

Contents

1	borge.borg	6
2	convention-go.borg	35
3	db.borg	39
4	ui.borg	47
5	cli.borg	59
6	history.borg	63
7	lib.borg	66
8	meta.borg	81
9	sexp.borg	88
10	lib/lang/ast.ml	93
11	lib/lang/balance.ml	95
12	lib/lang/error.ml	109
13	lib/lang/lexer.ml	110
14	lib/lang/parse.ml	113
15	lib/lang/print.ml	114
16	examples/crud-app/db.borg	118
17	examples/crud-app/generated.ml	119
18	examples/crud-app/ui.borg	121
19	lib/core/file_utils.ml	122
20	lib/core/project.ml	123
21	lib/core/spec.ml	125
22	lib/db/db_ast.ml	129
23	lib/db/db_json.ml	131
24	lib/db/db_parse.ml	133
25	lib/db/db_sql.ml	137
26	lib/db/db_validate.ml	143
27	lib/ui/ui_ast.ml	146
28	lib/ui/ui_clay.ml	150
29	lib/ui/ui_dream.ml	157
30	lib/ui/ui_json.ml	165
31	lib/ui/ui_parse.ml	168

32 lib/ui/ui_validate.ml	175
33 lib/book/book_manifest.ml	180
34 lib/book/book_print.ml	182
35 lib/bug/bug_ast.ml	191
36 lib/bug/bug_parse.ml	193
37 lib/bug/bug_registry.ml	195
38 lib/bug/bug_write.ml	196
39 lib/distributed/borg_comment.ml	197
40 lib/distributed/module_spec.ml	199
41 lib/doc/doc_coverage.ml	201
42 lib/doc/doc_detect.ml	205
43 lib/doc/doc_extract.ml	212
44 lib/format/fmt.ml	216
45 lib/format/nodes.ml	217
46 lib/future/depends.ml	219
47 lib/future/format.ml	220
48 lib/future/future.ml	221
49 lib/lock/lock.ml	222
50 lib/lock/lock_pi_launch.ml	226
51 lib/lock/lock_prompt.ml	227
52 lib/lock/prompt_minimal.ml	228
53 lib/merge/agent_submit.ml	231
54 lib/merge/branch.ml	235
55 lib/merge/conflict_detect.ml	238
56 lib/merge/lockfile.ml	241
57 lib/merge/merge_strategies.ml	244
58 lib/merge/queue_storage.ml	247
59 lib/merge/queue_types.ml	250
60 lib/merge/rollback.ml	253
61 lib/merge/worktree.ml	256
62 lib/check/check.ml	259
63 lib/check/code_quality.ml	260

64 lib/check/drift.ml	264
65 lib/check/dune__parse.ml	274
66 lib/check/lint.ml	277
67 lib/check/meta.ml	284
68 lib/check/report.ml	290
69 lib/check/surface.ml	292
70 lib/convention.ml	296
71 lib/go__doc__extract.ml	299
72 lib/go__parse.ml	302
73 lib/go__surface.ml	306
74 lib/metrics/stats.ml	311
75 lib/generate/agent.ml	314
76 lib/generate/generate.ml	319
77 lib/generate/review__quality.ml	322
78 lib/output/json__out.ml	324
79 bin/cmd/Abort.ml	330
80 bin/cmd/Balance.ml	331
81 bin/cmd/Book.ml	333
82 bin/cmd/Check.ml	334
83 bin/cmd/Commit.ml	336
84 bin/cmd/Diff.ml	338
85 bin/cmd/Drift.ml	339
86 bin/cmd/Execute.ml	345
87 bin/cmd/Fmt.ml	346
88 bin/cmd/Future.ml	348
89 bin/cmd/Generate.ml	350
90 bin/cmd/Inline.ml	355
91 bin/cmd/Issue.ml	356
92 bin/cmd/Lint.ml	358
93 bin/cmd/Log.ml	360
94 bin/cmd/Make.ml	361
95 bin/cmd/Merge.ml	363

96 bin/cmd/Merge_queue.ml	366
97 bin/cmd/Modules.ml	370
98 bin/cmd/Nodes.ml	372
99 bin/cmd/Normalize.ml	373
100 bin/cmd/Parse.ml	374
101 bin/cmd/Plan.ml	375
102 bin/cmd/Print.ml	377
103 bin/cmd/Report.ml	378
104 bin/cmd/Review.ml	380
105 bin/cmd/Stats.ml	383
106 bin/cmd/Undo.ml	386
107 bin/cmd/Version.ml	387
108 lib/review/extract_docs.ml	388
109 lib/review/extract_functions.ml	391
110 lib/review/extract_parsetree.ml	393
111 lib/review/review_agent.ml	394
112 lib/review/review_parse.ml	396
113 lib/review/review_prompt.ml	400
114 lib/review/review_types.ml	403
115 lib/review/semantic_review.ml	406
116 bin/borge.ml	412
117 bin/borge_balance.ml	414
118 bin/borge_check.ml	415
119 bin/borge_diff.ml	416
120 bin/borge_drift.ml	417
121 bin/borge_execute.ml	418
122 bin/borge_fmt.ml	419
123 bin/borge_generate.ml	420
124 bin/borge_inline.ml	421
125 bin/borge_lint.ml	422
126 bin/borge_log.ml	423
127 bin/borge_nodes.ml	424

128	bin/borge__normalize.ml	425
129	bin/borge__parse.ml	426
130	bin/borge__print.ml	427
131	bin/borge__report.ml	428
132	bin/borge__review.ml	429
133	bin/borge__stats.ml	430
134	bin/borge__undo.ml	431
135	bin/borge__version.ml	432
136	test/test__balance.ml	433
137	test/test__db__sql.ml	436
138	test/test__doc__coverage.ml	440
139	test/test__dune.ml	441
140	test/test__meta.ml	443
141	test/test__parse.ml	445
142	test/test__review.ml	447
143	test/test__slot__debug.ml	448
144	test/test__ui__clay.ml	449
145	test/test__ui__db.ml	452
146	test/test__ui__dream.ml	457

1 borge.borg

borge

A spec-first verification toolkit and gated agent loop that keeps spec and code aligned across commits – not an orchestrator

[status: in-progress]

– *agent note:* roerick, on what borge is for (2026-05-17): "Other people are solving this problem by spawning dozens of agents and having them work in parallel and at cross purposes. But I think we can achieve the same thing with massively fewer tokens by simply restricting and gating what the agent is allowed to do during a certain implementation phase. And the fact that we are adhering to spec as our source of truth is what's going to hopefully prevent a lot of the total running amok that we do tend to see with these agents."

The first spark was a question asked a few days earlier (2026-05-15): "is it possible to get postgres to output sexps as data." That sexp format became the through-line – the same structured language could flow from database to spec to parser to code to printed book. borge is the union of that format and this discipline: a spec-first verification toolkit that keeps agents aligned by gates rather than swarms.

program-architecture

How borge is structured: one root spec, inlined sub-specs, no orphans

[status: implemented]

The root borge.borg file is the architectural contract. It declares what modules exist and how they relate by inlining their specs. It is the AGENTS.md equivalent for borge projects.

(convention ocaml-dune) declares the structural rules for this project. Borge loads the rule set and uses it for structural drift detection. The convention covers: lib/ owns domain logic, bin/ owns CLI IO, dune files define libraries and executables, public_name conventions, test/ structure. Other conventions: go-standard, c-makefile, etc. This way lib.borg describes WHAT the lib does, not HOW OCaml works – the convention handles the how.

Every .borg file must either be inlined by a parent or declare no-inline itself. A .borg file with no parent and no no-inline declaration is orphaned – this is structural drift.

The inline directive is by filename and supports two target formats: (inline cli.borg) – the target is a .borg file containing the spec (inline login.mli) – the target is a .mli file; the spec lives in a (* borge (subsection ...) *) comment at the top of the .mli. The .mli's type declarations serve as the specification's structural contract; the borge comment adds status, verification criteria, and section nesting. The parent owns the architectural relationship. The child owns the detail.

Drift detection covers three layers: 1. Spec drift: .borg file describes something not in the code 2. Code drift: code implements something not in .borg 3. Structural drift: code organization violates the convention (e.g. domain logic in bin/ when ocaml-dune says lib/ owns it)

spec-interface-implementation-layers

The .borg ? .mli ? .ml layering borge is aware of

[status: planned]

Borge's project hierarchy has three layers, and borge is aware of each at the level appropriate to it:

`.borg` – intent. Borge OWNS this layer: reads, writes, validates, formats it. The spec is primary. `.mli` – interface contract. Borge is AWARE of this layer: reads it for drift checks, never restates it in the spec. The `.mli` is the integration contract between subtrees – what one module owes another. `.ml` – implementation. Borge CHECKS this layer against both `.borg` (does it match the intent?) and `.mli` (does it match the interface?). The `.ml` is the disposable, regenerable artifact.

The inline tree (root `.borg` inlines child `.borg` files) is the top of this hierarchy. Below it, each library’s `.borg` is aware of the `.mli` files in its scope and reads them as drift inputs. Borge does not duplicate `.mli` content into `.borg` – the `.mli` is the authoritative interface; the `.borg` is the authoritative intent; restating either in the other creates drift by design.

Formal requirement (planned): every dune library has a `.borg` file that is inlined by the root, and each such `.borg` is aware of the `.mli` files in its scope. A library with no `.borg`, or a `.borg` that ignores its `.mli` files, is structural drift. This keeps the loose tree loose (per-library `.borg`, writable by humans) while making the coverage complete.

modules

The modules that make up borge, inlined by filename
[status: implemented]

lang

The borge language: parsing, printing, and balancing
[status: implemented]

lib

Domain operations: checking, reporting, formatting, spec extraction
[status: implemented]

cli

CLI entrypoints: thin wrappers that call lib functions
[status: implemented]

history

Change tracking and provenance for `.borg` files
[status: implemented]

meta

Companion metadata files for structured analysis observations
[status: in-progress]

format

The file format
[status: implemented]

Verbatim strings use open-pipe to open and pipe-close to close. The parser tracks nesting depth. Content between the opening delimiter and the final closing delimiter that returns depth to 0 is `v_content`. The closing delimiter is NOT included in `v_content` – it is a delimiter, not content. Nested delimiters inside the content ARE included verbatim.

The inline directive: (inline filename.borg) declares that the named file is a sub-spec of this section. The file must exist and be parseable. The parent owns

the architectural relationship; the child owns the detail. Inline is by filename, relative to the parent file.

The no-inline declaration: (no-inline) in a project form means this project has no sub-specs. This is the legitimate standalone case. Every .borg file must either be inlined by a parent or declare no-inline. Any other .borg file is orphaned.

verification

Per-section acceptance criteria: how to know a feature works

[status: implemented]

The (verify ...) stanza defines acceptance criteria for a section. It lives inside any (section ...) or (subsection ...) form.

Syntax: (subsection login-page (status implemented) (verify (test "test/auth.ml") (build) (smoke "POST /api/login") (visual "/login"))))

Vocabulary is convention-defined: - ocaml-dune: build, test, smoke, visual, flow - go-standard: build, test, bench

Immutability: Verify stanzas are part of the spec. Agents MAY NOT add, modify, or remove verify items. Only humans edit verify. borge lint -verify checks for invalid methods. borge drift detects verify stanza modifications.

Execution model: borge does NOT run verification. It validates that: 1. Methods are valid for the convention (lint) 2. Verify stanzas are not modified by agents (drift) 3. Implemented sections with verify have agent notes (drift)

Agent workflow: When marking a section (status implemented), the agent reads the verify stanza, runs each check, and writes: (* agent note (| verified at commit abc123: build-pass, test-pass (22/22), smoke-pass (200 OK), visual-pass (login form visible) |) *) Verification claims are git-bound; freshness is computed from file hashes, not stored in .borg.

tool-pipeline

Defensive toolkit for writing .borg files

[status: implemented]

balance

Check paren balance. Run FIRST after every edit.

[status: implemented]

Balance is a fast character-level scanner that tracks paren nesting depth. It skips verbatim strings, quoted strings, and comments so only structural parens are counted. It also validates that structural ‘(’ on their own line align with their matching ‘)’. It is the #1 defensive tool: run it after every edit to catch structural errors before parse.

balance-verbose

Show paren stack on error: which (matches which)

[status: implemented]

When balance finds an unexpected close-paren, it reports the full paren stack: which open-paren is still open, at what line/col, and what keyword it belongs to. It also reports indent_mismatch errors when a structural ‘(’ on its own line does not align with its matching ‘)’.

With -repair, balance infers missing or extra ‘(’ from indentation and outputs a corrected file with properly indented new lines. The -verbose flag adds the raw paren stack list.

parse

Full syntax validation. Run after balance passes.

[status: implemented]

report

Show status summary across all .borg files. Use this to plan work.

[status: implemented]

CURRENT DRIFT: report treats all .borg files as peers and counts statuses independently. It does not walk the inline tree. This means inlined sub-specs are double-counted: if borge.borg says (subsection sexp (status implemented)) and sexp.borg also has its own statuses, both get counted. The fix: report should start from the root file, follow inline directives, and count each section once.

future

Show what's planned but not yet implemented - the roadmap view

[status: planned]

borge future scans all .borg files and extracts a roadmap: what's spec'd as 'planned' or 'in-progress' vs what's 'implemented'. This is the forward-looking companion to 'borg report' which shows current state.

Output format:

```
=== PLANNED === victor.borg:485 (section agentic-task-ui) - agent run
popup UI victor.borg:523 (section agentic-task-ui) auto-accept mode borge.borg:320
(subsection doc-coverage-command)
=== IN PROGRESS === victor.borg:237 (section codemirror-integration)
=== READY FOR REVIEW === victor.borg:120 (subsection job-panel)
- jobs complete (anything implemented but depends on planned items)
```

Usage: borge future # Show all planned work borge future -section X # Future work in specific section borge future -summary # Just counts: planned/in-prog/implemented

This command helps answer: - "What's next on the roadmap?" - "How much of the vision is implemented?" - "What depends on what?" - "Which sections are ready to work on?"

Integration with Victor IDE: - Shows in job panel as "12 items planned" - Click to see full future report - Can queue agents to work on planned sections

nodes

Dump the parsed AST with types and positions for debugging

[status: implemented]

Walks the AST and prints each node with its type, keyword (if list), source position, and child count. Example output: 1:1 List "project" [5 children] 1:9 Atom "borg" 1:16 List "doc" [2 children] This exposes what the parser actually sees - critical for debugging mismatches between source and AST.

lint

Semantic validation: invalid status, illegal placement, literacy.

[status: implemented]

borge lint checks for: - Invalid status values (not planned/in-progress/implemented/verified/etc) - Duplicate project names across .borg files - Orphaned .borg files (no inline parent, no no-inline) - Unknown comment types in annotated comments - Missing values in annotated comments - Deleted human-authored comments (compared to git HEAD) - Pending ask/response pairs without answers - (no-inline) on root files (root files can't inline anything) - Mechanical code quality: unsafe stdlib calls, polymorphic equality. borge lint -mechanical to enable.

borge lint -literacy checks .ml files for: - Missing literacy scores on agent-written doc comments (no [NNNN] after the authorship tag) - Plausible scores: if STORY >= 6 the comment must show narrative context; if EXPLAIN >= 6 it must walk through mechanics; if TEACH >= 6 it must contain a transferable principle. Catches inflated scores like [7777777] on a 2-line comment.

drift

Detect spec drift, code drift, and structural drift

[status: implemented]

Three layers of drift detection:

1. Spec drift: a .borg file describes something not in the codebase. E.g. cli.borg says (status implemented) for a command but no corresponding binary exists.
2. Code drift: the codebase has something not in any .borg file. E.g. a .ml file exists but nothing in the specs mentions it.
3. Structural drift: the code organization does not match the inline architecture. E.g. lib.borg says it owns Check and Report, but those modules live in bin/ instead. The root borge.borg says (inline lib.borg) but the actual dune library structure contradicts what lib.borg describes.

fnt

Canonicalize indentation. Run before commit.

[status: implemented]

Parses and re-prints with canonical indentation. Children of keyword forms (project, section, subsection, details) go on their own lines. Atom names stay inline with keyword. (doc ...) and (status ...) always print inline. Verbatim strings preserve their content verbatim.

fnt-check

Dry-run: exit 1 if formatting would change the file

[status: implemented]

borge fnt -check: parse the file, format it, compare to original. Exit 0 if identical, 1 if different. For CI pre-commit hooks. Also: borge fnt -diff shows the actual diff between current and formatted output.

check

Recursive health check across all .borg files.

[status: implemented]

worst-failure-modes

What goes wrong and which tool catches it

[status: implemented]

Missing open-paren: caught by balance. Extra close-paren: caught by balance. Verbatim not closed: caught by balance (context tracking). Annotated comment inside form body: caught by parse (syntax). Invalid status value: caught by lint (semantics). Duplicate project name: caught by check (cross-file). Bad indentation: caught by fnt (auto-fixes). Tracking progress: borge report shows what is done and what is next. Formatter producing unbalanced output: caught by fnt -check or by running borge balance on fnt output. Orphaned .borg file: caught by drift (structural). Domain logic in wrong layer: caught by drift (structural). Human comment deleted by agent: caught by lint (comment integrity). Undocumented function: caught by drift (code-documentation). Unsafe stdlib call (List.hd, List.assoc, Hashtbl.find, String.sub, int_of_string, open_in, input_line): caught by lint -mechanical. Full rule table in lib.borg section check / subsection code-quality. Polymorphic equality on strings/lists: planned - lint -mechanical with AST walker. Deeply nested match (> 3 levels): planned - lint -mechanical with AST walker. Exception: state- machine patterns (e.g., balance checker).

dead-code-integration

Enrich drift findings with `dead_code_analyzer` liveness data

[status: planned]

The `dead_code_analyzer` identifies OCaml functions that are never transitively called. Integrating its output into borge drift adds a liveness dimension to the existing spec-coverage analysis, producing four categories instead of two:

Spec'd + Alive -> healthy Spec'd + Dead -> spec rot: planned code that nobody uses Unspec'd + Alive -> doc gap (already flagged by drift) Unspec'd + Dead -> strong removal candidate

Integration approach (low effort): 1. borge drift shells out to `dead_code_analyzer` on the project 2. Parses its output to build a map of `function_name -> dead|alive` 3. Cross-references against the module surface already extracted by drift (`lib/check/surface.ml`) 4. Tags each function in `.borg.meta` with (dead true) or (dead false) 5. Upgrades drift findings: spec'd-but-dead functions become a new finding type (SpecRot) with higher severity

No new data model is needed – `.borg.meta` already carries per-function metadata. The `dead_code_analyzer` is maintained, works on modern OCaml, and is available via opam.

Future directions (separate subsections): - Clone detection (asak/inzad): find duplicated implementations that should share a single spec section - Exception tracking (Salto/reanalyze): verify that spec claims about error handling match the actual exception surface - AST-level pattern matching (ocaml-grep/osemgrep): upgrade `code_quality.ml` from regex to Parsetree-level matching

comment-integrity

Protect human-authored comments from silent agent deletion

[status: implemented]

Annotated comments with a human author (e.g. `(* roerick note ... *)`) are first-class spec content. Agents must never delete them silently.

For planning agents (borge plan): the agent edits `.borg` files in `.borge.lock/working/`. At commit time, borge checks whether any human-authored comments were deleted by comparing `working/` with `original/`. If deletions are found, the commit is blocked. The user must either restore the deleted comments or run `borge commit -force` to override. The diff `original/..working/` is shown.

For implementing/fixing agents (borge make, borge drift -fix): these agents cannot modify `.borg` files, so this check does not apply. Code file comment integrity may be added in the future.

`borge lint` (legacy) compares the working tree against git HEAD. If a human-authored comment is present in HEAD but missing in the working tree, lint emits a deletion warning. Nested comments are now captured by the parser and inspected by lint, so this no longer has the historical blind spot inside (project ...) or (section ...) forms. This still works for non-agent editing but is redundant when the agent workflow is used.

`borge normalize` is the human saying "I made changes on purpose." It runs `borge fmt`, then accepts all comment deletions as intentional by recording the current state as the new baseline. After `normalize`, lint will not warn about those deletions again.

code-documentation

Crawl `.ml` files for documentation coverage under `ocaml-dune` convention

[status: in-progress]

Under the `ocaml-dune` convention, borge drift checks code files too. For every `.ml` file it finds all `let` bindings (functions and values). A binding is documented if it has a borge-style annotated comment above it: `(* author note ([description of what this does]) *)`

Undocumented bindings are reported as a coverage gap. This makes literate programming the default: functions should explain themselves. To exempt a binding from documentation: `(* exempt doc *)`

The drift report shows: total bindings, documented count, undocumented count, coverage percentage. This is code drift by another name: the code exists but the spec (in the form of inline documentation) does not cover it.

doc-coverage-command

`borge drift -docs`: analyze documentation coverage

[status: implemented]

Implementation: `borge drift -docs` flag triggers documentation coverage analysis.

Algorithm (implemented in `lib/doc/`): 1. Find all `.ml` files (respecting `_build/.git` exclusion) via `Doc_extract.find_ml_files` 2. Extract all top-level let bindings via `Doc_extract.extract_bindings` 3. For each binding, check for preceding doc comment via `Doc_detect.extract_file_docs`: - `(** docstring *)` (ocaml doc style) - `(* author note (|...|) *)` borg-style comment - `(*| ... |*)` alternative borg format - `(* exempt doc *)` - marks as exempt, not undocumented 4. Skip bindings (handled by `Doc_extract` classification): - `let%test` (test bindings) - `let _ =` (ignored bindings) - Bindings starting with `_` (implicitly private) 5. Output: (documentation ...) block in `.borg.meta` via `Doc_coverage.generate_meta_block`

Coverage calculation (`Doc_coverage.calculate_file_coverage`): - documented = count of bindings with valid doc - exempt = count of bindings with `(* exempt doc *)` - total-bindings = all extracted bindings (after filtering) - undocumented = total - documented - exempt - coverage-percent = documented / (total - exempt) * 100

Drift findings produced: - undocumented-module: file with coverage < threshold - undocumented-function: specific binding missing doc (if `-verbose`)

CLI output (non-json): `lib/borge/spec.ml`: 38/45 documented (84.4%), 2 exempt, 5 undocumented `lib/borge/parse.ml`: 12/28 documented (42.9%) ? undocumented-module

Integration with stats: `borge stats` aggregates all (documentation ...) blocks from `.borg.meta` files for project-wide coverage report.

Remaining work: - Wire `Doc_coverage` into `borge_drift.ml` as `-docs` flag - Add `-verbose` flag for per-binding undocumented names - Add `Cmdliner` argument for threshold override

doc-formats

Supported documentation comment formats

[status: implemented]

OCaml supports multiple doc comment styles. Borge recognizes:

1. Standard docstring (ocaml doc style): `(** Convert a string to a status value. @raise Invalid_argument if string is unknown *) let status_of_string s = ...`

2. Borg-style annotated comment: `(* roerick note (| This function implements the status conversion logic. It handles all valid status strings defined in the spec. |) *) let status_of_string s = ...`

3. Borg-style annotated comment with literacy score: `(* agent note [5250] (| The balance checker is a paranoid paren counter. We do NOT trust the parser for structural validation because a syntax error in the parser itself could hide a real bug. So balance is a separate, dumber implementation.`

This is defense in depth: the parser and balancer are independent. They should agree. When they don't, at least one is wrong. `|) *) let check_balance input = ...`

4. Short borg comment: `(*| Convert string to status. Raises on invalid input. |*) let status_of_string s = ...`

5. Exemption marker: (* exempt doc *) let internal_helper x = ...

The literacy score [NNNN] is optional on borg-style comments. See the comment-writing-guide subsection for the full scoring system.

All formats are extracted during drift -docs analysis and contribute to coverage calculations.

drift-doc-integration

How doc coverage is reported in drift and enforced

[status: implemented]

When borge drift -docs runs, findings are generated in two places:

1. .borg.meta files get (documentation ...) blocks with full metrics. These are machine-readable and git-tracked.

2. Drift findings list includes coverage-related findings: - (finding undocumented-module (file PATH) (coverage PERCENT)) - (finding undocumented-function (file PATH) (name NAME)) (only with -verbose flag)

Thresholds and enforcement: - Coverage >= threshold: no finding (good)
- Coverage 50%-threshold: warning-level finding - Coverage < 50%: error-level finding - Default threshold: 80% (0.8)

Exit codes: - 0: all files meet threshold or above - 1: at least one file below 50% (error-level) - 2: at least one file between 50% and threshold (warning-level) only when -strict flag is passed; otherwise exit 0

-strict flag: Without -strict, warnings are printed but do not cause a non-zero exit. With -strict, any file below the threshold (including warning-level) causes a non-zero exit. This allows CI to gradually enforce coverage: start without -strict, fix the errors, then add -strict to catch warnings too.

The threshold is configurable per-project in borge.borg: (convention ocaml-dune (doc-coverage-threshold 0.7))

Remaining work: - Wire Doc_coverage.meets_threshold into drift exit code logic - Add -strict and -verbose Cmdliner args to borge_drift.ml - Read doc-coverage-threshold from borge.borg convention stanza

comment-writing-guide

How to write code comments – literacy scoring for agents and humans

[status: implemented]

Comments are not checkboxes. They are how the codebase tells its story. The goal is not coverage – it is literacy. Can someone read this file like a book and understand it?

Every borg-style comment carries a 4-digit literacy score:

(* agent note [STORY EXPLAIN TEACH EDGE] (| ... |) *)

Each digit is 0-7, self-declared by the author based on what the comment actually contains. Zeros are honest and common – most comments do not need to excel at everything.

1. STORY – Does this comment connect to a narrative? 0: No story. Just facts about this function. 5: Has context. Explains why this exists in the story of the module – what came before, what comes after, why you’re reading it now. 7: Reads like a book chapter. You’d publish it. Someone could understand the module’s architecture from the comments alone, without reading code.

2. EXPLAIN – Does it explain what’s happening in the code? 0: Doesn’t explain the implementation. 5: Explains the algorithm or data flow. Someone could understand how the code works without reading every line. 7: Walks you through it like a debugger session. Line by line where needed. Reveals the mechanics.

3. TEACH – Does it teach something transferable? 0: No transferable insight. 5: Teaches a technique, pattern, or gotcha – something you’ll use elsewhere in the codebase or in other projects. 7: Teaches a fundamental concept. You’ll remember it years later in unrelated code.

4. EDGE – Does it say what breaks? 0: No boundary discussion. Only the happy path. 5: Major edge cases, preconditions, or failure modes mentioned. You know at least one thing that can go wrong. 7: Every precondition stated, invariants explicit, failure modes catalogued. Reads like an incident report written by someone who got paged at 3am.

Examples:

A solid everyday comment – doesn't need to be a dissertation: (* agent note [5200] (| The balance checker is a paranoid paren counter. We do NOT trust the parser for structural validation because a parser bug could hide a real structural error. So balance is a separate, dumber implementation.

This is defense in depth – two independent checks that should agree. |) *)

A wrapper that needs almost no comment – honest zeros: (* agent note [5000] (| WRAPPER: balance-verbose is the same scanner as balance but with a paren stack for error reporting. Same algorithm, more memory. |) *)

An edge-case-heavy function that needs thorough warning: (* agent note [5557] (| Extracts named elements from a UI element tree.

Edge cases handled: - Empty tree: returns empty list (not an error) - Duplicate names: all occurrences collected, caller deduplicates if needed - Deep nesting: recurses through all children, O(depth) - Null names: elements without names are silently skipped (they're structural, not semantic)

Invariant: returned list preserves document order. Do not sort it – downstream tools expect FIFO. |) *)

Guidelines: - Score honestly. [5000] is not a failing grade – it means "I told the story but didn't explain mechanics, teach principles, or catalogue edge cases." That's fine for many functions. - Zeros are the default. Adding a dimension takes effort. If you didn't put edge cases in the comment, score EDGE as 0. Don't inflate scores to look good. - borge lint checks presence: every agent note must have a [NNNN] score block. Missing score is a lint warning. - borge lint checks plausibility: if you claim [7] on TEACH but the comment contains no transferable insight, lint suggests adjusting the score or adding content. - When a comment changes, reset the 4th digit (EDGE) and 1st digit (STORY if module context changed) to reflect unverified currency. Humans raise digits; agents should not claim currency they haven't verified. - Exempt internal helpers with (* exempt doc *) only if the name is truly self-documenting.

code-doc-drift-status

Mark inline code comments as drifted from implementation

[status: implemented]

When a code comment no longer accurately describes its function, it should be marked as drifted. This signals to agents and humans that the comment needs updating – it is not a silent lie, it is an explicit flag.

Syntax: add (status drifted) to the borg-comment: (* agent note (| WHAT: Parse a .borg file into a typed AST.

WHY: Downstream tools need structured access. |) (status drifted) *) let parse_file input = ...

Or in the short borg format: (*| Parse .borg into typed AST. | (status drifted) *)

Or in standard docstring: (** Parse a .borg file into a typed AST. @drifted comment does not mention error handling *)

When to mark drifted: - An agent modifies a function's behavior but doesn't update the comment (agent should mark it, not ignore it) - A human changes code and notices the comment is stale - borge review detects doc-code mismatch and marks it

When NOT to mark drifted: - Just update the comment instead. Drifted is for cases where you can't or won't fix it right now – you're acknowledging the gap, not endorsing it.

Drift status in borg-comments: The (status drifted) form inside a borg-comment is analogous to (status drifted) in a .borg section. Both say: this description once matched reality, but no longer.

Integration with borge review: - borge review checks for (status drifted) markers - If found, the finding is downgraded from "undocumented" to "drifted" - the function has a comment, but it's known-stale. This is better than undocumented (something was there) but worse than documented (it's accurate). - Coverage calculation: drifted comments count as documented for the numerator but are flagged separately in the report.

Integration with borge lint: - (status drifted) markers produce a `Stale_doc_comment` lint warning (not error - the gap is acknowledged). - Without the marker, a stale comment that doesn't match the code is an unacknowledged lie - that should be caught by borge review (agentic), not lint (static).

code-comment-integrity

Lint .ml files for missing documentation on exported bindings

[status: implemented]

The existing comment-integrity section protects .borg file comments from silent agent deletion. This section extends lint to .ml code files, enforcing the literate-programming rule that every exported function must have a doc comment.

This is the enforcement arm of code-documentation: `drift -docs` detects and reports coverage gaps; `lint -code-doc` makes it a pass/fail check.

New lint issue types for .ml files: - `Undocumented_binding`: exported binding at `PATH:LINE` with name `NAME` has no preceding doc comment and no `(* exempt doc *)` marker. Severity: warning by default, error with `-strict`. - `Stale_doc_comment`: a doc comment exists but the binding signature below it has changed since the doc was last reviewed (future: compare against .borg.meta reviewed-at timestamp). Severity: warning.

Integration with existing lint pipeline: - borge lint already walks .borg files. This adds a second pass over .ml files using `Doc_extract.extract_bindings` and `Doc_detect.extract_file_docs`. - New lint_issue variant: `| Undocumented_binding of { path; name; line } | Stale_doc_comment of { path; name; line; last_reviewed }` - These are included in `lint_result.issues` alongside existing .borg issues. - Error/warning classification follows drift-doc-integration thresholds: undocumented on exported binding = warning, undocumented with `-strict` = error.

Exemptions: - Bindings starting with `_` are internal, not checked. - `(* exempt doc *)` marker skips the binding. - Test bindings (`let%`) are skipped. - .mli-only modules: the .mli is the surface, but doc comments on .ml bindings are still expected. A binding with no .ml doc but a .mli doc is still flagged - the doc should live with the implementation.

CLI: - borge lint (existing): now also checks .ml files when the code-comment-integrity section is present and not (status planned). - borge lint `-code-doc`: explicitly enable .ml file checks even if section status is planned (for gradual adoption). - No separate subcommand - this integrates into the existing lint flow.

Exit code impact: - `Undocumented_binding` counts as warning in `lint_result`. - With `-strict`, counts as error. - lint already exits 1 if `error_count > 0`, 0 otherwise.

Future: - `Stale_doc_comment` needs .borg.meta reviewed-at tracking. - Could integrate with semantic review: if borge review finds low doc-accuracy, that could promote the lint issue from warning to error.

lint-code-doc-flag

borge lint `-code-doc`: enable .ml file doc checks

[status: implemented]

Adds a `Cmdliner -code-doc` flag to `borge_lint.ml`.

When passed (or when code-comment-integrity status is not planned), the lint runner does a second pass: 1. Find all .ml files via `Doc_extract.find_ml_files` 2. For each file, extract bindings and docs 3. For each exported binding without doc or exemption, emit `Undocumented_binding` lint issue 4. Add to the existing `lint_result` and print alongside .borg issues

Output format (consistent with existing lint): `? lib/borge/spec.ml:42: undocumented binding 'parse_section' ? lib/borge/drift.ml:15: undocumented binding 'check_code_drift'`

Or with `-strict`: `? lib/borge/spec.ml:42: undocumented binding 'parse_section'`

ast-position-tracking

Source position tracking on AST nodes for better diagnostics

[status: implemented]

Currently only annotated comments have `start_line/end_line`. The `sexp` type itself carries no position info. Adding `start_pos` and `end_pos` to every `sexp` node would enable: - balance to report which (at which line was closed early - nodes to show positions for each AST node - drift to map spec nodes to source locations - editors to jump to spec definitions The position type should include line, col, and offset to support both human-readable and machine-consumable output.

make

Agent-driven implementation – have an agent write code to match spec

[status: implemented]

`borge make` is the interactive entry point for agent-driven implementation. It opens a pi session with a prompt built from the spec, convention rules, existing code context, and the declared role. The agent writes code interactively; `borge` commits after (the journal is currently scaffolded – see lock-mechanism).

`borge make` Open pi to implement planned sections. The agent reads the full spec, writes code and tests to match planned sections, and journals status changes. Auto-commits on clean exit. Calls `borge balance` first to ensure structural soundness.

`borge make spec "description"` Single-shot spec generation. Not interactive. The agent produces a (status planned) section from a description. For interactive spec editing, use `borge plan` instead.

NOTE on the lockless single-shot path: `borge generate code` (aliased as `borge execute`) is a SEPARATE single-shot path that calls `pi -p`, prints code to stdout, and does NOT create a lock, run integrity checks, or commit. It is distinct from the gated `make/plan/drift -agent` loop above. Use it for quick one-off generation; use the gated loop for tracked work.

`borge drift -agent [-fix]` Open pi to check alignment between spec and code. Read-only by default. `-fix` allows creating journal entries with suggestions for spec updates.

`borge commit` Apply the current lock's journal to .borg files, run `fnt` and `balance`, `git commit`. Called automatically after `borge make/plan` unless `-no-commit`. Can be called manually after `-no-commit` sessions.

All agent-driven commands create a `.borg.lock/` directory (see lock-mechanism). The lock is removed after commit or abort.

make-command

Make code match spec – implement planned sections

[status: implemented]

`borge make`: open pi with the implementer role. The agent reads the entire spec, identifies planned sections that are ready to implement, writes code and tests. On clean exit: `borge` validates, commits. (The journal is scaffolded

but not yet written – the commit message is a file list, not a journal-derived summary.) On error or `-no-commit`: lock stays, user reviews.

plan-command

Edit the spec – add sections, refine descriptions

[status: implemented]

`borge plan`: open pi with the spec-writer role. The agent reads the spec, identifies gaps, adds sections, refines descriptions, updates statuses. Journals all changes.

drift-agent-command

Check and optionally fix alignment between spec and code

[status: implemented]

`borge drift -agent`: open pi with the observer role. The agent reads spec and code, reports findings, marks drifted sections. No code changes. `-fix` adds the ability to write code repairs and tests. Neither mode may modify `.borg` files except section statuses.

answer-command

Answer ask comments in `.borg` files

[status: planned]

`borge answer`: Scan `.borg` files for open ask comments and generate responses. An open ask is a `(* author ask (|...|) *)` comment without a corresponding `(* agent response (|...|) *)` or with an empty response slot.

Usage: `borge answer #` Answer asks in current file `borge answer -all #` Answer asks in all `.borg` files `borge answer -file path.borg #` Answer asks in specific file `borge answer -dry-run #` Show what would be answered

Process: 1. Parse specified `.borg` file(s) for ask comments 2. Filter to asks without responses (or empty responses) 3. For each open ask: - Build context from surrounding section - Query LLM with ask content + context - Insert `(* agent response (|answer|) *)` inline 4. Journal all responses (role: analyst)

Context building: - Parent section name and status - Sibling subsections (names only) - Preceding doc paragraphs (max 200 tokens) - The ask comment itself

What is NOT included: - Full implementation code - Other `.borg` files (unless `-all`) - Historical responses or changelog

Exit codes: - 0: All asks answered (or no asks found) - 1: LLM error or timeout - 2: Parse error in `.borg` file - 3: User abort (`-no-commit` mode)

context-management

Minimal context to respect token limits

[status: implemented]

The prompt sent to pi is intentionally minimal to avoid exhausting the agent's context window (~20K tokens consumed in seconds with full spec).

Context structure by role:

`borge make` (implementer) - Planned sections: name + one-line description (backlog) - Implemented sections: names only as guardrails - Existing module names: flat list - Max ~500 tokens total

`borge plan` (spec-writer) - All section names with statuses - Section nesting structure - No full doc bodies in prompt (agent explores files) - Max ~800 tokens total

`borge drift -agent` (observer/fixer) - Planned vs implemented diff - Changed module names since last commit - Optional: specific section under investigation - Max ~600 tokens total

What is NOT loaded: - Full .borg file contents (only summaries) - Module export lists (just names) - Convention rule details (just convention name) - Historical commits or diffs

The agent uses 'read' tool to explore files on demand rather than having everything pre-loaded.

loops-module

Extract the shared create/launch/teardown loop into a named, navigable module

[status: planned]

Today the agent loop is duplicated across bin/cmd/Make.ml, Plan.ml, and Drift.ml (run_agent) – the identical sequence of Lock.create ? write prompt ? Lock_pi_launch.launch ? on exit 0 Lock.teardown ~commit:true / Lock.remove, on failure keep lock. The loop exists but is not named; editing it means editing three copies.

Planned: lift the loop into lib/loop/ as a first-class module. Each command (make, plan, drift -agent, and the proposed regenerate) becomes a thin wrapper that instantiates and runs a named loop. Shared primitives – checkpoint (wait for user confirmation), run_to_completion (bounded loop with a completion criterion), gate_enforce (mode rules), verify (compile/drift/check at boundaries), build_context (per-role minimal context), invoke_agent (pi call now, rein phase later) – become small composable functions the loops compose.

A concrete loop is a record of params (role, gate mode, completion criterion, verification steps, checkpoint policy, context recipe) plus a body composing the primitives. Editing a loop edits its params; adding a loop instantiates the template.

This is future work that leans on rein: when borge migrates from pi to rein, these loop primitives map onto rein's effects (call_llm, call_tool, checkpoint, spawn_phase, telemetry, yield_artifact) with minimal rewrite. Building the module now against pi makes that migration a primitive swap, not a restructure. See future-work for the migration note.

execute

Make code match spec – interactive agent mode with role-gated behavior

[status: implemented]

– *agent note*: roerick, on the envisioned workflow (2026-05-17): "I'm kind of envisioning a workflow where we go like, okay, generate the code based on the spec. Okay, review the code that was created to determine if we've drifted from spec. Okay, fix any drift from the spec. Okay, review the code that's created again. And hopefully get to something that resembles the code. My hope is that we can achieve like a lot of development this way, basically by running Ralph loops over with using the spec as a source of truth."

Agent-driven commands open pi interactively with a minimal prompt built from summarized spec data. The agent's job depends on mode: implementer, spec-writer, or observer. Context is intentionally limited (see context-management subsection).

The agent operates in a .borge.lock/ workspace (see lock- mechanism). It edits .borg files in .borge.lock/working/ and code files in the repo. Commit applies working/ back to repo after integrity and balance checks.

Commands:

borge make Role: implementer CAN: write code, write tests, add journal entries (status changes, notes) CANNOT: modify .borg files (reads from working/, like all agents, but its writes go to code files and journal) CANNOT: delete human-authored comments INTENT: make code match the spec – planned sections are the backlog, implemented sections are guardrails

borge plan Role: spec-writer CAN: modify .borg files in .borg.lock/working/ CANNOT: write code files CANNOT: delete human-authored comments (integrity check blocks commit if violated –force to override) INTENT: edit the spec – add sections, refine descriptions, update priorities, reorder

borge drift –agent [–fix] Role: observer (default) or fixer (–fix) CAN (observer): create findings as journal entries, add comments to spec, change section status to drifted CANNOT (observer): write code files, modify .borg except via journal/status CAN (fixer): all observer permissions + write code repairs and tests CANNOT (fixer): modify .borg files except via journal/status INTENT: check alignment. –fix repairs drift.

borge commit Apply .borg.lock/working/ back to repo, run fmt and balance, git commit. Called automatically after make/plan unless –no-commit. Blocked by human comment deletion (unless –force).

borge abort Discard .borg.lock/. .borg files in repo are untouched. Code files may have been modified – use git to undo.

Why auto-commit? Git is the audit trail. Each session produces a commit. Revert if wrong. Review at PR level, not per-change.

inbox

The outlet the gate creates – agent feedback queued for human triage

[status: planned]

The inbox exists because the gate exists. Borge gates spec edits during build turns to stop the failure mode where an agent builds X and then rewrites the spec to match X. But a gated agent still needs a way to signal that the spec is wrong, incomplete, or ambiguous. The inbox is that outlet: it turns a forbidden action (edit the spec mid-turn) into a queued one (file a concern for human review). It is a routing layer, not a preservation layer.

As a consequence of being the gate's outlet, the inbox also absorbs the old issue-tracker section. Everything that needs human attention before it becomes a spec change goes through the inbox – bugs, drift findings, scope concerns, TODOs, external reports. One queue, one format, one lifecycle.

Why the inbox replaced the old bug tracker: The old .borg-bug format and the inbox concept were the same shape – both are "something that needs review before it affects the spec." Keeping them separate meant two file formats, two lifecycles, two sets of commands. A bug report from a chatbot and a scope concern from an agent are both inbox items that need triage.

The inbox also enforces spec-first discipline at the tool level – not just by convention. The current borge discipline says "no unspec'd code," but nothing prevents an agent from quietly editing a .borg file during implementation. The inbox fixes this:

- Agents in Plan mode CAN edit .borg files directly - Agents in any other mode CANNOT edit .borg files - Instead they file inbox items requesting spec changes - Inbox items require human review before becoming spec - This prevents unseen scope drift

Five sources of inbox items:

1. AGENT-FILED SCOPE CONCERNS. During implementation, if an agent discovers the spec is wrong or incomplete, it does NOT edit the spec. It files an inbox item describing the needed change and why. This forces the scope change to be visible and reviewed. (This replaces the old architectural-decisions subsection – same concept, now part of a unified queue.)

2. BUG REPORTS. From agents, users, or chatbots. A bug is an inbox item where something is broken. Drift findings from borge drift automatically create bug-type inbox items. Semantic review findings (from borge review) also create bug-type items.

3. TODO AGGREGATION. All TODO markers across .borg files and .ml files are surfaced as inbox items. A TODO in code is an implicit spec gap –

it says "I didn't implement what was planned" or "something was planned but not yet spec'd." Either way it needs triage.

4. **EXTERNAL FILINGS.** Users and chatbots can file inbox items directly – bug reports, scope concerns, feature requests, "this was implemented differently than I expected." These are the same shape as agent-filed items but come from outside the pipeline.

5. **DRIFT AND REVIEW FINDINGS.** borge drift and borge review produce findings automatically. These are linked to inbox items instead of being a parallel tracking system.

Inbox item types (source field): - scope-concern: spec is wrong, incomplete, or ambiguous - bug-report: something is broken - todo-aggregation: surfaced TODO marker - feature-request: user or chatbot wants something new - implementation-divergence: code doesn't match spec - drift-finding: from borge drift -agent - review-finding: from borge review (semantic or doc)

Lifecycle: triage ? acknowledged ? plan-approved ? spec-updated ? closed

The critical transition is plan-approved ? spec-updated. Only Plan mode can make it. Only after human review. This is what prevents scope drift.

Other resolutions: - wontfix: acknowledged but not acting on it - duplicate: same as an existing item - fixed-in-code: bug fixed without spec change

inbox-file-format

File format for inbox items

[status: planned]

Inbox items are stored as .borge-inbox files in the repo, tracked in git alongside code and specs. One queue, one format.

Scope concern example: (borge-inbox id:"INBOX-7" (title "Need error handling in db_sql") (status triage) (filed-by agent:implementer) (source scope-concern) (affects section:"lib.borg:db-sql") (created 2026-05-31) (doc (| During implementation of db_sql.ml, discovered that the spec doesn't cover connection retry logic. The spec says (status implemented) for db-sql but retry was never mentioned. Filing for scope review.))))

Bug report example: (borge-inbox id:"INBOX-12" (title "parse_expr crashes on nested parens") (status triage) (filed-by roerick) (source bug-report) (affects section:"lib.borg:parser") (created 2026-05-31) (doc (| borge parse crashes on the new borge.borg with nested verbatim strings. Repro: borge parse borge.borg.))))

Drift-finding example: (borge-inbox id:"INBOX-15" (title "Doc-code drift in lock_mechanism.ml") (status triage) (filed-by agent:drift) (source drift-finding) (affects section:"lib.borg:lock") (created 2026-05-31) (drift-report-id "DRIFT-2026-05-31-001") (doc (| Comment says "copies original/ back to the repo" but code actually copies from .borge.lock/snapshot/.))))

Fields: - id: unique identifier (INBOX-N) - title: brief description - status: triage | acknowledged | plan-approved | spec-updated | closed - filed-by: who filed it (roerick, agent:implementer, agent:drift, chatbot:name) - source: scope-concern | bug-report | todo-aggregation | feature-request | implementation-divergence | drift-finding | review-finding - affects: which .borg section is impacted - created: date - doc: full explanation - relates-to: optional reference to another INBOX-N - drift-report-id: optional link to drift report that spawned this item - resolution: optional (when date) (how text) (by who)

inbox-commands

CLI commands for inbox management

[status: planned]

borge inbox list [-status triage] [-source bug-report] List inbox items, optionally filtered by status and source.

borge inbox show INBOX-7 Show full details of an inbox item.

borge inbox file --title "description" --source scope-concern File a new inbox item. Used by agents, users, and chatbots. The --source flag determines the type. The --affects flag optionally links to a .borg section.

borge inbox acknowledge INBOX-7 Mark as acknowledged by human. Moves to acknowledged.

borge inbox approve INBOX-7 Approve for spec update. Moves to plan-approved. This DOES NOT update the spec yet – it just grants permission for Plan mode to incorporate it.

borge inbox close INBOX-7 [--resolution spec-updated|wontfix|duplicate|fixed-in-code] Close an inbox item with resolution.

borge inbox consolidate Aggregate all TODO markers across .borg and .ml files into inbox items. Deduplicates against existing items. Also surfaces (status planned) sections with no details.

The old commands are subsumed: - "borg bug new" ? "borg inbox file --source bug-report" - "borg bug list" ? "borg inbox list --source bug-report" - "borg bug show ID" ? "borg inbox show ID" - "borg bug close ID" ? "borg inbox close ID" - "borg drift-report" ? drift auto-creates inbox items

inbox-gating-rules

How borge modes enforce inbox usage

[status: planned]

The inbox is enforced through mode-specific gating. This is hard enforcement at the tool level, not convention.

In Plan mode: - CAN edit .borg files (including plan sections) - CAN close inbox items with spec-updated - This is the ONLY mode that can modify plan content

In Execute mode: - CANNOT edit .borg files (plan sections are frozen) - MUST file inbox items for any spec concerns - Can edit code files freely - CANNOT edit human-authored comments in any file (see comment-protection subsection)

In Audit/Review mode: - CANNOT edit .borg files - CAN file inbox items for drift/review findings - Drift and review findings auto-create inbox items - CANNOT edit human-authored comments in any file

In BorgEdit mode: - CAN edit .borg files (explicit escalation) - But changes are tracked and logged - Should still file inbox items for major changes - CANNOT edit human-authored comments in any file

The enforcement is at the tool level: rein's gate engine rejects edit/write tools targeting .borg files when not in Plan or BorgEdit mode. rein's gate engine also rejects edits to human-authored comments in .ml files (see comment-protection). Agents must use the inbox file command instead.

comment-protection

Protect human-authored comments in both .borg and .ml files

[status: planned]

The lock-mechanism already protects human-authored comments in .borg files (detecting deletions via diff at commit time). This extends the same principle to .ml files.

What counts as a human-authored comment: In .borg files: - (* roerick note ... *) - (* <name> ask ... *)

In .ml files: - (** ... *) ocamldoc comments (not authored by agents) - (* roerick ... *) or (* <human-name> ... *) - (*| ... |*) borge-style comments with human authorship

What does NOT count as human-authored: - (* agent note ([...]) *) - (* exempt doc *) - (* TODO: ... *) (these get aggregated into inbox) - Regular code comments without authorship markers

Enforcement: At the tool level, rein's gate engine inspects edit operations on both .borg and .ml files. If the edit would modify or delete a human-authored

comment, the gate BLOCKS it – regardless of mode. The only exception is when the human explicitly approves the edit (ASK).

This is stricter than the .borg-only protection in the lock-mechanism. The lock checks at commit time after the session. The gate checks BEFORE the edit happens.

Why: an agent should not be able to silently change what a human said in a comment. If the comment is wrong, file an inbox item with source scope-concern explaining the discrepancy. A human reviews it. The human can then run borge normalize to accept the change intentionally.

semantic-review

LLM-powered verification that function docs match implementation

[status: implemented]

Walks OCaml files, extracts functions and their docstrings, uses LLM to verify two things:

1. Doc-code alignment: Does the comment accurately describe what the function does and why it exists? 2. Internal consistency: Is the function well-constructed, or does it have internal contradictions (dead code paths, unreachable branches, parameters that are always ignored, logic that contradicts the comment)?

Both checks happen in a single LLM pass per function for efficiency. The prompt asks the LLM to evaluate both.

Process per function: 1. Extract: function name, signature, docstring, body 2. Prompt LLM with three questions: a) Is there a docstring present? If yes, does it accurately describe WHAT the function does and WHY it exists? (doc-code alignment) b) Are there (status drifted) markers in the comment? If so, the drift is acknowledged – flag as drifted rather than inaccurate. c) Is the function internally consistent? Are there dead branches, unused parameters, contradictory logic, or structural issues? (internal consistency) 3. Store result in .borg.meta under node metadata

What counts as doc-code drift: - Comment says "returns Error on invalid input" but the function actually raises an exception - Comment describes behavior that no longer exists - Comment omits important side effects or error cases - Comment says WHY but the reason has changed (e.g., "Workaround for bug X" but bug X was fixed)

What counts as internal inconsistency: - Parameter is accepted but never used in the body - Match branch is unreachable (covered by earlier branch) - Boolean parameter that is always passed the same value - Let binding that shadows an outer binding identically - Comment says one thing, code does another (this is also doc-code drift, but indicates the function itself may have been patched without thinking through the design)

Metadata structure: (function name:"parse_expr" (doc-present true) (doc-status accurate|drifted|missing) (doc-accuracy high|medium|low) (consistency consistent|questionable|inconsistent) (internal-issues "optional description") (confidence high|medium|low) (checked-at 2025-05-16T10:30:00Z))

doc-status values: - accurate: comment matches implementation - drifted: comment has (status drifted) marker OR LLM detected significant mismatch - missing: no doc comment present

consistency values: - consistent: no internal issues detected - questionable: minor issues (unused parameter, etc.) - inconsistent: major issues (dead branches, logic contradictions, code that can never execute)

Commands: - borge review FILE - borge review -dir DIR - borge review -all - borge review -stale - borge review -focus drift - borge review -focus consistency

Findings flow to inbox: - Low doc-accuracy ? creates inbox item (source: review-finding) - Missing doc on exported function ? inbox item - (status drifted) comment ? informational inbox item - Internal inconsistency ? inbox item (source: review-finding)

Auto-marking drifted comments: When borge review `-all` detects doc-code drift on a function that does NOT have (status drifted), it can optionally add the marker: `borge review -all -mark-drifted` This inserts (status drifted) into the comment. The marker means "a machine checked this and found drift" – it is not a fix, but it makes the lie explicit. Without `-mark-drifted`, review only reports findings.

distributed-specs

Module-level .borg files with loose function-level coupling

[status: planned]

Rather than one central borge.borg, each module folder has its own .borg file describing that module's exports and structure. Function-level detail lives in borg-comments within the .ml files themselves.

This creates a hierarchy: - repo/borge.borg: conventions, top-level goals - lib/core/lib_core.borg: module exports, dependencies - lib/core/spec.ml: functions with borg-comments

Borg-comment format (OCaml comment containing borg): `(* (fn extract_functions (doc "Pulls functions from code") (since "2025-05")) *)`

Drift checking (static/non-agentic): Only verifies referential integrity – does a function name listed in the module's exports actually exist in the code? Does a function in the code appear in the exports? No semantic comparison between module spec and function docs.

Drift checking (agentic): The agent reads module spec intent and function comments, determines if there's semantic drift, and can file inbox items for mismatches it finds.

This keeps the non-agentic path fast and simple while delegating semantic understanding to the LLM when needed.

todo-aggregation

Surfacing TODOs as inbox items

[status: planned]

TODOs are spec gaps. They say "I know I need to do something here but I haven't spec'd it yet." The inbox consolidates them.

TODO sources: - TODO in .borg files: `(* TODO: handle connection retry *)` - TODO in .ml files: `(* TODO: add error handling *)` - (status planned) sections with no details in .borg - Drift findings from borge drift

borge inbox consolidate scans all sources and creates or updates inbox items. Existing items are NOT duplicated – the consolidation checks for matches.

A TODO in code during implementation is NOT a license to implement without spec. It's a signal that the spec is incomplete. File it, triage it, plan it.

lock-mechanism

Transactional isolation for agent sessions – metadata, not workspace

[status: implemented]

Every agent-driven command (`make`, `plan`, `drift -agent`) creates a .borge.lock/ directory. The lock is a metadata store, NOT a workspace (the journal is scaffolded but not yet written – see known limitations below). The agent edits .borg files directly in the repo. The lock captures the session for audit, integrity checking, and abort/restore.

The agent's workspace IS the repo. The lock provides:

1. Concurrency prevention. If .borge.lock/ exists, refuse new agent sessions. One agent at a time prevents conflicting edits. The human can 'borge abort' to clear a stale lock.

2. Snapshot for integrity check. On lock creation, all .borg files are copied to .borge.lock/original/. At commit time, borge diffs original/ against the working

tree to detect deletions of human-authored comments. If found, commit is blocked unless `-force`.

3. Snapshot for abort. If the session goes wrong, ‘borge abort’ copies original/ back to the repo, restoring .borg files. Code file changes are NOT reverted – git handles that.

4. Audit trail. The lock contains: - prompt.md: the full prompt sent to pi - journal: structured log of proposed changes - role: the role declaration for the session - state: running | completed | failed

5. Commit message construction. Currently the commit message is a list of changed files. The journal is scaffolded (the path exists) but not yet written – a future version will derive the commit summary from a real journal.

Lock directory layout: .borg.lock/ original/ # snapshot of all .borg files at lock time role # role declaration (text + sexp) prompt.md # prompt sent to pi journal # structured session log (scaffolded, not yet written) state # running | completed | failed

Agent editing rules: - .borg files: edit in place (repo root). The agent uses normal paths like lib.borg, borge.borg, etc. - Code files: edit in place (repo tree). Standard paths. - At commit: diff checks, fmt, balance, git commit. - At abort: restore .borg files from original/. Code files stay as-is; use git to undo if needed.

Known limitation – line-equality integrity check: The integrity check compares original/ against the working tree line by line (split on newline, List.mem membership). A reformatted human comment – even a whitespace change – registers as both a deleted and an inserted comment, producing false positives. This is the source of the instability reported in comment preservation. A future fix should normalize whitespace and compare on author + comment body, not exact line text.

review

Review code quality against spec using LLM

[status: implemented]

Every file should carry a quality review – this is the review section in each .borg child spec. borge review generates a code-quality assessment for files under each spec section. The review answers: Does the code match the spec description? Are exports documented? Are there dead functions? Is error handling present? The review is stored as annotated comments in the spec or as a separate .review file.

review-command

Generate a per-file code quality review from spec

[status: implemented]

borge review FILE: read the spec for the file, read the file source, build prompt asking pi to assess: - Does every spec described feature exist in code? - Are all exports documented? (check .mli, or docstrings) - Are there dead functions (unused exports)? - Is error handling missing where the spec says ‘returns error’? - Is the literate description accurate? Output: findings as sexp, similar to drift findings. These are stored as annotated comments under the file’s spec section, or in a per-file .review.borg file.

literate-programming

The borge philosophy: spec is the primary artifact, code is derived

[status: implemented]

– *agent note*: roerick, on the original goal (2026-05-28): "we were initially going to build out a feature set where we A. implement a policy of comments for every function in our analyzed code and

B. implement a static analysis linter to make sure that each function has comments, a la literate programming. this way we can determine if functions are doing what they say, and better analyze drift."

The ambition grew beyond mechanical coverage (2026-06-19): "I'm generally not thrilled with the amount and quality of comments in the code. they're nice. but they feel like a checkbox, rather than like a living book or document which i can read to understand the codebase... maybe a scoring metric to show 'how effectively does this comment tell our story'." That tension – between checkbox coverage and readable narrative – is the animating problem of this section.

In a borge project, the spec is not documentation – it IS the source of truth. The .borg file is the artifact. Borge reads specs, validates them, reports on them, and helps agents edit them. But borge does NOT generate code. Every .borg file is both a human-readable design document and a machine-checkable contract.

The three pillars:

1. Spec first: write (status planned) sections before code. Use borge make spec to turn todos into structured specs.

2. Implement via agent: borge make opens an interactive session where an agent implements planned sections. The agent writes code AND updates the spec, journaling changes. This is NOT code generation – the spec is authored, not derived.

3. Verify continuously: borge balance, check, drift, and review keep spec and code aligned. Drift means the implementation has diverged from the spec. Fix the code or update the spec.

A completed project is one where every section is (status implemented), every file has a code review, and borge drift reports zero items.

book

Print the codebase as a paginated book, with a round trip back to the spec via the inbox

[status: implemented]

– *agent note*: Origin (roerick, 2026-06-27): "I want to print codebases. like i want to order them linearly from start to finish, and read them like a book. this is literate programming. i want to be able to print them out, with page numbers, then scan them back in as feedback to the LLM and have it spec changes."

– *agent note*: verified at this commit: print + manifest implemented. build-pass, runtest-pass, smoke-pass (borge book print . -> 363-page A4 PDF + manifest, line numbers render, deterministic tree_checksum). book-absorb remains (status planned) – the long-tail round trip.

The literate-programming section says the spec is the primary artifact and code is derived. The book is the physical reading surface for that derived artifact: a paginated, linearly ordered rendering of the codebase that a human can read away from a screen, mark up by hand, and feed back into the spec pipeline.

The book is NOT a weave. Borge does not tangle code out of prose and does not interleave spec and code the way WEB does. The spec stays in .borg files; the book is a read-only projection of already-written code. What makes it literate is that its order is the spec's order: the table of contents is the section tree, and each chapter is the code a section covers. You read the codebase the way it was specified.

The round trip:

code -> book (print) -> paper -> inbox (absorb) -> spec

Three of these four hops already exist. `print` produces the paper. `absorb` reads marked-up scans back and files inbox items; it does NOT edit the spec. The human triages the inbox in Plan mode, the same way every other scope concern becomes a spec change. A noisy OCR read produces a messy inbox item that can be closed, not a corrupted spec.

Why the book earns its keep even without the round trip: reading code on paper, in a fixed linear order, with no scrollback and no folding, surfaces structure that the editor hides. The absorb side is a bonus, not the justification.

Two render modes, shipped in order:

1. code-only (MVP). Each page is source with line numbers and a right-hand notes column. Ship this first.
2. woven (later). Interleave each section's spec text with the code it covers, chapter by chapter. More work; only worth it once code-only proves the book gets opened.

Scope: a chapter is the code covered by one `.borg` section. Whole-codebase printing is `print` every chapter in spec order, not a separate mode. Design for chapter-scoped printing and let the whole book fall out of it.

book-print

Render code to a paginated, annotated document

[status: implemented]

`borge book print [-section NAME] [-woven] [-o FILE]`

Linearizes code into a paginated document with page numbers, line numbers on every page, and a right-hand margin wide enough to write in.

Ordering is four-tier, data-driven from the project's own structure, so the book reads like a book rather than a lexical file dump:

tier 0: root `.borg` file (the architecture / introduction) via `Project.find_roots`.
 tier 1: other top-level `.borg` files (the design docs: `cli.borg`, `lib.borg`, `db.borg`, `ui.borg`, ...).
 tier 2: `.mli` files (module signatures – the interface layer; empty in this repo until interfaces are added).
 tier 3: `.ml` files (implementations), plus any sub-directory `.borg` files.

Tier 0 makes the book open with the spec – the primary artifact per literate-programming – not an alphabetical accident. Tiers 1-2 give the design and interface layer before the implementations.

Within tier 3, code files are ordered by a dependency rank, not lexically. The book parses 'open X' and 'Module.ref' from each `.ml` file, maps module names to their directory, and computes a longest-path rank per directory (lang has no deps -> rank 0; core depends on lang -> rank 1; check depends on core+lang -> rank 3; bin and test top the stack). Stdlib/external names (Format, String, List, ...) are excluded so a borge module that shadows a stdlib name (e.g. `lib/format`) doesn't create a false dependency. Files then sort by (rank, directory group `lib/bin/test`, lexical) – foundations precede dependents, so the book reads like a stack.

Full spec-order – where the `.borg` section tree is the table of contents and each chapter is the code a section covers – remains the long-tail feature, along with page-number cross-references for links between sections. `-section NAME` is a no-op in v1; it lands with spec-order.

Rendering target: LaTeX with the listings package – line numbers, framed code blocks, and a right-hand marginnote column for free; `pdflatex` to PDF. listings gives proper source typography without a custom line-numbering pass. groff/ms remains a viable fallback on systems without a TeX install (it is in OpenBSD base). The chosen backend is a build detail, not a spec invariant. The invariant is: paginated, numbered, with a notes column.

`.borg` files are NOT rendered as raw sexp. They are parsed (`Borge_lang.Parse`) and emitted as prose: section/subsection names as headings, (`doc ...`) strings as paragraphs, (`* author ... (|...|) *`) comments as attributed blockquotes (the

narrative voice), (status X) as an italic marker, (inline path) as a small note. Other structural forms (verify, depends-on, convention) are skipped – they are noise in the reading view. On parse failure the file falls back to a raw listing. The point: the spec reads as a document, not as data.

.ml/.mli files keep the listings treatment (line numbers, framed code) – they are code, and the book shows them as code.

Page furniture, every page: - page number (top or bottom corner) - chapter name (the section title) and section path - line numbers for the source on that page - a right-hand notes column, blank, for hand annotations

Output is a document file (PDF/PS, or text) plus a manifest sidecar (see book-manifest) on the same stem: foo.book.pdf and foo.book.manifest.

book-manifest

Sidecar mapping page N to file, with a tree checksum. Line numbers are on the page.

[status: implemented]

Every book print emits a manifest alongside the document: <stem>.book.manifest. Default gitignored (it is a build artifact); tracked only if the user wants a durable record.

The manifest is what makes the absorb round trip possible. Line numbers are printed on every page (listings numbers=left), so a margin note next to line 127 already names its own line – the LLM reads it off the page. The manifest only needs to answer "which file is page 14?" so a note in the gutter between two files is unambiguous.

Fields: - stem: the document stem the manifest belongs to - rendered_at: timestamp of the print run - backend: the renderer used (latex-listings, groff-ms, ...) - tree_checksum: digest of the exact tree printed (ordered file list plus each file's content hash) - pages: array of page entries, each with n (page number), file (source path on that page), and chapter (section title). Line numbers are NOT in the manifest – they are printed on the page itself and read directly by absorb.

tree_checksum is the stale-book guard. On absorb, if the working copy's checksum no longer matches the manifest, the book is stale relative to the code and the absorb step refuses. This stops a scan of a three-week-old book from filing inbox items against line numbers that have since moved.

book-absorb

Read marked-up scans and file inbox items – never edit the spec

[status: planned]

borge book absorb <scan...> [-stem NAME]

Takes scanned page images (PNG or PDF) of a previously printed book, extracts the margin annotations, and files one inbox item per annotation. It does NOT edit any .borg file. The round trip ends at the inbox, by design – the same place every other scope concern lands, and the only place a spec change is allowed to start.

Items conform to the inbox file format (see the inbox section). Pipeline per page:

1. A vision-capable LLM sees the page and returns the page number from the header, the printed code on the page for grounding, and each margin annotation with the line it sits next to and its text.
2. Map page to file:lines via the manifest.
3. Resolve the annotation to a .borg section using the manifest's section field.
4. Emit an inbox item: source: scope-concern | bug-report | feature-request (best-effort from the note) affects: section:"<section path from manifest>" filed-by: agent:book-absorb doc: the human's annotation, quoted verbatim, plus the file:lines it was anchored to

Fidelity is the real risk and is owned here, not elsewhere. The two hard problems are handwriting legibility and which line a scribble refers to. Miti-

gations, all on the print side so the absorb side can stay dumb: - structured annotation convention (circle the line, write the change intent in the margin) – documented in print output, not enforced - line numbers on every page (book-print) - generous line spacing and a wide notes column (book-print) - short intents, not essays (documented convention)

Safety: absorb never writes .borg. A bad OCR read yields a messy inbox item the human can close or edit; it cannot corrupt the spec. A –apply bypass that writes spec diffs directly is explicitly NOT in scope for v1; the inbox gate is the whole point.

Stale-book guard: if the manifest's tree_checksum does not match the working copy, absorb refuses and prints a message telling the human to reprint and rescan.

agent-discipline

Rules for AI agents working in borge – operationalizing spec-first development
[status: implemented]

– *agent note*: roerick, on the design choice that sets borge apart (2026-05-17): "Other people are solving this problem by spawning dozens of agents and having them work in parallel and at cross purposes. But I think we can achieve the same thing with massively fewer tokens by simply restricting and gating what the agent is allowed to do during a certain implementation phase."

The rules below are the operational form of that intuition: gating what an agent may touch is cheaper than orchestrating a swarm, and the spec is the gate.

The literate-programming section declares the philosophy. This section declares the operational rules that follow from it. These rules apply to AI agents (like pi) and humans alike, but AI agents are the primary audience because they are the ones most likely to violate them through scope creep.

Rule 1: No unspec'd code. Every .ml file, every module, every significant function must have a corresponding .borg section before it is written. If you want to add db_sql.ml, first add a (subsection db-sql) to lib.borg with (status planned) and a doc block describing what it does. Then implement. This is not bureaucracy – it is how borge prevents drift. Code without a spec is code without an owner.

Rule 2: Read the spec before building. Before implementing any feature, read the relevant .borg sections – especially their (details ...) blocks and (status ...). A (status planned) section with detailed doc is a design document: follow it. A section marked (status planned) with no details is a blank check: ask the human before filling it in. A feature not in any .borg file is out of scope – do not build it without explicit instruction.

Rule 3: Follow existing patterns. When the codebase has an established pattern (like generate.ml using Agent.run_pi_print for LLM-mediated generation), and the spec describes the same pattern (like ui.borg's generation-workflow section saying "assemble prompt, call the LLM"), follow that pattern. Do not invent alternative approaches (like template rendering) unless the spec explicitly calls for them.

Rule 4: Update spec status after implementation. When you implement something that was (status planned) in a .borg file, update it to (status implemented). When you add a new module, add a corresponding subsection. Stale statuses are a form of drift – they make borge report inaccurate and mislead the next agent about what is done vs. what is pending.

Rule 5: Check against spec during loops. In iterative workflows (Ralph loops, multi-step tasks), periodically verify: does my task list match what the .borg files say? Am I building something the spec describes, or am I building

something I invented? If a task item has no corresponding .borg section, stop and ask before proceeding.

Rule 6: Agents write only agent comments. An agent may only create, modify, or delete comments where the authorship is "agent" or "bot". Agents may NOT create, modify, or delete comments attributed to humans ("roerick" or any non-agent author). If an agent needs to quote a human, it uses (* agent note ([The user said: ...]) *). If a human wants a conversation captured in the spec, the human writes (* <name> ask ...) and the agent writes (* agent response ...). This rule is enforced by the lock integrity check (detects inserted and deleted human comments) and by lint (detects added human comments via git diff).

spec-language-design

Design principles for the borge spec language – declaration without evaluation
[*status: implemented*]

The .borg format is a declaration language, not a programming language. A .borg file states what is true and what should be true. The tools verify whether reality matches the statement. The spec language does not evaluate, branch, loop, or mutate. It declares.

This boundary – declaration without evaluation – is the defining architectural choice. Keeping it firm gives borge its most important properties:

- Forward compatibility: unknown forms survive parsing
- DSL extensibility: UI, DB, inbox coexist without parser changes
- Human readability: the format stays small and learnable
- Self-description: the grammar lives in borge.borg, not in types

The risk of over-typing the core is that every new keyword form becomes a compiler change. Borge avoids this. The core Ast.sexp type stays generic. Typed consumption happens in consumer modules (spec.ml, ui_parse.ml, db_parse.ml) that interpret the generic tree.

Where stronger types genuinely help is at the EXTRACTION boundary – the code that turns raw sexp into typed records. The current pattern is hand-rolled pattern matching repeated in every consumer. This is brittle: no exhaustiveness, silent skips, repeated logic.

GADTs for extraction combinators (lib/core/extract.ml): A small combinator library with a GADT encoding "this extractor produces type 'a from a sexp". Example:

```
val atom : string t val string : string t val keyword : string -> 'a t -> 'a t
val ( let* ) : 'a t -> ('a -> 'b t) -> 'b t
```

Then status extraction becomes:

```
let status_extractor = let* st = keyword "status" atom in match status_of_string st with
| Some s -> return s | None -> fail ("Invalid_status st")
```

This stays entirely in OCaml's type system. The .borg format does not change. UI and DB DSLs define their own extractors on top of this core, replacing ~200 lines of hand-rolled pattern matching in ui_parse.ml and db_parse.ml with composable, testable declarations.

Phantom types for the tool pipeline: Borge has a conceptual pipeline: balance -> parse -> lint -> drift -> review. A lightweight phantom type enforces this order:

```
type balanced type parsed type linted type 'stage file = { ast : Ast.file;
stage : 'stage } val balance : string -> balanced file result val parse : balanced
file -> parsed file result val lint : parsed file -> linted file result
```

Now lint cannot be called on a raw string. This is about the tools, not the spec language. It prevents real bugs with zero runtime cost.

Richer constraint forms (without evaluation): The spec language can express more expectations without becoming executable. Declared constraints are verified by tools, not evaluated by the language:

(depends-on STRING) -> resolved by borge future to build roadmaps (requires FORM) -> checked by borge lint (asserts FORM) -> verified by borge drift

These are declarations of intent. The language does not resolve, check, or verify them. The tools do. This keeps .borg as a vocabulary for expectations while the typechecker of reality remains outside the language.

Effects are not appropriate for borge: The current architecture is linear: read -> parse -> analyze -> produce findings. Effects shine with suspensible computations, plugin architectures, or complex resource management. Borge has none of these at scale. Effects would introduce non-local control flow into a simple linear pipeline, making it harder to follow.

The one future place effects might fit is the LLM call layer (generate.ml, review_agent.ml). A perform (LLM_query prompt) effect with a configurable handler could cleanly separate "request evaluation" from "how that request is fulfilled" (real model, cache, timeout, mock). But even there, a functor-based interface is sufficient today.

Self-describing validation: Since .borg files describe the .borg format in borge.borg itself, the tools could extract a schema from borge.borg and validate arbitrary .borg files against it. The spec describes the spec language. The tools validate conformity. This is the ultimate "types without evaluation": the language has no typechecker, but the tooling uses the self-description as a schema.

extract-combinators

GADT-based extraction library for typed sexp consumers

[status: planned]

lib/core/extract.ml: a GADT combinator library that replaces hand-rolled pattern matching in spec.ml, ui_parse.ml, and db_parse.ml.

Types: - 'a t : an extractor that produces 'a from a sexp

Primitives: - atom : string t - string : string t - int : int t - keyword : string -> 'a t -> 'a t - list : 'a t -> 'a list t - optional : 'a t -> 'a option t - record : fields -> 'a t

Combinators: - (let*) : sequential composition - return : lift pure value into extractor - fail : raise structured error with context

Usage in spec.ml: Replace the hand-rolled extract functions with combinator pipelines. The compiler checks exhaustiveness and type safety.

Migration plan: 1. Implement extract.ml with primitives and combinators 2. Port spec.ml to use combinators (smallest surface) 3. Verify edge case parity with existing tests 4. Port ui_parse.ml and db_parse.ml 5. Remove duplicated extraction helpers from each module

pipeline-phantoms

Phantom types enforcing tool pipeline order at compile time

[status: planned]

Add phantom types to the file processing pipeline to prevent tools from being called on data at the wrong stage.

Types: type balanced type parsed type linted type checked type 'stage file = { ast : Ast.file; stage : 'stage }

Functions (typed interface): Balance.balance : string -> balanced file result Parse.parse_file : string -> parsed file result Parse.from_balanced : balanced file -> parsed file result Lint.run : parsed file -> linted file result Check.run : linted file -> checked file result

This is zero-cost: the 'stage type parameter is erased at runtime. The typechecker prevents bugs like calling lint on a raw filename string or on a file that bypassed balance. It documents the intended pipeline in types.

richer-constraints

Declares more expectations in `.borg` without making it executable

[status: planned]

Extend the language vocabulary with forms that declare expectations. These are not evaluated by the language; they are verified by the tools.

(depends-on STRING) Declares that this section requires another section to be implemented first. Used by borge future to build dependency-aware roadmaps and detect ordering mistakes.

(requires FORM) Declares invariants this section depends on. Examples: (requires (min-coverage 0.8)) (requires (max-complexity 12)) Checked by borge lint. Violations produce findings.

(asserts FORM) Declares expectations about code reality. Examples: (asserts (module-surface "drift.ml" exports 12)) (asserts (no-orphan-files)) Verified by borge drift. Mismatches are drift findings.

All three forms are parseable by the generic sexp parser. Spec.ml recognizes them as first-class metadata. The tools decide whether and how to enforce them.

effects-policy

Why effects are deferred – and where they might fit later

[status: implemented]

Effects are deferred because borge's architecture does not need them. The tool pipeline is linear and deterministic. Adding effects would make control flow implicit and harder to trace.

Current position: do not use effects in borge.

Future consideration: the LLM abstraction layer (generate.ml, review_agent.ml) may benefit from an effect that separates "request evaluation" from "how it is fulfilled" when testing and mocking become a burden. Until then, functor-based interfaces are sufficient.

config

Project-level and user-level configuration for borge

[status: planned]

config-file

`.borgerc`: per-project configuration in sexp format

[status: planned]

Borge reads configuration from `.borgerc` files. Search order: 1. Project root: `.borgerc` (checked into git, shared by team) 2. User global: `~/.config/borge/config` (not in git, personal) Project config overrides user config. If both define the same key, the project value wins.

The file uses the same sexp format as `.borg` files. An empty or missing `.borgerc` means all defaults apply.

Config file format: (borge-config (developer-name "roerick") (api-keys (anthropic (env ANTHROPIC_API_KEY)) (openai (env OPENAI_API_KEY))))

Keys: - (developer-name STRING): the author name used for annotated comments. When set, borge make/plan/drift create comments with this authorship instead of "agent". When unset, defaults to "agent". This is the name that appears in (* developer-name note (|...|) *) comments.

- (api-keys ...): map of provider names to API key sources. Each key is either: (PROVIDER (env ENV_VAR_NAME)) – read from environment (PROVIDER (value "literal-key")) – literal string (NOT recommended for checked-in files; prefer env refs) When borge needs to call an LLM directly (not via pi), it reads the relevant provider key from config. If not configured, borge falls back to pi or `BORGE_AGENT_CMD` as it does today.

Why sexp format: consistency with the rest of borge. The parser already exists. No new syntax to learn. No TOML/YAML dependency. The config is a borge document.

Why not just env vars: env vars work but don't carry developer-name or project-scoped settings. The config file gives a named, version-controlled, discoverable home for project preferences that env vars can't provide.

Loading: lib/config.ml reads .borgerc, falls back to ~/.config/borge/config, merges project over user. The result is a typed config record available to all borge commands.

merge-queue

CI/CD pipeline for agentic development – isolated worktrees, batched merges
[status: planned]

agent-isolation

Every agent run is isolated in its own worktree and branch

[status: planned]

PROBLEM: Agents running concurrently clobber each other's changes, modify files directly in main worktree, create chaos that humans must clean up.

SOLUTION: Pi harness enforces isolation at the tool level. Every 'pi -mode agent' invocation:

1. Creates unique branch: agent/run-{timestamp}-{uuid}
2. Creates worktree: .pi/worktrees/agent-{id}/
3. Runs entirely within that worktree
4. On completion: submits to borge merge-queue
5. On failure: preserves worktree for debugging

HARNESS RULES (enforced by pi, not optional): - If .pi/concurrent.lock shows target files already locked, queue behind existing agent or refuse to start - If worktree creation fails, agent cannot proceed - If branch already exists, agent cannot proceed - Agent never modifies files outside its worktree

LOCKFILE FORMAT (.pi/concurrent.lock): { "active_runs": [{ "id": "agent-123", "branch": "agent/run-...", "files": ["lib/parser.ml"], "since": "2026-05-17T..." }] }

This prevents conflicts BEFORE they happen, not after.

merge-queue

Central queue of pending agent changes, processed atomically

[status: planned]

After agent completes in isolated worktree, it submits to the merge queue rather than applying directly:

```
borge merge-queue submit \ -from-worktree .pi/worktrees/agent-123/ \ -
summary "Generated parser section" \ -confidence 0.92 \ -affected-files lib/parser.ml,test/parser.ml
```

QUEUE STATE: - PENDING: Submitted, waiting for merge - READY: No conflicts with queued items before it - BLOCKED: Conflicts with earlier queue item - MERGING: Currently being processed - MERGED: Successfully merged to main - REJECTED: Merge failed, worktree preserved

CONFLICT DETECTION: borge analyzes file overlap between queue items: agent/run-123 touches lib/parser.ml agent/run-124 touches lib/parser.ml ? BLOCKED behind 123 agent/run-125 touches lib/lexer.ml ? READY (no overlap)

Commands: borge merge-queue list # Show pending items borge merge-queue show 123 # Diff for specific item borge merge -next # Merge first ready item borge merge -all # Merge all ready items borge merge -dry-run # Show what would happen

AUTO-MERGE AGENT (optional): - Daemon that continuously runs 'borge merge -all' - Configurable policies: batch size, delay between merges - Can pause on failure, wait for human review

merge-strategies

How to integrate agent changes into main

[status: planned]

MERGE STRATEGIES (configurable per project):

1. FAST-FORWARD (default for linear history): - If main hasn't moved since agent branched: `git merge -ff` - Cleanest history, no merge commits - Fails if main has new commits (use rebase instead)

2. REBASE-THEN-MERGE: - Rebase agent branch onto latest main - Then fast-forward merge - Linear history even when main moves - Risk: conflicts during rebase need agent re-run

3. MERGE-COMMIT: - Always create merge commit - Preserves agent branch history - Easier to revert entire agent run if needed

4. SQUASH: - Squash all agent commits to single commit - Clean history, lose agent's internal iterations - Good for: simple changes, auto-accepted runs

ROLLBACK ON FAILURE: If merge fails or CI rejects: 1. Revert merge commit (if created) 2. Mark queue item REJECTED 3. Preserve worktree for debugging 4. Notify agent owner (human or meta-agent) 5. Block dependent queue items

ATOMICITY: Either entire agent run merges or none of it does. No partial merges leaving codebase broken.

victor-integration

IDE surface for the curation loop – Victor runs the loop, borge verifies

[status: planned]

Victor's role is RUNNING THE CURATION LOOP, not passive observability. The vision: an agent proposes a hierarchical plan derived from the spec, and the human curates that plan before execution – pruning, reordering, expanding, rejecting branches. As inference cost trends toward zero, the human's leverage moves from prompt engineering to plan editing. Victor is the surface for that editing; borge remains the verification layer (drift/check/review) and the spec remains the source of truth.

The loop Victor runs: 1. Agent proposes a plan/tree derived from the spec 2. Human curates the tree in Victor (the editing primitives are still being designed – reroll, expand, reject, annotate are candidates, not settled) 3. Curated tree feeds back to the execution loop 4. borge verifies each node's output against its spec section 5. Human attention is routed to nodes that fail cheap verification (compiler, drift) and need expensive review

Victor does NOT own verification (borge does) and does NOT own execution (the agent loop does, via pi now and rein later). Victor owns the curation surface.

NOTE: this revises the earlier framing – "Victor is observability, not orchestration, no apply/discard buttons" – which described a passive queue viewer. The curation vision supersedes it: Victor participates in shaping the work before it runs, not just watching it after.

future-work

Planned improvements and architectural decisions

[status: planned]

Parser rewrite: see `sexp.borg` (subsection `sedlex-lexer`) and (subsection `menhir-parser`). Moved there because the parser is a `sexp` library concern, not a borge top-level concern.

Other possible future work: - Incremental parsing for editor integration - `.borg` language server protocol - Syntax highlighting definitions for common editors

Turn-level drift checking (future integration): Borge currently checks spec/code alignment at the commit boundary – after the agent session ends, before the

git commit. It does NOT check drift across turns within a session. Turn-level gating (rejecting an edit that violates the spec mid-session) is the role of a gate engine, which rein provides and borge does not yet integrate. When borge migrates from pi to rein, the loops-module primitives map onto rein's effects (`call_llm`, `call_tool`, `checkpoint`) and turn-level checking becomes available without restructuring.

Rejected alternative – multi-agent orchestration: A multi-agent architecture (spawning a hierarchy of agents, each owning a subtree, coordinating via a manager) was considered and declined. Org structures exist primarily to manage human cognitive limits – specialization because one person cannot hold "great DBA" and "great security" in one head. A frontier model does not share that constraint the same way; role prompting ("you are an expert codemonkey") is context priming, not a capability ceiling. Building an architecture whose premise is a hierarchy of differently-specialized agents encodes a limitation the underlying system does not have, and the coordination overhead (merge, conflict, translation losses, a manager agent worse at managing than doing) becomes its own complexity sink. A single linear loop with boundary verification is simpler and more LLM-native. Subagents remain an occasional invocation-level tool (cheap model for a `grep`), not the execution backbone. Model selection at call time, not a permanent agent hierarchy.

2 convention-go.borg

convention-go

go-standard convention: structural rules and code intelligence for Go projects using borge. Analogous to how ocaml-dune is the implicit convention for borge's own codebase.

[status: planned]

convention-definition

What (convention go-standard) declares about project structure

[status: planned]

When a root .borg file declares (convention go-standard), the following structural rules apply:

DIRECTORY LAYOUT: - cmd/ owns executable entrypoints (one dir per binary) - internal/ owns private packages (not importable externally) - pkg/ owns public library packages (importable by external) - api/ owns API definitions (protobuf, OpenAPI, etc.) - configs/ owns configuration files - test/ or *_test.go files own integration tests - go.mod at project root defines the module path

PACKAGE BOUNDARIES: - Each directory is a package; package name = dir name - Capitalized identifiers are exported (Go convention) - internal/ packages enforce privacy at compile time

BUILD: - go build ./... - compile all packages - go test ./... - run all tests - go vet ./... - static analysis - No dune files, no separate build system

DOC COMMENTS: - Go's `//` and `/* */` doc comments above declarations are the native format. golint / go vet enforce doc on exported declarations. - Borge annotated comments in Go nest the standard `(* *)` grammar inside Go's `/* */` block comment delimiters: OCaml: `(* author note (|description|) *)` Go: `/* (* author note (|description|) *) */` The inner `(* *)` is the same borge grammar that Doc_detect, Borg_comment, and all existing code already understands. No new parsing code needed for comment content.

DRIFT DETECTION: - Spec drift: same as OCaml - .borg section claims implemented but no corresponding code exists - Code drift: packages discovered from go.mod + directory structure vs sections in .borg files - Structural drift: domain logic in cmd/ when go-standard says pkg/ or internal/ owns it - Export drift: Go exported symbols (capitalized) vs what the spec describes

COMMENT INTEGRITY: - Same rules as OCaml: agent-authored comments can be modified by agents; human-authored comments cannot be silently deleted. The nested delimiters do not affect semantics.

comment-format

Borge comment grammar in Go source files

[status: planned]

THE KEY INSIGHT: Borge comments in Go nest the standard `(* *)` grammar inside Go's `/* */` block comment delimiters. This means the existing Doc_detect, Borg_comment, and Doc_extract modules work unchanged - they already know how to find `(* *)` and parse the borge grammar inside. The outer `/* */` is invisible to them.

Why not `/* */` as the borge delimiter? Because that would require duplicating the entire comment parsing pipeline: `is_borg_note`, `parse_borg_note`, `is_borg_short`, `is_exempt_marker`, `extract_comment_content`, the multi-line comment tracker in Doc_detect, etc. The nested approach trades a small visual cost for a large code savings: zero new comment parsing code.

Supported forms in Go files:

1. Annotated comment (full borge style): `/* (* agent note (| WHAT: Parse incoming HTTP requests. WHY: Handlers need structured access to request data. |) *) */ func ParseRequest(r *http.Request) (*Request, error) { ... }`

The outer `/* */` is Go's block comment. The inner `(* *)` is the standard borge grammar. `Doc_detect` finds the `(* *)` and processes it exactly as it would in an OCaml file.

2. Short borg comment: `/* (*| Parse incoming HTTP requests. |*) */ func ParseRequest(r *http.Request) (*Request, error) { ... }`

3. Exempt marker: `/* (* exempt doc *) */ func internalHelper(x int) int { ... }`

4. Drift marker inside a borge comment: `/* (* agent note (| WHAT: Parse incoming HTTP requests. WHY: Handlers need structured access to request data. |) (status drifted) *) */`

5. Function-level borg form: `/* (* (fn ParseRequest (doc "Parse incoming HTTP requests") (since "v0.1")) *) */ func ParseRequest(r *http.Request) (*Request, error) { ... }`

6. Go doc comments (standard `//` above declaration): `// ParseRequest parses an incoming HTTP request. func ParseRequest(r *http.Request) (*Request, error) { ... }`

These are standard Go doc comments. They are NOT borge comments – they cannot carry authorship or verbatim content. For doc coverage, they count as documented (`GoDocstring`) but lack the structured metadata that borge comments provide.

What does NOT work in Go: - Line comments (`//`) cannot carry borge annotations because they are single-line and cannot span the `(* *)` delimiters or `(|...|)` verbatim syntax required for borge comments. - A bare `/* (* ... */) */` without the outer `/* */` is also valid (Go allows nested appearances in comments) but the convention is to always use the outer wrapper for clarity.

NO NEW COMMENT PARSING MODULES NEEDED: Because borge comments use the same `(* *)` delimiters inside Go's `/* */` wrapper, the following modules work unchanged: - `Doc_detect` – finds `(* *)` and classifies borge comments - `Borg_comment` – extracts `(fn ...)` forms from `(* *)` - `Doc_extract` – counts bindings for coverage - `Doc_coverage` – computes coverage the same way

The only new modules are Go-specific: - `Go_parse` – discovers packages from `go.mod` + directories - `Go_surface` – extracts exported symbols from `.go` files - `Go_doc_extract` – extracts Go `func/type/var/const` bindings

modules

New modules needed for go-standard convention support

go-parse

Parse `go.mod` and directory structure for package discovery

[status: implemented]

Analogous to `Dune_parse`. Discovers Go project structure by parsing `go.mod` and walking the directory tree.

Types: - `go_module`: module path from `go.mod` (e.g. `"github.com/foo/bar"`) - `go_package`: directory path, package name, whether internal - `go_executable`: `cmd/` subdirectory name - `go_test`: `*_test.go` files or `test/` directory

Functions: - `parse_go_mod`: `string -> go_module` option - `discover_packages`: `string -> go_package` list - `discover_executables`: `string -> go_executable` list - `parse_all`: `string -> go_project` - `all_package_names`: `go_project -> string` list - `library_packages`: `go_project -> go_package` list - `executable_packages`: `go_project -> go_package` list

go-surface

Extract exported symbols from .go files

[status: implemented]

Analogous to Surface. Extracts the public interface of a Go package by scanning .go files for exported declarations.

Go export rule: a name starting with an uppercase letter is exported. This is enforced by the compiler – no .mli equivalent needed.

Types: - go_symbol_kind: Func | Method | Type | Var | Const - go_symbol: { name; kind; file; line; is_exported } - go_package_surface: { path; package_name; symbols; source }

Functions: - is_exported_name : string -> bool - extract_func_decl : string -> (string * go_symbol_kind) option - is_test_or_benchmark : string -> bool - extract_type_decl : string -> string option - extract_var_or_const_decl : string -> string option - extract_file_surface : string -> go_symbol list - find_go_files : string -> string list - extract_package_surface : string -> go_package_surface - exported_names : go_package_surface -> string list - all_names : go_package_surface -> string list

go-doc-extract

Extract Go bindings for documentation coverage

[status: implemented]

Analogous to Doc_extract. Extracts Go declaration bindings (func, type, var, const) for documentation coverage analysis.

Types: - go_binding_info: { name; line; is_exported; is_test; is_internal }

Functions: - extract_go_bindings : string -> go_binding_info list - find_go_files : string -> string list - extract_project_go_bindings : string -> (string * go_binding_info) list - count_stats : go_binding_info list -> int * int * int * int

NOTE: Doc detection for Go files uses the EXISTING Doc_detect module unchanged. Because borge comments in Go nest the standard (* *) grammar inside Go's /* */ block comments, Doc_detect already knows how to find and classify them. There is no go_doc_detect module – it is not needed.

Similarly, borge comments in Go files are extracted by the EXISTING Borg_comment module. There is no go_comment module.

convention-dispatch

How borge selects the right convention at runtime

[status: implemented]

Borge dispatches to the appropriate module set based on the convention declared in the root .borg file.

Convention resolution: 1. Read the root .borg file 2. Extract the (convention NAME) form 3. Dispatch to the appropriate module set:

| Feature | ocaml-dune | go-standard | |-----|-----|-----|
-----| | Package parse | Dune_parse | Go_parse | | Surface extr. | Surface |
Go_surface | | Binding extr. | Doc_extract | Go_doc_extract | | Doc detection | Doc_detect | Doc_detect (same!) | | Borg comments | Borg_comment |
Borg_comment (same!) | | Code drift | Drift (dune-aw.) | Drift (go-mod-aware) | | Borge comment | (* ... *) inside (* ... *) inside | | .ml file | /* */ wrapper in .go | | Docstring | (** ... *) | // above decl | | Exempt mark | (* exempt doc *) | /* (* exempt doc *) */ | | File extension | .ml / .mli | .go | | Excl. dirs | _build/ | vendor/ |

Notice: Doc_detect and Borg_comment are SHARED between conventions. This is the benefit of the nested comment approach – no duplicate parsing code.

Implementation: - `Convention.t` type (variant: `Ocaml_dune` | `Go_standard`)
- `Convention.resolve : string -> t` - Each command dispatches via `Convention`
Fallback: - If no `(convention ...)` form exists, assume `ocaml-dune`. - If
convention is unknown, warn and use `ocaml-dune` as default.

3 db.borg

borge-db

Database schema DSL – an extension of the borge language. Describes tables, columns, types, relations, CRUD operations, and row-level security in sexp. The spec is the artifact – borge does NOT generate SQL or code. Use borge make to have an agent edit the spec interactively. Postgres-first semantics.

[status: implemented]

relationship-to-borge

DB is an extension of the borge language, not a separate language

[status: implemented]

Borge DB is NOT a new language. It is a set of new sexp forms that live inside existing .borg files. Every borge command that works today works on DB sections without changes:

- borge balance: paren-checks DB sections like any other
- borge parse: parses DB forms as sexp like anything else
- borge fmt: formats DB sections with canonical indentation
- borge check: validates DB section files exist and are well-formed
- borge lint: validates DB semantics (and gains DB-aware checks)
- borge report: counts DB section statuses like any other section
- borge drift: detects drift between DB spec and actual schema
- borge nodes: dumps DB AST like any other section
- borge make (with appropriate prompt): agent edits DB spec interactively

The principle: borge DB sections are first-class borge citizens. They get the same tooling, the same workflow, and the same spec-is-source-of-truth discipline as every other section.

What IS new: 1. New sexp forms: (db), (table), (column), (type), (references), (operations), (crud), (query), (relation), (join), (groups), (group), (can), (ownership) 2. NO code generation – the spec IS the schema definition 3. Integration with borge-ui: pages can reference (data ...) to declare what queries they need

philosophy

– *agent note*: roerick (2026-05-22), setting the frame for the whole DB DSL: "The db.borg spec is the source of truth. The DB DSL does NOT generate SQL or code – the spec IS the artifact."

Borge DB describes database intent, not SQL. SQL is the implementation – it's the how. The spec is the what: what tables exist, what columns they have, what operations are available, who can access what.

Three core principles:

1. Schema is spec. The .borg file IS the schema definition. There is no code generation. The spec is read by humans and agents who implement it. If the implementation doesn't match, the spec is still correct – the implementation is wrong.
2. Permissions live on the model, not on the page. Row-level security is declared once on the table definition and enforced everywhere by Postgres. Pages don't filter data – the database does. Declare once, enforce everywhere.
3. Postgres-first. The DSL targets PostgreSQL specifically. RLS policies, UUID defaults, timestampz, and gen_random_uuid are first-class concerns, not afterthoughts. Other databases may work but Postgres is the primary target.

Why not just use SQL? For the same reason borge exists at all: the spec is the source of truth. SQL is the generated output. The spec is higher-level intent, diffable, reviewable, and checkable by borge's existing tooling. SQL migrations are derived artifacts.

schema

Table and column definitions

[status: implemented]

Tables are declared inside a (db ...) form:

```
(db (table users (column id (type uuid) (primary-key) (default gen_random_uuid))
(column email (type text) (unique) (not-null)) (column name (type text) (not-
null)) (column created-at (type timestamptz) (default now)))
(table posts (column id (type uuid) (primary-key) (default gen_random_uuid))
(column author-id (type uuid) (references users.id)) (column title (type text)
(not-null)) (column body (type text)) (column published (type boolean) (de-
fault false)) (column created-at (type timestamptz) (default now))))
```

Table names are symbols. Column definitions contain: - (type NAME): the Postgres type – uuid, text, int, boolean, timestamptz, jsonb, etc. - (primary-key): this column is the primary key - (unique): this column has a unique constraint - (not-null): this column cannot be null - (default EXPR): default value expression – gen_random_uuid, now, false, true, or any SQL expression - (references TABLE.COLUMN): foreign key constraint

Multiple columns can share a primary key: (column org-id (type uuid) (primary-key)) (column user-id (type uuid) (primary-key))

The generator produces CREATE TABLE statements with all constraints, defaults, and foreign keys.

column-types

Supported PostgreSQL column types

[status: implemented]

The (type ...) form accepts PostgreSQL type names directly:

Common types: uuid, text, varchar, int, bigint, boolean, timestamptz, date, jsonb, bytea, decimal, float

Array types: (type (array text)) – text[] (type (array uuid)) – uuid[]

The DSL does not abstract over types – it uses Postgres type names verbatim. This is opinionated but honest. If you're targeting Postgres, use Postgres types.

indexes

Index declarations for performance

[status: implemented]

Indexes are declared on tables:

```
(table users (column id (type uuid) (primary-key)) (column email (type
text) (unique) (not-null)) (column name (type text) (not-null)) (index email-
idx (columns email)) (index name-idx (columns name) (method gin)))
(index name (columns col1 col2) (method name) (unique)) - name: sym-
bol identifying the index - columns: one or more columns to index - method:
btree (default), gin, gist, hash, spgist, brin - unique: whether the index has a
uniqueness constraint
```

The generator produces CREATE INDEX statements.

operations

What you can do with the data – CRUD and queries

[status: implemented]

Operations declare what actions are available on a table:

```
(operations users (crud) ;; full CRUD: create, read, update, delete (query
by-email) ;; lookup by email (query recent (limit 10))) ;; recent users
```

```
(operations posts (crud (except delete)) ;; everything except delete (query
by-author) ;; posts by a specific author (query published-recent (limit 20))) ;;
recent published posts
```


(crud) declares standard operations: - (crud): create, read, update, delete – all five - (crud (except delete)): all except delete - (crud (except create delete)): only read and update - (crud (only read)): read-only

(query name) declares a named query: - name: symbol identifying the query - (limit N): optional result limit - The query details (WHERE clause, JOINS, etc.) are determined by the generator from context and naming conventions.

Operations are separate from the table schema because: - Not every table needs CRUD (e.g., join tables) - Not every table allows delete - Queries are application-specific, not schema-level - The generator produces route handlers from operations

relations

Joins and cross-table queries

[status: implemented]

Relations describe queries that span multiple tables:

(relation user-posts (from users) (join posts (on posts.author-id = users.id)) (select users.name posts.title posts.published) (query recent-posts-with-author))
(relation published-with-author (from posts) (join users (on users.id = posts.author-id)) (where (posts.published = true)) (select users.name posts.title posts.created-at) (query feed (limit 20)))

Parts: - (from TABLE): the primary table - (join TABLE (on CONDITION)): inner join - (left-join TABLE (on CONDITION)): left join - (where CONDITION): filter condition - (select COL1 COL2 ...): columns to return - (query NAME (limit N)): named query from this relation

The generator produces SQL views or application-level queries from relations. For simple cases, a view; for complex cases, a parameterized query function.

Relations integrate with the UI: a page can declare (data (relation-name query-name)) to indicate it uses a specific relation's query.

ownership

Row-level ownership – who owns a row

[status: implemented]

(ownership column-name) declares which column determines who owns a row. This is the most common RLS pattern:

(table posts (column id (type uuid) (primary-key)) (column author-id (type uuid) (references users.id)) (column title (type text) (not-null)) (column body (type text)) (column published (type boolean) (default false)) (ownership author-id))

(ownership author-id) means: - Users own rows where author-id matches their identity - The generator produces RLS policies: - SELECT: users can read their own rows (and published ones, if a group rule says so) - INSERT: users can insert rows with their own author-id - UPDATE: users can update their own rows - DELETE: users can delete their own rows

Ownership is a shorthand. Under the hood, it's equivalent to declaring a group with (can posts (all) (where author-id = current-user)). The shorthand exists because "author owns their own rows" is by far the most common pattern.

For more complex ownership models, use the groups system directly and skip (ownership ...).

groups

Access control groups and their capabilities

[status: implemented]

Groups define who can do what, at the row level. The generator produces Postgres RLS policies from group declarations.

(groups (group admin (can-all)) ;; admins can do everything (group author (can posts (create read update) (where author-id = current-user)) (can users (read (where id = current-user)))) (group reader (can posts (read (where published = true)))) (can users (read))))

(group name ...) declares a named group: - name: symbol identifying the group - Body: one or more (can ...) declarations

(can-all): shorthand for all operations on all tables. Equivalent to bypasses RLS. Used for admin/superuser roles.

(can TABLE OPERATIONS (where CONDITION)): - TABLE: which table this rule applies to - OPERATIONS: create, read, update, delete, or combinations - (read): can only read - (create read): can create and read - (create read update delete): full CRUD - (where CONDITION): row-level filter - (where author-id = current-user): column matches current user - (where published = true): column matches a value - (where id = current-user): specific row match

The generator translates groups into RLS policies:

(group admin (can-all)) ? ALTER TABLE ... ENABLE ROW LEVEL SECURITY; CREATE POLICY admin_all ON ... FOR ALL TO admin USING (true) WITH CHECK (true);

(group author (can posts (create read update) (where author-id = current-user))) ? CREATE POLICY author_posts ON posts FOR SELECT TO author USING (author_id = current_user_id); CREATE POLICY author_posts_insert ON posts FOR INSERT TO author WITH CHECK (author_id = current_user_id);

...

(group reader (can posts (read (where published = true)))) ? CREATE POLICY reader_posts ON posts FOR SELECT TO reader USING (published = true);

The (current-user) expression in (where ...) is resolved by the generator based on the convention. For Dream/wyo.auth, it becomes the session user ID. The mapping is convention-specific.

group-resolution

How group membership is determined at runtime

[status: planned]

The spec declares groups declaratively. How a user is assigned to a group at runtime is convention-specific:

- wyo.auth convention: group is stored as a column on the users table, resolved via the Dream session
- JWT convention: group is a claim in the token
- Application convention: group is resolved by a lookup query at request time

The generator produces the right resolution code based on the convention. The spec just says "these groups exist and these are their capabilities." The convention says "here's how you know which group a request belongs to."

A common pattern: the users table has a (column role (type text)) that stores the group name. The generator produces a session middleware that sets the Postgres role per request.

data-integration

How pages reference database queries

[status: planned]

Pages in borge-ui can declare what data they need using the (data ...) form:

(page dashboard (data (posts published-recent)) (use-layout sidebar-shell) (fill content ...))

(page profile (data (users by-id)) (data (posts by-author)) (use-layout app-shell) ...)

(data (TABLE QUERY)) means: this page loads the named query on the specified table. The generator produces: - A query function or ORM call in the

page handler - The data is available to the page template - RLS automatically filters results based on the user's group

The page doesn't declare permissions – the schema and groups handle that. The page just says "I need this data." Postgres RLS ensures the user only sees what they're allowed to see.

For relations:

(page feed (data (published-with-author feed)) (use-layout app-shell) ...)

This loads the "feed" query from the "published-with-author" relation.

full-example

A complete example: blog application database

[status: planned]

```
(db blog (table users (column id (type uuid) (primary-key) (default gen_random_uuid))
(column email (type text) (unique) (not-null)) (column name (type text) (not-null))
(column role (type text) (not-null) (default "reader")) (column created-at
(type timestampz) (default now)) (index email-idx (columns email))))
```

```
(table posts (column id (type uuid) (primary-key) (default gen_random_uuid))
(column author-id (type uuid) (references users.id) (not-null)) (column title
(type text) (not-null)) (column body (type text)) (column published (type
boolean) (default false)) (column created-at (type timestampz) (default now))
(column updated-at (type timestampz)) (ownership author-id) (index author-idx
(columns author-id)) (index published-idx (columns published created-at))))
```

```
(table comments (column id (type uuid) (primary-key) (default gen_random_uuid))
(column post-id (type uuid) (references posts.id) (not-null)) (column author-id
(type uuid) (references users.id) (not-null)) (column body (type text) (not-null))
(column created-at (type timestampz) (default now)) (ownership author-id)
(index post-idx (columns post-id)))
```

```
(operations users (crud (except delete)) (query by-email) (query by-id))
```

```
(operations posts (crud (except delete)) (query by-author) (query published-recent
(limit 20)) (query by-id))
```

```
(operations comments (crud (except delete)) (query by-post))
```

```
(relation post-with-author (from posts) (join users (on users.id = posts.author-id))
(select users.name posts.title posts.body posts.published posts.created-at)
(query detail))
```

```
(relation feed-with-authors (from posts) (join users (on users.id = posts.author-id))
(where (posts.published = true)) (select users.name posts.title posts.created-at)
(query feed (limit 20)))
```

```
(groups (group admin (can-all)) (group author (can posts (create read update)
(where author-id = current-user)) (can comments (create read update)
(where author-id = current-user)) (can users (read (where id = current-user))))
(group reader (can posts (read (where published = true))) (can comments
(read)) (can users (read)))))
```

convention-targets

Code generators: walk the typed DB AST and emit target source

[status: in-progress]

Generators are consumers of `db_ast` – the same relationship as `db_json` (which emits JSON) and `db_validate` (which emits issues). A generator walks the `db_app` tree and produces source code for a specific target.

Module layout: `lib/db/db_sql.ml` – postgres-sql generator (DDL)

CLI: `borge generate -target postgres-sql DB_SPEC.borg`

The generator exposes: `val generate : db_app -> string`

The string is the complete output: a .sql file containing all CREATE TABLE, CREATE INDEX, CREATE POLICY, and ALTER TABLE statements. The caller writes it to disk.

Event emission: Each successful generation prints a structured event to stderr: (generated db-APP-NAME postgres-sql at PATH) This provides the audit trail for future drift detection and the cybernetic loop.

Idempotency: The generated SQL uses IF NOT EXISTS for tables and indexes so the script can be re-run safely. RLS policies are DROP and re-CREATE since they have no IF NOT EXISTS in PostgreSQL. The script is ordered: tables first (respecting foreign keys), then indexes, then RLS enablement, then policies, then groups.

postgres-sql-target

Generate PostgreSQL DDL from a DB spec

[status: planned]

Target: PostgreSQL DDL. Produces a single .sql file that creates the complete schema: tables with constraints and defaults, indexes, row-level security, and group policies. The output can be run as a migration or as a complete schema bootstrap.

Mapping table – DB AST ? PostgreSQL DDL:

Tables: (table users (column id (type uuid) (primary-key) (default gen_random_uuid)) (column email (type text) (unique) (not-null))) ? CREATE TABLE IF NOT EXISTS users (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), email TEXT UNIQUE NOT NULL);

Column types are passed through verbatim since the DSL uses PostgreSQL type names directly. No translation needed.

Column constraints: (primary-key) ? PRIMARY KEY (unique) ? UNIQUE (not-null) ? NOT NULL (default EXPR) ? DEFAULT EXPR gen_random_uuid ? gen_random_uuid() now ? now() false ? FALSE true ? TRUE Other values ? passed through as-is

Foreign keys: (references TABLE.COLUMN) ? REFERENCES TABLE (COLUMN) Foreign key columns are implicitly NOT NULL if (not-null) is present, or nullable otherwise. ON DELETE CASCADE is NOT added by default – the spec must declare cascade behavior explicitly when needed (future extension).

Indexes: (index email-idx (columns email)) ? CREATE INDEX IF NOT EXISTS email-idx ON users (email);

(index name-idx (columns name) (method gin)) ? CREATE INDEX IF NOT EXISTS name-idx ON users USING gin (name);

(unique) flag: ? CREATE UNIQUE INDEX IF NOT EXISTS name-idx ON users (name);

Ownership: (ownership author-id) ? ALTER TABLE posts ENABLE ROW LEVEL SECURITY; CREATE POLICY posts_author_select ON posts FOR SELECT USING (author_id = current_user_id()); CREATE POLICY posts_author_insert ON posts FOR INSERT WITH CHECK (author_id = current_user_id()); CREATE POLICY posts_author_update ON posts FOR UPDATE USING (author_id = current_user_id()) WITH CHECK (author_id = current_user_id()); CREATE POLICY posts_author_delete ON posts FOR DELETE USING (author_id = current_user_id());

The ownership shorthand generates four policies (SELECT, INSERT, UPDATE, DELETE) all scoped to the ownership column matching the current user. The current_user_id() function is convention-specific – the generator emits a comment noting that this function must be defined by the application.

Groups: (group admin (can-all)) ? CREATE POLICY admin_all ON posts FOR ALL USING (true) WITH CHECK (true);

(group author (can posts (create read update) (where author-id = current-user))) ? CREATE POLICY author_posts_select ON posts FOR SELECT USING (author_id = current_user_id()); CREATE POLICY author_posts_insert ON posts FOR INSERT WITH CHECK (author_id = current_user_id()); CREATE POLICY author_posts_update ON posts FOR UPDATE USING (author_id = current_user_id()) WITH CHECK (author_id = current_user_id());

(group reader (can posts (read (where published = true)))) ? CREATE POLICY reader_posts_select ON posts FOR SELECT USING (published = true);

Group policy naming convention: GROUPNAME_TABLE_OPERATION e.g. author_posts_select, reader_posts_select This avoids collisions when multiple groups access the same table.

(can-all) generates a single policy for ALL operations with USING (true) and WITH CHECK (true) – effectively bypasses RLS. Used for admin/superuser roles.

The (where ...) clause: (where author-id = current-user) ? author_id = current_user_id() (where published = true) ? published = true (where id = current-user) ? id = current_user_id() The (current-user) token is the only special token in where clauses. It translates to current_user_id() in SQL. All other values are passed through as SQL literals. Boolean: true ? TRUE, false ? FALSE. String: passed as-is (caller must ensure SQL safety).

Operations and queries: (crud) ? No DDL generated. CRUD is handled by RLS. The (crud) form is for application-layer routing, not schema definition. (query by-email) ? Comment in the SQL file: – TODO: query by_email for table users Named queries are application-level concerns. The generator emits them as comments for documentation, not as SQL functions. Future: optional –create-views flag to generate views from (relation ...) forms.

Relations: (relation feed-with-authors ...) ? – TODO: relation feed_with_authors – FROM posts JOIN users ON users.id = posts.author_id – WHERE posts.published = true – SELECT users.name, posts.title, posts.created_at Same as queries: relations are documented as comments, not generated as views by default. The –create-views flag (future) would generate CREATE VIEW statements.

Generation order: 1. CREATE TABLE statements (topologically sorted by foreign key dependencies – referenced tables before referencing) 2. CREATE INDEX statements 3. ALTER TABLE ... ENABLE ROW LEVEL SECURITY 4. CREATE POLICY statements (ownership shorthand first, then explicit group policies) 5. Comments for operations, queries, and relations

Topological sort for foreign keys: The generator builds a dependency graph from (references) clauses and emits tables in an order where no table is created before the table it references. Circular references are an error (detected at generation time). Self-references are allowed (the column is nullable or deferrable).

Schema header: The generated file begins with a comment header: – Generated by borge generate –target postgres-sql – Source: path/to/spec.borg – Generated at: ISO-8601 timestamp – DO NOT EDIT: regenerate from spec This makes it clear the file is derived, not handwritten. Drift detection (borge drift) can check if the live schema matches this generated DDL.

agentic-editing

How agents edit DB specs using borge make

[status: implemented]

borge make

1. Agent reads the current .borg files with (db ...) sections 2. Agent identifies planned sections or gaps 3. Agent adds or modifies (table ...), (operations ...), etc. 4. Agent validates: borge balance, borge check 5. Agent commits changes

This is NOT code generation. The spec IS the artifact. The agent is helping write the spec, just as it helps write any other .borg section.

Drift detection: borge drift can check whether the actual database schema matches the spec by comparing against the migration history or by introspecting the live database. This is advisory – the spec is still the source of truth.

future-work

Possible extensibility for the DB DSL

[status: planned]

- Enums: (enum status (draft published archived)) - Triggers: (trigger update-timestamp (on update) (set updated-at = now)) - Checks: (check positive-price (column price > 0)) - Composite types and domains - Many-to-many join tables: (join-table user-roles users roles) - Soft delete: (soft-delete) instead of physical delete - Audit logging: (audit) tracks changes on a table
- Database functions and stored procedures - Schema namespacing (multiple schemas in one Postgres database) - Partial indexes with WHERE conditions
- Database-level constraints (exclusion constraints)

4 ui.borg

borge-ui

Declarative UI DSL – an extension of the borge language, not a separate one. Adds (theme), (component), (layout), (variant), (page), (routes) and related forms inside .borg sections. All existing borge commands work on UI sections. The spec is the artifact – borge does NOT generate code. Renderer-agnostic semantics for human interpretation and implementation.

[status: implemented]

relationship-to-borge

UI is an extension of the borge language, not a separate language

[status: implemented]

Borge UI is NOT a new language. It is a set of new sexp forms that live inside existing .borg files. Every borge command that works today works on UI sections without changes:

- borge balance: paren-checks UI sections like any other
- borge parse: parses UI forms as sexp like anything else
- borge fmt: formats UI sections with canonical indentation
- borge check: validates UI section files exist and are well-formed
- borge lint: validates UI semantics (and gains UI-aware checks)
- borge report: counts UI section statuses like any other section
- borge drift: detects drift between UI spec and generated code
- borge nodes: dumps UI AST like any other section
- borge make (with appropriate prompt): agent edits UI spec interactively

The new forms are just sexp – the parser already handles them. A (ui ...) section is another section body, same as (doc ...) or (details ...). Balance, formatting, and reporting all work on day one with zero code changes to the existing tools.

What IS new: 1. New sexp forms: (theme), (component), (layout), (variant), (page), (routes), (use), (fill), (use-layout), (slot) 2. NO code generation – the spec IS the UI definition 3. Future: semantic awareness – lint validates that (use button) refers to a defined component, drift checks generated UI code, etc. But this is incremental enhancement, not a prerequisite.

The principle: borge UI sections are first-class borge citizens. They get the same tooling, the same workflow, and the same spec-is-source-of-truth discipline as every other section.

philosophy

Borge UI is a declarative language for describing user interfaces. It borrows Clay's insight: UI layout is just nested boxes with flexbox-like rules. But where Clay is a C layout library, borge UI is a spec – it describes **what** the interface should look like, not **how** to render it.

The five core principles:

1. Describe intent, not pixels. Sizing defaults to fractional parts (flex portions), not absolute units. You say "sidebar gets 1 part, content gets 3 parts" and the generator handles the rest.
2. Renderer-agnostic. The same spec can generate HTML+CSS for a Dream web app, React components, Clay C code, or any other target. The convention system determines the output format.
3. Shared base, named variants. Responsive design is handled through named variants that overlay the base properties. You write shared style once, then describe only what changes at different sizes.
4. Theme variables as the foundation. Colors, spacing, font sizes, and corner radii are defined once as theme variables and referenced everywhere.

Change the theme, change the whole app. No hardcoded values scattered across elements.

5. Components, layouts, and pages are separate concerns. Components are small reusable pieces (buttons, cards, inputs). Layouts are page-level compositions with slots (nav bar, sidebar, content area). Pages fill layout slots with specific content. You don't use a layout the same way you use a component – they're different granularity.

Interactivity is a capability declaration, not a handler binding. (interactive click) means "this element is clickable" – the target generator decides how to implement that. The spec does not couple to any host language's function naming or event system.

theme

Theme variables – shared values referenced by all UI elements

[status: implemented]

The (theme ...) form defines named variables for colors, spacing, font sizes, and corner radii. Every UI element references these variables instead of hardcoding values. Change the theme, change the whole app.

(theme (palette bg-primary "#e0d7d2") (palette bg-surface "#f5f0eb") (palette text-primary "#1a1a1a") (palette text-muted "#666666") (palette accent "#a8421c") (spacing xs 4) (spacing sm 8) (spacing md 16) (spacing lg 24) (spacing xl 32) (font-size sm 12) (font-size base 14) (font-size lg 18) (font-size xl 24) (radius sm 4) (radius md 8) (radius lg 12))

Referencing palette variables in elements:

(bg (palette bg-primary)) (color (palette text-primary))

Referencing spacing, font-size, radius variables:

(padding (spacing md)) (text "Hello" (size (font-size lg))) (corner (radius md))

The generator resolves variables to literal values per target. CSS: var(-bg-primary) or inline. Clay: hex literal. React: theme context or CSS variable.

Two-phase color workflow:

1. Sketch phase: use keyword colors like (bg blue), (color white), (bg dark). The LLM picks actual hex values when generating code. Fast for prototyping.
2. Refinement phase: define a (palette ...) and reference it like (bg (palette primary)). Precise, unified, consistent across the whole app.

Keyword colors and palette references can coexist. The spec doesn't force precision before you're ready. A work-in-progress might have (bg blue) on some elements and (bg (palette primary)) on others. The palette wins during refinement.

Recognized keyword colors: red, orange, yellow, green, blue, purple, pink, white, black, gray, dark, light, transparent. The generator interprets these per target and convention.

Theme variables are flat: (palette name value), (spacing name value), (font-size name value), (radius name value). No nesting, no grouping beyond the category prefix. This keeps the theme simple to read and simple to generate from.

ui-form

The (ui ...) form – the core building block

[status: implemented]

A ui element is declared with the (ui name ...) form. Name is a symbol identifying the element. The body contains property declarations and children.

(ui sidebar (layout vertical) (width 1) (height grow) (padding (spacing md)) (gap (spacing sm)) (bg (palette bg-primary)) (children (ui header ...)))

Name is optional for anonymous elements: (ui (text "Status") (size (font-size sm)))

Properties can appear in any order. Children are declared explicitly with (children ...). This keeps the structure unambiguous – properties and children are never mixed.

layout-properties

How elements are arranged inside their parent

[status: implemented]

(layout direction) Direction: horizontal (default) or vertical. Children are laid out along the chosen axis.

(layout horizontal) – left to right (layout vertical) – top to bottom

sizing

How elements size themselves within their parent

[status: implemented]

(width N) – N fractional parts of parent (flex: N) (height N) – same, for height

(width grow) – fill remaining space after siblings are sized (height grow) – same, for height

(width full) – 100% of parent (height full) – same, for height

(width auto) – size to content (height auto) – size to content

Explicit units when needed: (width (px 300)) – 300 pixels (width (rem 2)) – 2rem (width (pct 25)) – 25% (height (vh 80)) – 80% of viewport height

Fractional parts are the default and preferred. A sidebar getting (width 1) and content getting (width 3) produces a 25/75 split without any pixel math. The generator translates "1 part" to whatever the target supports (flex-grow in CSS, SIZING_GROW in Clay, etc.).

The minimum: two numbers. If sidebar is 1 and content is 3, that's a 1:3 ratio. You don't need to think about pixels unless you specifically need them.

spacing

Padding, gaps, and alignment between children

[status: implemented]

(padding N) – equal padding on all sides (padding (top N) (right N) (bottom N) (left N)) – per-side

(gap N) – space between child elements

(align-x left) – horizontal alignment of children (align-x center) (align-x right)

(align-y top) – vertical alignment of children (align-y center) (align-y bottom)

Padding and gap values accept the same unit types as sizing: bare numbers are px (since fractional padding is rare), or use (rem 1), (pct 5), etc. Theme variable references also work: (padding (spacing md)), (gap (spacing sm)).

style-properties

Visual appearance – colors, borders, corners

[status: implemented]

(bg COLOR) – background color (color COLOR) – text color (inherits to children) (border (width N) (color COLOR)) – border on all sides (border (bottom (width N) (color COLOR))) – specific side (corner N) – uniform corner radius in px (corner (top-left N) (top-right N) (bottom-left N) (bottom-right N))

Color values accept three forms: - Hex: "#a8421c" or "#a8421c99" (with alpha) - Keyword: blue, red, green, dark, white, etc. For sketching – the LLM picks actual values when generating. - Palette reference: (palette name) For refined specs – references a (palette ...) theme variable.

The two-phase workflow: sketch with keywords, refine with palette. (bg blue) during prototyping, (bg (palette primary)) later.

For opacity: "#a8421c99" (hex with alpha) or (bg "#a8421c" (alpha 0.6)).

text

Text elements

[status: planned]

(text STRING) – text content (text STRING (size N)) – with font size (text STRING (size N) (color COLOR)) (text STRING (bold)) (placeholder STRING) – placeholder text for inputs

Text inherits color from parent unless overridden. Font size in px by default, or (size (rem 1.5)), or (size (font-size base)) for theme variable. (bold) and (italic) are boolean flags.

Text wrapping: (wrap words) is default – text wraps at word boundaries. (wrap none) disables wrapping. (wrap newlines) wraps only on newline characters.

interactive

Capability declarations – what the user can do to an element

[status: implemented]

(interactive click) – this element responds to clicks/taps (interactive type) – this element accepts text input (interactive hover) – this element responds to hover/focus

Interactive is optional. Omit it for non-interactive elements (labels, spacers, containers). When present, it tells the generator: "make this element respond to this kind of input."

The spec does NOT describe handler implementation – no function names, no callback wiring. That's code, not spec. The generator adds the right event bindings for the target: - HTML: an onclick attribute, event listener, or htmx trigger - React: an onClick prop - Clay: an OnHover callback

Example: (ui save-button (interactive click) (text "Save") (bg (palette accent)) (padding (top (spacing sm)) (right (spacing md)) (bottom (spacing sm)) (left (spacing md)))) (corner (radius sm)))

scroll

Scrollable containers

[status: implemented]

(scroll vertical) – scroll content vertically (scroll horizontal) – scroll content horizontally (scroll both) – scroll in both directions

floating

Elements that float above the normal layout flow

[status: implemented]

(float) – float above siblings, absolute positioned (float (attach parent)) – position relative to parent (float (offset (top 8))) – offset from attachment point

Floating elements don't affect their parent's size or sibling positioning. They stack on top. Use for tooltips, modals, dropdowns.

image

Image elements

[status: planned]

(image NAME) – display an image by name/reference (image NAME (width N) (height N)) – with explicit size

The image name is a logical reference. The generator maps it to the target's asset system (file path, URL, etc.).

variants

Named variants for responsive design

[status: implemented]

An element can have named variants. Each variant overlays the base properties – you only specify what changes.

```
(ui sidebar (padding (spacing md)) (gap (spacing sm)) (bg (palette bg-primary)) (children ...))
```

```
(variant wide (layout vertical) (width 1) (height grow))
```

```
(variant narrow (layout horizontal) (width full) (height auto)))
```

Shared properties (padding, gap, bg, children) are written once. Each variant only contains the properties that differ from the base.

Variant names are semantic: wide, narrow, compact, expanded, etc. The generator decides what breakpoints or conditions map to each variant. The spec says "there are two shapes"; the convention says "wide = >768px, narrow = <=768px".

Properties in a variant override the base. Properties not listed in a variant are inherited from the base unchanged.

Children declared in a variant replace the base children. If children don't change between variants, declare them in the base and omit from variants.

children

Declaring child elements

[status: implemented]

Children are declared explicitly with (children ...):

```
(ui card (layout vertical) (padding (spacing md)) (bg (palette bg-surface))
(corner (radius lg)) (children (text "Card Title" (size (font-size lg)) (bold))
(text "Card body text goes here." (size 14)) (ui action-row (layout horizontal)
(gap (spacing sm)) (children (ui btn (interactive click) (text "OK") (bg (palette
accent)))) (ui btn (interactive click) (text "Cancel") (bg "#999"))))))
```

Children are ordered: the order in (children ...) is the order on screen, following the layout direction.

An element with no (children ...) has no children.

components

Reusable UI pieces – defined once, used anywhere

[status: implemented]

A component is a reusable UI element defined with the (component name ...) form. Components encapsulate structure and style. They're the small pieces – buttons, cards, form fields – that get composed into layouts and pages.

Components have slots: places where content goes when the component is used.

Defining a component:

```
(component button (interactive click) (padding (top (spacing sm)) (right
(spacing md)) (bottom (spacing sm)) (left (spacing md))) (corner (radius sm))
(align-x center) (align-y center) (variant primary (bg (palette accent)) (color
"#ffffff")) (variant secondary (bg (palette bg-surface)) (color (palette text-
primary))))
```

```
(component card (layout vertical) (padding (spacing md)) (bg (palette bg-
surface)) (corner (radius lg)) (gap (spacing sm)) (children ..content)) ;; ~
default slot: children go here when used
```

Using a component with (use ...):

```
(ui save-btn (use button (variant primary)) (text "Save")) (ui info-card (use
card) (children (text "Welcome" (size (font-size lg))) (text "Details go here."
(size 14)))))
```

The `(use component-name)` form instantiates a component. Properties after `(use ...)` override component defaults. `(variant name)` selects which variant to apply.

Slots: `..content` – the default slot. Children of the using element fill this slot. `(slot name)` – named slot. Filled with `(fill name ...)`.

Named slots let a component define multiple insertion points. For example, a card with a header slot and a body slot:

```
(component card-with-header (layout vertical) (padding (spacing md)) (bg
(palette bg-surface)) (corner (radius lg)) (children (slot header) (ui divider
(height 1) (bg "#ddd")) ..content))
```

Filling named slots when using the component:

```
(ui my-card (use card-with-header) (fill header (text "Card Title" (size (font-
size lg)) (bold))) (children (text "Body text goes here.")))
```

Components are NOT layouts. A component is a small reusable piece. A layout is a page-level composition. You don't drop a layout into a card, and you don't slot a button into a page shell. Different granularity, different intent.

layouts

Page-level compositions with slots – structural templates for pages

[status: implemented]

A layout is a structural template that defines the shape of a page. It uses slots where page-specific content goes. Layouts are composed from components but are conceptually different: they're structural, not decorative.

Defining a layout:

```
(layout app-shell (ui root (layout vertical) (width full) (height full) (bg
(palette bg-primary)) (children (ui nav-bar (layout horizontal) (width full)
(height auto) (bg (palette accent)) (padding (spacing sm)) (align-y center) (gap
(spacing md)) (children (text "App Name" (size (font-size lg)) (color "#ffff"))
(ui spacer (width grow) (height 1)) (slot nav-content))) (ui body (layout hori-
zontal) (width full) (height grow) (children ..content)))))
```

`(slot name)` declares a named slot. `(fill name ...)` fills it. `..content` is the default slot – children of the using page or filling element go here.

Layouts often use components inside them:

```
(layout sidebar-shell (ui root (layout vertical) (width full) (height full) (chil-
dren (ui top-bar (layout horizontal) (width full) (height auto) (padding (spacing
sm)) (bg (palette accent)) (children ..nav)) (ui body (layout horizontal) (width
full) (height grow) (gap (spacing md)) (children (ui sidebar (layout vertical)
(width 1) (height grow) (bg (palette bg-surface)) (corner (radius md)) (padding
(spacing md)) (children ..sidebar)) (ui main (layout vertical) (width 3) (height
grow) (scroll vertical) (children ..content)))))
```

Layouts are NOT components. A layout defines page structure. A component defines a reusable piece. You compose layouts from components, but you don't compose components from layouts. They're separate because they serve different purposes and are used at different scales.

pages

Concrete pages – fill layout slots with specific content

[status: implemented]

A page is a concrete screen in the application. It picks a layout and fills its slots with specific content.

```
(page home (use-layout app-shell) (fill nav-content (ui nav-link (interac-
tive click) (text "Home"))) (ui nav-link (interactive click) (text "About"))) (fill
content (ui hero (layout vertical) (padding (spacing xl)) (align-x center) (gap
```

(spacing lg)) (children (text "Welcome" (size (font-size xl))) (text "Build something great." (size (font-size base))))))

(page dashboard (use-layout sidebar-shell) (fill nav (use button (variant secondary)) (text "Dashboard") (use button (variant secondary)) (text "Settings"))) (fill sidebar (use card) (children (text "Nav A"))) (use card) (children (text "Nav B"))) (fill content (use card) (children (text "Widget 1") (use card) (children (text "Widget 2")))))

(use-layout name) selects which layout the page uses. (fill slot-name ...) provides content for a named slot. Unfilled slots are empty – the generator produces an empty container in their place.

Pages can also override layout properties for per-page customization. For example, a page might change the nav bar background:

(page special (use-layout app-shell (override nav-bar (bg "#333"))) ...)

But prefer keeping layout overrides minimal. If multiple pages need the same override, make a new layout variant instead.

workflows

User journeys – sequences of actions across pages

[status: implemented]

A workflow defines how a user moves through the application. It connects pages via actions (click, type, hover) and outcomes (navigate to another page, submit a form).

(workflow login-to-dashboard (doc "Authenticate and land on dashboard") (step welcome-page (page home) (action click "get-started") (outcome navigate "/login"))) (step login-page (page login) (action type "username") (action type "password") (action click "sign-in") (outcome navigate "/dashboard"))) (step dashboard-landing (page dashboard) (action click "new-post")))

Every page in a UI spec should appear in at least one workflow. Pages not part of any workflow are considered dead code – they are reachable by URL but not by any user journey. Drift detection reports them as orphaned pages.

(workflow name) declares the workflow: - name: symbol identifying the user journey - Body: one or more (step ...) forms and optional (doc ...)

(step name ...) declares a single step in the journey: - (page PAGE_NAME): which page this step is on - (action click "element-name"): user clicks an element - (action type "input-name"): user types into an input - (action hover "element-name"): user hovers an element - (outcome navigate "/path"): next route (must be defined) - (outcome submit): form submission, next step continues - No outcome: terminal step (last step in the workflow)

Validation rules: - Every step must reference a defined page - Every outcome.navigate path must match a defined route - Every action must reference an (interactive ...) element on the referenced page - Every page should appear in at least one step - Every workflow must have a (doc ...) explaining the journey

routes

URL-to-page mapping for multi-page applications

[status: implemented]

The (routes ...) form maps URLs to pages. The generator uses this to produce the routing layer per target.

(routes (route "/" home) (route "/dashboard" dashboard) (route "/settings" settings))

Target-specific generation: - OCaml/Dream: route handler functions - React: router config (React Router, Next.js) - Clay: page switch function

Routes are flat: no nesting, no parameters (yet). Each route maps a path string to a page name. The page must be defined in the same spec.

Future: route parameters (/user/:id), nested routes, route guards (auth), and redirects.

app-structure

How a complete multi-page UI spec is organized

[status: implemented]

A complete application spec has one top-level (ui app-name ...) form containing theme, components, layouts, pages, and routes:

```
(ui my-app (theme (palette bg-primary "#e0d7d2") (palette bg-surface "#f5f0eb")
(palette text-primary "#1a1a1a") (palette accent "#a8421c") (spacing sm 8)
(spacing md 16) (spacing lg 24) (font-size sm 12) (font-size base 14) (font-size
lg 18) (radius sm 4) (radius md 8))
(component button (interactive click) (padding (top (spacing sm)) (right
(spacing md)) (bottom (spacing sm)) (left (spacing md)))) (corner (radius sm))
(variant primary (bg (palette accent)) (color "#ffffff")) (variant secondary (bg
(palette bg-surface)) (color (palette text-primary))))
(component card (layout vertical) (padding (spacing md)) (bg (palette bg-
surface)) (corner (radius lg)) (gap (spacing sm)) (children ..content))
(layout app-shell ...)
(layout sidebar-shell ...)
(page home (use-layout app-shell) ...) (page dashboard (use-layout sidebar-
shell) ...)
(routes (route "/" home) (route "/dashboard" dashboard))))
```

The order matters: theme first (everything references it), then components (layouts use them), then layouts (pages use them), then pages (routes reference them), then routes.

This is the full stack. A small spec might only have (ui name ...) with inline elements and no components or layouts. A large app has all five layers. The DSL scales from prototype to production.

full-example

A complete example: multi-page text editor

[status: planned]

```
(ui text-editor (theme (palette bg-primary "#e0d7d2") (palette bg-surface
"#f5f0eb") (palette bg-dark "#2a2520") (palette text-primary "#1a1a1a") (palette
text-muted "#888888") (palette accent "#a8421c") (spacing xs 4) (spacing sm
8) (spacing md 16) (spacing lg 24) (spacing xl 32) (font-size sm 12) (font-size
base 14) (font-size lg 18) (font-size xl 24) (radius sm 4) (radius md 8) (radius
lg 12))
(component btn (interactive click) (padding (top (spacing sm)) (right (spac-
ing md)) (bottom (spacing sm)) (left (spacing md)))) (corner (radius sm)) (bg
(palette bg-surface)) (color (palette text-primary)) (children ..content))
(component doc-item (interactive click) (padding (spacing sm)) (corner
(radius sm)) (children ..content) (variant active (bg (palette accent)) (color
"#ffffff")) (variant inactive (bg (palette bg-surface)) (color (palette text-muted))))
(layout editor-shell (ui root (layout vertical) (width full) (height full) (bg
(palette bg-primary)) (children (ui header (layout horizontal) (width full) (height
auto) (padding (spacing sm)) (gap (spacing md)) (align-y center) (bg (palette
bg-dark)) (children ..header-content)) (ui body (layout horizontal) (width full)
(height grow) (gap (spacing md)) (padding (spacing md)) (variant narrow (lay-
out vertical)) (children (ui sidebar (layout vertical) (width 1) (height grow)
(bg (palette bg-surface)) (corner (radius md)) (padding (spacing md)) (gap
(spacing sm)) (variant narrow (width full) (height auto)) (children ..sidebar))
(ui content (layout vertical) (width 3) (height grow) (bg (palette bg-surface))
(corner (radius md)) (padding (spacing md)) (scroll vertical) (variant narrow
(width full)) (children ..content))))))
(page editor (use-layout editor-shell) (fill header-content (ui btn-group (lay-
out horizontal) (gap (spacing sm)) (children (ui file-btn (use btn) (text "File"))
(ui edit-btn (use btn) (text "Edit")) (ui spacer (width grow) (height 1)) (ui
settings-btn (use btn) (text "Settings")))) (fill sidebar (ui title (use doc-item
```

```
(variant active)) (text "Chapter 1")) (ui title (use doc-item (variant inactive))
(text "Chapter 2")) (ui title (use doc-item (variant inactive)) (text "Chapter
3")))) (fill content (text "Chapter 1: The Beginning" (size (font-size xl)) (bold))
(text (| It was a dark and stormy night. The wind howled through the narrow
streets, rattling shutters and sending stray papers spiraling into the gutter. |)
(size (font-size base))))))
(routes (route "/" editor)))
```

convention-targets

Code generators: walk the typed UI AST and emit target source

[status: in-progress]

Generators are consumers of `ui_ast` – the same relationship as `ui_json` (which emits JSON) and `ui_validate` (which emits issues). A generator walks the `ui_app` tree and produces source code for a specific target.

Module layout: `lib/ui/ui_dream.ml` – ocaml-dream generator (`dream_html`)
`lib/ui/ui_clay.ml` – clay-c generator (C structs + Clay API calls)

CLI: `borge generate -target ocaml-dream UI_SPEC.borg` `borge generate -target clay-c UI_SPEC.borg`

Each generator exposes: `val generate : ui_app -> string`

The string is the complete output: a `.ml` file for ocaml-dream, a `.c` file + `.h` pair for clay-c. The caller writes it to disk.

Theme resolution: Before generation, all `Palette`/`Spacing_var`/`Font_var`/`Radius_var` references are resolved against the theme. If no theme exists, palette references are an error. Keyword colors (blue, dark, etc.) are resolved to hex defaults inside the generator.

Event emission: Each successful generation prints a structured event to `stderr`: (generated ui-APP-NAME TARGET at PATH) This provides the audit trail for future drift detection and the cybernetic loop.

ocaml-dream-target

Generate `dream_html` OCaml code from a UI spec

[status: planned]

Target: Victor's actual stack. Produces an OCaml module that opens `Dream_html` and constructs the full UI as `dream_html` nodes. The output is a `.ml` file that can be compiled into any Dream application.

Module structure of generated code: `let render_PAGE_NAME () : Html.node = ... let render_COMPONENT_NAME ~(content : Html.node list) () : Html.node = ... let register_routes () = Dream.router [...]`

Mapping table – UI AST ? `dream_html`:

Layout: `Horizontal ? Html.div ~a:[Html.a_class ["flex-row"]] [children] Vertical ? Html.div ~a:[Html.a_class ["flex-col"]] [children]`

Sizing: `(width N) ? Html.a_style "flex: N" (width grow) ? Html.a_style "flex: 1 1 0%" (width full) ? Html.a_style "width: 100%" (width auto) ? no style (natural sizing) (width (px 300)) ? Html.a_style "width: 300px" (width (rem 2)) ? Html.a_style "width: 2rem" (width (pct 25)) ? Html.a_style "width: 25%" (height ...) ? same as width with height property`

Colors: `(bg (palette X)) ? resolve from theme, emit Html.a_style "background-color: #hex" (bg "#a8421c") ? Html.a_style "background-color: #a8421c" (bg blue) ? resolve keyword to hex default (color ...) ? same pattern with "color" CSS property`

Spacing: `(padding N) ? Html.a_style "padding: Npx" (all sides when uniform) (padding (top N) (right N) (bottom N) (left N)) ? Html.a_style "padding: Tpx Rpx Bpx Lpx" (gap N) ? Html.a_style "gap: Npx" (flex gap property)`

Border: `(border (width N) (color C)) ? Html.a_style "border: Npx solid #hex" (border (bottom (width N) (color C))) ? Html.a_style "border-bottom: Npx solid #hex"`

Corner: (corner N) ? `Html.a_style "border-radius: Npx" (corner (top-left N) (top-right N) ...)` ? `Html.a_style "border-radius: TLpx TRpx BLpx BRpx"`

Alignment: (align-x left) ? `Html.a_class ["items-start"]` (align-x center) ? `Html.a_class ["items-center"]` (align-x right) ? `Html.a_class ["items-end"]` (align-y top) ? `Html.a_class ["justify-start"]` (align-y center) ? `Html.a_class ["justify-center"]` (align-y bottom) ? `Html.a_class ["justify-end"]`

Scroll: (scroll vertical) ? `Html.a_style "overflow-y: auto"` (scroll horizontal) ? `Html.a_style "overflow-x: auto"` (scroll both) ? `Html.a_style "overflow: auto"`

Interaction: (interactive click) ? `Html.a_on_click` handler or `Hx.hx_post` (convention-dependent; default: `htmlx`) (interactive type) ? input element with type attribute (interactive hover) ? `CSS :hover` (no code generated)

Float: (float) ? `Html.a_style "position: absolute"` (float (attach parent)) ? `Html.a_style "position: relative"` on parent (float (offset (top 8))) ? `Html.a_style "top: 8px"`

Variants: (variant wide ...) ? `@media (min-width: 768px) { ... }` emitted as a `<style>` block at the top of the page, or as a Tailwind responsive class if the convention specifies Tailwind. Default breakpoint: narrow=0-767, wide=768+

Components: (component X ...) ? `let render_x ~ (content : Html.node list) () : Html.node = ...` The function takes slot content as parameters. Default slot is `~content`. Named slots are additional labeled parameters.

Layouts: (layout X ...) ? `let render_x_layout ~ (slots) () : Html.node = ...` Slots are labeled parameters matching the slot names. Default slot is `~content`.

Pages: (page X (use-layout Y) (fill ...)) ? `let render_x () : Html.node = render_y_layout ~slot_name:content ()` Where content is the filled-in elements.

Routes: (route "/" home) ? `Dream.get "/" (fun _ -> Dream_html.respond (render_home ()))`

Style merging: Multiple (`a_style ...`) attributes on the same element are merged into a single attribute with semicolons. Multiple (`a_class ...`) lists are concatenated. This avoids invalid HTML from duplicate attributes.

Theme as CSS custom properties: When the spec has a theme, the generator emits a `<style>` block at the top of the first page root: `:root { -bg-primary: #e0d7d2; -bg-surface: #f5f0eb; -spacing-md: 16px; -font-size-lg: 18px; -radius-md: 8px; }` Then `(bg (palette bg-primary))` can emit either the resolved value directly or `var(-bg-primary)`. The default is to resolve inline for simplicity. A `-css-vars` flag forces `var()` references.

clay-c-target

Generate Clay C code from a UI spec – for raylib/pico rendering

[status: planned]

Target: Clay layout library rendered via raylib. Produces a pair of `.c/.h` files that construct Clay layout trees callable from a raylib render loop. Primary use: pico firmware screens, but also any C application using Clay.

Assumes: - Clay is initialized (`Clay_CreateArena`, `Clay_Initialize`) - raylib or equivalent renders the Clay output - The generated code is called once per frame inside the render loop

Module structure of generated code: `// ui_app_name.h` – declarations `void layout_app_name(void); void layout_PAGE_NAME(void); Clay_Children render_COMPONENT_NAME(void);`

`// ui_app_name.c` – definitions `Clay_Children render_COMPONENT_NAME(void) { ... }` `void layout_PAGE_NAME(void) { ... }` `void layout_app_name(int screen_width, int screen_height) { ... }`

The top-level layout function selects the active page based on application state (page switch function from the routes section). The caller sets a global variable or passes a page enum to select which page to render.

Mapping table – UI AST ? Clay C:

Layout: Horizontal ? `.layoutDirection = CLAY_LEFT_TO_RIGHT` Vertical ? `.layoutDirection = CLAY_TOP_TO_BOTTOM`

Sizing: (width N) ? `.sizing.width = { CLAY_SIZING_GROW(N), CLAY_SIZING_GROW(N) }` Clay's grow takes {min, max}. (width 1) emits grow with weight 1. (width 3) emits grow with weight 3. Clay distributes space proportionally. (width grow) ? `.sizing.width = { CLAY_SIZING_GROW(0), CLAY_SIZING_GROW(0) }` (width full) ? `.sizing.width = CLAY_SIZING_PERCENT(1.0f)` (100% of parent; Clay uses percent for this) (width auto) ? `.sizing.width = { CLAY_SIZING_FIT, CLAY_SIZING_FIT }` (width (px 300)) ? `.sizing.width = { CLAY_SIZING_FIXED(300), CLAY_SIZING_FIXED(300) }` Clay uses FIXED for pixel values. (height ...) ? same as width with `.sizing.height`

Colors: (bg "#e0d7d2") ? `.backgroundColor = { 224, 215, 210, 255 }` Hex ? RGBA. Alpha defaults to 255. (bg (palette X)) ? resolve from theme, then hex?RGBA (bg blue) ? resolve keyword to default hex, then hex?RGBA Default keyword map: red={220,60,60,255} blue={60,120,220,255} green={60,180,80,255} dark={40,40,40,255} white={255,255,255,255} black={0,0,0,255} gray={128,128,128,255} light={230,230,230,255} transparent={0,0,0,0} (color ...) ? `.textColor = { r, g, b, a }` Same hex?RGBA conversion.

Spacing: (padding N) ? `.padding = { N, N, N, N }` Uniform padding: { top, right, bottom, left } (padding (top T) (right R) (bottom B) (left L)) ? `.padding = { T, R, B, L }` (gap N) ? `.childGap = N`

Border: (border (width N) (color C)) ? Element rendered as `CLAY_RECTANGLE` with border: `.border = { N, N, N, N }` `.borderColor = { r, g, b, a }` (border (bottom (width N) (color C))) ? `.border = { 0, 0, N, 0 }` `.borderColor = { r, g, b, a, 0, 0, r, g, b, a, 0, 0, ... }` Clay uses per-side border widths.

Corner: (corner N) ? `.cornerRadius = { N, N, N, N }` Uniform: { topLeft, topRight, bottomLeft, bottomRight } (corner (top-left TL) (top-right TR) (bottom-left BL) (bottom-right BR)) ? `.cornerRadius = { TL, TR, BL, BR }`

Alignment: (align-x left) ? `.childAlignment.x = CLAY_ALIGN_X_LEFT` (align-x center) ? `.childAlignment.x = CLAY_ALIGN_X_CENTER` (align-x right) ? `.childAlignment.x = CLAY_ALIGN_X_RIGHT` (align-y top) ? `.childAlignment.y = CLAY_ALIGN_Y_TOP` (align-y center) ? `.childAlignment.y = CLAY_ALIGN_Y_CENTER` (align-y bottom) ? `.childAlignment.y = CLAY_ALIGN_Y_BOTTOM`

Scroll: (scroll vertical) ? `.scroll = { .vertical = true }` (scroll horizontal) ? `.scroll = { .horizontal = true }` (scroll both) ? `.scroll = { .vertical = true, .horizontal = true }`

Interaction: (interactive click) ? `.onHover = handle_ELEMENT_NAME_hover` The generator emits a stub handler: `void handle_ELEMENT_NAME_hover(Clay_HoverState state) { // TODO: implement click handling }` The stub is a placeholder. The application fills in the actual click behavior. (interactive hover) ? same as click (Clay uses `OnHover` for both) (interactive type) ? Not directly mappable. Clay doesn't have built-in text input. The generator emits a comment: `/* TODO: text input for ELEMENT_NAME */` and renders the element as a rectangle with a border to indicate an input field.

Float: (float) ? `.position = { .x = 0, .y = 0 }` `.elementIsFloating = true` (float (attach parent)) ? `.position = { .x = 0, .y = 0 }` (relative) (float (offset (top 8))) ? `.position = { .x = offsetX, .y = 8 }`

Text: Text elements are rendered via `CLAY_TEXT`: `CLAY_TEXT(CLAY_STRING("Hello"), (Clay_TextElementConfig) { .fontSize = 14, .textColor = { 26, 26, 26, 255 } });` (text STRING) ? `CLAY_TEXT` with default size/color (text STRING (size N)) ? `.fontSize = N` (text STRING (size (font-size X))) ? resolve from theme, `.fontSize = N` (text STRING (color C)) ? `.textColor = resolved RGBA` (text STRING (bold)) ? `.fontSize = N`, bold is noted in comment (Clay doesn't have a bold flag; the application loads a bold font and the generator emits a TODO)

Variants: (variant wide ...) ? The generator emits conditional layout: `if (screen_width >= 768) { // wide variant properties } else { // base or`

narrow variant properties } Screen dimensions are passed to the top-level layout function. Default breakpoint: 768px. (variant narrow ...) ? The else branch of the wide variant, or the conditional for screen_width < 768. If only one variant exists, the base properties are used when the variant condition is false.

Components: (component X ...) ? Clay_Children render_X(void) { ... } The function returns a Clay_Children that the caller can attach to a CLAY_CONTAINER. Slots are handled by having the caller provide child content between the CLAY_CONTAINER open and close.

Layouts: (layout X ...) ? void layout_X(int screen_width, int screen_height) { ... } Emits the full Clay element tree for this layout. Slot positions are marked with comments: /* SLOT: slot-name */ The application inserts content at these points.

Pages: (page X (use-layout Y) (fill ...)) ? void layout_page_X(int screen_width, int screen_height) { layout_Y(screen_width, screen_height); /* fill slot-name with page content */ } Page fills are inlined at the slot positions.

Routes: (routes ...) ? void layout_app(int screen_width, int screen_height) { switch (current_page) { case PAGE_HOME: layout_page_home(w, h); break; case PAGE_DASHBOARD: layout_page_dashboard(w, h); break; } } + enum page { PAGE_HOME, PAGE_DASHBOARD, ... }; + global: enum page current_page = PAGE_HOME; The application sets current_page to switch screens.

Resolving theme variables for Clay: Clay has no CSS variables. All references must be resolved to literal values at generation time. The generator: 1. Collects all Palette/Spacing_var/Font_var/Radius_var references 2. Looks up each in the theme 3. Substitutes the resolved value inline 4. Errors on unresolved references (no theme = error) This is stricter than the HTML target (which can use CSS vars) but necessary for C – there are no runtime style lookups.

C identifier sanitization: Element and component names are sanitized into valid C identifiers: sidebar ? sidebar nav-bar ? nav_bar bg-primary ? bg_primary .content ? (slot, not an identifier) Prefixes to avoid C keyword collisions: render_ for component functions layout_ for page functions handle_ for event handlers

agentic-editing

How agents edit UI specs using borge make

[status: implemented]

borge make

1. Agent reads the current .borg files with (ui ...) sections 2. Agent identifies planned sections or gaps 3. Agent adds or modifies (component ...), (layout ...), (page ...) 4. Agent validates: borge balance, borge check 5. Agent commits changes

This is NOT code generation. The spec IS the artifact. The agent is helping write the spec, just as it helps write any other .borg section.

Drift detection: borge drift can check whether implementation matches the spec. A section marked (status implemented) whose corresponding code doesn't exist is code drift.

future-work

Possible extensibility for the UI DSL

[status: planned]

- Animation/transition declarations - Grid layout beyond flexbox - Accessibility annotations: (aria label "...") or (role button) - Data binding: (text (bind model.field)) - Route parameters: /user/:id - Nested routes and route guards (auth) - Redirects between pages - Form validation declarations - Input types and constraints

5 cli.borg

borges-cli

Thin CLI entrypoints that call lib functions

[status: implemented]

commands

The individual CLI commands

[status: implemented]

balance

Fast paren balance check: run this FIRST before borge parse

[status: implemented]

balance-verbose

Show paren stack on error: which (matches which) and what keyword it opens

[status: implemented]

parse

Parse a .borg file and validate its structure

[status: implemented]

check

Check all .borg files for syntax, structure, and consistency

[status: implemented]

report

Show status summary across all .borg files

[status: implemented]

nodes

Dump the parsed AST with types and positions for debugging

[status: implemented]

lint

Semantic validation and mechanical code quality checks

[status: implemented]

drift

Detect spec drift, code drift, and structural drift

[status: implemented]

print

Parse and re-print a .borg file

[status: implemented]

book

Print the codebase as a paginated book: borge book print [DIR] [-o STEM].

Emits PDF + manifest sidecar

[status: implemented]

fmt

Auto-format a .borg file to canonical indentation

[status: implemented]

fmt-check

Dry-run: exit 1 if formatting would change the file. For CI.

[status: implemented]

fmt-diff

Show diff between current and formatted output

[status: implemented]

fmt-repair

Repair imbalanced files before formatting

[status: implemented]

When a .borg file has paren imbalance, borge fmt normally fails with a parse error. The `-repair` flag attempts to fix the imbalance first by inferring close-parens from indentation (Parinfer-style), then formats the repaired file.

Usage: `borg fmt -repair FILE`

Process: 1. Attempt normal parse + format 2. If parse fails, run balance repair on the file 3. Attempt parse + format on the repaired text 4. If both fail, report the parse error

This is the recovery path for agent edits that break structural balance. Instead of manual paren counting, the agent should run: `borg fmt -repair FILE`

expand

Expand a section by resolving ai-prompt directives

[status: planned]

version

Print the current version

[status: implemented]

normalize

Human-made-changes: fmt all files, accept comment deletions as intentional

[status: implemented]

inline

Show the inline tree: which .borg files inline which others

[status: implemented]

review

Non-deterministic advisory review: lint + drift

[status: implemented]

history-commands

Git operations that understand borge structure

[status: implemented]

log

git log filtered to .borg file changes
[status: implemented]

diff

git diff for .borg files with status annotations
[status: implemented]

undo

git revert HEAD – convenience command
[status: implemented]

agent-commands

Agent-driven commands that launch pi interactively
[status: implemented]

make

Implement planned sections interactively with pi
[status: implemented]

plan

Edit the spec interactively with pi
[status: implemented]

commit

Finalize the current agent session
[status: implemented]

abort

Abort session and restore .borg files from snapshot
[status: implemented]

output-conventions

How the CLIs produce output.
[status: implemented]

stats

Code intelligence metrics dashboard
[status: implemented]

make-spec

Single-shot spec generation from todo description
[status: planned]
borge make spec "description of what to build"
Single-shot LLM call that produces a .borg section. The agent reads the project context and returns a (status planned) section with proper structure.
This is NOT interactive – for interactive spec editing use borge plan instead.

exit-codes

0 success, 1 issues found, 2 usage error
[status: implemented]

human-output

Default. Formatted text for terminal consumption
[status: implemented]

json-output

-json flag. Structured output for machine consumption
[status: implemented]

quiet-mode

-quiet flag. Suppress all output except errors
[status: implemented]

issue

Issue tracker commands: new, list, show, close
[status: implemented]

modules

Check distributed module specs
[status: implemented]

review

Semantic code review (planned, not fully active)
[status: planned]

6 history.borg

borg-history

Git is the history engine. Every agent action gets a commit.

[status: implemented]

philosophy

Why git IS the history system

[status: implemented]

The original design had a separate .borg-history/ git repo for checkpoints. That was over-engineering. The project already has git. Every agent change gets a commit. The commit IS the checkpoint. The commit message IS the log. Git diff IS the semantic diff engine.

What makes borge history different from plain git is that borge understands the structure of .borg files. It can give you a semantic diff (status transitions, section additions, comment changes) instead of a text diff. But the storage is still git.

agent-commit-discipline

Agent must commit after every action

[status: implemented]

Every agent action ends with a git commit. No exceptions. This is what makes the audit trail work. If the agent does not commit, that is a process violation.

The commit workflow: 1. Agent makes changes 2. Agent runs borge balance + borge check 3. Agent git adds relevant files 4. Agent git commits with a descriptive message 5. Agent runs worktree check (see below)

If balance or check fails, the agent fixes before committing. The commit must never leave the worktree in a broken state.

worktree-check

Verify the worktree is clean and right before declaring done

[status: implemented]

Two levels of worktree verification:

Deterministic (borg check --worktree): - git diff is clean (no uncommitted changes) - borge balance passes on all .borg files - borge check passes on all .borg files - dune build succeeds - dune runtest passes This is pass/fail. CI-friendly. No judgment calls.

Non-deterministic (borg review): - borg lint: any human comment deletions? - borg drift: any undocumented functions? - Does the commit match what the agent claimed to do? - Are there orphaned .borg files? This is advisory. It surfaces information for human review. It does not pass or fail – it informs.

Agents run the deterministic check at the end of every action. Humans run the non-deterministic review when they want to understand what changed.

semantic-operations

Git operations that understand borge structure

[status: implemented]

borg log: git log but with borge-aware summaries. Shows only commits that modified .borg files.

borg diff: git diff for .borg files. Highlights status transitions and section changes.

borge undo: git revert HEAD. Convenience command. Current state is preserved because revert creates a new commit, not a destructive operation.

All of these are git operations that understand the borge model. No separate storage engine needed.

session-reports

Structured HTML output for every agent session

[status: planned]

Every borge command that opens a harness (make, plan, drift -agent) generates a structured HTML session report on commit. The report is a committed artifact in the repo, not a separate database. This makes agent sessions: - Auditable without digging through .borge.lock/ before deletion - Browseable by clicking through .borge-history/ - Renderable by victor (iframe or hx-get) and brochure (as part of the public site) - Persistent - they live in git, not in a database

The report replaces the need for victor's job queue output storage. Instead of streaming agent output to a PostgreSQL table, borge commits the report as HTML. Victor reads it. This is a major architectural simplification: borge owns the content, victor and brochure are just viewers.

report-generation

How session reports are generated during Lock.teardown

[status: planned]

On borge commit, before removing .borge.lock/:

1. Read .borge.lock/prompt.md - what was the agent asked
2. Read .borge.lock/journal - what the agent claimed to do
3. Read .borge.lock/role - implementer / spec-writer / observer
4. Read .borge.lock/state - completed / failed
5. Run git diff HEAD - what actually changed in code
6. Parse .borg status transitions (planned ? implemented)
7. Generate .borge-history/session-{timestamp}.html
8. Generate/update .borge-history/index.html (reverse chronological, newest first)
9. Git add .borge-history/ with the rest of the commit
10. THEN remove .borge.lock/

The report is pure static HTML with embedded CSS. No JavaScript framework. No external dependencies. The CSS uses the same variable set as victor so the visual language is consistent.

lib/lock/session_report.ml is the generator module. Called from Lock.teardown after integrity check passes.

report-structure

What each session report HTML page contains

[status: planned]

Each session-{timestamp}.html contains two views, switchable via a tab or toggle:

VIEW 1: DIFF (default) Shows what changed in the codebase: - File list with +/- line counts - Expandable per-file diffs - Semantic highlights for .borg files: Status transitions (planned ? implemented) in green New spec sections in blue Deleted human comments in red New agent comments in yellow - Standard diff coloring for code files - Summary: N files changed, N insertions, N deletions

VIEW 2: FULL SESSION Shows the complete agent session: - Role badge (Implementer / Spec-writer / Observer) - Prompt summary (first N lines, click to expand full) - Full journal entries (timestamped, structured) - State: Completed / Failed - Time elapsed (from lock creation to teardown) - Files touched summary table - Confirmed: integrity check passed / failed

Both views are in a single HTML file with CSS-driven tab switching. No JS required for basic functionality.

report-index

The `.borge-history/index.html` session browser

[status: planned]

`.borge-history/index.html` lists all sessions in reverse chronological order. Each entry shows: - Timestamp - Role badge - Session summary (first line of journal or prompt) - File count and +/- summary - Status (completed / failed)

Clicking an entry navigates to `session-{timestamp}.html`. The index defaults to highlighting the latest session.

`borge history`: Opens `file://` or launches a simple HTTP server pointing at `.borge-history/index.html`. If `$BROWSER` is set, opens in browser. Otherwise, prints the path.

`borge history -serve [-port 8080]`: Starts `python3 -m http.server` (or similar) at `.borge-history/` for interactive browsing.

diff-rendering

Semantic diff rendering for `.borg` files

[status: planned]

The diff view in session reports uses the same semantic awareness as `borge diff`: - Status transitions: (status planned) ? (status implemented) rendered as highlighted green with a transition arrow - New sections: (section ...) additions in blue - Deleted human comments: red with warning icon - New agent comments: yellow with agent icon - Plain text diff: standard +/- coloring

The diff renderer is a function: `git_diff_output ? structured_html`

It does NOT use `x-ocaml` (which is for embedding executable OCaml in web pages). Diff rendering is pure server-side HTML generation. `x-ocaml` could be used later for interactive code examples in reports, but is NOT needed for diff rendering.

For code file diffs (`.ml`, `.go`, `.c`, etc.), the renderer provides standard unified diff format with line numbers and syntax-aware highlighting where available.

victor-integration

How victor consumes session reports instead of job queue output

[status: planned]

With session reports in `borge`, `victor`'s architecture simplifies dramatically:

BEFORE (current): Victor job panel ? PostgreSQL job queue ? Worker process ? Stream output ? Store in DB ? Render in panel

AFTER (with session reports): Victor job panel ? Read `.borge-history/session-{id}.html`

The job panel becomes a list of links to session reports. No job queue table. No output streaming. No worker polling. No DB writes for content. Victor is a VIEWER for `borge`'s committed output, not a storage engine.

The PostgreSQL table for jobs shrinks to: `id`, `session_path`, `status`, `started_at`, `completed_at` The actual content lives in the repo.

Victor can still trigger `borge make / plan / drift` via `htmx POST`, but it no longer needs to store the output. It waits for the commit, then reads the session report.

See `victor.borg` section `session-reports-consumption` for the full `victor`-side spec.

brochure-integration

How brochure renders build history as part of the public site

[status: planned]

Brochure walks `.borge-history/` and includes it in the generated static site. The public gets a Build History page linked from the landing page. Each session is browsable with the same visual treatment as the rest of the brochure site.

See `brochure.borg` section `build-history` for the full brochure-side spec.

7 lib.borg

borg-lib

Domain operations: checking, reporting, formatting, spec extraction. Organized into concern-based subdirectories under lib/ with include_subdirs unqualified – same flat namespace, better layout.

[status: implemented]

core

Spec model: what borge files ARE. lib/core/

[status: implemented]

spec

Borge-specific types and operations on top of the sexp AST

[status: implemented]

Types: status (planned, in-progress, partial, implemented, drifted, blank).

Functions: status_of_string, string_of_status, project_name, count_sections, statuses, status_counts, inline_targets, has_no_inline.

file-utils

File discovery and reading shared by all commands

[status: implemented]

find_borg_files: recursively discover .borg files, skipping __build and .git.

read_file: read a file into a string.

project

Inline tree building and orphan detection

[status: implemented]

build_tree: recursively follow inline directives from a root .borg file, producing a project_node tree. Each node carries its path, project_name, children, and orphan status.

find_roots: find .borg files not inlined by any other file. find_orphans: find files not in any tree, no (no-inline), and don't inline any children themselves.

tree_status_counts: count statuses across the tree without double-counting inlined sub-specs.

format_tree: indented text representation of the tree.

check

Validation and analysis. lib/check/

[status: implemented]

check-command

Recursive health check across all .borg files

[status: implemented]

Walks a directory, finds .borg files, parses each, reports project name and validity. Returns a typed result per file with pass/fail status and warnings (orphan detection).

report

Status summary across all .borg files

[status: implemented]

Aggregates status counts per file and globally. Returns typed data: per-file (name, impl, in-progress, planned) and totals. Uses inline tree to avoid double-counting sub-specs. Reports orphaned files.

lint

Semantic validation of .borg files

[status: implemented]

Checks: invalid status values, duplicate project names, unknown comment types, missing comment values, deleted human comments (against git HEAD), pending ask/response slots, orphaned files. Since nested comments are now captured in the AST (see sexp.borg comment-attachment), lint inspects them as well through a single shared `Borge_lang.Ast.iter_comments` traversal – no duplicated comment-walking boilerplate, and no blind spot inside (section ...) or (project ...) forms.

code-quality

Mechanical code quality checks in .ml/.mli files

[status: implemented]

Scans OCaml source for mechanical anti-patterns via regex and targeted AST inspection. Integrates with borge lint –mechanical. Supports (* exempt: FUNC *) comments for intentional overrides.

Rules are defined in `lib/check/code_quality.ml`. Each rule specifies: the call pattern, severity (Error/Warning), and the preferred safe alternative. Errors crash the program on certain inputs; Warnings use exceptions for control flow, which is harder to reason about and easier to miss in review.

Implemented rules:

rule "List.hd" severity: Error pattern: `List.hd <expr>` why: Throws Failure on empty list. Pattern matching is the only correct fix. fix: `match expr with | [] -> ... | x :: _ -> ...`

rule "List.tl" severity: Error pattern: `List.tl <expr>` why: Throws Failure on empty list. fix: `match expr with | [] -> ... | _ :: xs -> ...`

rule "List.assoc" severity: Warning pattern: `List.assoc key assoc_list` why: Throws `Not_found` when key is absent. Almost all callers wrap it in `try/with` – the `_opt` variant does this correctly. fix: `List.assoc_opt key assoc_list`

rule "Hashtbl.find" severity: Warning pattern: `Hashtbl.find tbl key` why: Throws `Not_found` when key is absent. The `_opt` variant produces `None`, eliminating exception-based control flow. fix: `Hashtbl.find_opt tbl key`

rule "String.sub" severity: Warning pattern: `String.sub s start len` why: Throws `Invalid_argument` when bounds exceed string length. Many callers are guarded by `String.length`, but the scanner cannot verify this in all cases. fix: Guard with `String.length`; or use a bounds-safe helper; or exempt with (* exempt: String.sub *)

rule "Str.search_forward" severity: Warning pattern: `Str.search_forward regexp s pos` why: Throws `Not_found` when no match. Wrapping every call in `try/with` is noisy; consider a safe wrapper. fix: Wrap in `try/with Not_found` and handle `None`; or exempt if clearly guarded

rule "int_of_string" severity: Warning pattern: `int_of_string s` why: Throws Failure on non-numeric input. fix: `int_of_string_opt s`

rule "float_of_string" severity: Warning pattern: `float_of_string s` why: Throws Failure on non-numeric input. fix: `float_of_string_opt s`

rule "open_in" severity: Warning pattern: `open_in path` why: Requires manual `close_in`. Easy to leak channels on exceptions or early returns. fix: `In_channel.with_open_text path (fun ic -> ...)`

rule "input_line" severity: Warning pattern: input_line ic why: Throws End_of_file at EOF. In_channel.input_line returns None, which composes with option-handling. fix: In_channel.input_line ic

Planned rules (AST-based, not yet implemented):

rule "polymorphic equality" severity: planned Error pattern: expr_1 = expr_2 where either operand is string, list, or other structural type why: (= $\Rightarrow$) is polymorphic and performs physical equality on functions, incorrect comparison on floats, and structural recursion on lists. Use structural equality functions (String.equal, List.equal, etc.) fix: String.equal, Int.equal, List.equal, etc.

rule "deep match nesting" severity: planned Warning pattern: match nesting > 3 levels inside a single function (state-machine exclusions apply) why: Deeply nested match is hard to read and often indicates a missing helper extraction. State machines (e.g., balance checker) are legitimate. fix: Extract sub-functions; use result/bind patterns for early return

drift

Detect spec drift, code drift, and structural drift

[status: implemented]

Three drift layers: 1. Spec drift: .borg describes something not in code 2. Code drift: dune-aware – modules from dune files not in .borg 3. Structural drift: code organization does not match inline architecture

Generates .borg.meta files with findings, module surfaces, and dune snapshots. Supports –agent flag for LLM-driven semantic analysis.

dune-parse

Parse dune files for structural drift detection

[status: implemented]

Parses dune (sexp) files to extract library, executable, and test stanzas. Returns typed records with name, modules, public_name, and libraries. Used by drift for cross-referencing dune structure against .borg spec sections.

surface

Extract public module interface from .ml/.mli files

[status: implemented]

For modules with .mli: .mli is the authoritative surface. For modules without .mli: infer from top-level let bindings, type definitions, exceptions, and module declarations. Returns module_name, file path, and export list.

meta

Types and writer for .borg.meta companion files

[status: implemented]

.borg.meta files hold machine-generated observations: analyzed-at timestamp, content hashes, module surfaces, dune snapshots, and findings. The writer is idempotent and deterministic. Findings are sorted for clean git diffs. Agent findings merge (replace) on re-run; static findings are preserved.

format

Formatting and display. lib/format/

[status: implemented]

fnt

Auto-format a .borg file to canonical indentation

[status: implemented]

Parses and re-prints with canonical indentation. Supports `-check` (exit 1 if formatting would change) and `-diff` (show line-by-line diff).

nodes

Dump the parsed AST with types and positions for debugging

[status: implemented]

Walks the AST and prints each node with its type, keyword (if list), source position, and child count. Useful for debugging mismatches between source and AST.

generate

LLM-driven generation. `lib/generate/`

[status: implemented]

generate-command

Generate spec sections and code from specs via LLM

[status: implemented]

Two subcommands: `generate spec (todo -> .borg section)` and `generate code (spec -> implementation)`. Both use `pi -p` via `Agent.run_pi_print`. `Generate spec` takes free-form text and produces a structured (status planned) section. `Generate code` reads a planned section's spec text and module surfaces, then invokes `pi` to write the implementation.

agent

LLM-driven semantic drift analysis

[status: implemented]

Assembles context (spec + code + existing findings) into a prompt, invokes an LLM via `BORGE_AGENT_CMD` env var, parses the sexp response into typed findings. Parse failures are discarded – never corrupt `.borg.meta`. Agent findings merge with static findings (static preserved, agent replaced).

review-quality

Per-file LLM code quality assessment against spec

[status: implemented]

`review_quality.ml`: for each implemented section, find the corresponding source file, build a prompt comparing spec vs code, call `pi -p`, and return structured findings. Exposed via `borg review -agent`.

metrics

Code statistics. `lib/metrics/`

[status: implemented]

stats

Code intelligence metrics: lines, functions, exports, verified, literacy

[status: implemented]

Scans all `.ml` files and reports: line count, function count, average function length, export count, `verified_count`, doc coverage, mli status. Aggregates into project totals. Filters `_build`, `.git`, test dirs.

`verified_count`: bindings whose doc comment contains "verified" or "verif" – agent-verified claims. Counted per-file and aggregated project-wide as `total_verified`.

With `-literacy`, also reports per-file literacy averages: story, explain, teach, edge – the four dimensions of the comment-writing-guide scoring system. Shows

which files are narrative-strong vs mechanics-strong vs pedagogy-strong. Helps identify where the codebase tells its story well and where comments are just checkboxes.

output

Structured output formats. lib/output/
[status: implemented]

json

JSON serialization for `-json` flag across borge commands
[status: implemented]

Yojson-based serializers for Report, Check, Lint, Drift, Stats, and Balance result types. Each command's `-json` flag calls the corresponding `Json_out.to_json` function and prints the result.

json-out

Individual JSON output serializers
[status: implemented]

`Json_out` contains `to_json` functions for each command result.

ui

UI DSL types and parsing. lib/ui/
[status: implemented]

ui-ast

Typed AST for borge UI specs
[status: implemented]

`ui_ast.ml` defines typed representations: `ui_element`, `component`, `layout_def`, `page`, `route`, `theme`, `variant`, and all value types (sizing, `color_value`, `spacing_value`, etc.). The UI types are consumers of the generic `sexp` AST – they do NOT modify the parser.

ui-parse

`Sexp` to typed UI AST extraction
[status: implemented]

`ui_parse.ml` walks the generic `Ast.sexp` tree and extracts typed UI nodes. Functions like `parse_ui_element`, `parse_theme`, `parse_component`, `parse_layout_def`, `parse_page`, `parse_routes`, `parse_ui_app`. The parser stays generic; this module interprets the semantic structure of UI-specific forms.

ui-validate

Semantic validation for UI specs
[status: implemented]

Checks: variant names are unique per element, theme variable references (palette, spacing, font-size, radius) are defined, component and layout names are unique, page layout references point to defined layouts. Returns structured issues with severity (error/warning).

ui-json

JSON serialization for UI AST
[status: implemented]

`ui_json.ml` serializes the typed UI AST to yojson for `-json` output. Covers all types: sizing, color, spacing, elements, components, pages, routes, themes.

ui-dream

Dream HTML code generation from UI AST

[status: implemented]

ui_dream.ml generates Dream HTML output from the typed UI AST.

ui-clay

Clay CSS code generation from UI AST

[status: implemented]

ui_clay.ml generates CSS output from the typed UI AST.

lock

Transactional isolation for agent sessions. lib/lock/

[status: implemented]

lock-core

Lock directory creation, state management, commit/abort

[status: implemented]

Lock.create: snapshot all .borg files to .borge.lock/original/ Lock.set_state/get_state: running | completed | failed Lock.check_integrity: detect deleted human-authored comments Lock.restore: abort – restore .borg files from snapshot Lock.teardown: commit or abort with cleanup Lock.remove: delete lock directory after commit/abort

lock-prompt

Prompt builder for agent sessions

[status: implemented]

Role definitions: Implementer, Spec_writer, Observer, Fixer. Assembles prompts from spec files, module surfaces, and role declarations. Does not depend on Borge_lib internals to avoid cyclic deps.

lock-pi-launch

Launch pi interactively with structured prompt

[status: implemented]

Simple wrapper: pi @prompt_path. Direct invocation – pi is interactive so backgrounding loses status.

prompt-minimal

Minimal prompt builder for quick agent sessions

[status: implemented]

Prompt_minimal builds compact prompts without full context assembly.

db

DB DSL types and parsing. lib/db/

[status: implemented]

db-ast

Typed AST for borge DB specs

[status: implemented]

db_ast.ml defines typed representations: table_def, column, operations_def, relation_def, group_def, capability, and all value types (column_constraint, crud_spec, join_type, etc.). The DB types are consumers of the generic sexp AST.

db-parse

Sexp to typed DB AST extraction

[status: implemented]

db_parse.ml walks the generic Ast.sexp tree and extracts typed DB nodes.

Functions like parse_table, parse_column, parse_operations, parse_relation, parse_group, parse_db_app.

db-validate

Semantic validation for DB specs

[status: implemented]

Checks: table and column names are unique, (references table.column) refers to an existing table, (ownership column) refers to an existing column on the same table, operations reference existing tables, relation from-tables and join tables exist, group capability tables exist. Returns structured issues with severity (error/warning).

db-json

JSON serialization for DB AST

[status: implemented]

db_json.ml serializes the typed DB AST to yojson for -json output. Covers all types: tables, columns, operations, relations, groups, capabilities.

db-sql

SQL DDL generator from DB AST

[status: implemented]

db_sql.ml generates PostgreSQL DDL from the DB AST.

bug

Issue tracking for repo-stored bugs

[status: implemented]

bug-ast

Bug AST types and operations

[status: implemented]

Bug_ast defines bug type, status workflow, and helper functions.

bug-parse

Parse .borge-bug files

[status: implemented]

Bug_parse reads .borge-bug files into typed bug records.

bug-registry

Manage .borge-bug directory

[status: implemented]

Bug_registry handles file paths, listing, loading, saving bugs.

bug-write

Write bugs to .borge-bug files

[status: implemented]

Bug_write renders bug records to .borge-bug file format.

distributed

Distributed specs: module-level .borg files

[status: implemented]

borg-comment

Parse borg-comments from OCaml source

[status: implemented]

Extract `(* (fn name ...) *)` comments from .ml files.

module-spec

Discover and parse module .borg files

[status: implemented]

Find all .borg files, extract exports, check integrity.

review

Semantic code review with LLM

[status: implemented]

semantic-review

Verify function docs match implementation

[status: implemented]

semantic_review.ml implements per-function LLM verification.

Extraction: Uses compiler-libs/ocaml-common to parse .ml/.mli files into Parsetree. Extracts: - Value bindings (Pstr_value) - Docstrings (attribute "ocaml.doc") - Type signatures (from .mli when available)

Batching: Files with >20 functions are batched into groups of 15-20, alphabetically sorted for determinism.

LLM Prompt: See meta.borg (section semantic-review-integration) for the exact prompt template. Returns structured sexp that parses into semantic_finding record.

Error handling: Parse failures are logged to stderr with raw response preserved for debugging. Previous .borg.meta content is preserved – never corrupt metadata.

Data types: - doc_present: bool – is there a docstring/comment? - doc_accuracy: high | medium | low – does doc match code? - signature_match: accurate | mismatch | unknown - behavior_coverage: complete | partial | missing - structural_issues: string option (none, or description) - confidence: high | medium | low

Output: Writes (semantic-review ...) block to .borg.meta file sibling to the corresponding .borg file. See meta.borg for block format specification.

Commands exposed: - borge review FILE – review single file - borge review -dir DIR – review all .ml in directory - borge review -all – review entire project - borge review -stale – only review files with stale metadata

review-prompt

Prompt assembly for semantic review LLM queries

[status: implemented]

review_prompt.ml: builds prompts for semantic review.

Functions: - build_review_prompt: spec_section -> module_surface -> string - format_function_for_review: function_info -> string

The prompt includes: 1. The spec section describing this module (doc body) 2. List of functions with signatures and docstrings 3. Structured output constraint (sexp format only)

Token management: If prompt would exceed ~8k tokens, truncate function signatures (keep names only) and move full signatures to a follow-up batch.

review-parse

Parse LLM responses into typed semantic findings

[status: implemented]

review_parse.ml: parses sexp responses from LLM.

Handles: - Well-formed (findings ...) blocks ? semantic_finding list - Mal-formed responses ? Parse_error logged, empty list returned - Partially valid ? extracts valid findings, logs invalid remainder

Never raises on parse failure – always returns result type.

review-types

Types for semantic review findings

[status: implemented]

Review_types defines semantic_finding and related data types.

review-agent

LLM agent for semantic code review

[status: implemented]

Review_agent orchestrates LLM calls for semantic review.

extract-parsetree

OCaml parsetree extraction for semantic review

[status: implemented]

Extract_parsetree uses compiler-libs to parse .ml files into parsetree.

extract-functions

Function extraction from parsetree

[status: implemented]

Extract_functions extracts function bindings from OCaml parsetree.

extract-docs

Docstring extraction from parsetree

[status: implemented]

Extract_docs extracts docstrings from parsed OCaml source.

doc

Documentation coverage detection and enforcement for .ml files

[status: in-progress]

doc-extract

Extract top-level let bindings from OCaml source files

[status: implemented]

Gathers all top-level value bindings from .ml files, distinguishing between public exports and internal helpers. Filters out test bindings, ignored bindings, and local opens.

Types: - binding_info: { name; line; is_exported; is_test; is_internal }

Functions: - extract_bindings : string -> binding_info list Parse a .ml file and return all top-level let bindings with name, line number, and classification. - extract_binding_name : string -> string option Parse a let-binding

line to extract the binding name. - `find_ml_files` : string -> string list Recursively find .ml files under a directory, skipping `__build/` and `.git/`. - `extract_project_bindings` : string -> (string * binding_info) list All bindings from all .ml files, paired with file path. - `count_stats` : binding_info list -> int * int * int * int Returns (total, exported, tests, internal).

Classification rules: - `is_test`: line starts with `let%` - `is_internal`: name starts with `__` or is a test binding - `is_exported`: not internal and name starts with lowercase a-z

Skips `let open` and `let module` forms.

doc-detect

Detect documentation comment formats preceding let bindings

[status: implemented]

Identifies various documentation formats and exempt markers on lines preceding a let binding: - Standard docstrings: `(** ... *)` - Borg annotated: `(* author note (|...|) *)` - Short borg: `(*| ... |*)` - Exempt: `(* exempt doc *)`

Types: - `doc_kind`: Docstring of string | Borg_note of string*string | Borg_short of string | Exempt_marker - `doc_position`: Preceding of int | In-line | None_found - `binding_doc`: { binding_line : int; doc : (doc_kind * doc_position) option }

Functions: - `parse_doc_line` : string -> doc_kind option Classify a single line as one of the four doc kinds or return None. - `extract_file_docs` : string -> binding_doc list Scan a .ml file for all let bindings and their preceding doc comments. Returns one binding_doc per binding with the closest preceding comment. - `binding_has_doc` : binding_doc -> bool True if doc exists and is not Exempt_marker. - `binding_is_exempt` : binding_doc -> bool True if doc is Exempt_marker.

Detection logic for `extract_file_docs`: - Walk lines left to right, accumulating a `prev_docs` stack. - When a let-binding line is found, search `prev_docs` backward for the nearest `parse_doc_line` hit. - Stop searching at the first non-comment, non-empty line. - This handles multi-line comments and blank lines between comment and binding.

doc-coverage

Compute documentation coverage statistics and write .borg.meta blocks

[status: implemented]

Computes per-file and project-wide doc coverage, formats human-readable reports, and generates (documentation ...) blocks for .borg.meta files.

Types: - `file_coverage`: { path; total_bindings; documented; exempt; undocumented; coverage_percent; undocumented_names } - `project_coverage`: { files; total_files; total_bindings; total_documented; total_exempt; total_undocumented; overall_percent }

Functions: - `calculate_file_coverage` : string -> file_coverage Combine `Doc_extract.extract_bindings` with `Doc_detect.extract_file_docs`. Skip tests and internals. `coverage_percent` = `documented` / (`total` - `exempt`) * 100. - `calculate_dir_coverage` : string -> project_coverage All .ml files under a directory. - `format_coverage` : file_coverage -> string "path: N/M documented (X.X%), E exempt, U undocumented" - `format_project_summary` : project_coverage -> string Multi-line project summary. - `generate_meta_block` : file_coverage -> string Sexp block for .borg.meta: (documentation (analyzed-at ...) (file ...) ...) - `write_coverage_meta` : file_coverage -> string -> unit Write meta block to a .borg.meta path. - `meets_threshold` : file_coverage -> float -> bool Check if `coverage_percent` >= `threshold`.

Coverage calculation: `documented` = bindings with valid doc (not exempt) `exempt` = bindings with `(* exempt doc *)` `total_bindings` = all extracted bindings (after filtering tests/internals) `undocumented` = `total` - `documented` - `exempt` `coverage_percent` = `documented` / (`total` - `exempt`) * 100

Integration: called by borge drift -docs (see borge.borg code-documentation.doc-coverage-command).

config

Configuration loading from .borgerc files. lib/config/
[status: planned]

config-core

Read, merge, and expose borge configuration

[status: planned]

config.ml reads .borgerc files and exposes a typed config record to all borge commands.

Types: - api_key_source: Env of string | Literal of string - config: { developer_name : string option; api_keys : (string * api_key_source) list; }

Functions: - load_config : unit -> config Read project .borgerc, merge with ~/.config/borge/config, return merged config. Project values override user values. - load_project_config : unit -> config option Read ~/.borgerc only. None if file missing. - load_user_config : unit -> config option Read ~/.config/borge/config only. None if file missing. - resolve_api_key : config -> string -> string option Look up a provider key by name. Resolve (env X) by reading the env var. Return None if not found. - developer_name : config -> string Return the configured developer name, or "agent" if not set.

The developer_name function is used by lock-prompt, lint, and comment-generating code to set the authorship on annotated comments. Today these default to "agent"; with config, they use the developer's chosen name.

resolve_api_key is used by agent.ml and review_agent.ml when calling LLMs directly (not via pi). Today they only support BORGE_AGENT_CMD env var; config adds per-provider key resolution with env var indirection.

convention

Convention dispatch: select the right module set based on project convention
[status: implemented]

convention-core

Convention type, resolution from .borg files, fallback to ocaml-dune

[status: implemented]

Convention.ml provides convention-aware dispatch for borge commands. Reads the (convention NAME) form from the root .borg file and returns a typed Convention.t variant.

Types: - Convention.t: Ocaml_dune | Go_standard

Functions: - Convention.name : t -> string - Convention.source_extension : t -> string - Convention.interface_extension : t -> string option - Convention.excluded_dirs : t -> string list - Convention.of_string : string -> t option - Convention.resolve_from_file : string -> t - Convention.resolve : string -> t

merge

Merge queue and branch management. lib/merge/
[status: implemented]

worktree

Git worktree management

[status: implemented]

Worktree creates and manages git worktrees for agent isolation.

branch

Git branch management for agent runs

[status: implemented]

Branch creates and switches git branches for agent sessions.

lockfile

Agent session lockfile format

[status: implemented]

Lockfile defines the .borg.lock state format.

queue-types

Merge queue type definitions

[status: implemented]

Queue_types defines data types for the merge queue.

queue-storage

Merge queue persistent storage

[status: implemented]

Queue_storage persists merge queue state to disk.

conflict-detect

Detect merge conflicts between agent branches

[status: implemented]

Conflict_detect analyzes branches for potential merge conflicts.

rollback

Rollback failed agent sessions

[status: implemented]

Rollback reverts failed agent sessions and cleans up.

agent-submit

Submit agent work to merge queue

[status: implemented]

Agent_submit packages agent work for merge queue processing.

spec-in-code

Spec sections embedded in source files. lib/spec_in_code/

[status: planned]

spec-in-mli

Extract borge spec sections from OCaml .mli comments

[status: planned]

An .mli file may carry a borge spec section as a top-of-file comment. The comment uses the standard (* ... *) OCaml delimiter and contains a complete (section ...) or (subsection ...) form:

```
(* borge (subsection login (doc "Login form and handlers") (status im-
plemented) (endpoint GET (verify (build) (smoke "GET /login"))) (endpoint
POST (verify (build) (test "test/auth_login.ml")))) ) *)
```

The .mli is the declaration layer for OCaml modules. Putting the spec there means one file answers both "what must exist?" (types, signatures) and "why? is it done? how to verify?" (borg spec).

Types: - `mli_spec`: { `name` : string; `kind` : [Section | Subsection]; `status` : Spec.status option; `verify` : Spec.verify_item list; `children` : `mli_spec` list; `raw` : string; `line` : int }

Functions: - `extract_from_mli` : string -> `mli_spec` list Scan an .mli file for (* borge (section ...) *) and (* borge (subsection ...) *) comments. Returns all top-level spec sections found. Parses the borge sexp inside the comment using the existing `Borge_lang.Parse`. - `spec_to_section_mapping` : `mli_spec` -> Spec.section_mapping Convert to the existing section_mapping type so that drift, check, and report work unchanged. - `is_spec_comment` : string -> bool Returns true if a comment line starts a borge spec block.

Integration with `build_tree`: `project.ml:build_tree` is extended so that (inline target.mli) resolves targets ending in .mli by calling `extract_from_mli` instead of `parse_file`. The extracted `mli_spec` becomes a child node in the project tree. The project tree representation is unchanged – only the source format differs.

Index relationship: The root .borg file uses (inline auth/login.mli) to declare that login.mli carries the spec for the login subsection. The .mli file is the authority for the spec detail. The association to the implementation is implicit by convention: login.mli describes the module implemented in login.ml. No (implements ...) declaration is needed – surface extraction already pairs .mli with .ml.

Drift detection: Works identically. Borge reads the spec from login.mli's comment, reads the code surface from login.ml, and checks coherence. Drift does not need to change – the spec model (section_mapping) is the same regardless of whether the spec source was a .borg file or a .mli comment.

comment-parser

Extend `borg_comment.ml` to handle section/subsection forms

[status: planned]

Currently `borg_comment.ml` extracts leaf forms: (fn ...), (val ...), (type ...). For spec-in-mli, it must also extract nested forms: (section ...), (subsection ...) with arbitrary children including (verify ...), (endpoint ...), etc.

The existing `is_borg_form` checks the top-level atom. Extend to also match "section" and "subsection". The `parse_comment` function already calls `Borge_lang.Parse.parse` – the parser handles arbitrary nesting. The only change is what form types trigger extraction, and preserving the entire nested subtree rather than flattening to a (form_type, name) pair.

Functions: - `is_section_form` : string -> bool Recognizes (section ...) and (subsection ...) comment content, same pattern as existing `is_borg_form`. - `extract_section` : `Borge_lang.Ast.file` -> `mli_spec` list Walk parsed top-level forms, extract section/subsection nodes into the `mli_spec` tree type.

inline-resolution

Extend `build_tree` to resolve (inline file.mli) targets

[status: planned]

`project.ml:build_tree` currently only handles .borg targets (calls `parse_file`).

The extension:

1. Check file extension of inline target.
2. If target ends in .borg: existing path (`parse_file`).
3. If target ends in .mli: call `extract_from_mli`, convert each `mli_spec` to a `project_node`.
4. If target ends in anything else: error (unsupported).

The `project_node` type does not change. The source field is already a path string. The children come from the `mli_spec` tree (nested subsections become nested `project_nodes`).

Cycle detection: works identically – the visited list tracks paths regardless of extension.

`find_root_candidates`: must also consider `.mli` files that contain (no-inline) declarations in their spec comments. An `.mli` file whose spec comment does not declare (no-inline) and is not inlined by any parent is orphaned.

book

Codebase book printer. `lib/book/`
[status: planned]

book-print-impl

Linearize source files to a paginated PDF (four-tier order: root `.borg`, other top-level `.borg`, `.mli`, `.ml`)
[status: implemented]

– *agent note*: verified at this commit: `build-pass`, `smoke-pass` (borge book print . opens with `borge.borg` as the introduction, then the other 8 top-level `.borg` design docs, then `.ml` implementations; line numbers on every page; `tree_checksum` deterministic across runs). Uses `lstinputlisting` with sanitized temp files to dodge the UTF-8 and `\end{lstlisting}`-in-source problems.

`lib/book/print.ml` – the v1 linearizer and LaTeX emitter.

v1 scope: four-tier data-driven ordering (root `.borg` -> other top-level `.borg` -> `.mli` -> `.ml`), no full spec-order, no `–section`, no `–woven`. The root `.borg` (via `Project.find_roots`) opens the book as the introduction; other top-level `.borg` files follow as design docs; then `.mli` interfaces; then `.ml` implementations. Each file is read, sanitized, emitted as one LaTeX document, run through `pdflatex`.

Pipeline: 1. `collect_files` : `dir` -> string list `Roots` via `Project.find_roots`, normalized to paths relative to `dir`. Then a deterministic walk of all `.borg` (excl. `.borg.meta`), `.mli`, `.ml` files. Dedup roots from the walk. Sort by: (a) `tier_of`: tier 0 = root `.borg`, tier 1 = other top-level `.borg`, tier 2 = `.mli`, tier 3 = `.ml`. (b) within the code tier (2/3): a dependency rank computed from ‘open X’ and ‘Module.ref’ per file – module basenames map to directories; a longest-path rank per dir (`lang=0`, `core=1`, ...). Stdlib/external names are excluded so a project module that shadows a stdlib name (`lib/format`) doesn’t create a false dep. (c) then `dir_group` (`lib/` then `bin/` then `test/`). (d) then lexical. Foundations precede dependents. Skip `_build/`, `.git/`, and `.gitignore` build artifacts. 2. `file_to_latex` : `path` -> lines -> string For `.ml/.mli`: `\section{path} + \lstinputlisting` block (`numbers=left`, `frame=single`, `mono`, `breaklines`, wide right margin for marginnote annotations). For `.borg`: parse with `Borge_lang.Parse` and emit prose (`section/subsection` -> headings, `doc` -> paragraphs, annotated comments -> attributed blockquotes, `status` -> italic marker, `inline` -> small note; other forms skipped). Falls back to a raw listing on parse failure. 3. `render` : `dir` -> stem -> unit Assemble preamble + body + `\end{document}`, shell out to `pdflatex` (loop passes until `.aux/.toc` stabilize), write the manifest sidecar via `book-manifest-impl`.

Backend selection at runtime: `pdflatex` on `PATH` -> LaTeX; else `groff` on `PATH` -> `groff/ms`; else error. Not config.

Shell-out uses `Unix.open_process` and checks the exit code; a failed `pdflatex` is a hard error, not a silent fallback.

book-manifest-impl

Manifest sidecar: page N -> file, plus a tree checksum
[status: implemented]

– *agent note*: verified at this commit: `build-pass`. `start_page` per file parsed from the `.aux` (`\newlabel{file:...}`), exact – no estimation. Line numbers are NOT stored (printed on the page by listings).

lib/book/manifest.ml – the round-trip anchor. JSON (yojson).

Fields per book-manifest in borge.borg: stem, rendered_at, backend, tree_checksum, pages. tree_checksum is a sha1 over the ordered (path, content-sha1) list – ordered so a file reorder is detectable, not just content changes.

Line numbers are printed on the page by listings (numbers=left) and read off the page by absorb – the manifest never stores line ranges. pages entries carry only n (page number) and file (source path on that page), so a note in the gutter between two files is unambiguous. This avoids parsing PDF page breaks entirely: the line precision comes free from the rendered page, not from a sidecar.

Functions: - build : dir:string -> files:string list -> manifest - to_json : manifest -> string - tree_checksum : files:string list -> string

book-absorb-impl

Scan -> inbox items. NOT implemented in v1.

[status: planned]

lib/book/absorb.ml – stub in v1. The full pipeline (vision LLM per page, manifest lookup, inbox emission) is spec'd in borge.borg book-absorb but is out of scope for the first print pass. This subsection exists so the module has a home when it lands.

future

Future work analysis. lib/future/

[status: implemented]

depends

Dependency graph analysis for .borg files

[status: implemented]

Depends analyzes (depends-on ...) declarations across specs.

8 meta.borg

borge-meta

Companion metadata files for .borg specs – machine-generated observations about code reality

[status: in-progress]

purpose

.borg files are the source of truth for intent – what the project should be.
.borg.meta files are the source of truth for reality – what the project actually is.

A .borg.meta file sits next to its .borg sibling. It is sexp-formatted, machine-writable, git-tracked, and never hand-edited. It holds structured observations that borge commands produce: static analysis results, agent findings, module surfaces, timestamps.

The two-file model keeps the spec clean. Humans and agents author .borg files. Machines write .borg.meta files. You can look at either file to understand one concern, or look at both to see the gap.

file-layout

Sibling pairing – locality through proximity

[status: implemented]

Every .borg file may have a .borg.meta sibling:

borge.borg ? source of truth (intent) borge.borg.meta ? machine-generated (observations) cli.borg cli.borg.meta sexp.borg sexp.borg.meta

A .borg.meta file is created on first analysis and updated on subsequent runs. If no analysis has been run, the file doesn't exist – no empty stubs.

.borg.meta files are git-tracked. This means: - You can diff findings between commits - You can see when the agent's assessment changed - CI can check if findings are stale (timestamp too old)

.borg.meta files are listed in .gitignore only if the project opts out. By default they are tracked.

format

Sexp structure of a .borg.meta file

[status: implemented]

A .borg.meta file is a valid sexp file with its own top-level forms. It is NOT a (project ...) form – it's a flat sequence of observation blocks:

```
(meta borge (analyzed-at 2026-05-15T14:32:00Z) (module-surface (Spec (file
lib/borge/spec.ml) (exports status_of_string string_of_status project_name
find_count_sections statuses extract inline_targets has_no_inline status_counts))
(File_utils (file lib/borge/file_utils.ml) (exports read_file find_borg_files)))
(findings (finding unspecified-module (module Lexer) (file lib/sexp/lexer.ml)
(detail "no .borg section describes this module") (confidence high) (source
static) (at 2026-05-15T14:32:00Z)) (finding partial-implementation (section drift)
(detail "spec says drift checks undocumented bindings but code removed that
check") (confidence medium) (source agent) (at 2026-05-15T14:32:00Z))) (dune-
snapshot (library borge_sexp (modules ast balance error lexer parse print)
(public_name borge.sexp)) (library borge_lib (modules spec file_utils check
report fmt nodes project lint drift) (public_name borge.lib) (libraries borge.sexp
unix))))
```

The (meta ...) form's first argument is the project name from the sibling .borg file. This is for human readability – the pairing is by filename convention, not by name matching.

Block types: - (analyzed-at ...): ISO 8601 timestamp of last analysis - (module-surface ...): per-module export lists - (findings ...): structured observations with confidence - (dune-snapshot ...): parsed dune file structure

Additional block types may be added in the future. The parser should ignore unknown blocks for forward compatibility.

module-surface

Extracted public interface of each source module

[status: implemented]

The module surface is the set of names a module exports – its public interface, as seen from outside.

For OCaml with .mli files: the .mli IS the surface. Parse it.

For OCaml without .mli files (like borge itself): every top-level let binding, type definition, and exception is exported. Extract these from the .ml source. This is a heuristic – the real surface is what the compiler infers – but it's close enough for drift detection.

For other conventions in the future: each convention defines how to extract the module surface. Go uses exported names (capitalized). C uses header files. The convention declaration in the root .borg file determines the extraction method.

The module surface enables precise drift: - "spec says this module exports format_file, but it's not in the module surface" ? spec drift - "module exports check_structural_drift but no spec mentions it" ? code drift - "spec lists 5 functions, module exports 12" ? partial coverage

dune-snapshot

Parsed dune file structure for structural drift detection

[status: implemented]

For the ocaml-dune convention, dune files declare what exists: - (library ...) with (modules ...) and (libraries ...) - (executable ...) with name and dependencies

The dune snapshot captures this structure so borge can compare the build system's view of the project against the spec's view.

Drift examples: - dune says borge_lib has 9 modules, lib.borg has 7 sections - dune says borge_cmd depends on borge_lib, but cli.borg doesn't describe that dependency - a module appears in dune (modules ...) but has no .borg section

The snapshot is re-extracted on each analysis run. It's not authoritative – the dune file is – but having it in .borg.meta means you can diff between commits to see when structure changed.

findings

Structured observations – from static analysis and agent runs

[status: implemented]

A finding is a single structured observation: something that appears to be drift between spec and code.

Finding types (extensible – new types can be added):

unspecified-module: module exists in code, not in spec fields: module, file

phantom-spec: spec section describes something not in code fields: section

partial-implementation: spec says implemented, code is incomplete fields: section, detail source: agent only (can't detect statically without .mli diff)

status-mismatch: spec status contradicts code reality fields: section, spec-status, actual-status actual-status is inferred: "stub", "partial", "complete"

missing-export: spec describes an export that doesn't exist fields: section, export, module

extra-export: module exports something the spec doesn't mention fields: module, export

structural-drift: code organization violates convention fields: detail

Each finding has: - confidence: high | medium | low high = machine-verified (file doesn't exist, module not in dune) medium = heuristic (name matching, surface area comparison) low = opinion (agent assessment) - source: static | agent static = produced by borg drift without LLM agent = produced by borg drift -agent - at: ISO 8601 timestamp of when the finding was produced - detail: free-text explanation (required for agent, optional for static)

Findings are NOT ground truth. They are observations that may be wrong. A human or agent should review before acting on them.

drift-command-redesign

How borg drift changes with .borg.meta

[status: implemented]

Current drift: heuristic name-matching against filenames. Produces ad-hoc output to stdout. No persistence.

New drift writes .borg.meta files and produces structured output:

borg drift 1. Parse dune files ? dune-snapshot 2. Extract module surfaces ? module-surface 3. Cross-reference with .borg sections 4. Produce static findings 5. Write .borg.meta (one per .borg file that has findings) 6. Print human-readable summary to stdout

borg drift -agent (or borg drift agent) 1. Run all static drift steps first 2. For each .borg.meta with findings, assemble context: - the .borg section text - the corresponding module source - the existing findings 3. Prompt the LLM per section: "Here is a spec section and its code. Here are existing findings. Review for: partial implementations, silent scope changes, missing edge cases, status mismatches. Return findings in this format: ..." 4. Merge agent findings into .borg.meta - Keep existing static findings (they're machine-verified) - Replace agent findings from previous run (they're opinions) - Update analyzed-at timestamp 5. Print summary with source annotations

The -agent pass is additive. It doesn't redo static analysis. It layers semantic observations on top of structural ones.

agent-prompt-design

Structured output from LLM analysis

[status: implemented]

The agent prompt must constrain the LLM to return structured findings, not prose. This is critical: unstructured LLM output can't be diffed, tracked, or acted on programmatically.

The prompt template (rough sketch):

You are analyzing a codebase against its specification. For each section below, compare the spec description with the source code and return findings.

Spec section: {section_name} Status: {spec_status} Description: {spec_doc}

Source file: {module_path} Exports: {module_surface}

Return ONLY a sexp list of findings. Each finding is: (finding TYPE (section SECTION_NAME) (detail EXPLANATION) (confidence LEVEL))

Valid types: partial-implementation, status-mismatch, missing-export, extra-export, phantom-spec Valid confidence levels: high, medium, low

If no findings, return: (findings)

Example: (findings (finding partial-implementation (section drift) (detail "spec mentions undocumented-bindings check but it was removed") (confidence high)))

The agent output is parsed as sexp. Parse failures mean the agent produced garbage and the finding is discarded with a warning – it does NOT corrupt the .borg.meta file.

Multiple sections can be batched into one LLM call to reduce round-trips, or run one-at-a-time for precision. This should be configurable.

staleness

Detecting when metadata is out of date

[status: implemented]

.borg.meta files become stale when: - The .borg source file changed since last analysis - A source .ml file changed since last analysis - A dune file changed since last analysis

Staleness detection: - Store a content hash of the source files in the .borg.meta - On drift run, compare current hashes against stored hashes - If any hash differs, that block is stale - Print staleness warning even if no other findings

Alternatively: use git. If the .borg file or its referenced source files have commits after analyzed-at, the metadata might be stale. This is cheaper than hashing but less precise (a commit might not touch the relevant sections).

For the agent pass specifically: always re-run if metadata is stale. Agent findings about old code are worse than no findings.

writing

How .borg.meta files are written and preserved

[status: implemented]

Machine-written, never hand-edited. But this needs enforcement:

1. A header comment at the top of every .borg.meta file: (; DO NOT EDIT – generated by borge drift. Changes will be overwritten. ;)

2. borge fmt skips .borg.meta files – they’re not specs. borge check skips them. borge lint skips them.

3. The writer preserves formatting: consistent indentation, alphabetical ordering where order doesn’t matter, one finding per line. This makes git diffs useful.

4. The writer is idempotent: running drift twice with no code changes produces the same .borg.meta file. This means findings must be sorted and deterministic.

5. Agent findings merge, not append. If the agent previously reported (finding partial-implementation (section drift) ...) and now reports the same finding, it’s updated in place with the new timestamp. If the agent no longer reports it, it’s removed. The .borg.meta is a snapshot, not a log.

metrics

Static code intelligence stored in .borg.meta companion files

[status: implemented]

Every .borg.meta file stores machine-extracted code intelligence about its sibling .borg’s referenced source files. These metrics are generated on analysis, not hand-authored, and enable:

- borge stats: print a project health dashboard - borge review: surface files with declining quality - borge generate: inform the LLM about code size and shape

Metric blocks in .borg.meta:

(complexity (file lib/borge/meta.ml) (lines 488) (functions 27) (average-length 18) (max-nesting 4) (sentiment-preservation "high"))

(module-metrics (file lib/borge/nodes.ml) (lines 112) (functions 8) (exports 12) (mli-coverage 0.75) (doc-coverage 0.33))

(project-summary (files 24) (total-lines 4100) (total-functions 312) (plans 13) (implemented 19) (in-progress 2))

Extraction methods are convention-specific. For ocaml-dune: - lines: physical line count from .ml files - functions: top-level let bindings (heuristic for exports) - average-length: total lines / function count - max-nesting: maximum indentation depth - mli-coverage: exports with .mli documentation / total exports - doc-coverage: functions with borge-style doc comments / total

The sentiment-preservation field is LLM-assessed: does the spec's description of the code match the code's reality? High means the spec is accurate. Low means the code has drifted from what the spec claims.

These metrics enable a new command: `borg stats`. It reads all `.borg.meta` files, aggregates, and prints:

Project: borge Files: 24 Lines: 4100 Functions: 312 Coverage: mli 0.75 doc 0.33 Status: planned 13 implemented 19 in-progress 2 Quality drift: 1 file with declining coverage (lint.ml)

stats-command

Print aggregate code intelligence dashboard

[status: implemented]

`borg stats`: read all `.borg.meta` files, aggregate (metrics ...) blocks, and print a formatted summary. Show planned vs implemented counts, total lines, average function length, files with declining quality. This is the first thing a developer or agent reads when entering a project.

review-quality

Per-file code quality assessment with LLM

[status: implemented]

`borg review FILE`: read the spec for the file, read the file source, and ask the LLM to assess: Does the implementation match the spec description? Are exports documented? Are there dead functions? Is error handling present? The review is stored as annotated comments in the spec, or in a per-file `.review.borg`. The sentiment-preservation field in `.borg.meta` is updated.

future-blocks

Extensibility – additional block types

[status: planned]

The `.borg.meta` format is designed to grow. Future block types:

(test-coverage TYPICAL: (module X) (tested-by test_x.ml) (cases 12/15))

– which modules have test coverage, how complete

(dependencies (module X) (depends-on Y Z)) – module dependency graph extracted from source

(complexity (module X) (lines 230) (functions 15) (max-nesting 4)) – static complexity metrics

(agent-session SESSION_ID (started-at ...) (findings ...)) – if we want to track which agent run produced which findings, instead of just flattening into the current snapshot

Additional blocks must not break the parser. Unknown top-level forms inside (meta ...) are preserved on write and ignored on read unless the code explicitly handles them.

semantic-review-integration

How semantic review writes to `.borg.meta`

[status: planned]

Semantic review (`borg review`) produces per-function assessments that are stored in `.borg.meta` as a new block type:

(semantic-review (source-file lib/borge/spec.ml) (reviewed-at 2026-05-17T10:30:00Z) (reviewer agent:semantic-reviewer) (functions (function name:"status_of_string" (doc-present true) (doc-accuracy high) (signature-match accurate) (behavior-coverage complete) (structural-issues none) (confidence high) (checked-at 2026-05-17T10:30:00Z)) (function name:"project_name" (doc-present true) (doc-accuracy

medium) (signature-match accurate) (behavior-coverage partial) (structural-issues "doc claims X but code does Y when Z") (confidence medium) (checked-at 2026-05-17T10:30:00Z))))

Merge strategy: semantic review replaces previous review for the same file. Reviews are scoped per-file, not per-project, to allow incremental updates. The analyzed-at timestamp in the parent (meta ...) block is NOT updated by semantic review – semantic review is an additive layer, not a full re-analysis.

Staleness: a semantic review for file X is stale if: - The file's content hash changed since reviewed-at, OR - The corresponding .borg section describing this module changed

review-command

borge review implementation details

[status: planned]

Implementation strategy:

1. Extraction: Use OCaml's compiler-libs to parse .ml files into Parsetree. Extract: - Value bindings (let declarations) - Type signatures (val_decl in .mli files) - Docstrings attached via (** ... *) comments

2. Batching: Group functions by file. For files with >20 functions, batch into chunks of 15-20 to respect token limits. Sort functions alphabetically for determinism.

3. Prompt template (per batch):

You are reviewing OCaml code for alignment between documentation and implementation.

For each function below, assess: 1. Is there a docstring/doc comment present? 2. Does the documentation accurately describe the behavior? 3. Does the implementation match the claimed signature? 4. Are there edge cases or error paths not documented?

Return ONLY a sexp list in this exact format: (findings (function name:"FUNCTION_NAME" (doc-present BOOLEAN) (doc-accuracy high|medium|low) (signature-match accurate|mismatch|unknown) (behavior-coverage complete|partial|missing) (structural-issues "optional description or 'none'") (confidence high|medium|low)) ...)

Functions to review: {function_list_with_signatures_and_docstrings}

4. Parse handling: Parse failures mean the LLM produced malformed output. Log the failure with the raw response for debugging, but do NOT corrupt the .borg.meta. The previous review (if any) is preserved.

5. Merge: Replace any existing (semantic-review ...) block for this file with the new findings.

Commands: - borge review FILE: review single file - borge review -dir DIR: review all .ml files in directory - borge review -all: review all .ml files in project - borge review -stale: only review files with stale reviews

code-documentation-integration

How doc coverage tracking integrates with .borg.meta

[status: planned]

Code documentation tracking counts documented vs undocumented bindings in .ml files. Results stored in (documentation ...) block:

(documentation (analyzed-at 2026-05-17T11:00:00Z) (file lib/borge/spec.ml) (total-bindings 45) (documented 38) (exempt 2) (undocumented 5) (coverage-percent 84.4) (undocumented-names (name internal_helper_1) (name internal_helper_2) ...))

Binding detection: - Top-level let bindings: let f x = ... - let%test bindings: excluded (tests don't need docs) - let open M in: excluded (local opens)

Doc detection (OCaml): - (** docstring *) attached to binding - (* author note [|description|] *) borg-style comment on prior line - (* |description |*) alternative borg format - (* exempt doc *) – marks binding as exempt from coverage

Integration with drift: `borge drift --docs` flag runs documentation analysis and writes `.borg.meta` files. Drift findings can include undocumented-module for files with <50% coverage.

Integration with stats: `borge stats` shows coverage aggregate across all `.borg.meta` files.

extraction-strategy

How to extract bindings and doc comments from OCaml

[status: planned]

Two approaches: compiler-libs (accurate) vs regex (fast).

Compiler-libs approach (recommended): - Use `ocaml-common/parsetree` to parse `.ml` files - Walk the AST for toplevel `Pstr_value` bindings - Extract docstrings from attributes - For `.ml` files without `.mli`: all toplevel vals are exports - For `.ml` files with `.mli`: cross-reference to determine public vs private

Regex fallback (if compiler-libs is heavy): - Match `^let\\s+(rec\\s+)?([a-z][a-zA-Z0-9_]*)` - Look backward for comment before the `let` - Heuristic: skip lines containing `"let%"` - Less accurate but faster for large codebases

Current limitation: `metrics/stats.ml` uses regex and reports 0% coverage because it doesn't parse nested comments. This section describes the proper implementation using compiler-libs.

9 sexp.borg

borge-lang

The borge language: parsing, printing, and balancing

[status: in-progress]

ast

AST types for sexp, annotated comments, verbatim strings

[status: implemented]

Core types: sexp = Atom of pos * symbol | String of pos * string_value |

List of pos * sexp list

string_value = Quoted of quoted_string | Verbatim of verbatim_string

Comments: plain (;) and annotated (* *). Annotated comments carry authorship (Single or Multiple) and type (Untyped or Typed of symbol). Verbatim strings track nesting depth via v_content. Characters | and) written separately: the closing delimiter is NOT included in v_content; it is a delimiter re-added by print during output.

Positions: every sexp node carries a pos record with { line: int; col: int; offset: int }. The file type holds top_level_comments, top_level sexp_with_comments, trailing_comments, and nested_comments for round-tripping. nested_comments is a position-pinned list of comments that appear inside list forms (before any child sexp). See the comment-attachment section for why this is stored as side data and not as per-child comments_before. The module also exposes fold_comments and iter_comments, single traversals over all comments in a file (top-level, before each node, nested, trailing) used by lint and the book renderer so the walk is defined once.

parser

Hand-written recursive descent parser – to be replaced by sedlex+menhir

[status: implemented]

Current: parse.ml is a 311-line hand-written character-by-character parser. It uses a mutable state record { input; pos; line; column } and parses sexp forms by peek/advance. The Lexer module is limited to is_symbol_char/is_whitespace helpers – parse.ml does its own tokenization inline.

What it handles: - Atom: bare symbols (a-z, A-Z, 0-9, -, _, ., ?, !) - Quoted strings: "..." with \n, \t, \\\, \" escapes - Verbatim strings: open-pipe then close-pipe with nesting depth tracking - Plain comments: ; to end of line - Annotated comments: hash-open then star-close, with author [type] value format authorship (single or multiple), comment_type, and value (Quoted or Verbatim) - Lists: (...) with comments_before attachments

What it does NOT handle: - Nested OCaml comments: (* (* inner *) outer *) – the first *) closes the comment. This is the same limitation that makes doc_coverage unreliable in stats.ml. - Unicode: ASCII-only symbol characters - Good error messages: "unexpected character" with no grammar context. Menhir would say "expected RPAREN but got SYMBOL at line X, col Y, in rule list_tail". - Grammar-as-spec: the parser IS the spec, but it's imperative code, not a declarative grammar.

sedlex-lexer

Replace char helpers with a proper Unicode-aware tokenizer

[status: implemented]

sedlex produces a lexer from .lex (or inline) rules. Tokens are already defined in lexer.ml: LPAREN, RPAREN, HASH_LPAREN, STAR_RPAREN, VERB_OPEN, VERB_CLOSE, QUOTED of string, SYMBOL of string, SEMI_COLON of string, EOF

The new lexer will: - Actually produce these tokens (currently they're unused) - Handle Unicode identifiers via sedlex character classes - Handle nested comments by tracking depth in lexer state - Attach source positions to every token - Replace the inline char scanning in parse.ml

Key: the nested-comment problem is a LEXER concern, not a parser concern. sedlex tracks depth with a ref cell and only emits STAR_RPAREN when depth hits 0.

menhir-parser

Replace hand-written descent with LR(1) grammar

[status: implemented]

menhir produces a parser from .mly grammar rules. The grammar is small – borge sexp has ~8 production rules:

```
file ::= comment* (sexp comment*)* sexp ::= atom | string | list list ::=
LPAREN sexp* RPAREN string ::= QUOTED | verbatim verbatim ::= VERB_OPEN
content VERB_CLOSE comment ::= SEMICOLON text | HASH_LPAREN
author type value STAR_RPAREN author ::= SYMBOL | LPAREN SYMBOL+
RPAREN type ::= /* empty */ | SYMBOL value ::= /* empty */ |
string
```

Nested comments: comments appearing inside a list (between child sexps) are captured into file.nested_comments as a position-pinned (pos * comment_attachment) list, rather than discarded. The list spine (LPAREN sexp* RPAREN) is unchanged – comments remain side data, not children. Comment start_line/end_line are now filled from menhir \$startpos/\$endpos (previously hardcoded to 0). See comment-attachment for the future child-attachment plan.

The .mly becomes the canonical language definition – the spec IS the grammar. This is recursive: borge.borg describes the .borg format, and the .mly IS the .borg format, formally specified.

Benefits: - Grammar-as-spec: the .mly is the authoritative format def - Generated error messages with position and expected tokens - Incremental parsing possible (menhir –incremental) - The grammar is diffable and reviewable, unlike 311 lines of imperative state mutation

Migration plan: 1. Write parser.mly producing the same Ast types 2. Write sedlex lexer producing the existing token type 3. Replace Parse.parse_file with menhir entry point 4. Verify all tests pass (same AST, same behavior) 5. Remove hand-written parse.ml

print

Print AST back to .borg. Preserves comments, diffable output

[status: implemented]

Canonical formatting rules: keyword forms get children on their own lines. Atom names stay inline. (doc ...) and (status ...) always inline. Verbatim strings preserve content verbatim - the closing delimiter is re-added during printing, never stored in v_content.

Comments: plain as ;text, annotated as (* author type value *). The file type (with top_level_comments, trailing_comments, and nested_comments) enables full round-tripping: parse -> print produces diffable output. Nested comments are re-emitted in source position order so their lexical placement is preserved.

comment-attachment

Preserve and traverse comments inside nested lists

[status: implemented]

Most .borg files wrap everything in a single (project ...) or (section ...) form. Ordinary top-level comments live in file.top_level_comments and file.trailing_comments, and comments_before on sexp_with_comments handles comments before each

top-level node. But comments inside those top-level forms – before a (section ...) child inside (project ...), for example – are comments inside a list. Historically the parser discarded them via `inner_comment_list`. borge now captures them in `file.nested_comments` instead.

position-pinned

Nested comments as position-ordered side data

[status: implemented]

Implemented approach: nested comments are stored as a flat list keyed by their source position. They are re-emitted during print by interleaving them with the node stream in position order. This is sufficient for round-tripping and for consumers (lint, book renderer) that only need to see every comment once in source order. It does not model semantic attachment of a comment to a specific child node; that is deferred to future work (child-attachment).

child-attachment

Future: make List children carry `comments_before` directly

[status: planned]

Long-term design: change `Ast.sexp` List children from bare sexp to `sexp_with_comments`, so every child in a list has its own `comments_before`. This is the principled data model: a comment before (section foo ...) becomes structurally attached to that section. The migration is larger (~3-digit pattern sites in drift/lint/balance visitors) and will be done as a separate refactor.

balance

Fast paren balance check with lexical context tracking

[status: implemented]

Character-level state machine that counts only structural parens, skipping comments, verbatim strings, and quoted strings via a state stack. States: Normal, Plain_comment, Annotated_comment, Quoted_string, Verbatim(depth), Inline_example(depth).

Reports: Balanced with max nesting depth, or Imbalanced with structured error details (Unexpected_close with position pairing, Unclosed_parens with paren stack, Unclosed_context for un-closed lexical states).

This is NOT the parser – it's a fast pre-check. It does not build an AST. It stays character-level even after the menhir rewrite because speed matters for this tool. The parser rewrite does not affect `balance.ml`.

Verbatim and inline-example nesting: both track depth counters. Nested open-pipe close-pipe pairs are valid. Nested angle-bracket example delimiters are also valid. Balance counts the outer parens correctly.

enhanced-output

Structural skeleton, indent divergence, and smart close suggestions

[status: implemented]

When balance finds errors, it reports them concisely with one line per issue. It also validates that structural '(' on their own line match the column of their matching ') – catching the common agent mistake of closing the wrong form. Errors include: - unexpected_close: ')' with no matching '(' - unclosed_parens: EOF reached with open forms - unclosed_context: unclosed string/verbatim/comment - indent_mismatch: structural '(' at wrong column

With `-verbose`, balance shows the full paren stack.

indent-repair

Infer missing or extra parens from indentation (Parinfer-style)

[status: implemented]

Agents are good at indentation (visual/spatial) but bad at counting parens (symbolic/stateful). When a .borg file has balance errors, the single most useful recovery action is to treat indentation as the canonical structure and infer the parens from it.

The ‘-repair’ flag produces a corrected version of the file by reading indentation as a tree:

Algorithm (one-pass line scanner): 1. Track open forms with (line, indent, keyword). 2. For each new line at indent I, close every form whose indent $\geq$ I and has not been explicitly closed. 3. Missing closes become new lines at the correct indentation (one ‘)’ per line, innermost first). 4. Structural ‘)’ that don’t match their opener are trimmed. Inline ‘)’ that overflow the stack are trimmed. At EOF, close all remaining open forms. 5. Repaired file is guaranteed to pass ‘borge balance’.

Lexical context: the repair scanner MUST skip comments, verbatim strings, and quoted strings using the same state stack as the balance checker. It MUST NOT repair inside string literals or comments.

Output modes: - ‘borge balance -repair FILE’ prints the corrected file to stdout (exit 0 even if repairs were made). - ‘borge balance -repair-diff FILE’ shows a diff between original and repaired. - Combined with ‘-json’, outputs repair_actions: [{"action": "insert_line", "line": 12, "indent": 3, "reason": "close beta opened at line 4"}] plus repaired_text with the full corrected file.

Canonical format assumption: repair assumes the file follows borge canonical formatting (one form per line, strictly nested). Non-canonical files may still repair correctly but are not guaranteed.

enriched-json

Structured tree + repair actions in JSON output

[status: implemented]

When ‘-json’ is passed, balance produces a rich structured response that programmatic consumers (agents) can act on without parsing human text:

```
“json { "status": "imbalanced", "errors": [ { "type": "indent_mismatch", "pos": { "line": 6, "col": 3 },
"expected_col": 2, "keyword": "section", "open_pos": { "line": 2, "col": 2 } }, "structural_tree": [ { "line": 1, "depth": 1, "keyword": "project", "open_col": 1, "closed": false },
{ "line": 3, "depth": 2, "keyword": "section", "open_col": 2, "closed": false }, ... ], "indent_divergences": [ { "line": 13, "col": 3, "depth": 5, "expected_col": 5, "hint": "a ‘)
may be missing before this line" }, "repair_actions": [ { "action": "insert_line", "line": 5, "indent": 3,
"reason": "close beta opened at line 4" }, { "action": "remove", "line": 6, "column": 0,
"text": "" }, { "action": "remove", "line": 6, "column": 0,
"reason": "structural ‘) does not match open-paren indentation" } ],
"repaired_text": "(project test\n ..." }
```

The ‘structural_tree’ is a flat list of paren pairs in source order. Unclosed pairs are marked ‘“closed”: false’ with no ‘close_line’ field.

The ‘repair_actions’ list is only present when the file is imbalanced AND ‘-json’ is used. Each action: - action: “insert_line” | “remove” - line: 1-indexed position in the original - indent: for insert_line, the column of the new ‘)’ - text: for remove, the string being removed - reason: human-readable explanation

‘repaired_text’ contains the full corrected file.

error

Error types for parse and balance failures

[status: implemented]

parse_error: { message: string; line: int; column: int } Raised as Parse_error exception from parse.ml. After menhir: menhir produces its own error type.

The balance module has its own `error_detail` union: `Unexpected_close`, `Unclosed_parens`, `Unclosed_context`, `Indent_mismatch`.

lexer

Token types and helper predicates for sexp scanning

[status: implemented]

lexer.ml defines token types (`LPAREN`, `RPAREN`, `HASH_LPAREN`, `STAR_RPAREN`, `VERB_OPEN`, `VERB_CLOSE`, `QUOTED`, `SYMBOL`, `SEMICOLON`, `EOF`) and position tracking. Currently only `is_symbol_char` and `is_whitespace` are used by `parse.ml`. After `sedlex` rewrite, this module becomes the generated lexer that actually produces these tokens.

10 lib/lang/ast.ml

```

1  type pos = {
2      line : int;
3      col : int;
4      offset : int;
5  }
6
7  type verbatim_string = {
8      v_content : string;
9      (* The raw text between (| and |), newlines preserved *)
10 }
11
12 type quoted_string = {
13     q_content : string;
14     (* The decoded text, escapes already resolved *)
15 }
16
17 type string_value =
18     | Quoted of quoted_string
19     | Verbatim of verbatim_string
20
21 type symbol = string
22
23 type sexp =
24     | Atom of pos * symbol
25     | String of pos * string_value
26     | List of pos * sexp list
27
28 (** A plain (;) comment -- not part of the AST structure,
29     but preserved for round-tripping *)
30 type plain_comment = {
31     text : string;
32     line : int;
33 }
34
35 (** An annotated (* *) comment -- first-class AST node *)
36 type authorship =
37     | Single of symbol
38     | Multiple of symbol list
39
40 type comment_type =
41     | Untyped (* two-position: (* author value *) -- implicitly "note" *)
42     | Typed of symbol (* three-position: (* author type value *) *)
43
44 type annotated_comment = {
45     authorship : authorship;
46     comment_type : comment_type;
47     value : string_value option; (* None for empty (||) *)
48     start_line : int;
49     end_line : int;
50 }
51
52 (** Attachment: comments are attached to the next AST node *)
53 type comment_attachment =
54     | Plain of plain_comment
55     | Annotated of annotated_comment
56
57 type sexp_with_comments = {
58     comments_before : comment_attachment list;
59     node : sexp;
60     end_pos : pos; (* position after the closing delimiter *)
61 }
62
63 type file = {
64     top_level_comments : comment_attachment list;
65     (* Comments at the top of file before any sexp *)
66     top_level : sexp_with_comments list;
67     (* The main s-expressions in the file *)
68     trailing_comments : comment_attachment list;
69     (* Comments after the last sexp *)
70     nested_comments : (pos * comment_attachment) list;
71     (* Comments inside list forms, keyed by source position. Kept as

```

```

72     side data because Ast.List currently stores bare sexp children
73     without comment_attachment wrappers. See the comment-attachment
74     section in sexp.borg. *)
75 }
76
77 (** Single traversal over every sexp in a file, depth-first,
78     in source order. *)
79 let iter_sexps file f =
80   let rec go_sexp s =
81     f s;
82     match s with
83     | List (_, children) -> List.iter go_sexp children
84     | _ -> ()
85   and go_swc swc = go_sexp swc.node in
86   List.iter go_swc file.top_level
87
88 (** Fold over every sexp in a file, depth-first, in source order. *)
89 let fold_sexps file acc0 f =
90   let rec go_sexp acc s =
91     match s with
92     | List (_, children) -> List.fold_left go_sexp (f acc s) children
93     | _ -> f acc s
94   in
95   let go_swc acc swc = go_sexp acc swc.node in
96   List.fold_left go_swc acc0 file.top_level
97
98 (** Return all comments in source order: top-level leading, before each
99     top-level node, nested inside list forms, trailing. *)
100 let all_comments file =
101   file.top_level_comments
102   @ List.concat_map (fun swc -> swc.comments_before) file.top_level
103   @ List.map snd (List.sort (fun (p1, _) (p2, _) -> compare p1.offset
104     p2.offset) file.nested_comments)
104   @ file.trailing_comments
105
106 let iter_comments file f = List.iter f (all_comments file)
107
108 let fold_comments file acc0 f = List.fold_left f acc0 (all_comments file)
109
110 let start_pos_of = function
111   | Atom (p, _) | String (p, _) | List (p, _) -> p
112
113 let rec end_pos_of = function
114   | Atom (p, _) | String (p, _) | List (p, []) -> p
115   | List (_, children) -> end_pos_of (List.hd (List.rev children))
116
117 let nested_comments_acc : (pos * comment_attachment) list ref = ref []
118 let reset_nested_comments () = nested_comments_acc := []

```

11 lib/lang/balance.ml

```

1  (** Fast structural balance checker -- scans characters without fully
2     parsing.
3     Aware of all borge lexical contexts: comments, strings, verbatim
4     blocks. *)
5
6  type pos = { line : int; col : int }
7
8  type paren_frame = {
9     open_pos : pos;
10    keyword : string option; (** e.g. "section", "doc", None for bare lists
11                               *)
12  }
13
14  type error_detail =
15    | Unexpected_close of pos * pos option
16    | Unclosed_parens of paren_frame list
17    | Unclosed_context of pos * string
18    | Indent_mismatch of pos * paren_frame (** structural '(' column does
19      not match '(' column *)
20
21  type check_result =
22    | Balanced of { max_depth : int }
23    | Imbalanced of error_detail list
24
25  (* --- Detailed analysis types --- *)
26
27  type paren_pair = {
28    mutable open_pos : pos;
29    mutable close_pos : pos option;
30    keyword : string option;
31    depth : int; (** 1-indexed nesting level *)
32  }
33
34  type indent_divergence = {
35    line : int;
36    col : int;
37    depth : int;
38    expected_col : int;
39  }
40
41  type analysis = {
42    result : check_result;
43    pairs : paren_pair list;
44  }
45
46  (* agent note [5200] (|
47   *   WHAT: Convert a balance error detail to a human-readable string
48   *   describing the problem and where it occurred.
49   *
50   *   WHY: Error messages from the balance checker need to be presented
51   *   to users in a readable form. Each error variant has specific context.
52   * |) *)
53  let string_of_error = function
54    | Unexpected_close (p, None) ->
55      Printf.sprintf "Line %d, col %d: unexpected '%'" -- no matching '('"
56        p.line p.col
57    | Unexpected_close (p, Some open_p) ->
58      Printf.sprintf "Line %d, col %d: unexpected '%'" -- opened at line %d,
59        col %d"
60        p.line p.col open_p.line open_p.col
61    | Unclosed_parens frames ->
62      let count = List.length frames in
63      begin match List.rev frames with
64      | first :: _ ->
65        Printf.sprintf "End of file: %d unclosed '(' -- first opened at
66          line %d, col %d"
67            count first.open_pos.line first.open_pos.col
68      | [] -> "End of file: unclosed '('"
69      end
70    | Unclosed_context (p, ctx) ->
71      Printf.sprintf "End of file: unclosed %s (opened at line %d, col %d)"

```

```

65     ctx p.line p.col
66 | Indent_mismatch (p, frame) ->
67     let kw = match frame.keyword with Some k -> k | None -> "form" in
68     Printf.sprintf "Line %d, col %d: structural '%c' at column %d does not
        match '%c' at column %d (%s, opened line %d)"
69     p.line p.col p.col frame.open_pos.col kw frame.open_pos.line
70
71 (** Render the paren stack for verbose output *)
72 let string_of_paren_stack (frames : paren_frame list) =
73     let buf = Buffer.create 128 in
74     Buffer.add_string buf "Paren stack (innermost first):\n";
75     List.iter (fun (frame : paren_frame) ->
76         match frame.keyword with
77         | Some kw ->
78             Printf.bprintf buf " (%d:%d) %s\n" frame.open_pos.line
                frame.open_pos.col kw
79         | None ->
80             Printf.bprintf buf " (%d:%d) (\n" frame.open_pos.line
                frame.open_pos.col
81     ) frames;
82     Buffer.contents buf
83
84 module State = struct
85     type t =
86     | Normal
87     | Plain_comment
88     | Annotated_comment
89     | Quoted_string
90     | Verbatim of int
91     | Inline_example of int
92
93     let name = function
94     | Normal -> "normal"
95     | Plain_comment -> "; comment"
96     | Annotated_comment -> "(* *) comment"
97     | Quoted_string -> "\"...\" string"
98     | Verbatim _ -> "(|...|) string"
99     | Inline_example _ -> "<<...>> example"
100 end
101
102 type frame = { state : State.t; pos : pos }
103
104 (* agent note [5100] (|
105  *   WHAT: Create a position record from a line and column number.
106  *
107  *   WHY: Position tracking is used throughout balance checking to
108  *   report where errors occurred. This is a simple constructor.
109  * |) *)
110 let mk_pos line col = { line; col }
111
112 (** Peek at the keyword following an open paren.
113     Scans forward from 'start' looking for a symbol character.
114     Returns None if the next thing is a close-paren, quote, or another
115     open-paren. *)
116 let peek_keyword text start len =
117     let i = ref start in
118     (* Skip whitespace *)
119     while !i < len && (let ch = text.[!i] in ch = ' ' || ch = '\t' || ch =
        '\n' || ch = '\r') do
120         incr i
121     done;
122     if !i >= len then None
123     else begin
124         let ch = text.[!i] in
125         if ch = ')' || ch = '(' || ch = '"' then None
126         else begin
127             (* Read symbol characters *)
128             let buf = Buffer.create 32 in
129             while !i < len && Lexer.is_symbol_char text.[!i] do
130                 Buffer.add_char buf text.[!i];
131                 incr i
132             done;
133             let kw = Buffer.contents buf in

```



```

133     if kw = "" then None else Some kw
134   end
135 end
136
137 (** Main check: returns structured result without printing.
138     Fast path -- no pair tracking, minimal allocation. *)
139 let check text =
140   let len = String.length text in
141   let line = ref 1 in
142   let col = ref 1 in
143   let state_stack = ref [] in
144   let paren_stack : paren_frame list ref = ref [] in
145   let max_depth = ref 0 in
146   let errors = ref [] in
147   let seen_non_ws = ref false in
148
149   let push_state st =
150     state_stack := { state = st; pos = mk_pos !line !col } :: !state_stack
151   in
152   let pop_state () =
153     match !state_stack with
154     | [] -> ()
155     | _ :: rest -> state_stack := rest
156   in
157   let current_state () =
158     match !state_stack with
159     | [] -> State.Normal
160     | frame :: _ -> frame.state
161   in
162   let here () = mk_pos !line !col in
163
164   let i = ref 0 in
165   while !i < len do
166     let ch = text.[!i] in
167
168     if ch = '\n' then (
169       line := !line + 1;
170       col := 1;
171       seen_non_ws := false;
172       incr i;
173       if current_state () = State.Plain_comment then pop_state ();
174     ) else (
175       match current_state () with
176       | State.Normal ->
177         if ch = ';' then (
178           push_state State.Plain_comment;
179           incr i; incr col
180         ) else if ch = '(' && !i + 1 < len && text.[!i + 1] = '*' then (
181           push_state State.Annotated_comment;
182           seen_non_ws := false;
183           i := !i + 2; col := !col + 2
184         ) else if ch = '(' && !i + 1 < len && text.[!i + 1] = '|' then (
185           push_state (State.Verbatim 1);
186           seen_non_ws := false;
187           i := !i + 2; col := !col + 2
188         ) else if ch = '"' then (
189           push_state State.Quoted_string;
190           seen_non_ws := false;
191           incr i; incr col
192         ) else if ch = '(' then (
193           let keyword = peek_keyword text (!i + 1) len in
194           paren_stack := { open_pos = here (); keyword } :: !paren_stack;
195           max_depth := max !max_depth (List.length !paren_stack);
196           seen_non_ws := true;
197           incr i; incr col
198         ) else if ch = ')' then (
199           let is_structural = not !seen_non_ws in
200           seen_non_ws := true;
201           (match !paren_stack with
202            | [] ->
203              errors := Unexpected_close (here (), None) :: !errors;
204              incr i; incr col
205            | frame :: rest ->

```

```

206         if is_structural && frame.open_pos.col <> !col then (
207             errors := Indent_mismatch (here (), frame) :: !errors
208         );
209         paren_stack := rest;
210         incr i; incr col
211     )
212 ) else (
213     if ch <> ' ' && ch <> '\t' then seen_non_ws := true;
214     incr i; incr col
215 )
216
217 | State.Plain_comment ->
218     incr i; incr col
219
220 | State.Annotated_comment ->
221     if ch = '*' && !i + 1 < len && text.[!i + 1] = ')' then (
222         pop_state ();
223         seen_non_ws := true;
224         i := !i + 2; col := !col + 2
225     ) else (
226         incr i; incr col
227     )
228
229 | State.Quoted_string ->
230     if ch = '\\' && !i + 1 < len then (
231         i := !i + 2; col := !col + 2
232     ) else if ch = '"' then (
233         pop_state ();
234         seen_non_ws := true;
235         incr i; incr col
236     ) else (
237         incr i; incr col
238     )
239
240 | State.Verbatim depth ->
241     if ch = '(' && !i + 1 < len && text.[!i + 1] = '|' then (
242         push_state (State.Verbatim (depth + 1));
243         i := !i + 2; col := !col + 2
244     ) else if ch = '|' && !i + 1 < len && text.[!i + 1] = ')' then (
245         pop_state ();
246         seen_non_ws := true;
247         i := !i + 2; col := !col + 2
248     ) else if ch = '<' && !i + 1 < len && text.[!i + 1] = '>' then (
249         push_state (State.Inline_example 1);
250         i := !i + 2; col := !col + 2
251     ) else (
252         incr i; incr col
253     )
254
255 | State.Inline_example depth ->
256     if ch = '<' && !i + 1 < len && text.[!i + 1] = '>' then (
257         push_state (State.Inline_example (depth + 1));
258         i := !i + 2; col := !col + 2
259     ) else if ch = '>' && !i + 1 < len && text.[!i + 1] = '<' then (
260         pop_state ();
261         seen_non_ws := true;
262         i := !i + 2; col := !col + 2
263     ) else (
264         incr i; incr col
265     )
266 )
267 done;
268
269 (* Unclosed lexical contexts *)
270 let context_errors =
271     List.map (fun frame ->
272         Unclosed_context (frame.pos, State.name frame.state)
273     ) !state_stack
274 in
275
276 let paren_error =
277     match !paren_stack with
278     | [] -> []

```

```

279 | frames ->
280   [ Unclosed_parens (List.rev frames) ]
281 in
282
283 let all_errors = List.rev (!errors @ paren_error @ context_errors) in
284 if all_errors = [] then
285   Balanced { max_depth = !max_depth }
286 else
287   Imbalanced all_errors
288
289 (* agent note [5300] (|
290  *   WHAT: Run a structural analysis of a .borg file, recording
291  *   every paren pair (open and close), lexical errors, and
292  *   indent/depth divergences.
293  *
294  *   WHY: The enhanced balance report needs structural data to
295  *   produce the skeleton view, divergence hints, and smart
296  *   close suggestions for imbalanced files.
297  * |) *)
298 let analyze text =
299   let len = String.length text in
300   let line = ref 1 in
301   let col = ref 1 in
302   let state_stack = ref [] in
303   let paren_stack : paren_frame list ref = ref [] in
304   let pair_stack : paren_pair list ref = ref [] in
305   let all_pairs : paren_pair list ref = ref [] in
306   let max_depth = ref 0 in
307   let errors = ref [] in
308   let seen_non_ws = ref false in
309
310   let push_state st =
311     state_stack := { state = st; pos = mk_pos !line !col } :: !state_stack
312   in
313   let pop_state () =
314     match !state_stack with
315     | [] -> ()
316     | _ :: rest -> state_stack := rest
317   in
318   let current_state () =
319     match !state_stack with
320     | [] -> State.Normal
321     | frame :: _ -> frame.state
322   in
323   let here () = mk_pos !line !col in
324
325   let i = ref 0 in
326   while !i < len do
327     let ch = text.[!i] in
328
329     if ch = '\n' then (
330       line := !line + 1;
331       col := 1;
332       seen_non_ws := false;
333       incr i;
334       if current_state () = State.Plain_comment then pop_state ();
335     ) else (
336       match current_state () with
337       | State.Normal ->
338         if ch = ';' then (
339           push_state State.Plain_comment;
340           incr i; incr col
341         ) else if ch = '(' && !i + 1 < len && text.[!i + 1] = '*' then (
342           push_state State.Annotated_comment;
343           seen_non_ws := false;
344           i := !i + 2; col := !col + 2
345         ) else if ch = '(' && !i + 1 < len && text.[!i + 1] = '|' then (
346           push_state (State.Verbatim 1);
347           seen_non_ws := false;
348           i := !i + 2; col := !col + 2
349         ) else if ch = '"' then (
350           push_state State.Quoted_string;
351           seen_non_ws := false;

```

```

352     incr i; incr col
353   ) else if ch = '(' then (
354     let keyword = peek_keyword text (!i + 1) len in
355     let open_pos = here () in
356     let depth = List.length !paren_stack + 1 in
357     paren_stack := { open_pos; keyword } :: !paren_stack;
358     max_depth := max !max_depth depth;
359     let pair = { open_pos; close_pos = None; keyword; depth } in
360     pair_stack := pair :: !pair_stack;
361     all_pairs := pair :: !all_pairs;
362     seen_non_ws := true;
363     incr i; incr col
364   ) else if ch = ')' then (
365     let is_structural = not !seen_non_ws in
366     seen_non_ws := true;
367     (match !pair_stack with
368     | [] -> ()
369     | pair :: rest ->
370       pair.close_pos <- Some (here ());
371       pair_stack := rest;
372     );
373     (match !paren_stack with
374     | [] ->
375       errors := Unexpected_close (here (), None) :: !errors;
376       incr i; incr col
377     | frame :: rest ->
378       if is_structural && frame.open_pos.col <> !col then (
379         errors := Indent_mismatch (here (), frame) :: !errors;
380       );
381       paren_stack := rest;
382       incr i; incr col
383     )
384   ) else (
385     if ch <> ' ' && ch <> '\t' then seen_non_ws := true;
386     incr i; incr col
387   )
388
389 | State.Plain_comment ->
390   incr i; incr col
391
392 | State.Annotated_comment ->
393   if ch = '*' && !i + 1 < len && text.[!i + 1] = ')' then (
394     pop_state ();
395     seen_non_ws := true;
396     i := !i + 2; col := !col + 2
397   ) else (
398     incr i; incr col
399   )
400
401 | State.Quoted_string ->
402   if ch = '\\' && !i + 1 < len then (
403     i := !i + 2; col := !col + 2
404   ) else if ch = '"' then (
405     pop_state ();
406     seen_non_ws := true;
407     incr i; incr col
408   ) else (
409     incr i; incr col
410   )
411
412 | State.Verbatim_depth ->
413   if ch = '(' && !i + 1 < len && text.[!i + 1] = '|' then (
414     push_state (State.Verbatim (depth + 1));
415     i := !i + 2; col := !col + 2
416   ) else if ch = '|' && !i + 1 < len && text.[!i + 1] = ')' then (
417     pop_state ();
418     seen_non_ws := true;
419     i := !i + 2; col := !col + 2
420   ) else if ch = '<' && !i + 1 < len && text.[!i + 1] = '>' then (
421     push_state (State.Inline_example 1);
422     i := !i + 2; col := !col + 2
423   ) else (
424     incr i; incr col

```

```

425         )
426
427     | State.Inline_example depth ->
428         if ch = '<' && !i + 1 < len && text.[!i + 1] = '<' then (
429             push_state (State.Inline_example (depth + 1));
430             i := !i + 2; col := !col + 2
431         ) else if ch = '>' && !i + 1 < len && text.[!i + 1] = '>' then (
432             pop_state ();
433             seen_non_ws := true;
434             i := !i + 2; col := !col + 2
435         ) else (
436             incr i; incr col
437         )
438     )
439 done;
440
441 (* Unclosed lexical contexts *)
442 let context_errors =
443     List.map (fun frame ->
444         Unclosed_context (frame.pos, State.name frame.state)
445     ) !state_stack
446 in
447
448 let paren_error =
449     match !paren_stack with
450     | [] -> []
451     | frames ->
452         [ Unclosed_parens (List.rev frames) ]
453 in
454
455 let all_errors = List.rev (!errors @ paren_error @ context_errors) in
456 let result =
457     if all_errors = [] then
458         Balanced { max_depth = !max_depth }
459     else
460         Imbalanced all_errors
461 in
462 { result; pairs = List.rev !all_pairs }
463
464 (** Find indent divergences: open parens whose column does not
465     match their canonical depth. *)
466 let find_divergences (pairs : paren_pair list) =
467     List.filter_map (fun (pair : paren_pair) ->
468         let expected = pair.depth in
469         if pair.open_pos.col <> expected then
470             Some ({ line = pair.open_pos.line; col = pair.open_pos.col; depth =
471                 pair.depth; expected_col = expected } : indent_divergence)
472         else
473             None
474     ) pairs
475
476 (** Render indent divergences as human-readable hints. *)
477 let render_divergence d =
478     Printf.printf " Line %d, col %d: '(' at depth %d but column is %d\n"
479         (expected %d)\n"
480         d.line d.col d.depth d.col d.expected_col;
481     if d.col > d.expected_col then
482         Printf.printf " Hint: this form is over-indented -- check for an
483             extra '(' above this line\n"
484     else
485         Printf.printf " Hint: this form is under-indented -- a ')' may be
486             missing before this line\n"
487
488 (** Render a structural skeleton of all paren pairs with source previews. *)
489 let render_skeleton text pairs =
490     let lines = String.split_on_char '\n' text in
491     let max_line_num =
492         List.fold_left (fun acc p -> max acc p.open_pos.line) 0 pairs
493     in
494     let line_digits = String.length (string_of_int max_line_num) in
495     let fmt_line n = Printf.sprintf "%*d" line_digits n in
496     List.iter (fun p ->
497         let source =

```

```

494     try List.nth lines (p.open_pos.line - 1) with _ -> ""
495   in
496   let preview_len = 58 in
497   let preview =
498     let s = String.trim source in
499     let len = String.length s in
500     if len > preview_len then String.sub s 0 preview_len ^ "..." else s
501   in
502   let status = match p.close_pos with
503   | Some c -> Printf.sprintf "[closed:%d]" c.line
504   | None -> "[? UNCLOSED]"
505   in
506   let kw = match p.keyword with Some k -> k | None -> "" in
507   Printf.printf "  %s: %-58s %s %s\n" (fmt_line p.open_pos.line) preview
    status kw
508 ) pairs
509
510 (** Compute the indent (leading spaces) of a source line. *)
511 let line_indent line_text =
512   let len = String.length line_text in
513   let i = ref 0 in
514   while !i < len && line_text.[!i] = ' ' do incr i done;
515   !i
516
517 (** Find the next line at or before a given indent, starting from a line.
518   Returns the line number and text of the boundary line, or None. *)
519 let find_sibling_boundary text start_line target_indent =
520   let lines = String.split_on_char '\n' text in
521   let total = List.length lines in
522   let rec scan line_num =
523     if line_num > total then None
524     else begin
525       let line_text =
526         try List.nth lines (line_num - 1) with _ -> ""
527       in
528       let trimmed = String.trim line_text in
529       if trimmed <> "" then begin
530         let ind = line_indent line_text in
531         if ind <= target_indent then
532           Some (line_num, line_text)
533         else
534           scan (line_num + 1)
535       end else
536         scan (line_num + 1)
537     end
538   in
539   scan start_line
540
541 (** Render a smart close suggestion for a single unclosed paren frame. *)
542 let render_smart_suggestion text (pairs : paren_pair list) (frame :
    paren_frame) =
543   (* Find the pair matching this frame *)
544   let matching_pairs =
545     List.filter (fun p ->
546       p.close_pos = None &&
547       p.open_pos.line = frame.open_pos.line &&
548       p.open_pos.col = frame.open_pos.col
549     ) pairs
550   in
551   match matching_pairs with
552   | [] -> ()
553   | pair :: _ ->
554     let kw = match pair.keyword with Some k -> k | None -> "form" in
555     Printf.printf "  %s (line %d): missing close '%'\n" kw
    pair.open_pos.line;
556     let target_indent = pair.open_pos.col - 1 in
557     let search_start = pair.open_pos.line + 1 in
558     match find_sibling_boundary text search_start target_indent with
559     | None ->
560       let lines = String.split_on_char '\n' text in
561       let last_line = List.length lines in
562       Printf.printf "    Suggestion: add '%'\n" at the end of the file
    (after_line %d).\n" last_line;

```

```

563     Printf.printf "    Add: %s\n" (String.make target_indent ' ')
564   | Some (boundary_line, boundary_text) ->
565     let indent_str = String.make target_indent ' ' in
566     let prev_line_text =
567       try
568         let lines = String.split_on_char '\n' text in
569         List.nth lines (boundary_line - 2)
570       with _ -> ""
571     in
572     Printf.printf "    Suggestion: add '()' before line %d:\n"
boundary_line;
573     Printf.printf "        %3d | %s\n" (boundary_line - 1)
prev_line_text;
574     Printf.printf "        +      | %s\n" indent_str;
575     Printf.printf "        %3d | %s\n" boundary_line boundary_text;
576     Printf.printf "        (closes %s opened at line %d)\n" kw
pair.open_pos.line
577
578 (** Extract lines [start_line; end_line] inclusive from text *)
579 (* ?? Indent-based repair (Parinfer-style) ?????????????????????????? *)
580
581 type line_event = {
582   ev_line : int;
583   ev_open_col : int option;      (** column of first structural '(' *)
584   ev_keyword : string option;
585   ev_structural_close_col : int option; (** column of first structural ')' *)
586   ev_inline_close_count : int;   (** number of inline '()' *)
587 }
588
589 type repair_action =
590   | Insert_line of { line : int; indent : int; reason : string }
591   | Trim_end of { line : int; count : int; reason : string }
592
593 (** Per-line scanner: for every line, record first open-paren and total
594   close-parens, skipping comments/strings/verbatim. *)
595 let scan_line_events text =
596   let len = String.length text in
597   let line = ref 1 in
598   let col = ref 1 in
599   let state_stack = ref [] in
600   let events = ref [] in
601   let current_open = ref None in
602   let current_keyword = ref None in
603   let current_structural_close = ref None in
604   let current_inline_closes = ref 0 in
605   let seen_non_ws = ref false in
606
607   let push_state st = state_stack := st :: !state_stack in
608   let pop_state () = state_stack := (match !state_stack with _::t -> t | [] -> []) in
609   let current_state () = match !state_stack with [] -> State.Normal | s::_ -> s in
610   let flush_event () =
611     events := {
612       ev_line = !line;
613       ev_open_col = !current_open;
614       ev_keyword = !current_keyword;
615       ev_structural_close_col = !current_structural_close;
616       ev_inline_close_count = !current_inline_closes;
617     } :: !events;
618     current_open := None;
619     current_keyword := None;
620     current_structural_close := None;
621     current_inline_closes := 0;
622     seen_non_ws := false
623   in
624
625   let i = ref 0 in
626   while !i < len do
627     let ch = text.[!i] in
628
629     if ch = '\n' then (

```

```

630 flush_event ();
631 line := !line + 1;
632 col := 1;
633 incr i;
634 if current_state () = State.Plain_comment then pop_state ();
635 ) else (
636 match current_state () with
637 | State.Normal ->
638   if ch = ';' then (
639     push_state State.Plain_comment;
640     incr i; incr col
641   ) else if ch = '(' && !i + 1 < len && text.[!i + 1] = '*' then (
642     push_state State.Annotated_comment;
643     seen_non_ws := false;
644     i := !i + 2; col := !col + 2
645   ) else if ch = '(' && !i + 1 < len && text.[!i + 1] = '|' then (
646     push_state (State.Verbatim 1);
647     seen_non_ws := false;
648     i := !i + 2; col := !col + 2
649   ) else if ch = '"' then (
650     push_state State.Quoted_string;
651     seen_non_ws := false;
652     incr i; incr col
653   ) else if ch = '(' then (
654     if !current_open = None then (
655       current_open := Some !col;
656       current_keyword := peek_keyword text (!i + 1) len;
657     );
658     seen_non_ws := true;
659     incr i; incr col
660   ) else if ch = ')' then (
661     if not !seen_non_ws then (
662       if !current_structural_close = None then
663         current_structural_close := Some !col
664     ) else (
665       incr current_inline_closes
666     );
667     seen_non_ws := true;
668     incr i; incr col
669   ) else (
670     if ch <> ' ' && ch <> '\t' then seen_non_ws := true;
671     incr i; incr col
672   )
673
674 | State.Plain_comment ->
675   incr i; incr col
676
677 | State.Annotated_comment ->
678   if ch = '*' && !i + 1 < len && text.[!i + 1] = ')' then (
679     pop_state ();
680     seen_non_ws := true;
681     i := !i + 2; col := !col + 2
682   ) else (
683     incr i; incr col
684   )
685
686 | State.Quoted_string ->
687   if ch = '\\ ' && !i + 1 < len then (
688     i := !i + 2; col := !col + 2
689   ) else if ch = '"' then (
690     pop_state ();
691     seen_non_ws := true;
692     incr i; incr col
693   ) else (
694     incr i; incr col
695   )
696
697 | State.Verbatim depth ->
698   if ch = '|' && !i + 1 < len && text.[!i + 1] = '|' then (
699     push_state (State.Verbatim (depth + 1));
700     i := !i + 2; col := !col + 2
701   ) else if ch = '|' && !i + 1 < len && text.[!i + 1] = ')' then (
702     pop_state ();

```



```

703         seen_non_ws := true;
704         i := !i + 2; col := !col + 2
705     ) else if ch = '<' && !i + 1 < len && text.[!i + 1] = '<' then (
706         push_state (State.Inline_example 1);
707         i := !i + 2; col := !col + 2
708     ) else (
709         incr i; incr col
710     )
711
712     | State.Inline_example depth ->
713         if ch = '<' && !i + 1 < len && text.[!i + 1] = '<' then (
714             push_state (State.Inline_example (depth + 1));
715             i := !i + 2; col := !col + 2
716         ) else if ch = '>' && !i + 1 < len && text.[!i + 1] = '>' then (
717             pop_state ();
718             seen_non_ws := true;
719             i := !i + 2; col := !col + 2
720         ) else (
721             incr i; incr col
722         )
723     )
724 done;
725 (* Only flush final event if there is actual content -- prevents
726    a phantom empty line from stealing EOF repair actions. *)
727 if !current_open <> None || !current_structural_close <> None ||
728    !current_inline_closes > 0 then
729     flush_event ();
730 List.rev !events
731
732 type indent_form = {
733     if_line : int;
734     if_col : int;
735     if_keyword : string option;
736 }
737
738 (** Build repair actions from indentation-based tree inference.
739     Algorithm:
740     - Each line with an open-paren at col C starts a form.
741     - Before opening a new form at col C, close all forms with col > C.
742     - Structural ')' must match its opener's column; mismatches are trimmed.
743     - Inline ')' pops from the stack normally.
744     - At EOF, close everything remaining with properly indented new lines.
745 *)
746 let compute_repair_actions (events : line_event list) =
747     let stack : indent_form list ref = ref [] in
748     let actions : repair_action list ref = ref [] in
749
750     let insert_closes before_line forms =
751         (* Insert from innermost (highest indent) to outermost *)
752         let sorted = List.sort (fun a b -> compare b.if_col a.if_col) forms in
753         List.iter (fun form ->
754             actions := Insert_line { line = before_line - 1; indent = form.if_col;
755                 reason = Printf.sprintf "close %s opened at line %d" (match
756                     form.if_keyword with Some k -> k | None -> "form") form.if_line }
757             :: !actions
758         ) sorted
759     in
760
761     let close_forms target_col new_line =
762         let rec aux acc =
763             match !stack with
764             | [] -> acc
765             | top :: rest when top.if_col >= target_col ->
766                 stack := rest;
767                 aux (top :: acc)
768             | _ -> acc
769         in
770         let to_close = aux [] in
771         if to_close <> [] then insert_closes new_line to_close
772     in
773
774     List.iter (fun ev ->
775         (* Handle open first: close deeper forms, then push new form *)

```

```

774 (match ev.ev_open_col with
775 | Some c ->
776     close_forms c ev.ev_line;
777     stack := { if_line = ev.ev_line; if_col = c; if_keyword =
778               ev.ev_keyword } :: !stack
779 | None -> ());
780
781 (* Handle inline closes: pop from stack *)
782 if ev.ev_inline_close_count > 0 then (
783     let rec pop count =
784         if count <= 0 then ()
785         else match !stack with
786             | [] ->
787                 actions := Trim_end { line = ev.ev_line; count = count;
788                                       reason = "unexpected ' ' -- no matching open-paren" } ::
789                 !actions
790             | _ :: rest ->
791                 stack := rest;
792                 pop (count - 1)
793     in
794     pop ev.ev_inline_close_count
795 );
796
797 (* Handle structural close: must match top of stack *)
798 (match ev.ev_structural_close_col with
799 | Some c ->
800     (match !stack with
801     | top :: rest when top.if_col = c ->
802         stack := rest
803     | _ ->
804         (* Mismatched structural close or unexpected -- trim it *)
805         actions := Trim_end { line = ev.ev_line; count = 1;
806                               reason = "structural ' ' does not match open-paren
807                               indentation" } :: !actions
808         )
809 | None -> ());
810 );
811
812 ) events;
813
814 (* EOF: close any remaining *)
815 (match !stack with
816 | [] -> (
817 | remaining ->
818     let last_ev = match List.rev events with e::_ -> e.ev_line | [] -> 1
819     in
820     insert_closes (last_ev + 1) remaining;
821     stack := []
822 );
823
824 List.rev !actions
825
826 (** Apply repair actions to text, producing the corrected file. *)
827 let apply_repair_actions text actions =
828     let raw_lines = String.split_on_char '\n' text in
829     let lines, has_trailing_newline =
830         match List.rev raw_lines with
831         | "" :: rest -> List.rev rest, true
832         | _ -> raw_lines, false
833     in
834     let lines = Array.of_list lines in
835
836     (* Group actions by line *)
837     let inserts = Hashtbl.create 16 in
838     let trims = Hashtbl.create 16 in
839     List.iter (function
840         | Insert_line { line; indent; reason } ->
841             let existing = try Hashtbl.find inserts line with Not_found -> [] in
842             Hashtbl.replace inserts line ((indent, reason) :: existing)
843         | Trim_end { line; count; reason = _ } ->
844             let existing = try Hashtbl.find trims line with Not_found -> 0 in
845             Hashtbl.replace trims line (existing + count)
846     ) actions;
847

```

```

843 let result = ref [] in
844 for i = 0 to Array.length lines - 1 do
845   let line_num = i + 1 in
846   let line_text = lines.(i) in
847
848   (* Apply trim to current line *)
849   let trim_count = try Hashtbl.find trims line_num with Not_found -> 0 in
850   let trimmed =
851     let rec trim s n =
852       if n <= 0 then s
853       else
854         let len = String.length s in
855         if len > 0 && s.[len - 1] = ' ' then trim (String.sub s 0 (len -
1)) (n - 1)
856       else s
857     in
858     trim line_text trim_count
859   in
860   result := trimmed :: !result;
861
862   (* Insert new lines after this line *)
863   match try Some (Hashtbl.find inserts line_num) with Not_found -> None
with
864   | Some items ->
865     (* Sort by indent descending so innermost closes appear first *)
866     let sorted = List.sort (fun (a, _) (b, _) -> compare b a) items in
867     List.iter (fun (indent, _) ->
868       result := (String.make (indent - 1) ' ' ^ ")") :: !result
869     ) sorted
870   | None -> ()
871 done;
872
873 let text = String.concat "\n" (List.rev !result) in
874 if has_trailing_newline then text ^ "\n" else text
875
876 let repair_text =
877   let events = scan_line_events text in
878   let actions = compute_repair_actions events in
879   apply_repair_actions text actions
880
881 (** Enriched analysis with structural tree and repair actions. *)
882 type structural_node = {
883   node_line : int;
884   node_depth : int;
885   node_keyword : string option;
886   node_open_col : int;
887   node_closed : bool;
888   node_close_line : int option;
889 }
890
891 type enriched_analysis = {
892   enriched_result : check_result;
893   enriched_pairs : paren_pair list;
894   enriched_divergences : indent_divergence list;
895   enriched_tree : structural_node list;
896   enriched_repair_actions : repair_action list;
897 }
898
899 let analyze_enriched text =
900   let analysis = analyze_text text in
901   let tree =
902     List.map (fun (p : paren_pair) ->
903       { node_line = p.open_pos.line;
904         node_depth = p.depth;
905         node_keyword = p.keyword;
906         node_open_col = p.open_pos.col;
907         node_closed = p.close_pos <> None;
908         node_close_line = Option.map (fun (c : pos) -> c.line) p.close_pos;
909       }
910     ) analysis.pairs
911   in
912   let divergences = find_divergences analysis.pairs in
913   let actions =

```

```

914     match analysis.result with
915     | Balanced _ -> []
916     | Imbalanced _ ->
917         let events = scan_line_events text in
918         compute_repair_actions events
919   in
920   { enriched_result = analysis.result;
921     enriched_pairs = analysis.pairs;
922     enriched_divergences = divergences;
923     enriched_tree = tree;
924     enriched_repair_actions = actions;
925   }
926
927   let excerpt text start_line end_line =
928     let lines = String.split_on_char '\n' text in
929     let total = List.length lines in
930     let start_line = max 1 (min start_line total) in
931     let end_line = max 1 (min end_line total) in
932     let indexed = List.mapi (fun i l -> (i + 1, l)) lines in
933     List.filter_map (fun (i, line) ->
934       if i >= start_line && i <= end_line then
935         Some (Printf.sprintf "%3d | %s" i line)
936       else None
937     ) indexed
938
939   (** Print detailed report for a file *)
940   let report_file ?(verbose=false) path =
941     (* exempt: open_in *)
942     let ic = open_in path in
943     try
944       let n = in_channel_length ic in
945       let buf = Bytes.create n in
946       really_input ic buf 0 n;
947       close_in ic;
948       let text = Bytes.to_string buf in
949       let analysis = analyze text in
950       match analysis.result with
951       | Balanced { max_depth } ->
952         Printf.printf "? %s -- balanced (max nesting depth %d)\n" path
          max_depth;
953         true
954       | Imbalanced errs ->
955         Printf.printf "? %s -- %d issue(s):\n" path (List.length errs);
956         List.iter (fun err ->
957           Printf.printf " ? %s\n" (string_of_error err);
958           if verbose then begin
959             match err with
960             | Unclosed_parens frames ->
961               print_string (string_of_paren_stack frames)
962             | _ -> ()
963           end;
964         ) errs;
965         Printf.printf "\n Run 'borge balance --repair %s' to auto-fix.\n"
          path;
966         false
967     with e ->
968       close_in ic;
969       raise e

```

12 lib/lang/error.ml

```
1 type parse_error = {
2   message : string;
3   line : int;
4   column : int;
5 }
6
7 exception Parse_error of parse_error
8
9 (* agent note (|
10  *   WHAT: Raise a parse error with the given line, column, and
11  *   formatted message. Uses Format.kasprintf for printf-style formatting.
12  *
13  *   WHY: Parse errors need to be reported with location information
14  *   so users know where in the file the problem occurred.
15  * |) *)
16 let error ~line ~column fmt =
17   Format.kasprintf
18     (fun message -> raise (Parse_error { message; line; column }))
19     fmt
```

13 lib/lang/lexer.ml

```

1  (** Sedlex-based lexer for .borg sexp files.
2
3      Uses Sedlexing for position tracking and menhir integration,
4      but does character-by-character reading via Sedlexing.next
5      instead of the %sedlex PPX. This gives us proper positions
6      without the PPX syntax complexity.
7
8      Produces the token types below. Handles nested verbatim strings
9      via depth tracking. *)
10
11 type token =
12 | LPAREN      (* open paren *)
13 | RPAREN      (* close paren *)
14 | HASH_LPAREN (* annotated comment open *)
15 | STAR_RPAREN (* annotated comment close *)
16 | VERB of string (* verbatim string with content, (| already consumed *)
17 | QUOTED of string (* quoted string, escapes resolved *)
18 | SYMBOL of string (* bare word *)
19 | SEMICOLON of string (* plain comment to EOL *)
20 | EOF
21
22 type position = {
23   line : int;
24   column : int;
25 }
26
27 type located_token = {
28   token : token;
29   start : position;
30 }
31
32 (* agent note (|
33  * WHAT: Check if a character is valid in a borge symbol (bare word).
34  * Includes alphanumerics, dash, underscore, dot, question, exclamation.
35  *
36  * WHY: Symbol parsing needs to know where words end to correctly
37  * tokenize the input. This is the character class for valid symbols.
38  * |) *)
39 let is_symbol_char = function
40 | 'a'..'z' | 'A'..'Z' | '0'..'9' | '-' | '_' | '.' | '?' | '!' -> true
41 | _ -> false
42
43 (* agent note (|
44  * WHAT: Check if a character is whitespace (space, tab, newline, return).
45  *
46  * WHY: The lexer skips whitespace between tokens. This is the
47  * character class for whitespace.
48  * |) *)
49 let is_whitespace = function
50 | ' ' | '\t' | '\n' | '\r' -> true
51 | _ -> false
52
53 (** Peek at the next character without consuming it.
54     Sedlexing.rollback rolls back to the last start/mark checkpoint,
55     so we must call start before peeking to set a checkpoint,
56     then rollback to restore. *)
57 let peek lexbuf =
58   Sedlexing.start lexbuf;
59   match Sedlexing.next lexbuf with
60   | None -> None
61   | Some c ->
62     Sedlexing.rollback lexbuf;
63     Some (Uchar.to_char c)
64
65 (** Read next character, consuming it. *)
66 let next_char lexbuf =
67   match Sedlexing.next lexbuf with
68   | None -> None
69   | Some c -> Some (Uchar.to_char c)
70
71 (** Read a quoted string, resolving escapes. Opening quote consumed. *)

```

```

72 let read_quoted_contents lexbuf =
73   let buf = Buffer.create 64 in
74   let rec loop () =
75     match next_char lexbuf with
76     | None -> failwith "Unterminated quoted string"
77     | Some '"' -> Buffer.contents buf
78     | Some '\\' -> begin
79       match next_char lexbuf with
80       | None -> failwith "Unterminated quoted string escape"
81       | Some 'n' -> Buffer.add_char buf '\n'; loop ()
82       | Some 't' -> Buffer.add_char buf '\t'; loop ()
83       | Some '\\' -> Buffer.add_char buf '\\'; loop ()
84       | Some '"' -> Buffer.add_char buf '"'; loop ()
85       | Some 'r' -> Buffer.add_char buf '\r'; loop ()
86       | Some c -> Buffer.add_char buf c; loop ()
87     end
88   | Some c -> Buffer.add_char buf c; loop ()
89   in
90   loop ()
91
92 (** Read a plain comment to EOL. Opening semicolon consumed. *)
93 let read_plain_comment lexbuf =
94   let buf = Buffer.create 64 in
95   let rec loop () =
96     match next_char lexbuf with
97     | None | Some '\n' -> Buffer.contents buf
98     | Some c -> Buffer.add_char buf c; loop ()
99   in
100   loop ()
101
102 (** Read a symbol. *)
103 let read_symbol lexbuf first_char =
104   let buf = Buffer.create 32 in
105   Buffer.add_char buf first_char;
106   let rec loop () =
107     match peek lexbuf with
108     | Some c when is_symbol_char c ->
109       ignore (next_char lexbuf);
110       Buffer.add_char buf c; loop ()
111     | _ -> Buffer.contents buf
112   in
113   loop ()
114
115 (** Read verbatim string content. Opening (| consumed, closing |) consumed.
116     Tracks nesting depth. Returns the content between delimiters. *)
117 let read_verbatim_contents lexbuf =
118   let buf = Buffer.create 128 in
119   let depth = ref 1 in
120   let rec loop () =
121     match next_char lexbuf with
122     | None -> failwith "Unterminated verbatim string"
123     | Some '(' -> begin
124       match peek lexbuf with
125       | Some '|' ->
126         ignore (next_char lexbuf);
127         depth := !depth + 1;
128         Buffer.add_string buf "(|";
129         loop ()
130       | _ -> Buffer.add_char buf '('; loop ()
131     end
132   | Some '|' -> begin
133     match peek lexbuf with
134     | Some ')' ->
135       ignore (next_char lexbuf);
136       depth := !depth - 1;
137       if !depth > 0 then begin
138         Buffer.add_string buf "|)";
139         loop ()
140       end else
141         Buffer.contents buf
142     | _ -> Buffer.add_char buf '|'; loop ()
143   end
144   | Some c -> Buffer.add_char buf c; loop ()

```

```
145   in
146   loop ()
147
148   (** Main token reader. Skips whitespace between tokens. *)
149   let rec read_lexbuf =
150     match next_char lexbuf with
151     | None -> EOF
152     | Some ' ' | Some '\t' | Some '\n' | Some '\r' -> read_lexbuf
153     | Some '(' -> begin
154       match peek_lexbuf with
155       | Some '*' -> ignore (next_char lexbuf); HASH_LPAREN
156       | Some '|' -> ignore (next_char lexbuf); VERB (read_verbatim_contents
157         lexbuf)
158       | _ -> LPAREN
159     end
160     | Some ')' -> RPAREN
161     | Some '*' -> begin
162       match peek_lexbuf with
163       | Some ')' -> ignore (next_char lexbuf); STAR_RPAREN
164       | _ -> read_lexbuf (* stray *, skip *)
165     end
166     | Some '"' -> QUOTED (read_quoted_contents lexbuf)
167     | Some ';' -> SEMICOLON (read_plain_comment lexbuf)
168     | Some c when is_symbol_char c -> SYMBOL (read_symbol lexbuf c)
169     | Some _ -> read_lexbuf (* skip unknown chars *)
```


14 lib/lang/parse.ml

```

1  (** Parse .borg files using the menhir-generated parser and sedlex lexer. *)
2
3  open Ast [@@warning "-33"]
4
5  (** Create a Sedlexing.lexbuf from a string and set up position tracking. *)
6  let make_lexbuf input =
7    let lexbuf = Sedlexing.Latin1.from_string input in
8    Sedlexing.set_position lexbuf
9    { Lexing.pos_fname = "<input>";
10      pos_lnum = 1;
11      pos_bol = 0;
12      pos_cnum = 0 };
13    lexbuf
14
15  (** Adapt the sedlex lexer to work with menhir's expected interface.
16     Menhir expects a Lexing.lexbuf -> token function, but our lexer
17     uses Sedlexing.lexbuf. We wrap it. *)
18  let read_token (lexbuf : Sedlexing.lexbuf) : Parser.token =
19    let tok = Lexer.read lexbuf in
20    (* Convert Lexer.token to Parser.token -- they're the same types
21       since Parser.token is generated from the %token declarations
22       which match Lexer.token. But they're in different modules. *)
23    match tok with
24    | Lexer.LPAREN -> LPAREN
25    | Lexer.RPAREN -> RPAREN
26    | Lexer.HASH_LPAREN -> HASH_LPAREN
27    | Lexer.STAR_LPAREN -> STAR_LPAREN
28    | Lexer.VERB s -> VERB s
29    | Lexer.QUOTED s -> QUOTED s
30    | Lexer.SYMBOL s -> SYMBOL s
31    | Lexer.SEMICOLON s -> SEMICOLON s
32    | Lexer.EOF -> EOF
33
34  (** Bridge: menhir expects a Lexing.lexbuf, but our lexer uses Sedlexing.
35     We adapt tokens and copy Sedlexing positions into the dummy Lexing
36     lexbuf so menhir can report accurate $startpos/$endpos/$loc. *)
37  let parse_file input =
38    let sedlex_lexbuf = make_lexbuf input in
39    reset_nested_comments ();
40    let token_reader (lb : Lexing.lexbuf) =
41      let tok = read_token sedlex_lexbuf in
42      let start_p, end_p = Sedlexing.lexing_positions sedlex_lexbuf in
43      lb.Lexing.lex_start_p <- start_p;
44      lb.Lexing.lex_curr_p <- end_p;
45      tok
46    in
47    let dummy_lb = Lexing.from_string "" in
48    try
49      Parser.file token_reader dummy_lb
50    with
51    | Parser.Error ->
52      let pos, _ = Sedlexing.lexing_positions sedlex_lexbuf in
53      Error.error ~line:pos.Lexing.pos_lnum ~column:(pos.Lexing.pos_cnum -
54        pos.Lexing.pos_bol)
55        "Parse error"
56
57  (* agent note [5200] (|
58   * WHAT: Parse a borge file from a string input, returning the
59   * AST or raising a parse error.
60   *
61   * WHY: Convenience wrapper around parse_file for when input is
62   * already in memory as a string.
63   * |) *)
64  let parse input =
65    parse_file input

```

15 lib/lang/print.ml

```

1 open Ast
2
3 (* agent note [5200] (|
4  *   WHAT: Convert a quoted string AST node to its escaped
5  *   representation suitable for re-parsing.
6  *
7  *   WHY: When pretty-printing or re-serializing AST, quoted strings
8  *   need proper escape sequences for newlines, tabs, quotes, backslashes.
9  * |) *)
10 let string_of_quoted q =
11   let buf = Buffer.create (String.length q.q_content + 8) in
12   Buffer.add_char buf '"';
13   String.iter (fun ch ->
14     match ch with
15     | '\n' -> Buffer.add_string buf "\\n"
16     | '\t' -> Buffer.add_string buf "\\t"
17     | '\\' -> Buffer.add_string buf "\\\"
18     | '"' -> Buffer.add_string buf "\\\""
19     | c -> Buffer.add_char buf c
20   ) q.q_content;
21   Buffer.add_char buf '"';
22   Buffer.contents buf
23
24 (* agent note (|
25  *   WHAT: Convert a verbatim string AST node to its representation
26  *   in the (|content|) format.
27  *
28  *   WHY: Verbatim strings are printed with their delimiters for
29  *   round-tripping through the parser.
30  * |) *)
31 let string_of_verbatim v =
32   Printf.sprintf "(|%s|)" v.v_content
33
34 (* agent note (|
35  *   WHAT: Dispatch to the appropriate string converter based on
36  *   whether the value is Quoted or Verbatim.
37  *
38  *   WHY: String values in borge can be either quoted or verbatim.
39  *   This function handles both uniformly.
40  * |) *)
41 let string_of_string_value = function
42   | Quoted q -> string_of_quoted q
43   | Verbatim v -> string_of_verbatim v
44
45 (* exempt doc *)
46 and spaces n = String.make n ' '
47
48 (* agent note (|
49  *   WHAT: Convert a plain comment to its string representation
50  *   with semicolon prefix.
51  *
52  *   WHY: Plain comments are printed as ; comment for round-tripping.
53  * |) *)
54 let print_plain_comment c =
55   Printf.sprintf ";%s" c.text
56
57 (* agent note (|
58  *   WHAT: Convert an annotated comment to its borg representation
59  *   in the (* author type (|content|) *) format.
60  *
61  *   WHY: Annotated comments carry author and type metadata that must
62  *   be preserved for lint integrity checking.
63  * |) *)
64 let print_annotated_comment c =
65   let author_str = match c.authorship with
66     | Single a -> a
67     | Multiple auths -> Printf.sprintf "(%s)" (String.concat " " auths)
68   in
69   let type_str = match c.comment_type with
70     | Untyped -> ""
71     | Typed t -> Printf.sprintf "%s" t

```

```

72   in
73   let value_str = match c.value with
74   | None -> "()"
75   | Some sv -> string_of_string_value sv
76   in
77   Printf.sprintf "(* %s%s %s *)" author_str type_str value_str
78
79 (* agent note (|
80  *   WHAT: Dispatch to the appropriate printer based on comment type.
81  *
82  *   WHY: Comments can be plain or annotated, each with different
83  *   serialization formats.
84  * |) *)
85 let print_comment_attachment = function
86 | Plain c -> print_plain_comment c
87 | Annotated c -> print_annotated_comment c
88
89 let drain_comments indent buf pending offset_limit =
90   let rec loop () =
91     match !pending with
92     | (cpos, c) :: rest when cpos.offset < offset_limit ->
93       Buffer.add_char buf '\n';
94       Buffer.add_string buf (spaces (indent + 1));
95       Buffer.add_string buf (print_comment_attachment c);
96       pending := rest;
97       loop ()
98     | _ -> ()
99   in
100   loop ()
101
102 (* agent note (|
103  *   WHAT: Recursively pretty-print a single sexp with indentation.
104  *   Handles atoms, strings, and nested lists with proper formatting.
105  *
106  *   WHY: Core pretty-printing for the AST. Formats lists with keywords
107  *   differently from plain lists, adding newlines for readability.
108  * |) *)
109 let rec print_sexp indent buf pending sexp =
110   match sexp with
111   | Atom (_, sym) ->
112     Buffer.add_string buf sym
113   | String (_, sv) ->
114     Buffer.add_string buf (string_of_string_value sv)
115   | List (_, []) ->
116     Buffer.add_string buf "()"
117   | List (_, children) ->
118     if children = [] then Buffer.add_string buf "()"
119     else begin
120       let keyword =
121         match children with
122         | [Atom (_, "doc") | Atom (_, "status"); _] -> ""
123         | Atom (_, k) :: _ -> k
124         | _ -> ""
125       in
126       let has_keyword = keyword <> "" && keyword <> "doc" && keyword <>
"status" in
127       if has_keyword then begin
128         Buffer.add_char buf '(';
129         Buffer.add_string buf keyword;
130         let rest = match children with _ :: tl -> tl | [] -> [] in
131         (* First child is the name -- keep it inline *)
132         (* Only keep Atom names inline; lists go on their own line *)
133         let further = match rest with
134         | [] -> []
135         | [Atom (_, n)] ->
136           Buffer.add_string buf " ";
137           Buffer.add_string buf n; []
138         | Atom (_, n) :: tl ->
139           Buffer.add_string buf " ";
140           Buffer.add_string buf n; tl
141         | _ -> rest
142         in
143         further |> List.iter (fun child ->

```

```

144     drain_comments indent buf pending (Ast.start_pos_of
child).offset;
145     Buffer.add_char buf '\n';
146     Buffer.add_string buf (spaces (indent + 1));
147     print_sexp (indent + 1) buf pending child
148   );
149   drain_comments indent buf pending (Ast.end_pos_of sexp).offset;
150   Buffer.add_char buf '\n';
151   Buffer.add_string buf (spaces indent);
152   Buffer.add_char buf ')'
153 end else begin
154   match children with
155   | [Atom (_, "doc"); String (_, v)] ->
156     Buffer.add_string buf "(doc ";
157     Buffer.add_string buf (string_of_string_value v);
158     Buffer.add_char buf ')'
159   | [Atom (_, "status"); Atom (_, v)] ->
160     Buffer.add_string buf "(status ";
161     Buffer.add_string buf v;
162     Buffer.add_char buf ')'
163   | hd :: tl ->
164     Buffer.add_char buf '(';
165     drain_comments indent buf pending (Ast.start_pos_of
hd).offset;
166     print_sexp (indent + 1) buf pending hd;
167     tl |> List.iter (fun child ->
168       drain_comments indent buf pending (Ast.start_pos_of
child).offset;
169       Buffer.add_char buf '\n';
170       Buffer.add_string buf (spaces (indent + 1));
171       print_sexp (indent + 1) buf pending child
172     );
173     drain_comments indent buf pending (Ast.end_pos_of
sexp).offset;
174     Buffer.add_char buf '\n';
175     Buffer.add_string buf (spaces indent);
176     Buffer.add_char buf ')'
177   | _ -> Buffer.add_string buf "()"
178 end
179 end
180
181 (* agent note (|
182  * WHAT: Pretty-print an entire borge AST to a string with
183  * proper formatting and indentation.
184  *
185  * WHY: The primary output function for serializing parsed borge
186  * files back to text with readable formatting.
187  * |) *)
188 let print_file f =
189   let buf = Buffer.create 4096 in
190   let sorted_nested =
191     List.stable_sort (fun (p1, _) (p2, _) -> compare p1.offset p2.offset)
192     f.nested_comments
193   in
194   let pending = ref sorted_nested in
195   List.iter (fun c ->
196     Buffer.add_string buf (print_comment_attachment c);
197     Buffer.add_char buf '\n'
198   ) f.top_level_comments;
199
200   List.iter (fun { comments_before; node; _ } ->
201     List.iter (fun c ->
202       Buffer.add_string buf (print_comment_attachment c);
203       Buffer.add_char buf '\n'
204     ) comments_before;
205     print_sexp 0 buf pending node;
206     Buffer.add_char buf '\n'
207   ) f.top_level;
208
209   List.iter (fun c ->
210     Buffer.add_string buf (print_comment_attachment c);
211     Buffer.add_char buf '\n'

```

```
212   ) f.trailing_comments;  
213  
214   (* Defensive drain for any nested comment that was not emitted  
215      inside a list. By construction this should be empty. *)  
216   drain_comments 0 buf pending max_int;  
217  
218   Buffer.contents buf
```

16 examples/crud-app/db.borg

crud-app-db

Example DB spec for a task tracker

17 examples/crud-app/generated.ml

```

1 (* Generated by borge generate --target ocaml-dream *)
2 (* Source: examples/crud-app/ui.borg *)
3 (* DO NOT EDIT: regenerate from spec *)
4
5 open Dream_html
6
7 let theme_css = "<style>\n:root {\n  --success: #22c55e;\n  --danger:
  #ef4444;\n  --border: #e2e8f0;\n  --accent-hover: #2563eb;\n
  --bg-sidebar: #1e293b;\n  --accent: #3b82f6;\n  --bg: #f8f9fa;\n
  --text-primary: #1e293b;\n  --bg-surface: #ffffff;\n  --text-inverse:
  #ffffff;\n  --text-secondary: #64748b;\n  --spacing-lg: 24px;\n
  --spacing-md: 16px;\n  --spacing-xl: 32px;\n  --spacing-sm: 8px;\n
  --spacing-xs: 4px;\n  --font-size-base: 14px;\n  --font-size-lg:
  18px;\n  --font-size-xl: 24px;\n  --font-size-sm: 12px;\n
  --radius-lg: 8px;\n  --radius-md: 6px;\n  --radius-sm:
  4px;\n}\n</style>\n"
8
9 (* Variant media queries *)
10
11 let variant_css = "@media (max-width: 767px) {\n  .button.variant-danger {
  background-color: #ef4444 }\n}\n\n@media (max-width: 767px) {\n
  .button.variant-outline { background-color: #ffffff; color: #3b82f6
  }\n}\n\n@media (max-width: 767px) {\n  .nav_item.variant-active {
  background-color: #3b82f6 }\n}\n"
12
13 (* Components *)
14
15 let render_button ~(content : Html.node list) () : Html.node =
16   Html.div ~a:[Html.a_style "background-color: #3b82f6; border-radius: 6px;
   color: #ffffff"; Html.a_on_click (fun _ -> Dream.redirect "TODO")] []
17
18 let render_card ~(content : Html.node list) () : Html.node =
19   Html.div ~a:[Html.a_style "background-color: #ffffff; border-radius: 8px;
   padding: 16px"] []
20
21 let render_input_field ~(content : Html.node list) () : Html.node =
22   Html.div ~a:[Html.a_style "background-color: #ffffff; border-radius: 6px;
   width: 100%"; Html.a_on_click (fun _ -> Dream.redirect "TODO")] []
23
24 let render_nav_item ~(content : Html.node list) () : Html.node =
25   Html.div ~a:[Html.a_style "border-radius: 4px; color: #ffffff";
   Html.a_on_click (fun _ -> Dream.redirect "TODO")] []
26
27 (* Layouts *)
28
29 let render_app_shell_layout ~(content : Html.node list) () : Html.node =
30   Html.div ~a:[Html.a_class ["flex-row"]; Html.a_style "height: 100%;
   width: 100%"] [Html.div ~a:[Html.a_class ["flex-col"]; Html.a_style
   "background-color: #1e293b; gap: 8px; height: 100%; padding: 16px;
   width: 240px"] []; Html.div ~a:[Html.a_class ["flex-col"];
   Html.a_style "background-color: #f8f9fa; flex: 1 1 0%; gap: 16px;
   height: 100%; overflow-y: auto; padding: 24px"] []]
31
32 let render_sidebar_layout ~(content : Html.node list) () : Html.node
   =
33   Html.div ~a:[Html.a_class ["flex-col"]; Html.a_style "background-color:
   #1e293b; gap: 8px; height: 100%; padding: 16px; width: 100%"]
   [Html.div ~a:[Html.a_style "padding: 16px"] []; Html.div [] [];
   Html.div ~a:[Html.a_style "flex: 1 1 0%"] []; Html.div
   ~a:[Html.a_style "color: #ffffff; padding: 16px"] []]
34
35 (* Pages *)
36
37 let render_projects () : Html.node =
38   render_app_shell_layout ~content:[; ] ()
39
40 let render_tasks () : Html.node =
41   render_app_shell_layout ~content:[; ] ()
42
43 let render_task_detail () : Html.node =
44   render_app_shell_layout ~content:[; ] ()

```

```
45
46 (* Routes *)
47
48 let register_routes () = Dream.router [
49   Dream.get "/" (fun _ -> Dream_html.respond (render_projects ()));
50   Dream.get "/tasks" (fun _ -> Dream_html.respond (render_tasks ()));
51   Dream.get "/tasks/:id" (fun _ -> Dream_html.respond (render_task_detail
52     ()));
53 ]
```


18 examples/crud-app/ui.borg

crud-app-ui

Example UI spec for a task tracker

19 lib/core/file_utils.ml

```

1  (*| Shared file utilities used by all borge commands.
2
3      All functions are exception-safe: channels are closed even on error.
4  |*)
5
6  (** Read an entire file into a single string.
7      Raises Sys_error if the file cannot be opened. *)
8  let read_file path =
9      (* exempt: open_in *)
10     In_channel.with_open_text path (fun ic ->
11         let len = in_channel_length ic in
12         let buf = Bytes.create len in
13         really_input ic buf 0 len;
14         Bytes.to_string buf)
15
16  (** Read a file into a list of lines (without trailing newlines).
17      Returns [] if the file cannot be read. *)
18  let read_lines path =
19      try
20          (* exempt: input_line *)
21          In_channel.with_open_text path (fun ic ->
22              let rec loop acc =
23                  match In_channel.input_line ic with
24                  | Some line -> loop (line :: acc)
25                  | None -> List.rev acc
26              in
27              loop [])
28          with Sys_error _ -> []
29
30  (* agent note (|
31  *   WHAT: Recursively find all .borg files under a directory,
32  *   skipping _build and .git directories, excluding .borg.meta.
33  *   WHY: Used by lint, check, and drift to find all spec files.
34  * |) *)
35  let rec find_borg_files dir =
36      try
37          let entries = Sys.readdir dir in
38          Array.fold_left (fun acc name ->
39              if name = "_build" || name = ".git" then acc
40              else
41                  let path = Filename.concat dir name in
42                  if Sys.is_directory path then find_borg_files path @ acc
43                  else if Filename.check_suffix name ".borg" then begin
44                      (* Skip .borg.meta files -- they are machine-generated
45                       observations, not specs *)
46                      let meta_suffix = ".borg.meta" in
47                      let meta_len = String.length meta_suffix in
48                      if String.length name >= meta_len &&
49                         String.sub name (String.length name - meta_len) meta_len =
50                         meta_suffix
51                      then acc
52                      else path :: acc
53                  end
54                  else acc
55              ) [] entries
56          with Sys_error _ -> []

```

20 lib/core/project.ml

```

1 type project_node = {
2   path : string;
3   project_name : string option;
4   children : project_node list;
5   is_orphan : bool;
6 }
7
8 type build_error =
9   | File_not_found of string
10  | Parse_error of string * string (** path, message *)
11  | Cycle_detected of string list  (** the cycle path *)
12
13 (** Build the inline tree starting from a root .borg file.
14     Recursively follows (inline filename.borg) directives.
15     Filenames are resolved relative to the directory of the parent file. *)
16 let rec build_tree ?(visited=[]) path =
17   if List.mem path visited then
18     Error (Cycle_detected (path :: visited))
19   else if not (Sys.file_exists path) then
20     Error (File_not_found path)
21   else
22     try
23       let input = File_utils.read_file path in
24       let file = Borge_lang.Parse.parse_file input in
25       let name = Spec.project_name file in
26       let targets = Spec.inline_targets file in
27       let dir = Filename.dirname path in
28       let children_with_errors =
29         List.map (fun filename ->
30           let child_path =
31             if Filename.is_relative filename then
32               Filename.concat dir filename
33             else
34               filename
35           in
36           build_tree ~visited:(path :: visited) child_path
37         ) targets
38     in
39     (* Collect errors from children *)
40     let first_error = List.find_map (function Error e -> Some e | _ ->
41       None) children_with_errors in
42     match first_error with
43     | Some e -> Error e
44     | None ->
45       let children = List.filter_map (function Ok n -> Some n | _ ->
46         None) children_with_errors in
47       Ok { path; project_name = name; children; is_orphan = false }
48     with Borge_lang.Error.Parse_error e ->
49       Error (Parse_error (path, Printf.sprintf "parse error at %d:%d - %s"
50         e.line e.column e.message))
51
52 (** Find all .borg files that are NOT inlined by any other file.
53     These are the "root" candidates. *)
54 let find_root_candidates dir =
55   let all_files = File_utils.find_borg_files dir in
56   let all_targets =
57     List.concat_map (fun path ->
58       try
59         let input = File_utils.read_file path in
60         let file = Borge_lang.Parse.parse_file input in
61         let dir = Filename.dirname path in
62         List.map (fun filename ->
63           if Filename.is_relative filename then
64             Filename.concat dir filename
65           else
66             filename
67         ) (Spec.inline_targets file)
68       with _ -> []
69     ) all_files
70   in
71   List.filter (fun path -> not (List.mem path all_targets)) all_files

```

```

69
70 (** Find the root .borg file in a directory.
71    - If there's exactly one file not inlined by any other, that's the root.
72    - If there are multiple candidates, return all of them. *)
73 let find_roots dir =
74   find_root_candidates dir
75
76 (** Find all orphaned .borg files: not inlined by any parent, no
77    (no-inline) declaration,
78    and doesn't inline any children. Files that inline others are
79    implicitly roots. *)
80 let find_orphans dir =
81   let roots = find_root_candidates dir in
82   List.filter (fun path ->
83     try
84       let input = File_utils.read_file path in
85       let file = Borge_lang.Parse.parse_file input in
86       let has_inlines = Spec.inline_targets file <> [] in
87       let declares_no_inline = Spec.has_no_inline file in
88       not has_inlines && not declares_no_inline
89     with _ -> false
90   ) roots
91
92 (** Collect all paths in the tree (including root) *)
93 let rec tree_paths node =
94   node.path :: List.concat_map tree_paths node.children
95
96 (** Total status counts across the tree (no double-counting) *)
97 let tree_status_counts node =
98   let visited = Hashtbl.create 16 in
99   let rec walk n =
100     if Hashtbl.mem visited n.path then ()
101     else begin
102       Hashtbl.add visited n.path true;
103       List.iter walk n.children
104     end
105   in
106   walk node;
107   let all_files = Hashtbl.fold (fun path _ acc -> path :: acc) visited [] in
108   let impl = ref 0 in
109   let veri = ref 0 in
110   let prog = ref 0 in
111   let plan = ref 0 in
112   List.iter (fun path ->
113     try
114       let input = File_utils.read_file path in
115       let file = Borge_lang.Parse.parse_file input in
116       let counts = Spec.status_counts file in
117       impl := !impl + (match List.assoc_opt Spec.Implemented counts with
118         Some v -> v | None -> 0);
119       veri := !veri + (match List.assoc_opt Spec.Verified counts with
120         Some v -> v | None -> 0);
121       prog := !prog + (match List.assoc_opt Spec.In_progress counts with
122         Some v -> v | None -> 0);
123       plan := !plan + (match List.assoc_opt Spec.Planned counts with
124         Some v -> v | None -> 0);
125     with _ -> ()
126   ) all_files;
127   (!impl, !veri, !prog, !plan)
128
129 (** Format the tree as indented text *)
130 let rec format_tree ?(indent=0) node =
131   let pad = String.make (indent * 2) ' ' in
132   let name_str = match node.project_name with
133   | Some n -> Printf.sprintf "%s (%s)" (Filename.basename node.path) n
134   | None -> Filename.basename node.path
135   in
136   let orphan_str = if node.is_orphan then " [ORPHAN]" else "" in
137   let line = Printf.sprintf "%s%s%s" pad name_str orphan_str in
138   line :: List.concat_map (format_tree ~indent:(indent + 1)) node.children

```

21 lib/core/spec.ml

```

1 open Borge_lang.Ast
2
3 type status =
4   | Planned
5   | In_progress
6   | Partial
7   | Implemented
8   | Verified
9   | Drifted
10  | Blank
11
12 (* (fn status_of_string
13     (doc "Convert string to status type")
14     (since "v1.0")) *)
15 let status_of_string = function
16   | "planned" -> Some Planned
17   | "in-progress" -> Some In_progress
18   | "partial" -> Some Partial
19   | "implemented" -> Some Implemented
20   | "verified" -> Some Verified
21   | "drifted" -> Some Drifted
22   | "blank" -> Some Blank
23   | _ -> None
24
25 (* (fn string_of_status
26     (doc "Convert status type back to string")
27     (since "v1.0")) *)
28 let string_of_status = function
29   | Planned -> "planned"
30   | In_progress -> "in-progress"
31   | Partial -> "partial"
32   | Implemented -> "implemented"
33   | Verified -> "verified"
34   | Drifted -> "drifted"
35   | Blank -> "blank"
36
37 (** Extract the project name from a top-level (project name ...) form *)
38 let project_name (file : Borge_lang.Ast.file) : string option =
39   let rec find = function
40     | [] -> None
41     | { node = List (_, Atom (_, "project")) :: Atom (_, name) :: _; _ } ::
42       _ -> Some name
43     | _ :: rest -> find rest
44   in
45   find file.top_level
46
47 (** Count sections in the file *)
48 let count_sections (file : Borge_lang.Ast.file) : int =
49   let rec count_in_sexp = function
50     | List (_, Atom (_, kind) :: _)
51       when List.mem kind ["section"; "subsection"; "subsubsection"] -> 1
52     | List (_, sexps) -> List.fold_left (fun acc s -> acc + count_in_sexp
53     s) 0 sexps
54     | _ -> 0
55   in
56   let rec count_in_node = function
57     | [] -> 0
58     | { node; _ } :: rest -> count_in_sexp node + count_in_node rest
59   in
60   count_in_node file.top_level
61
62 (** Extract all status values from the file *)
63 let statuses (file : Borge_lang.Ast.file) : status list =
64   let rec extract = function
65     | List (_, Atom (_, "status") :: Atom (_, s) :: _) ->
66       (match status_of_string s with
67        | Some st -> [st]
68        | None -> [])
69     | List (_, sexps) -> List.concat_map extract sexps
70     | _ -> []

```

```

70   in
71   let rec walk = function
72     | [] -> []
73     | { node; _ } :: rest -> extract node @ walk rest
74   in
75   walk file.top_level
76
77 let status_order = [Planned; In_progress; Partial; Implemented; Verified;
78   Drifted; Blank]
79
80 (* agent note [5200] (|
81  *   WHAT: Ordered list of status values for display and comparison.
82  *   Used as the canonical ordering when computing status counts.
83  *   WHY: Provides a consistent sort order for reports and ensures
84  *   all status types appear in output even if not present in a file.
85  * |) *)
86
87 (** Extract inline target filenames from (inline filename.borg) forms *)
88 let inline_targets (file : Borge_lang.Ast.file) : string list =
89   let rec extract = function
90     | List (_, Atom (_, "inline") :: Atom (_, filename) :: _) -> [filename]
91     | List (_, sexps) -> List.concat_map extract sexps
92     | _ -> []
93   in
94   let rec walk = function
95     | [] -> []
96     | { node; _ } :: rest -> extract node @ walk rest
97   in
98   walk file.top_level
99
100 (** Check if the file declares (no-inline) *)
101 let has_no_inline (file : Borge_lang.Ast.file) : bool =
102   let rec extract = function
103     | List (_, [Atom (_, "no-inline")]) -> true
104     | List (_, sexps) -> List.exists extract sexps
105     | _ -> false
106   in
107   let rec walk = function
108     | [] -> false
109     | { node; _ } :: rest -> extract node || walk rest
110   in
111   walk file.top_level
112
113 (* agent note (|
114  *   WHAT: Count occurrences of each status value in a .borg file.
115  *   Returns an ordered list of status-count pairs using status_order.
116  *   WHY: Used by borge report to show implementation progress and
117  *   by borge drift to detect sections marked implemented that aren't.
118  * |) *)
119
120 (** Find status value in a list of sexp children *)
121 let rec find_status : Borge_lang.Ast.sexp list -> status option = function
122   | [] -> None
123   | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_, "status") ::
124     Borge_lang.Ast.Atom (_, v) :: _) :: _ ->
125     status_of_string v
126   | _ :: rest -> find_status rest
127
128 (** Extract (implements ...) file paths from a list of sexp children.
129  Returns paths as strings. Multiple implements forms accumulate. *)
130 let extract_implements children =
131   let rec find_impls = function
132     | [] -> []
133     | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_, "implements") ::
134       paths) :: rest ->
135       let files = List.filter_map (function
136         | Borge_lang.Ast.Atom (_, p) -> Some p
137         | _ -> None
138       ) paths in
139       files @ find_impls rest
140   in
141   find_impls children

```

```

140 (** A single verify criterion: method name and arguments *)
141 type verify_item = {
142   method_ : string;
143   args : string list;
144 }
145
146 (** Extract (verify ...) items from section children.
147   Each child of verify is a method call: (method_name arg1 arg2 ...).
148   Returns list of verify_item records. *)
149 let extract_verify (children : Borge_lang.Ast.sexp list) : verify_item list
150   =
151   let rec find_verify = function
152     | [] -> []
153     | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_, "verify") :: items)
154       :: rest ->
155       let vs = List.filter_map (function
156         | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_, m) :: args) ->
157         let arg_strs = List.filter_map (function
158           | Borge_lang.Ast.Atom (_, a) -> Some a
159           | Borge_lang.Ast.String (_, v) ->
160             Some (Borge_lang.Print.string_of_string_value v)
161           | _ -> None
162         ) args in
163         Some { method_ = m; args = arg_strs }
164         | _ -> None
165       ) items in
166       vs @ find_verify rest
167     | _ :: rest -> find_verify rest
168   in
169   find_verify children
170
171 (** Section mapping: name, status, explicitly declared files, and verify
172   items *)
173 type section_mapping = {
174   name : string;
175   status : status option;
176   implements : string list;
177   verify : verify_item list;
178 }
179
180 (** Extract all section mappings from a file *)
181 let extract_section_mappings (file : Borge_lang.Ast.file) : section_mapping
182   list =
183   let rec walk_sexp acc = function
184     | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_, kind) ::
185       rest_children)
186     when List.mem kind ["subsection"; "section"; "subsubsection"] ->
187     let name = match rest_children with
188       | Borge_lang.Ast.Atom (_, n) :: _ -> Some n
189       | _ -> None
190     in
191     let status = find_status rest_children in
192     let impls = extract_implements rest_children in
193     let verif = extract_verify rest_children in
194     let acc' = match name with
195       | Some n -> { name = n; status; implements = impls; verify =
196         verif } :: acc
197       | None -> acc
198     in
199     List.fold_left walk_sexp acc' rest_children
200   | Borge_lang.Ast.List (_, children) ->
201     List.fold_left walk_sexp acc children
202   | _ -> acc
203   in
204   List.fold_left (fun acc { node; _ } -> walk_sexp acc node) []
205     file.top_level
206   |> List.rev
207
208 let status_counts (file : Borge_lang.Ast.file) : (status * int) list =
209   let st_list = statuses file in
210   let init = List.map (fun s -> (s, 0)) status_order in
211   List.fold_left (fun acc st ->
212     let count =

```

```
206     match List.assoc_opt st acc with Some v -> v + 1 | None -> 1
207   in
208   (st, count) :: List.remove_assoc st acc
209 ) init st_list
210 |> List.sort (fun (a, _) (b, _) ->
211   let idx_a = List.find_index (fun s -> s = a) status_order |> Option.get
212   in
213   let idx_b = List.find_index (fun s -> s = b) status_order |> Option.get
214   in
215   compare idx_a idx_b
216 )
```


22 lib/db/db_ast.ml

```

1  (** Typed AST for borge DB specs.
2
3      Defines typed representations of database-specific forms:
4      tables, columns, operations, relations, ownership, and groups.
5      Extraction from sexp is in Parse.ml. This module is just types. *)
6
7  (** Column constraints *)
8  type column_constraint =
9      | Primary_key
10     | Unique
11     | Not_null
12     | Default of string          (** default expression as string *)
13     | References of string       (** table.column for foreign key *)
14
15  (** A column definition *)
16  type column = {
17      name : string;
18      typ : string;               (** PostgreSQL type name *)
19      constraints : column_constraint list;
20  }
21
22  (** Index method *)
23  type index_method =
24      | Btree
25      | Gin
26      | Gist
27      | Hash
28      | Spgist
29      | Brin
30
31  (** An index definition *)
32  type index_def = {
33      name : string;
34      columns : string list;
35      idx_method : index_method option; (** None = default btree *)
36      unique : bool;
37  }
38
39  (** A table definition *)
40  type table_def = {
41      name : string;
42      columns : column list;
43      indexes : index_def list;
44      ownership : string option;       (** column name for row-level ownership *)
45  }
46
47  (** CRUD specification *)
48  type crud_spec =
49      | Crud_all                      (** full CRUD *)
50      | Crud_except of string list    (** all except named operations *)
51      | Crud_only of string list      (** only named operations *)
52
53  (** A named query *)
54  type query_def = {
55      name : string;
56      limit : int option;
57  }
58
59  (** Operations on a table *)
60  type operations_def = {
61      table_name : string;
62      crud : crud_spec option;        (** None = no CRUD *)
63      queries : query_def list;
64  }
65
66  (** Join type *)
67  type join_type =
68      | Inner_join
69      | Left_join
70
71  (** A join clause *)

```

```

72 type join_clause = {
73   join_type : join_type;
74   table : string;
75   on_condition : string;          (** raw condition string *)
76 }
77
78 (** A relation (cross-table query) *)
79 type relation_def = {
80   name : string;
81   from_table : string;
82   joins : join_clause list;
83   where_clause : string option;   (** raw where condition *)
84   select_columns : string list;   (** table.column references *)
85   queries : query_def list;
86 }
87
88 (** A capability within a group *)
89 type capability = {
90   table : string;
91   operations : string list;       (** create, read, update, delete *)
92   where_clause : string option;   (** row-level filter *)
93 }
94
95 (** A group definition *)
96 type group_def = {
97   name : string;
98   can_all : bool;                (** true = bypass RLS *)
99   capabilities : capability list;
100 }
101
102 (** A complete DB application spec *)
103 type db_app = {
104   tables : table_def list;
105   operations : operations_def list;
106   relations : relation_def list;
107   groups : group_def list;
108 }

```

23 lib/db/db_json.ml

```

1  (** JSON serialization for borge DB specs.
2
3      Produces Yojson-compatible JSON from the typed DB AST.
4      Used by the --json flag on DB-related commands. *)
5
6  open Db_ast
7
8  (* exempt doc: simple JSON converter - name is self-documenting *)
9  let column_constraint_to_json = function
10 | Primary_key -> 'String "primary-key"
11 | Unique -> 'String "unique"
12 | Not_null -> 'String "not-null"
13 | Default expr -> 'Assoc [("default", 'String expr)]
14 | References ref -> 'Assoc [("references", 'String ref)]
15
16  (* exempt doc: JSON converter for column - name is self-documenting *)
17  let column_to_json (c : Db_ast.column) =
18    'Assoc [
19      ("name", 'String c.name);
20      ("type", 'String c.typ);
21      ("constraints", 'List (List.map column_constraint_to_json
22        c.constraints));
23    ]
24
25  (* exempt doc: JSON converter for index - name is self-documenting *)
26  let index_to_json (i : Db_ast.index_def) =
27    'Assoc [
28      ("name", 'String i.name);
29      ("columns", 'List (List.map (fun s -> 'String s) i.columns));
30      ("unique", 'Bool i.unique);
31    ]
32
33  (* exempt doc: JSON converter for table - name is self-documenting *)
34  let table_to_json (t : Db_ast.table_def) =
35    'Assoc [
36      ("name", 'String t.name);
37      ("columns", 'List (List.map column_to_json t.columns));
38      ("indexes", 'List (List.map index_to_json t.indexes));
39      ("ownership", match t.ownership with Some o -> 'String o | None ->
40        'Null);
41    ]
42
43  (* exempt doc: JSON converter for CRUD spec - name is self-documenting *)
44  let crud_spec_to_json (spec : Db_ast.crud_spec) =
45    match spec with
46    | Crud_all -> 'String "all"
47    | Crud_except ops -> 'Assoc [("except", 'List (List.map (fun s -> 'String
48      s) ops))]
49    | Crud_only ops -> 'Assoc [("only", 'List (List.map (fun s -> 'String s)
50      ops))]
51
52  (* exempt doc: JSON converter for query - name is self-documenting *)
53  let query_to_json (q : Db_ast.query_def) =
54    'Assoc [
55      ("name", 'String q.name);
56      ("limit", match q.limit with Some n -> 'Int n | None -> 'Null);
57    ]
58
59  (* exempt doc: JSON converter for operations - name is self-documenting *)
60  let operations_to_json (o : Db_ast.operations_def) =
61    'Assoc [
62      ("table", 'String o.table_name);
63      ("crud", match o.crud with Some c -> crud_spec_to_json c | None ->
64        'Null);
65      ("queries", 'List (List.map query_to_json o.queries));
66    ]
67
68  (* exempt doc: JSON converter for relation - name is self-documenting *)
69  let relation_to_json (r : Db_ast.relation_def) =
70    'Assoc [
71      ("name", 'String r.name);

```

```

67   ("from", 'String r.from_table);
68   ("joins", 'List (List.map (fun j ->
69     'Assoc [
70       ("type", 'String (match j.join_type with Inner_join -> "inner" |
71         Left_join -> "left"));
72       ("table", 'String j.table);
73       ("on", 'String j.on_condition);
74     ]) r.joins));
75   ("where", match r.where_clause with Some w -> 'String w | None ->
76     'Null);
77   ("select", 'List (List.map (fun s -> 'String s) r.select_columns));
78   ("queries", 'List (List.map query_to_json r.queries));
79 ]
80
81 (* exempt doc: JSON converter for capability - name is self-documenting *)
82 let capability_to_json (c : Db_ast.capability) =
83   'Assoc [
84     ("table", 'String c.table);
85     ("operations", 'List (List.map (fun s -> 'String s) c.operations));
86     ("where", match c.where_clause with Some w -> 'String w | None ->
87       'Null);
88   ]
89
90 (* exempt doc: JSON converter for group - name is self-documenting *)
91 let group_to_json (g : Db_ast.group_def) =
92   'Assoc [
93     ("name", 'String g.name);
94     ("can_all", 'Bool g.can_all);
95     ("capabilities", 'List (List.map capability_to_json g.capabilities));
96   ]
97
98 let db_app_to_json (app : Db_ast.db_app) =
99   'Assoc [
100     ("tables", 'List (List.map table_to_json app.tables));
101     ("operations", 'List (List.map operations_to_json app.operations));
102     ("relations", 'List (List.map relation_to_json app.relations));
103     ("groups", 'List (List.map group_to_json app.groups));
104   ]

```

24 lib/db/db_parse.ml

```

1 open Db_ast
2 (** Extract typed DB AST from generic sexp trees.
3
4     Walks the Ast.sexp tree and extracts database-specific typed
5     nodes: tables, columns, operations, relations, ownership, groups. *)
6
7 open Borge_lang.Ast
8
9 let atom_name = function
10   | Atom (_, name) -> Some name
11   | _ -> None
12
13 let string_value = function
14   | String (_, Quoted { q_content }) -> Some q_content
15   | _ -> None
16
17 (** Parse column constraints from a column body *)
18 let parse_constraint = function
19   | Atom (_, "primary-key") -> Some Primary_key
20   | Atom (_, "unique") -> Some Unique
21   | Atom (_, "not-null") -> Some Not_null
22   | List (_, [Atom (_, "default"); Atom (_, expr)]) ->
23     Some (Default expr)
24   | List (_, [Atom (_, "default"); String (_, Quoted { q_content })]) ->
25     Some (Default q_content)
26   | List (_, [Atom (_, "references"); Atom (_, ref_spec)]) ->
27     Some (References ref_spec)
28   | _ -> None
29
30 (** Parse a (column name (type T) ...) form *)
31 let parse_column = function
32   | List (_, Atom (_, "column") :: Atom (_, name) :: rest) ->
33     let typ = List.find_map (function
34       | List (_, [Atom (_, "type"); Atom (_, t)]) -> Some t
35       | _ -> None
36     ) rest in
37     let constraints = List.filter_map parse_constraint rest in
38     Some { name; typ = Option.value typ ~default:"text"; constraints }
39   | _ -> None
40
41 (** Parse index method *)
42 let parse_index_method = function
43   | Atom (_, "btree") -> Some Btree
44   | Atom (_, "gin") -> Some Gin
45   | Atom (_, "gist") -> Some Gist
46   | Atom (_, "hash") -> Some Hash
47   | Atom (_, "spgist") -> Some Spgist
48   | Atom (_, "brin") -> Some Brin
49   | _ -> None
50
51 (** Parse an (index name ...) form *)
52 let parse_index = function
53   | List (_, Atom (_, "index") :: Atom (_, name) :: rest) ->
54     let columns = List.find_map (function
55       | List (_, Atom (_, "columns") :: cols) ->
56         Some (List.filter_map atom_name cols)
57       | Atom (_, col) -> Some [col] (* single column shorthand *)
58       | _ -> None
59     ) rest in
60     let idx_method = List.find_map (function
61       | List (_, [Atom (_, "method"); m]) -> parse_index_method m
62       | _ -> None
63     ) rest in
64     let unique = List.exists (function
65       | Atom (_, "unique") -> true | _ -> false
66     ) rest in
67     Some { name; columns = Option.value columns ~default:[]; idx_method;
68           unique }
69   | _ -> None
70
71 (** Parse a (table name ...) form *)

```

```

71 let parse_table = function
72 | List (_, Atom (_, "table") :: Atom (_, name) :: rest) ->
73   let columns = List.filter_map parse_column rest in
74   let indexes = List.filter_map parse_index rest in
75   let ownership = List.find_map (function
76   | List (_, [Atom (_, "ownership"); Atom (_, col)]) -> Some col
77   | _ -> None
78   ) rest in
79   Some { name; columns; indexes; ownership }
80 | _ -> None
81
82 (** Parse CRUD specification *)
83 let parse_crud = function
84 | List (_, [Atom (_, "crud")]) ->
85   Some Crud_all
86 | List (_, [Atom (_, "crud"); List (_, Atom (_, "except") :: ops)]) ->
87   let names = List.filter_map atom_name ops in
88   Some (Crud_except names)
89 | List (_, [Atom (_, "crud"); List (_, Atom (_, "only") :: ops)]) ->
90   let names = List.filter_map atom_name ops in
91   Some (Crud_only names)
92 | _ -> None
93
94 (** Parse a (query name ...) form *)
95 let parse_query = function
96 | List (_, Atom (_, "query") :: Atom (_, name) :: rest) ->
97   let limit = List.find_map (function
98   | List (_, [Atom (_, "limit"); Atom (_, n)]) ->
99     int_of_string_opt n
100   | _ -> None
101   ) rest in
102   Some { name; limit }
103 | _ -> None
104
105 (** Parse an (operations table-name ...) form *)
106 let parse_operations = function
107 | List (_, Atom (_, "operations") :: Atom (_, table_name) :: rest) ->
108   let crud = List.find_map (fun s ->
109     Option.map (fun c -> c) (parse_crud s)
110   ) rest in
111   let queries = List.filter_map parse_query rest in
112   Some { table_name; crud; queries }
113 | _ -> None
114
115 (** Parse a join clause *)
116 let parse_join ~expected_keyword = function
117 | List (_, [Atom (_, kw); Atom (_, table); List (_, [Atom (_, "on"); Atom
118   (_, cond)])])
119   when kw = expected_keyword ->
120   let join_type = match kw with
121   | "join" -> Inner_join
122   | "left-join" -> Left_join
123   | _ -> Inner_join
124   in
125   Some { join_type; table; on_condition = cond }
126 | _ -> None
127
128 (** Parse a (relation name ...) form *)
129 let parse_relation = function
130 | List (_, Atom (_, "relation") :: Atom (_, name) :: rest) ->
131   let from_table = List.find_map (function
132   | List (_, [Atom (_, "from"); Atom (_, t)]) -> Some t
133   | _ -> None
134   ) rest in
135   let joins = List.filter_map (fun s ->
136     match parse_join ~expected_keyword:"join" s with
137     | Some j -> Some j
138     | None -> parse_join ~expected_keyword:"left-join" s
139   ) rest in
140   let where_clause = List.find_map (function
141   | List (_, Atom (_, "where") :: [Atom (_, cond)]) -> Some cond
142   | _ -> None
143   ) rest in
144   Some { name; from_table; joins; where_clause }
145 | _ -> None

```

```

143   let select_columns = List.find_map (function
144     | List (_, Atom (_, "select")) :: cols ->
145       Some (List.filter_map atom_name cols)
146     | _ -> None
147   ) rest in
148   let queries = List.filter_map parse_query rest in
149   Some { name; from_table = Option.value from_table ~default:"";
150         joins; where_clause; select_columns = Option.value
151         select_columns ~default:[];
152         queries }
153 | _ -> None
154
155 (** Parse a (can TABLE OPS (where COND)) capability *)
156 let parse_capability = function
157 | List (_, Atom (_, "can")) :: Atom (_, table) :: rest ->
158   let ops = List.filter_map (function
159     | Atom (_, op) when List.mem op ["create"; "read"; "update";
160     "delete"] ->
161       Some op
162     | _ -> None
163   ) rest in
164   let where_clause = List.find_map (function
165     | List (_, [Atom (_, "where"); Atom (_, cond)]) -> Some cond
166     | _ -> None
167   ) rest in
168   Some { table; operations = ops; where_clause }
169 | Atom (_, "can-all") ->
170   (* can-all is handled at the group level *)
171   None
172 | _ -> None
173
174 (** Parse a (group name ...) form *)
175 let parse_group = function
176 | List (_, Atom (_, "group")) :: Atom (_, name) :: rest ->
177   let can_all = List.exists (function
178     | List (_, [Atom (_, "can-all")]) -> true
179     | _ -> false
180   ) rest in
181   let capabilities = List.filter_map parse_capability rest in
182   Some { name; can_all; capabilities }
183 | _ -> None
184
185 (** Parse a complete DB application from a (db app-name ...) form *)
186 let parse_db_app = function
187 | List (_, Atom (_, "db")) :: _ :: body ->
188   let tables = List.filter_map parse_table body in
189   let operations = List.filter_map parse_operations body in
190   let relations = List.filter_map parse_relation body in
191   let groups =
192     List.concat_map (function
193       | List (_, Atom (_, "groups")) :: entries ->
194         List.filter_map parse_group entries
195       | _ -> []
196     ) body
197   in
198   Some { tables; operations; relations; groups }
199 | _ -> None
200
201 (** Parse all DB forms from a file's top-level sexps *)
202 let parse_file (file : Borge_lang.Ast.file) : db_app option =
203   let rec walk = function
204     | [] -> None
205     | { node; _ } :: rest ->
206       (match parse_db_app node with
207       | Some app -> Some app
208       | None -> walk rest)
209   in
210   match walk file.top_level with
211   | Some app -> Some app
212   | None ->
213     let sexps = List.map (fun swc -> swc.node) file.top_level in
214     let tables = List.filter_map parse_table sexps in
215     let operations = List.filter_map parse_operations sexps in

```

```
214   let relations = List.filter_map parse_relation sexps in
215   let groups = List.concat_map (function
216     | List (_, Atom (_, "groups") :: entries) ->
217       List.filter_map parse_group entries
218     | _ -> []
219   ) sexps in
220   Some { tables; operations; relations; groups }
```


25 lib/db/db_sql.ml

```

1  (*| PostgreSQL DDL generator for borge DB specs.
2
3      Walks the typed DB AST and produces a complete SQL file
4      with CREATE TABLE, CREATE INDEX, and RLS policy statements.
5      See db.borg section convention-targets / postgres-sql-target for
6      the full mapping specification. |*)
7
8  open Db_ast
9
10 (* --- Helper: SQL identifier escaping --- *)
11
12 (* exempt doc: trivial SQL id quote - name is self-documenting *)
13 let sql_id name =
14   (* Quote identifiers that might collide with SQL keywords *)
15   let need_quote = String.length name = 0 ||
16     let c = name.[0] in not ((c >= 'a' && c <= 'z') || (c >= 'A' && c <=
17       'Z')) || c = '_' in
18   if need_quote then Printf.sprintf "\"%s\"" name
19   else name
20
21 (* --- Helper: default value expression --- *)
22
23 let default_expr expr =
24   match String.lowercase_ascii expr with
25   | "gen_random_uuid" -> "gen_random_uuid()"
26   | "now" -> "now()"
27   | "false" -> "FALSE"
28   | "true" -> "TRUE"
29   | _ -> expr
30
31 (* --- Helper: column constraint to SQL --- *)
32
33 let column_constraint_to_sql = function
34   | Primary_key -> "PRIMARY KEY"
35   | Unique -> "UNIQUE"
36   | Not_null -> "NOT NULL"
37   | Default expr -> Printf.sprintf "DEFAULT %s" (default_expr expr)
38   | References ref ->
39     (* Parse table.column format *)
40     match String.split_on_char '.' ref with
41     | [table; col] ->
42       Printf.sprintf "REFERENCES %s (%s)" (sql_id table) (sql_id col)
43     | _ ->
44       Printf.sprintf "REFERENCES %s" (sql_id ref)
45
46 (* --- Helper: column to SQL line --- *)
47
48 let column_to_sql (c : column) =
49   let constraints = List.filter_map (function
50     | Primary_key | Unique | Not_null | Default _ | References _ as cc ->
51       Some (column_constraint_to_sql cc)
52   ) c.constraints in
53   let parts = [String.uppercase_ascii (sql_id c.typ)] @ constraints in
54   Printf.sprintf " %s %s" (sql_id c.name) (String.concat " " parts)
55
56 (* --- Helper: index method to SQL --- *)
57
58 let index_method_to_sql = function
59   | Btree -> "btree"
60   | Gin -> "gin"
61   | Gist -> "gist"
62   | Hash -> "hash"
63   | Spgist -> "spgist"
64   | Brin -> "brin"
65
66 (* --- Topological sort for foreign key dependencies --- *)
67
68 (*| Build a dependency graph from (references) constraints.
69   Returns tables in an order where no table is created before
70   the table it references. Circular references are an error. |*)

```

```

71 type dep_graph = (string, string list) Hashtbl.t
72
73 let build_dep_graph (tables : table_def list) =
74   let graph : dep_graph = Hashtbl.create 16 in
75   List.iter (fun (tbl : table_def) -> Hashtbl.add graph tbl.name []) tables;
76   List.iter (fun (t : table_def) ->
77     List.iter (fun (c : column) ->
78       List.iter (function
79         | References ref ->
80           (match String.split_on_char '.' ref with
81             | [dep_table; _] ->
82               (* t.name depends on dep_table *)
83               let deps = match Hashtbl.find_opt graph t.name with Some v ->
84                 v | None -> [] in
85               if not (List.mem dep_table deps) then
86                 Hashtbl.replace graph t.name (dep_table :: deps)
87             | _ -> ())
88         | _ -> ())
89       c.constraints
90     ) t.columns
91   ) tables;
92   graph
93
94 let topological_sort (tables : table_def list) =
95   let table_map = Hashtbl.create 16 in
96   List.iter (fun (tbl : table_def) -> Hashtbl.add table_map tbl.name tbl)
97     tables;
98   let graph = build_dep_graph tables in
99   let visited = Hashtbl.create 16 in
100   let in_stack = Hashtbl.create 16 in
101   let result = ref [] in
102   let rec visit name =
103     if Hashtbl.mem in_stack name then
104       failwith (Printf.sprintf "Circular foreign key dependency involving
105         table %s" name);
106     if Hashtbl.mem visited name then ()
107     else begin
108       Hashtbl.add in_stack name ();
109       let deps = match Hashtbl.find_opt graph name with Some v -> v | None
110         -> [] in
111       List.iter visit deps;
112       Hashtbl.remove in_stack name;
113       Hashtbl.add visited name ();
114       (match Hashtbl.find_opt table_map name with
115         | Some t -> result := t :: !result
116         | None -> ())
117     end
118   in
119   List.iter (fun (tbl : table_def) -> visit tbl.name) tables;
120   List.rev !result
121
122 (* --- Schema header --- *)
123
124 let schema_header source_path =
125   let timestamp =
126     let t = Unix.time () in
127     let tm = Unix.gmtime t in
128     Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
129       (tm.Unix.tm_year + 1900) tm.Unix.tm_mon tm.Unix.tm_mday
130       tm.Unix.tm_hour tm.Unix.tm_min tm.Unix.tm_sec
131   in
132   Printf.sprintf
133     "-- Generated by borge generate --target postgres-sql\n\
134     -- Source: %s\n\
135     -- Generated at: %s\n\
136     -- DO NOT EDIT: regenerate from spec\n"
137     source_path timestamp
138
139 (* --- CREATE TABLE generation --- *)
140
141 let generate_table (t : table_def) =
142   let cols = List.map column_to_sql t.columns in
143   let col_list = String.concat ",\n" cols in

```

```

140   Printf.sprintf "CREATE TABLE IF NOT EXISTS %s (\n%s\n);\n"
141     (sql_id t.name) col_list
142
143 (* --- CREATE INDEX generation --- *)
144
145 let generate_index (table_name : string) (idx : index_def) =
146   let col_list = String.concat ", " (List.map sql_id idx.columns) in
147   let method_clause = match idx.idx_method with
148     | Some m -> Printf.sprintf " USING %s" (index_method_to_sql m)
149     | None -> ""
150   in
151   let unique_clause = if idx.unique then "UNIQUE " else "" in
152   Printf.sprintf "CREATE %sINDEX IF NOT EXISTS %s ON %s%s (%s);\n"
153     unique_clause (sql_id idx.name) (sql_id table_name) method_clause
154     col_list
155
156 (* --- RLS: ownership shorthand policies --- *)
157
158 let generate_ownership_policies (t : table_def) =
159   match t.ownership with
160   | None -> ""
161   | Some col ->
162     let enable = Printf.sprintf "ALTER TABLE %s ENABLE ROW LEVEL
163       SECURITY;\n" (sql_id t.name) in
164     let ops = [
165       ("SELECT", "USING");
166       ("INSERT", "WITH CHECK");
167       ("UPDATE", "USING");
168       ("DELETE", "USING");
169     ] in
170     let policies = List.map (fun (op, _clause_type) ->
171       let using_or_check =
172         match op with
173         | "UPDATE" ->
174           Printf.sprintf " FOR %s USING (%s = current_user_id())\n WITH
175             CHECK (%s = current_user_id())"
176             op (sql_id col) (sql_id col)
177         | "SELECT" | "DELETE" ->
178           Printf.sprintf " FOR %s USING (%s = current_user_id())"
179             op (sql_id col)
180         | "INSERT" ->
181           Printf.sprintf " FOR %s WITH CHECK (%s = current_user_id())"
182             op (sql_id col)
183         | _ -> ""
184       in
185       Printf.sprintf "CREATE POLICY %s_%s_%s ON %s\n%s;"
186         (String.lowercase_ascii t.name) (sql_id col)
187         (String.lowercase_ascii op)
188         (sql_id t.name) using_or_check
189     ) ops in
190     enable ^ String.concat "\n" policies ^ "\n"
191
192 (* --- RLS: group policies --- *)
193
194 let operation_to_sql = function
195   | "create" -> "INSERT"
196   | "read" -> "SELECT"
197   | "update" -> "UPDATE"
198   | "delete" -> "DELETE"
199   | op -> String.uppercase_ascii op
200
201 let where_clause_to_sql (w : string) =
202   (* Replace current-user with current_user_id()
203     and convert hyphens to underscores for SQL column names *)
204   let w = String.map (fun c -> if c = '-' then '_' else c) w in
205   let needle = "current_user" in
206   let needle_len = String.length needle in
207   let replacement = "current_user_id()" in
208   let buf = Buffer.create (String.length w + 20) in
209   let i = ref 0 in
210   while !i <= String.length w - needle_len do
211     if (* exempt: String.sub *) String.sub w !i needle_len = needle then
212       begin

```

```

208     Buffer.add_string buf replacement;
209     i := !i + needle_len
210   end else begin
211     Buffer.add_char buf w.[!i];
212     incr i
213   end
214 done;
215 (* Copy remaining characters *)
216 while !i < String.length w do
217   Buffer.add_char buf w.[!i];
218   incr i
219 done;
220 Buffer.contents buf
221
222 let generate_capability_policies (group_name : string) (cap : capability) =
223   let table = sql_id cap.table in
224   let gn = String.lowercase_ascii group_name in
225   let tn = String.lowercase_ascii cap.table in
226   List.map (fun op ->
227     let sql_op = operation_to_sql op in
228     let policy_name = Printf.sprintf "%s_%s_%s" gn tn
      (String.lowercase_ascii sql_op) in
229     match op, cap.where_clause with
230     | "read", Some w ->
231       Printf.sprintf "CREATE POLICY %s ON %s FOR %s\n USING (%s);"
232         policy_name table sql_op (where_clause_to_sql w)
233     | "create", Some w ->
234       Printf.sprintf "CREATE POLICY %s ON %s FOR %s\n WITH CHECK (%s);"
235         policy_name table sql_op (where_clause_to_sql w)
236     | "update", Some w ->
237       Printf.sprintf "CREATE POLICY %s ON %s FOR %s\n USING (%s)\n WITH
238         CHECK (%s);"
239         policy_name table sql_op (where_clause_to_sql w)
240         (where_clause_to_sql w)
241     | "delete", Some w ->
242       Printf.sprintf "CREATE POLICY %s ON %s FOR %s\n USING (%s);"
243         policy_name table sql_op (where_clause_to_sql w)
244     | _, Some w ->
245       Printf.sprintf "CREATE POLICY %s ON %s FOR %s\n USING (%s);"
246         policy_name table sql_op (where_clause_to_sql w)
247     | _, None ->
248       Printf.sprintf "CREATE POLICY %s ON %s FOR %s\n USING (true);"
249         policy_name table sql_op
250   ) cap.operations
251
252 let generate_group_policies (g : group_def) =
253   if g.can_all then
254     (* can-all: single policy for ALL operations with USING (true) *)
255     [Printf.sprintf "CREATE POLICY %s_all ON %s FOR ALL\n USING (true)
256       WITH CHECK (true);"
257       (String.lowercase_ascii g.name)]
258     (* Note: table name needs to be filled in per-table. *)
259     (* This is handled below in generate_groups *)
260   else
261     List.concat_map (generate_capability_policies g.name) g.capabilities
262
263 (* --- Operations/queries as comments --- *)
264
265 let generate_operations_comments (ops : operations_def) =
266   let buf = Buffer.create 128 in
267   (match ops.crud with
268   | Some _ ->
269     Buffer.add_string buf (Printf.sprintf "-- Operations on %s: CRUD"
270       ops.table_name)
271   | None -> ());
272   List.iter (fun q ->
273     let limit_str = match q.limit with Some n -> Printf.sprintf " (limit
274       %d)" n | None -> "" in
275     Buffer.add_string buf (Printf.sprintf "\n-- Query: %s%s for table %s"
276       (String.lowercase_ascii q.name) limit_str (String.lowercase_ascii
277       ops.table_name))
278   ) ops.queries;
279   Buffer.contents buf

```

```

274
275 (* --- Relations as comments --- *)
276
277 let generate_relation_comment (r : relation_def) =
278   let joins = List.map (fun j ->
279     let jt = match j.join_type with Inner_join -> "JOIN" | Left_join ->
280       "LEFT JOIN" in
281     Printf.sprintf "%s %s ON %s" jt (String.lowercase_ascii j.table)
282     j.on_condition
283   ) r.joins in
284   let where_str = match r.where_clause with Some w -> Printf.sprintf "\n--
285     WHERE %s" w | None -> "" in
286   let select_str = String.concat ", " (List.map String.lowercase_ascii
287     r.select_columns) in
288   Printf.sprintf "-- Relation: %s\n-- FROM %s %s\n-- SELECT %s"
289     (String.lowercase_ascii r.name) (String.lowercase_ascii r.from_table)
290     (String.concat " " joins) where_str select_str
291
292 (* --- Main generation --- *)
293
294 let generate ?(source_path = "<spec>") (app : db_app) =
295   let buf = Buffer.create 4096 in
296
297   (* Header *)
298   Buffer.add_string buf (schema_header source_path);
299
300   (* 1. CREATE TABLE statements (topologically sorted) *)
301   let sorted_tables = topological_sort app.tables in
302   Buffer.add_string buf "\n-- Tables\n\n";
303   List.iter (fun (tbl : table_def) ->
304     Buffer.add_string buf (generate_table tbl);
305     Buffer.add_char buf '\n';
306     (* Indexes for this table *)
307     List.iter (fun idx ->
308       Buffer.add_string buf (generate_index tbl.name idx)
309     ) tbl.indexes;
310     if tbl.indexes <> [] then Buffer.add_char buf '\n';
311   ) sorted_tables;
312
313   (* 2. RLS: ownership policies *)
314   let tables_with_ownership = List.filter (fun (tbl : table_def) ->
315     tbl.ownership <> None) sorted_tables in
316   let tables_with_groups =
317     if app.groups = [] then []
318     else
319       (* All tables that have group policies need RLS enabled *)
320       let referenced = List.concat_map (fun g ->
321         if g.can_all then []
322         else List.map (fun c -> c.table) g.capabilities
323       ) app.groups in
324       List.filter (fun (tbl : table_def) -> List.mem tbl.name referenced)
325         sorted_tables
326   in
327   let tables_needing_rls =
328     List.filter (fun (tbl : table_def) ->
329       tbl.ownership <> None ||
330       List.exists (fun (tg : table_def) -> tg.name = tbl.name)
331         tables_with_groups
332     ) sorted_tables
333   in
334   if tables_needing_rls <> [] then begin
335     Buffer.add_string buf "\n-- Row-Level Security\n\n";
336     List.iter (fun (tbl : table_def) ->
337       Buffer.add_string buf (generate_ownership_policies tbl);
338       Buffer.add_char buf '\n';
339     ) tables_with_ownership;
340
341     (* 3. Group policies *)
342     List.iter (fun (g : group_def) ->
343       if g.can_all then (
344         (* can-all needs to be per-table *)
345         List.iter (fun (tbl : table_def) ->

```

```
339     let policy = Printf.sprintf "CREATE POLICY %s_all ON %s FOR ALL\n
340     USING (true) WITH CHECK (true);\n"
341     (String.lowercase_ascii g.name) (sql_id tbl.name) in
342     Buffer.add_string buf policy
343   ) sorted_tables;
344   Buffer.add_char buf '\n';
345   ) else
346     List.iter (fun p ->
347       Buffer.add_string buf p;
348       Buffer.add_char buf '\n';
349     ) (generate_group_policies g);
350   ) app.groups;
351 end;
352
353 (* 4. Operations/queries as comments *)
354 if app.operations <> [] then begin
355   Buffer.add_string buf "\n-- Operations & Queries\n\n";
356   List.iter (fun ops ->
357     let comment = generate_operations_comments ops in
358     if comment <> "" then
359       Buffer.add_string buf (comment ^ "\n\n")
360   ) app.operations;
361 end;
362
363 (* 5. Relations as comments *)
364 if app.relations <> [] then begin
365   Buffer.add_string buf "\n-- Relations\n\n";
366   List.iter (fun r ->
367     Buffer.add_string buf (generate_relation_comment r);
368     Buffer.add_string buf "\n\n";
369   ) app.relations;
370 end;
371 Buffer.contents buf
```

26 lib/db/db_validate.ml

```

1  (** Semantic validation for borge DB specs.
2
3      Checks that references are valid:
4      - (references table.column) refers to an existing table
5      - (ownership column) refers to an existing column on the same table
6      - Group capability tables exist in the schema
7      - Operations reference existing tables
8      - Table, group, and column names are unique *)
9
10 open Db_ast
11
12 type issue = {
13   message : string;
14   severity : [ `Error | `Warning ];
15 }
16
17 let err msg = { message = msg; severity = `Error }
18 let warn msg = { message = msg; severity = `Warning }
19
20 (** Build a lookup: table name -> column name list *)
21 let table_columns (tables : table_def list) : (string, string list)
22   Hashtbl.t =
23   let tbl = Hashtbl.create 16 in
24   List.iter (fun (t : table_def) ->
25     let cols = List.map (fun (c : column) -> c.name) t.columns in
26     Hashtbl.add tbl t.name cols
27   ) tables;
28   tbl
29
30 (** Get all table names *)
31 let table_names (tables : table_def list) : string list =
32   List.map (fun (t : table_def) -> t.name) tables
33
34 (** Check table name uniqueness *)
35 let check_table_uniqueness (tables : table_def list) : issue list =
36   let seen = Hashtbl.create 16 in
37   List.filter_map (fun (t : table_def) ->
38     if Hashtbl.mem seen t.name then
39       Some (err (Printf.sprintf "duplicate table name: %s" t.name))
40     else begin
41       Hashtbl.add seen t.name true;
42       None
43     end
44   ) tables
45
46 (** Check a column's references *)
47 let check_column (table_name : string) (col : column)
48   (tbl : (string, string list) Hashtbl.t) : issue list =
49   List.filter_map (function
50     | References ref_spec ->
51       (match String.split_on_char '.' ref_spec with
52        | [ref_table; _ref_col] ->
53          if not (Hashtbl.mem tbl ref_table) then
54            Some (err (Printf.sprintf
55              "table '%s': column '%s' references undefined table '%s'"
56              table_name col.name ref_table))
57          else None
58        | _ ->
59          Some (warn (Printf.sprintf
60            "table '%s': column '%s' has unusual references format: %s"
61            table_name col.name ref_spec)))
62     | _ -> None
63   ) col.constraints
64
65 (** Check ownership: the column must exist on the table *)
66 let check_ownership (table : table_def) : issue list =
67   match table.ownership with
68   | None -> []
69   | Some col_name ->
70     let col_names = List.map (fun (c : column) -> c.name) table.columns in
71     if not (List.mem col_name col_names) then

```

```

71     [err (Printf.sprintf "table '%s': ownership references undefined
72         column '%s'"
73         table.name col_name)]
74     else []
75
76 (** Check a table definition *)
77 let check_table (tbl : (string, string list) Hashtbl.t) (table : table_def)
78 : issue list =
79     let col_names = List.map (fun (c : column) -> c.name) table.columns in
80     let col_dup_issues =
81         let seen = Hashtbl.create 8 in
82         List.filter_map (fun (name : string) ->
83             if Hashtbl.mem seen name then
84                 Some (err (Printf.sprintf "table '%s': duplicate column name '%s'"
85                     table.name name))
86             else begin
87                 Hashtbl.add seen name true;
88                 None
89             end
90         ) col_names
91     in
92     let col_issues = List.concat_map
93         (fun (c : column) -> check_column table.name c tbl) table.columns in
94     let ownership_issues = check_ownership table in
95     col_dup_issues @ col_issues @ ownership_issues
96
97 (** Check operations reference existing tables *)
98 let check_operations (t_names : string list) (ops : operations_def) : issue
99 list =
100     if not (List.mem ops.table_name t_names) then
101         [err (Printf.sprintf "operations references undefined table: %s"
102             ops.table_name)]
103     else []
104
105 (** Check a relation's from-table and join tables *)
106 let check_relation (t_names : string list) (rel : relation_def) : issue
107 list =
108     let from_issues =
109         if rel.from_table <> "" && not (List.mem rel.from_table t_names) then
110             [err (Printf.sprintf "relation '%s': from-table '%s' is undefined"
111                 rel.name rel.from_table)]
112         else []
113     in
114     let join_issues = List.filter_map (fun (j : join_clause) ->
115         if not (List.mem j.table t_names) then
116             Some (err (Printf.sprintf "relation '%s': join table '%s' is
117                 undefined"
118                     rel.name j.table))
119         else None
120     ) rel.joins in
121     from_issues @ join_issues
122
123 (** Check group capability table references *)
124 let check_capability (t_names : string list) (cap : capability) : issue
125 list =
126     if not (List.mem cap.table t_names) then
127         [err (Printf.sprintf "group capability references undefined table: %s"
128             cap.table)]
129     else []
130
131 (** Check group name uniqueness and capability references *)
132 let check_groups (t_names : string list) (groups : group_def list) : issue
133 list =
134     let name_issues =
135         let seen = Hashtbl.create 8 in
136         List.filter_map (fun (g : group_def) ->
137             if Hashtbl.mem seen g.name then
138                 Some (err (Printf.sprintf "duplicate group name: %s" g.name))
139             else begin
140                 Hashtbl.add seen g.name true;
141                 None
142             end
143         ) groups
144     in

```



```
134   in
135   let cap_issues = List.concat_map (fun (g : group_def) ->
136     List.concat_map (check_capability t_names) g.capabilities
137   ) groups in
138   name_issues @ cap_issues
139
140 (** Validate the complete DB app spec *)
141 let validate (app : db_app) : issue list =
142   let tbl = table_columns app.tables in
143   let t_names = table_names app.tables in
144   let table_issues = check_table_uniqueness app.tables
145     @ List.concat_map (check_table tbl) app.tables in
146   let ops_issues = List.concat_map (check_operations t_names)
147     app.operations in
148   let rel_issues = List.concat_map (check_relation t_names) app.relations in
149   let group_issues = check_groups t_names app.groups in
150   table_issues @ ops_issues @ rel_issues @ group_issues
```

27 lib/ui/ui_ast.ml

```

1  (** Typed AST for borge UI specs.
2
3      The generic sexp parser produces Ast.sexp trees. This module
4      defines typed representations of UI-specific forms: elements,
5      themes, components, layouts, pages, routes, and variants.
6
7      Extraction from sexp is in Parse.ml. This module is just types. *)
8
9  (** Sizing values *)
10 type sizing =
11   | Flex of int          (** N fractional parts (flex: N) *)
12   | Grow                 (** fill remaining space *)
13   | Full                 (** 100% of parent *)
14   | Auto                 (** size to content *)
15   | Px of int            (** pixels *)
16   | Rem of float         (** rem units *)
17   | Pct of int           (** percentage *)
18   | Vh of int            (** viewport height percentage *)
19
20  (** Color values *)
21 type color_value =
22   | Hex of string        (** "#a8421c" *)
23   | Keyword of string    (** blue, red, dark, white, etc. *)
24   | Palette of string     (** (palette name) -- theme variable reference *)
25   | With_alpha of color_value * float (** color with opacity *)
26
27  (** Spacing values *)
28 type spacing_value =
29   | Spacing_px of int
30   | Spacing_rem of float
31   | Spacing_var of string (** (spacing name) -- theme variable reference *)
32
33  (** Font size values *)
34 type font_size_value =
35   | Font_px of int
36   | Font_rem of float
37   | Font_var of string    (** (font-size name) -- theme variable reference *)
38
39  (** Corner radius values *)
40 type radius_value =
41   | Radius_px of int
42   | Radius_var of string  (** (radius name) -- theme variable reference *)
43
44  (** Layout direction *)
45 type layout_direction =
46   | Horizontal
47   | Vertical
48
49  (** Alignment *)
50 type alignment =
51   | Align_start | Align_center | Align_end
52   [@@deriving eq]
53
54 let alignment_of_string = function
55   | "left" | "top" -> Some Align_start
56   | "center" -> Some Align_center
57   | "right" | "bottom" -> Some Align_end
58   | _ -> None
59
60  (** Scroll direction *)
61 type scroll_direction =
62   | Scroll_vertical
63   | Scroll_horizontal
64   | Scroll_both
65
66  (** Interactive capability *)
67 type interactive_type =
68   | Interactive_click
69   | Interactive_type
70   | Interactive_hover

```

```

71
72 (** Text wrapping *)
73 type text_wrap =
74   | Wrap_words
75   | Wrap_none
76   | Wrap_newlines
77
78 (** Float attachment *)
79 type float_spec =
80   | Float_simple
81   | Float_attach_parent
82   | Float_offset of (string * int) list (** offset pairs like (top 8) *)
83
84 (** Per-side padding *)
85 type padding =
86   | Padding_uniform of spacing_value
87   | Padding_sides of {
88     top : spacing_value;
89     right : spacing_value;
90     bottom : spacing_value;
91     left : spacing_value;
92   }
93
94 (** Border specification *)
95 type border_spec = {
96   width : int option;
97   color : color_value option;
98   side : string option; (** None = all sides, Some "bottom" = specific
99     side *)
100 }
101
102 (** Corner radius *)
103 type corner_radius =
104   | Corner_uniform of radius_value
105   | Corner_sides of {
106     top_left : radius_value;
107     top_right : radius_value;
108     bottom_left : radius_value;
109     bottom_right : radius_value;
110   }
111
112 (** Parsed theme variable entry *)
113 type theme_entry =
114   | Palette_entry of string * string (** name, hex value *)
115   | Spacing_entry of string * int (** name, px value *)
116   | Font_size_entry of string * int (** name, px value *)
117   | Radius_entry of string * int (** name, px value *)
118
119 (** A theme definition *)
120 type theme = {
121   entries : theme_entry list;
122 }
123
124 (** A named variant -- overrides base properties *)
125 type variant = {
126   name : string;
127   properties : ui_property list;
128   children : ui_element list option; (** None = inherit base children *)
129 }
130
131 and ui_property =
132   | P_layout of layout_direction
133   | P_width of sizing
134   | P_height of sizing
135   | P_padding of padding
136   | P_gap of spacing_value
137   | P_bg of color_value
138   | P_color of color_value
139   | P_border of border_spec
140   | P_corner of corner_radius
141   | P_scroll of scroll_direction
142   | P_float of float_spec
143   | P_interactive of interactive_type
144   | P_align_x of alignment

```

```

143 | P_align_y of alignment
144
145 (** A child of a UI element -- either a sub-element or a slot reference *)
146 and ui_child =
147 | El of ui_element          (** a concrete child element *)
148 | Slot_ref of string        (** ..name -- a named slot reference *)
149
150 (** A UI element -- the core building block *)
151 and ui_element = {
152   name : string option;      (** None for anonymous elements *)
153   properties : ui_property list;
154   children : ui_child list;  (** empty if no (children ...) *)
155   text : string option;      (** Some "text" for inline text nodes *)
156   variants : variant list;
157 }
158
159 (** Slot kinds inside components and layouts *)
160 type slot =
161 | Default_slot              (** ..content *)
162 | Named_slot of string      (** (slot name) *)
163
164 (** A component definition *)
165 type component = {
166   name : string;
167   properties : ui_property list;
168   slots : slot list;
169   children : ui_child list;
170   variants : variant list;
171 }
172
173 (** A layout definition *)
174 type layout_def = {
175   name : string;
176   root : ui_element;
177   slots : slot list;
178 }
179
180 (** A slot fill -- page provides content for a layout slot *)
181 type slot_fill = {
182   slot_name : string;
183   content : ui_element list;
184 }
185
186 (** A page definition *)
187 type page = {
188   name : string;
189   layout_name : string;
190   fills : slot_fill list;
191   override_element : (string * ui_property list) list; (** element name,
192   overridden props *)
193 }
194
195 (** A route mapping *)
196 type route = {
197   path : string;
198   page_name : string;
199 }
200
201 (** User action in a workflow step *)
202 type action =
203 | Click of string          (** element name *)
204 | Type of string           (** input element name *)
205 | Hover of string          (** element name *)
206
207 (** Outcome of a workflow step *)
208 type outcome =
209 | Navigate of string       (** route path *)
210 | Submit                  (** form submission *)
211 | No_outcome               (** terminal step *)
212
213 (** A single step in a user workflow *)
214 type step = {
215   name : string;

```

```
215   page_name : string;
216   actions : action list;
217   outcome : outcome;
218 }
219
220 (** A user workflow -- a journey through pages *)
221 type workflow = {
222   name : string;
223   doc : string option;
224   steps : step list;
225 }
226
227 (** A complete UI application spec *)
228 type ui_app = {
229   theme : theme option;
230   components : component list;
231   layouts : layout_def list;
232   pages : page list;
233   routes : route list;
234   elements : ui_element list; (** top-level elements not in a
235                                component/layout/page *)
236   workflows : workflow list;
237 }
```

28 lib/ui/ui__clay.ml

```

1  (*| Clay C generator for borge UI specs.
2
3      Walks the typed UI AST and produces a .c + .h file pair that
4      constructs Clay layout trees callable from a raylib render loop.
5      Primary use: pico firmware screens, but also any C application
6      using Clay. See ui.borg section convention-targets / clay-c-target
7      for the full mapping specification. |*)
8
9  open Ui_ast
10
11  (* --- Theme resolution (compile-time only -- C has no runtime style
12      lookups) --- *)
13
14  type theme_map = {
15      palette : (string, string) Hashtbl.t;
16      spacing : (string, int) Hashtbl.t;
17      font_size : (string, int) Hashtbl.t;
18      radius : (string, int) Hashtbl.t;
19  }
20
21  let empty_theme = {
22      palette = Hashtbl.create 0;
23      spacing = Hashtbl.create 0;
24      font_size = Hashtbl.create 0;
25      radius = Hashtbl.create 0;
26  }
27
28  let build_theme_map (t : theme option) =
29      match t with
30      | None -> empty_theme
31      | Some theme ->
32          let tm = { palette = Hashtbl.create 16; spacing = Hashtbl.create 16;
33                      font_size = Hashtbl.create 16; radius = Hashtbl.create 16 }
34          in
35          List.iter (function
36              | Palette_entry (name, value) -> Hashtbl.add tm.palette name value
37              | Spacing_entry (name, value) -> Hashtbl.add tm.spacing name value
38              | Font_size_entry (name, value) -> Hashtbl.add tm.font_size name value
39              | Radius_entry (name, value) -> Hashtbl.add tm.radius name value
40          ) theme.entries;
41          tm
42
43  (* --- Hex color to RGBA struct --- *)
44
45  (* Convert hex "#RRGGBB" to { R, G, B, 255 } *)
46  let hex_to_rgba hex =
47      if String.length hex >= 7 then
48          match int_of_string_opt ("0x" ^ (* exempt: String.sub *) String.sub hex
49              1 2),
50              int_of_string_opt ("0x" ^ (* exempt: String.sub *) String.sub hex
51              3 2),
52              int_of_string_opt ("0x" ^ (* exempt: String.sub *) String.sub hex
53              5 2) with
54          | Some r, Some g, Some b -> Printf.sprintf "{ %d, %d, %d, 255 }" r g b
55          | _ -> "{ 128, 128, 128, 255 }"
56      else "{ 128, 128, 128, 255 }"
57
58  (* Keyword color defaults in RGBA *)
59  let keyword_to_rgba = function
60      | "red" -> "{ 220, 60, 60, 255 }"
61      | "orange" -> "{ 220, 128, 60, 255 }"
62      | "yellow" -> "{ 220, 192, 60, 255 }"
63      | "green" -> "{ 60, 180, 80, 255 }"
64      | "blue" -> "{ 60, 120, 220, 255 }"
65      | "purple" -> "{ 128, 60, 220, 255 }"
66      | "pink" -> "{ 220, 60, 160, 255 }"
67      | "white" -> "{ 255, 255, 255, 255 }"
68      | "black" -> "{ 0, 0, 0, 255 }"
69      | "gray" -> "{ 128, 128, 128, 255 }"
70      | "dark" -> "{ 40, 40, 40, 255 }"
71      | "light" -> "{ 230, 230, 230, 255 }"

```

```

67 | "transparent" -> "{ 0, 0, 0, 0 }"
68 | _ -> "{ 136, 136, 136, 255 }"
69
70 (* Resolve a color to an RGBA C struct literal *)
71 let rec resolve_color_rgba (tm : theme_map) = function
72 | Hex h -> hex_to_rgba h
73 | Keyword k -> keyword_to_rgba k
74 | Palette name ->
75   (match Hashtbl.find_opt tm.palette name with
76   | Some v -> hex_to_rgba v
77   | None -> "/* UNRESOLVED_PALETTE */ { 0, 0, 0, 255 }")
78 | With_alpha (c, a) ->
79   let base = resolve_color_rgba tm c in
80   (* For alpha, we'd need to parse the struct -- just emit the base with
      a comment *)
81   Printf.sprintf "/* alpha %g */ %s" a base
82
83 (* --- C identifier sanitization --- *)
84
85 let sanitize_c_identifier name =
86   let name = String.map (fun c -> if c = '-' then '_' else c) name in
87   let name = String.map (fun c -> if c = ' ' then '_' else c) name in
88   (* Prefix reserved words *)
89   match name with
90   | "auto" | "break" | "case" | "char" | "const" | "continue"
91   | "default" | "do" | "double" | "else" | "enum" | "extern"
92   | "float" | "for" | "goto" | "if" | "int" | "long"
93   | "register" | "return" | "short" | "signed" | "sizeof"
94   | "static" | "struct" | "switch" | "typedef" | "union"
95   | "unsigned" | "void" | "volatile" | "while" ->
96     "ui_" ^ name
97   | _ -> name
98
99 (* --- Sizing to Clay C --- *)
100
101 let sizing_to_clay_width = function
102 | Flex n -> Printf.sprintf ".sizing.width = { CLAY_SIZING_GROW(%d),
      CLAY_SIZING_GROW(%d) }" n n
103 | Grow -> ".sizing.width = { CLAY_SIZING_GROW(0), CLAY_SIZING_GROW(0) }"
104 | Full -> ".sizing.width = CLAY_SIZING_PERCENT(1.0f)"
105 | Auto -> ".sizing.width = { CLAY_SIZING_FIT, CLAY_SIZING_FIT }"
106 | Px n -> Printf.sprintf ".sizing.width = { CLAY_SIZING_FIXED(%d),
      CLAY_SIZING_FIXED(%d) }" n n
107 | Rem n -> Printf.sprintf ".sizing.width = { CLAY_SIZING_FIXED(%d),
      CLAY_SIZING_FIXED(%d) }" (int_of_float (n *. 16.0)) (int_of_float (n
      *. 16.0))
108 | Pct n -> Printf.sprintf ".sizing.width = CLAY_SIZING_PERCENT(%g)"
      (float_of_int n /. 100.0)
109 | Vh n -> Printf.sprintf "/* vh not directly supported -- use FIXED */
      .sizing.width = { CLAY_SIZING_FIXED(%d), CLAY_SIZING_FIXED(%d) }" (n *
      8) (n * 8) (* rough estimation *)
110
111 let sizing_to_clay_height = function
112 | Flex n -> Printf.sprintf ".sizing.height = { CLAY_SIZING_GROW(%d),
      CLAY_SIZING_GROW(%d) }" n n
113 | Grow -> ".sizing.height = { CLAY_SIZING_GROW(0), CLAY_SIZING_GROW(0) }"
114 | Full -> ".sizing.height = CLAY_SIZING_PERCENT(1.0f)"
115 | Auto -> ".sizing.height = { CLAY_SIZING_FIT, CLAY_SIZING_FIT }"
116 | Px n -> Printf.sprintf ".sizing.height = { CLAY_SIZING_FIXED(%d),
      CLAY_SIZING_FIXED(%d) }" n n
117 | Rem n -> Printf.sprintf ".sizing.height = { CLAY_SIZING_FIXED(%d),
      CLAY_SIZING_FIXED(%d) }" (int_of_float (n *. 16.0)) (int_of_float (n
      *. 16.0))
118 | Pct n -> Printf.sprintf ".sizing.height = CLAY_SIZING_PERCENT(%g)"
      (float_of_int n /. 100.0)
119 | Vh n -> Printf.sprintf "/* vh */ .sizing.height = {
      CLAY_SIZING_FIXED(%d), CLAY_SIZING_FIXED(%d) }" (n * 8) (n * 8)
120
121 (* --- Spacing to Clay C --- *)
122
123 let resolve_spacing_c (tm : theme_map) = function
124 | Spacing_px n -> n
125 | Spacing_rem n -> int_of_float (n *. 16.0)

```

```

126 | Spacing_var name ->
127   (match Hashtbl.find_opt tm.spacing name with
128   | Some v -> v
129   | None -> 0)
130
131 let padding_to_clay (tm : theme_map) = function
132 | Padding_uniform s ->
133   let v = resolve_spacing_c tm s in
134   Printf.sprintf ".padding = { %d, %d, %d, %d }" v v v v
135 | Padding_sides { top; right; bottom; left } ->
136   Printf.sprintf ".padding = { %d, %d, %d, %d }"
137     (resolve_spacing_c tm top) (resolve_spacing_c tm right)
138     (resolve_spacing_c tm bottom) (resolve_spacing_c tm left)
139
140 (* --- Radius to Clay C --- *)
141
142 let resolve_radius_c (tm : theme_map) = function
143 | Radius_px n -> n
144 | Radius_var name ->
145   (match Hashtbl.find_opt tm.radius name with
146   | Some v -> v
147   | None -> 0)
148
149 let corner_to_clay (tm : theme_map) = function
150 | Corner_uniform r ->
151   let v = resolve_radius_c tm r in
152   Printf.sprintf ".cornerRadius = { %d, %d, %d, %d }" v v v v
153 | Corner_sides { top_left; top_right; bottom_left; bottom_right } ->
154   Printf.sprintf ".cornerRadius = { %d, %d, %d, %d }"
155     (resolve_radius_c tm top_left) (resolve_radius_c tm top_right)
156     (resolve_radius_c tm bottom_left) (resolve_radius_c tm bottom_right)
157
158 (* --- Alignment to Clay C --- *)
159
160 let align_x_to_clay = function
161 | Align_start -> "CLAY_ALIGN_X_LEFT"
162 | Align_center -> "CLAY_ALIGN_X_CENTER"
163 | Align_end -> "CLAY_ALIGN_X_RIGHT"
164
165 let align_y_to_clay = function
166 | Align_start -> "CLAY_ALIGN_Y_TOP"
167 | Align_center -> "CLAY_ALIGN_Y_CENTER"
168 | Align_end -> "CLAY_ALIGN_Y_BOTTOM"
169
170 (* --- Collect all Clay config fields from properties --- *)
171
172 let collect_clay_config (tm : theme_map) (props : ui_property list) =
173   let fields = ref [] in
174   List.iter (function
175 | P_layout Horizontal -> fields := ".layoutDirection =
176   CLAY_LEFT_TO_RIGHT" :: !fields
177 | P_layout Vertical -> fields := ".layoutDirection =
178   CLAY_TOP_TO_BOTTOM" :: !fields
179 | P_width s -> fields := "sizing_to_clay_width s :: !fields
180 | P_height s -> fields := "sizing_to_clay_height s :: !fields
181 | P_padding p -> fields := "padding_to_clay tm p :: !fields
182 | P_gap s -> fields := "Printf.sprintf ".childGap = %d"
183   (resolve_spacing_c tm s) :: !fields
184 | P_bg c -> fields := "Printf.sprintf ".backgroundColor = %s"
185   (resolve_color_rgba tm c) :: !fields
186 | P_color _ -> () (* handled in text elements *)
187 | P_border { width = Some w; color = Some c; _ } ->
188   fields := "Printf.sprintf ".border = { %d, %d, %d, %d }" w w w w ::
189   !fields;
190   fields := "Printf.sprintf ".borderColor = %s" (resolve_color_rgba tm
191   c) :: !fields
192 | P_border _ -> ()
193 | P_corner cr -> fields := "corner_to_clay tm cr :: !fields
194 | P_scroll Scroll_vertical -> fields := ".scroll = { .vertical = true
195   }" :: !fields
196 | P_scroll Scroll_horizontal -> fields := ".scroll = { .horizontal =
197   true }" :: !fields

```



```

190 | P_scroll Scroll_both -> fields := ".scroll = { .vertical = true,
    .horizontal = true }" :: !fields
191 | P_float _ -> fields := ".elementIsFloating = true" :: !fields
192 | P_interactive _ -> () (* handled via stub functions *)
193 | P_align_x a -> fields := Printf.sprintf ".childAlignment.x = %s"
    (align_x_to_clay a) :: !fields
194 | P_align_y a -> fields := Printf.sprintf ".childAlignment.y = %s"
    (align_y_to_clay a) :: !fields
195 ) props;
196 List.rev !fields
197
198 (* --- Element to Clay C --- *)
199
200 let rec element_to_clay (tm : theme_map) indent (el : ui_element) =
201   let indent_str = String.make indent ' ' in
202   (* Text content: emit CLAY_TEXT directly *)
203   match el.text with
204   | Some t ->
205     Printf.sprintf "%sCLAY_TEXT(%S, {});\n" indent_str t
206   | None ->
207     let config_fields = collect_clay_config tm el.properties in
208     let el_id = match el.name with Some n -> sanitize_c_identifier n | None
209       -> "element" in
210
211     let has_interactive = List.exists (function P_interactive _ -> true | _
212       -> false) el.properties in
213     let on_hover = if has_interactive then
214       Printf.sprintf "\n%s .onHover = handle_%s_hover," indent_str el_id
215     else "" in
216
217     let config_str = String.concat ("\n" ^ indent_str ^ " ") config_fields
218     in
219
220     let buf = Buffer.create 256 in
221
222     (* CLAY_CONTAINER open *)
223     Buffer.add_string buf (Printf.sprintf
224       "%sCLAY_CONTAINER(CLAY_IDI(%s), CLAY_LAYOUT_CONFIG({\n%s %s\n%s}),
225       {\n"
226       indent_str el_id indent_str config_str on_hover indent_str);
227
228     (* Children *)
229     List.iter (function
230       | El child -> Buffer.add_string buf (element_to_clay tm (indent + 4)
231         child)
232       | Slot_ref name -> Buffer.add_string buf (Printf.sprintf "%s/* slot: %s
233         */\n" indent_str name)
234     ) el.children;
235
236     (* CLAY_CONTAINER close *)
237     Buffer.add_string buf (Printf.sprintf "%s});\n" indent_str);
238
239     Buffer.contents buf
240
241 (* --- Component to C function --- *)
242
243 let component_to_clay (tm : theme_map) (comp : component) =
244   let fn_name = "render_" ^ sanitize_c_identifier comp.name in
245   let body = element_to_clay tm 4
246     { name = Some comp.name; properties = comp.properties;
247       children = comp.children; variants = comp.variants; text = None } in
248   Printf.sprintf "Clay_Children %s(void) {\n return (Clay_Children){\n%s
249     %s\n\n"
250     fn_name body
251
252 (* --- Layout to C function --- *)
253
254 let layout_to_clay (tm : theme_map) (layout : layout_def) =
255   let fn_name = "layout_" ^ sanitize_c_identifier layout.name in
256   let body = element_to_clay tm 4 layout.root in
257   Printf.sprintf "void %s(int screen_width, int screen_height) {\n%s\n\n"
258     fn_name body

```

```

253 (* --- Page to C function --- *)
254
255 let page_to_clay (_tm : theme_map) (pg : page) =
256   let fn_name = "layout_page_" ^ sanitize_c_identifier pg.name in
257   let layout_fn = "layout_" ^ sanitize_c_identifier pg.layout_name in
258   (* For now, just call the layout function. Slot filling would need
259      more sophisticated code generation -- emit the layout and add
260      comments where slot content goes. *)
261   let fill_comments = List.map (fun (fill : slot_fill) ->
262     Printf.sprintf " /* FILL: %s */\n" fill.slot_name
263   ) pg.fills in
264   let fills_str = String.concat "" fill_comments in
265   Printf.sprintf "void %s(int screen_width, int screen_height) {\n%s
266     %s(screen_width, screen_height);\n}\n\n"
267     fn_name fills_str layout_fn
268
269 (* --- Stub hover handlers for interactive elements --- *)
270
271 let collect_interactive_elements (app : ui_app) =
272   let result = ref [] in
273   let rec check_element (el : ui_element) =
274     List.iter (function
275       | P_interactive Interactive_click | P_interactive Interactive_hover ->
276         (match el.name with
277          | Some n -> result := n :: !result
278          | None -> ())
279       | _ -> ())
280     el.properties;
281   List.iter (function El child -> check_element child | Slot_ref _ -> ())
282     el.children
283   in
284   List.iter check_element app.elements;
285   List.iter (fun (comp : component) -> check_element
286     { name = Some comp.name; properties = comp.properties;
287       children = comp.children; variants = comp.variants; text = None })
288     app.components;
289   List.iter (fun layout -> check_element layout.root) app.layouts;
290   List.rev !result
291
292 (* --- Header file --- *)
293
294 let generate_header (app : ui_app) =
295   let buf = Buffer.create 1024 in
296   Buffer.add_string buf "/* Generated by borge generate --target clay-c
297     */\n";
298   Buffer.add_string buf "/* DO NOT EDIT: regenerate from spec */\n\n";
299   Buffer.add_string buf "#ifndef UI_APP_H\n#define UI_APP_H\n";
300   Buffer.add_string buf "#include \"clay.h\"\n\n";
301
302   (* Page enum *)
303   if app.routes <> [] then begin
304     Buffer.add_string buf "enum page {\n";
305     let routes = List.map (fun (r : route) ->
306       " PAGE_" ^ String.uppercase_ascii (sanitize_c_identifier r.page_name)
307     ) app.routes in
308     Buffer.add_string buf (String.concat ",\n" routes);
309     Buffer.add_string buf "\n};\n\n";
310     Buffer.add_string buf "extern enum page current_page;\n\n";
311   end;
312
313   (* Function declarations *)
314   List.iter (fun (comp : component) ->
315     Buffer.add_string buf (Printf.sprintf "Clay_Children
316       render_%s(void);\n" (sanitize_c_identifier comp.name))
317   ) app.components;
318   List.iter (fun (layout : layout_def) ->
319     Buffer.add_string buf (Printf.sprintf "void layout_%s(int screen_width,
320       int screen_height);\n" (sanitize_c_identifier layout.name))
321   ) app.layouts;
322   List.iter (fun (pg : page) ->
323     Buffer.add_string buf (Printf.sprintf "void layout_page_%s(int
324       screen_width, int screen_height);\n" (sanitize_c_identifier pg.name))
325   ) app.pages;

```

```

319   if app.routes <> [] then
320     Buffer.add_string buf "void layout_app(int screen_width, int
       screen_height);\n";
321
322   Buffer.add_string buf "\n#endif /* UI_APP_H */\n";
323   Buffer.contents buf
324
325   (* --- Main generation --- *)
326
327   let generate ?(source_path = "<spec>") (app : ui_app) =
328     let tm = build_theme_map app.theme in
329     let buf = Buffer.create 8192 in
330
331     (* Header *)
332     Buffer.add_string buf "/* Generated by borge generate --target clay-c
       */\n";
333     Buffer.add_string buf (Printf.sprintf "/* Source: %s */\n" source_path);
334     Buffer.add_string buf "/* DO NOT EDIT: regenerate from spec */\n";
335     Buffer.add_string buf "#include \"ui_app.h\"\n";
336
337     (* Page enum global *)
338     if app.routes <> [] then begin
339       Buffer.add_string buf "enum page current_page = PAGE_";
340       (match app.routes with r :: _ ->
341         Buffer.add_string buf (String.uppercase_ascii (sanitize_c_identifier
342           r.page_name))
343         | [] -> ());
344       Buffer.add_string buf ";\n";
345     end;
346
347     (* Stub hover handlers *)
348     let interactive_els = collect_interactive_elements app in
349     if interactive_els <> [] then begin
350       Buffer.add_string buf "/* Interactive handlers -- fill in actual
351         behavior */\n";
352       List.iter (fun el_name ->
353         let id = sanitize_c_identifier el_name in
354         Buffer.add_string buf (Printf.sprintf
355           "void handle_%s_hover(Clay_HoverState state) {\n  /* TODO:
356             implement click/hover for %s */\n}\n"
357           id el_name)
358         ) interactive_els;
359     end;
360
361     (* Components *)
362     if app.components <> [] then begin
363       Buffer.add_string buf "/* Components */\n";
364       List.iter (fun comp ->
365         Buffer.add_string buf (component_to_clay tm comp)
366       ) app.components;
367     end;
368
369     (* Layouts *)
370     if app.layouts <> [] then begin
371       Buffer.add_string buf "/* Layouts */\n";
372       List.iter (fun layout ->
373         Buffer.add_string buf (layout_to_clay tm layout)
374       ) app.layouts;
375     end;
376
377     (* Pages *)
378     if app.pages <> [] then begin
379       Buffer.add_string buf "/* Pages */\n";
380       List.iter (fun (pg : Ui_ast.page) ->
381         Buffer.add_string buf (page_to_clay tm pg)
382       ) app.pages;
383     end;
384
385     (* Top-level layout switch from routes *)
386     if app.routes <> [] then begin
387       Buffer.add_string buf "/* Route dispatcher */\n";
388       Buffer.add_string buf "void layout_app(int screen_width, int
389         screen_height) {\n";

```

```
386   Buffer.add_string buf "  switch (current_page) {\n";
387   List.iter (fun (r : route) ->
388     let page_id = String.uppercase_ascii (sanitize_c_identifier
389       r.page_name) in
389     Buffer.add_string buf (Printf.sprintf "    case PAGE_%s:
390       layout_page_%s(screen_width, screen_height); break;\n"
391         page_id (sanitize_c_identifier r.page_name))
391   ) app.routes;
392   Buffer.add_string buf "  }\n}\n";
393 end;
394
395 (* Elements -- top-level, not in a component/layout/page *)
396 if app.elements <> [] then begin
397   Buffer.add_string buf "/* Elements */\n\n";
398   List.iter (fun (el : Ui_ast.ui_element) ->
399     Buffer.add_string buf (element_to_clay tm 0 el)
400   ) app.elements;
401 end;
402
403 Buffer.contents buf
404
405 let generate_header_file = generate_header
```

29 lib/ui/ui_dream.ml

```

1  (*| dream_html OCaml generator for borge UI specs.
2
3     Walks the typed UI AST and produces an OCaml module that
4     constructs the full UI as dream_html nodes. The output is
5     a .ml file that can be compiled into any Dream application.
6     See ui.borg section convention-targets / ocaml-dream-target for
7     the full mapping specification. |*)
8
9  open Ui_ast
10
11  (* --- Theme resolution --- *)
12
13  type theme_map = {
14    palette : (string, string) Hashtbl.t;   (* name -> hex value *)
15    spacing : (string, int) Hashtbl.t;      (* name -> px value *)
16    font_size : (string, int) Hashtbl.t;    (* name -> px value *)
17    radius : (string, int) Hashtbl.t;       (* name -> px value *)
18  }
19
20  let empty_theme = {
21    palette = Hashtbl.create 0;
22    spacing = Hashtbl.create 0;
23    font_size = Hashtbl.create 0;
24    radius = Hashtbl.create 0;
25  }
26
27  let build_theme_map (t : theme option) =
28    match t with
29    | None -> empty_theme
30    | Some theme ->
31      let tm = {
32        palette = Hashtbl.create 16;
33        spacing = Hashtbl.create 16;
34        font_size = Hashtbl.create 16;
35        radius = Hashtbl.create 16;
36      } in
37      List.iter (function
38        | Palette_entry (name, value) -> Hashtbl.add tm.palette name value
39        | Spacing_entry (name, value) -> Hashtbl.add tm.spacing name value
40        | Font_size_entry (name, value) -> Hashtbl.add tm.font_size name value
41        | Radius_entry (name, value) -> Hashtbl.add tm.radius name value
42      ) theme.entries;
43      tm
44
45  (* Keyword color defaults *)
46  let keyword_to_hex = function
47    | "red" -> "#e04040"
48    | "orange" -> "#e08040"
49    | "yellow" -> "#e0c040"
50    | "green" -> "#40c060"
51    | "blue" -> "#4080e0"
52    | "purple" -> "#8040e0"
53    | "pink" -> "#e040a0"
54    | "white" -> "#ffffff"
55    | "black" -> "#000000"
56    | "gray" -> "#808080"
57    | "dark" -> "#282828"
58    | "light" -> "#e6e6e6"
59    | "transparent" -> "#00000000"
60    | _ -> "#888888" (* fallback gray *)
61
62  (* Resolve a color to a hex string *)
63  let rec resolve_color (tm : theme_map) = function
64    | Hex h -> h
65    | Keyword k -> keyword_to_hex k
66    | Palette name ->
67      (match Hashtbl.find_opt tm.palette name with
68      | Some v -> v
69      | None -> Printf.sprintf "/* UNRESOLVED PALETTE: %s */" name)
70    | With_alpha (c, a) ->
71      let hex = resolve_color tm c in

```

```

72 (* Convert hex + alpha to rgba *)
73 if String.length hex >= 7 then
74   match int_of_string_opt ("0x" ^ (* exempt: String.sub *) String.sub
75     hex 1 2),
76     int_of_string_opt ("0x" ^ (* exempt: String.sub *) String.sub
77       hex 3 2),
78     int_of_string_opt ("0x" ^ (* exempt: String.sub *) String.sub
79       hex 5 2) with
80   | Some r, Some g, Some b -> Printf.sprintf "rgba(%d, %d, %d, %g)" r g
81     b a
82   | _ -> hex
83   else hex
84
85 (* Resolve a spacing value to a CSS string *)
86 let resolve_spacing (tm : theme_map) = function
87   | Spacing_px n -> Printf.sprintf "%dpx" n
88   | Spacing_rem n -> Printf.sprintf "%grem" n
89   | Spacing_var name ->
90     (match Hashtbl.find_opt tm.spacing name with
91      | Some v -> Printf.sprintf "%dpx" v
92      | None -> Printf.sprintf "/* UNRESOLVED SPACING: %s */" name)
93
94 (* Resolve a font size to a CSS string *)
95 let resolve_font_size (tm : theme_map) = function
96   | Font_px n -> Printf.sprintf "%dpx" n
97   | Font_rem n -> Printf.sprintf "%grem" n
98   | Font_var name ->
99     (match Hashtbl.find_opt tm.font_size name with
100    | Some v -> Printf.sprintf "%dpx" v
101    | None -> Printf.sprintf "/* UNRESOLVED FONT-SIZE: %s */" name)
102
103 (* Resolve a radius value to a CSS string *)
104 let resolve_radius (tm : theme_map) = function
105   | Radius_px n -> Printf.sprintf "%dpx" n
106   | Radius_var name ->
107     (match Hashtbl.find_opt tm.radius name with
108    | Some v -> Printf.sprintf "%dpx" v
109    | None -> Printf.sprintf "/* UNRESOLVED RADIUS: %s */" name)
110
111 (* --- CSS property generation --- *)
112
113 let sizing_to_css = function
114   | Flex n -> Printf.sprintf "flex: %d" n
115   | Grow -> "flex: 1 1 0%"
116   | Full -> "width: 100%"
117   | Auto -> ""
118   | Px n -> Printf.sprintf "width: %dpx" n
119   | Rem n -> Printf.sprintf "width: %grem" n
120   | Pct n -> Printf.sprintf "width: %d%" n
121   | Vh n -> Printf.sprintf "height: %dvh" n
122
123 let sizing_to_css_height = function
124   | Flex n -> Printf.sprintf "flex: %d" n
125   | Grow -> "flex: 1 1 0%"
126   | Full -> "height: 100%"
127   | Auto -> ""
128   | Px n -> Printf.sprintf "height: %dpx" n
129   | Rem n -> Printf.sprintf "height: %grem" n
130   | Pct n -> Printf.sprintf "height: %d%" n
131   | Vh n -> Printf.sprintf "height: %dvh" n
132
133 let padding_to_css (tm : theme_map) = function
134   | Padding_uniform s -> Printf.sprintf "padding: %s" (resolve_spacing tm s)
135   | Padding_sides { top; right; bottom; left } ->
136     Printf.sprintf "padding: %s %s %s %s"
137       (resolve_spacing tm top) (resolve_spacing tm right)
138       (resolve_spacing tm bottom) (resolve_spacing tm left)
139
140 let corner_to_css (tm : theme_map) = function
141   | Corner_uniform r -> Printf.sprintf "border-radius: %s" (resolve_radius
142     tm r)
143   | Corner_sides { top_left; top_right; bottom_left; bottom_right } ->
144     Printf.sprintf "border-radius: %s %s %s %s"

```

```

140     (resolve_radius tm top_left) (resolve_radius tm top_right)
141     (resolve_radius tm bottom_left) (resolve_radius tm bottom_right)
142
143 let alignment_to_css_x = function
144 | Align_start -> "items-start"
145 | Align_center -> "items-center"
146 | Align_end -> "items-end"
147
148 let alignment_to_css_y = function
149 | Align_start -> "justify-start"
150 | Align_center -> "justify-center"
151 | Align_end -> "justify-end"
152
153 let scroll_to_css = function
154 | Scroll_vertical -> "overflow-y: auto"
155 | Scroll_horizontal -> "overflow-x: auto"
156 | Scroll_both -> "overflow: auto"
157
158 (* --- Collect styles from properties --- *)
159
160 let collect_styles (tm : theme_map) (props : ui_property list) =
161   let styles = ref [] in
162   let classes = ref [] in
163   List.iter (function
164 | P_layout Horizontal -> classes := "flex-row" :: !classes
165 | P_layout Vertical -> classes := "flex-col" :: !classes
166 | P_width s ->
167   (match sizing_to_css s with "" -> () | css -> styles := css ::
168   !styles)
169 | P_height s ->
170   (match sizing_to_css_height s with "" -> () | css -> styles := css ::
171   !styles)
172 | P_padding p -> styles := padding_to_css tm p :: !styles
173 | P_gap s -> styles := Printf.sprintf "gap: %s" (resolve_spacing tm s)
174   :: !styles
175 | P_bg c -> styles := Printf.sprintf "background-color: %s"
176   (resolve_color tm c) :: !styles
177 | P_color c -> styles := Printf.sprintf "color: %s" (resolve_color tm
178   c) :: !styles
179 | P_border { width = Some w; color = Some c; side } ->
180   (match side with
181   | Some s -> styles := Printf.sprintf "border-%s: %dpx solid %s" s w
182     (resolve_color tm c) :: !styles
183   | None -> styles := Printf.sprintf "border: %dpx solid %s" w
184     (resolve_color tm c) :: !styles)
185 | P_border { width = Some w; color = None; side } ->
186   (match side with
187   | Some s -> styles := Printf.sprintf "border-%s: %dpx solid" s w ::
188   !styles
189   | None -> styles := Printf.sprintf "border: %dpx solid" w :: !styles)
190 | P_border _ -> ()
191 | P_corner cr -> styles := corner_to_css tm cr :: !styles
192 | P_scroll d -> styles := scroll_to_css d :: !styles
193 | P_float Float_simple -> styles := "position: absolute" :: !styles
194 | P_float Float_attach_parent -> styles := "position: relative" ::
195   !styles
196 | P_float (Float_offset offsets) ->
197   styles := "position: absolute" :: !styles;
198   List.iter (fun (dir, n) ->
199     styles := Printf.sprintf "%s: %dpx" dir n :: !styles
200   ) offsets
201 | P_interactive _ -> () (* handled separately *)
202 | P_align_x a -> classes := alignment_to_css_x a :: !classes
203 | P_align_y a -> classes := alignment_to_css_y a :: !classes
204 ) props;
205 (List.rev !classes, List.rev !styles)
206
207 (* --- CSS custom properties from theme --- *)
208
209 let theme_to_css_vars (tm : theme_map) =
210   let buf = Buffer.create 256 in
211   Hashtbl.iter (fun name value ->
212     Buffer.add_string buf (Printf.sprintf " --%s: %s;\n" name value)

```

```

204   ) tm.palette;
205   Hashtbl.iter (fun name value ->
206     Buffer.add_string buf (Printf.sprintf " --spacing-%s: %dpx;\n" name
207       value)
208   ) tm.spacing;
209   Hashtbl.iter (fun name value ->
210     Buffer.add_string buf (Printf.sprintf " --font-size-%s: %dpx;\n" name
211       value)
212   ) tm.font_size;
213   Hashtbl.iter (fun name value ->
214     Buffer.add_string buf (Printf.sprintf " --radius-%s: %dpx;\n" name
215       value)
216   ) tm.radius;
217   if Buffer.length buf > 0 then
218     Printf.sprintf "<style>\n:root {\n%s}\n</style>\n" (Buffer.contents buf)
219   else ""
220
221   (* --- Style merging: deduplicate CSS properties --- *)
222
223   (*| Merge multiple CSS style strings by splitting on ',' and keeping
224     the last declaration for each property name. This prevents
225     invalid HTML from duplicate style attributes. |*)
226
227   let merge_css_styles (styles : string list) =
228     let seen = Hashtbl.create 16 in
229     (* Process in order -- later values override earlier ones *)
230     List.iter (fun s ->
231       let parts = String.split_on_char ',' s in
232       List.iter (fun part ->
233         let part = String.trim part in
234         if part <> "" then begin
235           match String.split_on_char ':' part with
236           | [key; value] ->
237             Hashtbl.replace seen (String.trim key) (String.trim value)
238           | _ -> () (* skip malformed *)
239         end
240       ) parts
241     ) styles;
242     let result = Hashtbl.fold (fun key value acc ->
243       (key ^ ": " ^ value) :: acc
244     ) seen [] in
245     String.concat "; " (List.sort String.compare result)
246
247   (* --- Variant ? @media blocks --- *)
248
249   let default_breakpoint = 768
250
251   let sanitize_name = function
252     | None -> "el"
253     | Some name ->
254       String.map (fun c -> if c = '-' then '_' else c) name
255
256   let variant_to_media_query (tm : theme_map) (el_name : string) (v :
257     variant) =
258     let v_classes, v_styles = collect_styles tm v.properties in
259     let class_str = String.concat " " v_classes in
260     let style_str = merge_css_styles v_styles in
261     let el_class = sanitize_name (Some el_name) in
262     let width = default_breakpoint in
263     (* Wide/narrow heuristic: "wide" or "expanded" = desktop, others = mobile
264       *)
265     let is_wide = List.exists (fun s ->
266       List.mem s ["wide"; "expanded"; "desktop"; "lg"; "xl"]
267     ) [v.name] in
268     let condition =
269       if is_wide then Printf.sprintf "min-width: %dpx" width
270       else Printf.sprintf "max-width: %dpx" (width - 1)
271     in
272     let rules = ref [] in
273     if class_str <> "" then
274       rules := Printf.sprintf ".%s.variant-%s { %s }" el_class v.name
275       class_str :: !rules;
276     if style_str <> "" then

```



```

271     rules := Printf.sprintf "%.s.variant-%s { %s }" el_class v.name
272     style_str :: !rules;
273   if !rules = [] then ""
274   else
275     Printf.sprintf "@media (%s) {\n  %s\n}\n" condition (String.concat "\n"
276       (List.rev !rules))
277
278 let collect_variant_styles (tm : theme_map) (el : ui_element) =
279   List.filter_map (fun (v : variant) ->
280     match el.name with
281     | Some n -> Some (variant_to_media_query tm n v)
282     | None -> None
283   ) el.variants
284   |> List.filter (fun s -> s <> "")
285
286 (* --- Element to dream_html --- *)
287
288 let has_click_interactive (props : ui_property list) =
289   List.exists (function P_interactive Interactive_click -> true | _ ->
290     false) props
291
292 let rec element_to_dream (tm : theme_map) (el : ui_element) =
293   (* Text content: emit txt node directly *)
294   match el.text with
295   | Some t ->
296     Printf.sprintf "txt %S" t
297   | None ->
298     let _name = sanitize_name el.name in
299     let classes, styles = collect_styles tm el.properties in
300     let has_click = has_click_interactive el.properties in
301
302     (* Build attributes list *)
303     let attr_parts = ref [] in
304     if classes <> [] then
305       attr_parts := Printf.sprintf "class_ \"%s\"\"
306         (String.concat " " classes) :: !attr_parts;
307     if styles <> [] then
308       attr_parts := Printf.sprintf "style_ \"%s\" \"\"
309         (merge_css_styles styles) :: !attr_parts;
310     if has_click then
311       attr_parts := "onclick \"/ * TODO: click handler */\" :: !attr_parts;
312
313     (* Build children -- slots are node list, elements are node,
314       so we build segments and splice with @ *)
315     let has_slots = List.exists (function Slot_ref _ -> true | _ -> false)
316       el.children in
317     let attrs_str = String.concat "; " (List.rev !attr_parts) in
318
319     if not has_slots then begin
320       (* Simple case: no slots, all elements *)
321       let children_code = List.filter_map (function
322         | El el -> Some (element_to_dream tm el)
323         | Slot_ref _ -> None
324       ) el.children in
325       let children_str = String.concat "; " children_code in
326       if attrs_str = "" && children_str = "" then
327         Printf.sprintf "div [] []"
328       else if attrs_str = "" then
329         Printf.sprintf "div [] [%s]" children_str
330       else if children_str = "" then
331         Printf.sprintf "div [%s] []" attrs_str
332       else
333         Printf.sprintf "div [%s] [%s]" attrs_str children_str
334     end else begin
335       (* Slots present: build spliced expression ([el1] @ nav @ [el2]) *)
336       let segments = List.filter_map (function
337         | El el ->
338           let code = element_to_dream tm el in
339           Some (Printf.sprintf "[%s]" code)
340         | Slot_ref name ->
341           Some (String.map (fun c -> if c = '-' then '_' else c) name)
342       ) el.children in

```

```

340     let children_expr = String.concat " @ " segments in
341     let children_expr = if children_expr = "" then "[]" else children_expr
342     in
343     if attrs_str = "" then
344       Printf.sprintf "div [] %s" children_expr
345     else
346       Printf.sprintf "div [%s] %s" attrs_str children_expr
347     end
348 (* --- Component to render function --- *)
349
350 let component_to_dream (tm : theme_map) (comp : component) =
351   let fn_name = "render_" ^ String.map (fun c -> if c = '-' then '_' else c) comp.name in
352   let body = element_to_dream tm
353     { name = Some comp.name; properties = comp.properties;
354       children = comp.children; variants = comp.variants; text = None } in
355   Printf.sprintf "let %s ~(_content : node list) () : node =\n %s\n"
356     fn_name body
357
358 (* --- Layout to render function --- *)
359
360 let layout_to_dream (tm : theme_map) (layout : layout_def) =
361   let fn_name = "render_" ^ String.map (fun c -> if c = '-' then '_' else c) layout.name ^ "_layout" in
362   let body = element_to_dream tm layout.root in
363   (* Build slot parameters from the layout's slot list *)
364   let slot_params = List.map (function
365     | Named_slot name -> Printf.sprintf "~(%s : node list)" (String.map
366       (fun c -> if c = '-' then '_' else c) name)
367     | Default_slot -> "~(content : node list)"
368   ) layout.slots
369   in
370   let params_str = String.concat " " slot_params in
371   Printf.sprintf "let %s %s () : node =\n %s\n"
372     fn_name params_str body
373
374 (* --- Page to render function --- *)
375
376 let page_to_dream (tm : theme_map) (_app : ui_app) (pg : page) =
377   let fn_name = "render_" ^ String.map (fun c -> if c = '-' then '_' else c) pg.name in
378   let layout_fn = "render_" ^ String.map (fun c -> if c = '-' then '_' else c) pg.layout_name ^ "_layout" in
379   (* Build named fill arguments from page fills *)
380   let fill_args = List.map (fun (fill : slot_fill) ->
381     let content = List.map (element_to_dream tm) fill.content in
382     let content_str = String.concat "; " content in
383     let slot_name = String.map (fun c -> if c = '-' then '_' else c)
384       fill.slot_name in
385     Printf.sprintf "~%s:[%s]" slot_name content_str
386   ) pg.fills in
387   let fill_str = String.concat " " fill_args in
388   Printf.sprintf "let %s () : node =\n %s %s ()\n"
389     fn_name layout_fn fill_str
390
391 (* --- Route to Dream handler --- *)
392
393 let route_to_dream (route : route) =
394   let page_fn = "render_" ^ String.map (fun c -> if c = '-' then '_' else c) route.page_name in
395   Printf.sprintf " Dream.get %S (fun _ -> Dream.html.respond (%s ()))"
396     route.path page_fn
397
398 (* --- Main generation --- *)
399
400 let generate ?(source_path = "<spec>") (app : ui_app) =
401   let tm = build_theme_map app.theme in
402   let buf = Buffer.create 4096 in
403
404   (* Header *)
405   Buffer.add_string buf (Printf.sprintf
406     "(* Generated by borge generate --target ocaml-dream *)\n\n"

```

```

405     (* Source: %s *)\n\
406     (* DO NOT EDIT: regenerate from spec *)\n\n\
407     open Dream_html\nopen HTML\n\n" source_path);
408
409     (* CSS custom properties from theme *)
410     let css_vars = theme_to_css_vars tm in
411     if css_vars <> "" then
412       Buffer.add_string buf (Printf.sprintf "let theme_css = %S\n\n"
413         css_vars);
414
415     (* Variant media queries from all elements *)
416     let rec collect_variants_from_element (el : ui_element) =
417       collect_variant_styles tm el @ List.concat_map (function El child ->
418         collect_variants_from_element child | Slot_ref _ -> []) el.children
419     in
420     let all_variants =
421       List.concat_map collect_variants_from_element app.elements
422       @ List.concat_map (fun (comp : component) ->
423         collect_variants_from_element
424           { name = Some comp.name; properties = comp.properties;
425             children = comp.children; variants = comp.variants; text = None
426           }) app.components
427       @ List.concat_map (fun (layout : layout_def) ->
428         collect_variants_from_element layout.root) app.layouts
429     in
430     if all_variants <> [] then begin
431       Buffer.add_string buf "(* Variant media queries *)\n\n";
432       Buffer.add_string buf (Printf.sprintf "let variant_css = %S\n\n"
433         (String.concat "\n" all_variants));
434     end;
435
436     (* Components *)
437     if app.components <> [] then begin
438       Buffer.add_string buf "(* Components *)\n\n";
439       List.iter (fun comp ->
440         Buffer.add_string buf (component_to_dream tm comp);
441         Buffer.add_char buf '\n';
442       ) app.components;
443     end;
444
445     (* Layouts *)
446     if app.layouts <> [] then begin
447       Buffer.add_string buf "(* Layouts *)\n\n";
448       List.iter (fun layout ->
449         Buffer.add_string buf (layout_to_dream tm layout);
450         Buffer.add_char buf '\n';
451       ) app.layouts;
452     end;
453
454     (* Pages *)
455     if app.pages <> [] then begin
456       Buffer.add_string buf "(* Pages *)\n\n";
457       List.iter (fun pg ->
458         Buffer.add_string buf (page_to_dream tm app pg);
459         Buffer.add_char buf '\n';
460       ) app.pages;
461     end;
462
463     (* Routes *)
464     if app.routes <> [] then begin
465       Buffer.add_string buf "(* Routes *)\n\n";
466       Buffer.add_string buf "let register_routes () = Dream.router [\n";
467       List.iter (fun r ->
468         Buffer.add_string buf (route_to_dream r);
469         Buffer.add_string buf ";\n";
470       ) app.routes;
471       Buffer.add_string buf "]\n";
472     end;
473
474     (* Elements -- top-level, not in a component/layout/page *)
475     if app.elements <> [] then begin
476       Buffer.add_string buf "(* Elements *)\n\n";
477       List.iter (fun (el : ui_element) ->

```

```
475     let el_name = match el.name with Some n -> String.map (fun c -> if c
476 = '-' then '_' else c) n | None -> "el" in
476     Buffer.add_string buf (Printf.sprintf "let _el_%s : node = %s\n"
477     el_name (element_to_dream tm el));
477 ) app.elements;
478 end;
479
480 Buffer.contents buf
```

30 lib/ui/ui_json.ml

```

1  (** JSON serialization for borge UI specs.
2
3      Produces Yojson-compatible JSON from the typed UI AST.
4      Used by the --json flag on UI-related commands. *)
5
6  open Ui_ast
7
8  (* exempt doc: simple JSON converter pattern - name is self-documenting *)
9  let sizing_to_json = function
10 | Flex n -> 'Assoc [("flex", 'Int n)]
11 | Grow -> 'String "grow"
12 | Full -> 'String "full"
13 | Auto -> 'String "auto"
14 | Px n -> 'Assoc [("px", 'Int n)]
15 | Rem n -> 'Assoc [("rem", 'Float n)]
16 | Pct n -> 'Assoc [("pct", 'Int n)]
17 | Vh n -> 'Assoc [("vh", 'Int n)]
18
19 (* exempt doc: simple recursive JSON converter - name is self-documenting *)
20 let rec color_to_json = function
21 | Hex s -> 'Assoc [("hex", 'String s)]
22 | Keyword s -> 'Assoc [("keyword", 'String s)]
23 | Palette s -> 'Assoc [("palette", 'String s)]
24 | With_alpha (c, a) -> 'Assoc [("color", color_to_json c); ("alpha",
    'Float a)]
25
26 (* exempt doc: simple JSON converter - name is self-documenting *)
27 let spacing_to_json = function
28 | Spacing_px n -> 'Int n
29 | Spacing_rem n -> 'Float n
30 | Spacing_var s -> 'Assoc [("spacing", 'String s)]
31
32 (* exempt doc: simple JSON converter - name is self-documenting *)
33 let theme_entry_to_json = function
34 | Palette_entry (name, value) -> 'Assoc [("palette", 'String name);
    ("value", 'String value)]
35 | Spacing_entry (name, value) -> 'Assoc [("spacing", 'String name);
    ("value", 'Int value)]
36 | Font_size_entry (name, value) -> 'Assoc [("font-size", 'String name);
    ("value", 'Int value)]
37 | Radius_entry (name, value) -> 'Assoc [("radius", 'String name);
    ("value", 'Int value)]
38
39 (* exempt doc: simple JSON converter - name is self-documenting *)
40 let padding_to_json = function
41 | Padding_uniform s -> 'Assoc [("uniform", spacing_to_json s)]
42 | Padding_sides { top; right; bottom; left } ->
    'Assoc [("top", spacing_to_json top); ("right", spacing_to_json right);
    ("bottom", spacing_to_json bottom); ("left", spacing_to_json
    left)]
43
44 (* exempt doc: simple JSON converter - name is self-documenting *)
45 let border_spec_to_json (b : Ui_ast.border_spec) =
46   'Assoc ((match b.width with Some w -> [("width", 'Int w)] | None -> [])
47     @ (match b.color with Some c -> [("color", color_to_json c)] |
48       None -> [])
49     @ (match b.side with Some s -> [("side", 'String s)] | None ->
50       []))
51
52 (* exempt doc: simple JSON converter - name is self-documenting *)
53 let radius_to_json = function
54 | Radius_px n -> 'Int n
55 | Radius_var s -> 'Assoc [("radius", 'String s)]
56
57 (* exempt doc: simple JSON converter - name is self-documenting *)
58 let corner_radius_to_json = function
59 | Corner_uniform r -> 'Assoc [("uniform", radius_to_json r)]
60 | Corner_sides { top_left; top_right; bottom_left; bottom_right } ->
    'Assoc [("top_left", radius_to_json top_left); ("top_right",
    radius_to_json top_right);

```

```

62         ("bottom_left", radius_to_json bottom_left); ("bottom_right",
        radius_to_json bottom_right)]
63
64 (* exempt doc: recursive JSON converter - name is self-documenting *)
65 let rec variant_to_json (v : Ui_ast.variant) =
66   'Assoc [
67     ("name", 'String v.name);
68     ("properties", 'List (List.map property_to_json v.properties));
69     ("children", match v.children with
70       | Some cs -> 'List (List.map ui_element_to_json cs)
71       | None -> 'Null);
72   ]
73
74 (* exempt doc: pattern match to JSON - name is self-documenting *)
75 and property_to_json = function
76   | P_layout d -> 'Assoc [("layout", 'String (match d with Horizontal ->
        "horizontal" | Vertical -> "vertical"))]
77   | P_width s -> 'Assoc [("width", sizing_to_json s)]
78   | P_height s -> 'Assoc [("height", sizing_to_json s)]
79   | P_padding p -> 'Assoc [("padding", padding_to_json p)]
80   | P_gap s -> 'Assoc [("gap", spacing_to_json s)]
81   | P_bg c -> 'Assoc [("bg", color_to_json c)]
82   | P_color c -> 'Assoc [("color", color_to_json c)]
83   | P_border b -> 'Assoc [("border", border_spec_to_json b)]
84   | P_corner c -> 'Assoc [("corner", corner_radius_to_json c)]
85   | P_scroll d -> 'Assoc [("scroll", 'String (match d with Scroll_vertical
        -> "vertical" | Scroll_horizontal -> "horizontal" | Scroll_both ->
        "both"))]
86   | P_float _ -> 'Assoc [("float", 'String "simple")]
87   | P_interactive t -> 'Assoc [("interactive", 'String (match t with
        Interactive_click -> "click" | Interactive_type -> "type" |
        Interactive_hover -> "hover"))]
88   | P_align_x a -> 'Assoc [("align-x", 'String (match a with Align_start ->
        "start" | Align_center -> "center" | Align_end -> "end"))]
89   | P_align_y a -> 'Assoc [("align-y", 'String (match a with Align_start ->
        "start" | Align_center -> "center" | Align_end -> "end"))]
90
91 (* exempt doc: recursive JSON converter - name is self-documenting *)
92 and ui_element_to_json el =
93   let props = 'List (List.map property_to_json el.properties) in
94   let children = 'List (List.map (function El el -> ui_element_to_json el |
        Slot_ref name -> 'Assoc [("slot", 'String name)]) el.children) in
95   let variants = 'List (List.map variant_to_json el.variants) in
96   'Assoc [
97     ("name", match el.name with Some n -> 'String n | None -> 'Null);
98     ("properties", props);
99     ("children", children);
100    ("variants", variants);
101    ("text", match el.text with Some t -> 'String t | None -> 'Null);
102  ]
103
104 (* exempt doc: JSON converter for component - name is self-documenting *)
105 let component_to_json (c : Ui_ast.component) =
106   'Assoc [
107     ("name", 'String c.name);
108     ("properties", 'List (List.map property_to_json c.properties));
109     ("slots", 'List (List.map (function Default_slot -> 'String "..content"
        | Named_slot s -> 'Assoc [("slot", 'String s)]) c.slots));
110     ("variants", 'List (List.map variant_to_json c.variants));
111   ]
112
113 (* exempt doc: JSON converter for layout - name is self-documenting *)
114 let layout_to_json (ly : Ui_ast.layout_def) =
115   'Assoc [
116     ("name", 'String ly.name);
117     ("root", ui_element_to_json ly.root);
118     ("slots", 'List (List.map (function Default_slot -> 'String "..content"
        | Named_slot s -> 'Assoc [("slot", 'String s)]) ly.slots));
119   ]
120
121 (* exempt doc: JSON converter for page - name is self-documenting *)
122 let page_to_json (p : Ui_ast.page) =
123   'Assoc [

```

```

124     ("name", 'String p.name);
125     ("layout", 'String p.layout_name);
126     ("fills", 'List (List.map (fun (f : Ui_ast.slot_fill) ->
127       'Assoc [( "slot", 'String f.slot_name); ("content", 'List (List.map
        ui_element_to_json f.content))]
        ) p.fills));
128   ]
129 ]
130
131 (* exempt doc: JSON converter for route - name is self-documenting *)
132 let route_to_json (r : Ui_ast.route) =
133   'Assoc [( "path", 'String r.path); ("page", 'String r.page_name)]
134
135 (* exempt doc: JSON converter for action - name is self-documenting *)
136 let action_to_json (a : Ui_ast.action) =
137   match a with
138   | Click name -> 'Assoc [( "click", 'String name)]
139   | Type name -> 'Assoc [( "type", 'String name)]
140   | Hover name -> 'Assoc [( "hover", 'String name)]
141
142 (* exempt doc: JSON converter for outcome - name is self-documenting *)
143 let outcome_to_json (o : Ui_ast.outcome) =
144   match o with
145   | Navigate path -> 'Assoc [( "navigate", 'String path)]
146   | Submit -> 'String "submit"
147   | No_outcome -> 'Null
148
149 (* exempt doc: JSON converter for step - name is self-documenting *)
150 let step_to_json (s : Ui_ast.step) =
151   'Assoc [
152     ("name", 'String s.name);
153     ("page", 'String s.page_name);
154     ("actions", 'List (List.map action_to_json s.actions));
155     ("outcome", outcome_to_json s.outcome);
156   ]
157
158 (* exempt doc: JSON converter for workflow - name is self-documenting *)
159 let workflow_to_json (wf : Ui_ast.workflow) =
160   'Assoc [
161     ("name", 'String wf.name);
162     ("doc", match wf.doc with Some d -> 'String d | None -> 'Null);
163     ("steps", 'List (List.map step_to_json wf.steps));
164   ]
165
166 (* exempt doc: JSON converter for full app - name is self-documenting *)
167 let ui_app_to_json app =
168   let theme = match app.theme with
169   | Some t -> 'Assoc [( "entries", 'List (List.map theme_entry_to_json
        t.entries))]
170   | None -> 'Null
171   in
172   'Assoc [
173     ("theme", theme);
174     ("components", 'List (List.map component_to_json app.components));
175     ("layouts", 'List (List.map layout_to_json app.layouts));
176     ("pages", 'List (List.map page_to_json app.pages));
177     ("routes", 'List (List.map route_to_json app.routes));
178     ("elements", 'List (List.map ui_element_to_json app.elements));
179     ("workflows", 'List (List.map workflow_to_json app.workflows));
180   ]

```

31 lib/ui/ui_parse.ml

```

1  (** Extract typed UI AST from generic sexp trees.
2
3      Walks the Ast.sexp tree produced by the borge parser and
4      extracts UI-specific typed nodes. The parser stays generic;
5      this module interprets the semantic structure of (ui ...),
6      (theme ...), (component ...), (layout ...), (page ...),
7      and (routes ...) forms. *)
8
9  open Ui_ast
10 open Borge_lang.Ast
11
12 (** Helpers for working with the sexp tree *)
13
14 let atom_name = function
15   | Atom (_, name) -> Some name
16   | _ -> None
17
18 let string_value = function
19   | String (_, Quoted { q_content }) -> Some q_content
20   | String (_, Verbatim { v_content }) -> Some v_content
21   | _ -> None
22
23 (** Find all children of a list form matching a keyword *)
24 let find_forms keyword sexps =
25   List.filter_map (fun swc ->
26     match swc.node with
27     | List (_, Atom (_, k) :: rest) when k = keyword ->
28       Some (k, rest, swc)
29     | _ -> None
30   ) sexps
31
32 (** Find first form matching a keyword *)
33 let find_form keyword sexps =
34   match find_forms keyword sexps with
35   | [x] -> Some x
36   | _ -> None
37
38 (** Extract the name from (keyword name ...) *)
39 let form_name = function
40   | Atom (_, name) :: _ -> Some name
41   | _ -> None
42
43 (** Parse sizing: integer = flex parts, or explicit units *)
44 let parse_sizing = function
45   | Atom (_, "grow") -> Some Grow
46   | Atom (_, "full") -> Some Full
47   | Atom (_, "auto") -> Some Auto
48   | Atom (_, s) ->
49     Option.map (fun n -> Flex n) (int_of_string_opt s)
50   | List (_, [Atom (_, "px"); Atom (_, n)]) ->
51     Option.map (fun v -> Px v) (int_of_string_opt n)
52   | List (_, [Atom (_, "rem"); Atom (_, n)]) ->
53     Option.map (fun v -> Rem v) (float_of_string_opt n)
54   | List (_, [Atom (_, "pct"); Atom (_, n)]) ->
55     Option.map (fun v -> Pct v) (int_of_string_opt n)
56   | List (_, [Atom (_, "vh"); Atom (_, n)]) ->
57     Option.map (fun v -> Vh v) (int_of_string_opt n)
58   | _ -> None
59
60 (** Parse color: hex string, keyword, or palette reference *)
61 let parse_color = function
62   | Atom (_, s) when String.length s > 0 && s.[0] = '#' ->
63     Some (Hex s)
64   | Atom (_, s) ->
65     (* Check if it's a known keyword color *)
66     let keywords = ["red"; "orange"; "yellow"; "green"; "blue";
67       "purple"; "pink"; "white"; "black"; "gray";
68       "dark"; "light"; "transparent"] in
69     if List.mem s keywords then Some (Keyword s)
70     else None
71   | List (_, [Atom (_, "palette"); Atom (_, name)]) ->

```



```

72   Some (Palette name)
73 | List (_, [Atom (_, "alpha"); _]) ->
74   (* (alpha 0.6) -- handled by caller *)
75   None
76 | _ -> None
77
78 (** Parse spacing: integer px, (rem N), or (spacing name) *)
79 let parse_spacing = function
80 | Atom (_, n) ->
81   Option.map (fun v -> Spacing_px v) (int_of_string_opt n)
82 | List (_, [Atom (_, "rem"); Atom (_, n)]) ->
83   Option.map (fun v -> Spacing_rem v) (float_of_string_opt n)
84 | List (_, [Atom (_, "spacing"); Atom (_, name)]) ->
85   Some (Spacing_var name)
86 | _ -> None
87
88 (** Parse font-size: integer px, (rem N), or (font-size name) *)
89 let parse_font_size = function
90 | Atom (_, n) ->
91   Option.map (fun v -> Font_px v) (int_of_string_opt n)
92 | List (_, [Atom (_, "rem"); Atom (_, n)]) ->
93   Option.map (fun v -> Font_rem v) (float_of_string_opt n)
94 | List (_, [Atom (_, "font-size"); Atom (_, name)]) ->
95   Some (Font_var name)
96 | _ -> None
97
98 (** Parse radius: integer px or (radius name) *)
99 let parse_radius = function
100 | Atom (_, n) ->
101   Option.map (fun v -> Radius_px v) (int_of_string_opt n)
102 | List (_, [Atom (_, "radius"); Atom (_, name)]) ->
103   Some (Radius_var name)
104 | _ -> None
105
106 (** Parse layout direction *)
107 let parse_layout = function
108 | Atom (_, "horizontal") -> Some Horizontal
109 | Atom (_, "vertical") -> Some Vertical
110 | _ -> None
111
112 (** Parse scroll direction *)
113 let parse_scroll = function
114 | Atom (_, "vertical") -> Some Scroll_vertical
115 | Atom (_, "horizontal") -> Some Scroll_horizontal
116 | Atom (_, "both") -> Some Scroll_both
117 | _ -> None
118
119 (** Parse interactive type *)
120 let parse_interactive = function
121 | Atom (_, "click") -> Some Interactive_click
122 | Atom (_, "type") -> Some Interactive_type
123 | Atom (_, "hover") -> Some Interactive_hover
124 | _ -> None
125
126 (** Parse a single UI property from a sexp form *)
127 let rec parse_property = function
128 | List (_, Atom (_, "layout") :: dir :: _) ->
129   Option.map (fun d -> P_layout d) (parse_layout dir)
130 | List (_, Atom (_, "width") :: s :: _) ->
131   Option.map (fun s -> P_width s) (parse_sizing s)
132 | List (_, Atom (_, "height") :: s :: _) ->
133   Option.map (fun s -> P_height s) (parse_sizing s)
134 | List (_, Atom (_, "padding") :: rest) ->
135   (match parse_padding rest with
136   | Some p -> Some (P_padding p)
137   | None -> None)
138 | List (_, Atom (_, "gap") :: s :: _) ->
139   Option.map (fun s -> P_gap s) (parse_spacing s)
140 | List (_, Atom (_, "bg") :: c :: _) ->
141   Option.map (fun c -> P_bg c) (parse_color c)
142 | List (_, Atom (_, "color") :: c :: _) ->
143   Option.map (fun c -> P_color c) (parse_color c)
144 | List (_, Atom (_, "scroll") :: d :: _) ->

```

```

145   Option.map (fun d -> P_scroll d) (parse_scroll d)
146 | List (_, Atom (_, "interactive") :: t :: _) ->
147   Option.map (fun t -> P_interactive t) (parse_interactive t)
148 | List (_, Atom (_, "align-x") :: a :: _) ->
149   (match alignment_of_string (Option.value (atom_name a) ~default:"") with
150   | Some a -> Some (P_align_x a)
151   | None -> None)
152 | List (_, Atom (_, "align-y") :: a :: _) ->
153   (match alignment_of_string (Option.value (atom_name a) ~default:"") with
154   | Some a -> Some (P_align_y a)
155   | None -> None)
156 | List (_, Atom (_, "border") :: _) ->
157   Some (P_border { width = None; color = None; side = None })
158 | List (_, Atom (_, "corner") :: r :: _) ->
159   Option.map (fun r -> P_corner (Corner_uniform r)) (parse_radius r)
160 | List (_, Atom (_, "float") :: _) ->
161   Some (P_float Float_simple)
162 | _ -> None
163
164 (** Skip forms that aren't properties (children, variant, etc.) *)
165 and is_property_form = function
166 | List (_, Atom (_, k) :: _) ->
167   not (List.mem k ["children"; "variant"; "slot"; "fill";
168                  "use"; "use-layout"; "text"; "image";
169                  "placeholder"])
170 | _ -> false
171
172 and parse_padding args = match args with
173 | [s] ->
174   Option.map (fun s -> Padding_uniform s) (parse_spacing s)
175 | _ ->
176   (* Per-side padding: (top N) (right N) (bottom N) (left N) *)
177   let get_side name =
178     List.find_map (function
179       | List (_, [Atom (_, n); v]) when n = name -> parse_spacing v
180       | _ -> None
181     ) args
182   in
183   (match get_side "top", get_side "right",
184    get_side "bottom", get_side "left" with
185   | Some t, Some r, Some b, Some l ->
186     Some (Padding_sides { top = t; right = r; bottom = b; left = l })
187   | _ -> None)
188
189 (** Parse a (variant name ...) form *)
190 and parse_variant = function
191 | List (_, Atom (_, "variant") :: Atom (_, name) :: rest) ->
192   let props, children_forms =
193     List.partition (function
194       | List (_, Atom (_, "children") :: _) -> false
195       | _ -> true
196     ) rest
197   in
198   let properties = List.filter_map parse_property
199     (List.filter is_property_form props) in
200   let children = List.filter_map parse_ui_element children_forms in
201   Some { name; properties; children = Some children }
202 | _ -> None
203
204 (** Parse a (ui name ...) form *)
205 and parse_ui_element = function
206 | List (_, Atom (_, "ui") :: rest) ->
207   let name, body =
208     match rest with
209     | Atom (_, n) :: body -> Some n, body
210     | body -> None, body
211   in
212   let children_forms, other =
213     List.partition (function
214       | List (_, Atom (_, "children") :: _) -> true
215       | _ -> false
216     ) body
217   in

```

```

218 let variant_forms, props_body =
219   List.partition (function
220     | List (_, Atom (_, "variant")) :: _ -> true
221     | _ -> false
222   ) other
223 in
224 let properties = List.filter_map parse_property
225   (List.filter is_property_form props_body) in
226 let children =
227   List.concat_map (function
228     | List (_, Atom (_, "children")) :: kids ->
229     List.filter_map (function
230       | Atom (_, s) when String.length s > 2 && String.sub s 0 2 =
231         ".." ->
232         Some (Slot_ref (String.sub s 2 (String.length s - 2)))
233         | kid ->
234         (match parse_ui_element kid with
235          | Some el -> Some (El el)
236          | None -> None)
237       ) kids
238     | _ -> []
239   ) children_forms
240 in
241 let variants = List.filter_map parse_variant variant_forms in
242 Some { name; properties; children; variants; text = None }
243 | List (_, [Atom (_, "use"); Atom (_, comp_name)]) ->
244 (* (use component-name) -- creates a component reference element *)
245 Some { name = Some comp_name; properties = []; children = []; variants
246   = []; text = None }
247 | List (_, [Atom (_, "text"); String (_, Quoted { q_content })]) ->
248 (* (text "...") -- inline text node *)
249 Some { name = None; properties = []; children = []; variants = []; text
250   = Some q_content }
251 | List (_, Atom (_, comp_name) :: args) ->
252 (* (component-name ...) -- a component use with slot children *)
253 let is_known_component = String.length comp_name > 0 &&
254   comp_name.[0] <> '_' &&
255   not (List.mem comp_name ["ui"; "layout"; "page"; "theme"; "variant";
256     "children"; "fill"]) in
257 if not is_known_component then None
258 else begin
259   let children = List.filter_map (function
260     | List (_, _) as kid ->
261     (match parse_ui_element kid with
262      | Some el -> Some (El el)
263      | None -> None)
264     | _ -> None
265   ) args in
266   Some { name = Some comp_name; properties = []; children; variants =
267     []; text = None }
268 end
269 | _ -> None
270
271 (** Parse a (theme ...) form *)
272 let parse_theme = function
273 | List (_, Atom (_, "theme")) :: entries ->
274 let entries = List.filter_map (function
275   | List (_, [Atom (_, "palette"); Atom (_, name); String (_, Quoted {
276     q_content })]) ->
277     Some (Palette_entry (name, q_content))
278   | List (_, [Atom (_, "palette"); Atom (_, name); Atom (_, value)]) ->
279     Some (Palette_entry (name, value))
280   | List (_, [Atom (_, "spacing"); Atom (_, name); Atom (_, n)]) ->
281     Option.map (fun v -> Spacing_entry (name, v)) (int_of_string_opt n)
282   | List (_, [Atom (_, "font-size"); Atom (_, name); Atom (_, n)]) ->
283     Option.map (fun v -> Font_size_entry (name, v)) (int_of_string_opt
284     n)
285   | List (_, [Atom (_, "radius"); Atom (_, name); Atom (_, n)]) ->
286     Option.map (fun v -> Radius_entry (name, v)) (int_of_string_opt n)
287   | _ -> None
288 ) entries in
289 Some { entries }
290 | _ -> None

```

```

284
285 (** Parse a (component name ...) form *)
286 and parse_component = function
287 | List (_, Atom (_, "component") :: Atom (_, name) :: rest) ->
288   let children_forms, props_body =
289     List.partition (function
290       | List (_, Atom (_, "children") :: _) -> true
291       | _ -> false
292     ) rest
293   in
294   let variant_forms, props_body' =
295     List.partition (function
296       | List (_, Atom (_, "variant") :: _) -> true
297       | _ -> false
298     ) props_body
299   in
300   let properties = List.filter_map parse_property
301     (List.filter is_property_form props_body') in
302   let children =
303     List.concat_map (function
304       | List (_, Atom (_, "children") :: kids) ->
305         List.filter_map (function
306           | Atom (_, s) when String.length s > 2 && String.sub s 0 2 =
307             ".." ->
308               Some (Slot_ref (String.sub s 2 (String.length s - 2)))
309             | kid ->
310               (match parse_ui_element kid with
311                | Some el -> Some (El el)
312                | None -> None)
313             ) kids
314       | _ -> []
315     ) children_forms
316   in
317   let variants = List.filter_map parse_variant variant_forms in
318   (* Extract slots from children *)
319   let _slots = List.concat_map (function
320     | List (_, Atom (_, "children") :: kids) ->
321       List.filter_map (function
322         | Atom (_, s) when String.length s > 2 && String.sub s 0 2 = ".."
323         ->
324           Some Default_slot
325         | List (_, [Atom (_, "slot"); Atom (_, name)]) ->
326           Some (Named_slot name)
327         | _ -> None
328       ) kids
329     | _ -> []
330   ) children_forms in
331   Some { name; properties; slots = _slots; children; variants }
332 | _ -> None
333
334 (** Parse a (layout name ...) form *)
335 and parse_layout_def = function
336 | List (_, Atom (_, "layout") :: Atom (_, name) :: rest) ->
337   let root = List.filter_map parse_ui_element rest in
338   let root_el = match root with
339   | [el] -> el
340   | _ -> { name = Some "root"; properties = []; children = List.map
341     (fun el -> El el) root; variants = []; text = None }
342   in
343   (* Extract named slots from the element tree *)
344   let rec collect_slots el =
345     List.concat_map (function
346       | Slot_ref name -> [Named_slot name]
347       | El child -> collect_slots child
348     ) el.children
349   in
350   let slots = collect_slots root_el in
351   Some { name; root = root_el; slots }
352 | _ -> None
353
354 (** Parse a (page name ...) form *)
355 and parse_page = function
356 | List (_, Atom (_, "page") :: Atom (_, name) :: rest) ->

```

```

354   let layout_name, rest' =
355     match rest with
356     | List (_, [Atom (_, "use-layout"); Atom (_, ln)]) :: more ->
357       ln, more
358     | _ -> "", rest
359   in
360   let fills = List.filter_map (function
361     | List (_, Atom (_, "fill")) :: Atom (_, slot_name) :: content ->
362     let content_els = List.filter_map parse_ui_element content in
363     Some { slot_name; content = content_els }
364     | _ -> None
365   ) rest' in
366   Some { name; layout_name; fills; override_element = [] }
367 | _ -> None
368
369 (** Parse a single action from a sexp form *)
370 let parse_action = function
371 | List (_, [Atom (_, "action"); Atom (_, "click"); String (_, Quoted {
372   q_content })]) ->
373   Some (Click q_content)
374 | List (_, [Atom (_, "action"); Atom (_, "click"); Atom (_, name)]) ->
375   Some (Click name)
376 | List (_, [Atom (_, "action"); Atom (_, "type"); String (_, Quoted {
377   q_content })]) ->
378   Some (Type q_content)
379 | List (_, [Atom (_, "action"); Atom (_, "type"); Atom (_, name)]) ->
380   Some (Type name)
381 | List (_, [Atom (_, "action"); Atom (_, "hover"); String (_, Quoted {
382   q_content })]) ->
383   Some (Hover q_content)
384 | List (_, [Atom (_, "action"); Atom (_, "hover"); Atom (_, name)]) ->
385   Some (Hover name)
386 | _ -> None
387
388 (** Parse an outcome from a sexp form *)
389 let parse_outcome = function
390 | List (_, [Atom (_, "outcome"); Atom (_, "navigate"); String (_, Quoted {
391   q_content })]) ->
392   Some (Navigate q_content)
393 | List (_, [Atom (_, "outcome"); Atom (_, "navigate"); Atom (_, path)]) ->
394   Some (Navigate path)
395 | List (_, [Atom (_, "outcome"); Atom (_, "submit")]) ->
396   Some Submit
397 | _ -> None
398
399 (** Parse a (step name ...) form *)
400 let parse_step = function
401 | List (_, Atom (_, "step") :: Atom (_, name) :: forms) ->
402   let page_name =
403     List.find_map (function
404       | List (_, [Atom (_, "page"); Atom (_, pn)]) -> Some pn
405       | _ -> None
406     ) forms
407   |> Option.value ~default:""
408   in
409   let actions = List.filter_map parse_action forms in
410   let outcome =
411     List.find_map (fun f ->
412       match parse_outcome f with Some o -> Some o | None -> None
413     ) forms
414   |> Option.value ~default:No_outcome
415   in
416   Some { name; page_name; actions; outcome }
417 | _ -> None
418
419 (** Parse a (workflow name ...) form *)
420 let parse_workflow = function
421 | List (_, Atom (_, "workflow") :: Atom (_, name) :: rest) ->
422   let doc_str =
423     List.find_map (function
424       | List (_, [Atom (_, "doc"); String (_, Quoted { q_content })]) ->
425       Some q_content

```

```

421 | List (_, [Atom (_, "doc"); String (_, Verbatim { v_content })])
    -> Some v_content
422 | _ -> None
423 ) rest
424 in
425 let steps = List.filter_map parse_step rest in
426 Some { name; doc = doc_str; steps }
427 | _ -> None
428
429 (** Parse a (routes ...) form *)
430 let parse_routes = function
431 | List (_, Atom (_, "routes") :: entries) ->
432   List.filter_map (function
433     | List (_, [Atom (_, "route"); String (_, Quoted { q_content })]; Atom
434       (_, page_name)]) ->
435     Some { path = q_content; page_name }
436   | _ -> None
437   ) entries
438 | _ -> []
439
440 (** Parse a complete UI application from a (ui app-name ...) form *)
441 let parse_ui_app = function
442 | List (_, Atom (_, "ui") :: _ :: body) ->
443   let theme = List.filter_map parse_theme body |> fun xs ->
444     match xs with [t] -> Some t | _ -> None in
445   let components = List.filter_map parse_component body in
446   let layouts = List.filter_map parse_layout_def body in
447   let pages = List.filter_map parse_page body in
448   let routes = List.concat_map (fun r ->
449     match parse_routes r with
450     | [] -> []
451     | routes -> routes
452   ) body in
453   let elements = List.filter_map parse_ui_element body in
454   let workflows = List.filter_map parse_workflow body in
455   Some { theme; components; layouts; pages; routes; elements; workflows }
456 | _ -> None
457
458 (** Parse all UI forms from a file's top-level sexps *)
459 let parse_file (file : Borge_lang.Ast.file) : ui_app option =
460 let rec walk = function
461 | [] -> None
462 | { node; _ } :: rest ->
463   (match parse_ui_app node with
464   | Some app -> Some app
465   | None -> walk rest)
466 in
467 match walk file.top_level with
468 | Some app -> Some app
469 | None ->
470   (* No top-level (ui ...) form -- collect any individual forms *)
471   let sexps = List.map (fun swc -> swc.node) file.top_level in
472   let theme = List.filter_map parse_theme sexps |> fun xs ->
473     match xs with [t] -> Some t | _ -> None in
474   let components = List.filter_map parse_component sexps in
475   let layouts = List.filter_map parse_layout_def sexps in
476   let pages = List.filter_map parse_page sexps in
477   let routes = List.concat_map (fun s ->
478     match parse_routes s with [] -> [] | r -> r
479   ) sexps in
480   let elements = List.filter_map parse_ui_element sexps in
481   let workflows = List.filter_map parse_workflow sexps in
482   Some { theme; components; layouts; pages; routes; elements; workflows }

```

32 lib/ui/ui_validate.ml

```

1  (** Semantic validation for borge UI specs.
2
3      Checks that references are valid:
4      - Variant names are unique per element
5      - Theme variable references refer to defined entries
6      - Page layout references are defined
7      - Component and layout name uniqueness *)
8
9  open Ui_ast
10
11  type issue = {
12      message : string;
13      severity : [ `Error | `Warning ];
14  }
15
16  let err msg = { message = msg; severity = `Error }
17  let warn msg = { message = msg; severity = `Warning }
18
19  (** Collect defined palette names from the theme *)
20  let palette_names (theme : theme option) : string list =
21      match theme with
22      | None -> []
23      | Some t ->
24          List.filter_map (function
25              | Palette_entry (name, _) -> Some name
26              | _ -> None
27          ) t.entries
28
29  (** Collect defined spacing names from the theme *)
30  let spacing_names (theme : theme option) : string list =
31      match theme with
32      | None -> []
33      | Some t ->
34          List.filter_map (function
35              | Spacing_entry (name, _) -> Some name
36              | _ -> None
37          ) t.entries
38
39  (** Collect defined font-size names from the theme *)
40  let font_size_names (theme : theme option) : string list =
41      match theme with
42      | None -> []
43      | Some t ->
44          List.filter_map (function
45              | Font_size_entry (name, _) -> Some name
46              | _ -> None
47          ) t.entries
48
49  (** Collect defined radius names from the theme *)
50  let radius_names (theme : theme option) : string list =
51      match theme with
52      | None -> []
53      | Some t ->
54          List.filter_map (function
55              | Radius_entry (name, _) -> Some name
56              | _ -> None
57          ) t.entries
58
59  (** Check a color value against defined palette names *)
60  let rec check_color (palette : string list) = function
61      | Palette name when not (List.mem name palette) ->
62          [err (Printf.sprintf "undefined palette reference: %s" name)]
63      | With_alpha (c, _) ->
64          check_color palette c
65      | _ -> []
66
67  (** Check a spacing value against defined spacing names *)
68  let check_spacing (spacings : string list) = function
69      | Spacing_var name when not (List.mem name spacings) ->
70          [err (Printf.sprintf "undefined spacing reference: %s" name)]
71      | _ -> []

```

```

72
73 (** Check a font-size value against defined font-size names *)
74 let check_font_size (font_sizes : string list) = function
75   | Font_var name when not (List.mem name font_sizes) ->
76     [err (Printf.sprintf "undefined font-size reference: %s" name)]
77   | _ -> []
78
79 (** Check a radius value against defined radius names *)
80 let check_radius (radii : string list) = function
81   | Radius_var name when not (List.mem name radii) ->
82     [err (Printf.sprintf "undefined radius reference: %s" name)]
83   | _ -> []
84
85 (** Check variant names are unique within an element *)
86 let check_variant_uniqueness (variants : variant list) : issue list =
87   let seen = Hashtbl.create 8 in
88   List.filter_map (fun (v : variant) ->
89     if Hashtbl.mem seen v.name then
90       Some (err (Printf.sprintf "duplicate variant name: %s" v.name))
91     else begin
92       Hashtbl.add seen v.name true;
93       None
94     end
95   ) variants
96
97 (** Validate a single UI property against theme definitions *)
98 let check_property (palette, spacings, _font_sizes, radii) = function
99   | P_bg c -> check_color palette c
100  | P_color c -> check_color palette c
101  | P_gap s -> check_spacing spacings s
102  | P_padding (Padding_uniform s) -> check_spacing spacings s
103  | P_padding (Padding_sides { top = t; right = r; bottom = b; left = l })
104    ->
105    check_spacing spacings t @ check_spacing spacings r
106    @ check_spacing spacings b @ check_spacing spacings l
107  | P_corner (Corner_uniform r) -> check_radius radii r
108  | _ -> []
109
110 (** Recursively validate a UI element *)
111 let rec check_element (theme_ctx, component_names) (el : ui_element) :
112   issue list =
113   let props = List.concat_map
114     (check_property theme_ctx) el.properties in
115   let variant_issues = check_variant_uniqueness el.variants in
116   let variant_props = List.concat_map (fun (v : variant) ->
117     List.concat_map (check_property theme_ctx) v.properties
118   ) el.variants in
119   let children = List.concat_map
120     (function El el -> check_element (theme_ctx, component_names) el |
121       Slot_ref _ -> []) el.children in
122   props @ variant_issues @ variant_props @ children
123
124 (** Validate a page: check layout reference *)
125 let check_page (layout_names : string list) (page : page) : issue list =
126   if page.layout_name <> "" && not (List.mem page.layout_name layout_names)
127   then
128     [err (Printf.sprintf "page '%s' references undefined layout: %s"
129       page.name page.layout_name)]
130   else []
131
132 (** Validate a component *)
133 let check_component (theme_ctx, component_names) (comp : component) : issue
134   list =
135   let props = List.concat_map
136     (check_property theme_ctx) comp.properties in
137   let variant_issues = check_variant_uniqueness comp.variants in
138   let children = List.concat_map
139     (function El el -> check_element (theme_ctx, component_names) el |
140       Slot_ref _ -> []) comp.children in
141   props @ variant_issues @ children
142
143 (** Validate a layout: check root element tree *)

```



```

138 let check_layout (theme_ctx, component_names) (ly : layout_def) : issue
    list =
139   check_element (theme_ctx, component_names) ly.root
140
141 (** Collect all named elements from a UI element tree *)
142 let rec collect_elements (el : Ui_ast.ui_element) acc =
143   let acc = match el.name with Some n -> n :: acc | None -> acc in
144   List.fold_left (fun acc -> function
145     | El child -> collect_elements child acc
146     | Slot_ref _ -> acc
147   ) acc el.children
148
149 (** Collect names of elements with (interactive ...) property *)
150 let rec collect_interactive_elements (el : Ui_ast.ui_element) acc =
151   let has_interactive = List.exists (function P_interactive _ -> true | _
152     -> false) el.properties in
153   let acc = match el.name with
154     | Some n when has_interactive -> n :: acc
155     | _ -> acc
156   in
157   List.fold_left (fun acc -> function
158     | El child -> collect_interactive_elements child acc
159     | Slot_ref _ -> acc
160   ) acc el.children
161
162 (** Build a map of page name ? interactive element names *)
163 let build_page_interactive_map (pages : page list) : (string, string list)
    Hashtbl.t =
164   let h = Hashtbl.create 16 in
165   List.iter (fun (p : page) ->
166     let all_fills = List.concat_map (fun (sf : slot_fill) -> sf.content)
167       p.fills in
168     let interactive_names = List.fold_left (fun acc el ->
169       collect_interactive_elements el acc
170     ) [] all_fills in
171     Hashtbl.add h p.name interactive_names
172   ) pages;
173   h
174
175 (** Check that every step references a defined page *)
176 let check_workflow_steps (page_names : string list) (wf : workflow) : issue
    list =
177   List.filter_map (fun (step : step) ->
178     if not (List.mem step.page_name page_names) then
179       Some (err (Printf.sprintf "workflow '%s' step '%s' references
180         undefined page: %s"
181           wf.name step.name step.page_name))
182     else
183       None
184   ) wf.steps
185
186 (** Check that every navigate outcome matches a defined route *)
187 let check_workflow_routes (route_paths : string list) (wf : workflow) :
    issue list =
188   List.filter_map (fun (step : step) ->
189     match step.outcome with
190     | Navigate path when not (List.mem path route_paths) ->
191       Some (err (Printf.sprintf "workflow '%s' step '%s' navigates to
192         undefined route: %s"
193           wf.name step.name path))
194     | _ -> None
195   ) wf.steps
196
197 (** Check that every action references an interactive element on the step's
    page *)
198 let check_workflow_actions (page_map : (string, string list) Hashtbl.t) (wf
    : workflow) : issue list =
199   List.concat_map (fun (step : step) ->
200     match Hashtbl.find_opt page_map step.page_name with
201     | None -> [] (* already caught by check_workflow_steps *)
202     | Some interactive_names ->
203       List.filter_map (function
204         | Click name when not (List.mem name interactive_names) ->

```

```

201         Some (err (Printf.sprintf "workflow '%s' step '%s' click action
references non-interactive '%s' on page '%s'"
202           wf.name step.name name step.page_name))
203         | Type name when not (List.mem name interactive_names) ->
204         Some (err (Printf.sprintf "workflow '%s' step '%s' type action
references non-interactive '%s' on page '%s'"
205           wf.name step.name name step.page_name))
206         | Hover name when not (List.mem name interactive_names) ->
207         Some (err (Printf.sprintf "workflow '%s' step '%s' hover action
references non-interactive '%s' on page '%s'"
208           wf.name step.name name step.page_name))
209         | _ -> None
210       ) step.actions
211     ) wf.steps
212
213 (** Check that every workflow has a doc *)
214 let check_workflow_doc (wf : workflow) : issue list =
215   match wf.doc with
216   | None -> [warn (Printf.sprintf "workflow '%s' has no (doc ...)
explaining the journey" wf.name)]
217   | Some _ -> []
218
219 (** Validate a single workflow *)
220 let check_workflow (page_names : string list) (route_paths : string list)
221   (page_map : (string, string list) Hashtbl.t) (wf : workflow) : issue
list =
222   check_workflow_steps page_names wf
223   @ check_workflow_routes route_paths wf
224   @ check_workflow_actions page_map wf
225   @ check_workflow_doc wf
226
227 (** Validate the complete UI app spec *)
228 let validate (app : ui_app) : issue list =
229   let palette = palette_names app.theme in
230   let spacings = spacing_names app.theme in
231   let font_sizes = font_size_names app.theme in
232   let radii = radius_names app.theme in
233   let theme_ctx = (palette, spacings, font_sizes, radii) in
234   let component_names = List.map (fun (c : component) -> c.name)
app.components in
235   let layout_names = List.map (fun (ly : layout_def) -> ly.name)
app.layouts in
236
237   (* Check component name uniqueness *)
238   let comp_name_issues =
239     let seen = Hashtbl.create 8 in
240     List.filter_map (fun (c : component) ->
241       if Hashtbl.mem seen c.name then
242         Some (err (Printf.sprintf "duplicate component name: %s" c.name))
243       else begin
244         Hashtbl.add seen c.name true;
245         None
246       end
247     ) app.components
248   in
249
250   (* Check layout name uniqueness *)
251   let layout_name_issues =
252     let seen = Hashtbl.create 8 in
253     List.filter_map (fun (ly : layout_def) ->
254       if Hashtbl.mem seen ly.name then
255         Some (err (Printf.sprintf "duplicate layout name: %s" ly.name))
256       else begin
257         Hashtbl.add seen ly.name true;
258         None
259       end
260     ) app.layouts
261   in
262
263   let component_issues = List.concat_map
264     (check_component (theme_ctx, component_names)) app.components in
265   let layout_issues = List.concat_map
266     (check_layout (theme_ctx, component_names)) app.layouts in

```

```

267 let page_issues = List.concat_map
268   (check_page layout_names) app.pages in
269 let element_issues = List.concat_map
270   (check_element (theme_ctx, component_names)) app.elements in
271
272 (* Workflow validation *)
273 let page_names = List.map (fun (p : page) -> p.name) app.pages in
274 let route_paths = List.map (fun (r : route) -> r.path) app.routes in
275 let page_map = build_page_interactive_map app.pages in
276 let workflow_issues = List.concat_map
277   (check_workflow page_names route_paths page_map) app.workflows in
278
279 (* Check for pages not appearing in any workflow step *)
280 let stepped_pages =
281   List.fold_left (fun acc (wf : workflow) ->
282     List.fold_left (fun acc (step : step) ->
283       step.page_name :: acc
284     ) acc wf.steps
285   ) [] app.workflows
286   |> List.sort_uniq String.compare
287 in
288 let unworked_page_issues =
289   List.filter_map (fun (p : page) ->
290     if not (List.mem p.name stepped_pages) then
291       Some (warn (Printf.sprintf "page '%s' is not part of any workflow"
292         p.name))
293     else
294       None
295   ) app.pages
296 in
297
298 (* Check workflow name uniqueness *)
299 let workflow_name_issues =
300   let seen = Hashtbl.create 8 in
301   List.filter_map (fun (wf : workflow) ->
302     if Hashtbl.mem seen wf.name then
303       Some (err (Printf.sprintf "duplicate workflow name: %s" wf.name))
304     else begin
305       Hashtbl.add seen wf.name true;
306       None
307     end
308   ) app.workflows
309 in
310
311 comp_name_issues @ layout_name_issues @ component_issues
312 @ layout_issues @ page_issues @ element_issues
313 @ workflow_issues @ unworked_page_issues @ workflow_name_issues

```

33 lib/book/book_manifest.ml

```

1 (* agent note (|
2  * WHAT: Manifest sidecar for the codebook round trip.
3  * JSON (yojson): stem, rendered_at, backend, tree_checksum,
4  * files (ordered), file_pages (file -> start_page).
5  * WHY: The round-trip anchor. Line numbers are printed on the
6  * page by listings, so the manifest never stores line ranges --
7  * it only records which file begins on which page (so a margin
8  * note in the gutter between two files is unambiguous) plus a
9  * tree checksum that lets absorb detect a stale book.
10 * |) *)
11
12 type page_entry = {
13   file : string;
14   start_page : int;
15 }
16
17 type manifest = {
18   stem : string;
19   rendered_at : string;
20   backend : string;
21   tree_checksum : string;
22   files : string list;
23   file_pages : page_entry list;
24 }
25
26 let content_digest path =
27   try
28     let content = File_utils.read_file path in
29     Digest.to_hex (Digest.string content)
30   with Sys_error _ -> ""
31
32 (* Ordered digest over (path, content digest). Ordered so a file
33  reorder is detectable, not just content changes. *)
34 let tree_checksum ~dir files =
35   let buf = Buffer.create 4096 in
36   List.iter (fun path ->
37     Buffer.add_string buf path;
38     Buffer.add_char buf '\x00';
39     Buffer.add_string buf (content_digest (Filename.concat dir path));
40     Buffer.add_char buf '\x01'
41   ) files;
42   Digest.to_hex (Digest.string (Buffer.contents buf))
43
44 let now_iso () =
45   let open Unix in
46   let t = time () in
47   let tm = localtime t in
48   Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02d"
49     (tm.tm_year + 1900) (tm.tm_mon + 1) tm.tm_mday
50     tm.tm_hour tm.tm_min tm.tm_sec
51
52 let build ~stem ~backend ~tree_checksum ~files ~file_pages =
53   {
54     stem;
55     rendered_at = now_iso ();
56     backend;
57     tree_checksum;
58     files;
59     file_pages;
60   }
61
62 let page_entry_to_json e =
63   'Assoc [
64     ("file", 'String e.file);
65     ("start_page", 'Int e.start_page);
66   ]
67
68 let to_json m =
69   let json =
70     'Assoc [
71       ("stem", 'String m.stem);

```

```
72     ("rendered_at", 'String m.rendered_at);
73     ("backend", 'String m.backend);
74     ("tree_checksum", 'String m.tree_checksum);
75     ("files", 'List (List.map (fun f -> 'String f) m.files));
76     ("file_pages", 'List (List.map page_entry_to_json m.file_pages));
77   ]
78   in
79   Yojson.Safe.pretty_to_string json
80
81 let write ~path m =
82   let oc = open_out path in
83   output_string oc (to_json m);
84   output_string oc "\n";
85   close_out oc
```

34 lib/book/book_print.ml

```

1 (* agent note (|
2  * WHAT: v1 codebook printer. Walks the source tree in
3  * deterministic filesystem-order, emits a LaTeX document with
4  * the listings package (line numbers, framed code, a wide
5  * right-hand margin for hand annotations), runs pdflatex until
6  * the .aux/.toc stabilize (a multi-page TOC grows across passes),
7  * and writes a manifest sidecar via Book_manifest.
8  * WHY: The "print" half of the book round trip (borge.borg
9  * section book). Code-only, no spec interleaving; spec-order
10 * and absorb are long-tail.
11 * |) *)
12
13 let skip_dirs =
14   ["_build"; ".git"; ".pi"; ".ralph"; ".borge-bugs"; ".borge.lock";
15    "_build_test"; "node_modules"; "_opam"; ".opamswitch"]
16
17 let is_source name =
18   (* .borg yes; .borg.meta no -- that's machine-generated, not the spec *)
19   (Filename.check_suffix name ".borg"
20    && not (Filename.check_suffix name ".borg.meta"))
21   || Filename.check_suffix name ".ml"
22   || Filename.check_suffix name ".mli"
23
24 (* Ordering tiers (lower sorts first):
25   0 = root .borg file (the architecture / introduction)
26   1 = other top-level .borg files (direct sub-specs, design docs)
27   2 = .mli files (module signatures -- interface layer)
28   3 = .ml files (implementations)
29   Within a tier, sort by directory group (lib/, bin/, test/) then lexical.
30   *)
31 let is_top_level_borg p =
32   not (String.contains p '/') && Filename.check_suffix p ".borg"
33
34 let dir_group p =
35   let len = String.length p in
36   if len >= 4 && String.sub p 0 4 = "lib/" then 0
37   else if len >= 4 && String.sub p 0 4 = "bin/" then 1
38   else if len >= 5 && String.sub p 0 5 = "test/" then 2
39   else 3
40
41 let file_kind p =
42   if Filename.check_suffix p ".mli" then 'Mli
43   else if Filename.check_suffix p ".ml" then 'Ml
44   else 'Borg
45
46 let tier_of ?(roots=[]) p =
47   if List.mem p roots then 0
48   else if is_top_level_borg p then 1
49   else match file_kind p with
50     | 'Borg -> 3 (* sub-directory .borg falls in with code *)
51     | 'Mli -> 2
52     | 'Ml -> 3
53
54 (* --- Dependency-aware ordering within the code tier ---
55   Parse 'open X' and 'Module.ref' from each .ml file, map module names
56   to their directory, and compute a dependency rank per directory
57   (longest path from a dependency root). The code tier then sorts by
58   (rank, dir_group, lexical) so foundations precede dependents --
59   the book reads like a stack, not a lexical shuffle. *)
60
61 (* Map every .ml module basename to its directory, relative to dir.
62   e.g. "Spec" -> "lib/core", "Drift" -> "lib/check". Basenames are
63   unique in this repo (include_subdirs unqualified requires it). *)
64 let module_dir_map ~dir =
65   let map = Hashtbl.create 256 in
66   let rec walk rel =
67     let d = if rel = "" then dir else Filename.concat dir rel in
68     let entries = try Sys.readdir d |> Array.to_list with Sys_error _ -> []
69     in
70     List.iter (fun name ->
71       if List.mem name skip_dirs then ()

```

```

70     else
71       let full = Filename.concat d name in
72       let r = if rel = "" then name else Filename.concat rel name in
73       if (try Sys.is_directory full with Sys_error _ -> false)
74       then walk r
75       else if Filename.check_suffix name ".ml" then
76         let mod_name = String.capitalize_ascii (Filename.chop_suffix name
77           ".ml") in
78         Hashtbl.replace map mod_name (Filename.dirname r)
79       ) (List.sort compare entries)
80     in
81     walk "";
82   map
83 (* Stdlib / external module names that must never be treated as project
84   deps,
85   even if a project module happens to share the basename (e.g. borge's
86   lib/format/format.ml shadows stdlib Format). 'Format' in source almost
87   always means stdlib formatting, not the project's fmt module. *)
88 let stdlib_names = [
89   "Format"; "String"; "List"; "Printf"; "Buffer"; "Bytes"; "Array";
90   "Scanf"; "Map"; "Set"; "Hashtbl"; "Queue"; "Stack"; "Stream";
91   "Char"; "Bool"; "Int"; "Float"; "Option"; "Result"; "Marshal";
92   "Obj"; "Lazy"; "Arg"; "Sys"; "Filename"; "Uchar"; "Lexing";
93   "Sedlexing"; "Parser"; "Parse"; "Lexer"; "Error"; "State";
94   "Unix"; "Str"; "Digest"; "Yojson"; "Cmdliner"; "Dream"; "Html";
95   "Atomic"; "Mutex"; "Condition"; "Domain"; "In_channel"; "Out_channel";
96   "LargeFile"; "Bigarray"; "Stdlib"; "Unit"; "Assert"; "Location";
97   "Longident"; "Asttypes"; "Parsetree"; "Ppxlib"; "Ast_iterator";
98   "Docstrings"; "Migrate_parsetree"; "Tbl"; "Either";
99 ]
100 (* Resolve a module reference (possibly qualified, e.g. Borge_lang.Ast)
101   to a directory. Take the first and last segments of the dotted name
102   and look each up; Borge_lang (the lang sublibrary) maps to lib/lang.
103   Stdlib/external names are skipped to avoid false deps. *)
104 let dir_of_ref ~module_map ~lang_dir name =
105   let parts = String.split_on_char '.' name in
106   let candidates =
107     match parts with
108     | [] -> []
109     | [x] -> [x]
110     | xs -> [List.hd xs; List.hd (List.rev xs)]
111   in
112   let lookup s =
113     if List.mem s stdlib_names then None
114     else if s = "Borge_lang" then Some lang_dir
115     else (try Some (Hashtbl.find module_map s) with Not_found -> None)
116   in
117   List.find_map lookup candidates
118
119 (* Directories a file depends on (excluding its own directory). *)
120 let file_deps ~dir ~module_map ~lang_dir path =
121   let full = Filename.concat dir path in
122   let content = try File_utils.read_file full with Sys_error _ -> "" in
123   let mydir = Filename.dirname path in
124   let seen = Hashtbl.create 16 in
125   let add_name name =
126     match dir_of_ref ~module_map ~lang_dir name with
127     | Some d when d <> mydir -> Hashtbl.replace seen d true
128     | _ -> ()
129   in
130   let scan re =
131     let rec loop start =
132       try
133         let _ = Str.search_forward re content start in
134         add_name (Str.matched_group 1 content);
135         loop (Str.match_end ())
136       with Not_found -> ()
137     in
138     loop 0
139   in
140   scan (Str.regexp "open[ \\t]+\\((([A-Za-z_][A-Za-z0-9_\\.]*\\))");

```

```

141   scan (Str.regexp "\\([A-Z][A-Za-z0-9_]*\\)\\.");
142   Hashtbl.fold (fun k _ acc -> k :: acc) seen []
143
144   (* Longest-path rank per directory. Roots (no internal deps) rank 0;
145      a directory that depends on a rank-n dir ranks at least n+1. Cycle-safe
146      via a visited mark written before recursing. *)
147   (* Root .borg files (the spec -- the primary artifact) relative to dir.
148      These become the introduction / front matter, ahead of the code. *)
149   let collect_borg_roots ~dir =
150     let roots = Project.find_roots dir in
151     List.filter_map (fun p ->
152       let d = Filename.concat dir "" in
153       let dlen = String.length d in
154       let rel =
155         if String.length p >= dlen && String.sub p 0 dlen = d
156         then String.sub p dlen (String.length p - dlen)
157         else if p = Filename.concat dir p then p
158         else p
159       in
160       if is_top_level_borg rel then Some rel else None
161     ) roots
162
163   let rec collect ~root ~rel acc =
164     let dir = if rel = "" then root else Filename.concat root rel in
165     let entries =
166       try Sys.readdir dir |> Array.to_list
167       with Sys_error _ -> []
168     in
169     List.fold_left (fun acc name ->
170       if List.mem name skip_dirs then acc
171       else
172         let full = Filename.concat dir name in
173         let r = if rel = "" then name else Filename.concat rel name in
174         if (try Sys.is_directory full with Sys_error _ -> false)
175         then collect ~root ~rel:r acc
176         else if is_source name then r :: acc
177         else acc
178       ) acc (List.sort compare entries)
179
180   (* Longest-path rank per directory. Roots (no internal deps) rank 0;
181      a directory that depends on a rank-n dir ranks at least n+1. Cycle-safe
182      via a visited mark written before recursing. *)
183   let compute_dir_ranks ~dir ~module_map ~lang_dir =
184     let files = collect ~root:dir ~rel:"" [] in
185     let edges = Hashtbl.create 64 in
186     List.iter (fun path ->
187       let mydir = Filename.dirname path in
188       List.iter (fun d ->
189         let s = try Hashtbl.find edges mydir with Not_found -> [] in
190         if not (List.mem d s) then Hashtbl.replace edges mydir (d :: s)
191       ) (file_deps ~dir ~module_map ~lang_dir path)
192     ) files;
193     let memo = Hashtbl.create 64 in
194     let rec rank d =
195       if Hashtbl.mem memo d then Hashtbl.find memo d
196       else begin
197         Hashtbl.add memo d 0; (* guard against cycles *)
198         let deps = try Hashtbl.find edges d with Not_found -> [] in
199         let r = match deps with
200           | [] -> 0
201           | xs -> 1 + List.fold_left (fun acc x -> max acc (rank x)) 0 xs
202         in
203         Hashtbl.replace memo d r; r
204       end
205     in
206     Hashtbl.iter (fun k _ -> ignore (rank k)) edges;
207     List.iter (fun p -> ignore (rank (Filename.dirname p))) files;
208     memo
209
210   let collect_files dir =
211     let module_map = module_dir_map ~dir in
212     let lang_dir = "lib/lang" in
213     let ranks = compute_dir_ranks ~dir ~module_map ~lang_dir in

```



```

214 let roots = collect_borg_roots ~dir in
215 let code = collect ~root:dir ~rel:"" [] in
216 let roots = List.sort_uniq compare roots in
217 let is_root p = List.mem p roots in
218 let code = List.filter (fun p -> not (is_root p)) code in
219 let files = roots @ code in
220 let rank_of p =
221   try Hashtbl.find ranks (Filename.dirname p) with Not_found -> 0
222 in
223 List.sort (fun a b ->
224   let ta = tier_of ~roots a and tb = tier_of ~roots b in
225   if ta <> tb then compare ta tb
226   else if ta = 2 || ta = 3 then begin
227     (* code tier: dependency rank first, then dir group, then lexical *)
228     let ra = rank_of a and rb = rank_of b in
229     if ra <> rb then compare ra rb
230   else begin
231     let ga = dir_group a and gb = dir_group b in
232     if ga <> gb then compare ga gb else compare a b
233   end
234   end else begin
235     (* borg tiers: dir group then lexical *)
236     let ga = dir_group a and gb = dir_group b in
237     if ga <> gb then compare ga gb else compare a b
238   end
239 ) files
240
241 (* Escape LaTeX special chars for use in \section{}, \markboth{}, headers.
242 Paths contain _, ., /, alphanumerics -- only _ needs escaping,
243 but we escape the full set to be safe. *)
244 let escape_text s =
245   let buf = Buffer.create (String.length s + 4) in
246   String.iter (fun c ->
247     match c with
248     | '\\' -> Buffer.add_string buf "\\textbackslash{"
249     | '~' -> Buffer.add_string buf "\\textasciicircum{"
250     | '~' -> Buffer.add_string buf "\\textasciitilde{"
251     | '_' | '&' | '%' | '#' | '$' | '{' | '}' ->
252       Buffer.add_char buf '\\'; Buffer.add_char buf c
253     | _ -> Buffer.add_char buf c
254   ) s;
255   Buffer.contents buf
256
257 let preamble () =
258   "\\documentclass[10pt,oneside]{article}\\n" ^
259   "\\usepackage[T1]{fontenc}\\n" ^
260   "\\usepackage{lmodern}\\n" ^
261   "\\usepackage{listings}\\n" ^
262   "\\usepackage{xcolor}\\n" ^
263   "\\usepackage[a4paper, left=2cm, right=7cm, top=2.5cm, bottom=2.5cm,
264   marginparwidth=5.5cm, marginparsep=8pt]{geometry}\\n" ^
265   "\\usepackage{fancyhdr}\\n" ^
266   "\\pagestyle{fancy}\\n" ^
267   "\\fancyhf{}\\n" ^
268   "\\fancyhead[L]{borge codebook}\\n" ^
269   (* \\leftmark = the first mark set on the page. Each file starts with
270   \\clearpage + \\markboth{file}{file}, so \\leftmark is the current
271   file with no lag. (\\rightmark lags by one section by design. *)
272   "\\fancyhead[R]{\\leftmark}\\n" ^
273   "\\fancyfoot[C]{\\thepage}\\n" ^
274   "\\renewcommand{\\headrulewidth}{0.4pt}\\n" ^
275   "\\lstset{basicstyle=\\ttfamily\\footnotesize, numbers=left,
276   numberstyle=\\tiny\\color{gray}, stepnumber=1, firstnumber=1,
277   frame=single, rulecolor=\\color{gray!50}, breaklines=true,
278   breakatwhitespace=true, showstringspaces=false, tabsize=2,
279   xleftmargin=2em, numbersep=10pt, columns=fullflexible,
280   keepspaces=true}\\n"
281
282 (* Sanitize UTF-8 to ASCII for pdflatex+listings, which is byte-oriented.
283 Maps common punctuation to ASCII equivalents, falls back to '?' for the
284 rest. The book is for reading code; ASCII-fied punctuation in comments
285 is fine, and this keeps the output deterministic. *)
286 let sanitize content =

```

```

281 let buf = Buffer.create (String.length content) in
282 let i = ref 0 in
283 let n = String.length content in
284 while !i < n do
285   let c = Char.code content.[!i] in
286   if c < 128 then begin
287     Buffer.add_char buf content.[!i];
288     incr i
289   end else begin
290     let len =
291       if c land 0xE0 = 0xC0 then 2
292       else if c land 0xF0 = 0xE0 then 3
293       else if c land 0xF8 = 0xF0 then 4
294       else 1
295     in
296     let seq =
297       if !i + len <= n then String.sub content !i len
298       else String.sub content !i (n - !i)
299     in
300     (match seq with
301      | "\xe2\x80\x94" -> Buffer.add_string buf "--" (* em dash *)
302      | "\xe2\x80\x93" -> Buffer.add_char buf '-' (* en dash *)
303      | "\xe2\x80\x99" | "\xe2\x80\x98" -> Buffer.add_char buf '\'',
304      | "\xe2\x80\x9c" | "\xe2\x80\x9d" -> Buffer.add_char buf '"',
305      | "\xc2\xa0" -> Buffer.add_char buf ' ' (* nbsp *)
306      | _ -> Buffer.add_char buf '?');
307     i := !i + len
308   end
309 done;
310 Buffer.contents buf
311
312 (* --- .borg -> LaTeX renderer (parses the sexp; no raw sexp in the PDF) ---
313 section/subsection -> headings, (doc ...) -> prose paragraphs,
314 (* author ... (|...|) *) comments -> attributed blockquotes,
315 (status X) -> an italic marker. Other forms (inline, verify,
316 depends-on, ...) are structural noise and are skipped. Falls back
317 to raw listing if the file fails to parse. *)
318 exception Fallback_raw
319
320 let string_value_text = function
321 | Borge_lang.Ast.Quoted q -> q.Borge_lang.Ast.q_content
322 | Borge_lang.Ast.Verbatim v -> v.Borge_lang.Ast.v_content
323
324 (* Emit prose: split on blank lines into paragraphs, escape, emit. *)
325 let render_prose buf text =
326   let paras = Str.split (Str.regexp "\n[ \t]*\n") text in
327   List.iter (fun p ->
328     let p = String.trim p in
329     if p <> "" then begin
330       Buffer.add_string buf (escape_text p);
331       Buffer.add_string buf "\n\n"
332     end
333   ) paras
334
335 let render_borg_comment buf c =
336   match c with
337   | Borge_lang.Ast.Plain _ -> () (* skip (; machine comments *)
338   | Borge_lang.Ast.Annotated ac ->
339     let author =
340       match ac.Borge_lang.Ast.authorship with
341       | Borge_lang.Ast.Single a -> a
342       | Borge_lang.Ast.Multiple xs -> String.concat "," xs
343     in
344     let typ = match ac.Borge_lang.Ast.comment_type with
345     | Borge_lang.Ast.Untyped -> ""
346     | Borge_lang.Ast.Typed t -> " ^ t"
347     in
348     let body =
349       match ac.Borge_lang.Ast.value with
350       | Some v -> String.trim (string_value_text v)
351       | None -> ""
352     in
353     if body <> "" then begin

```

```

354     Buffer.add_string buf "\\begin{quote}\\n";
355     Printf.bprintf buf "\\textit{-- %s%s:} " (escape_text author)
(escape_text typ);
356     render_prose buf body;
357     Buffer.add_string buf "\\end{quote}\\n\\n"
358     end
359
360 let rec render_borg_node buf depth pending node =
361   let open Borge_lang.Ast in
362   let drain limit =
363     let rec loop () =
364       match !pending with
365       | (cpos, c) :: rest when cpos.offset < limit ->
366         render_borg_comment buf c;
367         pending := rest;
368         loop ()
369       | _ -> ()
370     in
371     loop ()
372   in
373   match node with
374   | Atom _ | String _ -> ()
375   | List (_pos, children) as lst ->
376     let head_is s = match children with Atom (_, x) :: _ -> x = s | _ ->
false in
377     let rest = match children with _ :: r -> r | [] -> [] in
378     if head_is "project" || head_is "section" || head_is "subsection" then
begin
379       let name = match rest with Atom (_, n) :: _ -> n | _ -> "" in
380       let cmd =
381         if head_is "project" then "section*"
382         else if head_is "section" then "subsection*"
383         else "subsubsection*"
384       in
385       (* Drain nested comments that appear after the keyword atom but
386        before the name atom (or first inner child). *)
387       (match rest with
388        | Atom (p, _) :: _ -> drain p.offset
389        | child :: _ -> drain (start_pos_of child).offset
390        | [] -> drain (end_pos_of lst).offset);
391       Printf.bprintf buf "\\s{%s}\\n" cmd (escape_text name);
392       let inner = match rest with _ :: r -> r | _ -> [] in
393       List.iter (fun child ->
394         drain (start_pos_of child).offset;
395         render_borg_node buf (depth + 1) pending child
396       ) inner;
397       drain (end_pos_of lst).offset
398     end
399     else if head_is "doc" then begin
400       match rest with String (_, v) :: _ -> render_prose buf
(string_value_text v) | _ -> ()
401     end
402     else if head_is "details" then begin
403       List.iter (fun child ->
404         drain (start_pos_of child).offset;
405         render_borg_node buf depth pending child
406       ) rest;
407       drain (end_pos_of lst).offset
408     end
409     else if head_is "status" then begin
410       match rest with Atom (_, s) :: _ ->
411         Printf.bprintf buf "\\textit{[status: %s]}\\par\\n" (escape_text s)
412       | _ -> ()
413     end
414     else if head_is "inline" then begin
415       match rest with String (_, v) :: _ ->
416         Printf.bprintf buf "\\textit{[inlines %s]}\\par\\n" (escape_text
(string_value_text v))
417       | _ -> ()
418     end
419     else () (* skip structural forms: verify, depends-on, convention, ...
*)
420

```

```

421 let borg_to_latex ~content =
422   let open Borge_lang.Ast in
423   let file =
424     try Borge_lang.Parse.parse_file content
425     with _ -> raise Fallback_raw
426   in
427   let buf = Buffer.create 4096 in
428   let sorted_nested =
429     List.stable_sort (fun (p1, _) (p2, _) -> compare p1.offset p2.offset)
430     file.nested_comments
431   in
432   let pending = ref sorted_nested in
433   List.iter (render_borg_comment buf)
434     file.Borge_lang.Ast.top_level_comments;
435   List.iter (fun swc ->
436     List.iter (render_borg_comment buf) swc.Borge_lang.Ast.comments_before;
437     render_borg_node buf 0 pending swc.Borge_lang.Ast.node
438   ) file.Borge_lang.Ast.top_level;
439   Buffer.contents buf
440
441 let render_to_tex ~dir ~files ~tex_path =
442   let src_dir = Filename.concat (Filename.dirname tex_path) "src" in
443   (try Unix.mkdir src_dir 0o755 with Unix.Unix_error (Unix.EEXIST, _, _) ->
444     ());
445   let oc = open_out tex_path in
446   output_string oc (preamble ());
447   output_string oc "\\begin{document}\n";
448   output_string oc "\\tableofcontents\n";
449   List.iteri (fun i f ->
450     let escaped = escape_text f in
451     let full = Filename.concat dir f in
452     Printf.fprintf oc
453       "\\clearpage\n\\section{%s}\n\\label{file:%s}\n\\markboth{%s}{%s}\n"
454       escaped f escaped escaped;
455     if Filename.check_suffix f ".borg" then begin
456       (* Render the .borg spec as prose (parsed), not raw sexp. Falls
457        back to a raw listing if the file fails to parse. *)
458       let raw = try File_utils.read_file full with Sys_error _ -> "" in
459       let body =
460         try Some (borg_to_latex ~content:(sanitize raw))
461         with Fallback_raw -> None
462       in
463       match body with
464       | Some b -> output_string oc b
465       | None ->
466         let sanitized_path = Filename.concat src_dir (Printf.sprintf
467           "%04d.borg" i) in
468         let soc = open_out sanitized_path in
469         output_string soc (sanitize raw);
470         close_out soc;
471         Printf.fprintf oc "\\lstinputlisting[firstnumber=1]{%s}\n"
472           sanitized_path
473       end else begin
474         let sanitized_path = Filename.concat src_dir (Printf.sprintf
475           "%04d.ml" i) in
476         let content = sanitize (try File_utils.read_file full with Sys_error
477           _ -> "") in
478         let soc = open_out sanitized_path in
479         output_string soc content;
480         close_out soc;
481         Printf.fprintf oc "\\lstinputlisting[firstnumber=1]{%s}\n"
482           sanitized_path
483       end
484     ) files;
485   output_string oc "\\end{document}\n";
486   close_out oc
487
488 let has_cmd name =
489   Sys.command (Printf.sprintf "command -v %s >/dev/null 2>&1" name) = 0
490
491 let file_hash path =
492   try
493     let content = File_utils.read_file path in

```

```

485     Digest.to_hex (Digest.string content)
486   with Sys_error _ -> ""
487
488   (* Run pdflatex until the .aux and .toc hashes stop changing, max 5
489   passes. A multi-page TOC grows across passes: pass 1 has no .toc
490   (TOC renders 1 page, page numbers wrong), pass 2 renders the full
491   TOC (pushing content down), pass 3 displays + records the corrected
492   page numbers. Looping on the .aux+.toc hash catches the stabilization
493   the way latexmk does. *)
494   let run_pdflatex ~tmpdir ~tex =
495     let cmd =
496       Printf.sprintf "pdflatex -interaction=nonstopmode -halt-on-error
497         -output-directory=%s %s >/dev/null 2>&1"
498         (Filename.quote tmpdir) (Filename.quote tex)
499     in
500     let aux = Filename.concat tmpdir "book.aux" in
501     let toc = Filename.concat tmpdir "book.toc" in
502     let rec loop ~prev_hash ~passes =
503       let _ = Sys.command cmd in
504       let h = file_hash aux ^ ":" ^ file_hash toc in
505       if passes >= 5 then ()
506       else if h = prev_hash then ()
507       else loop ~prev_hash:h ~passes:(passes + 1)
508     in
509     loop ~prev_hash:"" ~passes:1
510
511   (* Parse book.aux for \newlabel{file:PATH}{sec}{PAGE}{...}{...}{} entries.
512   Returns (path, start_page) for every labeled file. *)
513   let aux_parse ~aux =
514     let s = (try File_utils.read_file aux with Sys_error _ -> "") in
515     let re = Str.regexp
516       "\\newlabel{file:[^}+\\)}{[^}+\\)}{[^}+\\)}{\\([0-9]+\\)}" in
517     let rec loop start acc =
518       try
519         let _ = Str.search_forward re s start in
520         let path = Str.matched_group 1 s in
521         let page = int_of_string (Str.matched_group 2 s) in
522         loop (Str.match_end ()) ((path, page) :: acc)
523       with Not_found -> List.rev acc
524     in
525     loop 0 []
526
527   let copy_file ~src ~dst =
528     let ic = open_in src in
529     let len = in_channel_length ic in
530     let buf = Bytes.create len in
531     really_input ic buf 0 len;
532     close_in ic;
533     let oc = open_out dst in
534     output_bytes oc buf;
535     close_out oc
536
537   let rm_rf dir =
538     let rec walk d =
539       List.iter (fun name ->
540         let full = Filename.concat d name in
541         if Sys.is_directory full then walk full
542         else Sys.remove full
543       ) (Sys.readdir d |> Array.to_list)
544     in
545     try walk dir; Unix.rmdir dir with _ -> ()
546
547   (* Returns the number of files printed. On failure, keeps tmpdir
548   and prints the log path so the user can diagnose. *)
549   let print ~dir ~stem =
550     if not (has_cmd "pdflatex") then begin
551       if has_cmd "groff" then
552         failwith "groff backend not implemented in v1 (install
553           pdflatex/texlive)"
554       else
555         failwith "no PDF backend found: need pdflatex on PATH"
556     end;
557     let files = collect_files dir in

```

```

555   if files = [] then
556     failwith "no source files found";
557   let tmpdir =
558     let rec try_n n =
559       if n > 50 then failwith "could not create temp dir";
560       let name = Printf.sprintf "/tmp/borgebook%d_%d" (Unix.getpid ()) n in
561       (try Unix.mkdir name 0o700; name
562        with Unix.Unix_error (Unix.EEXIST, _, _) -> try_n (n + 1))
563     in
564     try_n 0
565   in
566   let tex = Filename.concat tmpdir "book.tex" in
567   try
568     render_to_tex ~dir ~files ~tex_path:tex;
569     run_pdflatex ~tmpdir ~tex;
570     let aux = Filename.concat tmpdir "book.aux" in
571     let page_map = aux_parse ~aux in
572     let file_pages = List.map (fun f ->
573       { Book_manifest.file = f;
574         start_page = (try List.assoc f page_map with Not_found -> 0) }
575     ) files in
576     let checksum = Book_manifest.tree_checksum ~dir files in
577     let manifest = Book_manifest.build ~stem ~backend:"latex-listings"
578       ~tree_checksum:checksum ~files ~file_pages in
579     let pdf = Filename.concat tmpdir "book.pdf" in
580     if not (Sys.file_exists pdf) then
581       failwith (Printf.sprintf "pdflatex produced no pdf; see %s/book.log"
582         tmpdir);
582     copy_file ~src:pdf ~dst:(stem ^ ".pdf");
583     Book_manifest.write ~path:(stem ^ ".book.manifest") manifest;
584     rm_rf tmpdir;
585     List.length files
586   with e ->
587     prerr_endline (Printf.sprintf "borge book: pdflatex failed; inspect
588       %s/book.log" tmpdir);
589     raise e

```

35 lib/bug/bug_ast.ml

```

1  (* Bug AST - structured representation of .borge-bug files *)
2
3  type status =
4    | Triage
5    | Open
6    | In_progress
7    | Resolved
8    | Closed
9
10 (* agent note (|
11  *   WHAT: Convert a bug status to its string representation for
12  *   serialization in .borge-bug files.
13  *
14  *   WHY: Bug files need to persist status as text.
15  * |) *)
16 let string_of_status = function
17   | Triage -> "trriage"
18   | Open -> "open"
19   | In_progress -> "in-progress"
20   | Resolved -> "resolved"
21   | Closed -> "closed"
22
23 (* agent note (|
24  *   WHAT: Parse a string into a bug status, handling both
25  *   hyphenated and underscore variants.
26  *
27  *   WHY: Reading bug files requires converting text back to status type.
28  * |) *)
29 let status_of_string = function
30   | "trriage" -> Some Triage
31   | "open" -> Some Open
32   | "in-progress" | "in_progress" -> Some In_progress
33   | "resolved" -> Some Resolved
34   | "closed" -> Some Closed
35   | _ -> None
36
37 type resolution = {
38   when_ : string;
39   how : string;
40   by : string;
41 }
42
43 type bug = {
44   id : string;
45   title : string;
46   status : status;
47   created : string;
48   filed_by : string;
49   doc : string;
50   affects_section : string option;
51   relates_to : string list;
52   drift_report_id : string option;
53   resolution : resolution option;
54 }
55
56 (* agent note (|
57  *   WHAT: Generate a unique bug ID using the current timestamp.
58  *   Format: BORGE-{unix_timestamp}
59  *
60  *   WHY: New bugs need IDs for tracking and file naming.
61  * |) *)
62 let make_id () =
63   let ts = string_of_int (int_of_float (Unix.time ())) in
64   "BORGE-" ^ ts
65
66 (* agent note (|
67  *   WHAT: Create an empty bug record with default/placeholder values.
68  *
69  *   WHY: Starting point for creating a new bug.
70  * |) *)
71 let empty_bug = {

```

```
72   id = "";
73   title = "";
74   status = Triage;
75   created = "";
76   filed_by = "";
77   doc = "";
78   affects_section = None;
79   relates_to = [];
80   drift_report_id = None;
81   resolution = None;
82 }
```


36 lib/bug/bug_parse.ml

```

1  (* Parser for .borge-bug files *)
2
3  open Bug_ast
4  open Borge_lang.Ast
5
6  let extract_string sexp =
7      match sexp with
8      | Atom (_, s) -> s
9      | _ -> ""
10
11  let extract_sexp_list sexp =
12      match sexp with
13      | List (_, l) -> l
14      | _ -> []
15
16  let parse_status s =
17      match status_of_string s with
18      | Some st -> st
19      | None -> Triage
20
21  let parse_resolution = function
22      | List (_, sexps) ->
23          let fields = List.filter_map (function
24              | List (_, [Atom (_, "when"); Atom (_, v)]) -> Some ("when", v)
25              | List (_, [Atom (_, "how"); Atom (_, v)]) -> Some ("how", v)
26              | List (_, [Atom (_, "by"); Atom (_, v)]) -> Some ("by", v)
27              | _ -> None
28          ) sexps in
29          let find f = List.assoc_opt f fields in
30          (match find "when", find "how", find "by" with
31              | Some when_, Some how, Some by -> Some { when_; how; by }
32              | _ -> None)
33      | _ -> None
34
35  let parse_bug sexp =
36      match sexp with
37      | List (_, Atom (_, "borge-bug") :: List (_, [Atom (_, "id"); Atom (_, id)]) :: rest) ->
38          let bug = { empty_bug with id } in
39          List.fold_left (fun bug sexp ->
40              match sexp with
41              | List (_, [Atom (_, "title"); Atom (_, title)]) ->
42                  { bug with title }
43              | List (_, [Atom (_, "status"); Atom (_, status)]) ->
44                  { bug with status = parse_status status }
45              | List (_, [Atom (_, "created"); Atom (_, created)]) ->
46                  { bug with created }
47              | List (_, [Atom (_, "filed-by"); Atom (_, filed_by)]) ->
48                  { bug with filed_by }
49              | List (_, [Atom (_, "doc"); Atom (_, doc)]) ->
50                  { bug with doc }
51              | List (_, [Atom (_, "affects"); List (_, [Atom (_, "section"); Atom (_, section)])]) ->
52                  { bug with affects_section = Some section }
53              | List (_, [Atom (_, "relates-to"); List (_, [Atom (_, "bug"); Atom (_, bug_id)])]) ->
54                  { bug with relates_to = bug_id :: bug.relates_to }
55              | List (_, [Atom (_, "drift-report-id"); Atom (_, drift_id)]) ->
56                  { bug with drift_report_id = Some drift_id }
57              | List (_, Atom (_, "resolution") :: res_fields) ->
58                  { bug with resolution = parse_resolution (List ({ line = 0; col = 0; offset = 0 }, res_fields)) }
59              | _ -> bug
60          ) bug rest
61      | _ -> empty_bug
62
63  let parse_file path =
64      let content = File_utils.read_file path in
65      match Borge_lang.Parse.parse content with
66      | file ->
67          (match file.Borge_lang.Ast.top_level with

```

```
68 | [] -> empty_bug
69 | first :: _ -> parse_bug first.node)
70 | exception _ -> empty_bug
```

37 lib/bug/bug_registry.ml

```

1 (* Bug registry - manage .borge-bug files *)
2
3 open Bug_ast
4
5 let default_dir = ".borge-bugs"
6
7 let ensure_dir () =
8   if not (Sys.file_exists default_dir) then
9     Unix.mkdir default_dir 0o755
10
11 let bug_path id =
12   Filename.concat default_dir (id ^ ".borge-bug")
13
14 let list_bugs ~(status=None) () =
15   ensure_dir ();
16   let files = Array.to_list (Sys.readdir default_dir) in
17   let bug_files = List.filter (fun f -> Filename.check_suffix f
18     ".borge-bug") files in
19   let bugs = List.map (fun f ->
20     let path = Filename.concat default_dir f in
21     Bug_parse.parse_file path
22   ) bug_files in
23   match status with
24   | None -> bugs
25   | Some st -> List.filter (fun b -> b.status = st) bugs
26
27 let load_bug id =
28   let path = bug_path id in
29   if Sys.file_exists path then
30     Some (Bug_parse.parse_file path)
31   else None
32
33 let save_bug bug =
34   ensure_dir ();
35   let path = bug_path bug.id in
36   Bug_write.write bug path;
37   path
38
39 let close_bug id ~resolution ~by =
40   match load_bug id with
41   | None -> Error "Bug not found"
42   | Some bug ->
43     let res = {
44       when_ = string_of_int (int_of_float (Unix.time ())),
45       how = resolution;
46       by;
47     } in
48     let updated = { bug with status = Closed; resolution = Some res } in
49     let path = save_bug updated in
50     Ok path

```

38 lib/bug/bug_write.ml

```

1 (* Writer for .borge-bug files *)
2
3 open Bug_ast
4
5 let render_resolution (res : resolution) =
6   Printf.sprintf " (resolution (when %s) (how %s) (by %s))"
7     res.when_ res.how res.by
8
9 let render bug =
10  let lines = [
11    Printf.sprintf "(borge-bug (id \"%s\")" bug.id;
12    Printf.sprintf " (title \"%s\")" bug.title;
13    Printf.sprintf " (status %s)" (string_of_status bug.status);
14    Printf.sprintf " (created %s)" bug.created;
15    Printf.sprintf " (filed-by %s)" bug.filed_by;
16    Printf.sprintf " (doc \"%s\")" bug.doc;
17  ] in
18  let lines = match bug.affects_section with
19    | Some section -> lines @ [Printf.sprintf " (affects (section
20      \"%s\")" section]
21    | None -> lines
22  in
23  let lines = if bug.relates_to <> [] then
24    lines @ List.map (fun id -> Printf.sprintf " (relates-to (bug
25      \"%s\")" id) bug.relates_to
26  else lines
27  in
28  let lines = match bug.drift_report_id with
29    | Some id -> lines @ [Printf.sprintf " (drift-report-id \"%s\")" id]
30    | None -> lines
31  in
32  let lines = match bug.resolution with
33    | Some res -> lines @ [render_resolution res]
34    | None -> lines
35  in
36  lines @ ["]
37  |> String.concat "\n"
38
39 let write bug path =
40  let oc = open_out path in
41  output_string oc (render bug);
42  output_string oc "\n";
43  close_out oc

```

39 lib/distributed/borg_comment.ml

```

1  (* Parse borg-comments from OCaml source files
2
3  Borg-comments are OCaml comments containing valid borg s-expressions:
4  (* (fn name (doc "...") (exports ...)) *)
5
6  This module extracts them for drift checking against module specs. *)
7
8  type borg_comment = {
9    form_type : string; (* "fn", "val", "type", etc. *)
10   name : string;
11   properties : (string * string) list;
12   raw : string;
13   line : int;
14 }
15
16 let empty = {
17   form_type = "";
18   name = "";
19   properties = [];
20   raw = "";
21   line = 0;
22 }
23
24 (* Check if a string looks like a borg form: starts with (fn, (val, etc. *)
25 let is_borg_form s =
26   let trimmed = String.trim s in
27   if String.length trimmed < 2 then false
28   else if String.get trimmed 0 <> '(' then false
29   else
30     let rest = String.sub trimmed 1 (String.length trimmed - 1) in
31     List.exists (fun prefix ->
32       String.length rest >= String.length prefix &&
33       String.sub rest 0 (String.length prefix) = prefix
34     ) ["fn "; "val "; "type "; "module "; "doc "; "exports "; "since "]
35
36 (* Extract the form type and name from a borg s-exp *)
37 let extract_form_info sexp =
38   match sexp with
39   | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_, form_type) :: rest) ->
40     let name = match rest with
41       | Borge_lang.Ast.Atom (_, n) :: _ -> n
42       | _ -> ""
43     in
44     Some (form_type, name)
45   | _ -> None
46
47 (* Parse a single comment content *)
48 let parse_comment content ~line =
49   if not (is_borg_form content) then None
50   else
51     match Borge_lang.Parse.parse content with
52     | file ->
53       (match file.Borge_lang.Ast.top_level with
54        | [] -> None
55        | sexp_with_comments :: _ ->
56          match extract_form_info sexp_with_comments.Borge_lang.Ast.node
57            with
58            | Some (form_type, name) ->
59              Some { empty with form_type; name; raw = content; line }
60            | None -> None)
61     | exception _ -> None
62
63 (* Extract all borg-comments from OCaml source *)
64 let extract_from_source content =
65   let lines = String.split_on_char '\n' content in
66   let comments = ref [] in
67   let current_comment = ref [] in
68   let in_comment = ref false in
69   let start_line = ref 0 in
70   let line_num = ref 0 in

```

```

71 List.iter (fun line ->
72   incr line_num;
73   let trimmed = String.trim line in
74
75   (* Check for comment start *)
76   if not !in_comment then begin
77     if String.length trimmed >= 2 && String.sub trimmed 0 2 = "(" then
78       begin
79         in_comment := true;
80         start_line := !line_num;
81         current_comment := [line]
82       end
83     else begin
84       (* Inside comment, accumulate *)
85       current_comment := line :: !current_comment;
86
87       (* Check for comment end *)
88       if String.length trimmed >= 2 &&
89         (String.sub trimmed (String.length trimmed - 2) 2 = ")" ||
90          String.contains trimmed '*' && String.contains line ')') then
91         begin
92           in_comment := false;
93           let comment_text = String.concat "\n" (List.rev !current_comment) in
94           (match parse_comment comment_text ~line:!start_line with
95            | Some bc -> comments := bc :: !comments
96            | None -> ());
97           current_comment := []
98         end
99       end
100   ) lines;
101
102   List.rev !comments
103
104 (* Find all functions defined with borg-comments in a file *)
105 let find_functions path =
106   let content = File_utils.read_file path in
107   extract_from_source content
108   |> List.filter (fun bc -> bc.form_type = "fn" || bc.form_type = "val")
109
110 (* Get just the names for drift checking *)
111 let function_names path =
112   find_functions path
113   |> List.map (fun bc -> bc.name)
114
115 (* Render back to borg format for storage *)
116 let render_to_borg comments =
117   List.map (fun bc ->
118     Printf.sprintf "(fn %s%s)"
119       bc.name
120       (if bc.properties = [] then ""
121        else " " ^ String.concat " " (List.map (fun (k,v) -> Printf.sprintf
122          "(%s %s)" k v) bc.properties))
123   ) comments
124   |> String.concat "\n"

```

40 lib/distributed/module_spec.ml

```

1  (* Module spec discovery and parsing for distributed specs
2
3  Each module folder has its own .borg file describing exports.
4  This module discovers and parses them. *)
5
6  type module_def = {
7    path : string;          (* Path to the .borg file *)
8    module_name : string;   (* Derived from filename or spec *)
9    exports : string list;  (* Function/val names exported *)
10   raw_spec : string;      (* Raw borg content *)
11 }
12
13 (* Find all .borg files in subdirectories (excluding root) *)
14 let discover_specs dir =
15   let rec walk path acc =
16     try
17       let entries = Sys.readdir path in
18       Array.fold_left (fun acc entry ->
19         let full_path = Filename.concat path entry in
20         if Sys.is_directory full_path then
21           if entry = ".git" || entry = "_build" || entry = ".borge-bugs"
22         then
23           acc
24         else
25           walk full_path acc
26         else if Filename.check_suffix entry ".borg" && full_path <>
27           (Filename.concat dir "borge.borg") then
28             full_path :: acc
29         else
30           acc
31       ) acc entries
32     with _ -> acc
33   in
34   walk dir []
35
36 (* Extract exports from a parsed borg s-exp *)
37 let extract_exports sexp =
38   let rec find_exports = function
39     | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_, "exports") :: rest) ->
40       List.filter_map (function
41         | Borge_lang.Ast.Atom (_, name) -> Some name
42         | _ -> None
43       ) rest
44     | Borge_lang.Ast.List (_, items) ->
45       List.concat_map find_exports items
46     | _ -> []
47   in
48   match sexp with
49   | Borge_lang.Ast.List (_, _) -> find_exports sexp
50   | _ -> []
51
52 (* Parse a single module spec file *)
53 let parse_module_spec path =
54   let content = File_utils.read_file path in
55   let filename = Filename.basename path in
56   let module_name = Filename.remove_extension filename in
57
58   match Borge_lang.Parse.parse content with
59   | file ->
60     let exports = List.concat_map (fun sexp_with_comments ->
61       extract_exports sexp_with_comments.Borge_lang.Ast.node
62     ) file.Borge_lang.Ast.top_level in
63     Some { path; module_name; exports; raw_spec = content }
64   | exception _ ->
65     (* Failed to parse, return empty spec *)
66     Some { path; module_name; exports = []; raw_spec = content }
67
68 (* Get module specs for all discovered .borg files *)
69 let load_all_specs dir =
70   discover_specs dir
71   |> List.filter_map parse_module_spec

```

```

70
71 (* Find functions in implementation files that correspond to module spec *)
72 let find_implementation_files module_path =
73   let dir = Filename.dirname module_path in
74   try
75     Sys.readdir dir
76   |> Array.to_list
77   |> List.filter (fun f ->
78     Filename.check_suffix f ".ml" &&
79     not (Filename.check_suffix f "_test.ml"))
80   |> List.map (fun f -> Filename.concat dir f)
81   with _ -> []
82
83 (* Check referential integrity: names in spec vs names in code *)
84 let check_integrity module_spec impl_path =
85   let functions_in_code = Borg_comment.function_names impl_path in
86   let exports_set = List.map String.lowercase_ascii module_spec.exports in
87   let code_set = List.map String.lowercase_ascii functions_in_code in
88
89   (* Missing: in spec but not in code *)
90   let missing = List.filter (fun name ->
91     not (List.mem (String.lowercase_ascii name) code_set)
92   ) module_spec.exports in
93
94   (* Undocumented: in code but not listed in spec exports *)
95   let undocumented = List.filter (fun name ->
96     not (List.mem (String.lowercase_ascii name) exports_set)
97   ) functions_in_code in
98
99   (missing, undocumented)
100
101 (* Format an integrity report *)
102 let render_report module_name (missing, undocumented) =
103   let lines = ref [] in
104
105   if missing <> [] then begin
106     lines := Printf.sprintf " Missing implementations (in spec, not in
107       code):" :: !lines;
108     List.iter (fun name ->
109       lines := Printf.sprintf "    - %s" name :: !lines
110     ) missing
111   end;
112
113   if undocumented <> [] then begin
114     lines := Printf.sprintf " Undocumented functions (in code, not in
115       exports):" :: !lines;
116     List.iter (fun name ->
117       lines := Printf.sprintf "    - %s" name :: !lines
118     ) undocumented
119   end;
120
121   if !lines = [] then
122     Printf.sprintf "%s: ? all exports documented" module_name
123   else
124     Printf.sprintf "%s:\n%s" module_name (String.concat "\n" (List.rev !lines))

```


41 lib/doc/doc_coverage.ml

```

1  (* Calculate documentation coverage statistics
2  *
3  * agent note (|
4  *   WHAT: Computes per-file and project-wide documentation coverage
5  *   statistics against OCaml let bindings. Generates (documentation ...)
6  *   sexp blocks for .borg.meta files.
7  *
8  *   WHY: borge drift --docs needs machine-readable coverage data to
9  *   enforce documentation thresholds. The coverage calculation must
10  *   distinguish documented, exempt, drifted, and undocumented bindings.
11  * |) *)
12
13
14
15  (** Coverage statistics for a file *)
16  type file_coverage = {
17    path : string;
18    total_bindings : int;
19    documented : int;
20    exempt : int;
21    drifted : int;  (** Documented but marked (status drifted) *)
22    undocumented : int;
23    coverage_percent : float;
24    undocumented_names : string list;
25    drifted_names : string list;
26  }
27
28  (** Project-wide coverage summary *)
29  type project_coverage = {
30    files : file_coverage list;
31    total_files : int;
32    total_bindings : int;
33    total_documented : int;
34    total_exempt : int;
35    total_drifted : int;
36    total_undocumented : int;
37    overall_percent : float;
38  }
39
40  (* agent note (|
41  *   WHAT: Calculate documentation coverage for a single .ml file.
42  *   Extracts bindings and docs, matches them, and computes coverage
43  *   percentages. Skips test and internal bindings.
44  *
45  *   WHY: Per-file coverage is the unit of reporting for drift --docs.
46  *   Each file gets its own coverage stats and appears in the summary.
47  * |) *)
48  let calculate_file_coverage path =
49    let bindings = Doc_extract.extract_bindings path in
50    let binding_docs = Doc_detect.extract_file_docs path in
51
52    (* Match bindings with their docs *)
53    let matched = List.filter_map (fun (b : Doc_extract.binding_info) ->
54      if b.is_test || b.is_internal then
55        None  (* Skip tests and internal *)
56      else
57        let doc = List.find_opt (fun (bd : Doc_detect.binding_doc) ->
58          bd.binding_line = b.line
59        ) binding_docs in
60        Some (b, doc)
61    ) bindings in
62
63    let total = List.length matched in
64    let documented = List.filter (fun (_, doc_opt) ->
65      match doc_opt with
66      | Some bd -> Doc_detect.binding_has_doc bd
67      | None -> false
68    ) matched |> List.length
69    in
70    let exempt = List.filter (fun (_, doc_opt) ->
71      match doc_opt with

```

```

72 | Some bd -> Doc_detect.binding_is_exempt bd
73 | None -> false
74 ) matched |> List.length
75 in
76 let drifted = List.filter (fun (_, doc_opt) ->
77   match doc_opt with
78   | Some bd -> Doc_detect.binding_is_drifted bd &&
     Doc_detect.binding_has_doc bd
79   | None -> false
80 ) matched |> List.length
81 in
82 let drifted_names = List.filter_map (fun ((b : Doc_extract.binding_info),
     doc_opt) ->
83   match doc_opt with
84   | Some bd when Doc_detect.binding_is_drifted bd -> Some b.name
85   | _ -> None
86 ) matched in
87 let undocumented = total - documented - exempt in
88 let undoc_names = List.filter_map (fun ((b : Doc_extract.binding_info),
     doc_opt) ->
89   match doc_opt with
90   | Some bd when not (Doc_detect.binding_has_doc bd ||
     Doc_detect.binding_is_exempt bd) ->
91     Some b.name
92   | None -> Some b.name
93   | _ -> None
94 ) matched in
95
96 let percent =
97   if total - exempt <= 0 then 100.0
98   else float_of_int documented /. float_of_int (total - exempt) *. 100.0
99 in
100
101 {
102   path;
103   total_bindings = total;
104   documented;
105   exempt;
106   drifted;
107   undocumented;
108   coverage_percent = percent;
109   undocumented_names = undoc_names;
110   drifted_names;
111 }
112
113 (* agent note (|
114  * WHAT: Calculate documentation coverage for all .ml files
115  * under a directory, aggregating per-file stats into a project summary.
116  *
117  * WHY: borge drift --docs needs project-wide coverage to enforce
118  * thresholds at the repository level.
119  * |) *)
120 let calculate_dir_coverage dir =
121   let ml_files = Doc_extract.find_ml_files dir in
122   let file_coverages = List.map calculate_file_coverage ml_files in
123
124   let total_bindings = List.fold_left (fun acc (fc : file_coverage) ->
125     acc + fc.total_bindings
126   ) 0 file_coverages in
127   let total_documented = List.fold_left (fun acc fc ->
128     acc + fc.documented
129   ) 0 file_coverages in
130   let total_exempt = List.fold_left (fun acc fc ->
131     acc + fc.exempt
132   ) 0 file_coverages in
133   let total_drifted = List.fold_left (fun acc fc ->
134     acc + fc.drifted
135   ) 0 file_coverages in
136   let total_undocumented = List.fold_left (fun acc fc ->
137     acc + fc.undocumented
138   ) 0 file_coverages in
139
140   let overall =

```

```

141   if total_bindings - total_exempt <= 0 then 100.0
142   else float_of_int total_documented /. float_of_int (total_bindings -
143     total_exempt) *. 100.0
144   in
145   {
146     files = file_coverages;
147     total_files = List.length file_coverages;
148     total_bindings;
149     total_documented;
150     total_exempt;
151     total_drifted;
152     total_undocumented;
153     overall_percent = overall;
154   }
155
156 (* agent note (|
157  *   WHAT: Format per-file coverage as a human-readable string:
158  *   "path: N/M documented (X.X%), E exempt, U undocumented, D drifted"
159  *
160  *   WHY: Used by drift --docs to display per-file results.
161  *   Drifted count is only shown when > 0 to avoid noise.
162  * |) *)
163 let format_coverage (fc : file_coverage) =
164   let base = Printf.sprintf "%s: %d/%d documented (%.1f%%), %d exempt, %d
165     undocumented"
166     fc.path fc.documented fc.total_bindings fc.coverage_percent
167     fc.exempt fc.undocumented in
168   if fc.drifted > 0 then
169     base ^ Printf.sprintf ", %d drifted" fc.drifted
170   else
171     base
172
173 (* agent note (|
174  *   WHAT: Format project-wide coverage as a multi-line summary
175  *   showing total, documented, exempt, drifted, undocumented
176  *   counts, plus a list of files with undocumented bindings.
177  *
178  *   WHY: The primary output format for borge drift --docs.
179  *   Gives humans an at-a-glance view of project documentation health.
180  * |) *)
181 let format_project_summary (pc : project_coverage) =
182   let drifted_line =
183     if pc.total_drifted > 0 then [Printf.sprintf " Drifted: %d"
184       pc.total_drifted] else []
185   in
186   let undoc_files =
187     if pc.total_undocumented > 0 then
188       " " :: "Files with undocumented bindings:" ::
189       List.filter_map (fun (fc : file_coverage) ->
190         if fc.undocumented > 0 then
191           Some (Printf.sprintf " %s (%d undocumented)" fc.path
192             fc.undocumented)
193         else None
194       ) pc.files
195     else []
196   in
197   String.concat "\n" (
198     Printf.sprintf "Documentation Coverage Summary (%d files):"
199     pc.total_files ::
200     Printf.sprintf " Total bindings: %d" pc.total_bindings ::
201     Printf.sprintf " Documented: %d (%.1f%%)" pc.total_documented
202     pc.overall_percent ::
203     Printf.sprintf " Exempt: %d" pc.total_exempt ::
204     drifted_line @
205     Printf.sprintf " Undocumented: %d" pc.total_undocumented ::
206     undoc_files
207   )
208
209 (* agent note (|
210  *   WHAT: Generate a (documentation ...) sexp block for a .borg.meta
211  *   file, containing timestamp, file path, all coverage metrics,
212  *   and lists of drifted and undocumented binding names.

```

```

208 *
209 *   WHY: .borg.meta files are the machine-readable companion to .borg
210 *   specs. Borge stats and other tools read these blocks to produce
211 *   project-wide reports without re-scanning source files.
212 * |) *)
213 let generate_meta_block (fc : file_coverage) =
214   let timestamp =
215     let now = Unix.gmtime (Unix.time ()) in
216     Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
217       (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
218       now.tm_hour now.tm_min now.tm_sec
219   in
220   let buf = Buffer.create 512 in
221   Buffer.add_string buf "(documentation\n";
222   Buffer.add_string buf (Printf.sprintf " (analyzed-at \"%s\")\n"
223     timestamp);
224   Buffer.add_string buf (Printf.sprintf " (file \"%s\")\n" fc.path);
225   Buffer.add_string buf (Printf.sprintf " (total-bindings %d)\n"
226     fc.total_bindings);
227   Buffer.add_string buf (Printf.sprintf " (documented %d)\n"
228     fc.documented);
229   Buffer.add_string buf (Printf.sprintf " (exempt %d)\n" fc.exempt);
230   Buffer.add_string buf (Printf.sprintf " (drifted %d)\n" fc.drifted);
231   Buffer.add_string buf (Printf.sprintf " (undocumented %d)\n"
232     fc.undocumented);
233   Buffer.add_string buf (Printf.sprintf " (coverage-percent %.2f)\n"
234     fc.coverage_percent);
235   if fc.drifted_names <> [] then begin
236     Buffer.add_string buf " (drifted-names\n";
237     List.iter (fun name ->
238       Buffer.add_string buf (Printf.sprintf " (name \"%s\")\n" name)
239     ) fc.drifted_names;
240     Buffer.add_string buf " )\n";
241   end;
242   if fc.undocumented_names <> [] then begin
243     Buffer.add_string buf " (undocumented-names\n";
244     List.iter (fun name ->
245       Buffer.add_string buf (Printf.sprintf " (name \"%s\")\n" name)
246     ) fc.undocumented_names;
247     Buffer.add_string buf " ))\n";
248   end;
249   Buffer.add_string buf ")\n";
250   Buffer.contents buf
251
252 (* agent note (|
253 *   WHAT: Write a coverage metadata block to a .borg.meta file path.
254 *
255 *   WHY: Persists coverage data between runs so that drift --stale
256 *   can skip files that haven't changed.
257 * |) *)
258 let write_coverage_meta fc meta_path =
259   let content = generate_meta_block fc in
260   let oc = open_out meta_path in
261   output_string oc content;
262   close_out oc
263
264 (* agent note (|
265 *   WHAT: Check if a file's coverage meets the given threshold percentage.
266 *
267 *   WHY: Used by drift --docs to determine exit codes: files below
268 *   threshold produce warnings or errors depending on --strict.
269 * |) *)
270 let meets_threshold (fc : file_coverage) threshold =
271   fc.coverage_percent >= threshold

```

42 lib/doc/doc_detect.ml

```

1 (* Detect documentation comments and exempt markers in OCaml code
2  *
3  * agent note (|
4  *   WHAT: Identifies documentation formats, exempt markers, and drift
5  *   status markers preceding let bindings in OCaml source files.
6  *   Supports docstrings, borg-annotated comments, short borg comments,
7  *   exempt markers, and (status drifted) detection.
8  *
9  *   WHY: Downstream tools (doc_coverage, lint) need to know whether
10  *   a binding is documented, exempt, or has a known-stale comment.
11  *   The drifted status lets us distinguish between "no comment" and
12  *   "comment exists but is known to be inaccurate."
13  * |) *)
14
15 (** Literacy score for a comment: Story, Explain, Teach, Edge.
16     Each digit 0--7. Zeros are honest defaults. *)
17 type literacy_score = {
18   story : int;
19   explain : int;
20   teach : int;
21   edge : int;
22 }
23
24 (** Types of documentation found *)
25 type doc_kind =
26   | Docstring of string                (** (** ... *) *)
27   | Borg_note of string * string * literacy_score option (** author,
28   content, score *)
29   | Borg_short of string              (** (*| ... |*) *)
30   | Exempt_marker                    (** (* exempt doc *) *)
31
32 (** Where the doc was found relative to binding *)
33 type doc_position =
34   | Preceding of int                  (** N lines before binding *)
35   | Inline                          (** Same line as binding - rare *)
36   | None_found
37
38 (* agent note [5300] (|
39  *   WHAT: Check whether a comment string contains a (status drifted)
40  *   marker.
41  *   Scans the entire text for the literal substring "(status drifted)".
42  *
43  *   WHY: The (status drifted) marker signals that a comment is known
44  *   to be out of date with its function's implementation. This is
45  *   better than silently inaccurate documentation -- it makes the
46  *   lie explicit and actionable.
47  * |) *)
48 let has_drifted_marker text =
49   let rec loop i =
50     if i + 15 > String.length text then false
51     else if (* exempt: String.sub *) String.sub text i 15 = "(status
52     drifted)" then true
53     else loop (i + 1)
54   in
55   loop 0
56
57 (* agent note (|
58  *   WHAT: Check whether any doc_kind variant contains a drifted marker.
59  *
60  *   WHY: When extracting docs, we need to know if the comment is
61  *   drifted so we can classify it separately in coverage reports --
62  *   drifted comments count as documented but are flagged for update.
63  * |) *)
64 let doc_kind_is_drifted = function
65   | Docstring content -> has_drifted_marker content
66   | Borg_note (_, content, _) -> has_drifted_marker content
67   | Borg_short content -> has_drifted_marker content
68   | Exempt_marker -> false
69
70 (* agent note (|
71  *   WHAT: Parse a 4-digit literacy score [STEX] from a string.

```

```

69  *   Each digit 0--7 representing Story, Explain, Teach, Edge.
70  *   Returns None if no valid score pattern found.
71  * |) *)
72  let parse_literacy_score s =
73    try
74      let open_idx = String.index s '[' in
75      let close_idx = String.index_from s (open_idx + 1) ']' in
76      let digits = String.sub s (open_idx + 1) (close_idx - open_idx - 1) in
77      if String.length digits = 4 then
78        let is_digit c = c >= '0' && c <= '9' in
79        if is_digit digits.[0] && is_digit digits.[1] &&
80          is_digit digits.[2] && is_digit digits.[3] then
81          Some {
82            story = Char.code digits.[0] - Char.code '0';
83            explain = Char.code digits.[1] - Char.code '0';
84            teach = Char.code digits.[2] - Char.code '0';
85            edge = Char.code digits.[3] - Char.code '0';
86          }
87        else None
88      else None
89    with _ -> None
90
91  (** Documentation info for a binding *)
92  type binding_doc = {
93    binding_line : int;
94    doc : (doc_kind * doc_position) option;
95    is_drifted : bool; (** True when comment has (status drifted) marker *)
96  }
97
98  (* agent note (|
99  *   WHAT: Check if a line starts a standard OCaml docstring:
100  *   open-paren-star-star.
101  *
102  *   Excludes three-star which is a documentation-comment escape.
103  *
104  *   WHY: Standard docstrings are the most common OCaml documentation
105  *   format and must be recognized to calculate accurate coverage.
106  * |) *)
107  let is_docstring_start line =
108    String.starts_with ~prefix:"(**" (String.trim line) &&
109    not (String.starts_with ~prefix:"(***)" (String.trim line))
110
111  (* agent note (|
112  *   WHAT: Check if a line is a borg-annotated comment in the form
113  *   open-paren-star author note verbatim-content close-paren-star.
114  *   Validates the structure: open-paren-star then author name then
115  *   "note" then content.
116  *
117  *   WHY: Borg-annotated comments are the preferred documentation format
118  *   in borge projects because they carry authorship and can be
119  *   integrity-checked.
120  * |) *)
121  let is_borg_note line =
122    let trimmed = String.trim line in
123    if String.length trimmed < 4 then false
124    else if String.sub trimmed 0 2 = "(*" then
125      let inner = String.sub trimmed 2 (String.length trimmed - 2) in
126      let inner = String.trim inner in
127      (try
128        let space = String.index inner ' ' in
129        let word = (* exempt: String.sub *) String.sub inner 0 space in
130        let rest = String.sub inner space (String.length inner - space) |>
131          String.trim in
132        String.length word > 0 &&
133        word.[0] >= 'a' && word.[0] <= 'z' && (* lowercase name *)
134        String.starts_with ~prefix:"note " rest
135        with _ -> false)
136      else false
137
138  (* agent note (|
139  *   WHAT: Check if a line is a short borg comment using the
140  *   open-paren-pipe / close-pipe-paren format.
141  *
142  *   WHY: Short borg comments provide a lighter-weight alternative

```

```

139 *   to full borg-annotated comments when authorship tracking isn't needed.
140 * |) *)
141 let is_borg_short line =
142   let trimmed = String.trim line in
143   String.starts_with ~prefix:"(*|" trimmed
144
145 (* agent note (|
146 *   WHAT: Check if a line is an exemption marker using the
147 *   exempt-doc comment convention.
148 *
149 *   WHY: Some bindings are internal helpers whose purpose is obvious
150 *   from their call site. The exemption marker lets the author
151 *   explicitly opt out of documentation enforcement.
152 * |) *)
153 let is_exempt_marker line =
154   let trimmed = String.trim line in
155   if String.length trimmed >= 4 &&
156     String.sub trimmed 0 2 = "(" &&
157     String.sub trimmed (String.length trimmed - 2) 2 = ")" then
158     let inner = String.sub trimmed 2 (String.length trimmed - 4) |>
159       String.trim in
160     inner = "exempt" ||
161     inner = "exempt doc" ||
162     String.starts_with ~prefix:"exempt doc " inner ||
163     String.starts_with ~prefix:"exempt doc:" inner
164   else false
165
166 (* agent note (|
167 *   WHAT: Strip the (* and *) delimiters from a comment line,
168 *   returning the inner content.
169 *
170 *   WHY: Used by parse_borg_note and parse_doc_line to extract
171 *   the meaningful content from within comment delimiters.
172 * |) *)
173 let extract_comment_content line =
174   let trimmed = String.trim line in
175   if String.length trimmed < 4 then None
176   else
177     let inner = String.sub trimmed 2 (String.length trimmed - 4) |>
178       String.trim in
179     Some inner
180
181 (* agent note (|
182 *   WHAT: Parse a borg-annotated comment line to extract author and
183 *   content. Handles the borg-note format with author and
184 *   verbatim-wrapped content,
185 *   stripping the (|...|) wrapper if present.
186 *
187 *   WHY: Borg-annotated comments carry structured authorship data
188 *   that the linter uses for integrity checks. Parsing out the
189 *   author and content separately enables both lint and review.
190 * |) *)
191 let parse_borg_note line =
192   match extract_comment_content line with
193   | None -> None
194   | Some inner ->
195     try
196       let space = String.index inner ' ' in
197       let author = (* exempt: String.sub *) String.sub inner 0 space in
198       let rest = String.sub inner (space + 1) (String.length inner -
199         space - 1) |> String.trim in
200       if String.starts_with ~prefix:"note " rest then
201         let note_raw = String.sub rest 5 (String.length rest - 5) |>
202           String.trim in
203         let score = parse_literacy_score note_raw in
204         let note_after_score =
205           match score with
206           | Some _ ->
207             (try
208               let close_idx = String.index note_raw ']' in
209               String.sub note_raw (close_idx + 1) (String.length
210                 note_raw - close_idx - 1) |> String.trim
211             with _ -> note_raw)

```

```

207         | None -> note_raw
208     in
209     (* Remove surrounding (| ... |) if present *)
210     let note_content =
211         if String.starts_with ~prefix:"(|" note_after_score then
212             let len = String.length note_after_score in
213             if len >= 2 && String.sub note_after_score (len - 2) 2 = "|)"
214         then
215             String.sub note_after_score 2 (len - 4) |> String.trim
216         else note_after_score
217     in
218     Some (author, note_content, score)
219     else None
220     with _ -> None
221
222 (* agent note (|
223  * WHAT: Classify a single line as one of the four doc kinds
224  * (Docstring, Borg_note, Borg_short, Exempt_marker) or return
225  * None if the line is not a documentation comment.
226  *
227  * WHY: The doc detection pipeline needs to identify what kind
228  * of documentation precedes each binding so coverage and lint
229  * can apply the right rules.
230  * |) *)
231 let parse_doc_line line =
232     if is_exempt_marker line then Some Exempt_marker
233     else if is_docstring_start line then
234         extract_comment_content line |> Option.map (fun c -> Docstring c)
235     else if is_borg_short line then
236         let trimmed = String.trim line in
237         let inner = String.sub trimmed 3 (String.length trimmed - 5) |>
238             String.trim in
239         Some (Borg_short inner)
240     else if is_borg_note line then
241         parse_borg_note line |> Option.map (fun (a, c, s) -> Borg_note (a, c,
242             s))
243     else None
244
245 (* agent note (|
246  * WHAT: Find documentation for a binding at a specific line number
247  * by scanning backward from the binding line through preceding lines.
248  *
249  * WHY: Used for targeted lookups when we know a binding's line
250  * number and just need its associated doc comment.
251  * |) *)
252 let find_binding_doc lines binding_line =
253     let rec scan prev_lines remaining prev_line_no =
254         match remaining with
255         | [] -> None_found
256         | line :: rest ->
257             let line_no = prev_line_no + 1 in
258             if line_no = binding_line then
259                 (* This is the binding line - check for preceding doc *)
260                 let rec find_preceding = function
261                     | [] -> None_found
262                     | (doc_line, doc_content) :: prev ->
263                         match parse_doc_line doc_content with
264                         | Some _doc_kind -> Preceding (binding_line - doc_line)
265                         | None -> find_preceding prev
266                 in
267                 find_preceding prev_lines
268             else
269                 scan ((line_no, line) :: prev_lines) rest line_no
270     in
271     scan [] lines 0
272
273 (* agent note (|
274  * WHAT: Extract all binding docs from a .ml file by scanning
275  * for let bindings and their preceding doc comments. Handles
276  * both single-line and multi-line comments by tracking comment
277  * open/close state. For each binding found, records whether
278  * its comment has drifted status.

```



```

277 *
278 * WHY: The primary entry point for doc detection. Returns one
279 * binding_doc per binding with the closest preceding doc comment,
280 * used by Doc_coverage for coverage calculation and by Lint for
281 * code-doc checks.
282 * |) *)
283 let extract_file_docs path =
284   try
285     let lines = File_utils.read_lines path in
286
287     (* Find all let bindings and their docs.
288      * We track multi-line comment state: when we encounter a (*)
289      * that doesn't close on the same line, we note whether it
290      * started as a doc comment. When *) closes it, we treat the
291      * whole block as a doc line at the position of the closing paren-star.
292      *)
293     let rec scan line_no prev_docs acc in_ml_comment ml_comment_is_doc
294       ml_comment_first_line = function
295       | [] -> List.rev acc
296       | line :: rest ->
297         let line_no = line_no + 1 in
298         let trimmed = String.trim line in
299
300         if in_ml_comment then begin
301           (* Inside a multi-line comment: look for closing delimiter *)
302           let closes_here = String.length trimmed >= 2 &&
303             String.sub trimmed (String.length trimmed - 2) 2 = ")" in
304           if closes_here then
305             (* End of multi-line comment -- record it as a doc line
306              * if it started as a doc comment *)
307             let prev_docs' =
308               if ml_comment_is_doc then
309                 let marker = match ml_comment_first_line with
310                 | None -> "doc-comment"
311                 | Some first_line ->
312                   let score_opt = parse_literacy_score first_line in
313                   (match score_opt with
314                   | None -> "doc-comment"
315                   | Some s -> Printf.sprintf "borg-scored:%d%d%d"
316                     s.story s.explain s.teach s.edge)
317                 in
318                 (line_no, marker) :: prev_docs
319             else
320               (line_no, line) :: prev_docs
321             in
322             scan line_no prev_docs' acc false false None
323             (if String.starts_with ~prefix:"let" (String.trim
324               (String.sub trimmed 0 (String.length trimmed - 2))) then [line] else
325               rest)
326           else
327             scan line_no prev_docs acc true ml_comment_is_doc
328             ml_comment_first_line rest
329           end
330
331           else if String.starts_with ~prefix:"let" trimmed then
332             if String.starts_with ~prefix:"let open " trimmed ||
333               String.starts_with ~prefix:"let module " trimmed then
334               scan line_no ((line_no, line) :: prev_docs) acc false false
335             None rest
336           else
337             (* Found a binding -- check for preceding doc *)
338             let doc =
339               let rec find_doc = function
340               | [] -> None
341               | (doc_line, doc_content) :: prev ->
342                 (* "doc-comment" is our marker for a recognized
343                  * multi-line doc comment. "borg-scored:*" carries
344                  * a literacy score extracted from the first line. *)
345                 if doc_content = "doc-comment" then
346                   Some (Docstring "", Preceding (line_no - doc_line))
347                 else if String.starts_with ~prefix:"borg-scored:"
348                   doc_content then
349                     let digits = String.sub doc_content 12 4 in

```

```

342         let score = {
343             story = Char.code digits.[0] - Char.code '0';
344             explain = Char.code digits.[1] - Char.code '0';
345             teach = Char.code digits.[2] - Char.code '0';
346             edge = Char.code digits.[3] - Char.code '0';
347         } in
348         Some (Borg_note ("agent", "", Some score),
Preceding (line_no - doc_line))
349     else
350         match parse_doc_line doc_content with
351         | Some doc_kind -> Some (doc_kind, Preceding
(line_no - doc_line))
352         | None ->
353             let tc = String.trim doc_content in
354             if tc = "" then find_doc prev
355             else None
356         in
357         find_doc prev_docs
358     in
359     let is_drifted = match doc with
360     | None -> false
361     | Some (kind, _) -> doc_kind_is_drifted kind
362     in
363     scan line_no [] ({ binding_line = line_no; doc; is_drifted }
:: acc) false false None rest
364     else
365         (* Check if this line starts a multi-line comment that
366         doesn't close on the same line *)
367         let starts_ml_doc =
368             String.starts_with ~prefix: "(" trimmed &&
369             not (String.length trimmed >= 4 &&
370                 String.sub trimmed (String.length trimmed - 2) 2 = "*)")
371         &&
372         (is_docstring_start trimmed || is_borg_note trimmed ||
373         String.starts_with ~prefix: "(* agent note" trimmed ||
374         String.starts_with ~prefix: "(* roerick note" trimmed ||
375         String.starts_with ~prefix: "(* (fn" trimmed)
376     in
377     let starts_ml_nondoc =
378         String.starts_with ~prefix: "(" trimmed &&
379         not (String.length trimmed >= 4 &&
380             String.sub trimmed (String.length trimmed - 2) 2 = "*)")
381     &&
382     not starts_ml_doc
383     in
384     if starts_ml_doc then
385         let first_line = Some trimmed in
386         scan line_no prev_docs acc true true first_line rest
387     else if starts_ml_nondoc then
388         scan line_no prev_docs acc true false None rest
389     else
390         scan line_no ((line_no, line) :: prev_docs) acc false false
None rest
391     in
392     scan 0 [] [] false false None lines
393 with e ->
394     Printf.eprintf "Error extracting docs from %s: %s\n" path
(Printexc.to_string e);
395     []
396
397 (* agent note (|
398 *   WHAT: Check if a binding has documentation (not exempt, not missing).
399 *   Drifted comments still count as documented -- the gap is acknowledged,
400 *   which is better than having no comment at all.
401 *
402 *   WHY: Coverage calculation needs to distinguish documented from
403 *   undocumented. Drifted comments are still "there" even if stale.
404 * |) *)
405 let binding_has_doc (bd : binding_doc) =
406     match bd.doc with
407     | None -> false
408     | Some (Exempt_marker, _) -> false
409     | Some _ -> true

```

```
408
409 (* agent note (|
410 *   WHAT: Check if a binding is exempt from documentation requirement
411 *   (has the (* exempt doc *) marker).
412 *
413 *   WHY: Exempted bindings are excluded from coverage enforcement.
414 *   They don't count against the author -- they're explicitly opted out.
415 * |) *)
416 let binding_is_exempt (bd : binding_doc) =
417   match bd.doc with
418   | Some (Exempt_marker, _) -> true
419   | _ -> false
420
421 (* agent note (|
422 *   WHAT: Check if a binding has a drifted comment -- one marked
423 *   with (status drifted), indicating it's known to be stale.
424 *
425 *   WHY: Drifted comments get special treatment in reports: they're
426 *   documented (not missing) but flagged for update. This is the
427 *   middle ground between accurate and absent documentation.
428 * |) *)
429 let binding_is_drifted (bd : binding_doc) = bd.is_drifted
```

43 lib/doc/doc_extract.ml

```

1 (* Extract all let bindings from .ml files for documentation coverage
2  *)
3 (* agent note (|
4  *   WHAT: Gathers all top-level value bindings from OCaml source files,
5  *   distinguishing between public exports and internal helpers. Filters
6  *   out test bindings, ignored bindings, and local opens.
7  *)
8 (*   WHY: Downstream tools (doc_coverage, lint --code-doc) need to know
9  *   which bindings exist and whether they're exported, so they can
10  *   check documentation coverage and enforce the literate-programming rule.
11  * |) *)
12
13
14
15 (** Information about a binding extracted for doc coverage *)
16 type binding_info = {
17   name : string;
18   line : int;
19   is_exported : bool; (* Would this be visible in .mli? *)
20   is_test : bool;
21   is_internal : bool; (* Starts with _ or is let%test *)
22 }
23
24 (* agent note (|
25  *   WHAT: Parse a let-binding line to extract just the binding name.
26  *   Handles "let rec name", "let name", and various patterns after
27  *   the name (parameters, type annotations, etc.).
28  *)
29 (*   WHY: We need the binding name for coverage reporting -- the name
30  *   identifies which function is missing documentation.
31  * |) *)
32 let extract_binding_name line =
33   let trimmed = String.trim line in
34   let prefix =
35     if String.starts_with ~prefix:"let rec " trimmed then
36       "let rec "
37     else if String.starts_with ~prefix:"let " trimmed then
38       "let "
39     else
40       ""
41   in
42   if prefix = "" then
43     None
44   else
45     let rest = String.sub trimmed (String.length prefix) (String.length
46       trimmed - String.length prefix) in
47     let rest = String.trim rest in
48     (* Find first space, paren, or colon -- that's where the name ends *)
49     let name_end = try
50       let space = try String.index rest ' ' with Not_found -> max_int in
51       let paren = try String.index rest '(' with Not_found -> max_int in
52       let colon = try String.index rest ':' with Not_found -> max_int in
53       let equals = try String.index rest '=' with Not_found -> max_int in
54       min space (min paren (min colon equals))
55     with _ -> String.length rest in
56     let name = (* exempt: String.sub *) String.sub rest 0 name_end |>
57       String.trim in
58     if name = "" then None else Some name
59
60 (* agent note (|
61  *   WHAT: Check if a binding name is internal (starts with underscore).
62  *
63  *   WHY: Internal bindings are excluded from documentation coverage
64  *   enforcement -- they're implementation details, not public API.
65  * |) *)
66 let is_internal_name name =
67   name = "_" || String.starts_with ~prefix:"_" name
68
69 (* agent note (|
70  *   WHAT: Check if a line is a test binding (starts with let%).
71  *)

```

```

70  *   WHY: Test bindings are excluded from documentation coverage
71  *   enforcement -- they have different documentation needs than
72  *   production code.
73  * |) *)
74  let is_test_binding line =
75    let trimmed = String.trim line in
76    String.starts_with ~prefix:"let%" trimmed
77
78  (* agent note (|
79  *   WHAT: Check if a line is a local open or module binding.
80  *
81  *   WHY: Local opens (let open Foo in) and module bindings
82  *   (let module M = ...) are not real function definitions and
83  *   must be excluded from binding extraction.
84  * |) *)
85  let is_local_open line =
86    let trimmed = String.trim line in
87    String.starts_with ~prefix:"let open " trimmed ||
88    String.starts_with ~prefix:"let module " trimmed
89
90  (* agent note (|
91  *   WHAT: Determine if a binding would be exported in an .mli file
92  *   based on naming convention: lowercase start and not internal.
93  *
94  *   WHY: Only exported bindings need documentation per the
95  *   literate-programming rule. This heuristic approximates
96  *   .mli visibility without actually parsing the interface file.
97  * |) *)
98  let would_be_exported name =
99    not (is_internal_name name) && name.[0] >= 'a' && name.[0] <= 'z'
100
101  (* agent note (|
102  *   WHAT: Extract all top-level let bindings from a .ml file,
103  *   returning their names, line numbers, and classification
104  *   (exported, test, internal). Skips lines inside strings
105  *   and comments to avoid false positives.
106  *
107  *   WHY: This is the primary extraction function used by
108  *   Doc_coverage.calculate_file_coverage to discover which
109  *   bindings exist and need documentation checks.
110  * |) *)
111  let extract_bindings path =
112    try
113      (* exempt: open_in input_line *)
114      let channel = open_in path in
115      let bindings = ref [] in
116      let line_num = ref 0 in
117      let in_string = ref false in
118      let in_comment = ref false in
119
120      (try
121        while true do
122          let line = input_line channel in
123          incr line_num;
124          let trimmed = String.trim line in
125
126          (* Track whether we're inside a string or comment to
127           avoid extracting bindings from string literals or
128           comment blocks *)
129          let rec check_string_comments i =
130            if i >= String.length trimmed then ()
131            else match trimmed.[i] with
132            | '"' -> in_string := not !in_string; check_string_comments
133              (i+1)
134            | '(' when i+1 < String.length trimmed && trimmed.[i+1] = '*'
135              && not !in_string ->
136                in_comment := true; check_string_comments (i+2)
137            | '*' when i+1 < String.length trimmed && trimmed.[i+1] = ')'
138              && !in_comment ->
139                in_comment := false; check_string_comments (i+2)
140            | _ -> check_string_comments (i+1)
141          in
142          check_string_comments 0;

```

```

140
141     if not !in_string && not !in_comment && String.length line > 0 &&
142       line.[0] <> ' ' && line.[0] <> '\t' then
143       if String.starts_with ~prefix:"let" trimmed then
144         if is_local_open trimmed then
145           ()
146         else match extract_binding_name trimmed with
147           | None -> ()
148           | Some name ->
149             let is_test = is_test_binding trimmed in
150             let is_internal = is_internal_name name || is_test in
151             let is_exp = not is_internal && would_be_exported name in
152             bindings := {
153               name;
154               line = !line_num;
155               is_exported = is_exp;
156               is_test;
157               is_internal;
158             } :: !bindings
159       done
160     with End_of_file -> ());
161
162     close_in channel;
163     List.rev !bindings
164 with e ->
165   Printf.eprintf "Error extracting from %s: %s\n" path
166   (Printexc.to_string e);
167   []
168
169 (* agent note (|
170 *   WHAT: Recursively find all .ml files under a directory,
171 *   skipping _build/ and .git/ directories.
172 *
173 *   WHY: Doc_coverage needs to scan the entire project for
174 *   .ml files to calculate project-wide documentation coverage.
175 * |) *)
176 let find_ml_files dir =
177   let rec find acc path =
178     if Sys.is_directory path then
179       if Filename.basename path = "_build" || Filename.basename path =
180         ".git" then
181         acc
182       else
183         Sys.readdir path
184         |> Array.to_list
185         |> List.map (Filename.concat path)
186         |> List.fold_left find acc
187     else if Filename.check_suffix path ".ml" then
188       path :: acc
189     else
190       acc
191   in
192   find [] dir
193
194 (* agent note (|
195 *   WHAT: Extract bindings from all .ml files in a project,
196 *   returning (path, binding_info) pairs for every binding.
197 *
198 *   WHY: Used by higher-level analysis that needs to correlate
199 *   bindings with their source files for reporting.
200 * |) *)
201 let extract_project_bindings dir =
202   let ml_files = find_ml_files dir in
203   List.concat_map (fun path ->
204     let bindings = extract_bindings path in
205     List.map (fun b -> (path, b)) bindings
206   ) ml_files
207
208 (* agent note (|
209 *   WHAT: Count statistics for a list of bindings: total,
210 *   exported, tests, and internal counts.
211 *
212 *   WHY: Provides quick summary numbers for coverage reports

```

```
210  *   without needing to calculate full documentation coverage.
211  * |) *)
212  let count_stats bindings =
213    let total = List.length bindings in
214    let exported = List.filter (fun b -> b.is_exported) bindings |>
      List.length in
215    let tests = List.filter (fun b -> b.is_test) bindings |> List.length in
216    let internal = List.filter (fun b -> b.is_internal) bindings |>
      List.length in
217    (total, exported, tests, internal)
```

44 lib/format/fmt.ml

```

1 open Borge_lang
2
3 type fmt_result =
4   | Formatted of string
5   | CheckClean
6   | CheckDirty of string
7
8 (* agent note (|
9  *   WHAT: Format a borge file from a string, returning the
10  *   pretty-printed result.
11  *
12  *   WHY: Core formatting entry point - parses string input and
13  *   re-serializes with proper formatting.
14  * |) *)
15 let format_string input =
16   let file = Parse.parse input in
17   Print.print_file file
18
19 (* agent note (|
20  *   WHAT: Format a borge file at the given path, returning the
21  *   pretty-printed result wrapped in Formatted.
22  *
23  *   WHY: CLI entry point for formatting files.
24  * |) *)
25 let format_file path =
26   let input = File_utils.read_file path in
27   Formatted (format_string input)
28
29 (* agent note (|
30  *   WHAT: Remove trailing newline characters from a string.
31  *
32  *   WHY: Used for comparing formatted vs original content
33  *   to check if formatting changed anything.
34  * |) *)
35 let strip_trailing_newlines s =
36   let len = String.length s in
37   let i = ref (len - 1) in
38   while !i >= 0 && s.[!i] = '\n' do decr i done;
39   (* exempt: String.sub *) String.sub s 0 (!i + 1)
40
41 (* agent note (|
42  *   WHAT: Check if a file would be changed by formatting.
43  *   Returns CheckClean if identical, CheckDirty with formatted
44  *   content if different.
45  *
46  *   WHY: Used by borge fmt --check to verify files are formatted.
47  * |) *)
48 let check_file path =
49   let input = File_utils.read_file path in
50   let formatted = format_string input in
51   if strip_trailing_newlines input = strip_trailing_newlines formatted then
52     CheckClean
53   else CheckDirty formatted

```


45 lib/format/nodes.ml

```

1 open Borge_lang.Ast
2
3 type node_info = {
4   pos : pos;
5   kind : string;      (** "Atom", "String", "List" *)
6   keyword : string option; (** For List: first Atom's name, if any *)
7   child_count : int;   (** For List: number of children *)
8   value : string option; (** For Atom: the symbol; for String: truncated
9                           content *)
10 }
11
12 (* agent note (|
13  * WHAT: Recursively collect all AST nodes from a sexp tree,
14  * returning info for each node with position, kind, keyword, etc.
15  * Uses increasing indent for tracking depth.
16  *
17  * WHY: The nodes command needs to enumerate every node in a file
18  * for structural analysis and diffing.
19  * |) *)
20 let rec collect_nodes indent sexp =
21   let info = node_info_of_sexp sexp indent in
22   let children = child_nodes sexp in
23   info :: List.concat_map (collect_nodes (indent + 1)) children
24
25 and node_info_of_sexp sexp _indent =
26   match sexp with
27   | Atom (p, sym) ->
28     { pos = p; kind = "Atom"; keyword = None; child_count = 0; value =
29       Some sym }
30   | String (p, sv) ->
31     let val_str = string_value_summary sv in
32     { pos = p; kind = "String"; keyword = None; child_count = 0; value =
33       Some val_str }
34   | List (p, children) ->
35     let keyword = list_keyword children in
36     { pos = p; kind = "List"; keyword; child_count = List.length
37       children; value = None }
38
39 and child_nodes = function
40   | Atom _ | String _ -> []
41   | List (_, children) -> children
42
43 and list_keyword = function
44   | Atom (_, k) :: _ -> Some k
45   | _ -> None
46
47 and string_value_summary = function
48   | Quoted q ->
49     let s = q.q_content in
50     if String.length s > 30 then String.sub s 0 27 ^ "..." else s
51   | Verbatim v ->
52     let s = v.v_content in
53     if String.length s > 30 then String.sub s 0 27 ^ "...|" else s ^ "|"
54
55 type result = {
56   file_path : string;
57   nodes : node_info list;
58 }
59
60 (* agent note (|
61  * WHAT: Extract structural information from a file for the
62  * nodes report - parses and collects all nodes.
63  *
64  * WHY: Main entry point for borge nodes command.
65  * |) *)
66 let run path =
67   let input = File_utils.read_file path in
68   let file = Borge_lang.Parse.parse_file input in
69   let nodes = List.concat_map (fun { node; _ } ->
70     collect_nodes 0 node
71   ) file.top_level in

```

```

68   { file_path = path; nodes }
69
70 (* agent note (|
71  *   WHAT: Format a single node as a readable line with position,
72  *   kind, and relevant details.
73  *
74  *   WHY: Output formatting for the nodes report.
75  * |) *)
76 let format_node indent info =
77   let pos_str = Printf.sprintf "%d:%d" info.pos.line info.pos.col in
78   let indent_str = String.make (indent * 2) ' ' in
79   match info.kind, info.keyword, info.value with
80   | "Atom", _, Some sym ->
81     Printf.sprintf "%s%-5s Atom %s" indent_str pos_str sym
82   | "Atom", _, None ->
83     Printf.sprintf "%s%-5s Atom" indent_str pos_str
84   | "String", _, Some v ->
85     Printf.sprintf "%s%-5s String %S" indent_str pos_str v
86   | "String", _, None ->
87     Printf.sprintf "%s%-5s String" indent_str pos_str
88   | "List", Some kw, _ ->
89     Printf.sprintf "%s%-5s List \"%s\" [%d children]" indent_str pos_str
90     kw info.child_count
91   | "List", None, _ ->
92     Printf.sprintf "%s%-5s List [%d children]" indent_str pos_str
93     info.child_count
94   | _ ->
95     Printf.sprintf "%s%-5s %s" indent_str pos_str info.kind
96
97 (* agent note (|
98  *   WHAT: Format the complete nodes result as multi-line output.
99  *
100  *   WHY: CLI output for borge nodes.
101  * |) *)
102 let format_result result =
103   let buf = Buffer.create 4096 in
104   List.iter (fun info ->
105     Buffer.add_string buf (format_node 0 info);
106     Buffer.add_char buf '\n'
107   ) result.nodes;
108   Buffer.contents buf

```

46 lib/future/depends.ml

```
1 (* Parse dependency chains - STUB *)
2
3 type dep_graph = { nodes : string list; edges : (string * string) list }
4
5 let empty_graph = { nodes = []; edges = [] }
6 let build_graph _dir = empty_graph
7 let topological_sort _graph = []
```

47 lib/future/format.ml

```
1 (* Format roadmap output - STUB *)
2
3 open Future
4
5 let format ?_json ?_summary ?_section ?_show_deps _roadmap =
6   "=== PLANNED ===\n\n=== IN PROGRESS ===\n\n=== READY FOR REVIEW ===\n"
7
8 let format_summary roadmap =
9   Printf.sprintf "Planned: %d | In Progress: %d | Ready: %d"
10    (List.length roadmap.planned)
11    (List.length roadmap.in_progress)
12    (List.length roadmap.ready_for_review)
```

48 lib/future/future.ml

```
1 (* Scan specs for planned/in-progress work (borge future command) - STUB *)
2
3 type future_item = {
4   file : string;
5   line : int;
6   section_path : string;
7   status : Spec.status;
8   description : string;
9   depends_on : string list;
10 }
11
12 type roadmap = {
13   planned : future_item list;
14   in_progress : future_item list;
15   ready_for_review : future_item list;
16 }
17
18 let get_roadmap _dir = {
19   planned = [];
20   in_progress = [];
21   ready_for_review = [];
22 }
```

49 lib/lock/lock.ml

```

1 open Borge_lang
2
3 (** Lock mechanism for agent sessions.
4
5     Provides snapshot isolation for borge agent workflows.
6     On lock creation: copies all .borg files to .borg.lock/original/
7     On commit: integrity check, git commit, remove lock
8     On abort: restores .borg files from original/
9
10    The lock is purely metadata + snapshots. Agents edit repo files
11    directly. The lock provides integrity checking, concurrency
12    prevention, and audit trail. *)
13
14 let lock_dir = ".borg.lock"
15
16 (* agent note (|
17  *   WHAT: Get the path to the original files snapshot directory.
18  *
19  *   WHY: Lock mechanism copies .borg files to original/ for integrity
20  *   checking and restoration on abort.
21  * |) *)
22 let original_dir () = Printf.sprintf "%s/original" lock_dir
23
24 (* agent note (|
25  *   WHAT: Get the path to the agent prompt file.
26  *
27  *   WHY: The prompt is written to a file so the pi agent can read it.
28  * |) *)
29 let prompt_file () = Printf.sprintf "%s/prompt.md" lock_dir
30
31 (* agent note (|
32  *   WHAT: Get the path to the session journal file.
33  *
34  *   WHY: Journal tracks changes made during the agent session.
35  * |) *)
36 let journal_file () = Printf.sprintf "%s/journal" lock_dir
37
38 (* agent note (|
39  *   WHAT: Get the path to the role file that stores the agent role.
40  *
41  *   WHY: Role is persisted so we know what mode the agent was in.
42  * |) *)
43 let role_file () = Printf.sprintf "%s/role" lock_dir
44
45 (* agent note (|
46  *   WHAT: Get the path to the state file.
47  *
48  *   WHY: State tracks the lock lifecycle (created, exploring, completed,
49  *   etc).
50  * |) *)
51 let state_file () = Printf.sprintf "%s/state" lock_dir
52
53 (* agent note (|
54  *   WHAT: Check if a lock is currently active.
55  *
56  *   WHY: Commands that need isolation check for existing lock first.
57  * |) *)
58 let is_locked () =
59   Sys.file_exists lock_dir
60
61 (* agent note (|
62  *   WHAT: Create the lock directory structure and snapshot all
63  *   .borg files to the original/ subdirectory.
64  *
65  *   WHY: This is the first step of any agent session - establishes
66  *   the baseline for integrity checking.
67  * |) *)
68 let create () =
69   if is_locked () then
70     Error "Lock already exists -- borge abort to clear, or commit to finish"
71   else begin

```

```

71   (try Unix.mkdir lock_dir 0o755 with _ -> ());
72   (try Unix.mkdir (original_dir ()) 0o755 with _ -> ());
73   let borg_files = File_utils.find_borg_files "." in
74   List.iter (fun src ->
75     let dst = Printf.sprintf "%s/%s" (original_dir ()) (Filename.basename
76       src) in
77     let content = File_utils.read_file src in
78     let oc = open_out dst in
79     output_string oc content;
80     close_out oc
81   ) borg_files;
82   let oc = open_out (state_file ()) in
83   output_string oc "running\n";
84   close_out oc;
85   Ok ()
86 end
87
88 (* Mark state as completed or failed *)
89 let set_state st =
90   let oc = open_out (state_file ()) in
91   output_string oc (st ^ "\n");
92   close_out oc
93
94 (* Get current state *)
95 let get_state () =
96   if not (is_locked ()) then "no-lock"
97   else begin
98     match File_utils.read_lines (state_file ()) with
99     | st :: _ -> st
100    | [] -> "unknown"
101  end
102
103 (* Integrity check: compare original/ with working tree for deleted
104    or inserted human-authored annotated comments.
105    A human-authored comment: (* <author> <type> ... *) where
106    author != "agent" and author != "bot" *)
107 let check_integrity () : (string, string) result =
108   if not (is_locked ()) then
109     Error "No lock directory"
110   else begin
111     let original_files =
112       let d = original_dir () in
113       let files = Sys.readdir d in
114       Array.to_list files |> List.filter (fun f -> Filename.check_suffix f
115         ".borg")
116     |> List.map (fun f -> Printf.sprintf "%s/%s" d f)
117   in
118   let issues = ref [] in
119   List.iter (fun orig_path ->
120     let base = Filename.basename orig_path in
121     let work_path = base in
122     if Sys.file_exists work_path then begin
123       try
124         let orig_content = File_utils.read_file orig_path in
125         let work_content = File_utils.read_file work_path in
126         let orig_lines = String.split_on_char '\n' orig_content in
127         let work_lines = String.split_on_char '\n' work_content in
128         List.iter (fun line ->
129           let trimmed = String.trim line in
130           if String.length trimmed > 4 then begin
131             let prefix = (* exempt: String.sub *) String.sub trimmed 0 3
132             in
133             if prefix = "(* " then begin
134               let rest = String.sub trimmed 3 (String.length trimmed - 3)
135               in
136               let space_idx = try String.index rest ' ' with Not_found ->
137                 -1 in
138               if space_idx > 0 then begin
139                 let author = (* exempt: String.sub *) String.sub rest 0
140                   space_idx in
141                 if author <> "agent" && author <> "bot" then begin
142                   if not (List.mem line work_lines) then

```

```

137         issues := Printf.sprintf "%s: deleted comment by %s"
138         base author :: !issues
139         end
140         end
141         end
142         ) orig_lines;
143         (* Also detect inserted human-authored comments in working tree *)
144         List.iter (fun line ->
145             let trimmed = String.trim line in
146             if String.length trimmed > 4 then begin
147                 let prefix = (* exempt: String.sub *) String.sub trimmed 0 3
148
149                 in
150                     if prefix = "( " then begin
151                         let rest = String.sub trimmed 3 (String.length trimmed - 3)
152
153                         in
154                             let space_idx = try String.index rest ' ' with Not_found ->
155                                 -1 in
156                             if space_idx > 0 then begin
157                                 let author = (* exempt: String.sub *) String.sub rest 0
158                                     space_idx in
159                                 if author <> "agent" && author <> "bot" then begin
160                                     if not (List.mem line orig_lines) then
161                                         issues := Printf.sprintf "%s: inserted comment by %s"
162                                         base author :: !issues
163                                     end
164                                 end
165                             end
166                         ) work_lines
167                     with _ -> ()
168                 end
169             ) original_files;
170             if !issues = [] then Ok "pass"
171             else Error (String.concat "\n" !issues)
172         end
173
174 (* Restore .borg files from original/ -- abort *)
175 let restore () : unit =
176     let orig_dir = original_dir () in
177     let files = Sys.readdir orig_dir in
178     Array.iter (fun f ->
179         if Filename.check_suffix f ".borg" then begin
180             let src = Printf.sprintf "%s/%s" orig_dir f in
181             let dst = f in
182             let content = File_utils.read_file src in
183             let oc = open_out dst in
184             output_string oc content;
185             close_out oc
186         end
187     ) files
188
189 (* Remove lock directory *)
190 let remove () =
191     let rec rm_rf path =
192         if Sys.is_directory path then begin
193             let entries = Sys.readdir path in
194             Array.iter (fun e -> rm_rf (Filename.concat path e)) entries;
195             Unix.rmdir path
196         end else
197             Sys.remove path
198     in
199     if is_locked () then rm_rf lock_dir
200
201 (* Teardown: commit (if pass) or abort (if fail) *)
202 let teardown ~(commit : bool) : (string, string) result =
203     if not (is_locked ()) then
204         Error "No lock directory"
205     else if not commit then begin
206         restore ();
207         remove ();
208         Ok "aborted"
209     end else begin

```



```
204 match check_integrity () with
205 | Error msg ->
206   Error (Printf.sprintf "Integrity check failed:\n%s" msg)
207 | Ok _ ->
208   let borg_files = File_utils.find_borg_files "." in
209   let changed = ref [] in
210   List.iter (fun path ->
211     let orig = Printf.sprintf "%s/%s" (original_dir ())
212     (Filename.basename path) in
213     if Sys.file_exists orig then begin
214       let current = File_utils.read_file path in
215       let old = File_utils.read_file orig in
216       if current <> old then begin
217         try
218           let file = Parse.parse_file current in
219           let formatted = Print.print_file file in
220           let oc = open_out path in
221           output_string oc formatted;
222           close_out oc;
223           changed := path :: !changed
224         with _ -> ()
225       end
226     end
227   ) borg_files;
228   Ok (Printf.sprintf "Committed. Changed: %s" (String.concat ", "
229     !changed))
230 end
```

50 lib/lock/lock_pi_launch.ml

```
1 (* Launch pi interactively with a structured prompt.  
2    Returns the exit status of pi. *)  
3 let launch (prompt_path : string) : int =  
4   let cmd = Printf.sprintf "pi %s" prompt_path in  
5   (* Direct invocation -- pi is interactive, so backgrounding loses status.  
6      Future: detect tmux and open new window, detect ghostty and open new  
7      tab. *)  
   Sys.command cmd
```

51 lib/lock/lock_prompt.ml

```

1  (* Prompt builder for agent sessions.
2
3     Assembles structured prompts from the spec, convention rules,
4     module surfaces, and role declarations.
5
6     Does NOT depend on Borge_lib internals to avoid cyclic deps.
7     All data is passed in via function arguments. *)
8
9  type role =
10 | Implementer
11 | Spec_writer
12 | Observer
13 | Fixer
14
15 let role_name = function
16 | Implementer -> "implementer"
17 | Spec_writer -> "spec-writer"
18 | Observer -> "observer"
19 | Fixer -> "fixer"
20
21 let role_description = function
22 | Implementer ->
23   "You are an implementer. Your job is to make the code match the spec.
24   \\n      You may write code files and tests. You may NOT modify .borg
25   files \\n      directly. You may NOT delete human-authored comments."
26 | Spec_writer ->
27   "You are a spec-writer. Your job is to edit the spec to describe \\n
28   what the code should be. You may modify .borg files directly. \\n
29   You may NOT write code files."
30 | Observer ->
31   "You are an observer. Your job is to check whether the code matches \\n
32   the spec. You may create findings and add comments. You may NOT
33   write \\n      code files or modify .borg files."
34 | Fixer ->
35   "You are a fixer. Your job is to repair drift between code and spec.
36   \\n      You may write code repairs and tests. You may NOT modify .borg
37   files."
38
39 let make_prompt (role : role) ~(convention : string)
40   ~(spec_files : (string * string) list)
41   ~(module_surfaces : (string * string list) list) : string =
42   let buf = Buffer.create 4096 in
43
44   (* Role declaration *)
45   Buffer.add_string buf (role_description role);
46   Buffer.add_string buf "\\n\\n";
47
48   (* Project convention *)
49   Buffer.add_string buf (Printf.sprintf "Convention: %s\\n\\n" convention);
50
51   (* Spec files as context *)
52   Buffer.add_string buf "## SPEC FILES ##\\n\\n";
53   List.iter (fun (path, content) ->
54     Buffer.add_string buf (Printf.sprintf "### %s ###\\n" path);
55     Buffer.add_string buf content;
56     Buffer.add_string buf "\\n\\n"
57   ) spec_files;
58
59   (* Module surfaces for context *)
60   if module_surfaces <> [] then begin
61     Buffer.add_string buf "## EXISTING MODULES ##\\n\\n";
62     List.iter (fun (mod_name, exports) ->
63       if exports <> [] then
64         Buffer.add_string buf (Printf.sprintf "%s exports: %s\\n"
65           mod_name (String.concat " ", exports))
66     ) module_surfaces;
67     Buffer.add_string buf "\\n"
68   end;
69
70   Buffer.contents buf

```

52 lib/lock/prompt_minimal.ml

```

1  (* Lightweight prompt builder with minimal context
2
3  Instead of loading full .borg files, we extract:
4  - Planned sections (names + docs) -- the backlog
5  - Implemented section names -- guardrails
6  - Convention name only (not full rules)
7  - Module names only (not exports)
8
9  This keeps the prompt under ~2K tokens instead of 10K+ *)
10
11 type role =
12   | Implementer
13   | Spec_writer
14   | Observer
15   | Fixer
16
17 let role_name = function
18   | Implementer -> "implementer"
19   | Spec_writer -> "spec-writer"
20   | Observer -> "observer"
21   | Fixer -> "fixer"
22
23 let role_description = function
24   | Implementer ->
25     "You are an implementer. Make code match planned sections. " ^
26     "You may write code and tests. You may NOT modify .borg files."
27   | Spec_writer ->
28     "You are a spec-writer. Edit .borg files directly to describe intent. " ^
29     "You may NOT write code files."
30   | Observer ->
31     "You are an observer. Check that code matches spec. Read-only."
32   | Fixer ->
33     "You are a fixer. Repair drift by writing code. You may NOT modify
34     .borg."
35
36 (* Minimal section info: name, status, and short doc *)
37 type section_summary = {
38   name : string;
39   status : string;
40   doc : string;
41 }
42
43 (* Extract planned and implemented sections from a parsed file *)
44 let summarize_borg_file content : section_summary list =
45   (* Simple extraction: look for (section ... (status X) ... (doc "...")) *)
46   let lines = String.split_on_char '\n' content in
47   let sections = ref [] in
48   let current_section = ref None in
49   let current_status = ref "" in
50   let current_doc = ref "" in
51
52   List.iter (fun line ->
53     let trimmed = String.trim line in
54
55     (* Match (section name *)
56     if String.length trimmed > 10 &&
57       String.sub trimmed 0 9 = "(section " then
58       begin
59         let rest = String.sub trimmed 9 (String.length trimmed - 9) in
60         (* Extract section name until space or paren *)
61         let name_end = try String.index rest ' ' with Not_found ->
62           try String.index rest ')' with Not_found ->
63           String.length rest in
64         let name = (* exempt: String.sub *) String.sub rest 0 name_end in
65         current_section := Some name;
66         current_status := "";
67         current_doc := "";
68       end;
69
70     (* Match (status X) *)

```

```

69   if String.length trimmed > 9 &&
70     String.sub trimmed 0 8 = "(status " then
71     begin
72       let rest = String.sub trimmed 8 (String.length trimmed - 8) in
73       let status_end = try String.index rest ')' with Not_found ->
String.length rest in
74       let status = (* exempt: String.sub *) String.sub rest 0 status_end
in
75         current_status := status
76       end;
77
78       (* Match (doc "...") *)
79       if String.length trimmed > 7 &&
80         String.sub trimmed 0 6 = "(doc \"" then
81         begin
82           let rest = String.sub trimmed 6 (String.length trimmed - 6) in
83           (* Extract until closing quote *)
84           try
85             let doc_end = String.index rest '"' in
86             let doc = (* exempt: String.sub *) String.sub rest 0 doc_end in
87             current_doc := doc
88             with Not_found -> ()
89           end;
90
91           (* On section close, save if has name *)
92           if !current_section <> None && (String.length trimmed = 0 ||
trimmed.[0] = ')') then
93             begin
94               let name = Option.get !current_section in
95               if !current_status = "planned" || !current_status = "implemented"
then
96                 sections := { name; status = !current_status; doc = !current_doc
} :: !sections;
97                 current_section := None
98               end
99             ) lines;
100
101             List.rev !sections
102
103 let make_prompt_minimal (role : role) ~(planned : section_summary list)
104   ~(implemented : section_summary list) ~(modules : string list) : string
105   =
106   let buf = Buffer.create 2048 in
107   Buffer.add_string buf (role_description role);
108   Buffer.add_string buf "\n\n";
109
110   (* Backlog: planned sections *)
111   if planned <> [] then begin
112     Buffer.add_string buf "## BACKLOG (planned) ##\n";
113     List.iter (fun s ->
114       Buffer.add_string buf (Printf.sprintf "- %s\n" s.name s.doc)
115     ) planned;
116     Buffer.add_string buf "\n"
117   end;
118
119   (* Guardrails: implemented sections *)
120   if implemented <> [] then begin
121     Buffer.add_string buf "## GUARDRAILS (implemented) ##\n";
122     List.iter (fun s ->
123       Buffer.add_string buf (Printf.sprintf "- %s\n" s.name)
124     ) implemented;
125     Buffer.add_string buf "\n"
126   end;
127
128   (* Existing modules for context *)
129   if modules <> [] then begin
130     Buffer.add_string buf "## EXISTING MODULES ##\n";
131     List.iter (fun m ->
132       Buffer.add_string buf (Printf.sprintf "- %s\n" m)
133     ) modules;
134     Buffer.add_string buf "\n"
135   end;

```

```
136
137   Buffer.contents buf
138
139   (* DEPRECATED: full context version preserved for reference *)
140   let make_prompt = make_prompt_minimal
```

53 lib/merge/agent_submit.ml

```

1 (* Auto-submit agent runs to merge queue on completion
2  *
3  * roerick note (|
4  *   Hooks into agent completion to automatically submit worktrees
5  *   to the merge queue. Called by borge make/plan on clean exit.
6  *
7  *   Submission requires:
8  *   - Worktree is in clean state (no uncommitted changes)
9  *   - borge balance/check pass
10  *   - Confidence score from agent self-assessment
11  * |) *)
12
13 open Queue_types
14
15 (** Submission config *)
16 type submit_config = {
17   auto_submit : bool;          (* Whether to auto-submit on completion *)
18   min_confidence : float;      (* Minimum confidence to auto-submit *)
19   require_clean : bool;        (* Require clean worktree *)
20   require_checks : bool;       (* Require balance/check pass *)
21 }
22
23 (** Default config *)
24 let default_config = {
25   auto_submit = true;
26   min_confidence = 0.7;
27   require_clean = true;
28   require_checks = true;
29 }
30
31 (** Result of submission attempt *)
32 type submit_result =
33   | Submitted of item_id        (* Successfully submitted with ID *)
34   | Skipped of string           (* Skipped with reason *)
35   | Error of string             (* Error occurred *)
36
37 (** Run borge check on worktree *)
38 let run_checks worktree_path =
39   let cmd = Printf.sprintf "cd %s && borge balance >/dev/null 2>&1 && borge
40     check >/dev/null 2>&1" worktree_path in
41   Sys.command cmd = 0
42
43 (** Check if worktree is clean *)
44 let is_worktree_clean worktree_path =
45   let cmd = Printf.sprintf "cd %s && git status --porcelain | wc -l"
46     worktree_path in
47   let ic = Unix.open_process_in cmd in
48   try
49     (* exempt: input_line *) let line = input_line ic |> String.trim in
50     ignore (Unix.close_process_in ic);
51     line = "0"
52   with _ ->
53     ignore (Unix.close_process_in ic);
54     false
55
56 (** Get commit message for summary *)
57 let get_commit_summary worktree_path =
58   let cmd = Printf.sprintf "cd %s && git log -1 --pretty=format:'%s'"
59     worktree_path in
60   let ic = Unix.open_process_in cmd in
61   try
62     (* exempt: input_line *) let msg = input_line ic |> String.trim in
63     ignore (Unix.close_process_in ic);
64     if String.length msg > 80 then
65       (* exempt: String.sub *) String.sub msg 0 77 ^ "..."
66     else
67       msg
68   with _ ->
69     ignore (Unix.close_process_in ic);
70     "Agent run"

```

```

69 (** Estimate confidence based on agent self-assessment
70
71 Reads optional .borge/CONFIDENCE file that agent may have written *)
72 let read_confidence worktree_path =
73   let confidence_file = Filename.concat worktree_path ".borge/CONFIDENCE" in
74   if Sys.file_exists confidence_file then
75     let content_opt =
76       try Some (File_utils.read_file confidence_file |> String.trim)
77       with _ -> None
78     in
79     match content_opt with
80     | None -> 0.5
81     | Some content ->
82       match float_of_string_opt content with
83       | Some v -> v
84       | None -> 0.5
85   else
86     0.5 (* Default if agent didn't write confidence *)
87
88 (** Get affected files from git diff *)
89 let get_affected_files worktree_path =
90   let cmd = Printf.sprintf "cd %s && git diff --name-only HEAD~1 HEAD
91     2>/dev/null" worktree_path in
92   let ic = Unix.open_process_in cmd in
93   let files = ref [] in
94   (try
95     while true do
96       (* exempt: input_line *) let line = input_line ic in
97       if line <> "" then files := line :: !files
98     done
99   with End_of_file -> ());
100   ignore (Unix.close_process_in ic);
101   List.rev !files
102
103 (** Submit worktree to merge queue
104
105 Returns the queue item ID on success *)
106 let submit_worktree ~worktree_path ~summary ~confidence () =
107   (* Get branch from worktree *)
108   let branch =
109     let cmd = Printf.sprintf "cd %s && git branch --show-current"
110       worktree_path in
111     let ic = Unix.open_process_in cmd in
112     try
113       (* exempt: input_line *) let b = input_line ic |> String.trim in
114       ignore (Unix.close_process_in ic);
115       b
116     with _ ->
117       ignore (Unix.close_process_in ic);
118       "unknown"
119   in
120
121   let affected_files = get_affected_files worktree_path in
122
123   (* Check lockfile conflicts *)
124   let lock_conflicts = Lockfile.check_conflicts affected_files in
125   if lock_conflicts <> [] then
126     Error "Conflicts with active runs in lockfile"
127   else
128     (* Create queue item *)
129     let id =
130       let now = Unix.gmtime (Unix.time ()) in
131       Printf.sprintf "auto-%04d%02d%02d-%02d%02d-%04x"
132         (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
133         now.tm_hour now.tm_min now.tm_sec
134         (Random.int 0x10000)
135     in
136     let submitted_at =
137       let now = Unix.gmtime (Unix.time ()) in
138       Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
139         (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
140         now.tm_hour now.tm_min now.tm_sec
141     in

```



```

140   let item = {
141     id;
142     state = Pending;
143     branch;
144     worktree_path;
145     summary;
146     affected_files;
147     confidence;
148     submitted_at;
149     merged_at = None;
150     rejected_reason = None;
151   } in
152
153   (* Load queue, add item, recalculate, save *)
154   match Queue_storage.load () with
155   | Error e -> Error (Printf.sprintf "Failed to load queue: %s" e)
156   | Ok queue ->
157     let queue' = add_item queue item in
158     let queue'' = Conflict_detect.recalculate_states queue' in
159     match Queue_storage.save queue'' with
160     | Error e -> Error (Printf.sprintf "Failed to save queue: %s" e)
161     | Ok () -> Submitted id
162
163   (** Auto-submit on agent completion
164
165       Called by borge make/plan when agent exits cleanly.
166       Respects config settings for clean checks and confidence threshold. *)
167   let auto_submit ~config ~worktree_path () =
168     if not config.auto_submit then
169       Skipped "auto_submit disabled in config"
170     else
171       (* Check worktree is clean if required *)
172       if config.require_clean && not (is_worktree_clean worktree_path) then
173         Skipped "worktree has uncommitted changes"
174       else
175         (* Run checks if required *)
176         if config.require_checks && not (run_checks worktree_path) then
177           Skipped "borg balance/check failed"
178         else
179           (* Get summary and confidence *)
180           let summary = get_commit_summary worktree_path in
181           let confidence = read_confidence worktree_path in
182
183           (* Check confidence threshold *)
184           if confidence < config.min_confidence then
185             Skipped (Printf.sprintf "confidence %.2f below threshold %.2f"
186               confidence config.min_confidence)
187           else
188             submit_worktree ~worktree_path ~summary ~confidence ()
189
190   (** Manual submit with explicit parameters *)
191   let manual_submit ~worktree_path ~summary ~confidence () =
192     submit_worktree ~worktree_path ~summary ~confidence ()
193
194   (** Read config from .borge/config file
195
196       Simple key=value format:
197       auto_submit=true
198       min_confidence=0.7 *)
199   let read_config () =
200     let config_file = ".borge/config" in
201     if not (Sys.file_exists config_file) then
202       default_config
203     else
204       try
205         let lines = File_utils.read_file config_file |> String.split_on_char
206           '\n' in
207         List.fold_left (fun cfg line ->
208           let line = String.trim line in
209           if line = "" || line.[0] = '#' then
210             cfg
211           else
212             try

```

```
212         let eq = String.index line '=' in
213         let key = (* exempt: String.sub *) String.sub line 0 eq |>
String.trim in
214         let value = String.sub line (eq + 1) (String.length line - eq -
1) |> String.trim in
215         match key with
216         | "auto_submit" -> { cfg with auto_submit = bool_of_string
value }
217         | "min_confidence" ->
218             { cfg with min_confidence =
219                 match float_of_string_opt value with Some v -> v | None
-> cfg.min_confidence }
220         | "require_clean" -> { cfg with require_clean = bool_of_string
value }
221         | "require_checks" -> { cfg with require_checks =
bool_of_string value }
222         | _ -> cfg
223         with _ -> cfg
224         ) default_config lines
225     with _ -> default_config
```

54 lib/merge/branch.ml

```

1 (* Git branch management for agent runs
2  *
3  * roerick note (|
4  *   Manages git branches for agent runs with naming convention:
5  *   agent/run-{timestamp}-{uuid}
6  *
7  *   Provides creation, deletion, checking out, and conflict detection.
8  *   Tracks branch state for merge queue processing.
9  * |) *)
10
11 type branch_state =
12 | Active      (* Branch exists and has commits *)
13 | Merged      (* Branch has been merged to main *)
14 | Orphaned    (* Branch exists but no PR/worktree *)
15 | Conflicting (* Has merge conflicts with main *)
16
17 type branch_info = {
18   name : string;
19   state : branch_state;
20   base_commit : string;      (* Commit where branch diverged from main *)
21   head_commit : string;      (* Latest commit on branch *)
22   created_at : string;       (* ISO 8601 timestamp from name *)
23   author : string;           (* Last commit author *)
24 }
25
26 (** Generate branch name with timestamp and uuid *)
27 let generate_name () =
28   let timestamp =
29     let now = Unix.gmtime (Unix.time ()) in
30     Printf.sprintf "%04d%02d%02d-%02d%02d%02d"
31       (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
32       now.tm_hour now.tm_min now.tm_sec
33   in
34   let uuid =
35     Random.self_init ();
36     Printf.sprintf "%04x%04x" (Random.int 0x10000) (Random.int 0x10000)
37   in
38   Printf.sprintf "agent/run-%s-%s" timestamp uuid
39
40 (** Check if a branch name follows agent naming convention *)
41 let is_agent_branch name =
42   String.starts_with ~prefix:"agent/run-" name
43
44 (** Parse timestamp from agent branch name *)
45 let parse_timestamp name =
46   if is_agent_branch name then
47     try
48       (* agent/run-YYYYMMDD-HHMMSS-uuid *)
49       let prefix = "agent/run-" in
50       let rest = String.sub name (String.length prefix) (String.length name
51         - String.length prefix) in
52       let dash = String.index rest '-' in
53       let date = (* exempt: String.sub *) String.sub rest 0 dash in
54       let time_rest = String.sub rest (dash + 1) (String.length rest - dash
55         - 1) in
56       let dash2 = String.index time_rest '-' in
57       let time = (* exempt: String.sub *) String.sub time_rest 0 dash2 in
58       Some (Printf.sprintf "%s-%s" date time)
59     with _ -> None
60   else None
61
62 (** Run git command and return output *)
63 let git_cmd cmd =
64   let full_cmd = Printf.sprintf "git %s 2>&1" cmd in
65   let ic = Unix.open_process_in full_cmd in
66   let output = ref "" in
67   (try
68     while true do
69       (* exempt: input_line *) output := !output ^ input_line ic ^ "\n"
70     done
71   with End_of_file -> ());

```

```

70   let status = Unix.close_process_in ic in
71   (status, !output)
72
73   (** Create new agent branch from current HEAD *)
74   let create ?(from="HEAD") () =
75     let name = generate_name () in
76     match git_cmd (Printf.sprintf "checkout -b %s %s" name from) with
77     | Unix.WEXITED 0, _ -> Ok name
78     | _, err -> Error (Printf.sprintf "Failed to create branch: %s" err)
79
80   (** Create new agent branch in bare repo (for worktrees) *)
81   let create_bare ?(from="main") () =
82     let name = generate_name () in
83     (* First create branch in main repo *)
84     match git_cmd (Printf.sprintf "branch %s %s" name from) with
85     | Unix.WEXITED 0, _ -> Ok name
86     | _, err -> Error (Printf.sprintf "Failed to create branch: %s" err)
87
88   (** Delete a branch *)
89   let delete ?(force=false) name =
90     let flag = if force then "-D" else "-d" in
91     match git_cmd (Printf.sprintf "branch %s %s" flag name) with
92     | Unix.WEXITED 0, _ -> Ok ()
93     | _, err -> Error (Printf.sprintf "Failed to delete branch: %s" err)
94
95   (** List all agent branches *)
96   let list_agent_branches () =
97     match git_cmd "branch -a --format='%(%refname:short)'" with
98     | Unix.WEXITED 0, output ->
99       output
100       |> String.split_on_char '\n'
101       |> List.filter is_agent_branch
102       |> List.filter (fun s -> s <> "")
103     | _ -> []
104
105   (** Get branch info *)
106   let get_info name =
107     (* Get merge base with main *)
108     let base_commit =
109       match git_cmd (Printf.sprintf "merge-base %s main" name) with
110       | Unix.WEXITED 0, output -> String.trim output
111       | _ -> ""
112     in
113     (* Get HEAD commit *)
114     let head_commit =
115       match git_cmd (Printf.sprintf "rev-parse %s" name) with
116       | Unix.WEXITED 0, output -> String.trim output
117       | _ -> ""
118     in
119     (* Check if merged *)
120     let is_merged =
121       match git_cmd (Printf.sprintf "branch --merged main --list %s" name)
122       with
123       | Unix.WEXITED 0, output -> String.trim output <> ""
124       | _ -> false
125     in
126     (* Check for conflicts *)
127     let has_conflicts =
128       match git_cmd (Printf.sprintf "merge-tree $(git merge-base %s main)
129       main %s | grep -c '<<<<<<<' " name name) with
130       | Unix.WEXITED 0, "0" -> false
131       | _ -> true
132     in
133     let state =
134       if is_merged then Merged
135       else if has_conflicts then Conflicting
136       else Active
137     in
138     {
139       name;
140       state;
141       base_commit;
142       head_commit;

```

```

141     created_at = parse_timestamp name |> Option.value ~default:"unknown";
142     author = ""; (* Could get from git log *)
143 }
144
145 (** Check if branch has conflicts with main *)
146 let has_conflicts name =
147   (get_info name).state = Conflicting
148
149 (** Get list of files changed in branch compared to main *)
150 let changed_files name =
151   match git_cmd (Printf.sprintf "diff --name-only main...%s" name) with
152   | Unix.WEXITED 0, output ->
153     output
154     |> String.split_on_char '\n'
155     |> List.filter (fun s -> s <> "")
156   | _ -> []
157
158 (** Rebase branch onto main *)
159 let rebase_onto_main name =
160   match git_cmd (Printf.sprintf "rebase main %s" name) with
161   | Unix.WEXITED 0, _ -> Ok ()
162   | _, err ->
163     (* Abort failed rebase *)
164     ignore (git_cmd "rebase --abort");
165     Error (Printf.sprintf "Rebase failed: %s" err)
166
167 (** Merge branch into main (fast-forward only) *)
168 let merge_ff name =
169   match git_cmd (Printf.sprintf "merge --ff-only %s" name) with
170   | Unix.WEXITED 0, _ -> Ok ()
171   | _, err -> Error (Printf.sprintf "Fast-forward merge failed: %s" err)
172
173 (** Merge branch into main (create merge commit) *)
174 let merge_commit ~message name =
175   match git_cmd (Printf.sprintf "merge -m '%s' %s" message name) with
176   | Unix.WEXITED 0, _ -> Ok ()
177   | _, err -> Error (Printf.sprintf "Merge failed: %s" err)
178
179 (** Squash merge branch into main *)
180 let merge_squash ~message name =
181   match git_cmd (Printf.sprintf "merge --squash --no-commit %s" name) with
182   | Unix.WEXITED 0, _ ->
183     (* Commit with message *)
184     (match git_cmd (Printf.sprintf "commit -m '%s'" message) with
185     | Unix.WEXITED 0, _ -> Ok ()
186     | _, err -> Error (Printf.sprintf "Squash commit failed: %s" err))
187   | _, err -> Error (Printf.sprintf "Squash merge failed: %s" err)

```

55 lib/merge/conflict_detect.ml

```

1 (* File overlap analysis for merge queue conflict detection
2  *
3  * roerick note (|
4  *   Analyzes file overlap between queue items to detect conflicts.
5  *   Items that touch the same files cannot be merged in parallel.
6  *
7  *   Two items conflict if they share any file in their affected_files
8  *   lists.
9  * |) *)
10
11 open Queue_types
12
13 (** Conflict between two queue items *)
14 type conflict = {
15   item1_id : item_id;
16   item2_id : item_id;
17   conflicting_files : string list; (** Files that both items touch *)
18 }
19
20 (** Check if two items have file overlap *)
21 let has_overlap item1 item2 =
22   List.exists (fun f1 ->
23     List.exists (fun f2 -> f1 = f2) item2.affected_files
24   ) item1.affected_files
25
26 (** Find conflicting files between two items *)
27 let find_conflicting_files item1 item2 =
28   List.filter (fun f1 ->
29     List.exists (fun f2 -> f1 = f2) item2.affected_files
30   ) item1.affected_files
31
32 (** Find all conflicts for a given item against other items *)
33 let find_conflicts_for item other_items =
34   List.filter_map (fun other ->
35     if has_overlap item other then
36       Some {
37         item1_id = item.id;
38         item2_id = other.id;
39         conflicting_files = find_conflicting_files item other;
40       }
41     else
42       None
43   ) other_items
44
45 (** Check if an item is blocked by any items before it in queue *)
46 let is_blocked_by queue item_id =
47   match find_item queue item_id with
48   | None -> []
49   | Some item ->
50     let predecessors = predecessors queue item_id in
51     (* Only ACTIVE or MERGING predecessors can block *)
52     let active_predecessors = List.filter (fun p ->
53       p.state = Pending || p.state = Ready || p.state = Merging
54     ) predecessors in
55     find_conflicts_for item active_predecessors
56
57 (** Recalculate Ready/Blocked states for all items
58  Rules:
59  - First item is always Ready (unless already Merged/Rejected)
60  - Item is Blocked if it conflicts with any Ready/Merging item before it
61  - Item is Ready if no conflicts with Ready/Merging items before it *)
62 let recalculate_states queue =
63   let rec process items processed =
64     match items with
65     | [] -> List.rev processed
66     | item :: rest ->
67       if item.state = Merged || item.state = Rejected then
68         (* Keep terminal states *)
69         process rest (item :: processed)
70       else

```

```

71      (* Check for conflicts with active processed items *)
72      let active_processed = List.filter (fun p ->
73        p.state = Ready || p.state = Merging
74      ) processed in
75      let conflicts = find_conflicts_for item active_processed in
76      let new_state = if conflicts = [] then Ready else Blocked in
77      let new_item = { item with state = new_state } in
78      process rest (new_item :: processed)
79    in
80    { queue with items = process queue.items [] }
81
82  (** Find all pairs of conflicting items in queue *)
83  let find_all_conflicts queue =
84    let rec find_pairs = function
85      | [] -> pairs
86      | item :: rest ->
87        let new_pairs = List.filter_map (fun other ->
88          if has_overlap item other then
89            Some {
90              item1_id = item.id;
91              item2_id = other.id;
92              conflicting_files = find_conflicting_files item other;
93            }
94          else None
95        ) rest in
96        find (new_pairs @ pairs) rest
97    in
98    find [] queue.items
99
100  (** Group items into batches that can be merged in parallel
101
102      Items in same batch must not conflict with each other.
103      Items in later batches must wait for all earlier batches. *)
104  let group_into_batches queue =
105    let items = List.filter (fun i ->
106      i.state = Ready || i.state = Pending || i.state = Blocked
107    ) queue.items in
108
109    let rec group_batches remaining =
110      if remaining = [] then List.rev batches
111      else
112        (* Find largest set of non-conflicting items *)
113        let rec find_batch acc candidates =
114          match candidates with
115          | [] -> acc
116          | item :: rest ->
117            (* Check if item conflicts with any in acc *)
118            let conflicts_with_acc = List.exists (fun a -> has_overlap item
119              a) acc in
120            if conflicts_with_acc then
121              find_batch acc rest
122            else
123              find_batch (item :: acc) rest
124          in
125          let batch = find_batch [] remaining in
126          let remaining' = List.filter (fun r ->
127            not (List.exists (fun b -> b.id = r.id) batch)
128          ) remaining in
129          group (batch :: batches) remaining'
130        in
131        group [] items
132
133  (** Estimate merge order based on dependencies *)
134  let estimate_merge_order queue =
135    let batches = group_into_batches queue in
136    List.mapi (fun i batch ->
137      List.map (fun item -> (item.id, i + 1)) batch
138    ) batches
139    |> List.concat
140
141  (** Check if adding a new item would block existing items *)
142  let would_cause_new_blocks queue new_item =
143    let queue' = add_item queue new_item in

```

```
143   let conflicts = find_all_conflicts queue' in
144   List.filter (fun c -> c.item2_id = new_item.id) conflicts
145
146   (** Format conflict for display *)
147   let format_conflict (c : conflict) =
148     Printf.sprintf "%s <-> %s on %s"
149       c.item1_id c.item2_id
150       (String.concat ", " c.conflicting_files)
```


56 lib/merge/lockfile.ml

```

1 (* Concurrent lockfile for agent runs
2  *
3  * roerick note (|
4  *   Manages .pi/concurrent.lock for coordinating agent runs.
5  *
6  *   Lock format (JSON):
7  *   {
8  *     "active_runs": [
9  *       {
10 *         "id": "agent-123",
11 *         "branch": "agent/run-...",
12 *         "files": ["lib/parser.ml"],
13 *         "since": "2026-05-17T10:30:00Z"
14 *       }
15 *     ]
16 *   }
17 *
18 *   Provides acquire, release, and conflict checking.
19 * |) *)
20
21 type run_lock = {
22   id : string;
23   branch : string;
24   files : string list;
25   since : string; (* ISO 8601 *)
26 }
27
28 type lockfile = {
29   active_runs : run_lock list;
30 }
31
32 (** Default lockfile path *)
33 let default_path = ".pi/concurrent.lock"
34
35 (** Ensure .pi directory exists *)
36 let ensure_pi_dir () =
37   if not (Sys.file_exists ".pi") then
38     Unix.mkdir ".pi" 0o755
39
40 (** Read lockfile *)
41 let read ?(path=default_path) () =
42   if not (Sys.file_exists path) then
43     Ok { active_runs = [] }
44   else
45     try
46       let _content = File_utils.read_file path in
47       (* Simple JSON parsing for our specific format *)
48       (* For now, return empty - full JSON parsing would need yojson *)
49       Ok { active_runs = [] }
50     with e ->
51       Error (Printf.sprintf "Failed to read lockfile: %s"
52         (Printexc.to_string e))
53
54 (** Write lockfile *)
55 let write ?(path=default_path) lockfile =
56   ensure_pi_dir ();
57   try
58     let oc = open_out path in
59     (* Simple JSON output *)
60     output_string oc "{\n";
61     output_string oc "  \"active_runs\": [\n";
62     List.iteri (fun i run ->
63       output_string oc "    {\n";
64       output_string oc (Printf.sprintf "      \"id\": \"%s\", \n" run.id);
65       output_string oc (Printf.sprintf "      \"branch\": \"%s\", \n"
66         run.branch);
67       output_string oc "      \"files\": [";
68       List.iteri (fun j file ->
69         if j > 0 then output_string oc ", ";
70         output_string oc (Printf.sprintf "\"%s\" " file)
71       ) run.files;

```

```

70     output_string oc "],\n";
71     output_string oc (Printf.sprintf "        \\"since\\": \\"%s\\"\\n"
run.since);
72     output_string oc "    }";
73     if i < List.length lockfile.active_runs - 1 then
74         output_string oc ",\n"
75     else
76         output_string oc "\n"
77 ) lockfile.active_runs;
78     output_string oc " ]\n";
79     output_string oc "}\n";
80     close_out oc;
81     Ok ()
82 with e ->
83     Error (Printf.sprintf "Failed to write lockfile: %s"
(Printexc.to_string e))
84
85 (** Acquire lock for a new run
86
87     Checks for conflicts with existing runs.
88     Returns Error if any existing run touches the same files. *)
89 let acquire ~id ~branch ~files () =
90     match read () with
91     | Error e -> Error e
92     | Ok lockfile ->
93         (* Check for conflicts *)
94         let conflicts = List.filter (fun (run : run_lock) ->
95             List.exists (fun f1 ->
96                 List.exists (fun f2 -> f1 = f2) run.files
97             ) files
98         ) lockfile.active_runs in
99
100     if conflicts <> [] then
101         Error (Printf.sprintf "Conflict with existing run(s): %s"
(String.concat " ", " (List.map (fun r -> r.id) conflicts)))
102     else
103         let timestamp =
104             let now = Unix.gmtime (Unix.time ()) in
105             Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
106                 (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
107                 now.tm_hour now.tm_min now.tm_sec
108         in
109         let new_run = { id; branch; files; since = timestamp } in
110         let updated = { active_runs = new_run :: lockfile.active_runs } in
111         write updated
112
113 (** Release lock for a run
114
115     Removes the run from the active_runs list. *)
116 let release ~id () =
117     match read () with
118     | Error e -> Error e
119     | Ok lockfile ->
120         let filtered = List.filter (fun (run : run_lock) -> run.id <> id)
lockfile.active_runs in
121         write { active_runs = filtered }
122
123 (** Check if files conflict with any active run *)
124 let check_conflicts files =
125     match read () with
126     | Error _ -> [] (* If can't read, assume no conflicts *)
127     | Ok lockfile ->
128         List.filter (fun (run : run_lock) ->
129             List.exists (fun f1 ->
130                 List.exists (fun f2 -> f1 = f2) run.files
131             ) files
132         ) lockfile.active_runs
133
134 (** Check if any runs are active *)
135 let has_active_runs () =
136     match read () with
137     | Error _ -> false
138     | Ok lockfile -> lockfile.active_runs <> []

```

```
140
141 (** List all active runs *)
142 let list_active () =
143   match read () with
144   | Error _ -> []
145   | Ok lockfile -> lockfile.active_runs
146
147 (** Remove stale locks (runs older than threshold) *)
148 let prune_stale ~max_age_seconds () =
149   match read () with
150   | Error _ -> Ok ()
151   | Ok lockfile ->
152     let now = Unix.time () in
153     let fresh = List.filter (fun (run : run_lock) ->
154       (* Parse timestamp and check age *)
155       try
156         Scanf.sscanf run.since "%04d-%02d-%02dT%02d:%02d:%02dZ"
157           (fun y mo d h mi se ->
158             let tm = { Unix.tm_year = y - 1900; tm_mon = mo - 1; tm_mday
159               = d;
160                 tm_hour = h; tm_min = mi; tm_sec = se;
161                 tm_wday = 0; tm_yday = 0; tm_isdst = false }
162             in
163             let run_time = Unix.mktime tm |> fst in
164             now -. run_time < float_of_int max_age_seconds)
165           with _ -> true (* Keep if can't parse *)
166     ) lockfile.active_runs in
167     write { active_runs = fresh }
```

57 lib/merge/merge_strategies.ml

```

1 (* Git merge strategies for agent queue processing
2  *
3  * roerick note (|
4  *   Implements merge strategies:
5  *   - FAST-FORWARD: Only if main hasn't moved
6  *   - REBASE-THEN-MERGE: Rebase onto main, then fast-forward
7  *   - MERGE-COMMIT: Always create merge commit
8  *   - SQUASH: Squash all commits to single commit
9  *
10  *   Each strategy reports success or detailed error for rollback.
11  * |) *)
12
13 type strategy =
14   | FastForward
15   | RebaseThenMerge
16   | MergeCommit
17   | Squash
18
19 type merge_result =
20   | Success of string (* Commit hash *)
21   | Error of string   (* Error message *)
22   | Conflicts of string list (* List of conflicting files *)
23
24 (** Run git command and capture output *)
25 let git_cmd cmd =
26   let full_cmd = Printf.sprintf "git %s 2>&1" cmd in
27   let ic = Unix.open_process_in full_cmd in
28   let output = ref "" in
29   (try
30     while true do
31       (* exempt: input_line *) output := !output ^ input_line ic ^ "\n"
32     done
33   with End_of_file -> ());
34   let status = Unix.close_process_in ic in
35   (status, !output)
36
37 (** Check if fast-forward is possible *)
38 let can_fast_forward branch =
39   match git_cmd (Printf.sprintf "merge-base --is-ancestor HEAD %s" branch)
40     with
41   | Unix.WEXITED 0, _ -> true
42   | _ -> false
43
44 (** Check for merge conflicts before attempting *)
45 let check_conflicts branch =
46   (* Use merge-tree to check for conflicts without actually merging *)
47   match git_cmd (Printf.sprintf "merge-tree $(git merge-base HEAD %s) HEAD
48     %s | grep -c '<<<<<<<' branch branch) with
49   | Unix.WEXITED 0, "0\n" -> [] (* No conflicts *)
50   | Unix.WEXITED 0, _ ->
51     (* Has conflicts, get list of files *)
52     (match git_cmd (Printf.sprintf "merge-tree $(git merge-base HEAD %s)
53       HEAD %s | grep '<<<<<<<' branch branch) with
54     | Unix.WEXITED 0, output ->
55       output |> String.split_on_char '\n' |> List.filter (fun s -> s
56         <> "")
57     | _ -> ["unknown"])
58   | _ -> []
59
60 (** Fast-forward merge *)
61 let fast_forward_merge branch =
62   if not (can_fast_forward branch) then
63     Error (Printf.sprintf "Cannot fast-forward %s: main has new commits"
64       branch)
65   else
66     match git_cmd (Printf.sprintf "merge --ff-only %s" branch) with
67     | Unix.WEXITED 0, _ ->
68       (match git_cmd "rev-parse HEAD" with
69       | Unix.WEXITED 0, hash -> Success (String.trim hash)
70       | _ -> Error "Fast-forward succeeded but couldn't get commit hash")
71     | _, err -> Error (Printf.sprintf "Fast-forward failed: %s" err)

```

```

67
68 (** Rebase branch onto main, then fast-forward *)
69 let rebase_then_merge branch =
70   (* First check for conflicts *)
71   let conflicts = check_conflicts branch in
72   if conflicts <> [] then
73     Conflicts conflicts
74   else
75     (* Switch to branch and rebase *)
76     match git_cmd (Printf.sprintf "checkout %s" branch) with
77     | Unix.WEXITED 0, _ ->
78       (match git_cmd "rebase main" with
79       | Unix.WEXITED 0, _ ->
80         (* Rebase succeeded, switch back to main and fast-forward *)
81         (match git_cmd "checkout main" with
82         | Unix.WEXITED 0, _ ->
83           fast_forward_merge branch
84         | _, err ->
85           Error (Printf.sprintf "Failed to checkout main after
rebase: %s" err))
86         | _, err ->
87           (* Rebase failed, abort and report *)
88           ignore (git_cmd "rebase --abort");
89           ignore (git_cmd "checkout main");
90           if String.contains err 'c' then
91             Conflicts ["rebase conflict"]
92           else
93             Error (Printf.sprintf "Rebase failed: %s" err))
94         | _, err -> Error (Printf.sprintf "Failed to checkout branch: %s" err)
95
96 (** Create merge commit *)
97 let merge_commit ~message branch =
98   (* Check for conflicts first *)
99   let conflicts = check_conflicts branch in
100   if conflicts <> [] then
101     Conflicts conflicts
102   else
103     let merge_msg = Printf.sprintf "Merge branch '%s': %s" branch message in
104     match git_cmd (Printf.sprintf "merge -m '%s' %s" merge_msg branch) with
105     | Unix.WEXITED 0, _ ->
106       (match git_cmd "rev-parse HEAD" with
107       | Unix.WEXITED 0, hash -> Success (String.trim hash)
108       | _ -> Error "Merge succeeded but couldn't get commit hash")
109     | Unix.WEXITED 1, _ ->
110       (* May have conflicts, check for conflicted files *)
111       (match git_cmd "diff --name-only --diff-filter=U" with
112       | Unix.WEXITED 0, "" -> Error "Merge failed (no conflicts
detected)"
113       | Unix.WEXITED 0, files ->
114         Conflicts (files |> String.split_on_char '\n' |> List.filter
(fun s -> s <> ""))
115       | _ -> Conflicts ["unknown"])
116     | _, err -> Error (Printf.sprintf "Merge failed: %s" err)
117
118 (** Squash merge *)
119 let squash_merge ~message branch =
120   (* Check for conflicts first *)
121   let conflicts = check_conflicts branch in
122   if conflicts <> [] then
123     Conflicts conflicts
124   else
125     match git_cmd (Printf.sprintf "merge --squash --no-commit %s" branch)
with
126     | Unix.WEXITED 0, _ ->
127       (* Check if there are changes to commit *)
128       (match git_cmd "diff --cached --quiet" with
129       | Unix.WEXITED 0, _ ->
130         (* No changes - already up to date *)
131         ignore (git_cmd "merge --abort");
132         (match git_cmd "rev-parse HEAD" with
133         | Unix.WEXITED 0, hash -> Success (String.trim hash)
134         | _ -> Error "No changes but couldn't get commit hash")
135       | _ ->

```

```

136         (* Has changes, commit them *)
137         let squash_msg = Printf.sprintf "Squash merge %s: %s" branch
message in
138         (match git_cmd (Printf.sprintf "commit -m '%s'" squash_msg)
with
139             | Unix.WEXITED 0, _ ->
140                 (match git_cmd "rev-parse HEAD" with
141                     | Unix.WEXITED 0, hash -> Success (String.trim hash)
142                     | _ -> Error "Squash succeeded but couldn't get commit
hash")
143             | _, err -> Error (Printf.sprintf "Squash commit failed: %s"
err)))
144         | Unix.WEXITED 1, _ ->
145             Conflicts ["conflict"]
146         | _, err -> Error (Printf.sprintf "Squash merge failed: %s" err)
147
148 (** Execute merge with specified strategy *)
149 let execute ~strategy ~message branch =
150     (* Save current state for potential rollback *)
151     let before_merge =
152         match git_cmd "rev-parse HEAD" with
153             | Unix.WEXITED 0, hash -> Some (String.trim hash)
154             | _ -> None
155     in
156
157     let result = match strategy with
158         | FastForward -> fast_forward_merge branch
159         | RebaseThenMerge -> rebase_then_merge branch
160         | MergeCommit -> merge_commit ~message branch
161         | Squash -> squash_merge ~message branch
162     in
163
164     (before_merge, result)
165
166 (** String representation of strategy *)
167 let string_of_strategy = function
168     | FastForward -> "fast-forward"
169     | RebaseThenMerge -> "rebase-then-merge"
170     | MergeCommit -> "merge-commit"
171     | Squash -> "squash"
172
173 (** Parse strategy from string *)
174 let strategy_of_string = function
175     | "fast-forward" | "ff" -> Some FastForward
176     | "rebase-then-merge" | "rebase" -> Some RebaseThenMerge
177     | "merge-commit" | "merge" -> Some MergeCommit
178     | "squash" -> Some Squash
179     | _ -> None
180
181 (** Get default strategy for project *)
182 let default_strategy () =
183     (* Could read from config file *)
184     FastForward

```

58 lib/merge/queue_storage.ml

```

1 (* Merge queue state persistence
2  *
3  * roerick note (|
4  *   Serializes and persists merge queue state.
5  *   Storage format: .borge/queue/state.json
6  *
7  *   Operations:
8  *   - Load/save full queue state
9  *   - Atomic updates via write-to-temp-then-rename
10  *   - Backup previous state on write
11  * |) *)
12
13 open Queue_types
14
15 (** Default storage path *)
16 let default_path = ".borge/queue/state.json"
17
18 (** Create directory recursively *)
19 let rec mkdir_p path =
20   if not (Sys.file_exists path) then begin
21     let parent = Filename.dirname path in
22     if parent <> path && parent <> "." && parent <> "/" then
23       mkdir_p parent;
24     try Unix.mkdir path 0o755 with _ -> ()
25   end
26
27 (** Ensure directory exists *)
28 let ensure_dir path =
29   let dir = Filename.dirname path in
30   mkdir_p dir
31
32 (** Serialize queue to JSON string *)
33 let serialize queue =
34   let buf = Buffer.create 1024 in
35   Buffer.add_string buf "{\n";
36   Buffer.add_string buf "  \"items\": [\n";
37
38   List.iteri (fun i item ->
39     Buffer.add_string buf "    {\n";
40     Buffer.add_string buf (Printf.sprintf "      \"id\": \"%s\", \n"
41       item.id);
42     Buffer.add_string buf (Printf.sprintf "      \"state\": \"%s\", \n"
43       (string_of_state item.state));
44     Buffer.add_string buf (Printf.sprintf "      \"branch\": \"%s\", \n"
45       item.branch);
46     Buffer.add_string buf (Printf.sprintf "      \"worktree_path\":\n"
47       "\"%s\", \n" item.worktree_path);
48     Buffer.add_string buf (Printf.sprintf "      \"summary\": \"%s\", \n"
49       (String.escaped item.summary));
50     Buffer.add_string buf "      \"affected_files\": [";
51     List.iteri (fun j f ->
52       if j > 0 then Buffer.add_string buf ", ";
53       Buffer.add_string buf (Printf.sprintf "\"%s\"" (String.escaped f))
54     ) item.affected_files;
55     Buffer.add_string buf "], \n";
56     Buffer.add_string buf (Printf.sprintf "      \"confidence\": %.2f, \n"
57       item.confidence);
58     Buffer.add_string buf (Printf.sprintf "      \"submitted_at\": \"%s\""
59       item.submitted_at);
60     (match item.merged_at with
61     | Some t -> Buffer.add_string buf (Printf.sprintf "      \n"
62       "\"merged_at\": \"%s\"" t)
63     | None -> ());
64     (match item.rejected_reason with
65     | Some r -> Buffer.add_string buf (Printf.sprintf "      \n"
66       "\"rejected_reason\": \"%s\"" (String.escaped r))
67     | None -> ());
68     Buffer.add_string buf "    }\n";
69   if i < List.length queue.items - 1 then
70     Buffer.add_string buf ", \n"
71   else

```

```

63     Buffer.add_string buf "\n"
64   ) queue.items;
65
66   Buffer.add_string buf " ],\n";
67   Buffer.add_string buf (Printf.sprintf "  \"next_merge_id\": %s\n"
68     (match queue.next_merge_id with Some id -> Printf.sprintf "\"%s\"" id |
69       None -> "null"));
70   Buffer.add_string buf "}\n";
71   Buffer.contents buf
72
73   (** Parse queue from JSON string - simplified *)
74   let deserialize _json_str =
75     (* For now, return empty queue - full JSON parsing would need yojson *)
76     empty_queue
77
78   (** Save queue to file (atomic) *)
79   let save ?(path=default_path) queue =
80     ensure_dir path;
81     let json = serialize queue in
82     let temp_path = path ^ ".tmp" in
83     try
84       (* Write to temp file *)
85       let oc = open_out temp_path in
86       output_string oc json;
87       close_out oc;
88       (* Atomic rename *)
89       Sys.rename temp_path path;
90       Ok ()
91     with e ->
92       (* Clean up temp file on error *)
93       (try Sys.remove temp_path with _ -> ());
94       Error (Printf.sprintf "Failed to save queue: %s" (Printexc.to_string e))
95
96   (** Load queue from file *)
97   let load ?(path=default_path) () =
98     if not (Sys.file_exists path) then
99       Ok empty_queue
100     else
101       try
102         let json = File_utils.read_file path in
103         Ok (deserialize json)
104       with e ->
105         Error (Printf.sprintf "Failed to load queue: %s" (Printexc.to_string e))
106
107   (** Backup current state before write *)
108   let backup ?(path=default_path) () =
109     if Sys.file_exists path then
110       let backup_path = path ^ ".bak" in
111       try
112         let content = File_utils.read_file path in
113         let oc = open_out backup_path in
114         output_string oc content;
115         close_out oc;
116         Ok ()
117       with e ->
118         Error (Printf.sprintf "Failed to backup: %s" (Printexc.to_string e))
119     else
120       Ok ()
121
122   (** Save with backup *)
123   let save_with_backup ?(path=default_path) queue =
124     match backup ~path () with
125     | Error e -> Error e
126     | Ok () -> save ~path queue
127
128   (** Check if queue file exists *)
129   let exists ?(path=default_path) () =
130     Sys.file_exists path
131
132   (** Get last modified time *)
133   let last_modified ?(path=default_path) () =
134     if exists ~path () then

```



```
134     try
135         let stats = Unix.stat path in
136         Some stats.Unix.st_mtime
137     with _ -> None
138 else
139     None
```

59 lib/merge/queue_types.ml

```

1 (* Merge queue state types and transitions
2  *
3  * roerick note (|
4  *   Defines the merge queue state machine and data structures.
5  *
6  *   Queue item states:
7  *   - PENDING: Submitted, waiting for merge
8  *   - READY: No conflicts with queued items before it
9  *   - BLOCKED: Conflicts with earlier queue item
10  *   - MERGING: Currently being processed
11  *   - MERGED: Successfully merged to main
12  *   - REJECTED: Merge failed, worktree preserved
13  * |) *)
14
15 (** Unique queue item ID *)
16 type item_id = string
17
18 (** State of a queue item *)
19 type queue_state =
20 | Pending      (** Submitted, waiting for merge *)
21 | Ready        (** No conflicts with earlier items *)
22 | Blocked      (** Conflicts with earlier queue item *)
23 | Merging      (** Currently being processed *)
24 | Merged       (** Successfully merged to main *)
25 | Rejected     (** Merge failed, worktree preserved *)
26
27 (** A single item in the merge queue *)
28 type queue_item = {
29   id : item_id;
30   state : queue_state;
31   branch : string;          (** Git branch name *)
32   worktree_path : string;   (** Path to worktree *)
33   summary : string;         (** Description of changes *)
34   affected_files : string list; (** Files this item touches *)
35   confidence : float;       (** 0.0-1.0 confidence score *)
36   submitted_at : string;     (** ISO 8601 timestamp *)
37   merged_at : string option; (** When merged, if merged *)
38   rejected_reason : string option; (** Why rejected, if rejected *)
39 }
40
41 (** The full merge queue *)
42 type merge_queue = {
43   items : queue_item list;    (** All queue items, in order *)
44   next_merge_id : item_id option; (** Which item is being processed *)
45 }
46
47 (** Event for queue transitions *)
48 type queue_event =
49 | Submit of queue_item      (** New item submitted *)
50 | MarkReady of item_id      (** Item marked ready (conflicts resolved) *)
51 | MarkBlocked of item_id    (** Item blocked by earlier item *)
52 | StartMerge of item_id     (** Begin processing item *)
53 | CompleteMerge of item_id  (** Item successfully merged *)
54 | Reject of item_id * string (** Item rejected with reason *)
55 | Cancel of item_id         (** Item manually cancelled *)
56 | Refresh                   (** Recalculate all states *)
57
58 (** String representation of state *)
59 let string_of_state = function
60 | Pending -> "PENDING"
61 | Ready -> "READY"
62 | Blocked -> "BLOCKED"
63 | Merging -> "MERGING"
64 | Merged -> "MERGED"
65 | Rejected -> "REJECTED"
66
67 (** Parse state from string *)
68 let state_of_string = function
69 | "PENDING" -> Some Pending
70 | "READY" -> Some Ready
71 | "BLOCKED" -> Some Blocked

```

```

72 | "MERGING" -> Some Merging
73 | "MERGED" -> Some Merged
74 | "REJECTED" -> Some Rejected
75 | _ -> None
76
77 (** Create a new empty queue *)
78 let empty_queue = { items = []; next_merge_id = None }
79
80 (** Find item by ID *)
81 let find_item queue id =
82   List.find_opt (fun item -> item.id = id) queue.items
83
84 (** Find index of item in queue *)
85 let item_index queue id =
86   let rec find i = function
87     | [] -> None
88     | item :: rest -> if item.id = id then Some i else find (i + 1) rest
89   in
90   find 0 queue.items
91
92 (** Get items that come before a given item *)
93 let predecessors queue id =
94   match item_index queue id with
95   | None -> []
96   | Some idx ->
97     queue.items
98     |> List.mapi (fun i item -> (i, item))
99     |> List.filter (fun (i, _) -> i < idx)
100     |> List.map snd
101
102 (** Get all items in a specific state *)
103 let items_in_state queue state =
104   List.filter (fun item -> item.state = state) queue.items
105
106 (** Count items by state *)
107 let count_by_state queue =
108   let counts = [
109     ("pending", List.length (items_in_state queue Pending));
110     ("ready", List.length (items_in_state queue Ready));
111     ("blocked", List.length (items_in_state queue Blocked));
112     ("merging", List.length (items_in_state queue Merging));
113     ("merged", List.length (items_in_state queue Merged));
114     ("rejected", List.length (items_in_state queue Rejected));
115   ] in
116   counts
117
118 (** Check if an item can transition to a new state *)
119 let can_transition item new_state =
120   match item.state, new_state with
121   | Pending, Ready -> true
122   | Pending, Blocked -> true
123   | Ready, Merging -> true
124   | Ready, Blocked -> true (* Can become blocked if new conflicts *)
125   | Blocked, Ready -> true
126   | Merging, Merged -> true
127   | Merging, Rejected -> true
128   | _, Rejected -> true (* Can reject from any state (cancellation) *)
129   | _ -> false
130
131 (** Update item state (returns new queue) *)
132 let update_state queue id new_state =
133   { queue with
134     items = List.map (fun item ->
135       if item.id = id then
136         let merged_at =
137           if new_state = Merged && item.state <> Merged then
138             let now = Unix.gmtime (Unix.time ()) in
139             Some (Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
140               (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
141               now.tm_hour now.tm_min now.tm_sec)
142           else item.merged_at
143         in
144         { item with state = new_state; merged_at }

```

```
145     else item
146   ) queue.items
147 }
148
149 (** Remove item from queue *)
150 let remove_item queue id =
151   { queue with items = List.filter (fun item -> item.id <> id) queue.items }
152
153 (** Add new item to end of queue *)
154 let add_item queue item =
155   { queue with items = queue.items @ [item] }
156
157 (** Get next ready item (oldest ready item) *)
158 let next_ready queue =
159   queue.items
160   |> List.filter (fun item -> item.state = Ready)
161   |> List.sort (fun a b -> String.compare a.submitted_at b.submitted_at)
162   |> (function [] -> None | x :: _ -> Some x)
```

60 lib/merge/rollback.ml

```

1 (* Rollback failed merges
2  *
3  * roerick note (|
4  *   Reverts failed merges to restore main branch state.
5  *
6  *   Rollback strategies:
7  *   - SOFT: Reset to before merge (preserves working tree)
8  *   - HARD: Reset and clean working tree
9  *   - REVERT: Create new revert commit
10 *
11 *   Always preserves agent worktree for debugging.
12 * |) *)
13
14 type rollback_type =
15   | Soft      (* git reset --soft *)
16   | Hard      (* git reset --hard *)
17   | Revert    (* git revert *)
18
19 type rollback_result =
20   | Success
21   | Error of string
22
23 (** Run git command *)
24 let git_cmd cmd =
25   let full_cmd = Printf.sprintf "git %s 2>&1" cmd in
26   let ic = Unix.open_process_in full_cmd in
27   let output = ref "" in
28   (try
29     while true do
30       (* exempt: input_line *) output := !output ^ input_line ic ^ "\n"
31     done
32   with End_of_file -> ());
33   let status = Unix.close_process_in ic in
34   (status, !output)
35
36 (** Rollback to a specific commit
37
38     type_: Soft preserves changes in working tree
39           Hard removes all changes
40           Revert creates new commit
41     commit: Hash to rollback to
42     branch: Current branch name (for revert message *)
43 let rollback ~type_ ~commit ?(branch="") () =
44   match type_ with
45   | Soft ->
46     (* Soft reset - keeps changes staged *)
47     (match git_cmd (Printf.sprintf "reset --soft %s" commit) with
48      | Unix.WEXITED 0, _ ->
49        Printf.printf "Rolled back to %s (changes preserved in staging
50 area)\n" commit;
51        Success
52      | _, err -> Error (Printf.sprintf "Soft rollback failed: %s" err))
53   | Hard ->
54     (* Hard reset - removes all changes *)
55     (match git_cmd (Printf.sprintf "reset --hard %s" commit) with
56      | Unix.WEXITED 0, _ ->
57        Printf.printf "Rolled back hard to %s\n" commit;
58        Success
59      | _, err -> Error (Printf.sprintf "Hard rollback failed: %s" err))
60   | Revert ->
61     (* Create revert commits *)
62     (* First find commits since the target *)
63     (match git_cmd (Printf.sprintf "rev-list --reverse %s..HEAD" commit)
64      with
65      | Unix.WEXITED 0, commits_str ->
66        let commits =
67          commits_str
68          |> String.split_on_char '\n'
69          |> List.filter (fun s -> s <> "")

```

```

70     in
71     if commits = [] then
72         Success (* Nothing to revert *)
73     else begin
74         (* Revert each commit in reverse order *)
75         let rec revert_all = function
76             | [] -> Success
77             | c :: rest ->
78                 match git_cmd (Printf.sprintf "revert --no-commit %s" c)
79                     with
80                     | Unix.WEXITED 0, _ -> revert_all rest
81                     | _, err ->
82                         ignore (git_cmd "revert --abort");
83                         Error (Printf.sprintf "Revert failed at %s: %s" c
84                             err)
85                     in
86                         match revert_all commits with
87                         | Success ->
88                             (* Commit the reverts *)
89                             let commit_msg = Printf.sprintf "Revert failed merge of
90                                 %s" branch in
91                                 (match git_cmd (Printf.sprintf "commit -m '%s'"
92                                     commit_msg) with
93                                 | Unix.WEXITED 0, _ ->
94                                     Printf.printf "Created revert commits for failed
95                                     merge\n";
96                                     Success
97                                 | _, err -> Error (Printf.sprintf "Failed to commit
98                                     reverts: %s" err))
99                                 | Error e -> Error e
100                             end
101                         | _, err -> Error (Printf.sprintf "Failed to list commits to revert:
102                             %s" err))
103
104 (** Rollback a failed merge from merge_strategies result *)
105 let rollback_merge ~type_ ~before_merge ~branch result =
106     match result with
107     | Merge_strategies.Success _ ->
108         Printf.eprintf "Warning: Cannot rollback successful merge\n";
109         Error "Cannot rollback successful merge"
110     | Merge_strategies.Conflicts _files ->
111         (* Abort any ongoing merge *)
112         ignore (git_cmd "merge --abort");
113         (* Reset to before merge *)
114         (match before_merge with
115         | Some commit -> rollback ~type_ ~commit ~branch ()
116         | None ->
117             Printf.eprintf "Warning: No before-merge commit recorded\n";
118             ignore (git_cmd "merge --abort");
119             Success)
120     | Merge_strategies.Error msg ->
121         (* Abort any ongoing merge *)
122         ignore (git_cmd "merge --abort");
123         (* Reset to before merge *)
124         (match before_merge with
125         | Some commit ->
126             let result = rollback ~type_ ~commit ~branch () in
127             (match result with
128             | Success -> Printf.printf "Rolled back failed merge: %s\n" msg
129             | _ -> ());
130             result
131         | None ->
132             ignore (git_cmd "merge --abort");
133             Success)
134
135 (** Preserve worktree for debugging after rollback
136
137     Creates a reference to the failed state before rolling back. *)
138 let preserve_worktree ~branch ~worktree_path result =
139     (* let _failed_ref = Printf.sprintf "refs/failed-merges/%s" branch in *)
140     match result with

```

```
136 | Merge_strategies.Success _ -> () (* Don't preserve successes *)
137 | _ ->
138   (* Tag the failed state *)
139   ignore (git_cmd (Printf.sprintf "tag -f %s-failed HEAD" branch));
140   Printf.printf "Preserved failed state as %s-failed tag\n" branch;
141   Printf.printf "Worktree preserved at: %s\n" worktree_path
142
143 (** Full rollback workflow
144
145     1. Preserve worktree
146     2. Rollback main branch
147     3. Update queue item status to REJECTED *)
148 let full_rollback ~queue_item ~type_ ~before_merge () =
149   (* Preserve worktree for debugging *)
150   preserve_worktree ~branch:queue_item.Queue_types.branch
151   ~worktree_path:queue_item.Queue_types.worktree_path
152   (Merge_strategies.Error "rollback");
153
154   (* Rollback the merge *)
155   rollback_merge ~type_ ~before_merge ~branch:queue_item.Queue_types.branch
156   (Merge_strategies.Error "rollback")
157
158 (** Check if rollback is possible *)
159 let can_rollback commit =
160   match git_cmd (Printf.sprintf "cat-file -t %s" commit) with
161   | Unix.WEXITED 0, _ -> true
162   | _ -> false
```

61 lib/merge/worktree.ml

```

1 (* Git worktree management for agent isolation
2  *
3  * roerick note (|
4  *   Manages git worktrees for agent runs. Each agent gets an isolated
5  *   worktree at .pi/worktrees/agent-{id}/ to prevent file conflicts.
6  *   Provides create, remove, list, and path resolution functions.
7  * |) *)
8
9 (** Worktree information *)
10 type worktree = {
11   id : string;
12   path : string;
13   branch : string;
14   created_at : string; (* ISO 8601 *)
15 }
16
17 (** Default worktree base directory *)
18 let default_base_dir = ".pi/worktrees"
19
20 (** Ensure base directory exists *)
21 let ensure_base_dir () =
22   let dir = default_base_dir in
23   if not (Sys.file_exists dir) then
24     Unix.mkdir dir 0o755
25
26 (** Generate worktree ID from timestamp and uuid *)
27 let generate_id () =
28   let timestamp =
29     let now = Unix.gmtime (Unix.time ()) in
30     Printf.sprintf "%04d%02d%02d-%02d%02d%02d"
31       (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
32       now.tm_hour now.tm_min now.tm_sec
33   in
34   let uuid =
35     Random.self_init ();
36     Printf.sprintf "%04x" (Random.int 0x10000)
37   in
38   Printf.sprintf "agent-%s-%s" timestamp uuid
39
40 (** Create a new worktree
41  *
42  * Returns Error if:
43  *   - Git is not initialized
44  *   - Branch already exists
45  *   - Worktree path already exists *)
46 let create ~branch ?(base_dir=default_base_dir) () =
47   ensure_base_dir ();
48   let id = generate_id () in
49   let path = Filename.concat base_dir id in
50
51   (* Check if git is initialized *)
52   if not (Sys.file_exists ".git") then
53     Error "Not a git repository"
54   else if Sys.file_exists path then
55     Error (Printf.sprintf "Worktree path already exists: %s" path)
56   else
57     (* Create worktree using git worktree add *)
58     let cmd = Printf.sprintf "git worktree add -b %s %s 2>&1" branch path in
59     let ic = Unix.open_process_in cmd in
60     let output = ref "" in
61     (try
62       while true do
63         (* exempt: input_line *) output := !output ^ input_line ic ^ "\n"
64       done
65     with End_of_file -> ());
66     let status = Unix.close_process_in ic in
67
68     match status with
69     | Unix.WEXITED 0 ->
70       let timestamp =
71         let now = Unix.gmtime (Unix.time ()) in

```



```

72     Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
73     (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
74     now.tm_hour now.tm_min now.tm_sec
75   in
76   Ok { id; path; branch; created_at = timestamp }
77 | _ ->
78   Error (Printf.sprintf "Failed to create worktree: %s" !output)
79
80 (** Remove a worktree
81
82   Cleans up both the worktree directory and the git worktree record.
83   Preserves the branch unless delete_branch is true. *)
84 let remove ?(delete_branch=false) worktree =
85   if not (Sys.file_exists worktree.path) then
86     Error (Printf.sprintf "Worktree does not exist: %s" worktree.path)
87   else
88     (* Remove worktree using git worktree remove *)
89     let cmd = Printf.sprintf "git worktree remove %s 2>&1" worktree.path in
90     let ic = Unix.open_process_in cmd in
91     let output = ref "" in
92     (try
93       while true do
94         output := !output ^ input_line ic ^ "\n"
95       done
96     with End_of_file -> ());
97     let status = Unix.close_process_in ic in
98
99     match status with
100   | Unix.WEXITED 0 ->
101     (if delete_branch then
102       (* Also delete the branch *)
103       let cmd = Printf.sprintf "git branch -D %s 2>&1" worktree.branch
104       in
105       ignore (Sys.command cmd));
106     Ok ()
107 | _ ->
108   Error (Printf.sprintf "Failed to remove worktree: %s" !output)
109
110 (** List all worktrees
111
112   Returns list of borge-managed worktrees (those in .pi/worktrees/) *)
113 let list ?(base_dir=default_base_dir) () =
114   if not (Sys.file_exists base_dir) then []
115   else
116     Sys.readdir base_dir
117     |> Array.to_list
118     |> List.filter (fun name -> String.starts_with ~prefix:"agent-" name)
119     |> List.filter_map (fun id ->
120       let path = Filename.concat base_dir id in
121       if Sys.is_directory path then
122         (* Try to determine branch from git *)
123         let cmd = Printf.sprintf "cd %s && git branch --show-current
124         2>/dev/null" path in
125         let ic = Unix.open_process_in cmd in
126         (try
127           let branch = input_line ic |> String.trim in
128           ignore (Unix.close_process_in ic);
129           Some { id; path; branch; created_at = "unknown" }
130         with End_of_file | Sys_error _ ->
131           ignore (Unix.close_process_in ic);
132           None)
133       else None
134     )
135
136 (** Prune stale worktrees
137
138   Removes worktrees that no longer have valid git refs.
139   Useful for cleanup after interrupted agent runs. *)
140 let prune () =
141   let cmd = "git worktree prune 2>&1" in
142   let status = Sys.command cmd in
143   if status = 0 then Ok () else Error "Failed to prune worktrees"

```

```
143 (** Get worktree path by ID *)
144 let get_path id =
145   Filename.concat default_base_dir id
146
147 (** Check if a worktree exists *)
148 let exists id =
149   Sys.file_exists (get_path id)
150
151 (** Validate worktree is clean
152
153   Returns true if the worktree has no uncommitted changes *)
154 let is_clean worktree =
155   let cmd = Printf.sprintf "cd %s && git status --porcelain 2>/dev/null |
156     wc -l" worktree.path in
157   let ic = Unix.open_process_in cmd in
158   try
159     let line = input_line ic |> String.trim in
160     ignore (Unix.close_process_in ic);
161     line = "0"
162   with End_of_file | Sys_error _ ->
163     ignore (Unix.close_process_in ic);
164     false
```

62 lib/check/check.ml

```

1 open Borge_lang
2
3 type file_result =
4   | Ok of { path : string; project_name : string; form_count : int }
5   | Error of { path : string; message : string }
6
7 type warning =
8   | Orphan of string (** path to an orphaned .borg file *)
9
10 type result = {
11   files : file_result list;
12   warnings : warning list;
13   passed : int;
14   failed : int;
15 }
16
17 (* agent note (|
18  * WHAT: Parse a .borg file and validate its structure.
19  * Returns Ok with project name and form count, or Error with
20  * a descriptive message if parsing fails.
21  * WHY: Used by borge check to validate individual .borg files
22  * and ensure they have valid syntax and a project node.
23  * |) *)
24 let check_file path =
25   let input = File_utils.read_file path in
26   try
27     let file = Parse.parse_file input in
28     match Spec.project_name file with
29     | None ->
30       Error { path; message = "no project node found" }
31     | Some name ->
32       Ok { path; project_name = name; form_count = List.length
33         file.Ast.top_level }
34   with Error.Parse_error e ->
35     Error { path; message = Printf.sprintf "parse error at %d:%d - %s"
36       e.Error.line e.Error.column e.Error.message }
37
38 (* agent note (|
39  * WHAT: Check all .borg files in a directory for validity.
40  * Parses each file, validates structure, counts forms, and
41  * detects orphaned files.
42  * WHY: The main entry point for borge check command, providing
43  * a pass/fail validation of the project's spec files.
44  * |) *)
45 let run dir =
46   let files = File_utils.find_borg_files dir in
47   let results = List.map check_file files in
48   let passed = List.filter (function Ok _ -> true | Error _ -> false)
49     results |> List.length in
50   let failed = List.filter (function Ok _ -> false | Error _ -> true)
51     results |> List.length in
52   (* Orphan detection: files not in any inline tree and no no-inline
53     declaration *)
54   let orphans = Project.find_orphans dir in
55   let warnings = List.map (fun path -> Orphan path) orphans in
56   { files = results; warnings; passed; failed }

```

63 lib/check/code_quality.ml

```

1  (*| Mechanical code quality checker for OCaml source files.
2
3  Scans .ml/.mli files for mechanical anti-patterns:
4  - unsafe stdlib calls (List.hd, List.tl, List.assoc, Hashtbl.find,
5    String.sub, Str.search_forward, int_of_string, float_of_string,
6    open_in, input_line, etc.)
7
8  Regex-based phase: catches unambiguous calls that always warrant
9  attention. Planned AST-based phase for polymorphic equality and
10 deep match nesting.
11
12 Supports (* exempt: FUNC *) comments to suppress warnings on the
13 line where the call appears or the line immediately following.
14 |*)
15
16 type severity = Error | Warning
17
18 type issue = {
19   path : string;
20   line : int;
21   call : string;
22   severity : severity;
23   suggestion : string;
24 }
25
26 type rule = {
27   name : string;
28   pattern : Str.regexp;
29   severity : severity;
30   suggestion : string;
31 }
32
33 (* --- Rules --- *)
34
35 let rules : rule list = [
36   (* Unconditionally unsafe -- will crash on empty input *)
37   { name = "List.hd";
38     pattern = Str.regexp "List.hd[ \t]";
39     severity = Error;
40     suggestion = "Pattern-match on the list instead." };
41   { name = "List.tl";
42     pattern = Str.regexp "List.tl[ \t]";
43     severity = Error;
44     suggestion = "Pattern-match on the list instead." };
45
46   (* Exception-based control flow -- safe only when guarded. Flag all. *)
47   { name = "List.assoc";
48     pattern = Str.regexp "List.assoc[ \t]+[^\t;)]";
49     severity = Warning;
50     suggestion = "Use List.assoc_opt instead." };
51   { name = "Hashtbl.find";
52     pattern = Str.regexp "Hashtbl.find[ \t]+[^\t;)]";
53     severity = Warning;
54     suggestion = "Use Hashtbl.find_opt instead." };
55   { name = "String.sub";
56     pattern = Str.regexp "String.sub[ \t]+[^\t;)]";
57     severity = Warning;
58     suggestion = "Ensure bounds are valid, or use String.sub_opt when
59       available." };
60   { name = "Str.search_forward";
61     pattern = Str.regexp "Str.search_forward[ \t]+[^\t;)]";
62     severity = Warning;
63     suggestion = "Wrap in try/with Not_found or use a safe alternative." };
64   { name = "int_of_string";
65     pattern = Str.regexp "int_of_string[ \t]+[^\t;)]";
66     severity = Warning;
67     suggestion = "Use int_of_string_opt instead." };
68   { name = "float_of_string";
69     pattern = Str.regexp "float_of_string[ \t]+[^\t;)]";
70     severity = Warning;
71     suggestion = "Use float_of_string_opt instead." };

```

```

71 { name = "open_in";
72   pattern = Str.regexp "open_in[ \t]+[^\t;)]";
73   severity = Warning;
74   suggestion = "Use In_channel.with_open_text for automatic close." };
75 { name = "input_line";
76   pattern = Str.regexp "input_line[ \t]+[^\t;)]";
77   severity = Warning;
78   suggestion = "Use In_channel.input_line for safe None-returning
    variant." };
79 ]
80
81 (* --- Exemption support --- *)
82
83 let exempt_re = Str.regexp "\\(\\* exempt:.*/\\*\\)"
84
85 let extract_exemptions line =
86   try
87     (* exempt: Str.search_forward *)
88     ignore (Str.search_forward exempt_re line 0);
89     Str.matched_string line
90   with Not_found -> ""
91
92 let is_exempt_line line =
93   try
94     (* exempt: Str.search_forward *)
95     ignore (Str.search_forward exempt_re line 0);
96     true
97   with Not_found -> false
98
99 let exempts_input_line line =
100   let s = extract_exemptions line in
101   s <> "" && (String.contains s 'i' && String.contains s 'n') (* crude:
    contains "input_line" pattern *)
102
103 (* --- Scanning logic --- *)
104
105 let has_length_guard line =
106   try
107     let sub_pos = (* exempt: Str.search_forward *) Str.search_forward
        (Str.regexp "String\\.sub") line 0 in
108     let before = String.sub line 0 sub_pos in
109     let has_guard_re = Str.regexp "String\\.length\\|> \\|>= \\|try " in
110     ignore ((* exempt: Str.search_forward *) Str.search_forward
        has_guard_re before 0);
111     true
112   with Not_found -> false
113
114 (* Internal length guard: String.sub x (String.length x - N) M --
    self-bounded *)
115 let has_internal_length_guard line =
116   try
117     (* exempt: Str.search_forward *)
118     ignore (Str.search_forward (Str.regexp "String\\.sub.*String\\.length")
        line 0);
119     true
120   with Not_found -> false
121
122 (* Prefix comparison: String.sub x 0 N =|<> "str" -- if length check was on
    previous line *)
123 let is_prefix_comparison line prev =
124   try
125     (* exempt: Str.search_forward *)
126     ignore (Str.search_forward (Str.regexp "String\\.sub .* 0
        [0-9]+.*[<=>].*\\|>= \\|try "
        (* exempt: Str.search_forward *)
127     ignore (Str.search_forward (Str.regexp "< [0-9]+\\|> [0-9]+\\|>=
        [0-9]+") prev 0);
128     true
129   with Not_found -> false
130
131
132 (* Loop-invariant substring: String.sub x !i (!j - !i) *)
133 let is_loop_invariant line =
134   try

```

```

135   (* exempt: Str.search_forward *)
136   ignore (Str.search_forward (Str.regexp
137     "String\\.sub.*![a-z_]+.*(!.*-.*!)" line 0);
137   true
138   with Not_found -> false
139
140 let check_file path lines =
141   let issues = ref [] in
142   let line_num = ref 1 in
143   let prev_exempt = ref false in
144   let block_exempts_input = ref false in
145   let prev_line = ref "" in
146   List.iter (fun line ->
147     (* Block-level exemption: if line exempts both open_in and input_line,
148       carry the exemption through the block until close_in *)
149     if exempts_input_line line then block_exempts_input := true;
150     if String.contains line 'c' && String.contains line 'l' &&
151       String.contains line 'o' && String.contains line 's' &&
152       String.contains line 'e' && String.contains line '_' &&
153       String.contains line 'i' then
154       block_exempts_input := false;
155     let exempt = !prev_exempt || is_exempt_line line ||
156       !block_exempts_input in
157     prev_exempt := is_exempt_line line;
158     if not exempt then begin
159       List.iter (fun (rule : rule) ->
160         try
161           let _ = (* exempt: Str.search_forward *) Str.search_forward
162             rule.pattern line 0 in
163           (* Skip comment lines to reduce false positives in block comments
164             *)
165           if not (String.starts_with ~prefix:"(" (String.trim line)) then
166             (* Skip String.sub calls with obvious guards on same or
167               previous line *)
168             if rule.name = "String.sub" &&
169               (has_length_guard line ||
170                has_length_guard !prev_line ||
171                has_internal_length_guard line ||
172                is_prefix_comparison line !prev_line ||
173                is_loop_invariant line) then
174               ()
175             else
176               issues := { path; line = !line_num; call = rule.name;
177                 severity = rule.severity; suggestion =
178                   rule.suggestion } :: !issues
179               with Not_found -> ()
180             ) rules
181           end;
182           prev_line := line;
183           incr line_num
184         ) lines;
185       List.rev !issues
186
187 (*| Public entry point: scan a directory of .ml/.mli files for mechanical
188   issues. |*)
189 let run dir : issue list =
190   let rec find_files acc path =
191     if not (Sys.file_exists path) then acc
192     else if Sys.is_directory path then
193       if Filename.basename path = "_build" || Filename.basename path =
194         ".git" then
195         acc
196       else
197         Sys.readdir path
198         |> Array.to_list
199         |> List.map (Filename.concat path)
200         |> List.fold_left find_files acc
201     else if Filename.check_suffix path ".ml" || Filename.check_suffix path
202       ".mli" then
203       path :: acc
204     else
205       acc
206   in

```

```
199   let files = find_files [] dir in
200   List.concat_map (fun path ->
201     let lines = File_utils.read_lines path in
202     check_file path lines
203   ) files
```

64 lib/check/drift.ml

```

1 open Borge_lang
2
3 (** Extract position info from a sexp node *)
4 let pos_of_sexp = function
5   | Ast.Atom (p, _) -> p
6   | Ast.String (p, _) -> p
7   | Ast.List (p, _) -> p
8
9 (** Check if string contains a substring *)
10 let contains_sub s substr =
11   try
12     let _ = Str.search_forward (Str.regexp_string substr) s 0 in
13     true
14   with Not_found -> false
15
16 (** {1 Types} *)
17
18 type spec_drift = {
19   path : string;
20   section_name : string;
21   description : string;
22 }
23
24 type code_drift_item = {
25   path : string;
26   kind : string;
27   name : string;
28 }
29
30 type structural_drift_item = {
31   description : string;
32 }
33
34 type drift_result = {
35   spec_drift : spec_drift list;
36   code_drift : code_drift_item list;
37   structural_drift : structural_drift_item list;
38 }
39
40 (** {1 Spec drift: sections claiming implemented features that don't exist}
41   *)
42
43 let rec find_status : Ast.sexp list -> Spec.status option = function
44   | [] -> None
45   | Ast.List (_, Ast.Atom (_, "status") :: Ast.Atom (_, v) :: _) :: _ ->
46     Spec.status_of_string v
47   | _ :: rest -> find_status rest
48
49 (* agent note (|
50  *   WHAT: Extract the doc string from a sexp list.
51  *   Looks for a (doc ...) form and returns content truncated to 60 chars.
52  *   WHY: Used to show section descriptions in drift reports.
53  * |) *)
54
55 (* exempt: String.sub *)
56 let rec find_doc : Ast.sexp list -> string = function
57   | [] -> ""
58   | Ast.List (_, Ast.Atom (_, "doc") :: Ast.String (_, Ast.Verbatim v) ::
59     _) :: _ ->
60     let s = v.Ast.v_content in
61     if String.length s > 60 then String.sub s 0 57 ^ "..." else s
62   | Ast.List (_, Ast.Atom (_, "doc") :: Ast.String (_, Ast.Quoted q) :: _) ::
63     _ ->
64     let s = q.Ast.q_content in
65     if String.length s > 60 then String.sub s 0 57 ^ "..." else s
66   | _ :: rest -> find_doc rest
67
68 (* agent note (|
69  *   WHAT: Hardcoded list of known borge subcommands.
70  *   Distinguishes real commands from user-defined section names.
71  *   WHY: Spec drift detection needs to know if an implemented
72  *   section is a built-in command or custom feature.

```



```

69  * |) *)
70  let known_subcommands = [
71    "balance"; "parse"; "check"; "report"; "fmt"; "nodes"; "inline";
72    "version"; "print"; "lint"; "drift"; "normalize"; "review";
73    "log"; "diff"; "undo";
74    "balance-verbose"; "fmt-check"; "fmt-diff"; "worktree-check";
75  ]
76
77  (* agent note (|
78   * WHAT: Check if a name is a known built-in borge command.
79   * Returns Some true if known, None if not a command.
80   * WHY: Used during spec drift to distinguish real commands
81   * from user-defined sections marked implemented.
82   * |) *)
83  let feature_exists name =
84    if List.mem name known_subcommands then Some true
85    else None
86
87  (* agent note (|
88   * WHAT: Extract all implemented features from a .borg file.
89   * Walks the sexp tree looking for sections/subsections with
90   * (status implemented), returns their names and descriptions.
91   * WHY: Core spec drift detection: finds what the spec claims
92   * is implemented so we can verify it actually exists.
93   * |) *)
94  let find_implemented_features file =
95    let results = ref [] in
96    let rec walk_sexp = function
97      | Ast.List (_, Ast.Atom (_, kind) :: rest_children)
98        when List.mem kind ["subsection"; "section"; "subsubsection"] ->
99        let name = match rest_children with
100          | Ast.Atom (_, n) :: _ -> Some n
101          | _ -> None
102        in
103        let status = find_status rest_children in
104        let doc = find_doc rest_children in
105        (match name, status with
106         | Some n, Some (Spec.Implemented | Spec.Verified) ->
107           results := (n, doc) :: !results
108         | _ -> ());
109        List.iter walk_sexp rest_children
110      | Ast.List (_, children) ->
111        List.iter walk_sexp children
112      | _ -> ()
113    in
114    let rec walk_top = function
115      | [] -> ()
116      | { Ast.node; _ } :: rest -> walk_sexp node; walk_top rest
117    in
118    walk_top file.Ast.top_level;
119    List.rev !results
120
121  (* agent note (|
122   * WHAT: Check if the raw text of a section (from its first line
123   * to the closing of its list form) contains an agent note mentioning
124   * verification or "verified".
125   *
126   * WHY: Implemented sections with (verify ...) stanzas should have
127   * agent notes claiming verification was run. This is a coarse
128   * text check -- it scans for "agent note" and "verified|verif|verify"
129   * in the same block without full AST traversal.
130   * |) *)
131  let section_has_verified_note section_text =
132    let text = String.lowercase_ascii section_text in
133    let has_agent = contains_sub text "agent note" || contains_sub text
134      "agent response" in
135    let has_verify = contains_sub text "verified" || contains_sub text
136      "verif" in
137    has_agent && has_verify
138
139  (** Get raw text for each section from file content.
140   Returns (name, start_line, end_line) triples.**)
141  let extract_section_ranges (file : Ast.file) : (string * int * int) list =

```

```

140 let rec walk_sexp start_line acc = function
141 | Ast.List (_, Ast.Atom (_, kind) :: Ast.Atom (_, name) :: rest)
142 |> when kind = "subsection" || kind = "section" || kind =
143 |> "subsubsection" ->
144 |> let end_line = List.fold_left (fun max_line sexp ->
145 |> max max_line (walk_end sexp)
146 |> ) start_line rest in
147 |> (name, start_line, end_line) :: List.fold_left (walk_child
148 |> start_line) acc rest
149 | Ast.List (_, children) ->
150 |> List.fold_left (walk_child start_line) acc children
151 | _ -> acc
152 and walk_child _ acc node =
153 |> match node with
154 |> | Ast.List (_, Ast.Atom (_, kind) :: Ast.Atom (_, name) :: rest)
155 |> |> when kind = "subsection" || kind = "section" || kind =
156 |> |> "subsubsection" ->
157 |> |> let start_line = (pos_of_sexp node).Ast.line in
158 |> |> let end_line = List.fold_left (fun max_line sexp ->
159 |> |> max max_line (walk_end sexp)
160 |> |> ) start_line rest in
161 |> |> (name, start_line, end_line) :: List.fold_left (walk_child
162 |> |> start_line) acc rest
163 |> | Ast.List (_, children) ->
164 |> |> List.fold_left (walk_child 0) acc children
165 |> | _ -> acc
166 |> and walk_end = function
167 |> | Ast.Atom (p, _) | Ast.String (p, _) -> p.Ast.line
168 |> | Ast.List (p, []) -> p.Ast.line
169 |> | Ast.List (_, children) ->
170 |> |> List.fold_left (fun acc child -> max acc (walk_end child)) 0
171 |> |> children
172 |> in
173 |> let rec walk_top = function
174 |> | [] -> []
175 |> | { Ast.node; _ } :: rest ->
176 |> |> let line = (pos_of_sexp node).Ast.line in
177 |> |> walk_sexp line [] node @ walk_top rest
178 |> in
179 |> walk_top file.top_level
180
181 (** Extract raw text between start_line and end_line from file content. *)
182 let text_between_lines content start_line end_line =
183 |> let lines = String.split_on_char '\n' content in
184 |> let rec take acc curr = function
185 |> | [] -> List.rev acc
186 |> | line :: rest ->
187 |> |> if curr >= start_line && curr <= end_line then
188 |> |> take (line :: acc) (curr + 1) rest
189 |> |> else if curr > end_line then
190 |> |> List.rev acc
191 |> |> else
192 |> |> take acc (curr + 1) rest
193 |> in
194 |> String.concat "\n" (take [] 1 lines)
195
196 (** Find implemented sections with verify but no verified agent note. *)
197 let check_missing_verify_notes path file_content file =
198 |> let mappings = Spec.extract_section_mappings file in
199 |> let impl_with_verify = List.filter (fun (m : Spec.section_mapping) ->
200 |> |> match m.status with
201 |> |> | Some (Spec.Implemented | Spec.Verified) -> m.verify <> []
202 |> |> | _ -> false
203 |> ) mappings in
204 |> if impl_with_verify = [] then []
205 |> else begin
206 |> |> let ranges = extract_section_ranges file in
207 |> |> List.filter_map (fun (m : Spec.section_mapping) ->
208 |> |> |> match List.find_opt (fun (name, _, _) -> name = m.name) ranges with
209 |> |> |> | Some (_, start_line, end_line) ->
210 |> |> |> |> let section_text = text_between_lines file_content start_line
211 |> |> |> |> end_line in
212 |> |> |> |> if section_has_verified_note section_text then None

```

```

207         else Some { path; section_name = m.name;
208           description = "Implemented section with verify stanza has no
           agent verification note" }
209         | None -> None
210       ) impl_with_verify
211     end
212
213   (** Read file content from git HEAD *)
214   let read_git_head path =
215     let cmd = Printf.sprintf "git show HEAD:%s 2>/dev/null" (Filename.quote
       path) in
216     let ic = Unix.open_process_in cmd in
217     let buf = Buffer.create 1024 in
218     (try while true do Buffer.add_string buf (input_line ic ^ "\n") done with
       End_of_file -> ());
219     match Unix.close_process_in ic with
220     | Unix.WEXITED 0 -> Some (Buffer.contents buf)
221     | _ -> None
222
223   (** Compare verify stanzas between current and HEAD.
224     Returns spec_drift items for any existing section whose verify changed.
225     *)
226   let check_verify_drift path file =
227     let current =
228       try Spec.extract_section_mappings file
229       with _ -> []
230     in
231     let head = match read_git_head path with
232       | Some content ->
233         (try
234           let head_file = Parse.parse_file content in
235           Spec.extract_section_mappings head_file
236           with _ -> [])
237       | None -> []
238     in
239     let head_map = Hashtbl.create 16 in
240     List.iter (fun (m : Spec.section_mapping) ->
       Hashtbl.replace head_map m.name m.verify
241     ) head;
242     List.filter_map (fun (m : Spec.section_mapping) ->
243       let head_verify = try Hashtbl.find head_map m.name with Not_found -> []
244       in
245       if head_verify = [] then None (* section didn't exist at HEAD *)
246       else if head_verify <> m.verify then
247         Some { path; section_name = m.name;
248           description = "Verify stanza was modified since HEAD" }
249       else None
250     ) current
251
252   (* agent note (|
253    * WHAT: Check for spec drift in a directory.
254    * Finds all .borg files, extracts implemented features,
255    * and reports any that don't correspond to real commands.
256    * Also checks: verify stanzas modified since HEAD, and
257    * implemented sections with verify missing agent notes.
258    * WHY: One of the three drift detection modes in borge.
259    * |) *)
260   let check_spec_drift dir =
261     let tree_result =
262       let roots = Project.find_roots dir in
263       match roots with
264       | [] -> File_utils.find_borg_files dir
265       | root :: _ ->
266         (match Project.build_tree root with
267         | Ok tree -> Project.tree_paths tree
268         | Error _ -> File_utils.find_borg_files dir)
269     in
270     let basic_drift = List.concat_map (fun path ->
271       try
272         let input = File_utils.read_file path in
273         let file = Parse.parse_file input in
274         let sections = find_implemented_features file in
275         let missing_notes = check_missing_verify_notes path input file in

```

```

275     let verify_changed = check_verify_drift path file in
276     let feature_check = List.filter_map (fun (name, desc) ->
277       match feature_exists name with
278       | Some false ->
279         Some { path; section_name = name; description = desc }
280       | Some true | None -> None
281     ) sections in
282     feature_check @ missing_notes @ verify_changed
283   with _ -> []
284 ) tree_result in
285 basic_drift
286
287 (** {1 Dune-aware code drift} *)
288
289 (** Build a set of module names covered by .borg sections.
290     Uses explicit (implements ...) declarations when present,
291     falls back to convention-derived name normalization. *)
292 let build_covered_modules borg_files =
293   let names = Hashtbl.create 16 in
294   List.iter (fun path ->
295     try
296       let input = File_utils.read_file path in
297       let file = Parse.parse_file input in
298       (* Use the new extract_section_mappings which handles both
299          explicit (implements ...) and implicit section names *)
300       let mappings = Spec.extract_section_mappings file in
301       List.iter (fun (m : Spec.section_mapping) ->
302         if m.implements <> [] then
303           (* Explicit mapping: (implements lib/ui/ui_dream.ml) ? derive
304              module name from filename *)
305           List.iter (fun impl_path ->
306             try
307               let base = Filename.basename impl_path in
308               let mod_name = String.capitalize_ascii
309                 (Filename.remove_extension base) in
310               Hashtbl.replace names mod_name true
311             with _ -> ()
312           ) m.implements
313         else
314           (* Convention-derived: section "ui-dream" ? "Ui_dream" *)
315           let normalized = Convention.normalize_section_name m.name in
316           let mod_name = String.capitalize_ascii normalized in
317           Hashtbl.replace names mod_name true
318         ) mappings
319       with _ -> ()
320     ) borg_files;
321   names
322
323 (** Find .ml files recursively *)
324 let find_ml_files dir =
325   let rec find path =
326     try
327       let entries = Sys.readdir path in
328       Array.fold_left (fun acc entry ->
329         if entry = "_build" || entry = ".git" then acc
330         else
331           let full = Filename.concat path entry in
332           if Sys.is_directory full then find full @ acc
333           else if Filename.check_suffix entry ".ml" then full :: acc
334           else acc
335       ) [] entries
336     with Sys_error _ -> []
337   in
338   List.sort String.compare (find dir)
339
340 (** Cross-reference dune modules with .borg sections *)
341 let check_code_drift dir =
342   let borg_files = File_utils.find_borg_files dir in
343   let covered = build_covered_modules borg_files in
344   let dune_files = Dune_parse.parse_all dir in
345   (* Collect all lib modules from dune *)
346   let lib_modules = List.concat_map (fun df ->
347     List.concat_map (function

```

```

347     | Dune_parse.Library lib ->
348       List.map (fun m -> (lib.Dune_parse.name, m))
349       lib.Dune_parse.modules
350     | _ -> []
351   ) df.Dune_parse.stanzas
352 let unspecified = List.filter_map (fun (lib_name, mod_name) ->
353   let capitalized = String.capitalize_ascii mod_name in
354   if Hashtbl.mem covered capitalized then None
355   else Some { path = Printf.sprintf "lib/%s/%s.ml" lib_name mod_name;
356     kind = "unspecified-module"; name = capitalized }
357 ) lib_modules in
358 unspecified
359
360 (** {1 Structural drift} *)
361
362 let check_structural_drift _dir =
363   (* Domain logic in bin/ check is now covered by dune-aware analysis *)
364   []
365
366 (** {1 .borg.meta generation} *)
367
368 (** Generate findings from drift analysis *)
369 let generate_findings dir =
370   let timestamp = Meta.current_timestamp () in
371   let borg_files = File_utils.find_borg_files dir in
372   let dune_file = Dune_parse.parse_all dir in
373
374   (* Collect module surfaces for lib modules only *)
375   let lib_dune_files = List.filter (fun df ->
376     let dir = Filename.dirname df.Dune_parse.path in
377     String.length dir >= 5 && String.sub dir 0 5 = "./lib"
378   ) dune_file in
379   let lib_modules = List.concat_map (fun df ->
380     let dir = Filename.dirname df.Dune_parse.path in
381     List.concat_map (function
382       | Dune_parse.Library lib ->
383         List.map (fun m -> (dir, m)) lib.Dune_parse.modules
384       | _ -> []
385     ) df.Dune_parse.stanzas
386   ) lib_dune_files in
387   let all_surfaces = List.filter_map (fun (dir, mod_name) ->
388     let ml_path = Filename.concat dir (mod_name ^ ".ml") in
389     if Sys.file_exists ml_path then
390       Some (Surface.extract_surface ml_path)
391     else None
392   ) lib_modules in
393
394   (* Build dune snapshot *)
395   let dune_snapshot = List.concat_map (fun df ->
396     List.filter_map (function
397       | Dune_parse.Library lib ->
398         Some {
399           Meta.name = lib.Dune_parse.name;
400           Meta.modules = lib.Dune_parse.modules;
401           Meta.public_name = lib.Dune_parse.public_name;
402           Meta.libraries = lib.Dune_parse.libraries;
403         }
404       | _ -> None
405     ) df.Dune_parse.stanzas
406   ) dune_file in
407
408   (* Cross-reference: modules in dune but not in .borg *)
409   let covered = build_covered_modules borg_files in
410   let unspecified_findings = List.filter_map (fun (surface :
411     Surface.module_surface) ->
412     if Hashtbl.mem covered surface.module_name then None
413     else Some {
414       Meta.ft_type = Unspecified_module;
415       Meta.section = None;
416       Meta.module_ = Some surface.module_name;
417       Meta.file = Some surface.path;
418       Meta.export = None;

```

```

418     Meta.detail = Some (Printf.sprintf "module %s not mentioned in any
.borg spec" surface.module_name);
419     Meta.spec_status = None;
420     Meta.actual_status = None;
421     Meta.confidence = High;
422     Meta.source = Static;
423     Meta.at = timestamp;
424   }
425 ) all_surfaces in
426
427 (* Cross-reference: exports in module but not in spec *)
428 (* For each surface, check if all exports are described somewhere *)
429 let export_findings =
430   let surfaces_with_spec = List.filter (fun (surface :
Surface.module_surface) ->
431     Hashtbl.mem covered_surface.module_name
432   ) all_surfaces in
433   (* For now, just note modules with large surface area vs spec coverage
*)
434   List.filter_map (fun (surface : Surface.module_surface) ->
435     (* Heuristic: if a module has more than 10 exports and the spec
436       section exists but may not describe all of them, flag as
437       potential extra-export. This is a low-confidence finding. *)
438     if List.length surface.exports > 15 then
439       Some {
440         Meta.ft_type = Extra_export;
441         Meta.section = None;
442         Meta.module_ = Some surface.module_name;
443         Meta.file = Some surface.path;
444         Meta.export = None;
445         Meta.detail = Some (Printf.sprintf
446           "module %s has %d exports -- may not all be covered by spec"
447           surface.module_name (List.length surface.exports));
448         Meta.spec_status = None;
449         Meta.actual_status = None;
450         Meta.confidence = Low;
451         Meta.source = Static;
452         Meta.at = timestamp;
453       }
454     else None
455   ) surfaces_with_spec
456 in
457
458 (* Collect content hashes for staleness detection *)
459 let content_hashes = List.map (fun path ->
460   let hash = Meta.hash_file path in
461   (path, hash)
462 ) borg_files in
463
464 (* Write .borg.meta for the root borg file *)
465 let findings = Meta.sort_findings (unspecified_findings @
export_findings) in
466 let meta_surfaces = List.map (fun (s : Surface.module_surface) ->
467   { Meta.module_name = s.module_name; Meta.file = s.path; Meta.exports =
s.exports }
468 ) all_surfaces in
469
470 (* Generate the root meta file *)
471 let meta : Meta.meta = {
472   Meta.project_name = "borge";
473   Meta.analyzed_at = timestamp;
474   Meta.content_hashes = content_hashes;
475   Meta.module_surfaces = meta_surfaces;
476   Meta.dune_snapshot = dune_snapshot;
477   Meta.findings = findings;
478 } in
479 meta
480
481 (** Write .borg.meta files for all .borg files in the project *)
482 let write_meta_files dir =
483   let meta = generate_findings dir in
484   let borg_files = File_utils.find_borg_files dir in
485   List.iter (fun borg_path ->

```

```

486   let meta_path = Meta.meta_path_of borg_path in
487   (* Only write to the root .borg file's meta for now *)
488   (* Per-file meta would require splitting findings by file *)
489   if Filename.basename borg_path = "borge.borg" then begin
490     let project_name = try
491       let input = File_utils.read_file borg_path in
492       let file = Parse.parse_file input in
493       Spec.project_name file
494     with _ -> Some "borge"
495     in
496     let per_meta = { meta with Meta.project_name = Option.value
497       project_name ~default:"borge" } in
498     Meta.write_meta meta_path per_meta
499   end
500   ) borg_files;
501   meta
502 (** {1 Go-aware code drift} *)
503
504 (* agent note (|
505  *   WHAT: Check for code drift in a Go project by discovering
506  *   packages from directory structure and cross-referencing
507  *   against .borg spec sections.
508  *
509  *   WHY: Go projects don't have dune files -- packages are
510  *   discovered from the directory layout and go.mod.
511  * |) *)
512 let check_go_code_drift dir =
513   let borg_files = File_utils.find_borg_files dir in
514   let covered = build_covered_modules borg_files in
515   let project = Go_parse.parse_all dir in
516   (* Collect all library packages (not cmd executables) *)
517   let lib_packages = Go_parse.library_packages project in
518   let unspecified = List.filter_map (fun (pkg : Go_parse.go_package) ->
519     let capitalized = String.capitalize_ascii pkg.Go_parse.package_name in
520     if Hashtbl.mem covered capitalized then None
521     else Some { path = pkg.Go_parse.dir_path;
522       kind = "unspecified-package";
523       name = capitalized }
524   ) lib_packages in
525   unspecified
526
527 (* agent note (|
528  *   WHAT: Generate findings from drift analysis for a Go project.
529  *   Uses Go_parse for package discovery and Go_surface for
530  *   exported symbol extraction.
531  *
532  *   WHY: The meta generation pipeline needs Go-aware equivalents
533  *   of the dune-aware code in generate_findings.
534  * |) *)
535 let generate_go_findings dir =
536   let timestamp = Meta.current_timestamp () in
537   let borg_files = File_utils.find_borg_files dir in
538   let project = Go_parse.parse_all dir in
539
540   (* Collect package surfaces for library packages *)
541   let lib_packages = Go_parse.library_packages project in
542   let all_surfaces = List.filter_map (fun (pkg : Go_parse.go_package) ->
543     if Sys.is_directory pkg.Go_parse.dir_path then
544       Some (Go_surface.extract_package_surface pkg.Go_parse.dir_path)
545     else None
546   ) lib_packages in
547
548   (* Build package snapshot (analogous to dune_snapshot) *)
549   let package_snapshot = List.filter_map (fun (pkg : Go_parse.go_package) ->
550     let surface = Go_surface.extract_package_surface pkg.Go_parse.dir_path
551     in
552     let _exports = Go_surface.exported_names surface in
553     Some {
554       Meta.name = pkg.Go_parse.package_name;
555       Meta.modules = List.map (fun f -> Filename.chop_extension f)
556         pkg.Go_parse.go_files;
557       Meta.public_name = (match pkg.Go_parse.kind with

```

```

556         | Go_parse.Pkg -> Some pkg.Go_parse.package_name
557         | _ -> None);
558     Meta.libraries = [];
559 }
560 ) lib_packages in
561
562 (* Cross-reference: packages in go.mod but not in .borg *)
563 let covered = build_covered_modules borg_files in
564 let unspecified_findings = List.filter_map (fun (surface :
565     Go_surface.go_package_surface) ->
566     if Hashtbl.mem covered (String.capitalize_ascii
567         surface.Go_surface.package_name) then None
568     else Some {
569         Meta.ft_type = Unspecified_module;
570         Meta.section = None;
571         Meta.module_ = Some surface.Go_surface.package_name;
572         Meta.file = Some surface.Go_surface.path;
573         Meta.export = None;
574         Meta.detail = Some (Printf.sprintf "package %s not mentioned in any
575     .borg spec" surface.Go_surface.package_name);
576         Meta.spec_status = None;
577         Meta.actual_status = None;
578         Meta.confidence = High;
579         Meta.source = Static;
580         Meta.at = timestamp;
581     })
582 ) all_surfaces in
583
584 (* Cross-reference: exports in package but not in spec *)
585 let export_findings =
586     let surfaces_with_spec = List.filter (fun (surface :
587         Go_surface.go_package_surface) ->
588         Hashtbl.mem covered (String.capitalize_ascii
589             surface.Go_surface.package_name)
590     ) all_surfaces in
591     List.filter_map (fun (surface : Go_surface.go_package_surface) ->
592         let exported = Go_surface.exported_names surface in
593         if List.length exported > 15 then
594             Some {
595                 Meta.ft_type = Extra_export;
596                 Meta.section = None;
597                 Meta.module_ = Some surface.Go_surface.package_name;
598                 Meta.file = Some surface.Go_surface.path;
599                 Meta.export = None;
600                 Meta.detail = Some (Printf.sprintf
601                     "package %s has %d exports -- may not all be covered by spec"
602                     surface.Go_surface.package_name (List.length exported));
603                 Meta.spec_status = None;
604                 Meta.actual_status = None;
605                 Meta.confidence = Low;
606                 Meta.source = Static;
607                 Meta.at = timestamp;
608             }
609         else None
610     ) surfaces_with_spec
611 in
612
613 (* Collect content hashes for staleness detection *)
614 let content_hashes = List.map (fun path ->
615     let hash = Meta.hash_file path in
616     (path, hash)
617 ) borg_files in
618
619 let findings = Meta.sort_findings (unspecified_findings @
620     export_findings) in
621 let meta_surfaces = List.map (fun (s : Go_surface.go_package_surface) ->
622     let exports = Go_surface.exported_names s in
623     { Meta.module_name = s.Go_surface.package_name; Meta.file =
624         s.Go_surface.path; Meta.exports = exports }
625 ) all_surfaces in
626
627 let project_name = match project.Go_parse.go_mod with
628     | Some m -> m.Go_parse.module_path

```



```

622   | None -> "go-project"
623 in
624 let meta : Meta.meta = {
625   Meta.project_name = project_name;
626   Meta.analyzed_at = timestamp;
627   Meta.content_hashes = content_hashes;
628   Meta.module_surfaces = meta_surfaces;
629   Meta.dune_snapshot = package_snapshot;
630   Meta.findings = findings;
631 } in
632 meta
633
634 (** Write .borg.meta files for Go projects *)
635 let write_go_meta_files dir =
636   let meta = generate_go_findings dir in
637   let borg_files = File_utils.find_borg_files dir in
638   List.iter (fun borg_path ->
639     let meta_path = Meta.meta_path_of borg_path in
640     let project_name = try
641       let input = File_utils.read_file borg_path in
642       let file = Parse.parse_file input in
643       Spec.project_name file
644     with _ -> Some "go-project"
645     in
646     let per_meta = { meta with Meta.project_name = Option.value
647       project_name ~default:"go-project" } in
648     Meta.write_meta meta_path per_meta
649   ) borg_files;
650   meta
651
652 (** {1 Main entry point with convention dispatch} *)
653 let run dir =
654   let spec = check_spec_drift dir in
655   let convention = Convention.resolve dir in
656   let code = match convention with
657   | Convention.Go_standard -> check_go_code_drift dir
658   | Convention.Ocaml_dune -> check_code_drift dir
659   in
660   let structural = check_structural_drift dir in
661   { spec_drift = spec; code_drift = code; structural_drift = structural }

```

65 lib/check/dune_parse.ml

```

1 open Borge_lang
2
3 type dune_library = {
4   name : string;
5   public_name : string option;
6   modules : string list;
7   libraries : string list;
8 }
9
10 type dune_executable = {
11   names : string list;
12   public_names : string list;
13   libraries : string list;
14 }
15
16 type dune_test = {
17   name : string;
18   libraries : string list;
19 }
20
21 type dune_stanza =
22   | Library of dune_library
23   | Executables of dune_executable
24   | Test of dune_test
25
26 type dune_file = {
27   path : string;
28   stanzas : dune_stanza list;
29 }
30
31 (* agent note (|
32  *   WHAT: Dummy position value used when constructing AST nodes
33  *   that don't have real source locations.
34  *   WHY: Needed when building synthetic sexps from dune file parsing
35  *   where we don't have real line/column info.
36  * |) *)
37 let dummy_pos = { Ast.line = 0; col = 0; offset = 0 }
38
39 (** Extract a string list from a sexp list like (modules a b c) *)
40 let extract_string_list = function
41   | Ast.List (_, items) ->
42     List.filter_map (function
43       | Ast.Atom (_, s) -> Some s
44       | _ -> None
45     ) items
46   | _ -> []
47
48 (** Extract an optional string value from (key value) *)
49 let extract_optional_string key children =
50   List.find_map (function
51     | Ast.List (_, Ast.Atom (_, k) :: Ast.Atom (_, v) :: _)
52       when k = key -> Some v
53     | _ -> None
54   ) children
55
56 (** Extract a required string value from (key value) *)
57 let extract_string key children =
58   extract_optional_string key children
59
60 (** Parse a (library ...) stanza *)
61 let parse_library children =
62   let name = match extract_string "name" children with
63     | Some n -> n
64     | None -> ""
65   in
66   let public_name = extract_optional_string "public_name" children in
67   let modules = List.find_map (function
68     | Ast.List (_, Ast.Atom (_, "modules") :: rest) ->
69       Some (extract_string_list (Ast.List (dummy_pos, rest)))
70     | _ -> None
71   ) children |> function Some m -> m | None -> [] in

```

```

72   let libraries = List.find_map (function
73     | Ast.List (_, Ast.Atom (_, "libraries") :: rest) ->
74       Some (extract_string_list (Ast.List (dummy_pos, rest)))
75     | _ -> None
76   ) children |> function Some l -> l | None -> [] in
77   { name; public_name; modules; libraries }
78
79 (** Parse an (executables ...) or (executable ...) stanza *)
80 let parse_executables children =
81   let names = List.find_map (function
82     | Ast.List (_, Ast.Atom (_, "name") :: Ast.Atom (_, n) :: _) ->
83       Some [n]
84     | Ast.List (_, Ast.Atom (_, "names") :: rest) ->
85       Some (extract_string_list (Ast.List (dummy_pos, rest)))
86     | _ -> None
87   ) children |> function Some n -> n | None -> [] in
88   let public_names = List.find_map (function
89     | Ast.List (_, Ast.Atom (_, "public_name") :: Ast.Atom (_, n) :: _) ->
90       Some [n]
91     | Ast.List (_, Ast.Atom (_, "public_names") :: rest) ->
92       Some (extract_string_list (Ast.List (dummy_pos, rest)))
93     | _ -> None
94   ) children |> function Some n -> n | None -> [] in
95   let libraries = List.find_map (function
96     | Ast.List (_, Ast.Atom (_, "libraries") :: rest) ->
97       Some (extract_string_list (Ast.List (dummy_pos, rest)))
98     | _ -> None
99   ) children |> function Some l -> l | None -> [] in
100  { names; public_names; libraries }
101
102 (** Parse a (test ...) stanza *)
103 let parse_test children =
104   let name = match extract_string "name" children with
105     | Some n -> n
106     | None -> ""
107   in
108   let libraries = List.find_map (function
109     | Ast.List (_, Ast.Atom (_, "libraries") :: rest) ->
110       Some (extract_string_list (Ast.List (dummy_pos, rest)))
111     | _ -> None
112   ) children |> function Some l -> l | None -> [] in
113   { name; libraries }
114
115 (** Parse a single dune file *)
116 let parse_dune_file path =
117   let input = File_utils.read_file path in
118   let file = Parse.parse_file input in
119   let stanzas = List.filter_map (fun { Ast.node; _ } ->
120     match node with
121     | Ast.List (_, Ast.Atom (_, "library") :: children) ->
122       Some (Library (parse_library children))
123     | Ast.List (_, Ast.Atom (_, "executable") :: children)
124     | Ast.List (_, Ast.Atom (_, "executables") :: children) ->
125       Some (Executables (parse_executables children))
126     | Ast.List (_, Ast.Atom (_, "test") :: children) ->
127       Some (Test (parse_test children))
128     | _ -> None
129   ) file.Ast.top_level in
130   { path; stanzas }
131
132 (** Find all dune files recursively *)
133 let find_dune_files dir =
134   let rec find path =
135     try
136       let entries = Sys.readdir path in
137       Array.fold_left (fun acc entry ->
138         if entry = "_build" || entry = ".git" then acc
139         else
140           let full = Filename.concat path entry in
141           if Sys.is_directory full then find full @ acc
142           else if entry = "dune" then full :: acc
143           else acc
144       ) [] entries

```

```
145   with Sys_error _ -> []
146   in
147   List.sort String.compare (find_dir)
148
149   (** Parse all dune files in a project *)
150   let parse_all dir =
151     List.map parse_dune_file (find_dune_files dir)
152
153   (** Get all library names across all dune files *)
154   let all_library_names dune_files =
155     List.concat_map (fun { stanzas; _ } ->
156       List.filter_map (function
157         | Library lib -> Some lib.name
158         | _ -> None
159       ) stanzas
160     ) dune_files
161
162   (** Get all modules in a specific library *)
163   let modules_in_library dune_files lib_name =
164     List.find_map (fun { stanzas; _ } ->
165       List.find_map (function
166         | Library lib when lib.name = lib_name -> Some lib.modules
167         | _ -> None
168       ) stanzas
169     ) dune_files |> function Some m -> m | None -> []
170
171   (** Get all modules across all libraries *)
172   let all_modules dune_files =
173     List.concat_map (fun { stanzas; _ } ->
174       List.concat_map (function
175         | Library lib -> List.map (fun m -> (lib.name, m)) lib.modules
176         | _ -> []
177       ) stanzas
178     ) dune_files
```

66 lib/check/lint.ml

```

1 open Borge_lang
2
3 type lint_issue =
4   | Invalid_status of { path : string; value : string; line : int; col :
5     int }
6   | Duplicate_project_name of { name : string; paths : string list }
7   | No_inline_on_root of { path : string }
8   | Unknown_comment_type of { path : string; type_name : string; line : int
9     }
10  | Missing_comment_value of { path : string; author : string; type_name :
11    string option; line : int }
12  | Deleted_human_comment of { path : string; author : string; line : int }
13  | Inserted_human_comment of { path : string; author : string; line : int }
14  | Pending_response of { path : string; author : string; line : int;
15    question : string }
16  | Orphaned_file of { path : string }
17  | Invalid_verify_method of { path : string; method_ : string; line : int }
18  | Db_validation of { path : string; severity : [ 'Error | 'Warning ];
19    message : string }
20  | Ui_validation of { path : string; severity : [ 'Error | 'Warning ];
21    message : string }
22  | Undocumented_binding of { path : string; name : string; line : int }
23  | Stale_doc_comment of { path : string; name : string; line : int }
24  | Unsafe_call of { path : string; line : int; call : string; severity : [
25    'Error | 'Warning ]; suggestion : string }
26  | Missing_literacy_score of { path : string; name : string; line : int }
27  | Implausible_literacy_score of { path : string; name : string; line :
28    int; dimension : string; claimed : int; reason : string }
29
30 type lint_result = {
31   issues : lint_issue list;
32   error_count : int;
33   warning_count : int;
34 }
35
36 (* agent note (|
37  * WHAT: Extract position info from a sexp node.
38  * Returns the position regardless of whether node is Atom, String, or
39  * List.
40  * WHY: Used throughout lint to report line/column of issues.
41  * |) *)
42
43 let pos_of_sexp = function
44   | Ast.Atom (p, _) -> p
45   | Ast.String (p, _) -> p
46   | Ast.List (p, _) -> p
47
48 (** Check for invalid status values *)
49 let check_invalid_statuses path file =
50   let issues = ref [] in
51   Ast.iter_sexps file (function
52     | Ast.List (_, Ast.Atom (_, "status")) :: Ast.Atom (p, v) :: _ ->
53       (match Spec.status_of_string v with
54        | Some _ -> ()
55        | None -> issues := Invalid_status { path; value = v; line =
56          p.Ast.line; col = p.Ast.col } :: !issues)
57     | _ -> ()
58   );
59   List.rev !issues
60
61 (** Check for duplicate project names across files *)
62 let check_duplicate_names file_stats =
63   let names = List.filter_map (fun (path, name) ->
64     match name with Some n -> Some (n, path) | None -> None
65   ) file_stats in
66   let grouped = List.sort (fun (a, _) (b, _) -> String.compare a b) names in
67   let rec find_dups = function
68     | [] | [_] -> []
69     | (n1, p1) :: (n2, p2) :: rest when n1 = n2 ->
70       let paths = p1 :: p2 :: List.filter_map (fun (n, p) -> if n = n1
71         then Some p else None) rest in

```

```

60     Duplicate_project_name { name = n1; paths } :: find_dups
    (List.filter (fun (n, _) -> n <> n1) rest)
61 | _ :: rest -> find_dups rest
62 in
63 find_dups grouped
64
65 (* agent note (|
66  * WHAT: Extract author name from an annotated comment.
67  * Handles both single author (Single) and multiple authors (Multiple).
68  * WHY: Used to identify who wrote a comment that has issues.
69  * |) *)
70 let authorship_string = function
71 | Ast.Single a -> a
72 | Ast.Multiple auths -> String.concat " " auths
73
74 (** Check annotated comments for unknown types and missing values *)
75 let check_annotated_comments path file =
76   let issues = ref [] in
77   Ast.iter_comments file (function
78   | Ast.Annotated ac ->
79     let valid_types = ["note"; "design"; "ask"; "response"; "todo"] in
80     (match ac.Ast.comment_type with
81     | Ast.Typed t when not (List.mem t valid_types) ->
82       issues := Unknown_comment_type { path; type_name = t; line =
83         ac.Ast.start_line } :: !issues
84     | _ -> ());
85     (match ac.Ast.comment_type, ac.Ast.value with
86     | Ast.Typed _, None ->
87       issues := Missing_comment_value {
88         path; author = authorship_string ac.Ast.authorship;
89         type_name = (match ac.Ast.comment_type with Ast.Typed t ->
90           Some t | _ -> None);
91         line = ac.Ast.start_line
92       } :: !issues
93     | _ -> ());
94   | _ -> ());
95   List.rev !issues
96
97 (** Check for pending ask comments with empty response slots *)
98 (* exempt: String.sub *)
99 let check_pending_responses path file =
100   let asks = ref [] in
101   let responses = ref [] in
102   Ast.iter_comments file (function
103   | Ast.Annotated ac ->
104     (match ac.Ast.comment_type with
105     | Ast.Typed "ask" -> asks := ac :: !asks
106     | Ast.Typed "response" -> responses := ac :: !responses
107     | _ -> ());
108   | _ -> ());
109   (* An ask is "pending" if it has no response after it *)
110   List.filter_map (fun ask ->
111     let has_response = List.exists (fun resp ->
112       resp.Ast.start_line > ask.Ast.start_line && resp.Ast.start_line <=
113       ask.Ast.end_line + 5
114     ) !responses in
115     if has_response then None
116     else
117       let question = match ask.Ast.value with
118       | Some (Ast.Verbatim v) ->
119         let s = v.Ast.v_content in
120         if String.length s > 40 then String.sub s 0 37 ^ "..." else s
121       | Some (Ast.Quoted q) ->
122         let s = q.Ast.q_content in
123         if String.length s > 40 then String.sub s 0 37 ^ "..." else s
124       | None -> "?"
125       in
126       Some (Pending_response {
127         path;
128         author = authorship_string ask.Ast.authorship;
129         line = ask.Ast.start_line;

```

```

129     question
130   })
131 ) !asks
132
133 (** Check for human-authored comment deletions against git HEAD *)
134 let check_comment_integrity path =
135   let cmd = Printf.sprintf "git diff HEAD -- %s 2>/dev/null"
136     (Filename.quote path) in
137   let ic = Unix.open_process_in cmd in
138   let lines = ref [] in
139   (try while true do lines := input_line ic :: !lines done with End_of_file
140     -> ());
141   let _status = Unix.close_process_in ic in
142   let issues = ref [] in
143   List.iter (fun line ->
144     if String.length line > 0 && line.[0] = '-' then begin
145       let content = String.sub line 1 (String.length line - 1) in
146       if String.length content > 3 && String.sub content 0 3 = "( * " then
147         begin
148           let rest = String.sub content 3 (String.length content - 3) in
149           let space_idx = try String.index rest ' ' with Not_found -> -1 in
150           if space_idx > 0 then begin
151             let author = (* exempt: String.sub *) String.sub rest 0 space_idx
152             in
153             if author <> "agent" && author <> "bot" then
154               issues := Deleted_human_comment { path; author; line = 0 } ::
155                 !issues
156             end
157           end
158         end
159       ) (List.rev !lines);
160     (* Also detect added human comments in git diff *)
161     List.iter (fun line ->
162       if String.length line > 0 && line.[0] = '+' then begin
163         let content = String.sub line 1 (String.length line - 1) in
164         if String.length content > 3 && String.sub content 0 3 = "( * " then
165           begin
166             let rest = String.sub content 3 (String.length content - 3) in
167             let space_idx = try String.index rest ' ' with Not_found -> -1 in
168             if space_idx > 0 then begin
169               let author = (* exempt: String.sub *) String.sub rest 0 space_idx
170               in
171               if author <> "agent" && author <> "bot" then
172                 issues := Inserted_human_comment { path; author; line = 0 } ::
173                   !issues
174               end
175             end
176           end
177         ) (List.rev !lines);
178       !issues
179     )
180   ) (** Check DB forms inside .borg files using Db_validate *)
181 let check_db_specs path file =
182   (* Try to parse as DB spec *)
183   let db_app = Db_parse.parse_file file in
184   match db_app with
185   | None -> [] (* Not a DB file, or no (db ...) forms *)
186   | Some app ->
187     let issues = Db_validate.validate app in
188     List.map (fun (i : Db_validate.issue) ->
189       Db_validation {
190         path;
191         severity = i.severity;
192         message = i.message;
193       }
194     ) issues
195   ) (** Check UI forms inside .borg files using Ui_validate *)
196 let check_ui_specs path file =
197   let ui_app = Ui_parse.parse_file file in
198   match ui_app with
199   | None -> [] (* Not a UI file, or no (ui ...) forms *)
200   | Some app ->

```

```

194   let issues = Ui_validate.validate app in
195   List.map (fun (i : Ui_validate.issue) ->
196     Ui_validation {
197       path;
198       severity = i.severity;
199       message = i.message;
200     }
201   ) issues
202
203 (** Check .ml files for undocumented exported bindings.
204   Uses Doc_extract to find bindings and Doc_detect to check for
205   doc comments. Reports Undocumented_binding for each exported
206   binding without documentation or exemption, and Stale_doc_comment
207   for bindings with (status drifted) markers. *)
208 let check_code_doc dir =
209   let ml_files = Doc_extract.find_ml_files dir in
210   let issues = ref [] in
211   List.iter (fun path ->
212     let bindings = Doc_extract.extract_bindings path in
213     let docs = Doc_detect.extract_file_docs path in
214     List.iter (fun (b : Doc_extract.binding_info) ->
215       if not b.is_internal && not b.is_test then begin
216         let doc = List.find_opt (fun (bd : Doc_detect.binding_doc) ->
217           bd.binding_line = b.line
218         ) docs in
219         match doc with
220         | None ->
221           issues := Undocumented_binding { path; name = b.name; line =
222             b.line } :: !issues
223         | Some bd ->
224           if Doc_detect.binding_is_drifted bd then
225             issues := Stale_doc_comment { path; name = b.name; line =
226               b.line } :: !issues
227           else if not (Doc_detect.binding_has_doc bd) && not
228             (Doc_detect.binding_is_exempt bd) then
229             issues := Undocumented_binding { path; name = b.name; line =
230               b.line } :: !issues
231           end
232       ) bindings
233     ) ml_files;
234   List.rev !issues
235
236 (* agent note (|
237   * WHAT: Check .ml files for literacy score presence and plausibility.
238   * Agent-written comments must have [NNNN] scores.
239   * High scores must be supported by content.
240   * |) *)
241 let contains_sub s substr =
242   try
243     let _ = Str.search_forward (Str.regexp_string substr) s 0 in
244     true
245   with Not_found -> false
246
247 let check_literacy dir =
248   let ml_files = Doc_extract.find_ml_files dir in
249   let issues = ref [] in
250   List.iter (fun path ->
251     let bindings = Doc_extract.extract_bindings path in
252     let docs = Doc_detect.extract_file_docs path in
253     List.iter (fun (b : Doc_extract.binding_info) ->
254       if not b.is_internal && not b.is_test then begin
255         let doc = List.find_opt (fun (bd : Doc_detect.binding_doc) ->
256           bd.binding_line = b.line
257         ) docs in
258         match doc with
259         | Some bd when Doc_detect.binding_has_doc bd -> begin
260           match bd.doc with
261           | Some (Doc_detect.Borg_note (_, _content, None), _) ->
262             issues := Missing_literacy_score { path; name = b.name;
263               line = b.line } :: !issues
264           | Some (Doc_detect.Borg_note (_, content, Some score), _) ->
265             let content_lower = String.lowercase_ascii content in
266             let has_narrative =

```



```

262         String.length content > 200 ||
263         contains_sub content_lower "because" ||
264         contains_sub content_lower "this is why" ||
265         contains_sub content_lower "design reason" ||
266         contains_sub content_lower "the goal"
267     in
268     let has_explain =
269         contains_sub content_lower "step" ||
270         contains_sub content_lower "flow" ||
271         contains_sub content_lower "algorithm" ||
272         contains_sub content_lower "process" ||
273         String.length content > 400
274     in
275     let has_teach =
276         contains_sub content_lower "pattern" ||
277         contains_sub content_lower "technique" ||
278         contains_sub content_lower "principle" ||
279         contains_sub content_lower "gotcha" ||
280         contains_sub content_lower "lesson" ||
281         contains_sub content_lower "remember"
282     in
283     let has_edge =
284         contains_sub content_lower "edge" ||
285         contains_sub content_lower "precondition" ||
286         contains_sub content_lower "invariant" ||
287         contains_sub content_lower "assume" ||
288         contains_sub content_lower "break" ||
289         contains_sub content_lower "warning" ||
290         contains_sub content_lower "careful"
291     in
292     let check_dim claimed has_support reason =
293         if claimed >= 6 && not has_support then
294             issues := Implausible_literacy_score {
295                 path; name = b.name; line = b.line;
296                 dimension = dim; claimed; reason
297             } :: !issues
298     in
299     check "story" score.Doc_detect.story has_narrative
300         "No narrative context found (try adding 'because',
301         'design reason', or more detail)";
302     check "explain" score.Doc_detect.explain has_explain
303         "No mechanics explanation found (try adding 'step',
304         'flow', or algorithm detail)";
305     check "teach" score.Doc_detect.teach has_teach
306         "No transferable insight found (try adding 'pattern',
307         'principle', or 'gotcha')";
308     check "edge" score.Doc_detect.edge has_edge
309         "No boundary discussion found (try adding 'precondition',
310         'invariant', or 'careful')";
311     | _ -> ()
312     end
313     | _ -> ()
314     end
315 ) bindings
316 ) ml_files;
317 List.rev !issues
318
319 (* agent note (|
320 *   WHAT: Check for invalid verify methods in (verify ...) stanzas.
321 *   Walks each .borg file, extracts section mappings with verify items,
322 *   and confirms each method is in the convention's allowed list.
323 *   Returns Invalid_verify_method findings with line numbers.
324 *   WHY: Verify stanzas use convention-defined vocabulary. Unknown
325 *   methods indicate typos or wrong convention declarations.
326 * |) *)
327 let check_invalid_verify dir path file =
328     let convention = Convention.resolve dir in
329     let valid_methods = Convention.verify_methods convention in
330     let mappings = Spec.extract_section_mappings file in
331     let issues = ref [] in
332     List.iter (fun (m : Spec.section_mapping) ->
333         List.iter (fun (v : Spec.verify_item) ->
334             if not (List.mem v.method valid_methods) then begin

```

```

331         (* line number: we don't have exact line from verify_item,
332            so we report the section name's approximate position. *)
333         issues := Invalid_verify_method { path; method_ = v.method_; line =
334           0 } :: !issues
335       end
336     ) m.verify
337   ) mappings;
338   List.rev !issues
339
340 (* agent note (|
341  * WHAT: Run lint checks on all .borg files in a directory.
342  * Performs validation: status values, duplicate names, annotated
343  * comments, pending responses, orphans, and optional code-doc checks.
344  * WHY: Main entry point for borge lint command.
345  * |) *)
346 let run ~code_doc ~mechanical ~literacy dir =
347   let all_files = File_utils.find_borg_files dir in
348   let file_stats = List.filter_map (fun path ->
349     try
350       let input = File_utils.read_file path in
351       let file = Parse.parse_file input in
352       Some (path, Spec.project_name file, file)
353     with _ -> None
354   ) all_files in
355   (* Collect issues per category *)
356   let status_issues = List.concat_map (fun (path, _, file) ->
357     check_invalid_statuses path file
358   ) file_stats in
359   let name_issues = check_duplicate_names (List.map (fun (p, n, _) -> (p,
360     n)) file_stats) in
361   let comment_issues = List.concat_map (fun (path, _, file) ->
362     check_annotated_comments path file @
363     check_comment_integrity path
364   ) file_stats in
365   let pending_issues = List.concat_map (fun (path, _, file) ->
366     check_pending_responses path file
367   ) file_stats in
368   let orphan_issues = List.map (fun path ->
369     Orphaned_file { path }
370   ) (Project.find_orphans dir) in
371   let db_issues = List.concat_map (fun (path, _, file) ->
372     check_db_specs path file
373   ) file_stats in
374   let ui_issues = List.concat_map (fun (path, _, file) ->
375     check_ui_specs path file
376   ) file_stats in
377   let verify_issues = List.concat_map (fun (path, _, file) ->
378     check_invalid_verify dir path file
379   ) file_stats in
380   let (db_issues, ui_issues) = (db_issues, ui_issues) in
381   let code_doc_issues = if code_doc then check_code_doc dir else [] in
382   let mechanical_issues = if mechanical then
383     List.map (fun (i : Code_quality.issue) ->
384       Unsafe_call {
385         path = i.path;
386         line = i.line;
387         call = i.call;
388         severity = (match i.severity with Code_quality.Error -> 'Error |
389           Code_quality.Warning -> 'Warning);
390         suggestion = i.suggestion;
391       }
392     ) (Code_quality.run dir)
393   else [] in
394   let literacy_issues = if literacy then check_literacy dir else [] in
395   let all_issues = status_issues @ name_issues @ comment_issues @
396     pending_issues @ orphan_issues @ verify_issues @ db_issues @ ui_issues
397     @ code_doc_issues @ mechanical_issues @ literacy_issues in
398   let errors = List.filter (function
399     | Invalid_status _ | Duplicate_project_name _ | Orphaned_file _ |
400     Invalid_verify_method _
401     | Db_validation { severity = 'Error'; _ }
402     | Ui_validation { severity = 'Error'; _ }
403     | Unsafe_call { severity = 'Error'; _ } -> true

```

```
398 | _ -> false
399 ) all_issues in
400 let warnings = List.filter (function
401   | Deleted_human_comment _ | Inserted_human_comment _ | Pending_response
402   | Unknown_comment_type _
403   | Missing_comment_value _
404   | Db_validation { severity = 'Warning; _ }
405   | Ui_validation { severity = 'Warning; _ }
406   | Undocumented_binding _ | Stale_doc_comment _
407   | Unsafe_call { severity = 'Warning; _ }
408   | Missing_literacy_score _ | Implausible_literacy_score _ -> true
409   | _ -> false
410 ) all_issues in
{ issues = all_issues; error_count = List.length errors; warning_count =
  List.length warnings }
```

67 lib/check/meta.ml

```

1 (** .borg.meta file types and writer.
2
3     .borg.meta files are companion files to .borg specs. They hold
4     machine-generated observations about code reality: module surfaces,
5     dune snapshots, and structured findings from static and agent analysis.
6
7     Key invariants:
8     - .borg.meta files are machine-written, never hand-edited
9     - The writer is idempotent: same inputs ? same output
10    - Findings are sorted and deterministic for clean git diffs
11    - Agent findings replace (not append) previous agent findings
12    - Static findings are preserved across agent runs *)
13
14
15 (** {1 Types} *)
16
17 type confidence = High | Medium | Low
18
19 type finding_source = Static | Agent
20
21 type finding_type =
22   | Unspecified_module
23   | Phantom_spec
24   | Partial_implementation
25   | Status_mismatch
26   | Missing_export
27   | Extra_export
28   | Structural_drift
29
30 type finding = {
31   ft_type : finding_type;
32   section : string option;
33   module_ : string option;
34   file : string option;
35   export : string option;
36   detail : string option;
37   spec_status : string option;
38   actual_status : string option;
39   confidence : confidence;
40   source : finding_source;
41   at : string; (** ISO 8601 timestamp *)
42 }
43
44 type module_surface_entry = {
45   module_name : string;
46   file : string;
47   exports : string list;
48 }
49
50 type dune_library_entry = {
51   name : string;
52   modules : string list;
53   public_name : string option;
54   libraries : string list;
55 }
56
57 type content_hash = string
58
59 type meta = {
60   project_name : string;
61   analyzed_at : string;
62   content_hashes : (string * content_hash) list;
63   module_surfaces : module_surface_entry list;
64   dune_snapshot : dune_library_entry list;
65   findings : finding list;
66 }
67
68 (** {1 String conversions} *)
69
70 let string_of_confidence = function
71   | High -> "high"

```

```

72 | Medium -> "medium"
73 | Low -> "low"
74
75 (* agent note (|
76 *   WHAT: Convert a source variant to string for display.
77 *   WHY: Used when formatting meta file content.
78 * |) *)
79 let string_of_source = function
80 | Static -> "static"
81 | Agent -> "agent"
82
83 (* agent note (|
84 *   WHAT: Convert a source variant to string for display.
85 *   WHY: Used when formatting meta file content.
86 * |) *)
87 let string_of_finding_type = function
88 | Unspecified_module -> "unspecified-module"
89 | Phantom_spec -> "phantom-spec"
90 | Partial_implementation -> "partial-implementation"
91 | Status_mismatch -> "status-mismatch"
92 | Missing_export -> "missing-export"
93 | Extra_export -> "extra-export"
94 | Structural_drift -> "structural-drift"
95
96 (** {1 Sexp generation} *)
97
98 let sexp_of_string s =
99 (* Simple: if s contains spaces or special chars, quote it *)
100 if String.length s = 0 then "\""
101 else begin
102   let needs_quote = ref false in
103   String.iter (fun c ->
104     if c = ' ' || c = '(' || c = ')' || c = '"' || c = '\n' || c = '\t'
105     then
106       needs_quote := true
107   ) s;
108   if !needs_quote then
109     let buf = Buffer.create (String.length s + 4) in
110     Buffer.add_char buf ' ';
111     String.iter (fun c ->
112       if c = '"' then Buffer.add_string buf "\\\""
113       else if c = '\\' then Buffer.add_string buf "\\\\"
114       else Buffer.add_char buf c
115     ) s;
116     Buffer.add_char buf ' ';
117     Buffer.contents buf
118   else s
119 end
120
121 let sexp_of_findings findings =
122 let buf = Buffer.create 256 in
123 Buffer.add_string buf "(findings";
124 List.iter (fun f ->
125   Buffer.add_char buf '\n';
126   Buffer.add_string buf " (finding ";
127   Buffer.add_string buf (string_of_finding_type f.ft_type);
128   (match f.section with Some s -> Buffer.add_string buf (Printf.sprintf
129     "\n (section %s)" (sexp_of_string s)) | None -> ());
130   (match f.module_ with Some s -> Buffer.add_string buf (Printf.sprintf
131     "\n (module %s)" (sexp_of_string s)) | None -> ());
132   (match f.file with Some s -> Buffer.add_string buf (Printf.sprintf "\n
133     (file %s)" (sexp_of_string s)) | None -> ());
134   (match f.export with Some s -> Buffer.add_string buf (Printf.sprintf
135     "\n (export %s)" (sexp_of_string s)) | None -> ());
136   (match f.detail with Some s -> Buffer.add_string buf (Printf.sprintf
137     "\n (detail %s)" (sexp_of_string s)) | None -> ());
138   (match f.spec_status with Some s -> Buffer.add_string buf
139     (Printf.sprintf "\n (spec-status %s)" (sexp_of_string s)) | None ->
140     ());
141   (match f.actual_status with Some s -> Buffer.add_string buf
142     (Printf.sprintf "\n (actual-status %s)" (sexp_of_string s)) | None
143     -> ());

```

```

134   Buffer.add_string buf (Printf.sprintf "\n    (confidence %s)"
135     (string_of_confidence f.confidence));
136   Buffer.add_string buf (Printf.sprintf "\n    (source %s)"
137     (string_of_source f.source));
138   Buffer.add_string buf (Printf.sprintf "\n    (at %s)" (sexp_of_string
139     f.at));
140   Buffer.add_string buf ")";
141   ) findings;
142   Buffer.add_string buf ")";
143   Buffer.contents buf
144
145 let sexp_of_module_surfaces surfaces =
146   let buf = Buffer.create 256 in
147   Buffer.add_string buf "(module-surface";
148   List.iter (fun s ->
149     Buffer.add_char buf '\n';
150     Buffer.add_string buf (Printf.sprintf " (%s\n    (file %s)"
151       (sexp_of_string s.module_name) (sexp_of_string s.file));
152     Buffer.add_string buf "\n    (exports";
153     List.iter (fun e ->
154       Buffer.add_char buf ' ';
155       Buffer.add_string buf (sexp_of_string e)
156     ) s.exports;
157     Buffer.add_string buf "));";
158   ) surfaces;
159   Buffer.add_string buf ")";
160   Buffer.contents buf
161
162 let sexp_of_dune_snapshot libs =
163   let buf = Buffer.create 256 in
164   Buffer.add_string buf "(dune-snapshot";
165   List.iter (fun lib ->
166     Buffer.add_char buf '\n';
167     Buffer.add_string buf (Printf.sprintf " (library %s" (sexp_of_string
168       lib.name));
169     (match lib.public_name with
170     | Some pn -> Buffer.add_string buf (Printf.sprintf "\n    (public_name
171       %s)" (sexp_of_string pn))
172     | None -> ());
173     Buffer.add_string buf "\n    (modules";
174     List.iter (fun m -> Buffer.add_char buf ' '; Buffer.add_string buf
175       (sexp_of_string m)) lib.modules;
176     Buffer.add_string buf "));";
177     Buffer.add_string buf "\n    (libraries";
178     List.iter (fun l -> Buffer.add_char buf ' '; Buffer.add_string buf
179       (sexp_of_string l)) lib.libraries;
180     Buffer.add_string buf "));";
181   ) libs;
182   Buffer.add_string buf ")";
183   Buffer.contents buf
184
185 let sexp_of_content_hashes hashes =
186   let buf = Buffer.create 128 in
187   Buffer.add_string buf "(content-hashes";
188   List.iter (fun (path, hash) ->
189     Buffer.add_string buf (Printf.sprintf "\n (%s %s)" (sexp_of_string
190       path) (sexp_of_string hash));
191   ) hashes;
192   Buffer.add_string buf ")";
193   Buffer.contents buf
194
195 (** {1 Writing} *)
196
197 let sexp_of_documentation (doc : Doc_coverage.file_coverage) =
198   let buf = Buffer.create 256 in
199   Buffer.add_string buf "(documentation\n";
200   Buffer.add_string buf (Printf.sprintf " (file %s)\n" (sexp_of_string
201     doc.path));
202   Buffer.add_string buf (Printf.sprintf " (total-bindings %d)\n"
203     doc.total_bindings);
204   Buffer.add_string buf (Printf.sprintf " (documented %d)\n"
205     doc.documented);
206   Buffer.add_string buf (Printf.sprintf " (exempt %d)\n" doc.exempt);

```

```

195   Buffer.add_string buf (Printf.sprintf " (undocumented %d)\n"
196     doc.undocumented);
197   Buffer.add_string buf (Printf.sprintf " (coverage-percent %.2f)\n"
198     doc.coverage_percent);
199   if doc.undocumented_names <> [] then begin
200     Buffer.add_string buf " (undocumented-names\n";
201     List.iter (fun name ->
202       Buffer.add_string buf (Printf.sprintf " (name %s)\n"
203         (sexp_of_string name))
204     ) doc.undocumented_names;
205     Buffer.add_string buf " )\n";
206   end;
207   Buffer.add_string buf ")";
208   Buffer.contents buf
209
210 let string_of_meta m ?(docs=None) () =
211   let buf = Buffer.create 512 in
212   Buffer.add_string buf " (; DO NOT EDIT -- generated by borge drift.
213     Changes will be overwritten. ;)\n\n";
214   Buffer.add_string buf "(meta ";
215   Buffer.add_string buf (sexp_of_string m.project_name);
216   Buffer.add_char buf '\n';
217   Buffer.add_string buf (Printf.sprintf " (analyzed-at %s)\n"
218     (sexp_of_string m.analyzed_at));
219   if m.content_hashes <> [] then begin
220     Buffer.add_string buf " ";
221     Buffer.add_string buf (sexp_of_string m.content_hashes);
222     Buffer.add_char buf '\n';
223   end;
224   if m.module_surfaces <> [] then begin
225     Buffer.add_string buf " ";
226     Buffer.add_string buf (sexp_of_string m.module_surfaces);
227     Buffer.add_char buf '\n';
228   end;
229   if m.dune_snapshot <> [] then begin
230     Buffer.add_string buf " ";
231     Buffer.add_string buf (sexp_of_string m.dune_snapshot);
232     Buffer.add_char buf '\n';
233   end;
234   if m.findings <> [] then begin
235     Buffer.add_string buf " ";
236     Buffer.add_string buf (sexp_of_string m.findings);
237     Buffer.add_char buf '\n';
238   end;
239   (match docs with
240   | Some doc ->
241     Buffer.add_string buf " ";
242     Buffer.add_string buf (sexp_of_string doc);
243     Buffer.add_char buf '\n';
244   | None -> ());
245   Buffer.add_string buf ")\n";
246   Buffer.contents buf
247
248 (** Write a .borg.meta file *)
249 let write_meta path m =
250   let content = string_of_meta m ~docs:None () in
251   let oc = open_out path in
252   output_string oc content;
253   close_out oc
254
255 (** Write a .borg.meta file with documentation coverage *)
256 let write_meta_with_docs path m (doc : Doc_coverage.file_coverage) =
257   let content = string_of_meta m ~docs:(Some doc) () in
258   let oc = open_out path in
259   output_string oc content;
260   close_out oc
261
262 (** {1 Content hashing} *)
263
264 (** Simple hash of file contents for staleness detection *)
265 let content_hash content =
266   (* FNV-1a 32-bit hash, hex encoded *)
267   let h = ref (Int32.of_int 2166136261) in

```

```

263   String.iter (fun c ->
264     h := Int32.logxor !h (Int32.of_int (Char.code c));
265     h := Int32.mul !h (Int32.of_int 16777619)
266   ) content;
267   Printf.sprintf "%08lx" !h
268
269 (** Hash a file by reading it and computing content_hash *)
270 let hash_file path =
271   try content_hash (File_utils.read_file path)
272   with Sys_error _ -> ""
273
274 (** {1 Timestamps} *)
275
276 let current_timestamp () =
277   let cmd = "date -u +%Y-%m-%dT%H:%M:%SZ" in
278   let ic = Unix.open_process_in cmd in
279   let line = input_line ic in
280   let _ = Unix.close_process_in ic in
281   String.trim line
282
283 (** {1 Finding comparison for deterministic ordering} *)
284
285 let compare_findings a b =
286   let c = compare (string_of_finding_type a.ft_type)
287     (string_of_finding_type b.ft_type) in
288   if c <> 0 then c
289   else compare a.detail b.detail
290
291 (** Sort findings for deterministic output *)
292 let sort_findings findings =
293   List.sort compare_findings findings
294
295 (** {1 Merge logic} *)
296
297 (** Merge agent findings into existing findings.
298   - Keep all static findings from existing
299   - Remove all agent findings from existing
300   - Add new agent findings
301   - Sort for determinism *)
302 let merge_agent_findings existing new_agent =
303   let static_only = List.filter (fun f -> f.source = Static) existing in
304   sort_findings (static_only @ new_agent)
305
306 (** {1 Meta file path} *)
307
308 (** Get the .borg.meta path for a given .borg file *)
309 let meta_path_of borg_path =
310   borg_path ^ ".meta"
311
312 (** Check if a path is a .borg.meta file *)
313 let is_meta_file path =
314   String.length path > 10 &&
315   String.sub path (String.length path - 10) 10 = ".borg.meta"
316
317 (** {1 Reading .borg.meta files} *)
318
319 (** Parse a .borg.meta file. Returns None if file doesn't exist or parse
320   fails.
321   This is intentionally simple -- we don't need to parse our own output
322   back
323   into full typed structures. We just need to check if it exists and is
324   valid. *)
325 let read_meta_exists path =
326   Sys.file_exists path
327
328 (** Read the analyzed-at timestamp from a .borg.meta file *)
329 let read_analyzed_at path =
330   try
331     let input = File_utils.read_file path in
332     (* Quick: find (analyzed-at "...") *)
333     let re = Str.regexp {|(analyzed-at "[^"]*" )|} in
334     if Str.string_match re input 0 then
335       Some (Str.matched_group 1 input)

```



```
332     else None
333   with _ -> None
```

68 lib/check/report.ml

```

1 open Borge_lang
2
3 type file_stats = {
4   path : string;
5   project_name : string option;
6   implemented : int;
7   verified : int;
8   in_progress : int;
9   planned : int;
10 }
11
12 type totals = {
13   implemented : int;
14   verified : int;
15   in_progress : int;
16   planned : int;
17 }
18
19 type result = {
20   files : file_stats list;
21   totals : totals;
22   orphans : string list; (** .borg files not in the inline tree *)
23 }
24
25 (* agent note (|
26  * WHAT: Parse a .borg file and extract status counts.
27  * Returns file_stats with path, project name, and counts of
28  * implemented, in_progress, and planned sections.
29  * WHY: Used by borge report to compute per-file and aggregate
30  * statistics about spec completion.
31  * |) *)
32 let stats_of_file path =
33   let input = File_utils.read_file path in
34   try
35     let file = Parse.parse_file input in
36     let name = Spec.project_name file in
37     let counts = Spec.status_counts file in
38     let impl = match List.assoc_opt Spec.Implemented counts with Some v ->
39       v | None -> 0 in
40     let veri = match List.assoc_opt Spec.Verified counts with Some v -> v |
41       None -> 0 in
42     let plan = match List.assoc_opt Spec.Planned counts with Some v -> v |
43       None -> 0 in
44     let prog = match List.assoc_opt Spec.In_progress counts with Some v ->
45       v | None -> 0 in
46     Some { path; project_name = name; implemented = impl; verified = veri;
47           in_progress = prog; planned = plan }
48   with _ -> None
49
50 (** Run report with inline-tree awareness.
51     Builds the project tree from root file(s), walks it,
52     and only counts each .borg file once. Also detects orphans. *)
53 let run dir =
54   let roots = Project.find_roots dir in
55   let trees = List.filter_map (fun root ->
56     match Project.build_tree root with
57     | Ok tree -> Some tree
58     | Error _ -> None
59   ) roots in
60   (* Collect all paths from all trees (deduplicated by tree walker) *)
61   let tree_paths = List.concat_map Project.tree_paths trees in
62   (* Find orphans: files not in any tree *)
63   let all_files = File_utils.find_borg_files dir in
64   let orphans = List.filter (fun path ->
65     not (List.mem path tree_paths)
66   ) all_files in
67   (* Build per-file stats only for tree paths *)
68   let rows = List.filter_map stats_of_file tree_paths in
69   let tot_impl : int = List.fold_left (fun acc (r : file_stats) -> acc +
70     r.implemented) 0 rows in

```

```
65   let tot_veri : int = List.fold_left (fun acc (r : file_stats) -> acc +  
    r.verified) 0 rows in  
66   let tot_plan : int = List.fold_left (fun acc (r : file_stats) -> acc +  
    r.planned) 0 rows in  
67   let tot_prog : int = List.fold_left (fun acc (r : file_stats) -> acc +  
    r.in_progress) 0 rows in  
68   { files = rows; totals = { implemented = tot_impl; verified = tot_veri;  
    in_progress = tot_prog; planned = tot_plan }; orphans }
```

69 lib/check/surface.ml

```

1  (** Extract the public surface of an OCaml module.
2
3      For modules with .mli files: the .mli is the authoritative surface.
4      For modules without .mli: infer the surface from top-level bindings
5      in the .ml file (heuristic -- the real surface is what the compiler
6      infers).
7
8      Surface = { types, values, exceptions } where values are top-level
9      let bindings and types are type/exception/module declarations. *)
10
11 type module_surface = {
12   path : string;
13   module_name : string;
14   exports : string list;
15   source : [ 'Mli of string | 'Ml_inferred of string ];
16 }
17
18 (* agent note (|
19  *   WHAT: Check if a character is valid in an OCaml identifier.
20  *   Includes letters, digits, underscore, and prime.
21  *
22  *   WHY: Used by name extraction functions to properly parse
23  *   OCaml identifiers.
24  * |) *)
25 let is_ident_char c =
26   (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') ||
27   (c >= '0' && c <= '9') || c = '_' || c = '\'
28
29 (* agent note (|
30  *   WHAT: Extract the binding name from a top-level let expression.
31  *   Handles "let name", "let rec name", "let name : type", etc.
32  *   Returns None for let (), let _, let open, or non-let lines.
33  *
34  *   WHY: The core extraction function for inferring module surface
35  *   from .ml files without .mli.
36  * |) *)
37 (* exempt: String.sub *)
38 let extract_toplevel_let_name line =
39   let len = String.length line in
40   if len < 4 then None
41   else if String.sub line 0 4 <> "let " then None
42   else begin
43     let rest = (* exempt: String.sub *) String.sub line 4 (len - 4) in
44     let rest_len = String.length rest in
45     (* Skip 'rec ' *)
46     let i = ref 0 in
47     while !i + 3 < rest_len && (* exempt: String.sub *) String.sub rest !i
48       4 = "rec " do i := !i + 4 done;
49     (* Skip spaces *)
50     while !i < rest_len && rest.[!i] = ' ' do incr i done;
51     if !i >= rest_len then None
52     else begin
53       let ch = rest.[!i] in
54       (* let () = ... *)
55       if ch = '(' && !i + 1 < rest_len && rest.[!i + 1] = ')' then None
56       (* let _ = ... *)
57       else if ch = '_' then None
58       else begin
59         let start = !i in
60         while !i < rest_len && is_ident_char rest.[!i] do incr i done;
61         if !i > start then Some (String.sub rest start (!i - start))
62         else None
63       end
64     end
65   end
66
67 (* agent note (|
68  *   WHAT: Extract type name from a type definition line.
69  *   Handles "type t", "type name", "type 'a t", "type nonrec t", etc.
70  *
71  *   WHY: Part of surface inference - extracts type definitions from

```

```

70  * .ml files to build the module's export list.
71  * |) *)
72  (* exempt: String.sub *)
73  let extract_type_name line =
74    let len = String.length line in
75    if len < 5 then None
76    else if String.sub line 0 5 <> "type " then None
77    else begin
78      let rest = (* exempt: String.sub *) String.sub line 5 (len - 5) in
79      let i = ref 0 in
80      let rest_len = String.length rest in
81      (* Skip 'nonrec ' *)
82      while !i + 6 < rest_len && (* exempt: String.sub *) String.sub rest !i
83        7 = "nonrec " do i := !i + 7 done;
84      while !i < rest_len && rest.[!i] = ' ' do incr i done;
85      (* Skip 'rec ' (type rec doesn't exist but be safe) *)
86      if !i >= rest_len then None
87      else if rest.[!i] = '\\' then begin
88        (* 'a t -- type parameter, skip to the actual name *)
89        while !i < rest_len && rest.[!i] <> ' ' do incr i done;
90        while !i < rest_len && rest.[!i] = ' ' do incr i done;
91        if !i >= rest_len then None
92        else begin
93          let start = !i in
94          while !i < rest_len && is_ident_char rest.[!i] do incr i done;
95          if !i > start then Some (String.sub rest start (!i - start))
96          else None
97        end
98      end
99      let start = !i in
100      while !i < rest_len && is_ident_char rest.[!i] do incr i done;
101      if !i > start then Some (String.sub rest start (!i - start))
102      else None
103    end
104  end
105
106  (* agent note (|
107  *   WHAT: Extract exception name from an exception definition.
108  *   Handles "exception E" and "exception E of type".
109  *
110  *   WHY: Exceptions are part of the public module surface.
111  * |) *)
112  (* exempt: String.sub *)
113  let extract_exception_name line =
114    let len = String.length line in
115    if len < 10 then None
116    else if String.sub line 0 10 <> "exception " then None
117    else begin
118      let rest = (* exempt: String.sub *) String.sub line 10 (len - 10) in
119      let i = ref 0 in
120      let rest_len = String.length rest in
121      while !i < rest_len && rest.[!i] = ' ' do incr i done;
122      if !i >= rest_len then None
123      else begin
124        let start = !i in
125        while !i < rest_len && is_ident_char rest.[!i] do incr i done;
126        if !i > start then Some (String.sub rest start (!i - start))
127        else None
128      end
129    end
130  end
131
132  (* agent note (|
133  *   WHAT: Extract module name from a module declaration.
134  *   Handles "module M = ..." and "module type T = ...".
135  *
136  *   WHY: Module declarations are part of the public surface.
137  * |) *)
138  (* exempt: String.sub *)
139  let extract_module_name line =
140    let len = String.length line in
141    if len < 7 then None
142    else if String.sub line 0 7 <> "module " then None

```

```

142   else begin
143     let rest = (* exempt: String.sub *) String.sub line 7 (len - 7) in
144     let i = ref 0 in
145     let rest_len = String.length rest in
146     (* Skip 'type ' (module type ...) *)
147     if rest_len > 5 && String.sub rest 0 5 = "type " then None
148     else begin
149       (* Skip 'rec ' *)
150       while !i + 3 < rest_len && (* exempt: String.sub *) String.sub rest
151         !i 4 = "rec " do i := !i + 4 done;
152       while !i < rest_len && rest.[!i] = ' ' do incr i done;
153       if !i >= rest_len then None
154       else begin
155         let ch = rest.[!i] in
156         if ch >= 'A' && ch <= 'Z' then begin
157           let start = !i in
158           while !i < rest_len && is_ident_char rest.[!i] do incr i done;
159           Some ((* exempt: String.sub *) String.sub rest start (!i - start))
160         end
161       else None
162     end
163   end
164
165   (* agent note (|
166    *   WHAT: Extract all exports from an .ml or .mli file.
167    *   Only looks at column 0 (top-level) declarations.
168    *
169    *   WHY: The main surface extraction function for source files.
170    * |) *)
171   let extract_exports content =
172     let lines = String.split_on_char '\n' content in
173     let names = ref [] in
174     List.iter (fun line ->
175       match extract_toplevel_let_name line with
176       | Some n -> names := n :: !names
177       | None ->
178         match extract_type_name line with
179         | Some n -> names := n :: !names
180         | None ->
181           match extract_exception_name line with
182           | Some n -> names := n :: !names
183           | None ->
184             match extract_module_name line with
185             | Some n -> names := n :: !names
186             | None -> ()
187     ) lines;
188     List.sort String.compare (List.sort_uniq String.compare !names)
189
190   (* agent note (|
191    *   WHAT: Extract the surface of a single module given its .ml path.
192    *   If a .mli exists, use that instead (it's authoritative).
193    *
194    *   WHY: The main entry point for getting a module's public interface
195    *   for drift checking.
196    * |) *)
197   let extract_surface ml_path =
198     let mli_path = Filename.chop_extension ml_path ^ ".mli" in
199     let module_name = String.capitalize_ascii
200       (Filename.chop_extension (Filename.basename ml_path)) in
201     if Sys.file_exists mli_path then
202       let content = File_utils.read_file mli_path in
203       { path = mli_path; module_name; exports = extract_exports content;
204         source = 'Mli mli_path' }
205     else
206       let content = File_utils.read_file ml_path in
207       { path = ml_path; module_name; exports = extract_exports content;
208         source = 'ML_inferred ml_path' }
209
210   (* agent note (|
211    *   WHAT: Find .ml files in a directory recursively.
212    *   Excludes _build and .git directories.
213    *

```

```

214  *   WHY: Need to enumerate source files for surface extraction.
215  * |) *)
216  let find_ml_files dir =
217    let rec find path =
218      try
219        let entries = Sys.readdir path in
220        Array.fold_left (fun acc entry ->
221          if entry = "_build" || entry = ".git" then acc
222          else
223            let full = Filename.concat path entry in
224            if Sys.is_directory full then find full @ acc
225            else if Filename.check_suffix entry ".ml" then full :: acc
226            else acc
227          ) [] entries
228        with Sys_error _ -> []
229    in
230    List.sort String.compare (find dir)
231
232  (* agent note (|
233  *   WHAT: Extract surfaces for all modules listed in a dune library.
234  *   Matches .ml files to library module names and extracts each.
235  *
236  *   WHY: Used to build complete library surface for drift checking.
237  * |) *)
238  let surfaces_for_library ml_files lib_modules =
239    List.filter_map (fun mod_name ->
240      let capitalized = String.capitalize_ascii mod_name in
241      List.find_opt (fun path ->
242        String.capitalize_ascii
243          (Filename.chop_extension (Filename.basename path)) = capitalized
244      ) ml_files
245      |> Option.map (fun ml_path -> extract_surface ml_path)
246    ) lib_modules

```

70 lib/convention.ml

```

1 (** Convention dispatch: select the right module set based on the
2    project's declared convention.
3
4    A convention declares structural rules for a project -- directory
5    layout, build system, file extensions. Borge uses the convention
6    to dispatch to the appropriate parsing, surface extraction, and
7    doc detection modules.
8
9    Convention resolution reads the root .borg file and extracts the
10   (convention NAME) form. If none is found, defaults to ocaml-dune
11   for backward compatibility.
12
13   Comment parsing is SHARED across conventions. Borge comments in Go
14   source nest the standard (* *) grammar inside Go's /* */ block
15   comment delimiters: /* (* author note (|...|) *) */. This means
16   Doc_detect and Borg_comment work unchanged for both languages. *)
17
18 type t =
19   | Ocaml_dune
20   | Go_standard
21
22 (* agent note (|
23  *   WHAT: Convert a convention type to its string name.
24  *
25  *   WHY: Used in prompts and reports to identify the active convention.
26  * |) *)
27 let name = function
28   | Ocaml_dune -> "ocaml-dune"
29   | Go_standard -> "go-standard"
30
31 (* agent note (|
32  *   WHAT: Source file extension for this convention.
33  *
34  *   WHY: File discovery needs to know which extension to scan.
35  * |) *)
36 let source_extension = function
37   | Ocaml_dune -> ".ml"
38   | Go_standard -> ".go"
39
40 (* agent note (|
41  *   WHAT: Interface file extension for this convention.
42  *   Go has no separate interface files -- exports are determined
43  *   by capitalization.
44  *
45  *   WHY: Surface extraction checks for .mli in OCaml but not Go.
46  * |) *)
47 let interface_extension = function
48   | Ocaml_dune -> Some ".mli"
49   | Go_standard -> None
50
51 (* agent note (|
52  *   WHAT: Directories to exclude when scanning for source files.
53  *
54  *   WHY: Each convention has its own build artifact directories
55  *   that should not be scanned.
56  * |) *)
57 let excluded_dirs = function
58   | Ocaml_dune -> ["_build"; ".git"]
59   | Go_standard -> ["vendor"; ".git"; "node_modules"]
60
61 (* agent note (|
62  *   WHAT: Resolve a convention from its string name.
63  *   Returns None for unrecognized names.
64  *
65  *   WHY: The (convention NAME) form in .borg files stores the
66  *   convention as a plain atom. This function maps it to the
67  *   typed variant used for dispatch throughout borge.
68  * |) *)
69 let of_string = function
70   | "ocaml-dune" -> Some Ocaml_dune
71   | "go-standard" -> Some Go_standard

```



```

72 | _ -> None
73
74 (* agent note (|
75  * WHAT: Extract the convention from a root .borg file.
76  * Looks for (convention NAME) as a direct child of the
77  * (project ...) form. Returns Ocaml_dune as default if
78  * no convention is declared.
79  *
80  * WHY: Every borge command needs to know the convention to
81  * dispatch correctly. This reads it from the project root spec.
82  * |) *)
83 let resolve_from_file path =
84   try
85     let input = File_utils.read_file path in
86     let file = Borge_lang.Parse.parse_file input in
87     let rec find_convention = function
88       | [] -> Ocaml_dune (* default *)
89       | { Borge_lang.Ast.node; _ } :: rest ->
90         (match node with
91          | Borge_lang.Ast.List (_, children) ->
92            let rec scan = function
93              | [] -> find_convention rest
94              | Borge_lang.Ast.List (_, Borge_lang.Ast.Atom (_,
95                "convention")) :: Borge_lang.Ast.Atom (_, name) :: _ ->
96                (match of_string name with
97                 | Some t -> t
98                 | None ->
99                   Printf.eprintf "Warning: unknown convention '%s',
100                    defaulting to ocaml-dune\n" name;
101                   Ocaml_dune)
102              | _ :: scan_rest -> scan scan_rest
103            in
104            scan children
105          | _ -> find_convention rest)
106     in
107     find_convention file.Borge_lang.Ast.top_level
108   with _ ->
109     Ocaml_dune (* fallback on error *)
110
111 (* agent note (|
112  * WHAT: Resolve the convention from a project directory.
113  * Finds root .borg files and reads the convention from the
114  * first one. Falls back to Ocaml_dune if no roots are found.
115  *
116  * WHY: Most borge commands operate on a directory, not a specific
117  * file. This is the primary entry point for convention resolution.
118  * |) *)
119 let resolve_dir =
120   let roots = Project.find_roots dir in
121   match roots with
122   | [] -> Ocaml_dune
123   | root :: _ -> resolve_from_file root
124
125 (* agent note (|
126  * WHAT: Convert a section name to expected module name.
127  * For ocaml-dune: converts hyphenated sections to snake_case,
128  * then capitalizes to match OCaml module naming conventions.
129  * Example: "ui-dream" ? "Ui_dream".
130  *
131  * WHY: The drift detector needs to know which module a section
132  * corresponds to. This is the convention-derived default mapping.
133  * Explicit (implements ...) declarations override this.
134  * |) *)
135 let section_to_module_name = function
136   | Ocaml_dune ->
137     fun name ->
138       let normalized = String.map (fun c ->
139         if c = '-' then '_' else c
140       ) name in
141       String.capitalize_ascii normalized
142   | Go_standard ->
143     (* Go uses directories as package names; module name is just
144      the normalized section name. *)

```

```

143     fun name -> name
144
145 (* agent note (|
146  * WHAT: Suggest source file paths for a section based on convention.
147  * Returns relative paths (e.g. ["lib/ui/ui_dream.ml"]).
148  * The caller resolves these against the project root.
149  *
150  * WHY: This is the convention's default mapping. Agents may
151  * override with explicit (implements ...) declarations in .borg.
152  * For ocaml-dune, we look at dune module lists to find which
153  * library/directory contains the normalized module name.
154  * |) *)
155 let suggest_module_files _t ~section_name ~dune_modules =
156   (* dune_modules is a list of (library_name, module_name, file_path)
157     triples
158     produced by drift.ml from dune file parsing. *)
159   let normalized = String.map (fun c ->
160     if c = '-' then '_' else c
161   ) section_name in
162   let lookup = String.capitalize_ascii normalized in
163   List.filter_map (fun (_lib, mod_name, path) ->
164     if mod_name = lookup then Some path else None
165   ) dune_modules
166
167 (* agent note (|
168  * WHAT: Normalize a section name for convention comparison.
169  * Always returns lowercase with hyphens replaced by underscores.
170  * Used by drift detection when matching spec sections to code.
171  * |) *)
172 let normalize_section_name name =
173   String.map (fun c -> if c = '-' then '_' else Char.lowercase_ascii c) name
174
175 (* agent note (|
176  * WHAT: Return valid verify methods for a convention.
177  * Verify methods define how a section's correctness is confirmed.
178  * ocaml-dune supports: build, test, smoke, visual, flow.
179  * go-standard supports: build, test, bench.
180  *
181  * WHY: Lint uses this to validate (verify ...) stanzas in .borg files.
182  * Unknown methods are flagged as lint errors. Methods are
183  * convention-defined, not hardcoded into borge core.
184  * |) *)
185 let verify_methods = function
186   | Ocaml_dune -> ["build"; "test"; "smoke"; "visual"; "flow"]
187   | Go_standard -> ["build"; "test"; "bench"]

```

71 lib/go_doc_extract.ml

```

1  (** Extract Go binding information for documentation coverage.
2
3     Analogous to Doc_extract (OCaml). Gathers all top-level declarations
4     from Go source files, classifying them as exported, test, or internal.
5
6     Go-specific classification:
7     - func TestXxx(t *testing.T) ? test binding (excluded from doc
8       enforcement)
9     - func BenchmarkXxx(b *testing.B) ? test binding
10    - func (s *suite) TestXxx() ? test method
11    - func lowercaseName ? unexported (internal)
12    - func UppercaseName ? exported (needs doc)
13    - type/var/const follow the same capitalized = exported rule *)
14
15 type go_binding_info = {
16   name : string;
17   line : int;
18   is_exported : bool;
19   is_test : bool;
20   is_internal : bool;
21 }
22
23 (* agent note (|
24  *   WHAT: Extract all top-level Go declarations from a .go file.
25  *   Returns binding info with name, line, and classification
26  *   (exported, test, internal).
27  *
28  *   WHY: Documentation coverage analysis needs to discover which
29  *   declarations exist and whether they're exported (need docs)
30  *   or internal (excluded from enforcement).
31  * |) *)
32
33 let extract_go_bindings path =
34   let bindings = ref [] in
35   let line_num = ref 0 in
36   let in_block_comment = ref false in
37   let in_group_decl = ref false in
38
39   try
40     (* exempt: open_in input_line *)
41     let ic = open_in path in
42     (try
43       while true do
44         let line = input_line ic in
45         incr line_num;
46         let trimmed = String.trim line in
47
48         (* Skip block comments *)
49         if !in_block_comment then begin
50           if String.length trimmed >= 2 &&
51              String.sub trimmed (String.length trimmed - 2) 2 = "*/" then
52             in_block_comment := false;
53           ()
54         end
55         else if !in_group_decl then begin
56           if String.length trimmed >= 1 && trimmed.[0] = ')' then
57             in_group_decl := false
58           else begin
59             (* Parse names inside var/const group *)
60             let name, _ = Go_surface.extract_ident trimmed 0 in
61             if name <> "" then begin
62               let is_exp = Go_surface.is_exported_name name in
63               bindings := { name; line = !line_num;
64                             is_exported = is_exp;
65                             is_test = false;
66                             is_internal = not is_exp } :: !bindings
67             end
68           end
69         end
70       else begin
71         (* Block comment start *)
72         if String.length trimmed >= 2 &&

```

```

71         String.sub trimmed 0 2 = "/*" then begin
72         let content = String.sub trimmed 2 (String.length trimmed - 2)
in
73         if not (String.contains content '*' && String.contains content
74         '/') then
75             in_block_comment := true
76         end
77         (* Func declarations *)
78         else if String.starts_with ~prefix:"func " trimmed then begin
79             if Go_surface.is_test_or_benchmark trimmed then begin
80                 let name, _kind = match Go_surface.extract_func_decl trimmed
with
81                 | Some (n, k) -> (n, k)
82                 | None -> ("unknown", Go_surface.Func)
83             in
84                 bindings := { name; line = !line_num;
85                             is_exported = false; is_test = true;
86                             is_internal = true } :: !bindings
87             end
88             else
89                 match Go_surface.extract_func_decl trimmed with
90                 | Some (name, _kind) ->
91                     let is_exp = Go_surface.is_exported_name name in
92                     bindings := { name; line = !line_num;
93                                 is_exported = is_exp; is_test = false;
94                                 is_internal = not is_exp } :: !bindings
95                 | None -> ()
96             end
97             (* Type declarations *)
98             else if String.starts_with ~prefix:"type " trimmed then
99                 match Go_surface.extract_type_decl trimmed with
100                 | Some name ->
101                     let is_exp = Go_surface.is_exported_name name in
102                     bindings := { name; line = !line_num;
103                                 is_exported = is_exp; is_test = false;
104                                 is_internal = not is_exp } :: !bindings
105                 | None -> ()
106             end
107             (* Var declarations *)
108             else if String.starts_with ~prefix:"var " trimmed then
109                 (match Go_surface.extract_var_or_const_decl ~keyword:"var"
110                 trimmed with
111                 | Some name ->
112                     let is_exp = Go_surface.is_exported_name name in
113                     bindings := { name; line = !line_num;
114                                 is_exported = is_exp; is_test = false;
115                                 is_internal = not is_exp } :: !bindings
116                 | None ->
117                     let rest = String.sub trimmed 4 (String.length trimmed - 4)
118                 in
119                 |> String.trim in
120                     if String.length rest > 0 && rest.[0] = '(' then
121                         in_group_decl := true)
122                 (* Const declarations *)
123                 else if String.starts_with ~prefix:"const " trimmed then
124                     (match Go_surface.extract_var_or_const_decl ~keyword:"const"
125                     trimmed with
126                     | Some name ->
127                         let is_exp = Go_surface.is_exported_name name in
128                         bindings := { name; line = !line_num;
129                                     is_exported = is_exp; is_test = false;
130                                     is_internal = not is_exp } :: !bindings
131                     | None ->
132                         let rest = String.sub trimmed 6 (String.length trimmed - 6)
133                     in
134                     |> String.trim in
135                         if String.length rest > 0 && rest.[0] = '(' then
136                             in_group_decl := true)
137                     else ()
138                 end
139             done
140             with End_of_file -> ());
141         close_in ic;
142         List.rev !bindings
143     with Sys_error _ -> []
144

```

```

137 (* agent note (|
138 *   WHAT: Recursively find .go files under a directory.
139 *   Skips vendor/, .git/, testdata/, and _test.go files
140 *   (test files are excluded from doc coverage enforcement).
141 *
142 *   WHY: Coverage analysis scans non-test source files.
143 * |) *)
144 let find_go_files dir =
145   let rec find path =
146     try
147       let entries = Sys.readdir path in
148       Array.fold_left (fun acc entry ->
149         if entry = "vendor" || entry = ".git" ||
150           entry = "node_modules" || entry = "testdata" then acc
151         else if String.length entry > 0 && entry.[0] = '.' then acc
152         else
153           let full = Filename.concat path entry in
154           if Sys.is_directory full then find full @ acc
155           else if Filename.check_suffix entry ".go" &&
156             not (String.length entry >= 8 &&
157               String.sub entry (String.length entry - 8) 8 =
158                 "_test.go") then
159             full :: acc
160           else acc
161         ) [] entries
162       with Sys_error _ -> []
163   in
164   List.sort String.compare (find dir)
165
166 (* agent note (|
167 *   WHAT: Extract bindings from all .go files in a project,
168 *   returning (path, go_binding_info) pairs.
169 *
170 *   WHY: Project-wide coverage analysis needs all bindings
171 *   paired with their source files.
172 * |) *)
173 let extract_project_go_bindings dir =
174   let go_files = find_go_files dir in
175   List.concat_map (fun path ->
176     let bindings = extract_go_bindings path in
177     List.map (fun b -> (path, b)) bindings
178   ) go_files
179
180 (* agent note (|
181 *   WHAT: Count statistics for a list of bindings: total,
182 *   exported, tests, and internal counts.
183 *
184 *   WHY: Quick summary numbers for coverage reports.
185 * |) *)
186 let count_stats bindings =
187   let total = List.length bindings in
188   let exported = List.filter (fun b -> b.is_exported) bindings |>
189     List.length in
190   let tests = List.filter (fun b -> b.is_test) bindings |> List.length in
191   let internal = List.filter (fun b -> b.is_internal) bindings |>
192     List.length in
193   (total, exported, tests, internal)

```

72 lib/go_parse.ml

```

1  (** Parse Go project structure: go.mod and directory layout.
2
3      Analogous to Dune_parse. Discovers Go project structure by parsing
4      go.mod and walking the directory tree. The directory IS the package
5      in Go -- no separate build file needed.
6
7      Excluded directories: vendor/, .git/, node_modules/ *)
8
9  type go_module = {
10     module_path : string;  (* e.g. "github.com/foo/bar" *)
11     go_version : string;   (* e.g. "1.22" *)
12 }
13
14 type go_package_kind =
15   | Cmd      (* cmd/F00 -- executable (main package) *)
16   | Internal  (* internal/F00 -- private package *)
17   | Pkg       (* pkg/F00 -- public library package *)
18   | Standard  (* other dir with .go files *)
19
20 type go_package = {
21     dir_path : string;
22     package_name : string;
23     kind : go_package_kind;
24     go_files : string list;  (* .go files in this package *)
25     test_files : string list; (* *_test.go files in this package *)
26 }
27
28 type go_project = {
29     go_mod : go_module option;
30     packages : go_package list;
31 }
32
33 (* agent note (|
34  * WHAT: Parse go.mod to extract the module path and Go version.
35  * Handles the standard format:
36  *   module github.com/foo/bar
37  *   go 1.22
38  * Ignores require/replace/exclude stanzas.
39  *
40  * WHY: The module path is the root import path for the project.
41  * Go surface extraction and import resolution need it.
42  * |) *)
43 let parse_go_mod path =
44   try
45     (* exempt: open_in input_line *)
46     let ic = open_in path in
47     let module_path = ref None in
48     let go_version = ref None in
49     (try
50       while true do
51         let line = input_line ic |> String.trim in
52         if String.length line >= 7 && String.sub line 0 7 = "module " then
53           begin
54             let rest = String.sub line 7 (String.length line - 7) |>
55               String.trim in
56             module_path := Some rest
57           end
58         else if String.length line >= 3 && String.sub line 0 3 = "go " then
59           begin
60             let rest = String.sub line 3 (String.length line - 3) |>
61               String.trim in
62             go_version := Some rest
63           end
64         done
65       with End_of_file -> ());
66     close_in ic;
67     match !module_path with
68     | Some mp ->
69       Some { module_path = mp;
70             go_version = (match !go_version with Some v -> v | None ->
71               "") }

```

```

67 | None -> None
68 with Sys_error _ -> None
69
70 (* agent note (|
71 * WHAT: Find the go.mod file by walking up from a directory.
72 *
73 * WHY: Go tools find go.mod by walking up the directory tree.
74 * We follow the same convention so borge works from any subdir.
75 * |) *)
76 let find_go_mod dir =
77   let rec walk d =
78     let candidate = Filename.concat d "go.mod" in
79     if Sys.file_exists candidate then Some candidate
80     else
81       let parent = Filename.dirname d in
82       if parent = d then None (* reached root *)
83       else walk parent
84   in
85   walk dir
86
87 (* agent note (|
88 * WHAT: Check if a directory name should be skipped during
89 * Go package discovery. Skips hidden dirs, vendor, _, etc.
90 *
91 * WHY: Go package discovery must skip directories that don't
92 * represent importable packages -- vendor, hidden dirs, testdata.
93 * |) *)
94 let should_skip_dir name =
95   name = "vendor" ||
96   name = ".git" ||
97   name = "node_modules" ||
98   name = "testdata" ||
99   (String.length name > 0 && name.[0] = '.')
100
101 (* agent note (|
102 * WHAT: Classify a package directory by its position in the
103 * Go project layout.
104 *
105 * WHY: go-standard convention assigns meaning to directory
106 * positions: cmd/ = executables, internal/ = private, etc.
107 * Drift detection uses this classification.
108 * |) *)
109 let classify_package dir_path =
110   let base = Filename.basename dir_path in
111   let parent = Filename.dirname dir_path in
112   let parent_base = Filename.basename parent in
113   if parent_base = "cmd" then Cmd
114   else if parent_base = "internal" || String.length base >= 9 && String.sub
115     base 0 9 = "internal" then Internal
116   else if parent_base = "pkg" then Pkg
117   else Standard
118
119 (* agent note (|
120 * WHAT: Recursively discover Go packages in a directory.
121 * A directory is a package if it contains .go files.
122 * Returns go_package records with file listings.
123 *
124 * WHY: Go's package model is directory-based. Walking the tree
125 * is how we discover what packages exist for drift detection.
126 * |) *)
127 let discover_packages dir =
128   let packages = ref [] in
129   let rec walk path =
130     try
131       let entries = Sys.readdir path in
132       let go_files = ref [] in
133       let test_files = ref [] in
134       let subdirs = ref [] in
135       Array.iter (fun entry ->
136         if should_skip_dir entry then ()
137         else
138           let full = Filename.concat path entry in
139           if Sys.is_directory full then

```

```

139     subdirs := full :: !subdirs
140   else if Filename.check_suffix entry ".go" then
141     if String.length entry >= 8 &&
142       String.sub entry (String.length entry - 8) 8 = "_test.go"
143   then
144     test_files := entry :: !test_files
145   else
146     go_files := entry :: !go_files
147   else ()
148   ) entries;
149   (* This directory is a package if it has .go files *)
150   if !go_files <> [] || !test_files <> [] then begin
151     let package_name = Filename.basename path in
152     let kind = classify_package path in
153     packages := { dir_path = path; package_name; kind;
154                  go_files = List.sort String.compare !go_files;
155                  test_files = List.sort String.compare !test_files }
156               :: !packages
157   end;
158   List.iter walk (List.sort String.compare !subdirs)
159 with Sys_error _ -> ()
160 in
161   walk dir;
162   List.sort (fun a b -> String.compare a.dir_path b.dir_path) !packages
163
164 (* agent note (|
165  * WHAT: Extract executable names from cmd/ subdirectories.
166  * Each subdirectory of cmd/ is a binary entrypoint.
167  *
168  * WHY: Analogous to dune's (executables ...) stanza discovery.
169  * Drift detection and reports need to know what binaries exist.
170  * |) *)
171 let discover_executables dir =
172   let cmd_dir = Filename.concat dir "cmd" in
173   if not (Sys.is_directory cmd_dir) then []
174   else
175     try
176       Sys.readdir cmd_dir
177     |> Array.to_list
178     |> List.filter (fun name ->
179       not (should_skip_dir name) &&
180       Sys.is_directory (Filename.concat cmd_dir name))
181     |> List.sort String.compare
182   with Sys_error _ -> []
183
184 (* agent note (|
185  * WHAT: Full project scan combining go.mod parsing and directory
186  * walking. Returns a go_project with all discovered structure.
187  *
188  * WHY: The primary entry point for Go project discovery.
189  * Used by drift detection and reports.
190  * |) *)
191 let parse_all dir =
192   let go_mod = match find_go_mod dir with
193   | Some path -> parse_go_mod path
194   | None -> None
195   in
196   let packages = discover_packages dir in
197   { go_mod; packages }
198
199 (* agent note (|
200  * WHAT: Get all package names in the project.
201  *
202  * WHY: Quick summary for reports and drift detection.
203  * |) *)
204 let all_package_names project =
205   List.map (fun p -> p.package_name) project.packages
206
207 (* agent note (|
208  * WHAT: Get all packages that are libraries (not cmd executables).
209  *
210  * WHY: Drift detection cross-references library packages against
211  * .borg spec sections. Executables are thin wrappers by convention.

```



```
211  * |) *)
212  let library_packages project =
213    List.filter (fun p -> p.kind <> Cmd) project.packages
214
215  (* agent note (|
216   *   WHAT: Get all executable packages (cmd/ directories).
217   *
218   *   WHY: Analogous to Dune_parse.all_executables.
219   * |) *)
220  let executable_packages project =
221    List.filter (fun p -> p.kind = Cmd) project.packages
```

73 lib/go_surface.ml

```

1  (** Extract the public surface of a Go package.
2
3      Go export rule: a name starting with an uppercase letter is exported.
4      This is enforced by the compiler -- no .mli equivalent needed.
5
6      Analogous to Surface.ml (OCaml). The key difference: Go has no
7      separate interface file, so export is determined by capitalization
8      alone, making extraction simpler. *)
9
10 type go_symbol_kind =
11   | Func
12   | Method
13   | Type
14   | Var
15   | Const
16
17 type go_symbol = {
18   name : string;
19   kind : go_symbol_kind;
20   file : string;
21   line : int;
22   is_exported : bool;
23 }
24
25 type go_package_surface = {
26   path : string;
27   package_name : string;
28   symbols : go_symbol list;
29   source : [ 'Go_inferred of string ];
30 }
31
32 (* agent note (|
33   *   WHAT: Check if a Go identifier is exported (starts with uppercase).
34   *   Includes Unicode letters and underscore handling per Go spec.
35   *
36   *   WHY: Go's export rule is purely syntactic: uppercase first char
37   *   means exported. This is simpler than OCaml's .mli system.
38   * |) *)
39 let is_exported_name name =
40   String.length name > 0 &&
41   let c = name.[0] in
42   c >= 'A' && c <= 'Z'
43
44 (* agent note (|
45   *   WHAT: Check if a character is valid in a Go identifier.
46   *   Letters, digits, underscore. No prime (') unlike OCaml.
47   *
48   *   WHY: Name extraction needs to know where the identifier ends.
49   * |) *)
50 let is_ident_char c =
51   (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') ||
52   (c >= '0' && c <= '9') || c = '_'
53
54 (* agent note (|
55   *   WHAT: Extract the identifier starting at position i in a string.
56   *   Returns the identifier and the position after it.
57   *
58   *   WHY: Shared helper for parsing func/type/var/const names
59   *   from Go source lines.
60   * |) *)
61 let extract_ident line i =
62   let len = String.length line in
63   let start = ref i in
64   while !start < len && line.[!start] = ' ' do incr start done;
65   if !start >= len then ("", !start)
66   else begin
67     let fin = ref !start in
68     while !fin < len && is_ident_char line.[!fin] do incr fin done;
69     ((* exempt: String.sub *) String.sub line !start (!fin - !start), !fin)
70   end
71
```

```

72 (* agent note (|
73 *   WHAT: Extract the function name from a Go func declaration.
74 *   Handles:
75 *     func Foo(...) ? (Foo, Func)
76 *     func (t T) Foo(...) ? (Foo, Method)
77 *     func bar(...) ? (bar, Func) (unexported)
78 *   Returns None for func(), func init(), func main().
79 *
80 *   WHY: Function extraction is the primary way to discover
81 *   a Go package's public surface.
82 * |) *)
83 let extract_func_decl line =
84   let trimmed = String.trim line in
85   let len = String.length trimmed in
86   (* Must start with "func " *)
87   if len < 5 then None
88   else if String.sub trimmed 0 5 <> "func " then None
89   else begin
90     let rest = (* exempt: String.sub *) String.sub trimmed 5 (len - 5) in
91     let rest_len = String.length rest in
92     (* Check for method receiver: func (r Receiver) Name *)
93     let name, kind =
94       if rest_len > 0 && rest.[0] = '(' then begin
95         (* Method with receiver -- skip past the closing *)
96         let depth = ref 1 in
97         let i = ref 1 in
98         while !i < rest_len && !depth > 0 do
99           match rest.[!i] with
100             | '(' -> incr depth; incr i
101             | ')' -> decr depth; incr i
102             | _ -> incr i
103         done;
104         (* Now extract the method name *)
105         let (n, _) = extract_ident rest !i in
106         (n, Method)
107       end
108       else begin
109         (* Regular function *)
110         let (n, _) = extract_ident rest 0 in
111         (n, Func)
112       end
113     in
114     (* Skip init, main -- they're not "exported" in the API sense *)
115     if name = "" || name = "init" || name = "main" then None
116     else if name = "TestMain" then None (* test helper *)
117     else Some (name, kind)
118   end
119
120 (* agent note (|
121 *   WHAT: Check if a line is a Test or Benchmark declaration.
122 *   Pattern: func TestXxx(t *testing.T) or func BenchmarkXxx(b *testing.B)
123 *   Also handles method form: func (s *Suite) TestXxx()
124 *
125 *   WHY: Test and benchmark functions are excluded from documentation
126 *   coverage enforcement, just like let%test in OCaml.
127 * |) *)
128 let is_test_or_benchmark line =
129   let trimmed = String.trim line in
130   String.length trimmed >= 9 &&
131   String.sub trimmed 0 5 = "func " &&
132   (let rest = String.sub trimmed 5 (String.length trimmed - 5) in
133    String.length rest >= 4 &&
134    (String.sub rest 0 4 = "Test" || String.sub rest 0 9 = "Benchmark"))
135
136 (* agent note (|
137 *   WHAT: Extract a type name from a Go type declaration.
138 *   Handles: type Foo struct, type Foo interface, type Foo ...
139 *   Skips: type alias inside functions, type without a name.
140 *
141 *   WHY: Type declarations are part of a package's public surface.
142 * |) *)
143 let extract_type_decl line =
144   let trimmed = String.trim line in

```

```

145 let len = String.length trimmed in
146 if len < 5 then None
147 else if String.sub trimmed 0 5 <> "type " then None
148 else begin
149   let rest = (* exempt: String.sub *) String.sub trimmed 5 (len - 5) |>
150     String.trim in
151   let (name, _after) = extract_ident rest 0 in
152   if name = "" then None
153   (* Skip "type ( ..." -- grouped type declarations *)
154   else if name.[0] = '(' then None
155   else Some name
156 end
157
158 (* agent note (|
159 *   WHAT: Extract a name from a Go var or const declaration.
160 *   Handles: var Foo = ..., var ( ... ), const Foo = ...
161 *   For grouped declarations (var/const blocks), returns the
162 *   first name only -- the block will be parsed line by line.
163 *
164 *   WHY: Var and const declarations at package level are part
165 *   of the public surface.
166 * |) *)
167 let extract_var_or_const_decl ~keyword line =
168   let trimmed = String.trim line in
169   let len = String.length trimmed in
170   let kw_len = String.length keyword + 1 in (* +1 for the space *)
171   if len <= kw_len then None
172   else if (* exempt: String.sub *) String.sub trimmed 0 kw_len <> (keyword
173     ^ " ") then None
174   else begin
175     let rest = (* exempt: String.sub *) String.sub trimmed kw_len (len -
176       kw_len) |> String.trim in
177     (* Skip grouped declarations: var ( ... *)
178     if String.length rest > 0 && rest.[0] = '(' then None
179     else begin
180       let (name, _) = extract_ident rest 0 in
181       if name = "" then None
182       else Some name
183     end
184   end
185 end
186
187 (* agent note (|
188 *   WHAT: Extract all exported symbols from a .go file.
189 *   Scans line by line for func, type, var, const declarations.
190 *   Skips test/benchmark functions, init, main.
191 *
192 *   WHY: The main entry point for surface extraction from a single
193 *   Go source file. Used by drift detection and reports.
194 * |) *)
195 let extract_file_surface path =
196   let symbols = ref [] in
197   let line_num = ref 0 in
198   let in_block_comment = ref false in
199   let in_group_decl = ref false in
200   let group_kind = ref Var in (* var or const block *)
201
202   try
203     (* exempt: open_in input_line *)
204     let ic = open_in path in
205     (try
206       while true do
207         let line = input_line ic in
208         incr line_num;
209         let trimmed = String.trim line in
210
211         (* Track block comments -- skip declarations inside /* */ *)
212         if !in_block_comment then begin
213           if String.length trimmed >= 2 &&
214             String.sub trimmed (String.length trimmed - 2) 2 = "*/" then
215             in_block_comment := false;
216         end
217       end
218     end
219     (* Track grouped var/const declarations *)

```

```

215     else if !in_group_decl then begin
216       if String.length trimmed >= 1 && trimmed.[0] = ')' then
217         in_group_decl := false
218       else begin
219         (* Extract names inside the group *)
220         let (name, _) = extract_ident trimmed 0 in
221         if name <> "" && is_exported_name name then
222           symbols := { name; kind = !group_kind; file = path;
223             line = !line_num; is_exported = true } ::
!symbols
224       end
225     end
226   else begin
227     (* Detect block comment start *)
228     if String.length trimmed >= 2 &&
229       String.sub trimmed 0 2 = "/*" then begin
230       (* Check if it closes on the same line *)
231       let content = String.sub trimmed 2 (String.length trimmed - 2)
in
232       if not (String.contains content '*' && String.contains content
233         '/') then
234         in_block_comment := true
235       end
236     (* Try each declaration type *)
237     else if String.starts_with ~prefix:"func " trimmed then begin
238       if not (is_test_or_benchmark trimmed) then
239         match extract_func_decl trimmed with
240         | Some (name, kind) ->
241           let is_exp = is_exported_name name in
242           symbols := { name; kind; file = path;
243             line = !line_num; is_exported = is_exp } ::
!symbols
244         | None -> ()
245       end
246     else if String.starts_with ~prefix:"type " trimmed then
247       match extract_type_decl trimmed with
248       | Some name ->
249         let is_exp = is_exported_name name in
250         symbols := { name; kind = Type; file = path;
251           line = !line_num; is_exported = is_exp } ::
!symbols
252       | None -> ()
253     else if String.starts_with ~prefix:"var " trimmed then
254       (match extract_var_or_const_decl ~keyword:"var" trimmed with
255       | Some name ->
256         let is_exp = is_exported_name name in
257         symbols := { name; kind = Var; file = path;
258           line = !line_num; is_exported = is_exp } ::
!symbols
259       | None ->
260         (* Check for grouped var declaration: var ( *)
261         let rest = String.sub trimmed 4 (String.length trimmed - 4)
262         |> String.trim in
263         if String.length rest > 0 && rest.[0] = '(' then begin
264           in_group_decl := true;
265           group_kind := Var
266         end)
267       else if String.starts_with ~prefix:"const " trimmed then
268         (match extract_var_or_const_decl ~keyword:"const" trimmed with
269         | Some name ->
270           let is_exp = is_exported_name name in
271           symbols := { name; kind = Const; file = path;
272             line = !line_num; is_exported = is_exp } ::
!symbols
273         | None ->
274           let rest = String.sub trimmed 6 (String.length trimmed - 6)
275           |> String.trim in
276           if String.length rest > 0 && rest.[0] = '(' then begin
277             in_group_decl := true;
278             group_kind := Const
279           end)
280         else ()
281       end

```

```

279     done
280     with End_of_file -> ());
281     close_in ic;
282     List.sort (fun a b ->
283       let c = String.compare a.name b.name in
284       if c <> 0 then c else compare a.line b.line
285     ) !symbols
286     with Sys_error _ -> []
287
288 (* agent note (|
289  *   WHAT: Recursively find .go files in a directory.
290  *   Skips vendor/, .git/, node_modules/, and testdata/.
291  *
292  *   WHY: Surface extraction needs to enumerate source files.
293  * |) *)
294 let find_go_files dir =
295   let rec find path =
296     try
297       let entries = Sys.readdir path in
298       Array.fold_left (fun acc entry ->
299         if entry = "vendor" || entry = ".git" ||
300           entry = "node_modules" || entry = "testdata" then acc
301         else if String.length entry > 0 && entry.[0] = '.' then acc
302         else
303           let full = Filename.concat path entry in
304           if Sys.is_directory full then find full @ acc
305           else if Filename.check_suffix entry ".go" &&
306             not (String.length entry >= 8 &&
307               String.sub entry (String.length entry - 8) 8 =
308                 "_test.go") then
309             full :: acc
310           else acc
311         ) [] entries
312     with Sys_error _ -> []
313   in
314   List.sort String.compare (find dir)
315
316 (* agent note (|
317  *   WHAT: Extract the full surface of a Go package directory.
318  *   Combines all exported symbols from all .go files.
319  *
320  *   WHY: Used by drift detection and reports to build the
321  *   complete picture of a package's public API.
322  * |) *)
323 let extract_package_surface dir_path =
324   let go_files = find_go_files dir_path in
325   let all_symbols = List.concat_map extract_file_surface go_files in
326   let package_name = Filename.basename dir_path in
327   { path = dir_path; package_name;
328     symbols = all_symbols;
329     source = 'Go_inferred dir_path' }
330
331 (* agent note (|
332  *   WHAT: Get just the exported names from a package surface.
333  *   Unexported symbols are included in the surface record
334  *   but filtered out here for the summary view.
335  *
336  *   WHY: Drift detection and reports typically only care about
337  *   the public API, not internal helpers.
338  * |) *)
339 let exported_names surface =
340   List.filter_map (fun s ->
341     if s.is_exported then Some s.name else None
342   ) surface.symbols
343
344 (* agent note (|
345  *   WHAT: Get just the names (exported and unexported) from a surface.
346  *
347  *   WHY: Full name listing for verbose reports.
348  * |) *)
349 let all_names surface =
350   List.map (fun s -> s.name) surface.symbols

```

74 lib/metrics/stats.ml

```

1  (** Code intelligence metrics - static analysis of source files.
2
3      Collects lines of code, function count, average function length,
4      and module metadata for every .ml file in the project. *)
5
6  (** A single source file's metrics *)
7  type file_metrics = {
8      path : string;
9      module_name : string;
10     lines : int;
11     functions : int;
12     avg_length : float;
13     exports : int;
14     documented : int; (** exports with doc comments *)
15     verified_count : int; (** bindings with verified agent note *)
16     doc_coverage : float; (** documented / exports ratio, 0.0-1.0 *)
17     has_mli : bool;
18     literacy : (float * float * float * float) option; (** avg story,
19         explain, teach, edge *)
20 }
21
22 (** Aggregated project metrics *)
23 type project_metrics = {
24     files : file_metrics list;
25     total_lines : int;
26     total_functions : int;
27     total_exports : int;
28     total_documented : int;
29     total_verified : int;
30     doc_coverage : float;
31     avg_func_length : float;
32     avg_literacy : (float * float * float * float) option; (** project-wide
33         averages *)
34 }
35
36 (** Count the lines in a string (excluding blank lines and comments) *)
37 let count_lines content =
38     let lines = String.split_on_char '\n' content in
39     List.length lines
40
41 (** Count top-level let bindings at column 0.
42     Reuses Surface.extract_toplevel_let_name logic inline to avoid
43     dep on Surface (which uses File_utils). *)
44 let count_functions content =
45     let lines = String.split_on_char '\n' content in
46     List.fold_left (fun acc line ->
47         let len = String.length line in
48         if len < 4 then acc
49         else if String.sub line 0 4 <> "let " then acc
50         else begin
51             let rest = (* exempt: String.sub *) String.sub line 4 (len - 4) in
52             let rest_len = String.length rest in
53             let i = ref 0 in
54             (* Skip 'rec ' *)
55             while !i + 3 < rest_len && (* exempt: String.sub *) String.sub rest
56                 !i 4 = "rec " do i := !i + 4 done;
57             while !i < rest_len && rest.[!i] = ' ' do incr i done;
58             if !i >= rest_len then acc
59             else begin
60                 let ch = rest.[!i] in
61                 if ch = '(' && !i + 1 < rest_len && rest.[!i + 1] = ')' then acc
62                 (* let () *)
63                 else if ch = '_' then acc (* let _ *)
64                 else acc + 1
65             end
66         end
67     ) 0 lines
68
69 (** Extract literacy scores from all borg-note comments in a file.
70     Returns averages per dimension (story, explain, teach, edge) *)
71 let compute_literacy ml_path =

```

```

68   try
69     let docs = Doc_detect.extract_file_docs ml_path in
70     let scored = List.filter_map (fun (bd : Doc_detect.binding_doc) ->
71       match bd.doc with
72       | Some (Doc_detect.Borg_note (_, _, Some score), _) -> Some score
73       | _ -> None
74     ) docs in
75     if scored = [] then None
76     else begin
77       let total = float_of_int (List.length scored) in
78       let sum f = List.fold_left (fun acc s -> acc + f s) 0 scored in
79       Some (
80         float_of_int (sum (fun s -> s.Doc_detect.story)) /. total,
81         float_of_int (sum (fun s -> s.Doc_detect.explain)) /. total,
82         float_of_int (sum (fun s -> s.Doc_detect.teach)) /. total,
83         float_of_int (sum (fun s -> s.Doc_detect.edge)) /. total
84       )
85     end
86   with _ -> None
87
88 (** Count bindings with doc comments in a file.
89    Reuses Doc_detect which handles nested comments and strings. *)
90 let count_documented ml_path =
91   try
92     let docs = Doc_detect.extract_file_docs ml_path in
93     List.length (List.filter Doc_detect.binding_has_doc docs)
94   with _ -> 0
95
96 (** Count bindings whose doc contains a verification marker.
97    Checks for "verified" or "verif" in doc comment text. *)
98 let count_verified ml_path =
99   try
100     let docs = Doc_detect.extract_file_docs ml_path in
101     let is_verified (bd : Doc_detect.binding_doc) =
102       match bd.doc with
103       | None -> false
104       | Some (doc_kind, _) ->
105         let text = match doc_kind with
106         | Doc_detect.Docstring s -> s
107         | Doc_detect.Borg_note (_, content, _) -> content
108         | Doc_detect.Borg_short s -> s
109         | Doc_detect.Exempt_marker -> ""
110         in
111         let lower = String.lowercase_ascii text in
112         let rec contains_at i sub =
113           let sub_len = String.length sub in
114           if i + sub_len > String.length lower then false
115           else if String.sub lower i sub_len = sub then true
116           else contains_at (i + 1) sub
117         in
118         contains_at 0 "verified" || contains_at 0 "verif"
119     in
120     List.length (List.filter is_verified docs)
121   with _ -> 0
122
123 (** Compute average function length for a file.
124    Heuristic: total lines / function count. *)
125 let compute_avg_length lines functions =
126   if functions = 0 then 0.0
127   else float_of_int lines /. float_of_int functions
128
129 (** Measure a single .ml file *)
130 let measure_file ml_path =
131   let content = File_utils.read_file ml_path in
132   let module_name = String.capitalize_ascii
133     (Filename.chop_extension (Filename.basename ml_path)) in
134   let lines = count_lines content in
135   let functions = count_functions content in
136   let avg_length = compute_avg_length lines functions in
137   let mli_path = Filename.chop_extension ml_path ^ ".mli" in
138   let has_mli = Sys.file_exists mli_path in
139   let surface = Surface.extract_surface ml_path in
140   let exports = List.length surface.exports in

```



```

141 let documented = count_documented ml_path in
142 let verified_count = count_verified ml_path in
143 let doc_coverage = if exports = 0 then 1.0
144   else float_of_int documented /. float_of_int exports in
145 let literacy = compute_literacy ml_path in
146 { path = ml_path; module_name; lines; functions; avg_length; exports;
    documented; verified_count; doc_coverage; has_mli; literacy }
147
148 (** Find all .ml files in a directory, excluding _build, .git, and test
    dirs *)
149 let find_ml_files dir =
150   let is_test_module name =
151     String.length name >= 5 &&
152     (String.sub name 0 5 = "test_" || String.sub name 0 5 = "Test_")
153   in
154   let rec find_path =
155     try
156       let entries = Sys.readdir path in
157       Array.fold_left (fun acc entry ->
158         if entry = "_build" || entry = ".git" || entry = "test" then acc
159         else
160           let full = Filename.concat path entry in
161           if Sys.is_directory full then find full @ acc
162           else if Filename.check_suffix entry ".ml" then begin
163             let mod_name = Filename.chop_extension entry in
164             if is_test_module mod_name then acc
165             else full :: acc
166           end
167         else acc
168       ) [] entries
169     with Sys_error _ -> []
170   in
171   List.sort String.compare (find dir)
172
173 (** Run stats on a directory *)
174 let run dir =
175   let ml_files = find_ml_files dir in
176   let metrics = List.map measure_file ml_files in
177   let total_lines = List.fold_left (fun acc (m : file_metrics) -> acc +
178     m.lines) 0 metrics in
179   let total_functions = List.fold_left (fun acc (m : file_metrics) -> acc +
180     m.functions) 0 metrics in
181   let total_exports = List.fold_left (fun acc (m : file_metrics) -> acc +
182     m.exports) 0 metrics in
183   let total_documented = List.fold_left (fun acc (m : file_metrics) -> acc +
184     m.documented) 0 metrics in
185   let total_verified = List.fold_left (fun acc (m : file_metrics) -> acc +
186     m.verified_count) 0 metrics in
187   let doc_coverage = if total_exports = 0 then 1.0
188     else float_of_int total_documented /. float_of_int total_exports in
189   let avg_func_length = compute_avg_length total_lines total_functions in
190   let avg_literacy =
191     let all_scores = List.filter_map (fun (m : file_metrics) -> m.literacy)
192       metrics in
193     if all_scores = [] then None
194     else begin
195       let total = float_of_int (List.length all_scores) in
196       Some (
197         List.fold_left (fun acc (s, _, _, _) -> acc +. s) 0.0 all_scores /.
198         total,
199         List.fold_left (fun acc (_, e, _, _) -> acc +. e) 0.0 all_scores /.
200         total,
201         List.fold_left (fun acc (_, _, t, _) -> acc +. t) 0.0 all_scores /.
202         total,
203         List.fold_left (fun acc (_, _, _, g) -> acc +. g) 0.0 all_scores /.
204         total
205       )
206     end
207   in
208   { files = metrics; total_lines; total_functions; total_exports;
209     total_documented; total_verified; doc_coverage; avg_func_length;
210     avg_literacy }

```

75 lib/generate/agent.ml

```

1  (** Agent-driven drift analysis via pi in print mode.
2
3      Uses pi -p (single-shot, process and exit) to send structured
4      analysis prompts and parse sexp findings. No RPC protocol -- a
5      simple stdin/stdout pipe through temp files. No timeout -- runs
6      until the model completes.
7
8      Fallback: BORGE_AGENT_CMD env var for custom LLM invocation. *)
9
10 open Borge_lang
11
12 (** {1 Prompt assembly} *)
13
14 let build_prompt borg_path surfaces findings =
15   let buf = Buffer.create 1024 in
16   Buffer.add_string buf "You are analyzing a codebase against its
17     specification.\n";
18   Buffer.add_string buf "For each section below, compare the spec
19     description with\n";
20   Buffer.add_string buf "the source code and return findings.\n\n";
21   Buffer.add_string buf (Printf.sprintf "Spec file: %s\n" borg_path);
22   (try
23     let content = File_utils.read_file borg_path in
24     Buffer.add_string buf "\n--- SPEC ---\n";
25     Buffer.add_string buf content;
26     Buffer.add_string buf "\n--- END SPEC ---\n"
27   with _ -> ());
28   with _ -> ()
29   if surfaces <> [] then begin
30     Buffer.add_string buf "\n--- MODULE SURFACES ---\n";
31     List.iter (fun (s : Surface.module_surface) ->
32       Buffer.add_string buf (Printf.sprintf "Module %s (%s):\n exports:
33         %s\n"
34           s.module_name s.path (String.concat " " s.exports))
35     ) surfaces;
36     Buffer.add_string buf "--- END SURFACES ---\n"
37   end;
38   if findings <> [] then begin
39     Buffer.add_string buf "\n--- EXISTING STATIC FINDINGS ---\n";
40     List.iter (fun (f : Meta.finding) ->
41       Buffer.add_string buf (Printf.sprintf " [%s/%s] %s\n"
42         (Meta.string_of_source f.Meta.source)
43         (Meta.string_of_confidence f.Meta.confidence)
44         (match f.Meta.detail with Some d -> d | None ->
45           Meta.string_of_finding_type f.Meta.ft_type))
46     ) findings;
47     Buffer.add_string buf "--- END FINDINGS ---\n"
48   end;
49   Buffer.add_string buf "\nReturn ONLY a sexp list of findings. Each
50     finding is:\n";
51   Buffer.add_string buf "(finding TYPE\n";
52   Buffer.add_string buf "  (section SECTION_NAME)\n";
53   Buffer.add_string buf "  (detail EXPLANATION)\n";
54   Buffer.add_string buf "  (confidence LEVEL))\n";
55   Buffer.add_string buf "Valid types: partial-implementation,
56     status-mismatch,\n";
57   Buffer.add_string buf "              missing-export, extra-export,
58     phantom-spec\n";
59   Buffer.add_string buf "Valid confidence levels: high, medium, low\n";
60   Buffer.add_string buf "If no findings, return: (findings)\n";
61   Buffer.add_string buf "Example:\n";
62   Buffer.add_string buf "(findings\n";
63   Buffer.add_string buf "  (finding partial-implementation\n";
64   Buffer.add_string buf "    (section drift)\n";
65   Buffer.add_string buf "    (detail \"spec mentions undocumented-bindings
66     check but it was removed\")\n";
67   Buffer.add_string buf "    (confidence high)))\n";
68   Buffer.contents buf
69
70 (** {1 Agent output parsing} *)
71
72 let parse_agent_response response timestamp =

```

```

64   try
65     let file = Parse.parse_file response in
66     let findings = ref [] in
67     let rec walk = function
68       | [] -> ()
69       | { Ast.node; _ } :: rest ->
70         (match node with
71          | Ast.List (_, Ast.Atom (_, "findings") :: items) ->
72            List.iter (fun item ->
73              match item with
74              | Ast.List (_, Ast.Atom (_, "finding") :: Ast.Atom (_,
75                type_str) :: fields) ->
76                let ft_type = match type_str with
77                  | "partial-implementation" ->
78                    Meta.Partial_implementation
79                  | "status-mismatch" -> Meta.Status_mismatch
80                  | "missing-export" -> Meta.Missing_export
81                  | "extra-export" -> Meta.Extra_export
82                  | "phantom-spec" -> Meta.Pantom_spec
83                  | _ -> Meta.Partial_implementation
84                in
85                let section = List.find_map (function
86                  | Ast.List (_, Ast.Atom (_, "section") :: Ast.Atom (_,
87                    s) :: _) -> Some s
88                  | _ -> None
89                ) fields in
90                let detail = List.find_map (function
91                  | Ast.List (_, Ast.Atom (_, "detail") :: Ast.String
92                    (_, Ast.Quoted q) :: _) ->
93                    Some q.Ast.q_content
94                  | Ast.List (_, Ast.Atom (_, "detail") :: Ast.String
95                    (_, Ast.Verbatim v) :: _) ->
96                    Some v.Ast.v_content
97                  | _ -> None
98                ) fields in
99                let confidence =
100                  let c_opt = List.find_map (function
101                    | Ast.List (_, Ast.Atom (_, "confidence") ::
102                      Ast.Atom (_, c) :: _) ->
103                      (match c with
104                       | "high" -> Some Meta.High
105                       | "medium" -> Some Meta.Medium
106                       | "low" -> Some Meta.Low
107                       | _ -> Some Meta.Low)
108                    | _ -> None
109                  ) fields in
110                  match c_opt with Some c -> c | None -> Meta.Low
111                in
112                findings := {
113                  Meta.ft_type;
114                  Meta.section;
115                  Meta.module_ = None;
116                  Meta.file = None;
117                  Meta.export = None;
118                  Meta.detail;
119                  Meta.spec_status = None;
120                  Meta.actual_status = None;
121                  Meta.confidence;
122                  Meta.source = Meta.Agent;
123                  Meta.at = timestamp;
124                } :: !findings
125              | _ -> ()
126            ) items
127          | _ -> ()
128        );
129     walk rest
130   in
131     walk file.Ast.top_level;
132     List.rev !findings
133   with _ ->
134     []
135 (** {1 Pi -p mode integration} *)

```

```

131
132 let find_pi () =
133   try
134     let p = Sys.getenv "BORGE_PI_PATH" in
135     if Sys.file_exists p then Some p else None
136   with Not_found ->
137   try
138     let ic = Unix.open_process_in "command -v pi 2>/dev/null" in
139     let path = String.trim (input_line ic) in
140     let _ = Unix.close_process_in ic in
141     if String.length path > 0 && Sys.file_exists path then Some path
142     else None
143   with _ -> None
144
145 let extract_sexp_from_response text =
146   (* Scan the response for a sexp starting with (findings or (finding.
147   Returns the substring from the first (findings) we find, or
148   the full text if no clear starting point. *)
149   let len = String.length text in
150   let rec find start =
151     if start >= len then
152       text (* no sexp found, return whole thing for parse attempt *)
153     else if start + 8 <= len &&
154       (* exempt: String.sub *) String.sub text start 8 = "(findings"
155       then
156         (* exempt: String.sub *) String.sub text start (len - start)
157       else if start + 8 <= len &&
158         (* exempt: String.sub *) String.sub text start 8 = "(finding "
159         then
160           (* exempt: String.sub *) String.sub text start (len - start)
161         else
162           find (start + 1)
163   in
164   find 0
165
166 let run_pi_print prompt =
167   match find_pi () with
168   | None -> None
169   | Some pi_path ->
170     let tmp_prompt = Filename.temp_file "borge_prompt" ".txt" in
171     let tmp_out = Filename.temp_file "borge_response" ".txt" in
172     let oc_prompt = open_out tmp_prompt in
173     output_string oc_prompt prompt;
174     close_out oc_prompt;
175     Printf.eprintf " pi: running semantic analysis...";
176     flush stderr;
177     let cmd = Printf.sprintf "%s -p --no-session --no-tools < '%s' > '%s' 2>&1"
178       pi_path tmp_prompt tmp_out in
179     let start_time = Unix.gettimeofday () in
180     let pid = Unix.fork () in
181     if pid = 0 then begin
182       let _ = Sys.command cmd in
183       exit 0
184     end else begin
185       let running = ref true in
186       while !running do
187         match Unix.waitpid [Unix.WNOHANG] pid with
188         | 0, _ -> Printf.eprintf "."; flush stderr; Unix.sleep 1
189         | _, _ -> running := false
190       done;
191       let elapsed = Unix.gettimeofday () -. start_time in
192       Printf.eprintf " done (%.1fs)\n" elapsed;
193       let response =
194         try File_utils.read_file tmp_out with _ -> "" in
195       Sys.remove tmp_prompt;
196       Sys.remove tmp_out;
197       if String.length response > 0 then begin
198         let sexp_part = extract_sexp_from_response response in
199         if sexp_part != response then
200           Printf.eprintf " pi: extracted sexp from response\n";
201           Some sexp_part
202         end else begin

```

```

201     Printf.eprintf " pi: no response received\n";
202     None
203   end
204 end
205
206 (** {1 Fallback: BORGE_AGENT_CMD} *)
207
208 let get_agent_cmd () =
209   try Some (Sys.getenv "BORGE_AGENT_CMD")
210   with Not_found -> None
211
212 let run_agent_cmd cmd prompt =
213   Printf.eprintf " Using BORGE_AGENT_CMD (%s)...\n" cmd;
214   let tmp_in = Filename.temp_file "borge_agent" ".txt" in
215   let tmp_out = Filename.temp_file "borge_agent" ".out" in
216   let oc = open_out tmp_in in
217   output_string oc prompt;
218   close_out oc;
219   let ret = Sys.command (Printf.sprintf "%s < %s > %s 2>&1" cmd tmp_in
220     tmp_out) in
221   let response = if ret = 0 then
222     (try File_utils.read_file tmp_out with _ -> "") else "" in
223   Sys.remove tmp_in;
224   Sys.remove tmp_out;
225   if ret = 0 then Some response else None
226
227 (** {1 Main agent entry point} *)
228
229 let run_agent dir =
230   Printf.eprintf " Running static analysis...\n";
231   let timestamp = Meta.current_timestamp () in
232   let meta = Drift.generate_findings dir in
233   let n_static = List.length (List.filter (fun (f : Meta.finding) ->
234     f.Meta.source = Meta.Static) meta.Meta.findings) in
235   Printf.eprintf " Static: %d findings. Starting agent...\n" n_static;
236   let borg_files = File_utils.find_borg_files dir in
237   let dune_files = Dune_parse.parse_all dir in
238   let lib_dune_files = List.filter (fun df ->
239     let d = Filename.dirname df.Dune_parse.path in
240     String.length d >= 5 && String.sub d 0 5 = "./lib"
241   ) dune_files in
242   let lib_modules = List.concat_map (fun df ->
243     let dir = Filename.dirname df.Dune_parse.path in
244     List.concat_map (function
245       | Dune_parse.Library lib ->
246         List.map (fun m -> (dir, m)) lib.Dune_parse.modules
247       | _ -> []
248     ) df.Dune_parse.stanzas
249   ) lib_dune_files in
250   let all_surfaces = List.filter_map (fun (dir, mod_name) ->
251     let ml_path = Filename.concat dir (mod_name ^ ".ml") in
252     if Sys.file_exists ml_path then
253       Some (Surface.extract_surface ml_path)
254     else None
255   ) lib_modules in
256   let agent_findings =
257     let root_borg = List.filter (fun p ->
258       Filename.basename p = "borge.borg") borg_files in
259     List.concat_map (fun borg_path ->
260       let prompt = build_prompt borg_path all_surfaces meta.Meta.findings in
261       let response = match run_pi_print prompt with
262       | Some r -> Some r
263       | None ->
264         (match get_agent_cmd () with
265         | Some cmd -> run_agent_cmd cmd prompt
266         | None ->
267           Printf.eprintf " No LLM available. Install pi or set
268             BORGE_AGENT_CMD.\n";
269           None)
270     in
271     match response with
272     | Some r -> parse_agent_response r timestamp
273     | None -> []

```

```
272   ) root_borg
273   in
274   Printf.eprintf "  Agent: %d new findings.\n" (List.length agent_findings);
275   let merged_findings = Meta.merge_agent_findings meta.Meta.findings
       agent_findings in
276   let updated_meta = { meta with Meta.findings = merged_findings } in
277   List.iter (fun borg_path ->
278     if Filename.basename borg_path = "borge.borg" then begin
279       let meta_path = Meta.meta_path_of borg_path in
280       let project_name = try
281         let input = File_utils.read_file borg_path in
282         let file = Parse.parse_file input in
283         Spec.project_name file
284       with _ -> Some "borge"
285       in
286       let per_meta = { updated_meta with
287         Meta.project_name = Option.value project_name ~default:"borge" } in
288       Meta.write_meta meta_path per_meta
289     end
290   ) borg_files;
291   (updated_meta, agent_findings)
```

76 lib/generate/generate.ml

```

1 (** Generate spec sections and code from specs.
2
3     Two-step pipeline:
4     1. borge generate spec -- turns a todo into a .borg section
5     2. borge generate code -- implements a planned section from its spec
6
7     Both use pi -p (single-shot) via Agent.run_pi_print. *)
8
9 open Borge_lang
10
11 (** {1 Generate spec: todo ? .borg section} *)
12
13 let build_spec_prompt todo_text project_name convention =
14   let buf = Buffer.create 512 in
15   Buffer.add_string buf "You are a specification writer.\n";
16   Buffer.add_string buf "Given a task description, produce a .borg
17     section.\n\n";
18   Buffer.add_string buf "Rules:\n";
19   Buffer.add_string buf "- Use (section NAME) as the top-level form\n";
20   Buffer.add_string buf "- Include (doc \"...\") with a one-line
21     description\n";
22   Buffer.add_string buf "- Set (status planned)\n";
23   Buffer.add_string buf "- Add (details (doc (| ... |))) with
24     implementation notes\n";
25   Buffer.add_string buf "- Add (subsection ...) for each logical unit if
26     needed\n";
27   Buffer.add_string buf "- Return ONLY the sexp, no markdown fences, no
28     commentary\n\n";
29   Buffer.add_string buf (Printf.sprintf "Project: %s\n" project_name);
30   (match convention with
31   | Some c -> Buffer.add_string buf (Printf.sprintf "Convention: %s\n" c)
32   | None -> ());
33   Buffer.add_string buf (Printf.sprintf "\nTask:\n%s\n" todo_text);
34   Buffer.add_string buf "\n\nProduce a .borg section:\n";
35   Buffer.contents buf
36
37 (* agent note (|
38 *   WHAT: Generate a .borg spec section from a todo description
39 *   using the LLM. Parses the result and returns it as a string.
40 *
41 *   WHY: The borge spec --spec command uses this to turn a human
42 *   todo into structured borg specification.
43 * |) *)
44 let generate_spec todo_text dir =
45   let project_name =
46     let roots = Project.find_roots dir in
47     match roots with
48     | [root] ->
49       (try
50         let input = File_utils.read_file root in
51         let file = Parse.parse_file input in
52         (match Spec.project_name file with Some n -> n | None -> "unknown")
53       with _ -> "unknown")
54     | _ -> "unknown"
55   in
56   let prompt = build_spec_prompt todo_text project_name (Some
57     (Convention.name (Convention.resolve dir))) in
58   let response = Agent.run_pi_print prompt in
59   match response with
60   | None ->
61     Printf.eprintf "Error: no LLM response. Install pi or set
62       BORGE_AGENT_CMD.\n";
63     None
64   | Some text ->
65     (* Try to extract just the sexp from the response *)
66     let sexp = Agent.extract_sexp_from_response text in
67     (* Validate it parses *)
68     try
69       let _ = Parse.parse_file sexp in
70       Some sexp
71     with _ ->

```

```

65     Printf.eprintf "Warning: LLM output is not valid sexp. Raw
        output:\n%s\n" text;
66     Some text
67
68 (** {1 Generate code: spec ? implementation} *)
69
70 (* agent note (|
71  *   WHAT: Generate implementation code from a spec section using LLM.
72  *   Takes the section name, path to borg file, spec text, and surfaces.
73  *
74  *   WHY: The borge spec --code command uses this to have an agent
75  *   implement a planned section.
76  * |) *)
77
78 let build_code_prompt section_name borg_path spec_text surfaces =
79   let buf = Buffer.create 1024 in
80   Buffer.add_string buf "You are implementing a planned section of a
        specification.\n";
81   Buffer.add_string buf "Read the spec and write the implementation
        code.\n\n";
82   Buffer.add_string buf (Printf.sprintf "Section: %s\n" section_name);
83   Buffer.add_string buf (Printf.sprintf "Spec file: %s\n\n" borg_path);
84   Buffer.add_string buf "--- SPEC SECTION ---\n";
85   Buffer.add_string buf spec_text;
86   Buffer.add_string buf "\n--- END SPEC ---\n\n";
87   if surfaces <> [] then begin
88     Buffer.add_string buf "--- EXISTING MODULE SURFACES ---\n";
89     List.iter (fun (s : Surface.module_surface) ->
90       Buffer.add_string buf (Printf.sprintf "Module %s (%s): %s\n"
91         s.module_name s.path (String.concat " " s.exports))
92     ) surfaces;
93     Buffer.add_string buf "--- END SURFACES ---\n\n"
94   end;
95   Buffer.add_string buf "Rules:\n";
96   Buffer.add_string buf "- Write the implementation in OCaml\n";
97   Buffer.add_string buf "- Follow existing project conventions\n";
98   Buffer.add_string buf "- Add the module to the dune (modules ...)
        stanza\n";
99   Buffer.add_string buf "- Make sure dune build passes\n";
100  Buffer.add_string buf "- Return ONLY the code, no markdown fences\n";
101  Buffer.contents buf
102
103 (** Find a section by name in a parsed .borg file *)
104 let find_section_text borg_path section_name =
105   try
106     let input = File_utils.read_file borg_path in
107     let len = String.length input in
108     let start =
109       let target = Printf.sprintf "(section %s" section_name in
110       let rec search i =
111         if i + String.length target > len then raise Not_found
112         else if String.sub input i (String.length target) = target then i
113         else search (i + 1)
114       in
115       search 0
116     in
117     (* Find the matching closing paren by walking from start *)
118     let depth = ref 0 in
119     let fin = ref len in
120     for i = start to len - 1 do
121       if !fin = len then () (* already found end *)
122       else begin
123         if input.[i] = '(' then incr depth
124         else if input.[i] = ')' then begin
125           decr depth;
126           if !depth = 0 then fin := i + 1
127         end
128       end
129     done;
130     String.trim ((* exempt: String.sub *) String.sub input start (!fin -
        start))
131   with _ ->
132     Printf.sprintf "(section %s (status planned))" section_name

```



```

133
134 let generate_code section_name dir =
135   let borg_files = File_utils.find_borg_files dir in
136   let dune_file = Dune_parse.parse_all dir in
137   let lib_dune_files = List.filter (fun df ->
138     let d = Filename.dirname df.Dune_parse.path in
139     String.length d >= 5 && String.sub d 0 5 = "./lib"
140   ) dune_file in
141   let lib_modules = List.concat_map (fun df ->
142     let dir = Filename.dirname df.Dune_parse.path in
143     List.concat_map (function
144       | Dune_parse.Library lib ->
145         List.map (fun m -> (dir, m)) lib.Dune_parse.modules
146       | _ -> []
147     ) df.Dune_parse.stanzas
148   ) lib_dune_files in
149   let all_surfaces = List.filter_map (fun (dir, mod_name) ->
150     let ml_path = Filename.concat dir (mod_name ^ ".ml") in
151     if Sys.file_exists ml_path then
152       Some (Surface.extract_surface ml_path)
153     else None
154   ) lib_modules in
155   let matching_borg = List.filter (fun p ->
156     try
157       let input = File_utils.read_file p in
158       let file = Parse.parse_file input in
159       let found = ref false in
160       List.iter (fun { Ast.node; _ } ->
161         (match node with
162          | Ast.List (_, Ast.Atom (_, "section") :: Ast.Atom (_, name) :: _)
163            ->
164              if name = section_name then found := true
165              | _ -> ());
166          ) file.Ast.top_level;
167       !found
168     with _ -> false
169   ) borg_files in
170   match matching_borg with
171   | [] ->
172     Printf.eprintf "Error: no .borg file contains section '%s'\n"
173       section_name;
174     None
175   | borg_path :: _ ->
176     let spec_text = find_section_text borg_path section_name in
177     let prompt = build_code_prompt section_name borg_path spec_text
178     all_surfaces in
179     let response = Agent.run_pi_print prompt in
180     match response with
181     | None ->
182       Printf.eprintf "Error: no LLM response. Install pi or set
183         BORGE_AGENT_CMD.\n";
184       None
185     | Some text ->
186       Printf.printf "%s\n" text;
187       Some text

```

77 lib/generate/review__quality.ml

```

1  (** LLM-powered code quality review.
2
3      Compare code against its spec section and generate quality findings:
4      does the implementation match, are exports documented, are there
5      dead functions, is error handling present? *)
6
7  open Borge_lang
8
9  type review_finding = {
10     section : string;
11     file : string;
12     checks : string list; (** which checks passed *)
13     issues : string list;  (** which checks failed with detail *)
14     quality : [ 'high | 'medium | 'low ];
15 }
16
17 let build_review_prompt section_name borg_path spec_text source_path
18     source_content =
19   let buf = Buffer.create 1024 in
20   Buffer.add_string buf "You are reviewing code quality against a
21     specification.\n";
22   Buffer.add_string buf "Compare the spec section with the source code and
23     assess:\n\n";
24   Buffer.add_string buf "1. Does every spec-described feature exist in
25     code?\n";
26   Buffer.add_string buf "2. Are all exports documented? (check .mli or
27     docstrings)\n";
28   Buffer.add_string buf "3. Are there dead functions (unused exports)?\n";
29   Buffer.add_string buf "4. Is error handling present where the spec says
30     'returns error'?\n";
31   Buffer.add_string buf "5. Is the literate description accurate?\n\n";
32   Buffer.add_string buf (Printf.sprintf "Section: %s\n" section_name);
33   Buffer.add_string buf (Printf.sprintf "Spec file: %s\n\n" borg_path);
34   Buffer.add_string buf "--- SPEC ---\n";
35   Buffer.add_string buf spec_text;
36   Buffer.add_string buf "\n--- END SPEC ---\n\n";
37   Buffer.add_string buf (Printf.sprintf "Source file: %s\n" source_path);
38   Buffer.add_string buf "--- CODE ---\n";
39   Buffer.add_string buf source_content;
40   Buffer.add_string buf "\n--- END CODE ---\n\n";
41   Buffer.add_string buf "Return ONLY a sexp review:\n";
42   Buffer.add_string buf "(review\n";
43   Buffer.add_string buf "  (section SECTION_NAME)\n";
44   Buffer.add_string buf "  (file FILE_PATH)\n";
45   Buffer.add_string buf "  (quality high|medium|low)\n";
46   Buffer.add_string buf "  (checks (pass \"description\") ...) \n";
47   Buffer.add_string buf "  (issues (fail \"description\") ...))\n\n";
48   Buffer.add_string buf "If no issues, return:\n";
49   Buffer.add_string buf "(review (section SECTION) (file FILE) (quality
50     high) (checks) (issues))\n";
51   Buffer.contents buf
52
53 let review_section dir borg_path section_name =
54   (* Find the section text in the .borg file *)
55   let input = File_utils.read_file borg_path in
56   let len = String.length input in
57   let target = Printf.sprintf "(section %s" section_name in
58   let start =
59     let rec search i =
60       if i + String.length target > len then 0
61       else if String.sub input i (String.length target) = target then i
62       else search (i + 1)
63     in search 0
64   in
65   let spec_text =
66     if start = 0 then "(section unknown)"
67     else begin
68       (* Find the next section start after this one *)
69       let next_section =
70         let rec search i =
71           if i + 8 > len then len

```

```

65         else if i > start + String.length target &&
66             (* exempt: String.sub *) String.sub input i 8 =
67             "(section" then i
68                 else search (i + 1)
69                 in search (start + 1)
70                 in
71                 String.trim ((* exempt: String.sub *) String.sub input start
72                     (next_section - start))
73             end
74         in
75         (* Find the source file for this section *)
76         let ml_files = Stats.find_ml_files dir in
77         let matching_ml = List.filter (fun p ->
78             let base = Filename.chop_extension (Filename.basename p) in
79             String.lowercase_ascii base = String.lowercase_ascii section_name ||
80             String.lowercase_ascii base = String.lowercase_ascii (section_name ^
81                 "_cmd")
82         ) ml_files in
83         match matching_ml with
84         | ml_path :: _ ->
85             let source_content = File_utils.read_file ml_path in
86             let prompt = build_review_prompt section_name borg_path spec_text
87             ml_path source_content in
88             let response = Agent.run_pi_print prompt in
89             (match response with
90             | Some text -> Some (Agent.extract_sexp_from_response text)
91             | None -> None),
92             ml_path
93         | [] ->
94             Printf.eprintf " No source file found for section '%s'\n" section_name;
95             None, ""
96
97 (** Run quality review on all implemented sections *)
98 let run dir =
99     let borg_files = File_utils.find_borg_files dir in
100     let findings = ref [] in
101     List.iter (fun borg_path ->
102         try
103             let input = File_utils.read_file borg_path in
104             let file = Parse.parse_file input in
105             List.iter (fun { Ast.node; _ } ->
106                 (match node with
107                 | Ast.List (_, Ast.Atom (_, "section") :: Ast.Atom (_, name) ::
108                     Ast.List (_, Ast.Atom (_, "status") :: Ast.Atom (_,
109                         status) :: _) :: _) ->
110                     when status = "implemented" ->
111                         let result, ml_path = review_section dir borg_path name in
112                         (match result with
113                         | Some sexp ->
114                             findings := (name, ml_path, sexp) :: !findings
115                         | None -> ())
116                     | _ -> ())
117             ) file.Ast.top_level
118         with _ -> ()
119     ) borg_files;
120     List.rev !findings

```

78 lib/output/json_out.ml

```

1  (** JSON serialization for borge result types using yojson.
2
3      Each command produces typed result data. This module converts
4      those types to yojson for the --json flag. *)
5
6
7  (* ?? Report ????????????????????????????????????????????????????????? *)
8
9  (* exempt doc: simple JSON converter - name is self-documenting *)
10 let file_stats (r : Report.file_stats) =
11   'Assoc [
12     "path", 'String r.path;
13     "project_name", (match r.project_name with Some n -> 'String n | None
14       -> 'Null);
15     "implemented", 'Int r.implemented;
16     "verified", 'Int r.verified;
17     "in_progress", 'Int r.in_progress;
18     "planned", 'Int r.planned;
19   ]
20
21 (* exempt doc: simple JSON converter - name is self-documenting *)
22 let totals (t : Report.totals) =
23   'Assoc [
24     "implemented", 'Int t.implemented;
25     "verified", 'Int t.verified;
26     "in_progress", 'Int t.in_progress;
27     "planned", 'Int t.planned;
28   ]
29
30 (* exempt doc: simple JSON converter - name is self-documenting *)
31 let report (r : Report.result) =
32   'Assoc [
33     "files", 'List (List.map file_stats r.files);
34     "totals", totals r.totals;
35     "orphans", 'List (List.map (fun s -> 'String s) r.orphans);
36   ]
37
38 (* ?? Check ????????????????????????????????????????????????????????? *)
39
40 (* exempt doc: simple pattern match to JSON - name is self-documenting *)
41 let file_result = function
42   | Check.Ok { path; project_name; form_count } ->
43     'Assoc [
44       "status", 'String "ok";
45       "path", 'String path;
46       "project_name", 'String project_name;
47       "form_count", 'Int form_count;
48     ]
49   | Check.Error { path; message } ->
50     'Assoc [
51       "status", 'String "error";
52       "path", 'String path;
53       "message", 'String message;
54     ]
55
56 (* exempt doc: simple JSON converter - name is self-documenting *)
57 let check (r : Check.result) =
58   'Assoc [
59     "files", 'List (List.map file_result r.files);
60     "warnings", 'List (List.map (function
61       | Check.Orphan path -> 'Assoc [ "type", 'String "orphan"; "path",
62         'String path ]
63     ) r.warnings);
64     "passed", 'Int r.passed;
65     "failed", 'Int r.failed;
66   ]
67
68 (* ?? Lint ????????????????????????????????????????????????????????? *)
69
70 (* exempt doc: large pattern match to JSON - purpose clear from context *)
71 let lint_issue = function

```

```

70 | Lint.Invalid_status { path; value; line; col } ->
71   'Assoc [
72     "type", 'String "invalid_status";
73     "severity", 'String "error";
74     "path", 'String path;
75     "value", 'String value;
76     "line", 'Int line;
77     "col", 'Int col;
78   ]
79 | Lint.Duplicate_project_name { name; paths } ->
80   'Assoc [
81     "type", 'String "duplicate_project_name";
82     "severity", 'String "error";
83     "name", 'String name;
84     "paths", 'List (List.map (fun s -> 'String s) paths);
85   ]
86 | Lint.Orphaned_file { path } ->
87   'Assoc [
88     "type", 'String "orphaned_file";
89     "severity", 'String "error";
90     "path", 'String path;
91   ]
92 | Lint.Invalid_verify_method { path; method_; line } ->
93   'Assoc [
94     "type", 'String "invalid_verify_method";
95     "severity", 'String "error";
96     "path", 'String path;
97     "method", 'String method_;
98     "line", 'Int line;
99   ]
100 | Lint.No_inline_on_root { path } ->
101   'Assoc [
102     "type", 'String "no_inline_on_root";
103     "severity", 'String "warning";
104     "path", 'String path;
105   ]
106 | Lint.Unknown_comment_type { path; type_name; line } ->
107   'Assoc [
108     "type", 'String "unknown_comment_type";
109     "severity", 'String "warning";
110     "path", 'String path;
111     "comment_type", 'String type_name;
112     "line", 'Int line;
113   ]
114 | Lint.Missing_comment_value { path; author; type_name; line } ->
115   'Assoc [
116     "type", 'String "missing_comment_value";
117     "severity", 'String "warning";
118     "path", 'String path;
119     "author", 'String author;
120     "comment_type", (match type_name with Some t -> 'String t | None ->
121       'Null);
122     "line", 'Int line;
123   ]
124 | Lint.Deleted_human_comment { path; author; line } ->
125   'Assoc [
126     "type", 'String "deleted_human_comment";
127     "severity", 'String "warning";
128     "path", 'String path;
129     "author", 'String author;
130     "line", 'Int line;
131   ]
132 | Lint.Pending_response { path; author; line; question } ->
133   'Assoc [
134     "type", 'String "pending_response";
135     "severity", 'String "warning";
136     "path", 'String path;
137     "author", 'String author;
138     "line", 'Int line;
139     "question", 'String question;
140   ]
141 | Lint.Db_validation { path; severity; message } ->
142   'Assoc [

```

```

142     "type", 'String "db_validation";
143     "severity", 'String (match severity with 'Error -> "error" | 'Warning
-> "warning");
144     "path", 'String path;
145     "message", 'String message;
146   ]
147 | Lint.Ui_validation { path; severity; message } ->
148   'Assoc [
149     "type", 'String "ui_validation";
150     "severity", 'String (match severity with 'Error -> "error" | 'Warning
-> "warning");
151     "path", 'String path;
152     "message", 'String message;
153   ]
154 | Lint.Undocumented_binding { path; name; line } ->
155   'Assoc [
156     "type", 'String "undocumented_binding";
157     "path", 'String path;
158     "name", 'String name;
159     "line", 'Int line;
160   ]
161 | Lint.Stale_doc_comment { path; name; line } ->
162   'Assoc [
163     "type", 'String "stale_doc_comment";
164     "path", 'String path;
165     "name", 'String name;
166     "line", 'Int line;
167   ]
168 | Lint.Inserted_human_comment { path; author; line } ->
169   'Assoc [
170     "type", 'String "inserted_human_comment";
171     "severity", 'String "warning";
172     "path", 'String path;
173     "author", 'String author;
174     "line", 'Int line;
175   ]
176 | Lint.Unsafe_call { path; line; call; severity; suggestion } ->
177   'Assoc [
178     "type", 'String "unsafe_call";
179     "severity", 'String (match severity with 'Error -> "error" | 'Warning
-> "warning");
180     "path", 'String path;
181     "line", 'Int line;
182     "call", 'String call;
183     "suggestion", 'String suggestion;
184   ]
185 | Lint.Missing_literacy_score { path; name; line } ->
186   'Assoc [
187     "type", 'String "missing_literacy_score";
188     "severity", 'String "warning";
189     "path", 'String path;
190     "name", 'String name;
191     "line", 'Int line;
192   ]
193 | Lint.Implausible_literacy_score { path; name; line; dimension; claimed;
reason } ->
194   'Assoc [
195     "type", 'String "implausible_literacy_score";
196     "severity", 'String "warning";
197     "path", 'String path;
198     "name", 'String name;
199     "line", 'Int line;
200     "dimension", 'String dimension;
201     "claimed", 'Int claimed;
202     "reason", 'String reason;
203   ]
204
205 (* exempt doc: simple JSON wrapper - name is self-documenting *)
206 let lint (r : Lint.lint_result) =
207   'Assoc [
208     "issues", 'List (List.map lint_issue r.issues);
209     "error_count", 'Int r.error_count;
210     "warning_count", 'Int r.warning_count;

```

```

211   ]
212
213   (* ?? Drift ????????????????????????????????????????????????????????? *)
214
215   (* exempt doc: simple JSON converter - name is self-documenting *)
216   let spec_drift (d : Drift.spec_drift) =
217     'Assoc [
218       "path", 'String d.path;
219       "section_name", 'String d.section_name;
220       "description", 'String d.description;
221     ]
222
223   (* exempt doc: simple JSON converter - name is self-documenting *)
224   let code_drift_item (d : Drift.code_drift_item) =
225     'Assoc [
226       "path", 'String d.path;
227       "kind", 'String d.kind;
228       "name", 'String d.name;
229     ]
230
231   (* exempt doc: simple JSON converter - name is self-documenting *)
232   let structural_drift_item (d : Drift.structural_drift_item) =
233     'Assoc [
234       "description", 'String d.description;
235     ]
236
237   (* exempt doc: simple JSON wrapper - name is self-documenting *)
238   let drift (r : Drift.drift_result) =
239     'Assoc [
240       "spec_drift", 'List (List.map spec_drift r.spec_drift);
241       "code_drift", 'List (List.map code_drift_item r.code_drift);
242       "structural_drift", 'List (List.map structural_drift_item
243         r.structural_drift);
244     ]
245
246   (* ?? Stats ????????????????????????????????????????????????????????? *)
247
248   (* exempt doc: simple JSON converter - name is self-documenting *)
249   let file_metrics (m : Stats.file_metrics) =
250     'Assoc [
251       "path", 'String m.path;
252       "module_name", 'String m.module_name;
253       "lines", 'Int m.lines;
254       "functions", 'Int m.functions;
255       "avg_length", 'Float m.avg_length;
256       "exports", 'Int m.exports;
257       "documented", 'Int m.documented;
258       "verified_count", 'Int m.verified_count;
259       "doc_coverage", 'Float m.doc_coverage;
260       "has_mli", 'Bool m.has_mli;
261       "literacy", (match m.literacy with
262         | None -> 'Null
263         | Some (s, e, t, g) -> 'Assoc [
264           "story", 'Float s;
265           "explain", 'Float e;
266           "teach", 'Float t;
267           "edge", 'Float g;
268         ]);
269     ]
270
271   (* exempt doc: simple JSON wrapper - name is self-documenting *)
272   let stats (r : Stats.project_metrics) =
273     'Assoc [
274       "files", 'List (List.map file_metrics r.files);
275       "total_lines", 'Int r.total_lines;
276       "total_functions", 'Int r.total_functions;
277       "total_exports", 'Int r.total_exports;
278       "total_documented", 'Int r.total_documented;
279       "total_verified", 'Int r.total_verified;
280       "doc_coverage", 'Float r.doc_coverage;
281       "avg_func_length", 'Float r.avg_func_length;
282       "avg_literacy", (match r.avg_literacy with

```

```

283 | Some (s, e, t, g) -> 'Assoc [
284 | "story", 'Float s;
285 | "explain", 'Float e;
286 | "teach", 'Float t;
287 | "edge", 'Float g;
288 | ];
289 ]
290
291 (* ?? Balance ????????????????????????????????????????????????????????????? *)
292
293 (* exempt doc: simple JSON converter - name is self-documenting *)
294 let balance_pos (p : Borge_lang.Balance.pos) =
295   'Assoc [ "line", 'Int p.line; "col", 'Int p.col ]
296
297 (* exempt doc: simple JSON converter - name is self-documenting *)
298 let balance_paren_frame (f : Borge_lang.Balance.paren_frame) =
299   'Assoc [
300     "open_pos", balance_pos f.open_pos;
301     "keyword", (match f.keyword with Some k -> 'String k | None -> 'Null);
302   ]
303
304 (* exempt doc: pattern match to JSON - purpose clear from context *)
305 let balance_error_detail = function
306 | Borge_lang.Balance.Unexpected_close (p, None) ->
307   'Assoc [
308     "type", 'String "unexpected_close";
309     "pos", balance_pos p;
310   ]
311 | Borge_lang.Balance.Unexpected_close (p, Some open_p) ->
312   'Assoc [
313     "type", 'String "unexpected_close";
314     "pos", balance_pos p;
315     "matching_open", balance_pos open_p;
316   ]
317 | Borge_lang.Balance.Unclosed_parens frames ->
318   'Assoc [
319     "type", 'String "unclosed_parens";
320     "frames", 'List (List.map balance_paren_frame (List.rev frames));
321   ]
322 | Borge_lang.Balance.Unclosed_context (p, ctx) ->
323   'Assoc [
324     "type", 'String "unclosed_context";
325     "pos", balance_pos p;
326     "context", 'String ctx;
327   ]
328 | Borge_lang.Balance.Indent_mismatch (p, frame) ->
329   'Assoc [
330     "type", 'String "indent_mismatch";
331     "pos", balance_pos p;
332     "expected_col", 'Int frame.open_pos.col;
333     "keyword", (match frame.keyword with Some k -> 'String k | None ->
334   'Null);
335     "open_pos", balance_pos frame.open_pos;
336   ]
337
338 let structural_node (n : Borge_lang.Balance.structural_node) =
339   'Assoc [
340     "line", 'Int n.node_line;
341     "depth", 'Int n.node_depth;
342     "keyword", (match n.node_keyword with Some k -> 'String k | None ->
343   'Null);
344     "open_col", 'Int n.node_open_col;
345     "closed", 'Bool n.node_closed;
346   ] @ match n.node_close_line with
347 | Some l -> ["close_line", 'Int l]
348 | None -> []
349
350 let indent_divergence (d : Borge_lang.Balance.indent_divergence) =
351   'Assoc [
352     "line", 'Int d.line;
353     "col", 'Int d.col;
354     "depth", 'Int d.depth;
355     "expected_col", 'Int d.expected_col;

```



```

354     "hint", 'String (
355       if d.col > d.expected_col then
356         "this form is over-indented -- check for an extra ' above this
           line"
357       else
358         "this form is under-indented -- a ' may be missing before this
           line"
359     );
360   ]
361
362 let repair_action = function
363 | Borge_lang.Balance.Insert_line { line; indent; reason } ->
364   'Assoc [
365     "action", 'String "insert_line";
366     "line", 'Int line;
367     "indent", 'Int indent;
368     "reason", 'String reason;
369   ]
370 | Borge_lang.Balance.Trim_end { line; count; reason } ->
371   'Assoc [
372     "action", 'String "remove";
373     "line", 'Int line;
374     "column", 'Int 0;
375     "text", 'String (String.make count ' ');
376     "reason", 'String reason;
377   ]
378
379 (* exempt doc: simple JSON converter - name is self-documenting *)
380 let balance_path = function
381 | Borge_lang.Balance.Balanced { max_depth } ->
382   'Assoc [
383     "path", 'String path;
384     "status", 'String "balanced";
385     "max_depth", 'Int max_depth;
386   ]
387 | Borge_lang.Balance.Imbalanced details ->
388   'Assoc [
389     "path", 'String path;
390     "status", 'String "imbalanced";
391     "errors", 'List (List.map balance_error_detail details);
392   ]
393
394 let balance_enriched (path : string) (ea :
   Borge_lang.Balance.enriched_analysis) =
395   let repaired_text = Borge_lang.Balance.repair (File_utils.read_file path)
   in
396   let base = match ea.enriched_result with
397   | Balanced { max_depth } ->
398     'Assoc [
399       "path", 'String path;
400       "status", 'String "balanced";
401       "max_depth", 'Int max_depth;
402     ]
403   | Imbalanced details ->
404     'Assoc [
405       "path", 'String path;
406       "status", 'String "imbalanced";
407       "errors", 'List (List.map balance_error_detail details);
408     ]
409   in
410   match base with
411   | 'Assoc fields ->
412     'Assoc (fields @ [
413       "structural_tree", 'List (List.map structural_node ea.enriched_tree);
414       "indent_divergences", 'List (List.map indent_divergence
   ea.enriched_divergences);
415       "repair_actions", 'List (List.map repair_action
   ea.enriched_repair_actions);
416       "repaired_text", 'String repaired_text;
417     ])
418   | _ -> base

```

79 bin/cmd/Abort.ml

```

1  (** borge abort -- discard current lock and restore .borg files
2
3      Restores .borg files from .borge.lock/original/.
4      Removes the lock directory.
5      Code file changes are NOT reverted -- use git to undo those. *)
6
7  let run quiet =
8      if not (Borge_lib.Lock.is_locked ()) then begin
9          Printf.eprintf "No active lock. Nothing to abort.\n";
10         exit 1
11     end;
12
13     let state = Borge_lib.Lock.get_state () in
14     if not quiet then begin
15         Printf.printf "Lock status: %s\n" state;
16         Printf.printf "Restoring .borg files from .borge.lock/original/...\n";
17     end;
18
19     Borge_lib.Lock.restore ();
20
21     if not quiet then Printf.printf "Removing lock...\n";
22     Borge_lib.Lock.remove ();
23
24     if not quiet then begin
25         Printf.printf "Done. .borg files restored.\n";
26         Printf.printf "Code file changes remain -- use git to undo if
27             needed.\n";
28     end;
29     exit 0
30
31 open Cmdliner
32
33 (* exempt doc *)
34 let quiet =
35     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
36         errors")
37
38 (* exempt doc *)
39 let cmd : unit Cmd.t =
40     Cmd.v (Cmd.info "abort" ~doc:"abort the current agent session and restore
41         .borg files"
42         ~man:[ 'S "DESCRIPTION";
43             'P "Restores .borg files from .borge.lock/original/ and removes
44                 the lock.";
45             'P "Code file changes are NOT reverted -- use git checkout or git
46                 reset.";
47             'P "See also: borge commit (apply changes), borge make (start
48                 implementing)."] )
49     Term.(const run $ quiet)

```

80 bin/cmd/Balance.ml

```

1  open Borge_lang.Balance
2  open Borge_lib
3  open Cmdliner
4
5  let run path verbose json quiet do_repair do_repair_diff =
6    let text = File_utils.read_file path in
7    let result = check text in
8    let imbalanced = match result with Balanced _ -> false | Imbalanced _ ->
      true in
9
10   if do_repair || do_repair_diff then begin
11     let repaired = Borge_lang.Balance.repair text in
12     if do_repair_diff then begin
13       let original_lines = String.split_on_char '\n' text in
14       let repaired_lines = String.split_on_char '\n' repaired in
15       Printf.printf "--- %s\n+++ %s (repaired)\n" path path;
16       let rec show line_num = function
17         | [], [] -> ()
18         | o :: os, [] ->
19           Printf.printf "-%3d | %s\n" line_num o;
20           show (line_num + 1) (os, [])
21         | [], f :: fs ->
22           Printf.printf "+%3d | %s\n" line_num f;
23           show (line_num + 1) ([], fs)
24         | o :: os, f :: fs ->
25           if o = f then
26             Printf.printf " %3d | %s\n" line_num o
27           else begin
28             Printf.printf "-%3d | %s\n" line_num o;
29             Printf.printf "+%3d | %s\n" line_num f
30           end;
31           show (line_num + 1) (os, fs)
32       in
33       show 1 (original_lines, repaired_lines);
34       exit 0
35     end else begin
36       print_string repaired;
37       exit 0
38     end
39   end;
40
41   if json then begin
42     let ea = analyze_enriched text in
43     Printf.printf "%s\n" (Yojson.Basic.to_string (Json_out.balance_enriched
44       path ea));
45     exit (if imbalanced then 1 else 0)
46   end;
47   if quiet then exit (if imbalanced then 1 else 0);
48   let ok = Borge_lang.Balance.report_file ~verbose path in
49   if ok then exit 0 else exit 1
50
51 let path =
52   Arg.(required & pos 0 (some file) None & info [] ~docv:"FILE"
53     ~doc:"The .borg file to check for balance")
54
55 let verbose =
56   Arg.(value & flag & info ["v"; "verbose"] ~doc:"Show paren stack on
57     error")
58
59 let json =
60   Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
61
62 let quiet =
63   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
64     errors")
65
66 let repair_flag =
67   Arg.(value & flag & info ["repair"] ~doc:"Repair imbalanced parens using
68     indentation inference")
69
70 let repair_diff_flag =

```

```
67   Arg.(value & flag & info ["repair-diff"] ~doc:"Show diff of proposed
68       repairs instead of applying")
69 let cmd : unit Cmd.t =
70   Cmd.v (Cmd.info "balance" ~doc:"check structural balance of a .borg file"
71       ~man:[`S "DESCRIPTION";
72         'P "Fast balance check for .borg files. Scans characters without \
73             fully parsing. Understands comments, strings, and verbatim
74             blocks. \
75               Provides line-accurate errors with excerpts and suggestions.";
76         'P "With --verbose, shows the full paren stack on unclosed-paren \
77             errors, listing each open paren with its position and
78             keyword.";
79         'P "With --json, outputs structured JSON with structural_tree, \
80             indent_divergences, and repair_actions.";
81         'P "With --repair, outputs a corrected version of the file by \
82             inferring parens from indentation (Parinfer-style).";
83         'P "With --repair-diff, shows what --repair would change."])
84   Term.(const run $ path $ verbose $ json $ quiet $ repair_flag $
85       repair_diff_flag)
```

81 bin/cmd/Book.ml

```

1 open Borge_lib
2
3 let print_run dir stem =
4   let dir = match dir with Some d -> d | None -> "." in
5   try
6     let n = Book_print.print ~dir ~stem in
7     Printf.printf "wrote %s.pdf and %s.book.manifest (%d files)\n" stem
8       stem n;
9     exit 0
10  with
11  | Failure msg -> prerr_endline ("borge book: " ^ msg); exit 1
12  | e -> prerr_endline ("borge book: " ^ Printexc.to_string e); exit 1
13
14 open Cmdliner
15
16 let dir_arg =
17   Arg.(value & pos 0 (some string) None & info [] ~docv:"DIR"
18     ~doc:"Source directory to print (defaults to the current directory)")
19
20 let stem_arg =
21   Arg.(value & opt string "borge-book" & info ["o"] ~docv:"STEM"
22     ~doc:"Output stem: writes STEM.pdf and STEM.book.manifest")
23
24 let print_cmd : unit Cmd.t =
25   Cmd.v (Cmd.info "print" ~doc:"print the codebase as a paginated PDF book"
26     ~man:[ 'S "DESCRIPTION";
27       'P "Walks the source tree in filesystem-order, emits a paginated \
28         PDF with line numbers on every page and a wide right-hand \
29         margin for hand annotations. Writes a <STEM>.book.manifest \
30         sidecar (file identity + tree checksum) alongside the PDF.";
31       'S "EXAMPLES";
32       'P "borge book print .          # writes borge-book.pdf";
33       'P "borge book print lib -o lib  # writes lib.pdf" ])
34   Term.(const print_run $ dir_arg $ stem_arg)
35
36 let cmd : unit Cmd.t =
37   Cmd.group (Cmd.info "book" ~doc:"print the codebase as a book")
38     [print_cmd]

```

82 bin/cmd/Check.ml

```

1 open Borge_lib
2
3 (** Print a human-readable summary of check results *)
4 let print_result (result : Check.result) =
5   List.iter (function
6     | Check.Ok { path; project_name; form_count } ->
7       Printf.printf " ? %s: project '%s' (%d forms)\n" path project_name
8         form_count
9     | Check.Error { path; message } ->
10       Printf.printf " ? %s: %s\n" path message
11   ) result.files;
12 List.iter (function
13   | Check.Orphan path ->
14     Printf.printf " ? %s: orphaned .borg file (no parent, no
15       no-inline)\n" path
16   ) result.warnings;
17 Printf.printf "\n%d files checked. %d passed. %d failed. %d warnings.\n"
18   (List.length result.files) result.passed result.failed (List.length
19     result.warnings)
20
21 (** Run all checks on a worktree: diff clean, balance, check, build, test *)
22 let run_worktree dir quiet =
23   let failures = ref [] in
24   (* 1. git diff is clean *)
25   let cmd = Printf.sprintf "git -C %s diff --quiet 2>/dev/null"
26     (Filename.quote dir) in
27   if Sys.command cmd <> 0 then
28     failures := "git diff is not clean (uncommitted changes)" :: !failures;
29   (* 2. borge balance passes on all .borg files *)
30   let borg_files = File_utils.find_borg_files dir in
31   List.iter (fun path ->
32     let input = File_utils.read_file path in
33     match Borge_lang.Balance.check input with
34     | Borge_lang.Balance.Balanced _ -> ()
35     | Borge_lang.Balance.Imbalanced _ ->
36       failures := Printf.sprintf "borg balance: %s is imbalanced" path
37         :: !failures
38   ) borg_files;
39   (* 3. borge check passes *)
40   let check_result = Check.run dir in
41   if check_result.failed > 0 then
42     failures := "borg check: some files failed" :: !failures;
43   (* 4. dune build succeeds *)
44   let cmd = Printf.sprintf "cd %s && dune build 2>/dev/null"
45     (Filename.quote dir) in
46   if Sys.command cmd <> 0 then
47     failures := "dune build failed" :: !failures;
48   (* 5. dune runtest passes *)
49   let cmd = Printf.sprintf "cd %s && dune runtest 2>/dev/null"
50     (Filename.quote dir) in
51   if Sys.command cmd <> 0 then
52     failures := "dune runtest failed" :: !failures;
53   (* Report *)
54   if !failures = [] then begin
55     if not quiet then Printf.printf "? Worktree check passed -- clean and
56       ready.\n";
57     exit 0
58   end else begin
59     Printf.eprintf "? Worktree check failed:\n";
60     List.iter (fun f -> Printf.eprintf " - %s\n" f) (List.rev !failures);
61     exit 1
62   end
63
64 let run_dir worktree json quiet =
65   if not (Sys.is_directory dir) then (
66     Printf.eprintf "Error: '%s' is not a directory\n" dir;
67     exit 2
68   );
69   if worktree then run_worktree dir quiet
70   else begin
71     let result = Check.run dir in

```

```

64   if json then begin
65     Printf.printf "%s\n" (Yojson.Basic.to_string (Json_out.check result));
66     exit 0
67   end;
68   if not quiet then begin
69     Printf.printf "Checking .borg files in '%s'...\n\n" dir;
70     print_result result;
71   end;
72   if result.failed > 0 then exit 1 else exit 0
73 end
74
75 open Cmdliner
76
77 (* exempt doc *)
78 let dir =
79   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
80     ~doc:"Directory to scan for .borg files")
81
82 (* exempt doc *)
83 let worktree =
84   Arg.(value & flag & info ["worktree"] ~doc:"Deterministic check: clean
85     diff, balance, check, build, test")
86
87 (* exempt doc *)
88 let json =
89   Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
90
91 (* exempt doc *)
92 let quiet =
93   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
94     errors")
95
96 let cmd : unit Cmd.t =
97   Cmd.v (Cmd.info "check" ~doc:"recursive health check across all .borg
98     files"
99     ~man:[`S "DESCRIPTION";
100       `P "Finds all .borg files recursively and checks that each one \
101         parses correctly and has a project node.";
102       `P "With --worktree, runs a deterministic pass/fail check: \
103         git diff clean, balance passes, check passes, dune build,
104         dune test.";
105       `P "With --json, outputs structured JSON instead of formatted
106         text."])
107   Term.(const run $ dir $ worktree $ json $ quiet)

```

83 bin/cmd/Commit.ml

```

1  (** borge commit -- finalize the current agent session
2
3      Applies changes from the agent session:
4      1. Runs integrity check (unless --force)
5      2. Formats all .borg files
6      3. Runs balance check
7      4. Git commits the changes
8      5. Removes the lock directory
9
10     Called automatically after make/plan/drift --agent unless --no-commit.
11     Can be called manually after --no-commit sessions. *)
12
13 open Borge_lib
14
15 let run force dir quiet =
16   ignore dir;
17   if not (Lock.is_locked ()) then begin
18     Printf.eprintf "Error: No active lock. Nothing to commit.\n";
19     Printf.eprintf "Did you mean to run 'borg make' or 'borg plan'
20     first?\n";
21     exit 1
22   end;
23
24   let state = Lock.get_state () in
25   if state = "committed" then begin
26     Printf.eprintf "Error: Lock already committed.\n";
27     exit 1
28   end;
29
30   if state = "no-lock" then begin
31     Printf.eprintf "Error: Lock state is invalid.\n";
32     exit 1
33   end;
34
35   (* Integrity check unless --force *)
36   if not force then begin
37     match Lock.check_integrity () with
38     | Error msg ->
39       Printf.eprintf "Integrity check failed:\n%s\n" msg;
40       Printf.eprintf "Run 'borg commit --force' to override or 'borg
41       abort' to discard.\n";
42       exit 1
43     | Ok _ -> ()
44   end else begin
45     if not quiet then Printf.printf "--force: skipping integrity check.\n"
46   end;
47
48   (* Run teardown with commit *)
49   match Lock.teardown ~commit:true with
50   | Ok msg ->
51     if not quiet then Printf.printf "%s\n" msg;
52     Lock.remove ();
53     if not quiet then Printf.printf "Committed and lock removed.\n";
54     exit 0
55   | Error msg ->
56     Printf.eprintf "Commit failed: %s\n" msg;
57     Printf.eprintf "Run 'borg commit --force' to override or 'borg abort'
58     to discard.\n";
59     exit 1
60
61 open Cmdliner
62
63 let force =
64   Arg.(value & flag & info ["force"; "f"]
65     ~doc:"Force commit even if integrity check fails")
66
67 let dir =
68   Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
69     ~doc:"Project directory")
70
71 let quiet =

```



```
69   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
    errors")
70
71   let cmd : unit Cmd.t =
72     Cmd.v (Cmd.info "commit" ~doc:"commit the current agent session"
73       ~man:[ 'S "DESCRIPTION";
74         'P "Applies changes from an agent session and commits them.";
75         'P "Runs integrity check, formats .borg files, and creates a git
76           commit.";
77         'P "Called automatically after make/plan unless --no-commit.";
78         'P "Use --force to override integrity check failures.";
79         'S "INTEGRITY CHECK";
80         'P "The integrity check verifies that human-authored comments
81           were not";
82         'P "deleted during the agent session. If deletions are detected,";
83         'P "the commit is blocked unless --force is used." ])
84     Term.(const run $ force $ dir $ quiet)
```

84 bin/cmd/Diff.ml

```

1
2 (** borge diff: git diff for .borg files only *)
3
4 let run dir quiet =
5   let cmd = Printf.sprintf
6     "git -C %s diff -- *.borg 2>/dev/null"
7     (Filename.quote dir) in
8   let ic = Unix.open_process_in cmd in
9   let lines = ref [] in
10  (* exempt: input_line *) (try while true do lines := input_line ic ::
11    !lines done with End_of_file -> ());
12  let _status = Unix.close_process_in ic in
13  match List.rev !lines with
14  | [] ->
15    if not quiet then Printf.printf "No uncommitted changes to .borg
16    files.\n";
17    exit 0
18  | diff_lines ->
19    if quiet then exit 1;
20    Printf.printf "Uncommitted changes to .borg files:\n\n";
21    List.iter (fun line ->
22      if String.length line > 0 then begin
23        match line.[0] with
24        | '+' when String.length line > 1 && line.[1] <> '+' ->
25          if String.length line > 10 && String.sub line 1 10 = "(status
26          " then
27            Printf.printf " ? %s [status change]\n" line
28          else
29            Printf.printf " %s\n" line
30        | '-' when String.length line > 1 && line.[1] <> '-' ->
31          if String.length line > 10 && String.sub line 1 10 = "(status
32          " then
33            Printf.printf " ? %s [status change]\n" line
34          else
35            Printf.printf " %s\n" line
36        | _ ->
37          Printf.printf " %s\n" line
38      end
39    ) diff_lines;
40    exit 1
41
42 open Cmdliner
43
44 (* exempt doc *)
45 let dir =
46   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
47     ~doc:"Path to git repository (default: current directory)")
48
49 (* exempt doc *)
50 let quiet =
51   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
52     errors")
53
54 (* exempt doc *)
55 let cmd : unit Cmd.t =
56   Cmd.v (Cmd.info "diff" ~doc:"git diff for .borg files with status
57     annotations"
58     ~man:[ 'S "DESCRIPTION";
59       'P "Shows git diff filtered to .borg files. Highlights status \
60         transitions and section changes for easy review." ])
61   Term.(const run $ dir $ quiet)

```

85 bin/cmd/Drift.ml

```

1 (** borge drift -- detect and optionally fix drift
2
3   Without flags: static drift analysis
4   With --agent: opens pi with observer role (read-only checks)
5   With --agent --fix: opens pi with fixer role (can write repairs) *)
6
7 open Borge_lib
8
9 (* agent note (|
10  *   WHAT: Gather spec files and module surface data needed for
11  *   agent-driven drift analysis prompt. Finds all .borg files and
12  *   extracts module exports from dune files.
13  *
14  *   WHY: The agent needs context about what specs exist and what
15  *   modules are available to check for drift.
16  * |) *)
17 let gather_prompt_data () =
18   let spec_files =
19     File_utils.find_borg_files "."
20     |> List.map (fun path -> (path, File_utils.read_file path))
21   in
22   let convention = Convention.resolve "." in
23   let module_surfaces =
24     match convention with
25     | Convention.Go_standard ->
26       (try
27         let project = Go_parse.parse_all "." in
28         let lib_packages = Go_parse.library_packages project in
29         List.filter_map (fun (pkg : Go_parse.go_package) ->
30           let surface = Go_surface.extract_package_surface
31             pkg.Go_parse.dir_path in
32             Some (pkg.Go_parse.package_name, Go_surface.exported_names
33               surface)
34         ) lib_packages
35       with _ -> [])
36     | Convention.Ocaml_dune ->
37       (try
38         let dune_files = Dune_parse.parse_all "." in
39         let lib_modules = List.concat_map (fun df ->
40           let dir = Filename.dirname df.Dune_parse.path in
41           List.concat_map (function
42             | Dune_parse.Library lib ->
43               List.map (fun m -> (dir, m)) lib.Dune_parse.modules
44             | _ -> [])
45           ) df.Dune_parse.stanzas
46         ) dune_files in
47         List.filter_map (fun (dir, mod_name) ->
48           let ml_path = Filename.concat dir (mod_name ^ ".ml") in
49           if Sys.file_exists ml_path then
50             try
51               let surface = Surface.extract_surface ml_path in
52               Some (mod_name, surface.Surface.exports)
53             with _ -> Some (mod_name, [])
54           else None
55         ) lib_modules
56       with _ -> [])
57   in
58   (spec_files, module_surfaces)
59
60 (* agent note (|
61  *   WHAT: Write the agent prompt to a temporary file in .borge.lock/.
62  *   Combines spec files and module surfaces into a prompt for the
63  *   observer or fixer agent role.
64  *
65  *   WHY: The agent needs its instructions written to disk before
66  *   pi can execute them.
67  * |) *)
68 let write_prompt role =
69   let spec_files, module_surfaces = gather_prompt_data () in
70   let convention_name = Convention.name (Convention.resolve ".") in

```

```

69   let prompt = Lock_prompt.make_prompt role ~convention:convention_name
70     ~spec_files ~module_surfaces in
71   let path = Lock.prompt_file () in
72   let oc = open_out path in
73   output_string oc prompt;
74   close_out oc;
75   path
76 (* agent note (|
77  * WHAT: Run the interactive agent mode for drift analysis.
78  * Creates a lock, writes prompt, launches pi session, and handles
79  * teardown (commit or abort) based on success/failure.
80  *
81  * WHY: This is the agent-driven alternative to static drift checking.
82  * Allows the LLM to understand spec intent and find semantic drift.
83  * |) *)
84 let run_agent ~fix ~no_commit ~dir ~quiet =
85   (* Agent mode is interactive; quiet flag is only relevant for final
86      output *)
87   ignore quiet;
88   ignore dir;
89   let role = if fix then Lock_prompt.Fixer else Lock_prompt.Observer in
90   let role_name = if fix then "fixer" else "observer" in
91   (match Lock.create () with
92   | Error msg ->
93     Printf.eprintf "Error: %s\n" msg;
94     exit 1
95   | Ok () -> ());
96
97   let prompt_path = write_prompt role in
98   Printf.printf "Lock created at .borge.lock/\n";
99   Printf.printf "Running drift analysis with role: %s\n" role_name;
100  Printf.printf "Prompt written to %s\n" prompt_path;
101
102  let exit_status = Lock_pi_launch.launch prompt_path in
103
104  if exit_status = 0 then begin
105    Printf.printf "\nAgent session completed (exit 0).\n";
106    if no_commit then begin
107      Printf.printf "--no-commit: changes left in exploring.\n";
108      Printf.printf "Run 'borge commit' to finalize or 'borge abort' to
109      discard.\n";
110      Lock.set_state "completed";
111      exit 0
112    end else begin
113      match Lock.teardown ~commit:true with
114      | Ok msg ->
115        Printf.printf "%s\n" msg;
116        Lock.remove ();
117        Printf.printf "Committed and lock removed.\n";
118        exit 0
119      | Error msg ->
120        Printf.eprintf "Commit failed: %s\n" msg;
121        Printf.eprintf "Run 'borge commit --force' to override or 'borge
122        abort' to discard.\n";
123        Lock.set_state "failed";
124        exit 1
125      end
126    end else begin
127      Printf.eprintf "\nAgent session failed (exit %d).\n" exit_status;
128      Printf.eprintf "Lock kept at .borge.lock/ for review.\n";
129      Lock.set_state "failed";
130      exit exit_status
131    end
132  end
133 (* agent note (|
134  * WHAT: Run static (non-agentic) drift analysis on a directory.
135  * Checks spec drift, code drift, and optionally module integrity.
136  * Prints results and returns exit code based on findings.
137  *
138  * WHY: The fast, non-LLM path for drift detection. Used for
139  * CI/CD and quick checks without agent overhead.

```

```

138  * |) *)
139  let run_static dir json quiet modules =
140    (* Run classic drift analysis *)
141    let _meta = Drift.write_meta_files dir in
142    let result = Drift.run dir in
143
144    (* Optionally run module integrity check *)
145    let module_issues = if modules then begin
146      let specs = Module_spec.load_all_specs dir in
147      List.concat_map (fun (spec : Module_spec.module_def) ->
148        let impl_files = Module_spec.find_implementation_files spec.path in
149        List.concat_map (fun impl_path ->
150          let (missing, undocumented) = Module_spec.check_integrity spec
151            impl_path in
152          List.map (fun name ->
153            Printf.sprintf "module %s: missing implementation of %s"
154              spec.module_name name
155                ) missing @
156              List.map (fun name ->
157                Printf.sprintf "module %s: undocumented function %s"
158                  spec.module_name name
159                    ) undocumented
160                ) impl_files
161          ) specs
162      end else [] in
163
164    if json then begin
165      (* TODO: include module_issues in JSON *)
166      if not quiet then Printf.printf "%s\n" (Yojson.Basic.to_string
167        (Json_out.drift result));
168      let total = List.length result.spec_drift + List.length
169        result.code_drift +
170        List.length result.structural_drift + List.length
171        module_issues in
172      exit (if total > 0 then 1 else 0)
173    end;
174
175    if quiet then exit 0;
176
177    Printf.printf "Drift report for '%s'\n\n" dir;
178    if result.spec_drift = [] then
179      Printf.printf "Spec drift: none\n"
180    else begin
181      Printf.printf "Spec drift (%d sections):\n" (List.length
182        result.spec_drift);
183      List.iter (fun (d : Drift.spec_drift_item) ->
184        Printf.printf " ? %s: '%s' marked implemented but not found in
185          code\n"
186          d.path d.section_name
187        ) result.spec_drift
188    end;
189
190    if result.code_drift = [] then
191      Printf.printf "Code drift: none\n"
192    else begin
193      Printf.printf "Code drift (%d items):\n" (List.length
194        result.code_drift);
195      List.iter (fun (d : Drift.code_drift_item) ->
196        Printf.printf " ? %s: %s (%s)\n" d.path d.name d.kind
197        ) result.code_drift
198    end;
199
200    if result.structural_drift = [] then
201      Printf.printf "Structural drift: none\n"
202    else begin
203      Printf.printf "Structural drift (%d items):\n" (List.length
204        result.structural_drift);
205      List.iter (fun (d : Drift.structural_drift_item) ->
206        Printf.printf " ? %s\n" d.description
207        ) result.structural_drift
208    end;
209
210    let total = List.length result.spec_drift + List.length result.code_drift
211      +
212      List.length result.structural_drift + List.length
213      module_issues in

```

```

199   if module_issues <> [] then begin
200     Printf.printf "Module integrity issues (%d):\n" (List.length
      module_issues);
201     List.iter (Printf.printf " ? %s\n") module_issues
202   end;
203   Printf.printf "\nTotal drift: %d item(s)\n" total;
204   if total > 0 then exit 1 else exit 0
205
206 (* agent note (|
207  * WHAT: Run documentation coverage analysis using Doc_coverage.
208  * Reports documented/undocumented/exempt counts per file with
209  * optional verbose output showing specific binding names.
210  *
211  * WHY: Implements borge drift --docs for literate programming
212  * enforcement. Exit codes indicate coverage thresholds.
213  * |) *)
214 (* agent note (|
215  * WHAT: Read the doc-coverage-threshold from the borge.borg
216  * convention stanza. Returns the default 0.8 if not found.
217  *
218  * WHY: Coverage thresholds are configurable per project, so teams
219  * can set their own standards. Reading from the spec file keeps
220  * the threshold consistent across all borge commands.
221  * |) *)
222 let read_coverage_threshold dir =
223   let borg_files = File_utils.find_borg_files dir in
224   let rec try_files = function
225     | [] -> 0.8 (* default *)
226     | path :: rest ->
227       try
228         let input = File_utils.read_file path in
229         (* Look for (convention ocaml-dune (doc-coverage-threshold N)) *)
230         let re = Str.regexp "doc-coverage-threshold[ \\t]+\\([0-9.]+\\)" in
231         if Str.string_match re input 0 then
232           let v = Str.matched_group 1 input in
233           (match float_of_string_opt v with Some f -> f | None ->
234            try_files rest)
235         else
236           try_files rest
237       with _ -> try_files rest
238   in
239   try_files borg_files
240
241 let run_docs dir ~strict ~verbose =
242   Printf.printf "Documentation coverage analysis for '%s'\n\n" dir;
243   let coverage = Doc_coverage.calculate_dir_coverage dir in
244   Printf.printf "%s\n" (Doc_coverage.format_project_summary coverage);
245   if verbose then begin
246     List.iter (fun (fc : Doc_coverage.file_coverage) ->
247       if fc.undocumented_names <> [] then begin
248         Printf.printf "\nUndocumented in %s:\n" fc.path;
249         List.iter (fun name ->
250           Printf.printf " - %s\n" name
251         ) fc.undocumented_names
252       end;
253       if fc.drifted_names <> [] then begin
254         Printf.printf "\nDrifted in %s:\n" fc.path;
255         List.iter (fun name ->
256           Printf.printf " ~ %s (status drifted)\n" name
257         ) fc.drifted_names
258       end
259     ) coverage.files
260   end;
261   let threshold = read_coverage_threshold dir in
262   (* Exit code logic per drift-doc-integration spec *)
263   let has_error = List.exists (fun (fc : Doc_coverage.file_coverage) ->
264     fc.coverage_percent < 50.0
265   ) coverage.files in
266   let has_warning = List.exists (fun (fc : Doc_coverage.file_coverage) ->
267     fc.coverage_percent >= 50.0 && fc.coverage_percent < (threshold *.
268     100.0)
269   ) coverage.files in
270   if has_error then exit 1

```

```

269     else if strict && has_warning then exit 2
270     else exit 0
271
272 let run_dir agent fix no_commit json quiet modules docs strict verbose =
273   if docs then run_docs dir ~strict ~verbose
274   else if agent then run_agent ~fix ~no_commit ~dir ~quiet
275   else if fix then begin
276     Printf.eprintf "Error: --fix requires --agent\n";
277     exit 1
278   end
279   else run_static dir json quiet modules
280
281 open Cmdliner
282
283 (* exempt doc: cmdliner argument definitions - purpose is self-evident *)
284 let dir =
285   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
286     ~doc:"Directory to check for drift")
287
288 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
289 let agent =
290   Arg.(value & flag & info ["agent"]
291     ~doc:"Open pi interactively for agent-powered drift analysis")
292
293 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
294 let fix =
295   Arg.(value & flag & info ["fix"]
296     ~doc:"Allow agent to write code repairs (requires --agent)")
297
298 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
299 let no_commit =
300   Arg.(value & flag & info ["no-commit"]
301     ~doc:"Leave changes in working tree; do not auto-commit")
302
303 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
304 let json =
305   Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
306
307 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
308 let quiet =
309   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
310     errors")
311
312 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
313 let modules =
314   Arg.(value & flag & info ["modules"; "m"] ~doc:"Include module integrity
315     checks")
316
317 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
318 let docs =
319   Arg.(value & flag & info ["docs"; "d"] ~doc:"Analyze documentation
320     coverage")
321
322 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
323 let strict =
324   Arg.(value & flag & info ["strict"] ~doc:"Treat warnings as errors in
325     docs mode")
326
327 (* exempt doc: cmdliner argument definition - purpose is self-evident *)
328 let verbose =
329   Arg.(value & flag & info ["verbose"; "v"] ~doc:"Show per-binding
330     undocumented/drifted names in docs mode")
331
332 let cmd : unit Cmd.t =
333   Cmd.v (Cmd.info "drift" ~doc:"detect spec, code, and structural drift"
334     ~man:[ 'S "DESCRIPTION";
335       'P "Checks for three layers of drift:";
336       'P "1. Spec drift: .borg describes something not in the code";
337       'P "2. Code drift: code has something not in any .borg spec";
338       'P "3. Structural drift: code organization violates the
339         convention";
340       'S "DOCUMENTATION MODE";
341       'P "With --docs, analyzes documentation coverage of .ml files." ]
342   )

```

```
336         'P "Reports which bindings have/don't have documentation.";
337         'P "With --docs --strict, warnings also cause non-zero exit.";
338         'P "With --docs --verbose, shows per-binding undocumented/drifted
names.";
339         'S "AGENT MODE";
340         'P "With --agent, opens pi interactively with observer role.";
341         'P "The agent checks whether code matches spec and reports
findings.";
342         'P "With --agent --fix, the agent can write code repairs.";
343         'P "Auto-commits on clean exit (use --no-commit to review
first)."]
344 Term.(const run $ dir $ agent $ fix $ no_commit $ json $ quiet $ modules
$ docs $ strict $ verbose)
```


86 bin/cmd/Execute.ml

```

1  open Borge_lib
2
3  let run section dir quiet =
4    ignore quiet;
5    if section = "" then begin
6      Printf.eprintf "Error: SECTION argument is required.\n";
7      exit 1
8    end;
9    match Generate.generate_code section dir with
10   | Some _ ->
11     exit 0
12   | None ->
13     Printf.eprintf "Failed to execute section '%s'.\n" section;
14     exit 1
15
16  open Cmdliner
17
18  (* exempt doc *)
19  let section =
20    Arg.(value & pos 0 string "" & info [] ~docv:"SECTION"
21      ~doc:"Section name to execute (implement)")
22
23  (* exempt doc *)
24  let dir =
25    Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
26      ~doc:"Project directory")
27
28  (* exempt doc *)
29  let quiet =
30    Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
31      errors")
32
33  (* exempt doc *)
34  let cmd : unit Cmd.t =
35    Cmd.v (Cmd.info "execute" ~doc:"execute (implement) a planned section"
36      ~man:[ 'S "DESCRIPTION";
37        'P "Alias for 'borge generate code'. Reads the spec for a planned
38          \
39            section and invokes an LLM to produce the implementation. \
40            Planned sections are unimplemented tasks -- 'execute' means \
41            'implement this task'." ])
42    Term.(const run $ section $ dir $ quiet)

```

87 bin/cmd/Fmt.ml

```

1 open Borge_lib
2
3 let show_diff original formatted =
4   let orig_lines = String.split_on_char '\n' original in
5   let fmt_lines = String.split_on_char '\n' formatted in
6   Printf.printf "--- original\n+++ formatted\n";
7   let rec show line_num = function
8     | [], [] -> ()
9     | o :: os, [] ->
10      Printf.printf "-%3d | %s\n" line_num o;
11      show (line_num + 1) (os, [])
12     | [], f :: fs ->
13      Printf.printf "+%3d | %s\n" line_num f;
14      show (line_num + 1) ([], fs)
15     | o :: os, f :: fs ->
16      if o = f then
17        Printf.printf " %3d | %s\n" line_num o
18      else begin
19        Printf.printf "-%3d | %s\n" line_num o;
20        Printf.printf "+%3d | %s\n" line_num f
21      end;
22      show (line_num + 1) (os, fs)
23   in
24   show 1 (orig_lines, fmt_lines)
25
26 let check_or_repair path repair quiet =
27   let input = File_utils.read_file path in
28   try
29     let formatted = Fmt.format_string input in
30     if Fmt.strip_trailing_newlines input = Fmt.strip_trailing_newlines
31        formatted then
32       Fmt.CheckClean
33     else
34       Fmt.CheckDirty formatted
35   with Borge_lang.Error.Parse_error _ as exn ->
36     if repair then (
37       let repaired = Borge_lang.Balance.repair input in
38       try
39         let formatted = Fmt.format_string repaired in
40         if not quiet then Printf.eprintf "borge fmt: repaired imbalance
41 before formatting\n";
42         Fmt.CheckDirty formatted
43       with Borge_lang.Error.Parse_error _ ->
44         raise exn
45     ) else
46       raise exn
47
48 let format_or_repair path repair quiet =
49   let input = File_utils.read_file path in
50   try
51     Fmt.Formatted (Fmt.format_string input)
52   with Borge_lang.Error.Parse_error _ as exn ->
53     if repair then (
54       let repaired = Borge_lang.Balance.repair input in
55       try
56         let formatted = Fmt.format_string repaired in
57         if not quiet then Printf.eprintf "borge fmt: repaired imbalance
58 before formatting\n";
59         Fmt.Formatted formatted
60       with Borge_lang.Error.Parse_error _ ->
61         raise exn
62     ) else
63       raise exn
64
65 let run path check diff output quiet repair =
66   if check then
67     match check_or_repair path repair quiet with
68     | Fmt.CheckClean ->
69       if not quiet then Printf.printf "%s: already formatted\n"
70          (Filename.basename path);
71       exit 0

```

```

68 | Fmt.CheckDirty formatted ->
69   if not quiet then Printf.printf "%s: needs formatting\n"
   (Filename.basename path);
70   (match output with
71     | Some dst ->
72       let oc = open_out dst in
73       output_string oc formatted;
74       close_out oc
75     | None -> print_endline formatted);
76   exit 1
77 | Fmt.Formatted _ -> (* shouldn't happen in check mode *) exit 1
78 else if diff then begin
79   let original = File_utils.read_file path in
80   match format_or_repair path repair quiet with
81   | Fmt.Formatted formatted ->
82     if not quiet then show_diff original formatted;
83     if Fmt.strip_trailing_newlines original =
84       Fmt.strip_trailing_newlines formatted
85     then exit 0 else exit 1
86   | _ -> exit 1
87 end else
88   match format_or_repair path repair quiet with
89   | Fmt.Formatted txt ->
90     (match output with
91       | Some dst ->
92         let oc = open_out dst in
93         output_string oc txt;
94         close_out oc;
95         if not quiet then Printf.printf "%s\n" (Filename.basename dst)
96       | None -> if not quiet then print_endline txt else ());
97     exit 0
98   | _ -> (* check results in format mode shouldn't happen *) exit 1
99
100 open Cmdliner
101
102 let path =
103   Arg.(required & pos 0 (some file) None & info [] ~docv:"FILE"
104     ~doc:"The .borg file to format")
105
106 let output =
107   Arg.(value & opt (some file) None & info ["o"] ~docv:"FILE"
108     ~doc:"Write output to FILE (default: stdout)")
109
110 let check =
111   Arg.(value & flag & info ["check"] ~doc:"Exit 1 if formatting would
112     change the file")
113
114 let diff =
115   Arg.(value & flag & info ["diff"] ~doc:"Show diff between current and
116     formatted output")
117
118 let quiet =
119   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
120     errors")
121
122 let repair =
123   Arg.(value & flag & info ["repair"] ~doc:"Repair paren imbalance from
124     indentation before formatting")
125
126 let cmd : unit Cmd.t =
127   Cmd.v (Cmd.info "fmt" ~doc:"auto-format a .borg file to canonical
128     indentation"
129     ~man:[`S "DESCRIPTION";
130       `P "Parses the .borg file and re-prints it with canonical
131         indentation.";
132       `P "Use --check for CI (exit 1 if formatting would change). \
133         Use --diff to see what would change.";
134       `P "Use --repair to auto-fix paren imbalance before formatting \
135         (infers close-parens from indentation)."])
136   Term.(const run $ path $ check $ diff $ output $ quiet $ repair)

```

88 bin/cmd/Future.ml

```

1  (* borge future -- show what's planned but not yet implemented
2
3      Displays the roadmap of spec'd work that's planned or in-progress *)
4
5  open Borge_lib
6
7  (** Display the roadmap of planned/in-progress spec sections *)
8  let run_dir json summary section verbose =
9      (* Get roadmap from future module *)
10     let roadmap = Future.get_roadmap dir in
11
12     if summary then begin
13         Printf.printf "%s\n" (Format.format_summary roadmap);
14         exit 0
15     end;
16
17     if json then begin
18         Printf.printf "%s\n" (Format.format ~json:true roadmap);
19         exit 0
20     end;
21
22     (* Filter by section if specified *)
23     let roadmap' = match section with
24         | Some s -> Format.filter_by_section s roadmap
25         | None -> roadmap
26     in
27
28     if verbose then begin
29         (* Show dependency graph *)
30         let graph = Depends.build_graph dir in
31         Printf.printf "%s\n" (Format.format roadmap');
32         Printf.printf "\n%s\n" (Format.format_dependency_graph graph);
33         Printf.printf "\n%s\n" (Format.format_completion_order graph);
34     end else
35         Printf.printf "%s\n" (Format.format roadmap');
36
37     exit 0
38
39  open Cmdliner
40
41  (* exempt doc *)
42  let dir =
43      Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
44          ~doc:"Project directory to scan")
45
46  (* exempt doc *)
47  let json =
48      Arg.(value & flag & info ["json"; "j"]
49          ~doc:"Output as JSON")
50
51  (* exempt doc *)
52  let summary =
53      Arg.(value & flag & info ["summary"; "s"]
54          ~doc:"Show summary counts only")
55
56  (* exempt doc *)
57  let section =
58      Arg.(value & opt (some string) None & info ["section"]
59          ~docv:"NAME" ~doc:"Filter to specific section")
60
61  (* exempt doc *)
62  let verbose =
63      Arg.(value & flag & info ["verbose"; "v"]
64          ~doc:"Show dependency graph and completion order")
65
66  let cmd : unit Cmd.t =
67      Cmd.v (Cmd.info "future" ~doc:"show planned and in-progress work"
68          ~man:[ 'S "DESCRIPTION";
69              'P "Scans all .borg files and displays what's marked as planned";
70              'P "or in-progress. This is your project's roadmap.";
71              'P "';

```

```
72         'P "Output is grouped by status:";
73         'P "  PLANNED          - Not yet started";
74         'P "  IN PROGRESS     - Currently being worked on";
75         'P "  READY FOR REVIEW - Implemented but depends on planned
items";
76         'S "FILTERING";
77         'P "Use --section to view only a specific part of the spec.";
78         'P "Use --summary for a quick count overview.";
79         'S "EXAMPLES";
80         'P "borge future          # Show full roadmap";
81         'P "borge future --summary # Just the counts";
82         'P "borge future --section db # Only database-related items";])
83 Term.(const run $ dir $ json $ summary $ section $ verbose)
```

89 bin/cmd/Generate.ml

```

1 open Borge_lib
2
3 (** Generate a .borg section from a todo description using LLM *)
4 let run_spec todo dir quiet =
5   match Generate.generate_spec todo dir with
6   | Some sexp ->
7     if not quiet then Printf.printf "%s\n" sexp;
8     exit 0
9   | None ->
10    Printf.eprintf "Failed to generate spec.\n";
11    exit 1
12
13 (** Generate code to implement a planned section *)
14 let run_code section dir quiet =
15   ignore quiet;
16   match Generate.generate_code section dir with
17   | Some _ ->
18     exit 0
19   | None ->
20    Printf.eprintf "Failed to generate code for section '%s'.\n" section;
21    exit 1
22
23 (** Inspect a UI spec: parse, validate, summarize *)
24 let run_ui section dir json quiet =
25   let borg_files = File_utils.find_borg_files dir in
26   let matching_borg = List.filter (fun p ->
27     try
28       let input = File_utils.read_file p in
29       let file = Borge_lang.Parse.parse_file input in
30       let app = Ui_parse.parse_file file in
31       match app with
32       | Some a ->
33         List.exists (fun (pg : Ui_ast.page) -> pg.Ui_ast.name = section)
34         a.Ui_ast.pages
35         || List.exists (fun (ly : Ui_ast.layout_def) -> ly.Ui_ast.name =
36         section) a.Ui_ast.layouts
37         || List.exists (fun (c : Ui_ast.component) -> c.Ui_ast.name =
38         section) a.Ui_ast.components
39         | None -> false
40     with _ -> false
41   ) borg_files in
42   match matching_borg with
43   | [] ->
44     Printf.eprintf "No .borg file contains UI section '%s'\n" section;
45     exit 1
46   | borg_path :: _ ->
47     let input = File_utils.read_file borg_path in
48     let file = Borge_lang.Parse.parse_file input in
49     (match Ui_parse.parse_file file with
50     | Some app ->
51       if json then begin
52         let json_val = Ui_json.ui_app_to_json app in
53         Printf.printf "%s\n" (Yojson.Basic.pretty_to_string json_val)
54       end else if not quiet then begin
55         Printf.printf "UI spec found in %s\n" borg_path;
56         (match app.Ui_ast.theme with
57         | Some t ->
58           let palette_count = List.filter_map (function
59           Ui_ast.Palette_entry _ -> Some () | _ -> None) t.Ui_ast.entries in
60           Printf.printf "  theme: %d palette entries, %d spacing, %d
61           font-size, %d radius\n"
62             (List.length palette_count)
63             (List.filter_map (function Ui_ast.Spacing_entry _ -> Some () |
64             _ -> None) t.Ui_ast.entries |> List.length)
65             (List.filter_map (function Ui_ast.Font_size_entry _ -> Some ()
66             | _ -> None) t.Ui_ast.entries |> List.length)
67             (List.filter_map (function Ui_ast.Radius_entry _ -> Some () | _
68             -> None) t.Ui_ast.entries |> List.length)
69             | None -> ());
70         Printf.printf "  components: %d\n" (List.length
71         app.Ui_ast.components);

```

```

63     List.iter (fun (c : Ui_ast.component) -> Printf.printf "      - %s\n"
c.Ui_ast.name) app.Ui_ast.components;
64     Printf.printf " layouts: %d\n" (List.length app.Ui_ast.layouts);
65     List.iter (fun (ly : Ui_ast.layout_def) -> Printf.printf "      -
%s\n" ly.Ui_ast.name) app.Ui_ast.layouts;
66     Printf.printf " pages: %d\n" (List.length app.Ui_ast.pages);
67     List.iter (fun (p : Ui_ast.page) -> Printf.printf "      - %s
(layout: %s)\n" p.Ui_ast.name p.Ui_ast.layout_name) app.Ui_ast.pages;
68     Printf.printf " routes: %d\n" (List.length app.Ui_ast.routes);
69     List.iter (fun (r : Ui_ast.route) -> Printf.printf "      - %s ->
%s\n" r.Ui_ast.path r.Ui_ast.page_name) app.Ui_ast.routes;
70     let issues = Ui_validate.validate app in
71     if issues <> [] then begin
72         Printf.printf " validation issues: %d\n" (List.length issues);
73         List.iter (fun (i : Ui_validate.issue) ->
74             Printf.printf "      [%s] %s\n"
75                 (match i.severity with 'Error -> "ERROR" | 'Warning -> "WARN")
76                 i.message
77             ) issues
78     end
79 end;
80 exit 0
81 | None ->
82     Printf.eprintf "Failed to parse UI spec from %s\n" borg_path;
83     exit 1)
84
85 (** Inspect a DB spec: parse, validate, summarize *)
86 let run_db section dir json quiet =
87     let borg_files = File_utils.find_borg_files dir in
88     let matching_borg = List.filter (fun p ->
89         try
90             let input = File_utils.read_file p in
91             let file = Borge_lang.Parse.parse_file input in
92             let app = Db_parse.parse_file file in
93             match app with
94             | Some a ->
95                 List.exists (fun (t : Db_ast.table_def) -> t.Db_ast.name = section)
a.Db_ast.tables
96             | None -> false
97         with _ -> false
98     ) borg_files in
99     match matching_borg with
100 | [] ->
101     Printf.eprintf "No .borg file contains DB table '%s'\n" section;
102     exit 1
103 | borg_path :: _ ->
104     let input = File_utils.read_file borg_path in
105     let file = Borge_lang.Parse.parse_file input in
106     (match Db_parse.parse_file file with
107     | Some app ->
108         if json then begin
109             let json_val = Db_json.db_app_to_json app in
110             Printf.printf "%s\n" (Yojson.Basic.pretty_to_string json_val)
111         end else if not quiet then begin
112             Printf.printf "DB spec found in %s\n" borg_path;
113             Printf.printf " tables: %d\n" (List.length app.Db_ast.tables);
114             List.iter (fun (t : Borge_lib.Db_ast.table_def) ->
115                 Printf.printf "      - %s (%d columns" t.Db_ast.name (List.length
t.Db_ast.columns);
116                 (match t.Db_ast.ownership with Some o -> Printf.printf ",
ownership: %s" o | None -> ());
117                 Printf.printf ")\n"
118             ) app.Db_ast.tables;
119             Printf.printf " operations: %d\n" (List.length
app.Db_ast.operations);
120             List.iter (fun (o : Borge_lib.Db_ast.operations_def) ->
121                 Printf.printf "      - %s" o.Db_ast.table_name;
122                 (match o.Db_ast.crud with Some _ -> Printf.printf " (CRUD)" |
None -> ());
123                 Printf.printf "\n"
124             ) app.Db_ast.operations;
125             Printf.printf " relations: %d\n" (List.length
app.Db_ast.relations);

```

```

126     List.iter (fun (r : Borge_lib.Db_ast.relation_def) ->
127         Printf.printf "    - %s (from %s, %d joins)\n" r.Db_ast.name
128         r.Db_ast.from_table (List.length r.Db_ast.joins)
129         ) app.Db_ast.relations;
130     Printf.printf "    groups: %d\n" (List.length app.Db_ast.groups);
131     List.iter (fun (g : Borge_lib.Db_ast.group_def) ->
132         Printf.printf "    - %s%s\n" g.Db_ast.name (if g.Db_ast.can_all
133         then " (can-all)" else "")
134         ) app.Db_ast.groups;
135     let issues = Db_validate.validate app in
136     if issues <> [] then begin
137         Printf.printf "    validation issues: %d\n" (List.length issues);
138         List.iter (fun (i : Db_validate.issue) ->
139             Printf.printf "        [%s] %s\n"
140             (match i.severity with 'Error -> "ERROR" | 'Warning -> "WARN")
141             i.message
142         ) issues
143     end
144     end;
145     exit 0
146 | None ->
147     Printf.eprintf "Failed to parse DB spec from %s\n" borg_path;
148     exit 1)
149
150 open Cmdliner
151
152 (* exempt doc *)
153 let spec_cmd : unit Cmd.t =
154     let todo =
155         Arg.(value & pos 0 string "" & info [] ~docv:"TODO"
156             ~doc:"Todo description to turn into a .borg section")
157     in
158     let dir =
159         Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
160             ~doc:"Project directory")
161     in
162     let quiet =
163         Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output
164             except errors")
165     in
166     Cmd.v (Cmd.info "spec" ~doc:"generate a .borg section from a todo
167         description"
168         ~man:['S "DESCRIPTION";
169             'P "Takes a free-form todo description and uses an LLM to produce
170             \
171                 a structured .borg section with (status planned)."]
172         Term.(const run_spec $ todo $ dir $ quiet))
173
174 (* exempt doc *)
175 let code_cmd : unit Cmd.t =
176     let section =
177         Arg.(value & pos 0 string "" & info [] ~docv:"SECTION"
178             ~doc:"Section name to implement")
179     in
180     let dir =
181         Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
182             ~doc:"Project directory")
183     in
184     let quiet =
185         Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output
186             except errors")
187     in
188     Cmd.v (Cmd.info "code" ~doc:"generate implementation code from a planned
189         section"
190         ~man:['S "DESCRIPTION";
191             'P "Reads the spec for a planned section and uses an LLM to \
192                 generate the implementation code. The section should be \
193                 marked (status planned) in a .borg file."]
194         Term.(const run_code $ section $ dir $ quiet))
195
196 (* exempt doc *)
197 let ui_cmd : unit Cmd.t =
198     let section =

```



```

192   Arg.(value & pos 0 string "" & info [] ~docv:"SECTION"
193         ~doc:"UI section name (page, layout, or component)")
194   in
195   let dir =
196     Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
197           ~doc:"Project directory")
198   in
199   let json =
200     Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
201   in
202   let quiet =
203     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output
204           except errors")
205   in
206   Cmd.v (Cmd.info "ui" ~doc:"inspect a UI spec section"
207         ~man:[`S "DESCRIPTION";
208               `P "Reads a (ui ...) spec from a .borg file, parses it, \
209                   validates it, and prints a summary. The UI spec IS the \
210                   artifact -- no code is generated. Use borge make to edit \
211                   UI specs interactively."])
212   Term.(const run_ui $ section $ dir $ json $ quiet)
213
214 (* exempt doc *)
215 let db_cmd : unit Cmd.t =
216   let section =
217     Arg.(value & pos 0 string "" & info [] ~docv:"TABLE"
218           ~doc:"DB table name")
219   in
220   let dir =
221     Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
222           ~doc:"Project directory")
223   in
224   let json =
225     Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
226   in
227   let quiet =
228     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output
229           except errors")
230   in
231   Cmd.v (Cmd.info "db" ~doc:"inspect a DB spec section"
232         ~man:[`S "DESCRIPTION";
233               `P "Reads a (db ...) spec from a .borg file, parses it, \
234                   validates it, and prints a summary. The DB spec IS the \
235                   artifact -- no code is generated. Use borge make to edit \
236                   DB specs interactively."])
237   Term.(const run_db $ section $ dir $ json $ quiet)
238
239 (** Generate target source code from a .borg spec file *)
240 let run_target target borg_path quiet =
241   if target = "" then begin
242     Printf.eprintf "Error: TARGET is required (postgres-sql, ocaml-dream,
243                   clay-c)\n";
244     exit 1
245   end;
246   if borg_path = "" then begin
247     Printf.eprintf "Error: SPEC path is required\n";
248     exit 1
249   end;
250   let input = File_utils.read_file borg_path in
251   let file = Borge_lang.Parse.parse_file input in
252   match target with
253   | "postgres-sql" ->
254     (match Db_parse.parse_file file with
255     | Some app ->
256       let sql = Db_sql.generate ~source_path:borg_path app in
257       Printf.printf "%s\n" sql;
258       if not quiet then
259         Printf.eprintf "(generated db postgres-sql at %s)\n" borg_path
260     | None ->
261       Printf.eprintf "Failed to parse DB spec from %s\n" borg_path;
262       exit 1)
263   | "ocaml-dream" ->
264     (match Ui_parse.parse_file file with

```

```

262 | Some app ->
263   let ml = Ui_dream.generate ~source_path:borg_path app in
264   Printf.printf "%s\n" ml;
265   if not quiet then
266     Printf.eprintf "(generated ui ocaml-dream at %s)\n" borg_path
267 | None ->
268   Printf.eprintf "Failed to parse UI spec from %s\n" borg_path;
269   exit 1)
270 | "clay-c" ->
271   (match Ui_parse.parse_file file with
272   | Some app ->
273     let c_code = Ui_clay.generate ~source_path:borg_path app in
274     let h_code = Ui_clay.generate_header_file app in
275     Printf.printf "/* --- ui_app.h --- */\n%s\n/* --- ui_app.c --- */\n%s\n" h_code c_code;
276     if not quiet then
277       Printf.eprintf "(generated ui clay-c at %s)\n" borg_path
278   | None ->
279     Printf.eprintf "Failed to parse UI spec from %s\n" borg_path;
280     exit 1)
281 | _ ->
282   Printf.eprintf "Unknown target '%s'. Supported: postgres-sql,
283   ocaml-dream, clay-c\n" target;
284   exit 1
285
286 (* exempt doc *)
287 let target_cmd : unit Cmd.t =
288   let target =
289     Arg.(value & pos 0 string "" & info [] ~docv:"TARGET"
290     ~doc:"Code generation target: postgres-sql, ocaml-dream, or clay-c")
291   in
292   let borg_path =
293     Arg.(value & pos 1 file "" & info [] ~docv:"SPEC"
294     ~doc:"Path to .borg spec file")
295   in
296   let quiet =
297     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress event emission
298     on stderr")
299   in
300   Cmd.v (Cmd.info "target" ~doc:"generate target source code from a spec"
301   ~man:[S "DESCRIPTION";
302   'P "Reads a .borg spec file and generates target source code \
303   from the typed AST. Supports three targets:";
304   'I ("postgres-sql", "PostgreSQL DDL (CREATE TABLE, indexes, RLS
305   policies)");
306   'I ("ocaml-dream", "OCaml dream_html code (components, layouts,
307   routes)");
308   'I ("clay-c", "C code using Clay layout API (for raylib/pico)"]])
309   Term.(const run_target $ target $ borg_path $ quiet)
310
311 let cmd : unit Cmd.t =
312   let info = Cmd.info "generate" ~doc:"generate spec sections or code from
313   specs"
314   ~man:[S "DESCRIPTION";
315   'P "Generate spec sections, inspect DB/UI specs, or implement \
316   code from specs.";
317   'S "SUBCOMMANDS";
318   'I ("spec", "Turn a todo into a structured .borg section");
319   'I ("code", "Implement a planned section from its spec");
320   'I ("ui", "Inspect a UI spec section");
321   'I ("db", "Inspect a DB spec section");
322   'I ("target", "Generate target source code from a spec")]
323   in
324   Cmd.group info [spec_cmd; code_cmd; ui_cmd; db_cmd; target_cmd]
325
326 (** Note: ui_cmd and db_cmd inspect specs (parse, validate, summarize).
327   They do NOT generate code. The spec IS the artifact.
328   Use 'borge make' for agent-driven spec editing. *)

```

90 bin/cmd/Inline.ml

```

1  open Borge_lib
2
3  let run dir quiet =
4    let roots = Project.find_roots dir in
5    if roots = [] then begin
6      if not quiet then Printf.printf "No root .borg files found in '%s'\n"
7        dir;
8      exit 1
9    end;
10   List.iter (fun root ->
11     match Project.build_tree root with
12     | Error (Project.File_not_found path) ->
13       Printf.eprintf "Error: file not found: %s\n" path;
14       exit 2
15     | Error (Project.Parse_error (path, msg)) ->
16       Printf.eprintf "Error: parse error in %s: %s\n" path msg;
17       exit 2
18     | Error (Project.Cycle_detected path) ->
19       Printf.eprintf "Error: inline cycle detected: %s\n" (String.concat
20         " -> " path);
21       exit 2
22     | Ok tree ->
23       if not quiet then begin
24         let lines = Project.format_tree tree in
25         List.iter print_endline lines
26       end
27   ) roots;
28   exit 0
29
30 open Cmdliner
31
32 let dir =
33   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
34     ~doc:"Directory containing .borg files")
35
36 let quiet =
37   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
38     errors")
39
40 let cmd : unit Cmd.t =
41   Cmd.v (Cmd.info "inline" ~doc:"show the inline tree for a borge project"
42     ~man:[ 'S "DESCRIPTION";
43       'P "Finds the root .borg file and follows inline directives to \
44         build and display the project tree. Also shows orphaned files \
45         that are not part of any inline tree." ])
46   Term.(const run $ dir $ quiet)

```

91 bin/cmd/Issue.ml

```

1  (** borge issue -- issue tracker commands *)
2
3  open Borge_lib
4
5  (* exempt doc *)
6  let now () =
7    let tm = Unix.localtime (Unix.time ()) in
8    Printf.sprintf "%04d-%02d-%02d"
9      (tm.Unix.tm_year + 1900)
10     (tm.Unix.tm_mon + 1)
11     tm.Unix.tm_mday
12
13  (* exempt doc *)
14  let get_user () =
15    try Sys.getenv "USER" with Not_found -> "unknown"
16
17  (** Create a new bug with a title and optional open status *)
18  let cmd_new title open_status with_commit =
19    ignore with_commit;
20    let id = Bug_ast.make_id () in
21    let status = if open_status then Bug_ast.Open else Bug_ast.Triage in
22    let bug = {
23      Bug_ast.id;
24      title;
25      status;
26      created = now ();
27      filed_by = get_user ();
28      doc = "";
29      affects_section = None;
30      relates_to = [];
31      drift_report_id = None;
32      resolution = None;
33    } in
34    let path = Bug_registry.save_bug bug in
35    Printf.printf "Created bug %s\n" id;
36    Printf.printf "  Status: %s\n" (Bug_ast.string_of_status status);
37    Printf.printf "  File: %s\n" path;
38    if open_status then
39      Printf.printf "Run 'borge issue doc %s' to add description.\n" id
40
41  (** List all bugs, optionally filtered by status *)
42  let cmd_list status_opt =
43    let filter_status = match status_opt with
44      | Some s -> Bug_ast.status_of_string s
45      | _ -> None
46    in
47    let bugs = Bug_registry.list_bugs ~status:filter_status () in
48    if bugs = [] then
49      Printf.printf "No bugs found.\n"
50    else begin
51      Printf.printf "Bugs (%d total):\n" (List.length bugs);
52      List.iter (fun (b : Bug_ast.bug) ->
53        Printf.printf "  %-20s [%8s] %s\n"
54          b.id (Bug_ast.string_of_status b.status) b.title
55      ) bugs
56    end
57
58  (** Show detailed info for a specific bug by ID *)
59  let cmd_show id =
60    match Bug_registry.load_bug id with
61    | None ->
62      Printf.eprintf "Bug '%s' not found.\n" id;
63      exit 1
64    | Some bug ->
65      Printf.printf "%s: %s\n" bug.id bug.title;
66      Printf.printf "Status: %s\n" (Bug_ast.string_of_status bug.status);
67      Printf.printf "Created: %s by %s\n" bug.created bug.filed_by;
68      if bug.doc <> "" then
69        Printf.printf "\n%s\n" bug.doc;
70      (match bug.affects_section with
71      | Some s -> Printf.printf "\nAffects: %s\n" s

```

```

72         | None -> ();
73         (match bug.drift_report_id with
74         | Some d -> Printf.printf "Drift report: %s\n" d
75         | None -> ())
76
77 (** Close a bug by ID, marking it as resolved *)
78 let cmd_close id with_commit =
79     ignore with_commit;
80     match Bug_registry.close_bug id ~resolution:"closed" ~by:(get_user ())
81     with
82     | Error msg ->
83         Printf.eprintf "Error: %s\n" msg;
84         exit 1
85     | Ok path ->
86         Printf.printf "Closed bug %s\n" id;
87         Printf.printf "   File: %s\n" path
88
89 open Cmdliner
90
91 (* exempt doc *)
92 let title =
93     Arg.(required & pos 0 (some string) None & info [] ~docv:"TITLE"
94     ~doc:"Bug title")
95
96 (* exempt doc *)
97 let id =
98     Arg.(required & pos 0 (some string) None & info [] ~docv:"ID"
99     ~doc:"Bug ID (e.g., BORGE-1234567890)")
100
101 (* exempt doc *)
102 let status_opt =
103     Arg.(value & opt (some string) None & info ["status"] ~docv:"STATUS"
104     ~doc:"Filter by status (triage, open, in-progress, resolved, closed)")
105
106 (* exempt doc *)
107 let open_flag =
108     Arg.(value & flag & info ["open"] ~doc:"Create as open (not triage)")
109
110 (* exempt doc *)
111 let with_commit_flag =
112     Arg.(value & flag & info ["commit"] ~doc:"Commit changes to git")
113
114 let new_cmd =
115     Cmd.v (Cmd.info "new" ~doc:"create a new bug")
116     Term.(const cmd_new $ title $ open_flag $ with_commit_flag)
117
118 let list_cmd =
119     Cmd.v (Cmd.info "list" ~doc:"list bugs")
120     Term.(const cmd_list $ status_opt)
121
122 let show_cmd =
123     Cmd.v (Cmd.info "show" ~doc:"show bug details")
124     Term.(const cmd_show $ id)
125
126 let close_cmd =
127     Cmd.v (Cmd.info "close" ~doc:"close a bug")
128     Term.(const cmd_close $ id $ with_commit_flag)
129
130 let cmd : unit Cmd.t =
131     Cmd.group (Cmd.info "issue" ~doc:"issue tracker")
132     [new_cmd; list_cmd; show_cmd; close_cmd]

```

92 bin/cmd/Lint.ml

```

1 open Borge_lib
2
3 let run dir json quiet code_doc mechanical literacy =
4   let result = Lint.run ~code_doc ~mechanical ~literacy dir in
5   if json then begin
6     Printf.printf "%s\n" (Yojson.Basic.to_string (Json_out.lint result));
7     exit (if result.error_count > 0 then 1 else 0)
8   end;
9   if not quiet then Printf.printf "Linting .borg files in '%s'...\n\n" dir;
10  let print_issue = function
11    | Lint.Invalid_status { path; value; line; col } ->
12      Printf.printf " ? %s:%d:%d: invalid status value '%s'\n" path line
13        col value
14    | Lint.Duplicate_project_name { name; paths } ->
15      Printf.printf " ? duplicate project name '%s' in: %s\n" name
16        (String.concat ", " paths)
17    | Lint.Orphaned_file { path } ->
18      Printf.printf " ? %s: orphaned .borg file\n" path
19    | Lint.Invalid_verify_method { path; method; line } ->
20      Printf.printf " ? %s:%d: invalid verify method '%s'\n" path line
21        method
22    | Lint.Unknown_comment_type { path; type_name; line } ->
23      Printf.printf " ? %s:%d: unknown comment type '%s'\n" path line
24        type_name
25    | Lint.Missing_comment_value { path; author; type_name; line } ->
26      let type_str = match type_name with Some t -> Printf.sprintf " %s"
27        t | None -> "" in
28      Printf.printf " ? %s:%d: comment by '%s%s' missing value\n" path
29        line author type_str
30    | Lint.No_inline_on_root { path } ->
31      Printf.printf " ? %s: (no-inline) on root file\n" path
32    | Lint.Deleted_human_comment { path; author; _ } ->
33      Printf.printf " ? %s: human-authored comment by '%s' was
34        deleted\n" path author
35    | Lint.Inserted_human_comment { path; author; _ } ->
36      Printf.printf " ? %s: human-authored comment by '%s' was
37        inserted\n" path author
38    | Lint.Pending_response { path; author; line; question } ->
39      Printf.printf " ? %s:%d: unanswered ask by '%s': %s\n" path line
40        author question
41    | Lint.Db_validation { path; severity; message } ->
42      let mark = match severity with 'Error -> "?" | 'Warning -> "?" in
43      Printf.printf " %s %s: DB: %s\n" mark path message
44    | Lint.Ui_validation { path; severity; message } ->
45      let mark = match severity with 'Error -> "?" | 'Warning -> "?" in
46      Printf.printf " %s %s: UI: %s\n" mark path message
47    | Lint.Undocumented_binding { path; name; line } ->
48      Printf.printf " ? %s:%d: undocumented binding '%s'\n" path line
49        name
50    | Lint.Stale_doc_comment { path; name; line } ->
51      Printf.printf " ? %s:%d: drifted doc comment on '%s'\n" path line
52        name
53    | Lint.Missing_literacy_score { path; name; line } ->
54      Printf.printf " ? %s:%d: missing literacy score on '%s'\n" path
55        line name
56    | Lint.Implausible_literacy_score { path; name; line; dimension;
57      claimed; reason } ->
58      Printf.printf " ? %s:%d: %s=%d on '%s' is implausible -- %s\n"
59        path line dimension claimed name reason
60    | Lint.Unsafe_call { path; line; call; severity; suggestion } ->
61      let mark = match severity with 'Error -> "?" | 'Warning -> "?" in
62      Printf.printf " %s %s:%d: unsafe call '%s' -- %s\n" mark path line
63        call suggestion
64  in
65  if quiet then begin
66    (* In quiet mode, only print errors to stderr, suppress warnings *)
67    List.iter (fun issue ->
68      match issue with
69      | Lint.Invalid_status _ | Lint.Duplicate_project_name _ |
70      Lint.Orphaned_file _ | Lint.Invalid_verify_method _
71      | Lint.Unsafe_call { severity = 'Error; _ } ->

```

```

57         print_issue issue
58     | _ -> ()
59 ) result.issues
60 end else begin
61     List.iter print_issue result.issues;
62     Printf.printf "\n%d errors, %d warnings\n" result.error_count
        result.warning_count;
63 end;
64 if result.error_count > 0 then exit 1 else exit 0
65
66 open Cmdliner
67
68 let dir =
69     Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
70         ~doc:"Directory to lint")
71
72 let json =
73     Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
74
75 let quiet =
76     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
        errors")
77
78 let code_doc =
79     Arg.(value & flag & info ["code-doc"] ~doc:"Also check .ml files for
        undocumented bindings")
80
81 let mechanical =
82     Arg.(value & flag & info ["mechanical"] ~doc:"Check .ml/.mli files for
        unsafe stdlib calls")
83
84 let literacy =
85     Arg.(value & flag & info ["literacy"; "l"] ~doc:"Check .ml files for
        literacy scores")
86
87 let cmd : unit Cmd.t =
88     Cmd.v (Cmd.info "lint" ~doc:"semantic validation of .borg files"
89         ~man:[ 'S "DESCRIPTION";
90             'P "Checks for: invalid status values, duplicate project names, \
        unknown comment types, missing comment values, deleted human \
        comments, pending ask/response slots, and orphaned files.";
91             'P "With --code-doc, also checks .ml files for undocumented \
        exported bindings and drifted doc comments.";
92             'P "With --mechanical, also checks .ml/.mli files for unsafe \
        standard library calls (List.hd, List.assoc, Hashtbl.find,
93             etc.).";
94             'P "With --literacy, also checks .ml files for missing or
        implausible \
        literacy scores on doc comments.";
95             'P "With --json, outputs structured JSON instead of formatted
        text." ])
96
97 Term.(const run $ dir $ json $ quiet $ code_doc $ mechanical $ literacy)
100

```

93 bin/cmd/Log.ml

```

1
2 (** borge log: git log with borge-aware summaries *)
3
4 let run dir quiet =
5   let cmd = Printf.sprintf
6     "git -C %s log --oneline --no-decorate -20 -- *.borg 2>/dev/null"
7     (Filename.quote dir) in
8   let ic = Unix.open_process_in cmd in
9   let lines = ref [] in
10  (* exempt: input_line *) (try while true do lines := input_line ic ::
11    !lines done with End_of_file -> ());
12  let _status = Unix.close_process_in ic in
13  match List.rev !lines with
14  | [] ->
15    if not quiet then Printf.printf "No borge commit history found.\n";
16    exit 0
17  | commits ->
18    if quiet then exit 0;
19    Printf.printf "Borge commit history (last %d):\n\n" (List.length
20      commits);
21    List.iter (fun line ->
22      Printf.printf "  %s\n" line
23    ) commits;
24    exit 0
25
26 open Cmdliner
27
28 let dir =
29   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
30     ~doc:"Path to git repository (default: current directory)")
31
32 let quiet =
33   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
34     errors")
35
36 let cmd : unit Cmd.t =
37   Cmd.v (Cmd.info "log" ~doc:"git log showing only borge-related commits"
38     ~man:[ 'S "DESCRIPTION";
39       'P "Shows git log filtered to commits that modified .borg files. \
40         This gives a borge-aware view of project history." ])
41   Term.(const run $ dir $ quiet)

```


94 bin/cmd/Make.ml

```

1 (** borge make -- implement code from spec
2
3     Opens pi interactively with the implementer role.
4     Creates lock, builds MINIMAL prompt (summarized, not full spec),
5     launches pi. Auto-commits on clean exit unless --no-commit. *)
6
7 open Borge_lib
8
9 let gather_minimal_data () =
10   (* Summarize all .borg files instead of loading full contents *)
11   let summaries =
12     File_utils.find_borg_files "."
13     |> List.concat_map (fun path ->
14       try
15         let content = File_utils.read_file path in
16         Prompt_minimal.summarize_borg_file content
17       with _ -> []
18     )
19   in
20   let planned = List.filter (fun (s : Prompt_minimal.section_summary) ->
21     s.Prompt_minimal.status = "planned") summaries in
22   let implemented = List.filter (fun (s : Prompt_minimal.section_summary)
23     -> s.Prompt_minimal.status = "implemented") summaries in
24   let modules =
25     try
26       let dune_files = Dune_parse.parse_all "." in
27       let lib_modules = List.concat_map (fun df ->
28         let dir = Filename.dirname df.Dune_parse.path in
29         List.concat_map (function
30           | Dune_parse.Library lib ->
31             List.map (fun m -> (dir, m)) lib.Dune_parse.modules
32           | _ -> []
33         ) df.Dune_parse.stanzas
34       ) dune_files in
35       List.map snd lib_modules
36     with _ -> []
37   in
38   (planned, implemented, modules)
39
40 let write_prompt role =
41   let planned, implemented, modules = gather_minimal_data () in
42   let prompt = Prompt_minimal.make_prompt role ~planned ~implemented
43     ~modules in
44   let path = Lock.prompt_file () in
45   let oc = open_out path in
46   output_string oc prompt;
47   close_out oc;
48   path
49
50 let run_no_commit dir quiet =
51   ignore dir; (* TODO: support non-root dirs *)
52   (match Lock.create () with
53   | Error msg ->
54     Printf.eprintf "Error: %s\n" msg;
55     exit 1
56   | Ok () -> ());
57
58   let prompt_path = write_prompt Prompt_minimal.Implementer in
59   if not quiet then begin
60     Printf.printf "Lock created at .borg.lock/\n";
61     Printf.printf "Prompt written to %s\n" prompt_path;
62   end;
63
64   let exit_status = Lock_pi_launch.launch prompt_path in
65
66   if exit_status = 0 then begin
67     if not quiet then Printf.printf "\nAgent session completed (exit 0).\n";
68     if no_commit then begin
69       if not quiet then begin
70         Printf.printf "--no-commit: changes left in working tree.\n";

```

```

68     Printf.printf "Run 'borge commit' to finalize or 'borge abort' to
discard.\n";
69     end;
70     Lock.set_state "completed";
71     exit 0
72 end else begin
73     match Lock.teardown ~commit:true with
74     | Ok msg ->
75         if not quiet then Printf.printf "%s\n" msg;
76         Lock.remove ();
77         if not quiet then Printf.printf "Committed and lock removed.\n";
78         exit 0
79     | Error msg ->
80         Printf.eprintf "Commit failed: %s\n" msg;
81         Printf.eprintf "Run 'borge commit --force' to override or 'borge
abort' to discard.\n";
82         Lock.set_state "failed";
83         exit 1
84     end
85 end else begin
86     Printf.eprintf "\nAgent session failed (exit %d).\n" exit_status;
87     Printf.eprintf "Lock kept at .borge.lock/ for review.\n";
88     Lock.set_state "failed";
89     exit exit_status
90 end
91
92 open Cmdliner
93
94 (* exempt doc *)
95 let no_commit =
96     Arg.(value & flag & info ["no-commit"]
97         ~doc:"Leave changes in working tree; do not auto-commit")
98
99 (* exempt doc *)
100 let dir =
101     Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
102         ~doc:"Project directory")
103
104 (* exempt doc *)
105 let quiet =
106     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
errors")
107
108 let cmd : unit Cmd.t =
109     Cmd.v (Cmd.info "make" ~doc:"make code match spec [launches pi]"
110         ~man:[ 'S "DESCRIPTION";
111             'P "Opens an interactive pi session with role: implementer.";
112             'P "The agent reads the full spec and writes code + tests.";
113             'P "Auto-commits on clean exit (use --no-commit to keep lock).";
114             'P "See also: borge plan (edit spec), borge drift (check
alignment)."] )
115     Term.(const run $ no_commit $ dir $ quiet)

```

95 bin/cmd/Merge.ml

```

1  (* borge merge -- process merge queue
2
3      Commands:
4      --next: Merge the first ready item
5      --all: Merge all ready items in order
6      --dry-run: Show what would be merged without executing *)
7
8  open Borge_lib
9  open Queue_types
10
11  let run_merge ~item ~strategy ~dry_run () =
12      let strategy = Merge_strategies.strategy_of_string strategy |>
13          Option.value ~default:Merge_strategies.FastForward in
14
15      Printf.printf "Merging %s (%s)\n" item.id item.summary;
16      Printf.printf "  Branch: %s\n" item.branch;
17      Printf.printf "  Strategy: %s\n" (Merge_strategies.string_of_strategy
18          strategy);
19      Printf.printf "  Files: %d\n" (List.length item.affected_files);
20
21      if dry_run then begin
22          Printf.printf "  [DRY-RUN] Would merge with %s\n"
23              (Merge_strategies.string_of_strategy strategy);
24          Ok ()
25      end else begin
26          (* Update state to Merging *)
27          (match Queue_storage.load () with
28          | Error e -> Error (Printf.sprintf "Failed to load queue: %s" e)
29          | Ok queue ->
30              let queue' = update_state queue item.id Merging in
31              ignore (Queue_storage.save queue');
32
33              (* Execute merge *)
34              let before_merge =
35                  match Merge_strategies.git_cmd "rev-parse HEAD" with
36                  | Unix.WEXITED 0, hash -> Some (String.trim hash)
37                  | _ -> None
38              in
39
40              let message = Printf.sprintf "Merge %s: %s" item.id item.summary in
41              let _, result = Merge_strategies.execute ~strategy ~message
42                  item.branch in
43
44              (* Update state based on result *)
45              (match Queue_storage.load () with
46              | Ok queue ->
47                  let queue'' = match result with
48                  | Success _ ->
49                      Printf.printf " ? Merged successfully\n";
50                      update_state queue item.id Merged
51                  | Conflicts files ->
52                      Printf.printf " ? Conflicts in: %s\n" (String.concat
53                          ", " files);
54                      let item' = { item with
55                          state = Rejected;
56                          rejected_reason = Some ("Conflicts: " ^ String.concat
57                              ", " files)
58                      } in
59                      { queue with items = List.map (fun i ->
60                          if i.id = item.id then item' else i) queue.items }
61                  | Error msg ->
62                      Printf.printf " ? Merge failed: %s\n" msg;
63                      let item' = { item with
64                          state = Rejected;
65                          rejected_reason = Some msg
66                      } in
67                      { queue with items = List.map (fun i ->
68                          if i.id = item.id then item' else i) queue.items }
69              in
70              ignore (Queue_storage.save queue'');
71

```

```

66         (* Rollback if failed *)
67         (match result with
68         | Success _ -> ()
69         | _ ->
70             (match Merge_strategies.git_cmd "merge --abort" with
71             | _ -> ());
72             ignore (Rollback.rollback_merge ~type_:Soft
~before_merge ~branch:item.branch result));
73
74         Ok ()
75         | Error _ -> Ok ()))
76     end
77
78 let merge_next ~strategy ~dry_run () =
79     match Queue_storage.load () with
80     | Error e ->
81         Printf.eprintf "Error loading queue: %s\n" e;
82         exit 1
83     | Ok queue ->
84         match next_ready queue with
85         | None ->
86             Printf.printf "No ready items in queue\n";
87             exit 0
88         | Some item ->
89             (match run_merge ~item ~strategy ~dry_run () with
90             | Ok () -> exit 0
91             | Error e ->
92                 Printf.eprintf "Merge failed: %s\n" e;
93                 exit 1)
94
95 let merge_all ~strategy ~dry_run () =
96     let rec merge_loop count =
97         match Queue_storage.load () with
98         | Error e ->
99             Printf.eprintf "Error loading queue: %s\n" e;
100             exit 1
101         | Ok queue ->
102             match next_ready queue with
103             | None ->
104                 Printf.printf "Merged %d item(s)\n" count;
105                 exit 0
106             | Some item ->
107                 (match run_merge ~item ~strategy ~dry_run () with
108                 | Ok () -> merge_loop (count + 1)
109                 | Error e ->
110                     Printf.eprintf "Merge failed: %s\n" e;
111                     exit 1)
112     in
113     merge_loop 0
114
115 open Cmdliner
116
117 (* exempt doc *)
118 let strategy =
119     Arg.(value & opt string "fast-forward"
120     & info ["strategy"; "s"]
121     ~docv:"STRATEGY"
122     ~doc:"Merge strategy: fast-forward, rebase, merge, or squash")
123
124 (* exempt doc *)
125 let dry_run =
126     Arg.(value & flag & info ["dry-run"; "n"]
127     ~doc:"Show what would be merged without executing")
128
129 (* exempt doc *)
130 let next =
131     Arg.(value & flag & info ["next"]
132     ~doc:"Merge the first ready item")
133
134 (* exempt doc *)
135 let all =
136     Arg.(value & flag & info ["all"; "a"]
137     ~doc:"Merge all ready items")

```

```

138
139 let run strategy dry_run next all =
140   if not next && not all then begin
141     Printf.eprintf "Error: specify --next or --all\n";
142     exit 1
143   end;
144   if next && all then begin
145     Printf.eprintf "Error: cannot specify both --next and --all\n";
146     exit 1
147   end;
148   if next then
149     merge_next ~strategy ~dry_run ()
150   else
151     merge_all ~strategy ~dry_run ()
152
153 let cmd : unit Cmd.t =
154   Cmd.v (Cmd.info "merge" ~doc:"process merge queue"
155     ~man:[`S "DESCRIPTION";
156       `P "Merges ready items from the agent merge queue.";
157       `P "Use --next to merge one item, --all to merge all ready
158         items.";
159       `P "Supports multiple merge strategies via --strategy.";
160       `S "EXAMPLES";
161       `P "borge merge --next                # Merge first ready
162         item";
163       `P "borge merge --all --strategy squash # Squash merge all
164         ready";
165       `P "borge merge --next --dry-run       # Preview without
166         merging";])
167   Term.(const run $ strategy $ dry_run $ next $ all)

```

96 bin/cmd/Merge_queue.ml

```

1  (* borge merge-queue -- manage agent merge queue
2
3  Commands:
4  - list: Show pending merge queue items
5  - show ID: Show details of specific item
6  - submit --worktree PATH: Submit worktree to queue
7  - cancel ID: Cancel a queued item *)
8
9  open Borge_lib
10 open Queue_types
11
12 (* agent note (|
13  * WHAT: List all pending items in the merge queue, displaying
14  * their ID, state, branch, confidence, and summary.
15  *
16  * WHY: The primary command to see what's in the queue waiting
17  * to be merged.
18  * |) *)
19 let list_items () =
20   match Queue_storage.load () with
21   | Error e ->
22     Printf.eprintf "Error loading queue: %s\n" e;
23     exit 1
24   | Ok queue ->
25     if queue.items = [] then
26       Printf.printf "Merge queue is empty.\n"
27     else begin
28       Printf.printf "Merge Queue (%d items):\n\n" (List.length
29         queue.items);
30       Printf.printf "%-10s %-12s %-20s %-8s %s\n" "ID" "STATE" "BRANCH"
31         "CONF" "SUMMARY";
32       Printf.printf "%s\n" (String.make 80 '-');
33       List.iter (fun (item : queue_item) ->
34         let id_short =
35           if String.length item.id > 8 then String.sub item.id 0 8 else
36             item.id
37         in
38         Printf.printf "%-10s %-12s %-20s %6.0f%% %.40s\n"
39           id_short
40           (string_of_state item.state)
41           (Filename.basename item.branch)
42           (item.confidence *. 100.0)
43           item.summary
44       ) queue.items;
45       let counts = count_by_state queue in
46       Printf.printf "\nSummary: %d pending, %d ready, %d blocked, %d
47         merging, %d merged, %d rejected\n"
48         (List.assoc_opt "pending" counts |> Option.value ~default:0)
49         (List.assoc_opt "ready" counts |> Option.value ~default:0)
50         (List.assoc_opt "blocked" counts |> Option.value ~default:0)
51         (List.assoc_opt "merging" counts |> Option.value ~default:0)
52         (List.assoc_opt "merged" counts |> Option.value ~default:0)
53         (List.assoc_opt "rejected" counts |> Option.value ~default:0);
54     end
55
56 (* exempt doc: CLI command handler - purpose is clear from context *)
57 let show_item id =
58   match Queue_storage.load () with
59   | Error e ->
60     Printf.eprintf "Error loading queue: %s\n" e;
61     exit 1
62   | Ok queue ->
63     match find_item queue id with
64     | None ->
65       (* Try to find by prefix *)
66       let matches = List.filter (fun (i : queue_item) ->
67         String.starts_with ~prefix:id i.id
68       ) queue.items in
69       (match matches with
70       | [item] -> show_item_detail item
71       | [] ->

```

```

68         Printf.eprintf "Item not found: %s\n" id;
69         exit 1
70     | _ ->
71         Printf.eprintf "Multiple items match prefix '%s', please use
full ID\n" id;
72         exit 1)
73     | Some item -> show_item_detail item
74
75 (* exempt doc: CLI detail printer - purpose is clear from context *)
76 let show_item_detail (item : queue_item) =
77     Printf.printf "Merge Queue Item: %s\n" item.id;
78     Printf.printf "  State: %s\n" (string_of_state item.state);
79     Printf.printf "  Branch: %s\n" item.branch;
80     Printf.printf "  Worktree: %s\n" item.worktree_path;
81     Printf.printf "  Summary: %s\n" item.summary;
82     Printf.printf "  Confidence: %.2f\n" item.confidence;
83     Printf.printf "  Submitted: %s\n" item.submitted_at;
84     Printf.printf "  Affected files: %d\n" (List.length item.affected_files);
85     List.iter (Printf.printf "    - %s\n") (List.take 10 item.affected_files);
86     if List.length item.affected_files > 10 then
87         Printf.printf "    ... and %d more\n" (List.length item.affected_files
- 10);
88     (match item.merged_at with
89     | Some t -> Printf.printf "  Merged: %s\n" t
90     | None -> ());
91     (match item.rejected_reason with
92     | Some r -> Printf.printf "  Rejected: %s\n" r
93     | None -> ());
94
95 (* Show conflicts if blocked *)
96 if item.state = Blocked then
97     let queue = match Queue_storage.load () with Ok q -> q | Error _ ->
empty_queue in
98     let conflicts = Conflict_detect.is_blocked_by queue item.id in
99     if conflicts <> [] then begin
100         Printf.printf "\n  Blocked by conflicts with:\n";
101         List.iter (fun (c : Conflict_detect.conflict) ->
102             Printf.printf "    - %s: %s\n" c.item1_id
103             (String.concat ", " c.conflicting_files)
104         ) conflicts
105     end
106
107 (* agent note (|
108 *   WHAT: Submit a completed worktree to the merge queue.
109 *   Validates the worktree exists, extracts branch name, detects
110 *   affected files, checks for lockfile conflicts, and creates
111 *   a queue item for the worktree.
112 *
113 *   WHY: This is how agent worktrees get merged into main after
114 *   successful completion.
115 * |) *)
116 let submit_worktree ~worktree_path ~summary ~confidence =
117     (* Validate worktree exists *)
118     if not (Sys.file_exists worktree_path) then begin
119         Printf.eprintf "Error: Worktree not found: %s\n" worktree_path;
120         exit 1
121     end;
122
123     (* Get branch name from worktree *)
124     let branch =
125         let cmd = Printf.sprintf "cd %s && git branch --show-current
2>/dev/null" worktree_path in
126         let ic = Unix.open_process_in cmd in
127         try
128             (* exempt: input_line *) let b = input_line ic |> String.trim in
129             ignore (Unix.close_process_in ic);
130             b
131         with End_of_file ->
132             ignore (Unix.close_process_in ic);
133             Printf.eprintf "Error: Could not determine branch for worktree\n";
134             exit 1
135     in
136

```

```

137 (* Get affected files *)
138 let affected_files = Branch.changed_files branch in
139
140 (* Check for lockfile conflicts *)
141 let conflicts = Lockfile.check_conflicts affected_files in
142 if conflicts <> [] then begin
143   Printf.eprintf "Error: Files conflict with active runs:\n";
144   List.iter (fun (run : Lockfile.run_lock) ->
145     Printf.eprintf " - %s: %s\n" run.id (String.concat " " run.files)
146   ) conflicts;
147   exit 1
148 end;
149
150 (* Create queue item *)
151 let id =
152   let timestamp = Unix.gmtime (Unix.time ()) in
153   Printf.sprintf "mq-%04d%02d%02d-%02d%02d%02d-%04x"
154     (timestamp.tm_year + 1900) (timestamp.tm_mon + 1) timestamp.tm_mday
155     timestamp.tm_hour timestamp.tm_min timestamp.tm_sec
156     (Random.int 0x10000)
157 in
158 let submitted_at =
159   let now = Unix.gmtime (Unix.time ()) in
160   Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
161     (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
162     now.tm_hour now.tm_min now.tm_sec
163 in
164 let item = {
165   id;
166   state = Pending;
167   branch;
168   worktree_path;
169   summary;
170   affected_files;
171   confidence;
172   submitted_at;
173   merged_at = None;
174   rejected_reason = None;
175 } in
176
177 (* Load queue, add item, recalculate states, save *)
178 (match Queue_storage.load () with
179 | Error e ->
180   Printf.eprintf "Error loading queue: %s\n" e;
181   exit 1
182 | Ok queue ->
183   let queue' = add_item queue item in
184   let queue'' = Conflict_detect.recalculate_states queue' in
185   (match Queue_storage.save queue'' with
186   | Error e ->
187     Printf.eprintf "Error saving queue: %s\n" e;
188     exit 1
189   | Ok () ->
190     Printf.printf "Submitted item %s to merge queue\n" id;
191     (match item.state with
192     | Ready -> Printf.printf "Item is READY and can be merged\n"
193     | Blocked -> Printf.printf "Item is BLOCKED - conflicts with
194       earlier items\n"
195     | _ -> ())))
196
197 let cancel_item id =
198   (match Queue_storage.load () with
199   | Error e ->
200     Printf.eprintf "Error loading queue: %s\n" e;
201     exit 1
202   | Ok queue ->
203     match find_item queue id with
204     | None ->
205       Printf.eprintf "Item not found: %s\n" id;
206       exit 1
207     | Some item ->
208       if item.state = Merging then begin
209         Printf.eprintf "Cannot cancel item currently being merged\n";

```



```

209         exit 1
210     end;
211     let queue' = remove_item queue id in
212     (match Queue_storage.save queue' with
213     | Error e ->
214         Printf.eprintf "Error saving queue: %s\n" e;
215         exit 1
216     | Ok () ->
217         Printf.printf "Cancelled item %s\n" id))
218
219 open Cmdliner
220
221 (* exempt doc: cmdliner command definition - purpose is clear *)
222 let list_cmd : unit Cmd.t =
223     Cmd.v (Cmd.info "list" ~doc:"List all queue items")
224     Term.(const list_items $ const ())
225
226 (* exempt doc: cmdliner command definition - purpose is clear *)
227 let show_cmd : unit Cmd.t =
228     let id =
229         Arg.(required & pos 0 (some string) None & info [] ~docv:"ID"
230             ~doc:"Item ID (or prefix)")
231     in
232     Cmd.v (Cmd.info "show" ~doc:"Show item details")
233     Term.(const show_item $ id)
234
235 (* exempt doc: cmdliner command definition - purpose is clear *)
236 let submit_cmd : unit Cmd.t =
237     let worktree =
238         Arg.(required & opt (some string) None & info ["worktree"; "w"]
239             ~docv:"PATH" ~doc:"Path to agent worktree")
240     in
241     let summary =
242         Arg.(value & opt string "" & info ["summary"; "s"]
243             ~docv:"TEXT" ~doc:"Summary of changes")
244     in
245     let confidence =
246         Arg.(value & opt float 0.5 & info ["confidence"; "c"]
247             ~docv:"0.0-1.0" ~doc:"Confidence score (0.0-1.0)")
248     in
249     Cmd.v (Cmd.info "submit" ~doc:"Submit worktree to merge queue")
250     Term.(const submit_worktree $ worktree $ summary $ confidence)
251
252 let cancel_cmd : unit Cmd.t =
253     let id =
254         Arg.(required & pos 0 (some string) None & info [] ~docv:"ID"
255             ~doc:"Item ID to cancel")
256     in
257     Cmd.v (Cmd.info "cancel" ~doc:"Cancel a queued item")
258     Term.(const cancel_item $ id)
259
260 let cmd : unit Cmd.t =
261     Cmd.v (Cmd.info "merge-queue" ~doc:"manage agent merge queue"
262         ~man:[ 'S "DESCRIPTION";
263             'P "Manages the queue of agent changes waiting to be merged.";
264             'P "Items are submitted after agent runs complete.";
265             'P "The queue processes items in order, checking for conflicts.";
266             'S "COMMANDS";
267             'I ("list", "Show all queue items with their states");
268             'I ("show ID", "Show detailed info for an item");
269             'I ("submit --worktree PATH", "Submit a worktree to the queue");
270             'I ("cancel ID", "Remove an item from the queue")]])
271     (Cmd.group (Cmd.info "merge-queue") [list_cmd; show_cmd; submit_cmd;
        cancel_cmd])

```

97 bin/cmd/Modules.ml

```

1 (** borge modules -- check distributed module specs
2
3     Discovers module-level .borg files and checks referential
4     integrity between spec exports and code implementations. *)
5
6 open Borge_lib
7
8 (** Check all module specs for referential integrity *)
9 let run_check ~dir ~json =
10   let specs = Module_spec.load_all_specs dir in
11
12   if specs = [] then begin
13     Printf.printf "No module specs found in %s\n" dir;
14     exit 0
15   end;
16
17   let reports = List.map (fun (spec : Module_spec.module_def) ->
18     let impl_files = Module_spec.find_implementation_files spec.path in
19     let merged_issues = List.fold_left (fun acc impl_path ->
20       let issues = Module_spec.check_integrity spec impl_path in
21       let (missing1, undoc1) = acc in
22       let (missing2, undoc2) = issues in
23       (missing1 @ missing2, undoc1 @ undoc2)
24     ) ([], []) impl_files in
25     (spec, merged_issues)
26   ) specs in
27
28   if json then begin
29     (* TODO: JSON output *)
30     Printf.printf "{\"modules\": []}\n"
31   end else begin
32     Printf.printf "Module spec integrity check for '%s'\n\n" dir;
33
34     let total_issues = ref 0 in
35     List.iter (fun ((spec : Module_spec.module_def), issues) ->
36       let report = Module_spec.render_report spec.module_name issues in
37       Printf.printf "%s\n\n" report;
38       let (missing, undoc) = issues in
39       total_issues := !total_issues + List.length missing + List.length
40         undoc
41     ) reports;
42
43     Printf.printf "Total issues: %d\n" !total_issues;
44     if !total_issues > 0 then exit 1 else exit 0
45   end
46
47 (** List all discovered module specs without checking *)
48 let run_list ~dir =
49   let specs = Module_spec.load_all_specs dir in
50   if specs = [] then
51     Printf.printf "No module specs found.\n"
52   else begin
53     Printf.printf "Module specs found:\n";
54     List.iter (fun (spec : Module_spec.module_def) ->
55       Printf.printf "  %s (%d exports)\n" spec.module_name (List.length
56         spec.exports)
57     ) specs
58   end
59
60 (** Main entry point: route to check/list/function-names *)
61 let run function_names path dir list json =
62   if list then run_list ~dir
63   else if function_names then begin
64     match path with
65     | None ->
66       Printf.eprintf "Error: --functions requires --path FILE\n";
67       exit 1
68     | Some p ->
69       let names = Borg_comment.function_names p in
70       Printf.printf "Functions in %s:\n" p;
71       List.iter (Printf.printf "  - %s\n") names

```

```

70   end
71   else run_check ~dir ~json
72
73 open Cmdliner
74
75 (* exempt doc *)
76 let function_names_flag =
77   Arg.(value & flag & info ["functions"] ~doc:"List functions in a file")
78
79 (* exempt doc *)
80 let path =
81   Arg.(value & opt (some string) None & info ["path"] ~docv:"FILE"
82     ~doc:"Path to OCaml file")
83
84 (* exempt doc *)
85 let dir =
86   Arg.(value & opt string "." & info ["dir"; "d"] ~docv:"DIR"
87     ~doc:"Project directory")
88
89 (* exempt doc *)
90 let list_flag =
91   Arg.(value & flag & info ["list"; "l"] ~doc:"List discovered module
92     specs")
93
94 (* exempt doc *)
95 let json =
96   Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
97
98 let cmd : unit Cmd.t =
99   Cmd.v (Cmd.info "modules" ~doc:"check distributed module specs"
100     ~man:[S "DESCRIPTION";
101       'P "Discovers module-level .borg files and checks integrity.";
102       'P "Verifies that exported functions exist in code (and vice
103         versa).";
104       'P "Non-semantic check: only name referential integrity.";
105       'S "EXAMPLES";
106       'P "borg modules                # check all modules";
107       'P "borg modules --list          # list discovered specs";
108       'P "borg modules --functions --path lib/core/spec.ml"]])
109   Term.(const run $ function_names_flag $ path $ dir $ list_flag $ json)

```

98 bin/cmd/Nodes.ml

```
1 open Borge_lib
2
3 let run path quiet =
4   let result = Nodes.run path in
5   if not quiet then begin
6     let output = Nodes.format_result result in
7     print_string output;
8   end;
9   exit 0
10
11 open Cmdliner
12
13 let path =
14   Arg.(required & pos 0 (some file) None & info [] ~docv:"FILE"
15     ~doc:"The .borg file to analyze")
16
17 let quiet =
18   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
19     errors")
20
21 let cmd : unit Cmd.t =
22   Cmd.v (Cmd.info "nodes" ~doc:"dump the parsed AST with types and
23     positions"
24     ~man:[ 'S "DESCRIPTION";
25       'P "Parses the .borg file and prints each AST node with its type,
26         \
27           keyword (if list), source position, and child count." ])
28   Term.(const run $ path $ quiet)
```

99 bin/cmd/Normalize.ml

```

1 open Borge_lib
2
3 (** Normalize: fmt all .borg files, accept comment deletions as intentional.
4     This is the human saying "I made changes on purpose." *)
5
6 let run dir quiet =
7   if not quiet then Printf.printf "Normalizing .borg files in '%s'...\n"
8     dir;
9   (* Find all .borg files via the project tree *)
10  let roots = Project.find_roots dir in
11  let tree_paths = match roots with
12    | [] -> File_utils.find_borg_files dir
13    | _ ->
14      List.concat_map (fun root ->
15        match Project.build_tree root with
16        | Ok tree -> Project.tree_paths tree
17        | Error _ -> [root]
18      ) roots
19  in
20  (* Format each file in place *)
21  let formatted = ref 0 in
22  let unchanged = ref 0 in
23  List.iter (fun path ->
24    let input = File_utils.read_file path in
25    let file = Borge_lang.Parse.parse_file input in
26    let output = Borge_lang.Print.print_file file in
27    if Fmt.strip_trailing_newlines input <> Fmt.strip_trailing_newlines
28      output then begin
29      let oc = open_out path in
30      output_string oc output;
31      close_out oc;
32      if not quiet then Printf.printf "  formatted: %s\n"
33        (Filename.basename path);
34      incr formatted
35    end else
36      incr unchanged
37  ) tree_paths;
38  if not quiet then begin
39    Printf.printf "\n%d files formatted, %d unchanged.\n" !formatted
40      !unchanged;
41    Printf.printf "Comment deletions are now accepted as intentional.\n";
42  end;
43  exit 0
44
45 open Cmdliner
46
47 let dir =
48   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
49     ~doc:"Directory containing .borg files")
50
51 let quiet =
52   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
53     errors")
54
55 let cmd : unit Cmd.t =
56   Cmd.v (Cmd.info "normalize" ~doc:"fmt all files and accept comment
57     deletions"
58     ~man:[ 'S "DESCRIPTION";
59       'P "Runs borge fmt on all .borg files in the project tree. \
60         After normalize, any comment deletions are considered
61         intentional -- \
62         borge lint will no longer warn about them." ])
63   Term.(const run $ dir $ quiet)

```

100 bin/cmd/Parse.ml

```

1 open Borge_lib
2
3 (** Read entire file contents into a string *)
4 let parse_file path = File_utils.read_file path
5
6 (** Parse a .borg file and optionally print diagnostics *)
7 let run path json diagnostics quiet =
8   let input = parse_file path in
9   try
10    let file = Borge_lang.Parse.parse input in
11    if json then
12      Printf.printf "JSON output not yet implemented\n"
13    else if diagnostics then
14      Printf.printf "Diagnostics output not yet implemented\n"
15    else if not quiet then begin
16      let name = Borge_lib.Spec.project_name file in
17      let sections = Borge_lib.Spec.count_sections file in
18      let statuses = Borge_lib.Spec.statuses file in
19      Printf.printf "Valid .borg file: %s\n" path;
20      (match name with
21       | Some n -> Printf.printf " Project: %s\n" n
22       | None -> ());
23      Printf.printf " Sections: %d\n" sections;
24      let by_status = List.fold_left (fun acc st ->
25        let count = match List.assoc_opt st acc with Some v -> v + 1 | None
26        -> 1 in
27        List.sort (fun (a, _) (b, _) -> compare a b) ((st, count) ::
28        List.remove_assoc st acc)
29        ) [] statuses in
30      List.iter (fun (st, count) ->
31        Printf.printf " %s: %d\n" (Borge_lib.Spec.string_of_status st)
32        count
33        ) by_status
34      end
35    with
36    | Borge_lang.Error.Parse_error e ->
37      Printf.eprintf "Parse error at line %d, column %d: %s\n" e.line
38      e.column e.message;
39      exit 1
40
41 open Cmdliner
42
43 (* exempt doc *)
44 let path =
45   Arg.(required & pos 0 (some file) None & info [] ~docv:"FILE"
46   ~doc:"The .borg file to parse")
47
48 (* exempt doc *)
49 let json =
50   Arg.(value & flag & info ["json"] ~doc:"Output AST as JSON")
51
52 (* exempt doc *)
53 let diagnostics =
54   Arg.(value & flag & info ["diagnostics"] ~doc:"Output LSP-compatible
55   diagnostics")
56
57 (* exempt doc *)
58 let quiet =
59   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
60   errors")
61
62 let cmd : unit Cmd.t =
63   Cmd.v (Cmd.info "parse" ~doc:"parse a .borg file and validate its
64   structure"
65   ~man:['S "DESCRIPTION";
66   'P "Parse a .borg file and validate its structure. \
67   Exit 0 if the file is valid, 1 if not."])
68   Term.(const run $ path $ json $ diagnostics $ quiet)

```

101 bin/cmd/Plan.ml

```

1 (** borge plan -- edit the spec
2
3     Opens pi interactively with the spec-writer role.
4     Creates lock, builds prompt, launches pi.
5     Auto-commits on clean exit unless --no-commit. *)
6
7 open Borge_lib
8
9 let gather_prompt_data () =
10   let spec_files =
11     File_utils.find_borg_files "."
12     |> List.map (fun path -> (path, File_utils.read_file path))
13   in
14   let module_surfaces =
15     try
16       let dune_files = Dune_parse.parse_all "." in
17       let lib_modules = List.concat_map (fun df ->
18         let dir = Filename.dirname df.Dune_parse.path in
19         List.concat_map (function
20           | Dune_parse.Library lib ->
21             List.map (fun m -> (dir, m)) lib.Dune_parse.modules
22           | _ -> [])
23         ) df.Dune_parse.stanzas
24       ) dune_files in
25     List.filter_map (fun (dir, mod_name) ->
26       let ml_path = Filename.concat dir (mod_name ^ ".ml") in
27       if Sys.file_exists ml_path then
28         try
29           let surface = Surface.extract_surface ml_path in
30           Some (mod_name, surface.Surface.exports)
31         with _ -> Some (mod_name, [])
32       else None
33     ) lib_modules
34   with _ -> []
35 in
36   (spec_files, module_surfaces)
37
38 let write_prompt role =
39   let spec_files, module_surfaces = gather_prompt_data () in
40   let prompt = Lock_prompt.make_prompt role ~convention:(Convention.name
41     (Convention.resolve ".")) ~spec_files ~module_surfaces in
42   let path = Lock_prompt_file () in
43   let oc = open_out path in
44   output_string oc prompt;
45   close_out oc;
46   path
47
48 let run_no_commit dir quiet =
49   ignore dir;
50   (match Lock.create () with
51   | Error msg ->
52     Printf.eprintf "Error: %s\n" msg;
53     exit 1
54   | Ok () -> ());
55
56 let prompt_path = write_prompt Lock_prompt.Spec_writer in
57 if not quiet then begin
58   Printf.printf "Lock created at .borge.lock/\n";
59   Printf.printf "Prompt written to %s\n" prompt_path;
60 end;
61
62 let exit_status = Lock_pi_launch.launch prompt_path in
63
64 if exit_status = 0 then begin
65   if not quiet then Printf.printf "\nAgent session completed (exit 0).\n";
66   if no_commit then begin
67     if not quiet then begin
68       Printf.printf "--no-commit: changes left in working tree.\n";
69       Printf.printf "Run 'borge commit' to finalize or 'borge abort' to
70       discard.\n";
71     end;

```

```

70     Lock.set_state "completed";
71     exit 0
72 end else begin
73     match Lock.teardown ~commit:true with
74     | Ok msg ->
75         if not quiet then Printf.printf "%s\n" msg;
76         Lock.remove ();
77         if not quiet then Printf.printf "Committed and lock removed.\n";
78         exit 0
79     | Error msg ->
80         Printf.eprintf "Commit failed: %s\n" msg;
81         Printf.eprintf "Run 'borg commit --force' to override or 'borg
abort' to discard.\n";
82         Lock.set_state "failed";
83         exit 1
84     end
85 end else begin
86     Printf.eprintf "\nAgent session failed (exit %d).\n" exit_status;
87     Printf.eprintf "Lock kept at .borg.lock/ for review.\n";
88     Lock.set_state "failed";
89     exit exit_status
90 end
91
92 open Cmdliner
93
94 (* exempt doc *)
95 let no_commit =
96     Arg.(value & flag & info ["no-commit"]
97         ~doc:"Leave changes in working tree; do not auto-commit")
98
99 (* exempt doc *)
100 let dir =
101     Arg.(value & opt dir "." & info ["dir"; "d"] ~docv:"DIR"
102         ~doc:"Project directory")
103
104 (* exempt doc *)
105 let quiet =
106     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
errors")
107
108 let cmd : unit Cmd.t =
109     Cmd.v (Cmd.info "plan" ~doc:"edit the spec interactively [launches pi]"
110         ~man:['S "DESCRIPTION";
111             'P "Opens an interactive pi session with role: spec-writer.";
112             'P "The agent reads the full spec and adds/refines sections.";
113             'P "Auto-commits on clean exit (use --no-commit to keep lock).";
114             'P "See also: borge make (write code), borge drift (check
alignment)."]])
115     Term.(const run $ no_commit $ dir $ quiet)

```


102 bin/cmd/Print.ml

```
1 open Borge_lib
2
3 let run path quiet =
4   let input = File_utils.read_file path in
5   let file = Borge_lang.Parse.parse_file input in
6   let output = Borge_lang.Print.print_file file in
7   if not quiet then print_string output;
8   exit 0
9
10 open Cmdliner
11
12 let path =
13   Arg.(required & pos 0 (some file) None & info [] ~docv:"FILE"
14     ~doc:"The .borg file to print")
15
16 let quiet =
17   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
18     errors")
19
20 let cmd : unit Cmd.t =
21   Cmd.v (Cmd.info "print" ~doc:"parse and re-print a .borg file"
22     ~man:[ 'S "DESCRIPTION";
23       'P "Parses the .borg file and re-prints it with canonical
24         formatting. \
25           Useful for verifying parse round-trips." ])
26   Term.(const run $ path $ quiet)
```

103 bin/cmd/Report.ml

```

1 open Borge_lib
2
3 (** Pad a string to reach a minimum length with spaces *)
4 let pad_right n s =
5   let len = String.length s in
6   if len >= n then s else s ^ String.make (n - len) ' '
7
8 (** Run report analysis on all .borg files in a directory *)
9 let run dir json quiet =
10  let result = Report.run dir in
11  if json then begin
12    if not quiet then Printf.printf "%s\n" (Yojson.Basic.to_string
13      (Json_out.report result));
14    exit 0
15  end;
16  if quiet then exit 0;
17  Printf.printf "Borge Report -- %d .borg file(s) in '%s'\n\n" (List.length
18    result.files) dir;
19  let max_name = List.fold_left (fun acc (r : Report.file_stats) ->
20    let display = match r.project_name with
21      | Some n -> Printf.sprintf "%s (%s)" (Filename.basename r.path) n
22      | None -> Filename.basename r.path
23    in
24    max acc (String.length display)
25  ) 10 result.files in
26  Printf.printf "%s implemented verified in-progress planned\n"
27    (pad_right max_name "file");
28  Printf.printf "%s\n" (String.make (max_name + 38) '-');
29  List.iter (fun (r : Report.file_stats) ->
30    let display = match r.project_name with
31      | Some n -> Printf.sprintf "%s (%s)" (Filename.basename r.path) n
32      | None -> Filename.basename r.path
33    in
34    Printf.printf "%s      %3d      %3d      %3d      %3d\n"
35      (pad_right max_name display) r.implemented r.verified r.in_progress
36      r.planned
37  ) result.files;
38  if result.files <> [] then (
39    Printf.printf "%s\n" (String.make (max_name + 38) '-');
40    Printf.printf "%s      %3d      %3d      %3d      %3d\n"
41      (pad_right max_name "total") result.totals.implemented
42      result.totals.verified result.totals.in_progress result.totals.planned
43  );
44  let total = result.totals.implemented + result.totals.verified +
45    result.totals.in_progress + result.totals.planned in
46  Printf.printf "\n%d implemented | %d verified | %d in-progress | %d
47    planned | %d total tracked\n"
48    result.totals.implemented result.totals.verified
49    result.totals.in_progress result.totals.planned total;
50  if result.totals.planned > 0 then (
51    let pct = float_of_int (result.totals.implemented +
52      result.totals.verified) *. 100.0 /. float_of_int total in
53    Printf.printf "Completion: %.1f%%\n" pct
54  );
55  if result.orphans <> [] then begin
56    Printf.printf "\nOrphaned .borg files (no parent, no no-inline):\n";
57    List.iter (fun path ->
58      Printf.printf " ? %s\n" path
59    ) result.orphans
60  end;
61  exit 0
62
63 open Cmdliner
64
65 (* exempt doc *)
66 let dir =
67   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
68     ~doc:"Directory to scan for .borg files")
69
70 (* exempt doc *)
71 let json =

```

```
63   Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
64
65   (* exempt doc *)
66   let quiet =
67     Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
        errors")
68
69   let cmd : unit Cmd.t =
70     Cmd.v (Cmd.info "report" ~doc:"show status summary across all .borg files"
71       ~man:[`S "DESCRIPTION";
72         `P "Finds all .borg files recursively and reports how many nodes \
73           are planned, in-progress, and implemented.";
74         `P "With --json, outputs structured JSON instead of formatted
        text."])
75     Term.(const run $ dir $ json $ quiet)
```

104 bin/cmd/Review.ml

```

1  (** borge review -- semantic code review
2
3      Uses LLM to verify function documentation matches implementation.
4      Now uses the new semantic review modules from lib/review/. *)
5
6  open Borge_lib
7
8  let now () =
9      let tm = Unix.localtime (Unix.time ()) in
10     Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
11         (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday
12         tm.Unix.tm_hour tm.Unix.tm_min tm.Unix.tm_sec
13
14  let call_pi prompt =
15      (* Use the new review_agent to call LLM *)
16      match Agent.run_pi_print prompt with
17      | Some response -> response
18      | None ->
19          "(function name:\\"unknown\\"\\n (doc-present false)\\n (doc-accuracy
20             low)\\n (signature-match unknown)\\n (behavior-coverage missing)\\n
21             (structural-issues \\"LLM unavailable\\"))\\n (confidence low))"
22
23  let review_file path ~dry_run =
24      if dry_run then Printf.printf "[DRY-RUN] Would review: %s\\n" path;
25
26      let content = File_utils.read_file path in
27      let functions = Semantic_review.extract_functions content in
28
29      if functions = [] then begin
30          Printf.printf "No functions found in %s\\n" path;
31          exit 0
32      end;
33
34      Printf.printf "Found %d functions in %s\\n\\n" (List.length functions) path;
35
36      let results = List.map (fun (info : Semantic_review.function_info) ->
37          if dry_run then begin
38              Printf.printf "Function: %s (line %d)\\n" info.name info.line;
39              (match info.docstring with
40              | None -> Printf.printf " No docstring\\n"
41              | Some _ -> Printf.printf " Has docstring\\n");
42              Printf.printf "\\n";
43              (info, Semantic_review.empty_result)
44          end else begin
45              let prompt = Semantic_review.build_prompt info in
46              Printf.printf "Function: %s (line %d) - calling LLM...\\n" info.name
47                  info.line;
48              let response = call_pi prompt in
49              let result = Semantic_review.parse_response response in
50              Printf.printf " Doc present: %b, Status: %s, Accuracy: %s,
51                  Consistency: %s\\n"
52                  result.Semantic_review.doc_present
53                  (match result.Semantic_review.doc_status with
54                  | Review_types.Doc_accurate -> "accurate"
55                  | Review_types.Doc_drifted -> "drifted"
56                  | Review_types.Doc_missing -> "missing")
57                  (match result.Semantic_review.doc_accuracy with
58                  | 'High -> "high" | 'Medium -> "medium" | 'Low -> "low" | 'Unknown
59                      -> "unknown")
60                  (match result.Semantic_review.consistency with
61                  | Review_types.Cons_consistent -> "consistent"
62                  | Review_types.Cons_questionable -> "questionable"
63                  | Review_types.Cons_inconsistent -> "inconsistent");
64              (info, result)
65          end
66      ) functions in
67
68      (* Write metadata if not dry-run *)
69      if not dry_run then begin
70          let meta_path = path ^ ".borg.meta" in
71          let metadata = List.concat_map (fun (info, result) ->

```

```

67     Semantic_review.render_metadata info result
68   ) results in
69   let oc = open_out meta_path in
70   output_string oc "(semantic-review\n";
71   output_string oc (Printf.sprintf " (timestamp \"%s\")\n" (now ()));
72   output_string oc (Printf.sprintf " (file \"%s\")\n" path);
73   List.iter (fun line -> output_string oc (line ^ "\n")) metadata;
74   output_string oc "\n";
75   close_out oc;
76   Printf.printf "\nMetadata written to %s\n" meta_path;
77   end
78
79 let review_dir dir ~dry_run ~recursive =
80   ignore recursive; (* TODO: implement recursive *)
81   Printf.printf "Reviewing directory: %s\n" dir;
82   let files = Array.to_list (Sys.readdir dir) in
83   let ml_files = List.filter (fun f -> Filename.check_suffix f ".ml") files
84     in
85   List.iter (fun f ->
86     review_file (Filename.concat dir f) ~dry_run
87   ) ml_files
88
89 let review_stale dir ~dry_run =
90   ignore dry_run;
91   Printf.printf "Reviewing stale files in: %s\n" dir;
92   let reviews = Semantic_review.review_stale dir in
93   if reviews = [] then
94     Printf.printf "No stale files found.\n"
95   else
96     List.iter (fun (review : Borge_lib.Review_types.semantic_review) ->
97       Printf.printf " Reviewed %s: %d functions\n"
98         review.source_file
99         (List.length review.functions)
100     ) reviews
101
102 (* agent note (|
103  * WHAT: Filter review findings by focus area. When set to "drift",
104  * only report doc-code alignment issues. When "consistency", only
105  * report internal consistency issues. When unset, report everything.
106  *
107  * WHY: Large codebases may generate many findings. Focus lets
108  * the user target one category at a time for manageable fixes.
109  * |) *)
110 let filter_by_focus findings focus =
111   match focus with
112   | None -> findings
113   | Some "drift" ->
114     List.filter (fun (f : Review_types.function_finding) ->
115       f.doc_status <> Review_types.Doc_accurate || f.doc_accuracy =
116         Review_types.Low
117     ) findings
118   | Some "consistency" ->
119     List.filter (fun (f : Review_types.function_finding) ->
120       f.consistency <> Review_types.Cons_consistent
121     ) findings
122   | _ -> findings
123
124 (* agent note (|
125  * WHAT: Insert (status drifted) markers into source files for
126  * functions where the LLM found doc-code drift without an
127  * existing drifted marker.
128  *
129  * WHY: Auto-marking makes drift explicit without requiring manual
130  * editing. It's a convenience feature for bulk review runs.
131  * The marker means "a machine checked this and found drift."
132  * |) *)
133 let mark_drifted_in_file path findings =
134   ignore (File_utils.read_file path);
135   (* For each finding with doc_status = Doc_drifted, find the
136   comment preceding the function and insert (status drifted)
137   TODO: implement actual file modification -- for now, report which
138   functions would be marked *)
139   List.iter (fun (f : Review_types.function_finding) ->

```

```

138     if f.doc_status = Review_types.Doc_drifted then begin
139         Printf.printf " Marking %s as drifted in %s\n" f.name path
140     end
141 ) findings
142
143 let run file dir all stale dry_run focus mark_drifted =
144     ignore focus;
145     ignore mark_drifted;
146     if stale then
147         review_stale (match dir with Some d -> d | None -> ".") ~dry_run
148     else
149         match file, dir, all with
150         | Some f, None, false -> review_file f ~dry_run
151         | None, Some d, false -> review_dir d ~dry_run ~recursive:false
152         | None, None, true -> review_dir "." ~dry_run ~recursive:true
153         | _ ->
154             Printf.eprintf "Error: specify --file, --dir, --all, or --stale\n";
155             exit 1
156
157 open Cmdliner
158
159 let file =
160     Arg.(value & opt (some string) None & info ["file"; "f"]
161         ~doc:"Review a specific file")
162
163 let dir =
164     Arg.(value & opt (some string) None & info ["dir"; "d"]
165         ~doc:"Review files in directory")
166
167 let all =
168     Arg.(value & flag & info ["all"; "a"] ~doc:"Review all .ml files in
169         project")
170
171 let stale =
172     Arg.(value & flag & info ["stale"; "s"]
173         ~doc:"Only review files with stale metadata")
174
175 let dry_run =
176     Arg.(value & flag & info ["dry-run"; "n"]
177         ~doc:"Show what would be reviewed without calling LLM")
178
179 let focus =
180     Arg.(value & opt (some string) None & info ["focus"]
181         ~docv:"FOCUS"
182         ~doc:"Filter findings: drift or consistency")
183
184 let mark_drifted =
185     Arg.(value & flag & info ["mark-drifted"]
186         ~doc:"Auto-insert (status drifted) markers on drifted comments")
187
188 let cmd : unit Cmd.t =
189     Cmd.v (Cmd.info "review" ~doc:"semantic code review (requires LLM)"
190         ~man:[ 'S "DESCRIPTION";
191             'P "Analyzes OCaml functions and verifies documentation with
192                 LLM.";
193             'P "Checks doc-code alignment (WHAT + WHY) and internal
194                 consistency.";
195             'P "Stores results in .borg.meta files for tracking.";
196             'P "Use --dry-run to preview without LLM calls.";
197             'P "Use --stale to only review changed files.";
198             'P "Use --focus drift to only show doc-code drift findings.";
199             'P "Use --focus consistency to only show internal consistency
200                 findings.";
201             'P "Use --mark-drifted to auto-insert (status drifted) markers.";
202             'S "EXAMPLES";
203             'P "borge review --file lib/core/spec.ml";
204             'P "borge review --dir lib/core --dry-run";
205             'P "borge review --all";
206             'P "borge review --stale";
207             'P "borge review --all --focus drift";
208             'P "borge review --all --mark-drifted"]])
209
210 Term.(const run $ file $ dir $ all $ stale $ dry_run $ focus $
211     mark_drifted)

```

105 bin/cmd/Stats.ml

```

1 open Borge_lib
2
3 (** Pad a string to the right with spaces to reach length n *)
4 let pad_right n s =
5   let len = String.length s in
6   if len >= n then s else s ^ String.make (n - len) ' '
7
8 (** Print a bar chart for a literacy dimension *)
9 let bar n =
10  let filled = int_of_float (n) in
11  String.make (min filled 7) '#' ^ String.make (max 0 (7 - filled)) ' '
12
13 (** Run literacy analysis and print results *)
14 let run_literacy dir =
15   Printf.printf "Literacy Metrics -- %s\n\n" dir;
16   let result = Stats.run dir in
17   Printf.printf "Per-file literacy scores:\n";
18   Printf.printf " %-30s story explain teach edge profile\n" "module";
19   Printf.printf " %s\n" (String.make 75 '-');
20   List.iter (fun (m : Stats.file_metrics) ->
21     match m.literacy with
22     | None ->
23       Printf.printf " %-30s no scores\n" m.module_name
24     | Some (s, e, t, g) ->
25       let profile =
26         if s >= 5.0 && e < 3.0 && t < 3.0 && g < 3.0 then
27           "narrative-strong"
28         else if s < 3.0 && e >= 5.0 && t < 3.0 && g < 3.0 then
29           "mechanics-strong"
30         else if s < 3.0 && e < 3.0 && t >= 5.0 && g < 3.0 then
31           "pedagogy-strong"
32         else if s < 3.0 && e < 3.0 && t < 3.0 && g >= 5.0 then
33           "edge-strong"
34         else "balanced"
35       in
36       Printf.printf " %-30s %.1f%s %.1f%s %.1f%s %.1f%s %s\n"
37         m.module_name
38         s (bar s) e (bar e) t (bar t) g (bar g) profile
39   ) result.files;
40   (match result.avg_literacy with
41   | None -> Printf.printf "\nNo literacy scores found in project.\n"
42   | Some (s, e, t, g) ->
43     Printf.printf "\nProject averages:\n";
44     Printf.printf " Story: %.1f %s\n" s (bar s);
45     Printf.printf " Explain: %.1f %s\n" e (bar e);
46     Printf.printf " Teach: %.1f %s\n" t (bar t);
47     Printf.printf " Edge: %.1f %s\n" g (bar g);
48     let insight =
49       if e < 2.0 && s >= 4.0 then
50         "Narrative is strong but mechanics are weak. Comments tell you
51         WHY but not HOW."
52       else if s < 2.0 && e >= 4.0 then
53         "Mechanics are strong but narrative is weak. Comments walk
54         through code but don't explain why it exists."
55       else if t < 1.0 && s >= 3.0 && e >= 3.0 then
56         "Solid documentation but limited teaching. Comments explain this
57         code but don't help with other code."
58       else if g < 1.0 then
59         "Boundary conditions missing. Comments describe happy path but
60         don't say what breaks."
61       else
62         "Balanced literacy across all four dimensions."
63     in
64     Printf.printf "\n %s\n" insight);
65   exit 0
66
67 (** Run documentation coverage analysis and print results *)
68 let run_coverage dir =
69   Printf.printf "Documentation Coverage -- %s\n\n" dir;
70   let coverage = Doc_coverage.calculate_dir_coverage dir in
71   if coverage.files = [] then begin

```

```

64   Printf.printf "No .ml files found.\n";
65   exit 0
66 end;
67 Printf.printf "Overall: %.1f%% (%d/%d documented, %d exempt, %d
   undocumented)\n\n"
68   coverage.overall_percent
69   coverage.total_documented
70   (coverage.total_bindings - coverage.total_exempt)
71   coverage.total_exempt
72   coverage.total_undocumented;
73 Printf.printf "Per-file coverage:\n";
74 List.iter (fun (fc : Doc_coverage.file_coverage) ->
75   Printf.printf " %-40s %6.1f%% (%d/%d doc, %d undoc)%s\n"
76     (Filename.basename fc.path)
77     fc.coverage_percent
78     fc.documented
79     (fc.total_bindings - fc.exempt)
80     fc.undocumented
81     (if fc.undocumented_names <> [] then
82       let rec take n = function
83         | [] -> []
84         | _ when n <= 0 -> []
85         | x :: xs -> x :: take (n - 1) xs
86       in
87         ": " ^ String.concat ", " (take 3 fc.undocumented_names)
88       else ""
89     ) coverage.files;
90   if coverage.total_undocumented > 0 then exit 1 else exit 0
91
92 let run_dir json quiet coverage literacy =
93   if literacy then run_literacy dir
94   else if coverage then run_coverage dir
95   else
96     let result = Stats.run_dir in
97     if json then begin
98       if not quiet then Printf.printf "%s\n" (Yojson.Basic.to_string
99         (Json_out.stats result));
100     end;
101     if quiet then exit 0;
102     Printf.printf "Borge Stats -- %d source file(s) in '%s'\n\n" (List.length
103       result.files) dir;
104     if result.files = [] then begin
105       Printf.printf "No .ml files found.\n";
106       exit 0
107     end;
108     let max_name = List.fold_left (fun acc (m : Stats.file_metrics) ->
109       max acc (String.length m.module_name)
110     ) 4 result.files in
111     Printf.printf "%s lines funcs avg exports docs verif mli\n"
112       (pad_right max_name "module");
113     Printf.printf "%s\n" (String.make (max_name + 57) '-');
114     List.iter (fun (m : Stats.file_metrics) ->
115       Printf.printf "%s %5d %6d %3.0f %7d %3.0f%% %5d %s\n"
116         (pad_right max_name m.module_name)
117         m.lines m.functions m.avg_length m.exports
118         (m.doc_coverage *. 100.0)
119         m.verified_count
120         (if m.has_mli then "yes" else "no")
121     ) result.files;
122     Printf.printf "%s\n" (String.make (max_name + 57) '-');
123     Printf.printf "%s %5d %6d %3.0f %7d %3.0f%% %5d\n"
124       (pad_right max_name "total")
125       result.total_lines result.total_functions result.avg_func_length
126       result.total_exports
127       (result.doc_coverage *. 100.0)
128       result.total_verified;
129     Printf.printf "\n%d files | %d lines | %d functions | %d exports | %d
   verified | doc %.0f%% | avg %.1f lines/function\n"
130       (List.length result.files) result.total_lines result.total_functions
131       result.total_exports result.total_verified (result.doc_coverage *.
132         100.0) result.avg_func_length;
133   exit 0

```



```
130
131 open Cmdliner
132
133 (* exempt doc *)
134 let dir =
135   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
136     ~doc:"Directory to scan for .ml files")
137
138 (* exempt doc *)
139 let json =
140   Arg.(value & flag & info ["json"] ~doc:"Output as JSON")
141
142 (* exempt doc *)
143 let quiet =
144   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
145     errors")
146
147 (* exempt doc *)
148 let coverage =
149   Arg.(value & flag & info ["coverage"; "c"] ~doc:"Show documentation
150     coverage analysis")
151
152 let literacy =
153   Arg.(value & flag & info ["literacy"; "l"] ~doc:"Show literacy scoring
154     metrics")
155
156 let cmd : unit Cmd.t =
157   Cmd.v (Cmd.info "stats" ~doc:"show code intelligence metrics"
158     ~man:[ 'S "DESCRIPTION";
159       'P "Scans all .ml files and reports lines of code, function
160         count, \
161           average function length, export count, and .mli coverage.";
162       'P "With --json, outputs structured JSON instead of formatted
163         text.";
164       'P "With --coverage, shows documentation coverage analysis.";
165       'P "With --literacy, shows per-file literacy scores: story,
166         explain, teach, edge." ])
167   Term.(const run $ dir $ json $ quiet $ coverage $ literacy)
```

106 bin/cmd/Undo.ml

```

1 (** borge undo: git revert HEAD -- convenience command *)
2
3 let run dir quiet =
4   if not quiet then begin
5     Printf.printf "This will revert the last commit (git revert HEAD).\n";
6     Printf.printf "The current state is preserved because revert creates a
       new commit.\n\n";
7   end;
8   (* Show what we're reverting *)
9   let cmd = Printf.sprintf "git -C %s log -1 --oneline" (Filename.quote
       dir) in
10  let ic = Unix.open_process_in cmd in
11  (try
12    (* exempt: input_line *) let line = input_line ic in
13    if not quiet then Printf.printf "Last commit: %s\n\n" line
14  with End_of_file -> ());
15  ignore (Unix.close_process_in ic);
16  let cmd = Printf.sprintf "git -C %s revert --no-edit HEAD 2>&1"
       (Filename.quote dir) in
17  let ret = Sys.command cmd in
18  if ret = 0 then begin
19    if not quiet then Printf.printf "Reverted successfully.\n";
20    exit 0
21  end else begin
22    Printf.eprintf "Revert failed. You may need to resolve conflicts
       manually.\n";
23    exit 1
24  end
25
26 open Cmdliner
27
28 let dir =
29   Arg.(value & pos 0 dir "." & info [] ~docv:"DIR"
30     ~doc:"Path to git repository (default: current directory)")
31
32 let quiet =
33   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
       errors")
34
35 let cmd : unit Cmd.t =
36   Cmd.v (Cmd.info "undo" ~doc:"revert the last commit (git revert HEAD)"
37     ~man:[ 'S "DESCRIPTION";
38       'P "Convenience command that runs git revert HEAD. This creates \
39         a new commit that undoes the last one -- the original commit \
40         is preserved in history, so nothing is lost." ])
41   Term.(const run $ dir $ quiet)

```

107 bin/cmd/Version.ml

```
1 (* exempt doc *)
2 let version = "0.1.0"
3
4 let run quiet =
5   if not quiet then Printf.printf "borge %s\n" version;
6   exit 0
7
8 open Cmdliner
9
10 (* exempt doc *)
11 let quiet =
12   Arg.(value & flag & info ["quiet"; "q"] ~doc:"Suppress all output except
13     errors")
14
15 (* exempt doc *)
16 let cmd : unit Cmd.t =
17   Cmd.v (Cmd.info "version" ~doc:"print the current version"
18     ~man:[ 'S "DESCRIPTION"; 'P "Prints the borge version number." ])
19   Term.(const run $ quiet)
```

108 lib/review/extract_docs.ml

```

1  (* Extract documentation comments from OCaml source files
2  *
3  * roerick note (|
4  *   Handles multiple doc comment formats:
5  *   1. Standard docstring: (** ... *) attribute on let binding
6  *   2. Borg-style: (* author note (|...|) *) on preceding line
7  *   3. Short borg: (*| ... |*) on preceding line
8  *   4. Exempt marker: (* exempt doc *) marks binding as exempt
9  * |) *)
10
11 (** Types of documentation comments found *)
12 type doc_type =
13   | Ocamldoc of string (** Standard (** ... *) docstring *)
14   | Borg_note of string * string (** (* author note (|...|) *) *)
15   | Borg_short of string (** (*| ... |*) *)
16   | Exempt (** (* exempt doc *) *)
17
18 (** Documentation found for a binding *)
19 type binding_doc = {
20   line : int;
21   doc_type : doc_type;
22   content : string;
23 }
24
25 (** Regex patterns for borg-style comments *)
26 let borg_note_pattern =
27   Str.regexp "(*\\s*\\([a-z]+\\)\\s+note\\s+(\\([\\^]\\)*\\)|)\\s*\\s*)"
28
29 (** Regex pattern for short borg comments: open-pipe-content-pipe-close *)
30 let borg_short_pattern =
31   Str.regexp "(*|\\s*\\([\\^]\\)*\\)|\\s*\\s*)"
32
33 (** Regex pattern for exemption markers in comments *)
34 let exempt_pattern =
35   Str.regexp "(*\\s*exempt\\s+doc\\s*\\s*)"
36
37 (** Extract comment body from a line *)
38 let extract_comment_content line =
39   let stripped = String.trim line in
40   if String.length stripped >= 4 &&
41     String.sub stripped 0 2 = "(" &&
42     String.sub stripped (String.length stripped - 2) 2 = ")" then
43     Some (String.sub stripped 2 (String.length stripped - 4) |> String.trim)
44   else
45     None
46
47 (** Check if a line is a standalone borg comment *)
48 let is_borg_comment line =
49   Str.string_match borg_note_pattern line 0 ||
50   Str.string_match borg_short_pattern line 0 ||
51   Str.string_match exempt_pattern line 0
52
53 (** Parse a borg-style comment *)
54 let parse_borg_comment line =
55   if Str.string_match borg_note_pattern line 0 then
56     let author = Str.matched_group 1 line in
57     let content = try Str.matched_group 2 line with Not_found -> "" in
58     Some (Borg_note (author, content))
59   else if Str.string_match borg_short_pattern line 0 then
60     let content = try Str.matched_group 1 line with Not_found -> "" in
61     Some (Borg_short content)
62   else if Str.string_match exempt_pattern line 0 then
63     Some Exempt
64   else
65     None
66
67 (** Read file and extract all doc comments with their line numbers *)
68 let extract_all_docs path =
69   try
70     (* exempt: open_in input_line *)
71     let channel = open_in path in

```

```

72   let docs = ref [] in
73   let line_num = ref 0 in
74   let prev_comment = ref None in
75
76   (try
77     while true do
78       let line = input_line channel in
79       incr line_num;
80       let trimmed = String.trim line in
81
82       (* Check for borg-style comment *)
83       (match parse_borg_comment trimmed with
84       | Some doc_type ->
85         let content = match doc_type with
86         | Borg_note (_, c) -> c
87         | Borg_short c -> c
88         | Exempt -> "exempt"
89         | Ocamlloc _ -> "" (* shouldn't happen *)
90         in
91         prev_comment := Some { line = !line_num; doc_type; content }
92       | None ->
93         (* Check for standard (* ... *) comment that might be borg *)
94         if String.length trimmed >= 4 &&
95           String.sub trimmed 0 2 = "(*" &&
96           String.sub trimmed (String.length trimmed - 2) 2 = "*)" then
97           let inner = String.sub trimmed 2 (String.length trimmed - 4)
98           |> String.trim in
99           if inner = "exempt doc" || inner = "exempt" then
100             prev_comment := Some { line = !line_num; doc_type =
101             Exempt; content = "exempt" }
102           else
103             prev_comment := None
104           else
105             prev_comment := None)
106         done
107       with End_of_file -> ());
108   close_in channel;
109   List.rev !docs
110   with e ->
111     Printf.eprintf "Error reading %s: %s\n" path (Printexc.to_string e);
112     []
113
114 (** Find doc comment preceding a line *)
115 let find_preceding_doc docs target_line =
116   (* Look for the closest doc that is on an earlier line *)
117   let rec find_closest = function
118   | [] -> None
119   | doc :: rest when doc.line < target_line ->
120     (* Check if there's another doc between this one and the target *)
121     (match find_closest rest with
122     | Some closer when closer.line < target_line -> Some closer
123     | _ -> Some doc)
124   | _ :: rest -> find_closest rest
125   in
126   find_closest (List.rev docs)
127
128 (** Check if a binding at a given line has documentation *)
129 let has_doc docs line =
130   match find_preceding_doc docs line with
131   | Some { doc_type = Exempt; _ } -> 'Exempt
132   | Some { doc_type = (Ocamlloc _ | Borg_note _ | Borg_short _); _ } ->
133     'Documented
134   | None -> 'Undocumented
135
136 (** Extract all docs from a string (for testing) *)
137 let extract_all_docs_from_string content =
138   let lines = String.split_on_char '\n' content in
139   let docs = ref [] in
140   let line_num = ref 0 in
141   List.iter (fun line ->
142     incr line_num;

```

```
142   let trimmed = String.trim line in
143
144   (* Check for borg-style comment *)
145   (match parse_borg_comment trimmed with
146   | Some doc_type ->
147       let content = match doc_type with
148       | Borg_note (_, c) -> c
149       | Borg_short c -> c
150       | Exempt -> "exempt"
151       | Ocamldoc _ -> "" (* shouldn't happen *)
152       in
153       docs := { line = !line_num; doc_type; content } :: !docs
154   | None ->
155       (* Check for standard (* ... *) comment that might be borg *)
156       if String.length trimmed >= 4 &&
157         String.sub trimmed 0 2 = "(" &&
158         String.sub trimmed (String.length trimmed - 2) 2 = ")" then
159         let inner = String.sub trimmed 2 (String.length trimmed - 4) |>
String.trim in
160         if inner = "exempt doc" || inner = "exempt" then
161             docs := { line = !line_num; doc_type = Exempt; content =
"exempt" } :: !docs)
162   ) lines;
163
164   List.rev !docs
```

109 lib/review/extract_functions.ml

```

1 (* Extract function information from OCaml source files
2  *
3  * roerick note (|
4  *   Uses regex-based extraction to find let bindings, their names,
5  *   signatures, and doc comments. This is a practical first-pass
6  *   implementation. Future versions may use compiler-libs for
7  *   more accurate AST-based extraction.
8  * |) *)
9
10 type function_info = {
11   name : string;
12   signature : string;
13   docstring : string option;
14   line : int;
15   is_recursive : bool;
16   is_test : bool; (* let%test binding *)
17   is_ignored : bool; (* let _ = ... *)
18 }
19
20 (** Extract function name from a let binding line *)
21 let extract_name_from_let line =
22   let trimmed = String.trim line in
23   let prefix =
24     if String.length trimmed > 8 && String.sub trimmed 0 8 = "let rec " then
25       Some "let rec "
26     else if String.length trimmed > 4 && String.sub trimmed 0 4 = "let "
27     then
28       Some "let "
29     else
30       None
31   in
32   match prefix with
33   | None -> None
34   | Some pfx ->
35     let rest = String.sub trimmed (String.length pfx) (String.length
36       trimmed - String.length pfx) in
37     let rest = String.trim rest in
38     (* Find first space or parenthesis to get function name *)
39     let name_end = try
40       let space_pos = String.index rest ' ' in
41       let paren_pos = try String.index rest '(' with Not_found ->
42         space_pos in
43       min space_pos paren_pos
44     with Not_found ->
45       try String.index rest '(' with Not_found -> String.length rest
46     in
47     let name = (* exempt: String.sub *) String.sub rest 0 name_end in
48     (* Check for parameters to build signature *)
49     let params = try
50       let param_start = name_end in
51       let param_end = try String.index rest '=' with Not_found ->
52         String.length rest in
53       if param_start < param_end then
54         (* exempt: String.sub *) String.sub rest param_start (param_end -
55           param_start) |> String.trim
56       else
57         ""
58     with _ -> "" in
59     Some (name, params)
60
61 (** Check if a line is a test binding *)
62 let is_test_line line =
63   let trimmed = String.trim line in
64   String.length trimmed > 4 && String.sub trimmed 0 4 = "let%"
65
66 (** Check if a line is an ignored binding *)
67 let is_ignored_line line =
68   let trimmed = String.trim line in
69   String.length trimmed > 6 && String.sub trimmed 0 6 = "let _ "
70
71 (** Check if a line contains an attribute *)

```

```

67 let has_attribute line =
68   String.contains line '[' && String.contains line ']'
69
70 (** Extract functions from a file *)
71 let extract_from_file path =
72   try
73     (* exempt: open_in input_line *)
74     let channel = open_in path in
75     let functions = ref [] in
76     let current_doc = ref [] in
77     let line_num = ref 0 in
78
79     (try
80       while true do
81         let line = input_line channel in
82         incr line_num;
83         let trimmed = String.trim line in
84
85         (* Check for doc comment *)
86         if String.length trimmed > 2 && String.sub trimmed 0 2 = "(" then
87           current_doc := line :: !current_doc
88         else if String.length trimmed > 4 &&
89           (String.sub trimmed 0 4 = "let " ||
90            (String.length trimmed > 8 && String.sub trimmed 0 8 =
91             "let rec ")) then
92           (* Check for let%test or let%lwt etc. *)
93           if String.length trimmed > 4 && trimmed.[3] = '%' then
94             current_doc := [] (* Reset doc for test bindings *)
95           else
96             match extract_name_from_let line with
97             | Some (name, params) ->
98               let is_rec = String.starts_with ~prefix:"let rec " trimmed
99               in
100               let is_ign = name = "_" || String.starts_with ~prefix:"_"
101               name in
102               let doc_opt = if !current_doc = [] then None else Some
103               (String.concat "\n" (List.rev !current_doc)) in
104               let sig_str = if params = "" then name else Printf.sprintf
105               "%s %s" name params in
106               functions := {
107                 name;
108                 signature = sig_str;
109                 docstring = doc_opt;
110                 line = !line_num;
111                 is_recursive = is_rec;
112                 is_test = false;
113                 is_ignored = is_ign;
114               } :: !functions;
115               current_doc := []
116             | None ->
117               current_doc := []
118           done
119         with End_of_file -> ());
120
121     close_in channel;
122     List.rev !functions
123   with e ->
124     Printf.eprintf "Error reading %s: %s\n" path (Printexc.to_string e);
125     []

```


110 lib/review/extract__parsetree.ml

```

1 (* Extract OCaml parsetree from .ml/.mli files
2  *
3  * roerick note (|
4  *   Placeholder for compiler-libs integration.
5  *   Currently provides stub types and functions that will be
6  *   replaced with actual compiler-libs Parsetree extraction.
7  *
8  *   The full implementation will use ocaml-compiler-libs to parse
9  *   .ml/.mli files into the Parsetree module for accurate AST
10  *   extraction.
11  * |) *)
12
13 (* Stub types that mirror Parsetree concepts *)
14 type structure = unit (* Placeholder *)
15 type signature = unit (* Placeholder *)
16 type structure_item = unit (* Placeholder *)
17 type location = {
18   pos_fname : string;
19   pos_lnum : int;
20   pos_cnum : int;
21   pos_bol : int;
22 }
23
24 (* Stub parse function *)
25 let parse_file path =
26   (* For now, just return empty structure - extraction happens via regexes
27    * *)
28   ignore path;
29   Ok ()
30
31 (* Stub interface parse *)
32 let parse_interface path =
33   ignore path;
34   Ok ()
35
36 (* Get location from structure item - stub *)
37 let location_of_structure_item _item =
38   { pos_fname = ""; pos_lnum = 0; pos_cnum = 0; pos_bol = 0 }
39
40 (* Check if item is value binding - stub *)
41 let is_value_binding _item =
42   false
43
44 (* Extract docstring - stub, actual extraction is in extract_docs.ml *)
45 let extract_docstring _attrs =
46   None
47
48 (* Check for exempt marker - stub *)
49 let has_exempt_marker _attrs =
50   false
51
52 (* Get line from location *)
53 let line_of_loc loc =
54   loc.pos_lnum
55
56 (* Get column from location *)
57 let col_of_loc loc =
58   loc.pos_cnum - loc.pos_bol

```

111 lib/review/review_agent.ml

```

1 (* LLM integration for semantic review
2  *
3  * roerick note (|
4  *   Handles calling the LLM via BORGE_AGENT_CMD for semantic review.
5  *   Uses the same Agent.run_pi_print mechanism as other borge
6  *   generation commands for consistency.
7  * |) *)
8
9 open Review_types
10
11 (** Run semantic review on a single function *)
12 let review_function (info : Extract_functions.function_info) :
13   function_finding option =
14   let prompt = Review_prompt.build_function_prompt info in
15   match Agent.run_pi_print prompt with
16   | None ->
17     Printf.eprintf "Warning: No LLM response for function %s\n" info.name;
18     None
19   | Some response ->
20     let timestamp =
21       let now = Unix.gmtime (Unix.time ()) in
22       Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
23         (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
24         now.tm_hour now.tm_min now.tm_sec
25     in
26     match Review_parse.parse_response_with_timestamp response timestamp
27     with
28     | Ok [finding] ->
29       Some finding
30     | Ok (finding :: rest) ->
31       (* Multiple findings, take the first or the one matching our
32        function name *)
33       if info.name = finding.name then
34         Some finding
35       else
36         List.find_opt (fun (f : function_finding) -> f.name =
37           info.name) (finding :: rest)
38     | Ok [] ->
39       (* No findings - assume no issues *)
40       Some {
41         name = info.name;
42         doc_present = info.docstring <> None;
43         doc_status = if info.docstring = None then Doc_missing else
44           Doc_drifted;
45         doc_accuracy = if info.docstring = None then Low else Medium;
46         consistency = Cons_consistent;
47         internal_issues = None;
48         signature_match = Unknown;
49         behavior_coverage = if info.docstring = None then Missing else
50           Partial;
51         structural_issues = None;
52         confidence = Low;
53         checked_at = timestamp;
54       }
55     | Error e ->
56       Printf.eprintf "Warning: Failed to parse LLM response for %s:
57       %s\n" info.name e;
58       None
59
60 (** Run semantic review on a batch of functions *)
61 let review_batch (functions : Extract_functions.function_info list) :
62   function_finding list =
63   match functions with
64   | [] -> []
65   | [f] ->
66     (match review_function f with
67     | Some r -> [r]
68     | None -> [])
69   | _ ->
70     (* Batch review *)
71     let prompt = Review_prompt.build_batch_prompt functions in

```

```

64   match Agent.run_pi_print prompt with
65   | None ->
66       Printf.eprintf "Warning: No LLM response for batch review\n";
67       []
68   | Some response ->
69       let timestamp =
70           let now = Unix.gmtime (Unix.time ()) in
71           Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
72               (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
73               now.tm_hour now.tm_min now.tm_sec
74       in
75       match Review_parse.parse_response_with_timestamp response timestamp
76       with
77       | Ok findings -> findings
78       | Error e ->
79           Printf.eprintf "Warning: Failed to parse batch response: %s\n"
80           e;
81           []
82
83   (** Run semantic review on a file *)
84   let review_file path =
85       Printf.printf "Reviewing %s...\n%! " path;
86
87       (* Extract functions from file *)
88       let functions = Extract_functions.extract_from_file path in
89
90       if functions = [] then begin
91           Printf.printf " No functions found in %s\n" path;
92           None
93       end else begin
94           Printf.printf " Found %d functions\n" (List.length functions);
95
96           (* Batch functions to respect token limits *)
97           let batches = Review_prompt.batch_functions functions ~max_tokens:6000
98           in
99           Printf.printf " Split into %d batches\n" (List.length batches);
100
101           (* Review each batch *)
102           let all_findings = List.concat_map review_batch batches in
103
104           let timestamp =
105               let now = Unix.gmtime (Unix.time ()) in
106               Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
107                   (now.tm_year + 1900) (now.tm_mon + 1) now.tm_mday
108                   now.tm_hour now.tm_min now.tm_sec
109           in
110
111           Some {
112               source_file = path;
113               reviewed_at = timestamp;
114               reviewer = "agent:semantic-reviewer";
115               functions = all_findings;
116           }
117       end
118
119   (** Run semantic review comparing spec to implementation *)
120   let review_spec_section ~spec_section ~module_surface ~impl_path =
121       let functions = Extract_functions.extract_from_file impl_path in
122
123       let prompt = Review_prompt.build_spec_comparison_prompt
124           ~spec_section
125           ~module_surface
126           ~functions
127       in
128
129       match Agent.run_pi_print prompt with
130       | None ->
131           Printf.eprintf "Warning: No LLM response for spec comparison\n";
132           None
133       | Some response ->
134           (* For spec comparison, we return the raw response for now
135              - would need additional parsing for drift findings *)
136           Some response

```

112 lib/review/review__parse.ml

```

1 (* Parse LLM responses into typed semantic findings
2  *
3  * roerick note (|
4  *   Parses sexp responses from semantic review LLM queries.
5  *   Never raises on parse failure -- always returns result type
6  *   to avoid corrupting .borg.meta files.
7  * |) *)
8
9 open Review_types
10
11 (** Parse result type *)
12 type 'a parse_result =
13   | Ok of 'a
14   | Error of string
15
16 (** Extract sexp from response (handles markdown fences) *)
17 let extract_sexp response =
18   let lines = String.split_on_char '\n' response in
19   let rec find_start = function
20     | [] -> None
21     | line :: rest ->
22       let trimmed = String.trim line in
23       if String.length trimmed >= 2 && String.sub trimmed 0 2 = "(" then
24         Some (line :: rest)
25       else if trimmed = "```" then
26         find_end rest
27       else
28         find_start rest
29   and find_end = function
30     | [] -> None
31     | line :: rest ->
32       if String.trim line = "```" then
33         find_start rest
34       else
35         Some (line :: "(" :: rest) (* Add opening paren if missing *)
36   in
37   match find_start lines with
38   | Some lines ->
39     let sexp_str = String.concat "\n" lines in
40     Some sexp_str
41   | None -> None
42
43 (** Parse a single sexp atom *)
44 let parse_atom s =
45   let s = String.trim s in
46   if String.length s >= 2 && s.[0] = '"' && s.[String.length s - 1] = '"'
47   then
48     String.sub s 1 (String.length s - 2)
49   else
50     s
51
52 (** Simple sexp parsing - extracts key-value pairs from function blocks *)
53 let rec parse_function_fields acc = function
54   | [] -> Ok (List.rev acc)
55   | field :: rest ->
56     let field = String.trim field in
57     if String.length field = 0 then
58       parse_function_fields acc rest
59     else if field.[0] = '(' then
60       (* Extract field name and value *)
61       let close_paren = try String.index field ')' with Not_found ->
62         String.length field - 1 in
63       let inner = (* exempt: String.sub *) String.sub field 1
64         (close_paren - 1) in
65       let parts = String.split_on_char ' ' (String.trim inner) in
66       (match parts with
67       | name :: values ->
68         let value = String.concat " " values in
69         parse_function_fields ((name, value) :: acc) rest
70       | [] -> parse_function_fields acc rest)
71     else

```

```

69     parse_function_fields acc rest
70
71 (** Parse function block from sexp string *)
72 let parse_function_block sexp_str =
73   (* Extract name from (function name:"..." ...) *)
74   let name_pattern = Str.regexp "name:\\\\"([^\"]*)\\\\" in
75   if Str.string_match name_pattern sexp_str 0 then
76     let name = Str.matched_group 1 sexp_str in
77
78     (* Extract other fields *)
79     let doc_present =
80       if Str.string_match (Str.regexp "(doc-present[ ]+true)" sexp_str 0
81         then true
82         else if Str.string_match (Str.regexp_string "(doc-present false)"
83           sexp_str 0 then false
84           else false
85     in
86
87     let doc_accuracy =
88       if Str.string_match (Str.regexp "(doc-accuracy[ ]+high)" sexp_str 0
89         then High
90         else if Str.string_match (Str.regexp "(doc-accuracy[ ]+medium)"
91           sexp_str 0 then Medium
92           else if Str.string_match (Str.regexp "(doc-accuracy[ ]+low)"
93             sexp_str 0 then Low
94             else Low
95     in
96
97     let signature_match =
98       if Str.string_match (Str.regexp "(signature-match[ ]+accurate)"
99         sexp_str 0 then Accurate
100       else if Str.string_match (Str.regexp "(signature-match[
101         ]+mismatch)" sexp_str 0 then Mismatch
102       else Unknown
103     in
104
105     let (behavior_coverage : behavior_coverage) =
106       if Str.string_match (Str.regexp "(behavior-coverage[ ]+complete)"
107         sexp_str 0 then Complete
108       else if Str.string_match (Str.regexp "(behavior-coverage[
109         ]+partial)" sexp_str 0 then Partial
110       else Missing
111     in
112
113     let structural_issues =
114       if Str.string_match (Str.regexp "(structural-issues[\\t ]+\\\""
115         sexp_str 0 then
116         let issues_start = Str.match_end () in
117         match String.index_from sexp_str issues_start '"' with
118         | exception Not_found -> None
119         | issues_end ->
120           let issues = (* exempt: String.sub *) String.sub sexp_str
121             issues_start (issues_end - issues_start) in
122           if issues = "none" || issues = "" then None else Some issues
123       else
124         None
125     in
126
127     let (conf : confidence) =
128       if Str.string_match (Str.regexp "(confidence[ ]+high)" sexp_str 0
129         then High
130       else if Str.string_match (Str.regexp "(confidence[ ]+medium)"
131         sexp_str 0 then Medium
132       else Low
133     in
134
135     let doc_status =
136       if Str.string_match (Str.regexp "(doc-status[ \\t]+accurate)" sexp_str
137         0 then Doc_accurate
138       else if Str.string_match (Str.regexp "(doc-status[ \\t]+drifted)"
139         sexp_str 0 then Doc_drifted
140       else if Str.string_match (Str.regexp "(doc-status[ \\t]+missing)"
141         sexp_str 0 then Doc_missing

```

```

126     else if doc_present then Doc_accurate else Doc_missing
127 in
128
129 let (consistency : consistency) =
130   if Str.string_match (Str.regexp "(consistency[ \t]+consistent)")
131     sexp_str 0 then Cons_consistent
132   else if Str.string_match (Str.regexp "(consistency[ \t]+questionable)") sexp_str 0 then Cons_questionable
133   else if Str.string_match (Str.regexp "(consistency[ \t]+inconsistent)") sexp_str 0 then Cons_inconsistent
134   else Cons_consistent
135 in
136
137 let internal_issues =
138   if Str.string_match (Str.regexp "(internal-issues[ \t ]+\\\"") sexp_str
139     0 then
140     let issues_start = Str.match_end () in
141     match String.index_from sexp_str issues_start ''' with
142     | exception Not_found -> None
143     | issues_end ->
144       let issues = (* exempt: String.sub *) String.sub sexp_str
145         issues_start (issues_end - issues_start) in
146       if issues = "none" || issues = "" then None else Some issues
147   else
148     None
149 in
150
151 Ok {
152   name;
153   doc_present;
154   doc_status;
155   doc_accuracy;
156   consistency;
157   internal_issues;
158   signature_match;
159   behavior_coverage;
160   structural_issues;
161   confidence = conf;
162   checked_at = ""; (* Will be filled in later *)
163 }
164 else
165   Error "No name found in function block"
166
167 (** Parse a (findings ...) block containing multiple (function ...) blocks
168   *)
169 let parse_findings response =
170   match extract_sexp response with
171   | None -> Error "No sexp found in response"
172   | Some sexp_str ->
173     (* Find all (function ...) blocks *)
174     let function_pattern = Str.regexp "(function[ \t ]+)" in
175     let rec find_functions acc pos =
176       try
177         let _ = (* exempt: Str.search_forward *) Str.search_forward
178           function_pattern sexp_str pos in
179         let start = Str.match_beginning () in
180         (* Find matching closing paren by counting *)
181         let depth = ref 1 in
182         let i = ref (start + 9) in (* Skip "(function" *)
183         while !depth > 0 && !i < String.length sexp_str do
184           match sexp_str.[i] with
185           | '(' -> incr depth; incr i
186           | ')' -> decr depth; incr i
187           | '"' -> (* Skip string *)
188             incr i;
189             while !i < String.length sexp_str && sexp_str.[i] <> '"' do
190               if sexp_str.[i] = '\\' then incr i;
191               incr i
192             done;
193             incr i
194         | _ -> incr i
195       done;

```

```
191     let func_str = (* exempt: String.sub *) String.sub sexp_str start
192     (!i - start) in
193     find_functions (func_str :: acc) !i
194     with Not_found -> List.rev acc
195   in
196   let function_blocks = find_functions [] 0 in
197   let findings = List.filter_map (fun block ->
198     match parse_function_block block with
199     | Ok finding -> Some finding
200     | Error _ -> None
201   ) function_blocks in
202
203   if findings = [] && not (String.contains sexp_str '(') then
204     Error "No findings could be parsed"
205   else
206     Ok findings
207
208   (** Parse response with timestamp *)
209   let parse_response_with_timestamp response timestamp =
210     match parse_findings response with
211     | Ok findings ->
212       Ok (List.map (fun f -> { f with checked_at = timestamp }) findings)
213     | Error e ->
214       Error e
215
216   (** Returns true if response is valid sexp findings *)
217   let is_valid_response response =
218     match parse_findings response with
219     | Ok [] -> true (* Empty findings is valid *)
220     | Ok _ -> true
221     | Error _ -> false
```

113 lib/review/review_prompt.ml

```

1 (* Build prompts for semantic review LLM queries
2  *
3  * roerick note (|
4  *   Assembles prompts for semantic review that constrain the LLM
5  *   to return structured sexp output. This is critical for parseability.
6  * |) *)
7
8 (** Build a prompt for reviewing a single function.
9   The prompt asks the LLM to evaluate three dimensions:
10   1. Doc-code alignment (WHAT and WHY)
11   2. Internal consistency (dead branches, unused params, contradictions)
12   3. Overall review quality *)
13 let build_function_prompt (info : Extract_functions.function_info) =
14   let doc_section = match info.docstring with
15   | Some d -> Printf.sprintf "Documentation:\n%s\n\n" d
16   | None -> "Documentation: [none]\n\n"
17   in
18   Printf.sprintf {|Review this OCaml function:
19
20 Function name: %s
21 Signature: %s
22 Line: %d
23 Recursive: %s
24
25 %s
26 Implementation:
27 %s
28
29 Evaluate the following and return ONLY a sexp response in this exact format:
30
31 (function name:"%s"
32  (doc-present %s)
33  (doc-status accurate|drifted|missing)
34  (doc-accuracy high|medium|low)
35  (consistency consistent|questionable|inconsistent)
36  (internal-issues "description or none")
37  (signature-match accurate|mismatch|unknown)
38  (behavior-coverage complete|partial|missing)
39  (structural-issues "description or none")
40  (confidence high|medium|low))
41
42 Evaluation criteria:
43
44 Doc-code alignment -- check both WHAT and WHY:
45 - doc-present: Does any documentation exist for this function?
46 - doc-status: Is the doc accurate (matches code), drifted (has
47   (status drifted) marker or clearly stale), or missing?
48 - doc-accuracy: Does the documentation answer:
49   1. WHAT does this function do? (inputs, outputs, side effects)
50   2. WHY does this function exist? (design reason, constraint)
51   A doc that only restates the signature scores medium at best.
52
53 Internal consistency -- check for code problems:
54 - consistency: Is the function internally consistent?
55   - consistent: no issues detected
56   - questionable: minor issues (unused parameter, redundant check)
57   - inconsistent: major issues (unreachable branch, parameter
58     always passed the same value, logic contradicts the doc)
59 - internal-issues: Describe any problems found, or "none"
60
61 Other checks:
62 - signature-match: Does the implementation match the signature?
63 - behavior-coverage: Does the doc cover all behaviors and edge cases?
64 - structural-issues: Code smells, complexity, unclear flow
65 - confidence: Your confidence in this assessment
66 |}
67   info.name
68   info.signature
69   info.line
70   (if info.is_recursive then "yes" else "no")
71   doc_section

```



```

72   "[implementation not shown - infer from signature]"
73   info.name
74   (if info.docstring <> None then "true" else "false")
75
76 (** Build a batched prompt for reviewing multiple functions
77
78   Token management: If prompt would exceed ~8k tokens, this function
79   should be called with smaller batches. *)
80 let build_batch_prompt (functions : Extract_functions.function_info list) =
81   let buf = Buffer.create 4096 in
82   Buffer.add_string buf "Review these OCaml functions:\n\n";
83
84   List.iter (fun (info : Extract_functions.function_info) ->
85     Buffer.add_string buf (Printf.sprintf "--- Function: %s ---\n"
86       info.name);
87     Buffer.add_string buf (Printf.sprintf "Signature: %s\n" info.signature);
88     Buffer.add_string buf (Printf.sprintf "Line: %d\n" info.line);
89     (match info.docstring with
90     | Some d -> Buffer.add_string buf (Printf.sprintf "Doc: %s\n" d)
91     | None -> Buffer.add_string buf "Doc: [none]\n");
92     Buffer.add_string buf "\n"
93   ) functions;
94
95   Buffer.add_string buf {|For each function, return a (function ...) sexp
96   block:
97
98   (function name:"FUNCTION_NAME"
99     (doc-present true|false)
100     (doc-status accurate|drifted|missing)
101     (doc-accuracy high|medium|low)
102     (consistency consistent|questionable|inconsistent)
103     (internal-issues "description or none")
104     (signature-match accurate|mismatch|unknown)
105     (behavior-coverage complete|partial|missing)
106     (structural-issues "description or none")
107     (confidence high|medium|low))
108
109   Return all findings wrapped in a (findings ...) block:
110   (findings
111     (function name:"func1" ...)
112     (function name:"func2" ...)
113     ...)
114
115   If no findings for a function, omit it from the response.
116   For doc-status: accurate = doc matches code, drifted = doc is
117   stale or has (status drifted) marker, missing = no doc.
118   For consistency: consistent = no issues, questionable = minor
119   (unused param), inconsistent = major (dead branch, logic error).
120   |});
121
122   Buffer.contents buf
123
124 (** Build prompt comparing spec section to implementation file *)
125 let build_spec_comparison_prompt ~spec_section ~module_surface ~functions =
126   let buf = Buffer.create 4096 in
127   Buffer.add_string buf "Compare this spec section to its
128   implementation:\n\n";
129   Buffer.add_string buf "--- SPEC SECTION ---\n";
130   Buffer.add_string buf spec_section;
131   Buffer.add_string buf "\n--- END SPEC ---\n\n";
132
133   Buffer.add_string buf "--- MODULE EXPORTS ---\n";
134   Buffer.add_string buf (Printf.sprintf "Module: %s\n"
135     module_surface.Surface.module_name);
136   Buffer.add_string buf (Printf.sprintf "Exports: %s\n" (String.concat ", "
137     module_surface.exports));
138   Buffer.add_string buf (Printf.sprintf "File: %s\n\n" module_surface.path);
139
140   Buffer.add_string buf "--- FUNCTIONS TO REVIEW ---\n";
141   List.iter (fun (info : Extract_functions.function_info) ->
142     Buffer.add_string buf (Printf.sprintf "- %s: %s\n" info.name
143       info.signature)
144   ) functions;

```

```

139   Buffer.add_string buf "\n";
140
141   Buffer.add_string buf {|Check for these drift indicators:
142 1. Spec says function exists but it's not in the module
143 2. Spec behavior doesn't match implementation
144 3. Implementation has functions not in spec (potential scope creep)
145 4. Documentation in spec is stale compared to code
146
147 Return findings as sexp:
148 (findings
149   (finding type:"phantom-spec|missing-implementation|doc-stale|scope-creep"
150     (function NAME)
151     (detail "description")
152     (confidence high|medium|low))
153   ...)
154
155 If no drift detected, return: (findings)
156 |});
157
158   Buffer.contents buf
159
160 (** Estimate token count (rough approximation) *)
161 let estimate_tokens text =
162   (* Rough estimate: ~4 chars per token for English text *)
163   String.length text / 4
164
165 (** Split functions into batches that fit within token limit *)
166 let batch_functions (functions : Extract_functions.function_info list)
167   ~max_tokens =
168   let rec batch acc current_batch current_tokens = function
169     | [] ->
170       if current_batch = [] then List.rev acc
171       else List.rev (List.rev current_batch :: acc)
172     | f :: rest ->
173       let prompt = build_function_prompt f in
174       let tokens = estimate_tokens prompt in
175       if current_tokens + tokens > max_tokens && current_batch <> [] then
176         batch (List.rev current_batch :: acc) [f] tokens rest
177       else
178         batch acc (f :: current_batch) (current_tokens + tokens) rest
179   in
180   batch [] [] 0 functions

```

114 lib/review/review_types.ml

```

1 (* Semantic review types for borge review command
2  *
3  * roerick note (|
4  *   Defines data types for semantic review findings and documentation
5  *   coverage.
6  *   These types map directly to the (semantic-review ...) and
7  *   (documentation ...)
8  *   blocks in .borg.meta files per meta.borg spec.
9  * |) *)
10
11 (** Confidence level for a finding *)
12 type confidence = High | Medium | Low
13
14 (** Documentation accuracy assessment *)
15 type accuracy = High | Medium | Low
16
17 (** Documentation status -- whether the doc matches the implementation *)
18 type doc_status = Doc_accurate | Doc_drifted | Doc_missing
19
20 (** Internal consistency assessment *)
21 type consistency = Cons_consistent | Cons_questionable | Cons_inconsistent
22
23 (** Signature match assessment *)
24 type signature_match = Accurate | Mismatch | Unknown
25
26 (** Behavior coverage assessment *)
27 type behavior_coverage = Complete | Partial | Missing
28
29 (** Per-function semantic finding *)
30 type function_finding = {
31   name : string;
32   doc_present : bool;
33   doc_status : doc_status;
34   doc_accuracy : accuracy;
35   consistency : consistency;
36   internal_issues : string option; (* None or "none" means no issues *)
37   signature_match : signature_match;
38   behavior_coverage : behavior_coverage;
39   structural_issues : string option; (* None or "none" means no issues *)
40   confidence : confidence;
41   checked_at : string; (* ISO 8601 timestamp *)
42 }
43
44 (** Semantic review block for a file *)
45 type semantic_review = {
46   source_file : string;
47   reviewed_at : string; (* ISO 8601 timestamp *)
48   reviewer : string; (* e.g., "agent:semantic-reviewer" *)
49   functions : function_finding list;
50 }
51
52 (** Documentation coverage for a file *)
53 type documentation = {
54   analyzed_at : string; (* ISO 8601 timestamp *)
55   file : string;
56   total_bindings : int;
57   documented : int;
58   exempt : int;
59   undocumented : int;
60   coverage_percent : float;
61   undocumented_names : string list;
62 }
63
64 (** Content hash for staleness detection *)
65 type content_hash = {
66   file_path : string;
67   hash : string; (* e.g., SHA256 or MD5 *)
68 }
69
70 (** Convert doc_status to string *)
71 let string_of_doc_status : doc_status -> string = function

```

```

70 | Doc_accurate -> "accurate"
71 | Doc_drifted -> "drifted"
72 | Doc_missing -> "missing"
73
74 (** Convert consistency to string *)
75 let string_of_consistency : consistency -> string = function
76 | Cons_consistent -> "consistent"
77 | Cons_questionable -> "questionable"
78 | Cons_inconsistent -> "inconsistent"
79
80 (** Convert confidence to string *)
81 let string_of_confidence : confidence -> string = function
82 | High -> "high"
83 | Medium -> "medium"
84 | Low -> "low"
85
86 (** Convert accuracy to string *)
87 let string_of_accuracy : accuracy -> string = function
88 | High -> "high"
89 | Medium -> "medium"
90 | Low -> "low"
91
92 (** Convert signature_match to string *)
93 let string_of_signature_match = function
94 | Accurate -> "accurate"
95 | Mismatch -> "mismatch"
96 | Unknown -> "unknown"
97
98 (** Convert behavior_coverage to string *)
99 let string_of_behavior_coverage = function
100 | Complete -> "complete"
101 | Partial -> "partial"
102 | Missing -> "missing"
103
104 (** Parse doc_status from string *)
105 let doc_status_of_string = function
106 | "accurate" -> Some Doc_accurate
107 | "drifted" -> Some Doc_drifted
108 | "missing" -> Some Doc_missing
109 | _ -> None
110
111 (** Parse consistency from string *)
112 let consistency_of_string = function
113 | "consistent" -> Some Cons_consistent
114 | "questionable" -> Some Cons_questionable
115 | "inconsistent" -> Some Cons_inconsistent
116 | _ -> None
117
118 (** Parse confidence from string *)
119 let confidence_of_string = function
120 | "high" -> Some High
121 | "medium" -> Some Medium
122 | "low" -> Some Low
123 | _ -> None
124
125 (** Parse accuracy from string *)
126 let accuracy_of_string = function
127 | "high" -> Some High
128 | "medium" -> Some Medium
129 | "low" -> Some Low
130 | _ -> None
131
132 (** Parse signature_match from string *)
133 let signature_match_of_string = function
134 | "accurate" -> Some Accurate
135 | "mismatch" -> Some Mismatch
136 | "unknown" -> Some Unknown
137 | _ -> None
138
139 (** Parse behavior_coverage from string *)
140 let behavior_coverage_of_string = function
141 | "complete" -> Some Complete
142 | "partial" -> Some Partial

```

```
143 | "missing" -> Some Missing
144 | _ -> None
```

115 lib/review/semantic_review.ml

```

1 (* Semantic code review - orchestration layer
2  *
3  * roerick note (|
4  *   Orchestrates the semantic review pipeline:
5  *   1. Extract functions from source files
6  *   2. Build prompts and call LLM
7  *   3. Parse responses into structured findings
8  *   4. Write findings to .borg.meta files
9  *
10 *   This module ties together extract_parsetree, extract_functions,
11 *   review_prompt, review_parse, and review_agent into a cohesive
12 *   workflow for borge review command.
13 * |) *)
14
15 open Review_types
16 open Borge_lang
17
18 (** Review a single file and return findings *)
19 let review_file = Review_agent.review_file
20
21 (** Review all .ml files in a directory *)
22 let review_dir dir =
23   ignore (File_utils.find_borg_files dir);
24   let ml_files =
25     Sys.readdir dir
26     |> Array.to_list
27     |> List.filter (fun f -> Filename.check_suffix f ".ml")
28     |> List.map (Filename.concat dir)
29   in
30
31   let findings = List.filter_map (fun path ->
32     if Sys.file_exists path && not (Sys.is_directory path) then
33       Review_agent.review_file path
34     else
35       None
36   ) ml_files in
37
38   findings
39
40 (** Review files that have changed since last review *)
41 let review_stale dir =
42   (* Find all .ml files *)
43   let ml_files =
44     let rec find_ml acc dir =
45       if Sys.file_exists dir && Sys.is_directory dir then
46         try
47           Sys.readdir dir
48           |> Array.to_list
49           |> List.fold_left (fun acc f ->
50             let path = Filename.concat dir f in
51             if Sys.is_directory path then
52               if f <> "_build" && f <> ".git" then
53                 find_ml acc path
54             else
55               acc
56             else if Filename.check_suffix f ".ml" then
57               path :: acc
58             else
59               acc
60           ) acc
61         with _ -> acc
62       else
63         acc
64     in
65     find_ml [] dir
66   in
67
68   (* Filter to files that have changed since last reviewed_at *)
69   (* For now, review all files - staleness detection needs .borg.meta
70   reader *)
71   let findings = List.filter_map Review_agent.review_file ml_files in

```

```

71   findings
72
73   (** Review files related to a spec section *)
74   let review_section ~borg_file ~section_name =
75     (* Find the section in the borg file *)
76     try
77       let input = File_utils.read_file borg_file in
78       let _file = Parse.parse_file input in
79
80       (* Look for matching section in exports or surface *)
81       (* This is simplified - would need proper section extraction *)
82       let section_text =
83         try
84           let start = (* exempt: Str.search_forward *) Str.search_forward
85             (Str.regexp ("(section " ^ section_name)) input 0 in
86           (* Find matching closing paren *)
87           let depth = ref 1 in
88           let i = ref (start + 9 + String.length section_name) in
89           while !depth > 0 && !i < String.length input do
90             match input.[!i] with
91             | '(' -> incr depth; incr i
92             | ')' -> decr depth; incr i
93             | '"' ->
94               incr i;
95               while !i < String.length input && input.[!i] <> '"' do
96                 if input.[!i] = '\\' then incr i;
97                 incr i
98             | _ -> incr i
99             done;
100             (* exempt: String.sub *) String.sub input start (!i - start)
101             with Not_found -> ""
102           in
103           if section_text = "" then
104             Error "Section not found"
105           else
106             (* Find implementation file - heuristic based on section name *)
107             let impl_filename =
108               Str.global_replace (Str.regexp "_") "-" section_name ^ ".ml"
109             in
110             let impl_path = Filename.concat "lib" impl_filename in
111
112             if Sys.file_exists impl_path then
113               match Review_agent.review_file impl_path with
114               | Some review -> Ok review
115               | None -> Error "Review failed"
116             else
117               Error (Printf.sprintf "Implementation file not found: %s" impl_path)
118             with e ->
119               Error (Printexc.to_string e)
120
121       (** Convert semantic review to metadata string *)
122       let review_to_metadata (review : semantic_review) : string =
123         let buf = Buffer.create 1024 in
124         Buffer.add_string buf "(semantic-review\n";
125         Buffer.add_string buf (Printf.sprintf " (source-file \"%s\")\n"
126           review.source_file);
127         Buffer.add_string buf (Printf.sprintf " (reviewed-at \"%s\")\n"
128           review.reviewed_at);
129         Buffer.add_string buf (Printf.sprintf " (reviewer \"%s\")\n"
130           review.reviewer);
131         Buffer.add_string buf " (functions\n";
132
133         List.iter (fun (f : function_finding) ->
134           Buffer.add_string buf (Printf.sprintf " (function name: \"%s\")\n"
135             f.name);
136           Buffer.add_string buf (Printf.sprintf " (doc-present %b)\n"
137             f.doc_present);

```

```

138   Buffer.add_string buf (Printf.sprintf "      (doc-status %s)\n"
139     (string_of_doc_status f.doc_status));
140   Buffer.add_string buf (Printf.sprintf "      (doc-accuracy %s)\n"
141     (string_of_accuracy f.doc_accuracy));
142   Buffer.add_string buf (Printf.sprintf "      (consistency %s)\n"
143     (string_of_consistency f.consistency));
144   (match f.internal_issues with
145   | Some i -> Buffer.add_string buf (Printf.sprintf "
146     (internal-issues \"%s\")\n" i)
147   | None -> ());
148   Buffer.add_string buf (Printf.sprintf "      (signature-match %s)\n"
149     (string_of_signature_match f.signature_match));
150   Buffer.add_string buf (Printf.sprintf "      (behavior-coverage %s)\n"
151     (string_of_behavior_coverage f.behavior_coverage));
152   (match f.structural_issues with
153   | Some i -> Buffer.add_string buf (Printf.sprintf "
154     (structural-issues \"%s\")\n" i)
155   | None -> ());
156   Buffer.add_string buf (Printf.sprintf "      (confidence %s)\n"
157     (string_of_confidence f.confidence));
158   Buffer.add_string buf (Printf.sprintf "      (checked-at \"%s\")\n"
159     f.checked_at);
160 ) review.functions;
161
162 Buffer.add_string buf "  )\n";
163 Buffer.add_string buf ")\n";
164 Buffer.contents buf
165
166 (** Legacy function extraction - kept for backward compatibility *)
167 let extract_functions_simple content =
168   (* Simple regex-based extraction for files that don't parse with
169     compiler-libs *)
170   let lines = String.split_on_char '\n' content in
171   let functions = ref [] in
172   let current_doc = ref [] in
173   let line_num = ref 0 in
174
175   List.iter (fun line ->
176     incr line_num;
177     let trimmed = String.trim line in
178
179     if String.length trimmed > 2 && String.sub trimmed 0 2 = "(" then
180       current_doc := line :: !current_doc
181     else if String.length trimmed > 7 && String.sub trimmed 0 7 = "let rec"
182     then
183       let name = try
184         let rest = String.sub trimmed 8 (String.length trimmed - 8) in
185         let space_pos = try String.index rest ' ' with Not_found ->
186           String.length rest in
187         (* exempt: String.sub *) String.sub rest 0 space_pos
188       with _ -> "unknown" in
189       functions := {
190         Extract_functions.name;
191         signature = "";
192         docstring = if !current_doc = [] then None else Some (String.concat
193           "\n" (List.rev !current_doc));
194         line = !line_num;
195         is_recursive = true;
196         is_test = false;
197         is_ignored = false;
198       } :: !functions;
199       current_doc := []
200     else if String.length trimmed > 3 && String.sub trimmed 0 3 = "let" then
201       let name = try
202         let rest = String.sub trimmed 4 (String.length trimmed - 4) in
203         let space_pos = try String.index rest ' ' with Not_found ->
204           String.length rest in
205         (* exempt: String.sub *) String.sub rest 0 space_pos
206       with _ -> "unknown" in
207       functions := {
208         Extract_functions.name;
209         signature = "";

```



```

196     docstring = if !current_doc = [] then None else Some (String.concat
197     "\n" (List.rev !current_doc));
198     line = !line_num;
199     is_recursive = false;
200     is_test = false;
201     is_ignored = false;
202     } :: !functions;
203     current_doc := []
204   ) lines;
205   List.rev !functions
206
207 (** Backwards compatibility for cmd/Review.ml *)
208
209 type function_info = {
210   name : string;
211   signature : string;
212   docstring : string option;
213   body : string;
214   line : int;
215 }
216
217 type review_result = {
218   doc_present : bool;
219   doc_status : Review_types.doc_status;
220   doc_accuracy : [ 'High | 'Medium | 'Low | 'Unknown ];
221   consistency : Review_types.consistency;
222   internal_issues : string option;
223   structural_issues : string option;
224   confidence : [ 'High | 'Medium | 'Low ];
225 }
226
227 let empty_result = {
228   doc_present = false;
229   doc_status = Review_types.Doc_missing;
230   doc_accuracy = 'Unknown;
231   consistency = Review_types.Cons_consistent;
232   internal_issues = None;
233   structural_issues = None;
234   confidence = 'Low;
235 }
236
237 let extract_functions content : function_info list =
238   let simple_list = extract_functions_simple content in
239   List.map (fun (info : Extract_functions.function_info) -> {
240     name = info.name;
241     signature = info.signature;
242     docstring = info.docstring;
243     body = ""; (* Not used by old code *)
244     line = info.line;
245   }) simple_list
246
247 let build_prompt (info : function_info) =
248   let doc_part = match info.docstring with
249   | None -> "No docstring found."
250   | Some d -> Printf.sprintf "Docstring:\n%s" d
251   in
252   Printf.sprintf {|Review this OCaml function:
253
254   Function name: %s
255   Body:
256   %s
257
258   %s
259
260   Evaluate:
261   1. Is there a docstring? (yes/no)
262   2. If yes, does it accurately describe what the function does?
263     (high/medium/low/unknown)
264   3. Are there structural issues (complexity, unclear flow)?
265     Describe briefly or say "none"
266   4. Your confidence in this assessment (high/medium/low)
267

```

```

268 Respond in this format:
269 Doc present: yes/no
270 Accuracy: high/medium/low/unknown
271 Issues: description or "none"
272 Confidence: high/medium/low
273 |}
274   info.name info.body doc_part
275
276 (** Legacy parse_response for backwards compatibility *)
277 let parse_response response : review_result =
278   let lines = String.split_on_char '\n' response in
279   let result = ref empty_result in
280
281   List.iter (fun line ->
282     let line = String.trim line in
283     if String.length line > 12 && String.sub line 0 12 = "Doc present:" then
284       result := { !result with doc_present =
285         let v = String.trim (String.sub line 12 (String.length line - 12))
286         in
287         v = "yes" }
288     else if String.length line > 11 && String.sub line 0 11 = "Doc status:"
289     then
290       let v = String.trim (String.sub line 11 (String.length line - 11)) in
291       let st = match v with
292       | "accurate" -> Review_types.Doc_accurate | "drifted" ->
293       Review_types.Doc_drifted | _ -> Review_types.Doc_missing
294       in
295       result := { !result with doc_status = st }
296     else if String.length line > 10 && String.sub line 0 10 = "Accuracy:"
297     then
298       let v = String.trim (String.sub line 10 (String.length line - 10)) in
299       let acc = match v with
300       | "high" -> 'High' | "medium" -> 'Medium' | "low" -> 'Low' | _ ->
301       'Unknown
302       in
303       result := { !result with doc_accuracy = acc }
304     else if String.length line > 12 && String.sub line 0 12 =
305     "Consistency:" then
306       let v = String.trim (String.sub line 12 (String.length line - 12)) in
307       let cons = match v with
308       | "consistent" -> Review_types.Cons_consistent | "questionable" ->
309       Review_types.Cons_questionable | _ -> Review_types.Cons_inconsistent
310       in
311       result := { !result with consistency = cons }
312     else if String.length line > 15 && String.sub line 0 15 = "Internal
313     issues:" then
314       let issues = String.trim (String.sub line 15 (String.length line -
315       15)) in
316       result := { !result with internal_issues = if issues = "none" then
317       None else Some issues }
318     else if String.length line > 7 && String.sub line 0 7 = "Issues:" then
319       let issues = String.trim (String.sub line 7 (String.length line - 7))
320       in
321       result := { !result with structural_issues = if issues = "none" then
322       None else Some issues }
323     else if String.length line > 11 && String.sub line 0 11 = "Confidence:"
324     then
325       let v = String.trim (String.sub line 11 (String.length line - 11)) in
326       let conf = match v with
327       | "high" -> 'High' | "medium" -> 'Medium' | _ -> 'Low
328       in
329       result := { !result with confidence = conf }
330   ) lines;
331
332   !result
333
334 (** Legacy render_metadata for backwards compatibility *)
335 let render_metadata (info : function_info) (result : review_result) =
336   let doc_acc = match result.doc_accuracy with
337   | 'High' -> "high" | 'Medium' -> "medium" | 'Low' -> "low" | 'Unknown' ->
338   "unknown"
339   in
340   let conf = match result.confidence with

```

```

327 | 'High -> "high" | 'Medium -> "medium" | 'Low -> "low"
328 in
329 let doc_st = match result.doc_status with
330 | Review_types.Doc_accurate -> "accurate" | Review_types.Doc_drifted ->
  "drifted" | Review_types.Doc_missing -> "missing"
331 in
332 let cons = match result.consistency with
333 | Review_types.Cons_consistent -> "consistent" |
  Review_types.Cons_questionable -> "questionable" |
  Review_types.Cons_inconsistent -> "inconsistent"
334 in
335 let lines = [
336   Printf.sprintf " (function name: \"%s\" line: %d" info.name info.line;
337   Printf.sprintf " (doc-present %b)" result.doc_present;
338   Printf.sprintf " (doc-status %s)" doc_st;
339   Printf.sprintf " (doc-accuracy %s)" doc_acc;
340   Printf.sprintf " (consistency %s)" cons;
341   Printf.sprintf " (confidence %s)" conf;
342 ] in
343 let lines = match result.internal_issues with
344 | Some i -> lines @ [Printf.sprintf " (internal-issues \"%s\")" i]
345 | None -> lines
346 in
347 let lines = match result.structural_issues with
348 | Some i -> lines @ [Printf.sprintf " (structural-issues \"%s\")" i]
349 | None -> lines
350 in
351 lines @ [" )"]

```

116 bin/borge.ml

```

1  (** borge -- a spec-driven code orchestrator *)
2
3  open Borge_cmd
4  open Cmdliner
5
6  let cmds = [
7    Abort.cmd;
8    Balance.cmd;
9    Book.cmd;
10   Issue.cmd;
11   Check.cmd;
12   Commit.cmd;
13   Diff.cmd;
14   Drift.cmd;
15   Execute.cmd;
16   Fmt.cmd;
17   Generate.cmd;
18   Inline.cmd;
19   Lint.cmd;
20   Log.cmd;
21   Make.cmd;
22   (* Merge_queue.cmd; Merge.cmd; Future.cmd; *) (* Phase 3.12/4.4: CLI
      integration pending build system fix *)
23   Modules.cmd;
24   Nodes.cmd;
25   Normalize.cmd;
26   Parse.cmd;
27   Plan.cmd;
28   Print.cmd;
29   Report.cmd;
30   Review.cmd;
31   Stats.cmd;
32   Undo.cmd;
33   Version.cmd;
34 ]
35
36 let () =
37   let doc = "a spec-driven code orchestrator" in
38   let man = [
39     'S "DESCRIPTION";
40     'P "Borge reads .borg files (sexp-based specs) and provides tools to \
41       validate, format, check, and orchestrate your project's
42       specification.";
43     'P "Every .borg file is a git-tracked source of truth. Borge
44       understands \
45       the inline tree, comment integrity, and drift between spec and
46       code.";
47     'S "COMMANDS";
48     'P "Run borge <command> --help for details on any command.";
49     'I ("balance", "Check structural balance (run this FIRST after edits)");
50     'I ("parse", "Full syntax validation");
51     'I ("check", "Recursive health check (add --worktree for CI)");
52     'I ("report", "Status summary dashboard");
53     'I ("fmt", "Auto-format to canonical indentation");
54     'I ("nodes", "AST dump with positions");
55     'I ("book", "print the codebase as a paginated PDF book");
56     'I ("inline", "Show the inline project tree");
57     'I ("lint", "Semantic validation");
58     'I ("drift", "Detect spec/code/structural drift");
59     'I ("normalize", "Fmt all + accept comment deletions");
60     'I ("execute", "Execute (implement) a planned section");
61     'I ("generate", "Generate spec sections or code from specs");
62     'I ("stats", "Code intelligence metrics (lines, functions, exports)");
63     'I ("review", "Advisory review (lint + drift)");
64     'I ("log", "Git log for .borg changes");
65     'I ("diff", "Git diff with status annotations");
66     'I ("undo", "Revert last commit");
67     'S "WORKFLOW";
68     'P "1. Edit .borg files in your editor";
69     'P "2. Run 'borg balance' to catch structural errors";
70     'P "3. Run 'borg fmt' to canonicalize indentation";

```

```
68     'P "4. Run 'borge check' to validate the whole project";
69     'P "5. Run 'borge report' to see what's done and what's next";
70     'S "EXIT CODES";
71     'P "0 success (no issues found)";
72     'P "1 issues found (check failed, lint errors, etc.)";
73     'P "2 usage error (wrong arguments)";
74   ] in
75   let info = Cmd.info "borge" ~version:"0.1.0" ~doc ~man in
76   let cmd = Cmd.group info cmds in
77   exit (Cmd.eval cmd)
```

117 bin/borge_balance.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Balance.cmd |> Cmd.eval |> exit
```

118 bin/borge__check.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Check.cmd |> Cmd.eval |> exit
```

119 bin/borge_diff.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Diff.cmd |> Cmd.eval |> exit
```


120 bin/borge_drift.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Drift.cmd |> Cmd.eval |> exit
```

121 bin/borge_execute.ml

```
1 let () = Borge_cmd.Execute.cmd |> Cmd.eval |> exit
```

122 bin/borge_fmt.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Fmt.cmd |> Cmd.eval |> exit
```

123 bin/borge_generate.ml

```
1 let () = Borge_cmd.Generate.cmd |> Cmd.eval |> exit
```

124 bin/borge__inline.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Inline.cmd |> Cmd.eval |> exit
```

125 bin/borge__lint.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Lint.cmd |> Cmd.eval |> exit
```

126 bin/borge__log.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Log.cmd |> Cmd.eval |> exit
```

127 bin/borge__nodes.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Nodes.cmd |> Cmd.eval |> exit
```


128 bin/borge_normalize.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Normalize.cmd |> Cmd.eval |> exit
```

129 bin/borge_parse.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Parse.cmd |> Cmd.eval |> exit
```

130 bin/borge__print.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Print.cmd |> Cmd.eval |> exit
```

131 bin/borge_report.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Report.cmd |> Cmd.eval |> exit
```

132 bin/borge_review.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Review.cmd |> Cmd.eval |> exit
```

133 bin/borge_stats.ml

```
1 let () = Borge_cmd.Stats.cmd |> Cmd.eval |> exit
```

134 bin/borge_undo.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Undo.cmd |> Cmd.eval |> exit
```

135 bin/borge_version.ml

```
1 open Cmdliner
2 let () = Borge_cmd.Version.cmd |> Cmd.eval |> exit
```


136 test/test_balance.ml

```

1 open Borge_lang.Balance
2
3 let assert_balanced s =
4   match check s with
5   | Balanced _ -> ()
6   | Imbalanced errs ->
7     Alcotest.fail (Printf.sprintf "Expected balanced, got errors: %s"
8       (String.concat "; " (List.map string_of_error errs)))
9
10 let assert_imbalanced s =
11   match check s with
12   | Balanced _ -> Alcotest.fail "Expected imbalanced, got balanced"
13   | Imbalanced _ -> ()
14
15 let assert_analyze_pairs_expected s expected_pairs =
16   let analysis = analyze s in
17   let actual =
18     List.map (fun (p : Borge_lang.Balance.paren_pair) ->
19       (p.open_pos.line, p.close_pos |> Option.map (fun (c :
20         Borge_lang.Balance.pos) -> c.line), p.keyword)
21     ) analysis.pairs
22   in
23   if actual <> expected_pairs then
24     Alcotest.fail (Printf.sprintf "Expected pairs %s, got %s"
25       (String.concat "; " (List.map (fun (l, c, k) ->
26         Printf.sprintf "(%d,%s,%s)" l (match c with Some x ->
27         string_of_int x | None -> "None") (match k with Some x -> x | None ->
28         "-")
29       ) expected_pairs))
30     (String.concat "; " (List.map (fun (l, c, k) ->
31       Printf.sprintf "(%d,%s,%s)" l (match c with Some x ->
32       string_of_int x | None -> "None") (match k with Some x -> x | None ->
33       "-")
34     ) actual)))
35
36 let assert_divergences s expected_count =
37   let analysis = analyze s in
38   let divs = find_divergences analysis.pairs in
39   if List.length divs <> expected_count then
40     Alcotest.fail (Printf.sprintf "Expected %d divergences, got %d"
41       expected_count (List.length divs))
42
43 let test_simple_balanced () =
44   assert_balanced "(project test)"
45
46 let test_unclosed () =
47   assert_imbalanced "(project test"
48
49 let test_extra_close () =
50   assert_imbalanced "project test)"
51
52 let test_nested_balanced () =
53   assert_balanced "(project test (section a (subsection b)))"
54
55 let test_verbatim_ignores_parens () =
56   assert_balanced "(doc (|hello (world)|))"
57
58 let test_comment_ignores_parens () =
59   assert_balanced "; this has ) parens\n(project test)"
60
61 let test_annotated_comment () =
62   assert_balanced "(* agent note (|ok|) *)\n(project test)"
63
64 let test_analyze_records_pairs () =
65   assert_analyze_pairs_expected
66     "(project test (section a))"
67     [ (1, Some 1, Some "project"); (1, Some 1, Some "section") ]
68
69 let test_analyze_unclosed_pairs () =
70   let analysis = analyze "(project test (section a" in
71   match analysis.pairs with

```

```

66 | [p1; p2] ->
67   if p1.close_pos <> None then Alcotest.fail "project should be
   unclosed";
68   if p2.close_pos <> None then Alcotest.fail "section should be
   unclosed";
69   if p1.keyword <> Some "project" then Alcotest.fail "wrong keyword for
   project";
70   if p2.keyword <> Some "section" then Alcotest.fail "wrong keyword for
   section";
71 | _ -> Alcotest.fail (Printf.sprintf "Expected 2 pairs, got %d"
   (List.length analysis.pairs))
72
73 let test_divergence_detected () =
74   (* section gamma at col 2 but depth 3 ? under-indented *)
75   assert_divergences
76     "(project test\n (section alpha\n (doc \"hello\")\n (section beta\n
   (doc \"world\")))"
77     2
78
79 let test_indent_mismatch_detected () =
80   (* section closed at col 3 but opened at col 2 *)
81   let input = "(project test\n (section alpha\n (subsection beta\n (doc
   \"hello\")\n )\n )\n (subsection gamma\n (doc \"world\")\n )\n)\n"
   in
82   match check input with
83   | Balanced _ -> Alcotest.fail "Expected indent_mismatch, got balanced"
84   | Imbalanced errs ->
85     let has_mismatch = List.exists (function
86       | Borge_lang.Balance.Indent_mismatch _ -> true
87       | _ -> false
88     ) errs in
89     if not has_mismatch then
90       Alcotest.fail (Printf.sprintf "Expected indent_mismatch among
   errors: %s"
91         (String.concat "; " (List.map Borge_lang.Balance.string_of_error
   errs)))
92
93 let assert_repair_balances broken =
94   let repaired = repair broken in
95   match check repaired with
96   | Balanced _ -> ()
97   | Imbalanced errs ->
98     Alcotest.fail (Printf.sprintf "Repair produced imbalanced output: %s"
99       (String.concat "; " (List.map string_of_error errs)))
100
101 let test_repair_missing_closes () =
102   let broken = "(project test\n (doc \"hello\")\n (section alpha\n
   (subsection beta\n (doc \"world\")\n (status implemented)\n )\n)\n"
   in
103   assert_repair_balances broken;
104   let repaired = repair broken in
105   (* The repaired file should have ) on the right lines to close forms *)
106   if not (String.contains repaired ')') then
107     Alcotest.fail "Repaired file has no close parens"
108
109 let test_repair_extra_closes () =
110   let broken = "(project test\n (section alpha\n (doc \"hello\")\n )\n
   )\n)\n" in
111   assert_repair_balances broken
112
113 let test_repair_verbatim_preserved () =
114   let broken = "(doc (|hello (world)|)\n(project test\n" in
115   let repaired = repair broken in
116   (* Debug: *)
117   (* Printf.printf "REPAIRED: %S\n" repaired; *)
118   assert_repair_balances broken;
119   (* Verbatim string should still contain (world) *)
120   if not ((String.contains repaired '(') && (String.contains repaired 'w'))
   then
121     Alcotest.fail "Verbatim content was corrupted by repair"
122
123 let test_enriched_has_tree () =
124   let ea = analyze_enriched "(project test (section a))" in

```

```

125   if List.length ea.enriched_tree <> 2 then
126       Alcotest.fail (Printf.sprintf "Expected 2 tree nodes, got %d"
127           (List.length ea.enriched_tree));
128   match ea.enriched_repair_actions with
129   | [] -> ()
130   | _ -> Alcotest.fail "Balanced file should have no repair actions"
131
132 let test_enriched_has_repair_actions_on_broken () =
133   let broken = "(project test\n (section alpha\n (doc \"hello\")\n
134       (section beta\n (doc \"world\")))" in
135   let ea = analyze_enriched broken in
136   if ea.enriched_repair_actions = [] then
137       Alcotest.fail "Broken file should have repair actions"
138
139 let () =
140   Alcotest.run "Balance checker" [
141       "balance", [
142           Alcotest.test_case "simple balanced" 'Quick test_simple_balanced;
143           Alcotest.test_case "unclosed" 'Quick test_unclosed;
144           Alcotest.test_case "extra close" 'Quick test_extra_close;
145           Alcotest.test_case "nested balanced" 'Quick test_nested_balanced;
146           Alcotest.test_case "verbatim ignores parens" 'Quick
147               test_verbatim_ignores_parens;
148           Alcotest.test_case "plain comment ignores parens" 'Quick
149               test_comment_ignores_parens;
150           Alcotest.test_case "annotated comment ignored" 'Quick
151               test_annotated_comment;
152           Alcotest.test_case "analyze records pairs" 'Quick
153               test_analyze_records_pairs;
154           Alcotest.test_case "analyze unclosed pairs" 'Quick
155               test_analyze_unclosed_pairs;
156           Alcotest.test_case "divergence detected" 'Quick
157               test_divergence_detected;
158           Alcotest.test_case "indent mismatch detected" 'Quick
159               test_indent_mismatch_detected;
160           Alcotest.test_case "repair missing closes" 'Quick
161               test_repair_missing_closes;
162           Alcotest.test_case "repair extra closes" 'Quick
163               test_repair_extra_closes;
164           Alcotest.test_case "repair verbatim preserved" 'Quick
165               test_repair_verbatim_preserved;
166           Alcotest.test_case "enriched has tree" 'Quick test_enriched_has_tree;
167           Alcotest.test_case "enriched has repair actions" 'Quick
168               test_enriched_has_repair_actions_on_broken;
169       ]
170   ]

```

137 test/test_db_sql.ml

```

1  (* Test db_sql generator *)
2  open Borge_lib
3
4  (* Helper: check if string contains substring *)
5  let contains haystack needle =
6    (* exempt: Str.search_forward *)
7    try let _ = Str.search_forward (Str.regexp_string needle) haystack 0 in
8      true
9    with Not_found -> false
10
11  (* Helper: find position of substring *)
12  let find_pos haystack needle =
13    (* exempt: Str.search_forward *)
14    try Str.search_forward (Str.regexp_string needle) haystack 0
15    with Not_found -> -1
16
17  let test_simple_table () =
18    let open Db_ast in
19    let app = {
20      tables = [
21        { name = "users";
22          columns = [
23            { name = "id"; typ = "uuid"; constraints = [Primary_key; Default
24              "gen_random_uuid"] };
25            { name = "email"; typ = "text"; constraints = [Unique; Not_null]
26            };
27            { name = "created_at"; typ = "timestampz"; constraints =
28              [Default "now"] };
29          ];
30          indexes = [{ name = "email_idx"; columns = ["email"]; idx_method =
31            None; unique = false }];
32          ownership = None;
33        };
34      ];
35      operations = [];
36      relations = [];
37      groups = [];
38    } in
39    let sql = Db_sql.generate ~source_path:"test.borg" app in
40    Alcotest.(check bool) "contains CREATE TABLE" true (contains sql "CREATE
41      TABLE IF NOT EXISTS users");
42    Alcotest.(check bool) "contains uuid type" true (contains sql "UUID");
43    Alcotest.(check bool) "contains PRIMARY KEY" true (contains sql "PRIMARY
44      KEY");
45    Alcotest.(check bool) "contains NOT NULL" true (contains sql "NOT NULL");
46    Alcotest.(check bool) "contains gen_random_uuid()" true (contains sql
47      "gen_random_uuid()");
48    Alcotest.(check bool) "contains UNIQUE" true (contains sql "UNIQUE");
49    Alcotest.(check bool) "contains CREATE INDEX" true (contains sql "CREATE
50      INDEX IF NOT EXISTS email_idx");
51    Alcotest.(check bool) "contains schema header" true (contains sql
52      "Generated by borge generate")
53
54  let test_foreign_key_ordering () =
55    let open Db_ast in
56    let app = {
57      tables = [
58        (* posts references users -- must come after users *)
59        { name = "posts";
60          columns = [
61            { name = "id"; typ = "uuid"; constraints = [Primary_key] };
62            { name = "author_id"; typ = "uuid"; constraints = [References
63              "users.id"] };
64          ];
65          indexes = [];
66          ownership = None;
67        };
68        { name = "users";
69          columns = [
70            { name = "id"; typ = "uuid"; constraints = [Primary_key] };
71          ];
72        };
73      ];
74      operations = [];
75      relations = [];
76      groups = [];
77    } in
78    let sql = Db_sql.generate ~source_path:"test.borg" app in
79    Alcotest.(check bool) "contains foreign key" true (contains sql "REFERENCES
80      users.id");
81    Alcotest.(check bool) "contains schema header" true (contains sql
82      "Generated by borge generate")

```

```

61     indexes = [];
62     ownership = None;
63   };
64 ];
65 operations = [];
66 relations = [];
67 groups = [];
68 } in
69 let sql = Db_sql.generate ~source_path:"test.borg" app in
70 let users_pos = find_pos sql "CREATE TABLE IF NOT EXISTS users" in
71 let posts_pos = find_pos sql "CREATE TABLE IF NOT EXISTS posts" in
72 Alcotest.(check bool) "users before posts" true (users_pos < posts_pos)
73
74 let test_ownership_qls () =
75   let open Db_ast in
76   let app = {
77     tables = [
78       { name = "posts";
79         columns = [
80           { name = "id"; typ = "uuid"; constraints = [Primary_key] };
81           { name = "author_id"; typ = "uuid"; constraints = [Not_null] };
82         ];
83         indexes = [];
84         ownership = Some "author_id";
85       };
86     ];
87     operations = [];
88     relations = [];
89     groups = [];
90   } in
91   let sql = Db_sql.generate ~source_path:"test.borg" app in
92   Alcotest.(check bool) "contains ENABLE ROW LEVEL SECURITY" true (contains
93     sql "ENABLE ROW LEVEL SECURITY");
94   Alcotest.(check bool) "contains SELECT policy" true (contains sql "FOR
95     SELECT USING");
96   Alcotest.(check bool) "contains INSERT policy" true (contains sql "FOR
97     INSERT WITH CHECK");
98   Alcotest.(check bool) "contains UPDATE policy" true (contains sql "FOR
99     UPDATE USING");
100   Alcotest.(check bool) "contains DELETE policy" true (contains sql "FOR
101     DELETE USING");
102   Alcotest.(check bool) "contains current_user_id()" true (contains sql
103     "current_user_id()")
104
105 let test_group_policies () =
106   let open Db_ast in
107   let app = {
108     tables = [
109       { name = "posts";
110         columns = [
111           { name = "id"; typ = "uuid"; constraints = [Primary_key] };
112           { name = "author_id"; typ = "uuid"; constraints = [Not_null] };
113           { name = "published"; typ = "boolean"; constraints = [] };
114         ];
115         indexes = [];
116         ownership = None;
117       };
118     ];
119     operations = [];
120     relations = [];
121     groups = [
122       { name = "admin"; can_all = true; capabilities = [] };
123       { name = "reader";
124         can_all = false;
125         capabilities = [
126           { table = "posts"; operations = ["read"]; where_clause = Some
127             "published = true" };
128         ];
129       };
130     ];
131   } in
132   let sql = Db_sql.generate ~source_path:"test.borg" app in

```

```

126   Alcotest.(check bool) "contains admin can-all policy" true (contains sql
127     "admin_all");
128   Alcotest.(check bool) "admin USING true" true (contains sql "USING (true)
129     WITH CHECK (true)");
130   Alcotest.(check bool) "reader SELECT policy" true (contains sql
131     "reader_posts_select");
132   Alcotest.(check bool) "reader where published = true" true (contains sql
133     "published = true")
134
135 let test_operations_as_comments () =
136   let open Db_ast in
137   let app = {
138     tables = [
139       { name = "users";
140         columns = [{ name = "id"; typ = "uuid"; constraints = [Primary_key]
141       }];
142       indexes = [];
143       ownership = None;
144     };
145   ];
146   operations = [
147     { table_name = "users"; crud = Some Crud_all; queries = [{ name =
148       "by_email"; limit = None } ] };
149   ];
150   relations = [];
151   groups = [];
152 } in
153 let sql = Db_sql.generate ~source_path:"test.borg" app in
154 Alcotest.(check bool) "contains operations comment" true (contains sql
155   "-- Operations on users");
156 Alcotest.(check bool) "contains query comment" true (contains sql "--
157   Query: by_email")
158
159 let test_relations_as_comments () =
160   let open Db_ast in
161   let app = {
162     tables = [
163       { name = "users";
164         columns = [{ name = "id"; typ = "uuid"; constraints = [Primary_key]
165       }];
166       indexes = [];
167       ownership = None;
168     };
169     { name = "posts";
170       columns = [
171         { name = "id"; typ = "uuid"; constraints = [Primary_key] };
172         { name = "author_id"; typ = "uuid"; constraints = [References
173           "users.id" ] };
174       ];
175       indexes = [];
176       ownership = None;
177     };
178   ];
179   operations = [];
180   relations = [
181     { name = "user_posts";
182       from_table = "users";
183       joins = [{ join_type = Inner_join; table = "posts"; on_condition =
184         "posts.author_id = users.id" }];
185       where_clause = None;
186       select_columns = ["users.name"; "posts.title"];
187       queries = [];
188     };
189   ];
190   groups = [];
191 } in
192 let sql = Db_sql.generate ~source_path:"test.borg" app in
193 Alcotest.(check bool) "contains relation comment" true (contains sql "--
194   Relation: user_posts")
195
196 ;;
197
198 let () =
199   Alcotest.run "db_sql" [

```

```
187 "db_sql", [  
188   Alcotest.test_case "simple table" 'Quick test_simple_table;  
189   Alcotest.test_case "foreign key ordering" 'Quick  
190   test_foreign_key_ordering;  
191   Alcotest.test_case "ownership RLS" 'Quick test_ownership_rls;  
192   Alcotest.test_case "group policies" 'Quick test_group_policies;  
193   Alcotest.test_case "operations as comments" 'Quick  
194   test_operations_as_comments;  
195   Alcotest.test_case "relations as comments" 'Quick  
196   test_relations_as_comments;  
197 ];
```

138 test/test_doc_coverage.ml

```

1  (* Tests for documentation coverage system *)
2
3  open Borge_lib
4
5  let test_doc_extract () =
6    let code = {|
7  (* Public function with doc *)
8  let public_func x = x + 1
9
10 let private_helper x = x
11
12 let _ = ignore ()
13
14 let%test "test" = true
15 |}
16    in
17    (* Write temp file *)
18    let tmp = Filename.temp_file "test" ".ml" in
19    let oc = open_out tmp in
20    output_string oc code;
21    close_out oc;
22    let bindings = Doc_extract.extract_bindings tmp in
23    Sys.remove tmp;
24    (* Should find bindings *)
25    Alcotest.(check bool) "Found bindings" true (List.length bindings >= 2)
26
27  let test_doc_detect () =
28    let code = {|
29  (* This is a doc comment *)
30  let foo x = x
31
32  (* exempt doc *)
33  let bar y = y
34  |}
35    in
36    let tmp = Filename.temp_file "test" ".ml" in
37    let oc = open_out tmp in
38    output_string oc code;
39    close_out oc;
40    let docs = Doc_detect.extract_file_docs tmp in
41    Sys.remove tmp;
42    Alcotest.(check bool) "Found docs" true (List.length docs > 0)
43
44  let test_doc_coverage () =
45    let fc : Doc_coverage.file_coverage = {
46      path = "test.ml";
47      total_bindings = 5;
48      documented = 3;
49      exempt = 1;
50      drifted = 0;
51      undocumented = 1;
52      coverage_percent = 75.0;
53      undocumented_names = ["helper"];
54      drifted_names = [];
55    }
56    in
57    Alcotest.(check (float 0.01)) "Coverage calc" 75.0 fc.coverage_percent;
58    let meta = Doc_coverage.generate_meta_block fc in
59    Alcotest.(check bool) "Meta block contains file" true (String.contains
60      meta 't')
61
62  let () =
63    Alcotest.run "Doc coverage tests" [
64      ("extract", [Alcotest.test_case "Extract bindings" 'Quick
65        test_doc_extract]);
66      ("detect", [Alcotest.test_case "Detect docs" 'Quick test_doc_detect]);
67      ("coverage", [Alcotest.test_case "Generate meta" 'Quick
68        test_doc_coverage]);
69    ]

```


139 test/test__dune.ml

```

1 (** Unit tests for dune parser and module surface extractor.
2     These tests work on string inputs, not filesystem paths. *)
3
4 (* --- Dune parse tests --- *)
5
6 let test_dune_library () =
7   let input = "(library\n  (name my_lib)\n  (public_name my.lib)\n
8     (modules foo bar baz)\n  (libraries unix str))" in
9   let file = Borge_lang.Parse.parse_file input in
10  let stanza = match file.Borge_lang.Ast.top_level with
11    | { Borge_lang.Ast.node = Borge_lang.Ast.List (_, Borge_lang.Ast.Atom
12      (_, "library")) :: children; _ } :: _ ->
13      Borge_lib.Dune_parse.parse_library children
14    | _ -> Alcotest.fail "no library stanza"
15  in
16  Alcotest.(check string) __LOC__ "my_lib" stanza.Borge_lib.Dune_parse.name;
17  Alcotest.(check (option string)) __LOC__ (Some "my.lib")
18    stanza.Borge_lib.Dune_parse.public_name;
19  Alcotest.(check int) __LOC__ 3 (List.length
20    stanza.Borge_lib.Dune_parse.modules);
21  Alcotest.(check bool) __LOC__ true (List.mem "foo"
22    stanza.Borge_lib.Dune_parse.modules);
23  Alcotest.(check bool) __LOC__ true (List.mem "unix"
24    stanza.Borge_lib.Dune_parse.libraries)
25
26 let test_dune_executables () =
27   let input = "(executables\n  (names my_app my_tool)\n  (libraries my_lib
28     cmdliner))" in
29   let file = Borge_lang.Parse.parse_file input in
30   let stanza = match file.Borge_lang.Ast.top_level with
31     | { Borge_lang.Ast.node = Borge_lang.Ast.List (_, Borge_lang.Ast.Atom
32       (_, "executables")) :: children; _ } :: _ ->
33       Borge_lib.Dune_parse.parse_executables children
34     | _ -> Alcotest.fail "no executables stanza"
35   in
36   Alcotest.(check int) __LOC__ 2 (List.length
37     stanza.Borge_lib.Dune_parse.names);
38   Alcotest.(check bool) __LOC__ true (List.mem "my_app"
39     stanza.Borge_lib.Dune_parse.names);
40   Alcotest.(check bool) __LOC__ true (List.mem "my_lib"
41     stanza.Borge_lib.Dune_parse.libraries)
42
43 let test_dune_test_stanza () =
44   let input = "(test\n  (name test_foo)\n  (libraries my_lib alcotest))" in
45   let file = Borge_lang.Parse.parse_file input in
46   let stanza = match file.Borge_lang.Ast.top_level with
47     | { Borge_lang.Ast.node = Borge_lang.Ast.List (_, Borge_lang.Ast.Atom
48       (_, "test")) :: children; _ } :: _ ->
49       Borge_lib.Dune_parse.parse_test children
50     | _ -> Alcotest.fail "no test stanza"
51   in
52   Alcotest.(check string) __LOC__ "test_foo"
53     stanza.Borge_lib.Dune_parse.name;
54   Alcotest.(check bool) __LOC__ true (List.mem "my_lib"
55     stanza.Borge_lib.Dune_parse.libraries)
56
57 (* --- Surface extraction tests --- *)
58
59 let test_surface_exports () =
60   let content = "let foo x y = x + y\nlet bar = 42\nlet _ = ignore ()\nlet
61     () = print_endline \"hello\"\ntype t = int\nexception Error of
62     string\nmodule M = struct end\n" in
63   let exports = Borge_lib.Surface.extract_exports content in
64   Alcotest.(check bool) __LOC__ true (List.mem "foo" exports);
65   Alcotest.(check bool) __LOC__ true (List.mem "bar" exports);
66   Alcotest.(check bool) __LOC__ true (List.mem "t" exports);
67   Alcotest.(check bool) __LOC__ true (List.mem "Error" exports);
68   Alcotest.(check bool) __LOC__ true (List.mem "M" exports);
69   Alcotest.(check bool) __LOC__ false (List.mem "_" exports)
70
71 let test_surface_rec_bindings () =

```

```

56   let content = "let rec fib n = if n < 2 then n else fib (n-1) + fib
                    (n-2)\nlet rec fact n = if n = 0 then 1 else n * fact (n-1)\n" in
57   let exports = Borge_lib.Surface.extract_exports content in
58   Alcotest.(check bool) __LOC__ true (List.mem "fib" exports);
59   Alcotest.(check bool) __LOC__ true (List.mem "fact" exports)
60
61   let test_surface_type_declarations () =
62   let content = "type result = { ok : bool; msg : string }\ntype status =
                    Implemented | Planned | In_progress\ntype 'a tree = Leaf | Node of 'a
                    tree * 'a * 'a tree\n" in
63   let exports = Borge_lib.Surface.extract_exports content in
64   Alcotest.(check bool) __LOC__ true (List.mem "result" exports);
65   Alcotest.(check bool) __LOC__ true (List.mem "status" exports);
66   Alcotest.(check bool) __LOC__ true (List.mem "tree" exports)
67
68   let () =
69   Alcotest.run "dune & surface tests" [
70     "dune", [
71       Alcotest.test_case "library stanza" 'Quick test_dune_library;
72       Alcotest.test_case "executables stanza" 'Quick test_dune_executables;
73       Alcotest.test_case "test stanza" 'Quick test_dune_test_stanza;
74     ];
75     "surface", [
76       Alcotest.test_case "exports" 'Quick test_surface_exports;
77       Alcotest.test_case "rec bindings" 'Quick test_surface_rec_bindings;
78       Alcotest.test_case "type declarations" 'Quick
79         test_surface_type_declarations;
80     ];
81   ]

```

140 test/test__meta.ml

```

1 open Borge_lib
2
3 let test_meta_write () =
4   let meta : Meta.meta = {
5     Meta.project_name = "test-project";
6     Meta.analyzed_at = "2026-01-01T00:00:00Z";
7     Meta.content_hashes = [("./foo.borg", "abcdef12")];
8     Meta.module_surfaces = [{
9       Meta.module_name = "Foo";
10      Meta.file = "./lib/foo.ml";
11      Meta.exports = ["bar"; "baz"];
12    }];
13    Meta.dune_snapshot = [{
14      Meta.name = "test_lib";
15      Meta.modules = ["foo"];
16      Meta.public_name = Some "test.lib";
17      Meta.libraries = ["unix"];
18    }];
19    Meta.findings = [{
20      Meta.ft_type = Meta.Unspecified_module;
21      Meta.section = None;
22      Meta.module_ = Some "Foo";
23      Meta.file = Some "./lib/foo.ml";
24      Meta.export = None;
25      Meta.detail = Some "module Foo not mentioned in any .borg spec";
26      Meta.spec_status = None;
27      Meta.actual_status = None;
28      Meta.confidence = Meta.High;
29      Meta.source = Meta.Static;
30      Meta.at = "2026-01-01T00:00:00Z";
31    }];
32  } in
33  let content = Meta.string_of_meta meta ~docs:None () in
34  Alcotest.(check bool) __LOC__ true (String.length content > 0);
35  Alcotest.(check bool) __LOC__ true
36    (let s = "DO NOT EDIT" in
37     String.length content >= String.length s &&
38     (* exempt: Str.search_forward *)
39     try ignore (Str.search_forward (Str.regexp_string s) content 0); true
40     with Not_found -> false);
41  Alcotest.(check bool) __LOC__ true (String.contains content '(')
42
43 let test_meta_merge () =
44   let static_finding : Meta.finding = {
45     Meta.ft_type = Meta.Unspecified_module;
46     Meta.section = None;
47     Meta.module_ = Some "Foo";
48     Meta.file = None;
49     Meta.export = None;
50     Meta.detail = None;
51     Meta.spec_status = None;
52     Meta.actual_status = None;
53     Meta.confidence = Meta.High;
54     Meta.source = Meta.Static;
55     Meta.at = "2026-01-01T00:00:00Z";
56   } in
57   let old_agent_finding = {
58     static_finding with
59     Meta.ft_type = Meta.Partial_implementation;
60     Meta.source = Meta.Agent;
61     Meta.detail = Some "old agent finding";
62   } in
63   let new_agent_finding = {
64     static_finding with
65     Meta.ft_type = Meta.Status_mismatch;
66     Meta.source = Meta.Agent;
67     Meta.detail = Some "new agent finding";
68   } in
69   let merged = Meta.merge_agent_findings [static_finding;
70     old_agent_finding] [new_agent_finding] in
71   Alcotest.(check int) __LOC__ 2 (List.length merged);

```

```
71 let has_static = List.exists (fun f -> f.Meta.source = Meta.Static)
    merged in
72 let has_old_agent = List.exists (fun f ->
73   f.Meta.source = Meta.Agent && f.Meta.detail = Some "old agent finding"
74 ) merged in
75 let has_new_agent = List.exists (fun f ->
76   f.Meta.source = Meta.Agent && f.Meta.detail = Some "new agent finding"
77 ) merged in
78 Alcotest.(check bool) __LOC__ true has_static;
79 Alcotest.(check bool) __LOC__ false has_old_agent;
80 Alcotest.(check bool) __LOC__ true has_new_agent
81
82 let test_content_hash () =
83   let h1 = Meta.content_hash "hello world" in
84   let h2 = Meta.content_hash "hello world" in
85   let h3 = Meta.content_hash "goodbye world" in
86   Alcotest.(check string) __LOC__ h1 h2;
87   Alcotest.(check bool) __LOC__ true (h1 <> h3)
88
89 let test_is_meta_file () =
90   Alcotest.(check bool) __LOC__ true (Meta.is_meta_file "foo.borg.meta");
91   Alcotest.(check bool) __LOC__ false (Meta.is_meta_file "foo.borg");
92   Alcotest.(check bool) __LOC__ false (Meta.is_meta_file "meta");
93   Alcotest.(check bool) __LOC__ false (Meta.is_meta_file "")
94
95 let () =
96   Alcotest.run "meta tests" [
97     "meta", [
98       Alcotest.test_case "write" 'Quick test_meta_write;
99       Alcotest.test_case "merge" 'Quick test_meta_merge;
100      Alcotest.test_case "hash" 'Quick test_content_hash;
101      Alcotest.test_case "is_meta_file" 'Quick test_is_meta_file;
102    ];
103   ]
```

141 test/test_parse.ml

```

1 let parse_ok input =
2   try
3     let _ = Borge_lang.Parse.parse input in
4     true
5   with Borge_lang.Error.Parse_error _ -> false
6
7 let test_simple_project () =
8   let input = "(project hello\n  (doc \"A project\")\n  (status planned))"
9   in
10  Alcotest.(check bool) __LOC__ true (parse_ok input)
11
12 let test_annotated_comment () =
13   let input = "(* roerick note (|Hello|) *)\n\n(project test\n  (doc\n  \"Test\")\n  (status planned))" in
14   Alcotest.(check bool) __LOC__ true (parse_ok input)
15
16 let test_verbatim_string () =
17   let input = "(project test\n  (doc (|A multi-line\ndoc string|))\n  (status planned))" in
18   Alcotest.(check bool) __LOC__ true (parse_ok input)
19
20 let test_ask_response () =
21   let input = "(project test\n  (doc \"Test\")\n  (* roerick ask (|Hello?|)\n  *)\n  (* agent response (|) *)\n  (status planned))" in
22   Alcotest.(check bool) __LOC__ true (parse_ok input)
23
24 let test_design () =
25   let input = "(* roerick design (|Build it|) *)\n\n(project test\n  (doc\n  \"Test\")\n  (status planned))" in
26   Alcotest.(check bool) __LOC__ true (parse_ok input)
27
28 let test_plain_comments () =
29   let input = "; comment\n\n(project test\n  (doc \"Test\")\n  (status\n  planned))" in
30   Alcotest.(check bool) __LOC__ true (parse_ok input)
31
32 let test_bad_unclosed_paren () =
33   let input = "(project test" in
34   Alcotest.(check bool) __LOC__ false (parse_ok input)
35
36 let test_project_name () =
37   let input = "(project my-project\n  (doc \"Test\")\n  (status planned))"
38   in
39   let file = Borge_lang.Parse.parse input in
40   let name = Borge_lib.Spec.project_name file in
41   Alcotest.(check (option string)) __LOC__ (Some "my-project") name
42
43 let test_inline_targets () =
44   let input = "(project test\n  (section foo\n  (doc \"Foo\")\n  (status\n  planned)\n  (inline bar.borg)\n  (inline baz.borg)))" in
45   let file = Borge_lang.Parse.parse input in
46   let targets = Borge_lib.Spec.inline_targets file in
47   Alcotest.(check (list string)) __LOC__ ["bar.borg"; "baz.borg"] targets
48
49 let test_inline_targets_empty () =
50   let input = "(project test\n  (doc \"No inlines\")\n  (status planned))"
51   in
52   let file = Borge_lang.Parse.parse input in
53   let targets = Borge_lib.Spec.inline_targets file in
54   Alcotest.(check (list string)) __LOC__ [] targets
55
56 let test_has_no_inline () =
57   let input = "(project test\n  (doc \"Standalone\")\n  (status planned)\n  (no-inline))" in
58   let file = Borge_lang.Parse.parse input in
59   Alcotest.(check bool) __LOC__ true (Borge_lib.Spec.has_no_inline file)
60
61 let test_has_no_inline_false () =
62   let input = "(project test\n  (doc \"Has parent\")\n  (status planned))"
63   in
64   let file = Borge_lang.Parse.parse input in

```

```
61   Alcotest.(check bool) __LOC__ false (Borge_lib.Spec.has_no_inline file)
62
63 let () =
64   Alcotest.run "borge parse tests" [
65     "basic", [
66       Alcotest.test_case "simple project" 'Quick test_simple_project;
67       Alcotest.test_case "annotated comment" 'Quick test_annotated_comment;
68       Alcotest.test_case "verbatim string" 'Quick test_verbatim_string;
69       Alcotest.test_case "ask/response" 'Quick test_ask_response;
70       Alcotest.test_case "design comment" 'Quick test_design;
71       Alcotest.test_case "plain comments" 'Quick test_plain_comments;
72       Alcotest.test_case "bad paren" 'Quick test_bad_unclosed_paren;
73       Alcotest.test_case "project name" 'Quick test_project_name;
74       Alcotest.test_case "inline targets" 'Quick test_inline_targets;
75       Alcotest.test_case "inline targets empty" 'Quick
76       test_inline_targets_empty;
77       Alcotest.test_case "has no-inline" 'Quick test_has_no_inline;
78       Alcotest.test_case "no-inline false" 'Quick test_has_no_inline_false;
79     ];
79   ]
```

142 test/test_review.ml

```
1 (* Tests for semantic review system *)
2
3 open Borge_lib
4
5 let test_extract_functions () =
6   let code = "let foo x = x + 1\n" in
7   let _functions = Semantic_review.extract_functions code in
8   (* Just verify extraction runs without error *)
9   Alcotest.(check pass) "Extraction runs" () ()
10
11 let test_review_types () =
12   Alcotest.(check string) "High to string" "high"
13     (Review_types.string_of_confidence Review_types.High);
14   match Review_types.confidence_of_string "high" with
15   | Some _ -> Alcotest.(check pass) "Parse works" () ()
16   | None -> Alcotest.fail "Parse failed"
17
18 let () =
19   Alcotest.run "Review tests" [
20     ("extract", [Alcotest.test_case "Functions" `Quick
21       test_extract_functions]);
21     ("types", [Alcotest.test_case "Conversions" `Quick test_review_types]);
22   ]
```

143 test/test_slot_debug.ml

```

1 open Borge_lib
2
3 let input = {(ui test
4   (layout app-shell
5     (ui root
6       (layout horizontal)
7       (children
8         (ui sidebar (children ..nav))
9         (ui main (children ..content))))))} in
10 let file = Borge_lang.Parse.parse_file input in
11 let app = Ui_parse.parse_file file in
12 match app with
13 | None -> print_endline "PARSE FAILED"
14 | Some a ->
15   Printf.printf "layouts: %d\n" (List.length a.Ui_ast.layouts);
16   List.iter (fun l ->
17     Printf.printf "  layout %s, slots: %d\n" l.Ui_ast.name (List.length
18       l.Ui_ast.slots);
19     let rec print_el indent el =
20       Printf.printf "%s  el %s, children: %d\n" (String.make indent ' ')
21         (match el.Ui_ast.name with Some n -> n | None -> "?")
22         (List.length el.Ui_ast.children);
23       List.iter (print_el (indent + 2)) el.Ui_ast.children
24     in
25     print_el 0 l.Ui_ast.root
26   ) a.Ui_ast.layouts
27 ;;
28
29 let input = {(ui test2
30   (layout app-shell
31     (ui root
32       (layout horizontal)
33       (children
34         (ui sidebar (children ..nav))
35         (ui main (children ..content))))))
36   (page home
37     (use-layout app-shell)
38     (fill nav (text "HomeNav"))
39     (fill content (text "Welcome")))} in
40 let file = Borge_lang.Parse.parse_file input in
41 let app = Ui_parse.parse_file file in
42 match app with
43 | None -> print_endline "PAGE PARSE FAILED"
44 | Some a ->
45   Printf.printf "pages: %d\n" (List.length a.Ui_ast.pages);
46   List.iter (fun p ->
47     Printf.printf "  page %s (layout: %s), fills: %d\n" p.Ui_ast.name
48       p.Ui_ast.layout_name (List.length p.Ui_ast.fills);
49     List.iter (fun f ->
50       Printf.printf "    fill slot=%s, content: %d elements\n"
51         f.Ui_ast.slot_name (List.length f.Ui_ast.content)
52       ) p.Ui_ast.fills
53   ) a.Ui_ast.pages

```


144 test/test_ui_clay.ml

```

1 (* Test ui_clay generator *)
2 open Borge_lib
3
4 let contains haystack needle =
5   (* exempt: Str.search_forward *)
6   try let _ = Str.search_forward (Str.regexp_string needle) haystack 0 in
7     true
8   with Not_found -> false
9
10 let test_simple_element () =
11   let open Ui_ast in
12   let app = {
13     theme = None;
14     components = [];
15     layouts = [];
16     pages = [];
17     routes = [];
18     workflows = [];
19     elements = [
20       { name = Some "box";
21         properties = [P_layout Vertical; P_width (Flex 1); P_bg (Hex
22           "#333")];
23         children = [];
24         variants = []; text = None;
25       };
26     ];
27   } in
28   let c = Ui_clay.generate ~source_path:"test.borg" app in
29   Alcotest.(check bool) "contains CLAY_CONTAINER" true (contains c
30     "CLAY_CONTAINER");
31   Alcotest.(check bool) "contains TOP_TO_BOTTOM" true (contains c
32     "CLAY_TOP_TO_BOTTOM");
33   Alcotest.(check bool) "contains SIZING_GROW" true (contains c
34     "CLAY_SIZING_GROW");
35   Alcotest.(check bool) "contains background RGBA" true (contains c
36     "backgroundColor");
37   Alcotest.(check bool) "contains header" true (contains c "Generated by
38     borge generate")
39
40 let test_color_rgba () =
41   let open Ui_ast in
42   let app = {
43     theme = None;
44     components = [];
45     layouts = [];
46     pages = [];
47     routes = [];
48     workflows = [];
49     elements = [
50       { name = Some "box";
51         properties = [P_bg (Hex "#e0d7d2")];
52         children = [];
53         variants = []; text = None;
54       };
55     ];
56   } in
57   let c = Ui_clay.generate ~source_path:"test.borg" app in
58   Alcotest.(check bool) "hex to RGBA" true (contains c "224, 215, 210, 255")
59
60 let test_keyword_color () =
61   let open Ui_ast in
62   let app = {
63     theme = None;
64     components = [];
65     layouts = [];
66     pages = [];
67     routes = [];
68     workflows = [];
69     elements = [
70       { name = Some "box";
71         properties = [P_bg (Keyword "dark")];

```

```

65     children = [];
66     variants = []; text = None;
67   };
68 ];
69 } in
70 let c = Ui_clay.generate ~source_path:"test.borg" app in
71 Alcotest.(check bool) "keyword dark RGBA" true (contains c "40, 40, 40,
    255")
72
73 let test_c_sanitization () =
74   let open Ui_ast in
75     let app = {
76       theme = None;
77       components = [];
78       layouts = [];
79       pages = [];
80       routes = [];
81       workflows = [];
82       elements = [
83         { name = Some "nav-bar";
84           properties = [P_layout Horizontal];
85           children = [];
86           variants = []; text = None;
87         };
88       ];
89     } in
90     let c = Ui_clay.generate ~source_path:"test.borg" app in
91     Alcotest.(check bool) "hyphen to underscore" true (contains c "nav_bar")
92
93 let test_routes_enum () =
94   let open Ui_ast in
95     let app = {
96       theme = None;
97       components = [];
98       layouts = [];
99       pages = [];
100      routes = [{ path = "/"; page_name = "home" }; { path = "/dashboard";
    page_name = "dashboard" }];
101      workflows = [];
102      elements = [];
103    } in
104    let c = Ui_clay.generate ~source_path:"test.borg" app in
105    Alcotest.(check bool) "contains enum page" true (contains c "enum page");
106    Alcotest.(check bool) "contains PAGE_HOME" true (contains c "PAGE_HOME");
107    Alcotest.(check bool) "contains layout_app switch" true (contains c
    "layout_app")
108
109 let test_header_file () =
110   let open Ui_ast in
111     let app = {
112       theme = None;
113       components = [{ name = "button"; properties = [P_interactive
    Interactive_click]; slots = []; children = []; variants = [] }];
114       layouts = [];
115       pages = [];
116       routes = [{ path = "/"; page_name = "home" }];
117       workflows = [];
118       elements = [];
119     } in
120     let h = Ui_clay.generate_header_file app in
121     Alcotest.(check bool) "contains include guard" true (contains h "#ifndef
    UI_APP_H");
122     Alcotest.(check bool) "contains render_button declaration" true (contains
    h "render_button");
123     Alcotest.(check bool) "contains enum page" true (contains h "enum page")
124   ;;
125
126 let () =
127   Alcotest.run "ui_clay" [
128     "ui_clay", [
129       Alcotest.test_case "simple element" 'Quick test_simple_element;
130       Alcotest.test_case "color RGBA" 'Quick test_color_rgba;
131       Alcotest.test_case "keyword color" 'Quick test_keyword_color;

```

```
132     Alcotest.test_case "C sanitization" 'Quick test_c_sanitization;  
133     Alcotest.test_case "routes enum" 'Quick test_routes_enum;  
134     Alcotest.test_case "header file" 'Quick test_header_file;  
135 ];  
136 ]
```

145 test/test_ui_db.ml

```

1 open Borge_lib
2
3 let test_ui_parse () =
4   let input = |(ui my-app
5     (theme
6       (palette bg-primary "#e0d7d2")
7       (palette accent "#a8421c")
8       (spacing sm 8) (spacing md 16)
9       (font-size base 14) (font-size lg 18)
10      (radius sm 4) (radius md 8))
11     (component button
12       (interactive click)
13       (padding (spacing md))
14       (corner (radius sm))
15       (variant primary (bg (palette accent)) (color "#ffffff"))
16       (variant secondary (bg (palette bg-primary))))
17     (layout app-shell
18       (ui root
19         (layout vertical)
20         (width full)
21         (height full)
22         (children ..content)))
23     (page home
24       (use-layout app-shell)
25       (fill content
26         (text "Hello" (size (font-size lg))))))
27     (routes
28       (route "/" home)))|} in
29   let file = Borge_lang.Parse.parse_file input in
30   let app = Ui_parse.parse_file file in
31   match app with
32   | None -> Alcotest.fail "parse returned None"
33   | Some a ->
34     Alcotest.(check int) "theme present" 1 (match a.Ui_ast.theme with Some
35       _ -> 1 | None -> 0);
36     Alcotest.(check int) "components" 1 (List.length a.Ui_ast.components);
37     Alcotest.(check int) "layouts" 1 (List.length a.Ui_ast.layouts);
38     Alcotest.(check int) "pages" 1 (List.length a.Ui_ast.pages);
39     Alcotest.(check int) "routes" 1 (List.length a.Ui_ast.routes)
40
41 let test_db_parse () =
42   let input = |(db blog
43     (table users
44       (column id (type uuid) (primary-key) (default gen_random_uuid))
45       (column email (type text) (unique) (not-null))
46       (column name (type text) (not-null))
47       (column role (type text) (not-null) (default "reader"))
48       (column created-at (type timestampz) (default now)))
49     (table posts
50       (column id (type uuid) (primary-key) (default gen_random_uuid))
51       (column author-id (type uuid) (references users.id) (not-null))
52       (column title (type text) (not-null))
53       (column published (type boolean) (default false))
54       (ownership author-id))
55     (operations posts
56       (crud (except delete))
57       (query by-author
58         (query published-recent (limit 20)))
59       (relation post-with-author
60         (from posts)
61         (join users (on users.id = posts.author-id))
62         (select users.name posts.title posts.published)
63         (query detail)))
64     (groups
65       (group admin (can-all))
66       (group author
67         (can posts (create read update) (where author-id = current-user)))
68       (group reader
69         (can posts (read (where published = true))))))|} in
70   let file = Borge_lang.Parse.parse_file input in
71   let app = Db_parse.parse_file file in

```

```

71 match app with
72 | None -> Alcotest.fail "parse returned None"
73 | Some a ->
74   Alcotest.(check int) "tables" 2 (List.length a.Db_ast.tables);
75   Alcotest.(check int) "operations" 1 (List.length a.Db_ast.operations);
76   Alcotest.(check int) "relations" 1 (List.length a.Db_ast.relations);
77   Alcotest.(check int) "groups" 3 (List.length a.Db_ast.groups)
78
79 let test_ui_validate () =
80   let input = {|(ui my-app
81     (theme (palette accent "#a8421c") (spacing md 16))
82     (component button
83       (bg (palette nonexistent))
84       (padding (spacing nonexistent))
85       (variant primary (bg (palette accent)))
86       (variant primary (bg (palette accent))))
87     (layout app-shell
88       (ui root (children ..content)))
89     (page home
90       (use-layout nonexistent-layout))}|} in
91   let file = Borge_lang.Parse.parse_file input in
92   let app = Ui_parse.parse_file file in
93   match app with
94   | None -> Alcotest.fail "parse returned None"
95   | Some a ->
96     let issues = Ui_validate.validate a in
97     let errors = List.filter (fun (i : Ui_validate.issue) -> i.severity =
98       'Error) issues in
99     (* Should catch: undefined palette ref, undefined spacing ref,
100      duplicate variant name, undefined layout ref *)
101     Alcotest.(check int) "validation errors" 4 (List.length errors)
102
103 let test_db_validate () =
104   let input = {|(db blog
105     (table users
106       (column id (type uuid) (primary-key))
107       (column id (type text)))
108     (table posts
109       (column id (type uuid) (primary-key))
110       (column author-id (type uuid) (references nonexistent.id))
111       (ownership nonexistent-col))
112     (table users
113       (column id (type uuid) (primary-key)))
114     (operations nonexistent-posts (crud))
115     (relation bad-rel (from nonexistent))
116     (groups
117       (group admin (can-all))
118       (group admin (can-all))
119       (group reader (can nonexistent-table (read))))}|} in
120   let file = Borge_lang.Parse.parse_file input in
121   let app = Db_parse.parse_file file in
122   match app with
123   | None -> Alcotest.fail "parse returned None"
124   | Some a ->
125     let issues = Db_validate.validate a in
126     let errors = List.filter (fun (i : Db_validate.issue) -> i.severity =
127       'Error) issues in
128     (* Should catch: duplicate column, undefined ref table, undefined
129      ownership col,
130      duplicate table, undefined ops table, undefined from-table in
131      relation,
132      duplicate group, undefined cap table *)
133     Alcotest.(check int) "validation errors" 8 (List.length errors)
134
135 let test_ui_parse_empty () =
136   let input = "(ui empty-app)" in
137   let file = Borge_lang.Parse.parse_file input in
138   let app = Ui_parse.parse_file file in
139   (match app with
140   | None -> Alcotest.fail "parse returned None"
141   | Some a ->
142     Alcotest.(check int) "no theme" 0 (match a.Ui_ast.theme with Some _ ->
143       1 | None -> 0);

```

```

139   Alcotest.(check int) "no components" 0 (List.length
140     a.Ui_ast.components);
141   Alcotest.(check int) "no pages" 0 (List.length a.Ui_ast.pages))
142
143 let test_ui_parse_nested_elements () =
144   let input = {|(ui nested
145     (ui outer
146       (layout horizontal)
147       (width full)
148       (height grow)
149       (children
150         (ui left (width 1) (height grow))
151         (ui right (width 3) (height grow)
152           (children
153             (ui header (height auto))
154             (ui content (height grow))))))}}|} in
155   let file = Borge_lang.Parse.parse_file input in
156   let app = Ui_parse.parse_file file in
157   (match app with
158   | None -> Alcotest.fail "parse returned None"
159   | Some a ->
160     Alcotest.(check int) "1 element" 1 (List.length a.Ui_ast.elements);
161     match a.Ui_ast.elements with
162     | [el] ->
163       Alcotest.(check int) "outer has 2 children" 2 (List.length
164         el.Ui_ast.children);
165       (match el.Ui_ast.children with
166       | [_; El right] ->
167         Alcotest.(check int) "right has 2 children" 2 (List.length
168           right.Ui_ast.children)
169       | _ -> Alcotest.fail "wrong children count")
170     | _ -> Alcotest.fail "wrong element count")
171
172 let test_db_parse_minimal () =
173   let input = {|(db minimal
174     (table items
175       (column id (type uuid) (primary-key))))|} in
176   let file = Borge_lang.Parse.parse_file input in
177   let app = Db_parse.parse_file file in
178   (match app with
179   | None -> Alcotest.fail "parse returned None"
180   | Some a ->
181     Alcotest.(check int) "1 table" 1 (List.length a.Db_ast.tables);
182     (match a.Db_ast.tables with
183     | t :: _ ->
184       Alcotest.(check string) "table name" "items" t.Db_ast.name;
185       Alcotest.(check int) "1 column" 1 (List.length t.Db_ast.columns);
186       (match t.Db_ast.columns with
187       | col :: _ ->
188         Alcotest.(check string) "column name" "id" col.Db_ast.name
189       | [] -> Alcotest.fail "expected at least 1 column")
190     | [] -> Alcotest.fail "expected at least 1 table"))
191
192 let test_db_parse_crud_variants () =
193   let input = {|(db crud-test
194     (table t1 (column id (type int) (primary-key)))
195     (operations t1 (crud))
196     (table t2 (column id (type int) (primary-key)))
197     (operations t2 (crud (except delete)))
198     (table t3 (column id (type int) (primary-key)))
199     (operations t3 (crud (only read))))|} in
200   let file = Borge_lang.Parse.parse_file input in
201   let app = Db_parse.parse_file file in
202   (match app with
203   | None -> Alcotest.fail "parse returned None"
204   | Some a ->
205     Alcotest.(check int) "3 tables" 3 (List.length a.Db_ast.tables);
206     Alcotest.(check int) "3 ops" 3 (List.length a.Db_ast.operations);
207     (match a.Db_ast.operations with
208     | [o1; o2; o3] ->
209       (match o1.Db_ast.crud with Some (Db_ast.Crud_all) -> () | _ ->
210       Alcotest.fail "o1 not Crud_all");

```

```

207     (match o2.Db_ast.crud with Some (Db_ast.Crud_except ["delete"]) -> ()
208   | _ -> Alcotest.fail "o2 not Crud_except");
209   (match o3.Db_ast.crud with Some (Db_ast.Crud_only ["read"]) -> () | _
210 -> Alcotest.fail "o3 not Crud_only")
211 | _ -> Alcotest.fail "wrong ops count"))
212
213 let test_ui_workflow_parse () =
214   let input = {
215     | (ui my-app
216       (theme
217         (palette bg-primary "#e0d7d2")
218         (spacing sm 8) (spacing md 16))
219       (component button
220         (interactive click)
221         (padding (spacing sm))
222         (text "Sign In"))
223       (layout app-shell
224         (ui root
225           (layout vertical)
226           (width full)
227           (height full)
228           (children ..content)))
229       (page home
230         (use-layout app-shell)
231         (fill content
232           (ui get-started (interactive click) (text "Get Started"))))
233       (page login
234         (use-layout app-shell)
235         (fill content
236           (ui username-input (interactive type))
237           (ui password-input (interactive type))
238           (ui sign-in-btn (interactive click) (text "Sign In"))))
239       (page dashboard
240         (use-layout app-shell)
241         (fill content
242           (ui new-post-btn (interactive click) (text "New Post"))))
243       (routes
244         (route "/" home)
245         (route "/login" login)
246         (route "/dashboard" dashboard))
247       (workflow login-to-dashboard
248         (doc "Authenticate and land on dashboard")
249         (step welcome
250           (page home)
251           (action click "get-started")
252           (outcome navigate "/login"))
253         (step auth
254           (page login)
255           (action type "username-input")
256           (action type "password-input")
257           (action click "sign-in-btn")
258           (outcome navigate "/dashboard"))
259         (step landing
260           (page dashboard)
261           (action click "new-post-btn"))))
262     } in
263   let file = Borge_lang.Parse.parse_file input in
264   let app = Ui_parse.parse_file file in
265   match app with
266   | None -> Alcotest.fail "parse returned None"
267   | Some a ->
268     let open Ui_ast in
269     Alcotest.(check int) "workflows" 1 (List.length a.workflows);
270     (match a.workflows with
271     | [wf] ->
272       Alcotest.(check string) "workflow name" "login-to-dashboard" wf.name;
273       Alcotest.(check int) "workflow steps" 3 (List.length wf.steps);
274       let auth_steps = List.filter (fun (s : Ui_ast.step) -> s.name =
275         "auth") wf.steps in
276       Alcotest.(check int) "found auth step" 1 (List.length auth_steps);
277       (match auth_steps with
278       | [s] -> Alcotest.(check int) "workflow actions in auth step" 3
279         (List.length s.actions)
280       | _ -> Alcotest.fail "expected exactly 1 auth step")
281     | _ -> Alcotest.fail "expected 1 workflow")

```

```

276
277 let test_ui_workflow_validate () =
278   let input = {|(ui my-app
279     (theme (palette accent "#a8421c"))
280     (component button
281       (interactive click)
282       (padding (spacing sm))
283       (text "Sign In"))
284     (layout app-shell
285       (ui root (layout vertical) (width full) (height full) (children
286         ..content)))
287     (page home
288       (use-layout app-shell)
289       (fill content
290         (ui get-started (interactive click) (text "Get Started"))))
291     (page login
292       (use-layout app-shell)
293       (fill content
294         (ui sign-in-btn (interactive click) (text "Sign In"))))
295     (page orphan
296       (use-layout app-shell)
297       (fill content (text "No one visits me")))
298     (routes
299       (route "/" home))
300     (workflow bad-journey
301       (step welcome
302         (page home)
303         (action click "get-started")
304         (outcome navigate "/login"))
305       (step bad-page
306         (page nonexistent)
307         (action click "sign-in-btn")))] in
308   let file = Borge_lang.Parse.parse_file input in
309   let app = Ui_parse.parse_file file in
310   match app with
311   | None -> Alcotest.fail "parse returned None"
312   | Some a ->
313     let issues = Ui_validate.validate a in
314     let errors = List.filter (fun (i : Ui_validate.issue) -> i.severity =
315       'Error) issues in
316     let warnings = List.filter (fun (i : Ui_validate.issue) -> i.severity =
317       'Warning) issues in
318     (* Errors: undefined spacing ref (button padding), undefined route
319       (/login),
320       undefined page (nonexistent) *)
321     Alcotest.(check int) "workflow errors" 3 (List.length errors);
322     (* Warnings: unworked pages login+orphan, workflow missing doc *)
323     Alcotest.(check int) "workflow warnings" 3 (List.length warnings)
324
325 let () =
326   Alcotest.run "UI/DB tests" [
327     "ui", [Alcotest.test_case "parse" 'Quick test_ui_parse;
328       Alcotest.test_case "validate" 'Quick test_ui_validate;
329       Alcotest.test_case "parse-empty" 'Quick test_ui_parse_empty;
330       Alcotest.test_case "parse-nested" 'Quick
331         test_ui_parse_nested_elements;
332       Alcotest.test_case "workflow-parse" 'Quick
333         test_ui_workflow_parse;
334       Alcotest.test_case "workflow-validate" 'Quick
335         test_ui_workflow_validate];
336     "db", [Alcotest.test_case "parse" 'Quick test_db_parse;
337       Alcotest.test_case "validate" 'Quick test_db_validate;
338       Alcotest.test_case "parse-minimal" 'Quick test_db_parse_minimal;
339       Alcotest.test_case "parse-crud" 'Quick
340         test_db_parse_crud_variants];
341   ]

```


146 test/test_ui_dream.ml

```

1 (* Test ui_dream generator *)
2 open Borge_lib
3
4 let contains haystack needle =
5   (* exempt: Str.search_forward *)
6   try let _ = Str.search_forward (Str.regexp_string needle) haystack 0 in
7     true
8   with Not_found -> false
9
10 let test_simple_element () =
11   let open Ui_ast in
12   let app = {
13     theme = None;
14     components = [];
15     layouts = [];
16     pages = [];
17     routes = [];
18     workflows = [];
19     elements = [
20       { name = Some "box";
21         properties = [P_layout Vertical; P_width (Flex 1); P_bg (Hex
22           "#333")];
23         children = [];
24         variants = []; text = None;
25       };
26     ];
27   } in
28   let ml = Ui_dream.generate ~source_path:"test.borg" app in
29   Alcotest.(check bool) "contains flex-col" true (contains ml "flex-col");
30   Alcotest.(check bool) "contains flex: 1" true (contains ml "flex: 1");
31   Alcotest.(check bool) "contains background-color" true (contains ml
32     "background-color");
33   Alcotest.(check bool) "contains header" true (contains ml "Generated by
34     borge generate")
35
36 let test_theme_resolution () =
37   let open Ui_ast in
38   let app = {
39     theme = Some { entries = [
40       Palette_entry ("bg-primary", "#e0d7d2");
41       Spacing_entry ("md", 16);
42       Font_size_entry ("lg", 18);
43       Radius_entry ("sm", 4);
44     ];
45     components = [];
46     layouts = [];
47     pages = [];
48     routes = [];
49     workflows = [];
50     elements = [
51       { name = Some "box";
52         properties = [P_bg (Palette "bg-primary"); P_padding
53           (Padding_uniform (Spacing_var "md"))];
54         children = [];
55         variants = []; text = None;
56       };
57     ];
58   } in
59   let ml = Ui_dream.generate ~source_path:"test.borg" app in
60   Alcotest.(check bool) "palette resolved to hex" true (contains ml
61     "#e0d7d2");
62   Alcotest.(check bool) "spacing resolved" true (contains ml "16px");
63   Alcotest.(check bool) "contains CSS vars" true (contains ml
64     "--bg-primary")
65
66 let test_component () =
67   let open Ui_ast in
68   let app = {
69     theme = None;
70     components = [
71       { name = "button";

```

```

65     properties = [P_interactive Interactive_click; P_padding
66     (Padding_uniform (Spacing_px 8))];
67     slots = [Default_slot];
68     children = [];
69     variants = [];
70   };
71   ];
72   layouts = [];
73   pages = [];
74   routes = [];
75   workflows = [];
76   elements = [];
77 } in
78 let ml = Ui_dream.generate ~source_path:"test.borg" app in
79 Alcotest.(check bool) "contains render_button" true (contains ml
80   "render_button");
81 Alcotest.(check bool) "contains click handler" true (contains ml
82   "onclick");
83
84 let test_routes () =
85   let open Ui_ast in
86     let app = {
87       theme = None;
88       components = [];
89       layouts = [];
90       pages = [];
91       routes = [{ path = "/"; page_name = "home" }; { path = "/dashboard";
92         page_name = "dashboard" }];
93       workflows = [];
94       elements = [];
95     } in
96     let ml = Ui_dream.generate ~source_path:"test.borg" app in
97     Alcotest.(check bool) "contains Dream.get" true (contains ml "Dream.get");
98     Alcotest.(check bool) "contains register_routes" true (contains ml
99       "register_routes");
100     Alcotest.(check bool) "contains / route" true (contains ml "\\\/\\\/")
101   ;;
102
103 let test_variant_media_queries () =
104   let open Ui_ast in
105     let app = {
106       theme = None;
107       components = [];
108       layouts = [];
109       pages = [];
110       routes = [];
111       workflows = [];
112       elements = [
113         { name = Some "sidebar";
114           properties = [P_layout Vertical; P_width (Flex 1); P_padding
115             (Padding_uniform (Spacing_px 16))];
116           children = [];
117           variants = [
118             { name = "wide";
119               properties = [P_layout Vertical; P_width (Flex 1); P_height
120                 Grow];
121               children = None };
122             { name = "narrow";
123               properties = [P_layout Horizontal; P_width Full; P_height Auto];
124               children = None };
125           ];
126           text = None;
127         };
128       ];
129     } in
130     let ml = Ui_dream.generate ~source_path:"test.borg" app in
131     Alcotest.(check bool) "contains variant_css" true (contains ml
132       "variant_css");
133     Alcotest.(check bool) "contains @media" true (contains ml "@media");
134     Alcotest.(check bool) "contains min-width: 768px" true (contains ml
135       "min-width: 768px")
136   
```

```

129 let test_style_merging () =
130   let open Ui_ast in
131   let app = {
132     theme = None;
133     components = [];
134     layouts = [];
135     pages = [];
136     routes = [];
137     workflows = [];
138     elements = [
139       { name = Some "box";
140         properties = [P_bg (Hex "#333"); P_padding (Padding_uniform
141           (Spacing_px 16)); P_gap (Spacing_px 8)];
142         children = [];
143         variants = []; text = None;
144       };
145     ]
146   } in
147   let ml = Ui_dream.generate ~source_path:"test.borg" app in
148   (* All three CSS properties should be in a single a_style attribute *)
149   Alcotest.(check bool) "contains background-color" true (contains ml
150     "background-color");
151   Alcotest.(check bool) "contains padding" true (contains ml "padding");
152   Alcotest.(check bool) "contains gap" true (contains ml "gap")
153   ;;
154
155 let () =
156   Alcotest.run "ui_dream" [
157     "ui_dream", [
158       Alcotest.test_case "simple element" 'Quick test_simple_element;
159       Alcotest.test_case "theme resolution" 'Quick test_theme_resolution;
160       Alcotest.test_case "component" 'Quick test_component;
161       Alcotest.test_case "routes" 'Quick test_routes;
162       Alcotest.test_case "variant media queries" 'Quick
163         test_variant_media_queries;
164       Alcotest.test_case "style merging" 'Quick test_style_merging;
165     ]
166   ];

```